

MicroBlaze SoC Design Flow: From RTL to Custom AXI IP and DDR Integration using Vivado and Vitis

תוכן עניינים:

Chapter 1: RTL Design, Integration and ILA Debug of a Chasing LEDs Core on VCU118.....	2
Chapter 2: MicroBlaze Processor Infrastructure and UART Communication on Zynq-7000.....	48
Chapter 3: Custom AXI4-Lite IP with a Read-Only Version Register on VCU118...	94
Chapter 4: System Integration and Software Control via JTAG-UART on VCU118	141
Chapter 5: DDR Memory Integration and AXI GPIO Pushbutton Interface on Artix-7 AC701.....	181

Chapter 1: RTL Design, Integration and ILA Debug of a Chasing LEDs Core on VCU118

מבוא:

בפרק זה נממש ליבת RTL המפעילה שמונה לדים בתבנית רצה על גבי לוח VCU118. הליבה מורכבת ממחלק תדר המייצר פולס צר בקצב 1Hz וממכונת מצבים המזיזה את הLED הדולק בכל פולס. את התכן נאמת בסימולציה ב-ModelSim ובחומרה באמצעות ILA.

דרישות קדם:

חומרה:

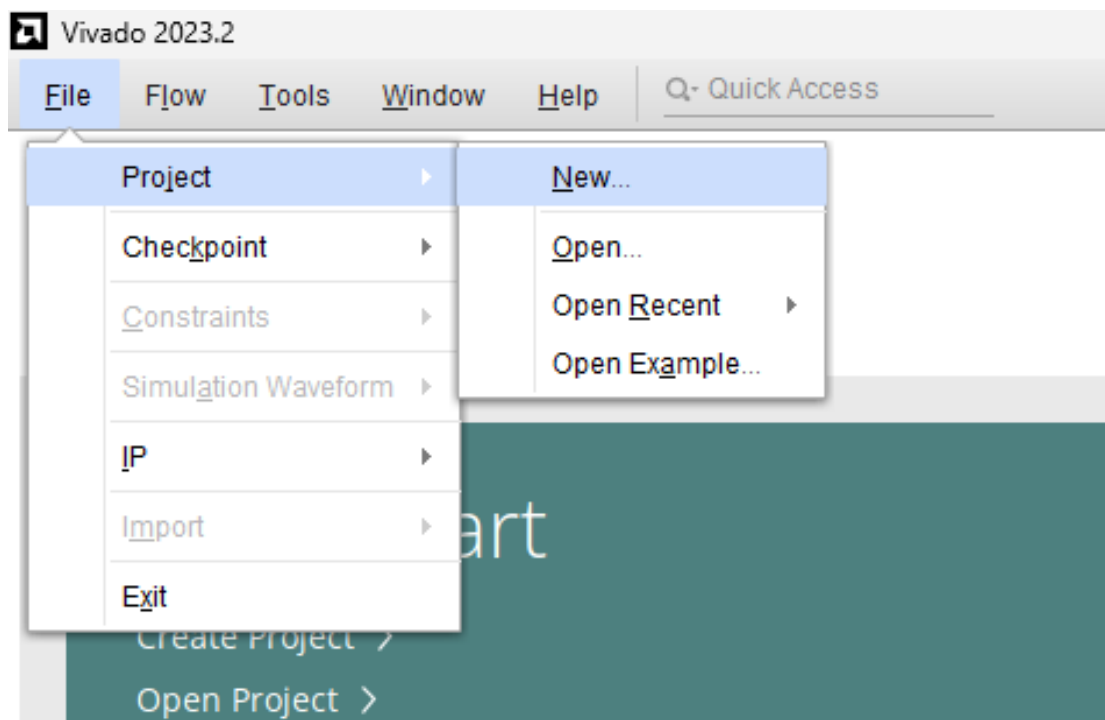
לוח פיתוח Xilinx VCU118
כבל USB-JTAG לצריבה ודיבוג
ספק כוח 12 וולט ללוח

תוכנה:

Vivado Design Suite
ModelSim לסימולציה

שלב 1:

נפתח את Vivado, ניגש לתפריט File ונבחר Project ואז New.



נזין שם לפרויקט Chasing_LEDs_Core, נבחר מיקום שמירה ונסמן יצירת תת-תיקיה לפרויקט.

New Project

Project Name
Enter a name for your project and specify a directory where the project data files will be stored.

Project name: Chasing_LEDs_Core

Project location: N:/412/4412_Comp_DevTeam/Yair_Osh/Yair_Tayri/VCU118_LED_Control_System_Project/Chasing_LEDs_RTL_01

☒ Create project subdirectory

Project will be created at: N:/412/4412_Comp_DevTeam/Yair_Osh/Yair_Tayri/VCU118_LED_Control_System_Project/Chasing_LEDs_RTL_01/Chasing_LEDs_Core

< Back Next > Finish Cancel

נבחר RTL Project ונסמן שלא להגדיר קבצי מקור כרגע.

New Project

Project Type
Specify the type of project to create.

☒ **RTL Project**
You will be able to add sources, create block designs in IP Integrator, generate IP, run RTL analysis, synthesis, implementation, design planning and analysis.
☒ Do not specify sources at this time
☐ Project is an extensible Vitis platform

☐ **Post-synthesis Project**
You will be able to add sources, view device resources, run design analysis, planning and implementation.
☐ Do not specify sources at this time

☐ **I/O Planning Project**
Do not specify design sources. You will be able to view part/package resources.

☐ **Imported Project**
Create a Vivado project from a Synplify Project File.

☐ **Example Project**
Create a new Vivado project from a predefined template.

< Back Next > Finish Cancel

בלשונית Boards נבחר את VCU118 Evaluation Platform ונסיים את האשף.

New Project

Default Part
Choose a default AMD part or board for your project.

Parts | **Boards**

To fetch the latest available boards from git repository, click on 'Refresh' button. [Dismiss](#)

[Reset All Filters](#)

Vendor: All Name: All Board Rev: Latest

Search: vcu118 (1 match)

Display Name	Preview	Status	Vendor	File Version	Part	I/O Pin Count
Virtex UltraScale+ VCU118 Evaluation Platform		Installed	xilinx.com	2.4	xcvu59p-flga2104-2L-e	2104

Refresh

< Back Next > Finish Cancel

שלב 2:

תחת IP INTEGRATOR נבחר Create Block Design ונשאיר את השם design_1.

Create Block Design

Please specify name of block design.

Design name: design_1

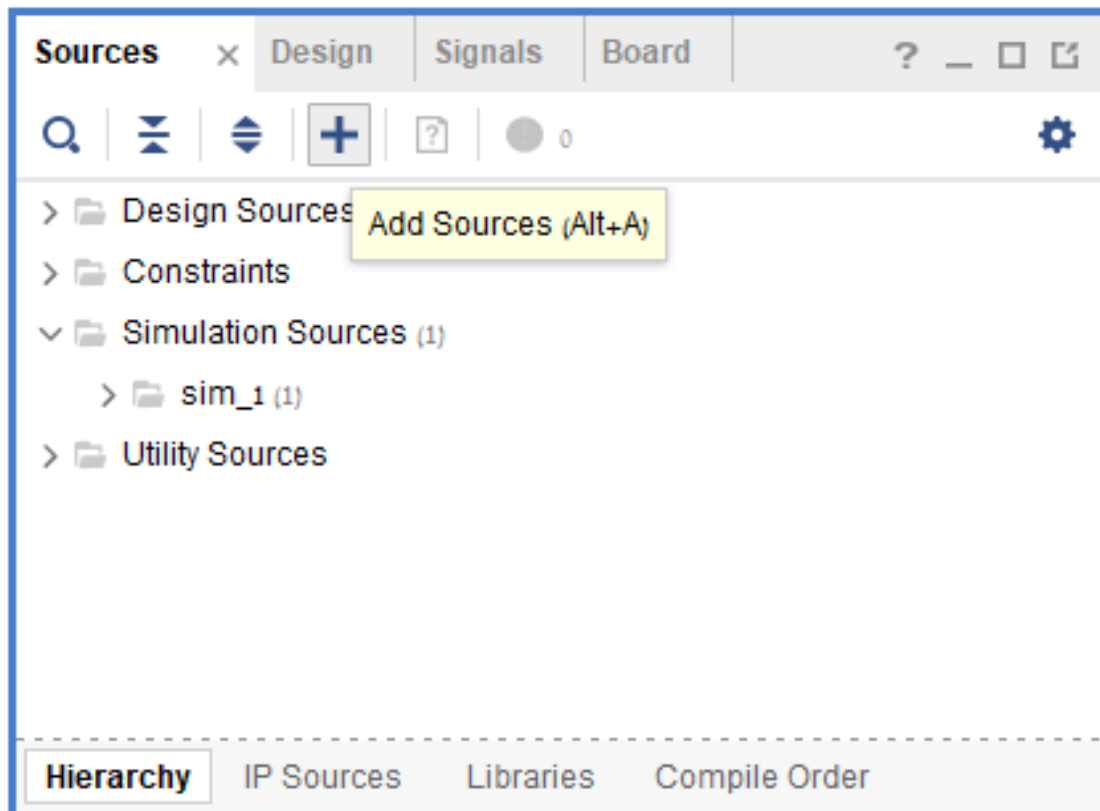
Directory: <Local to Project>

Specify source set: Design Sources

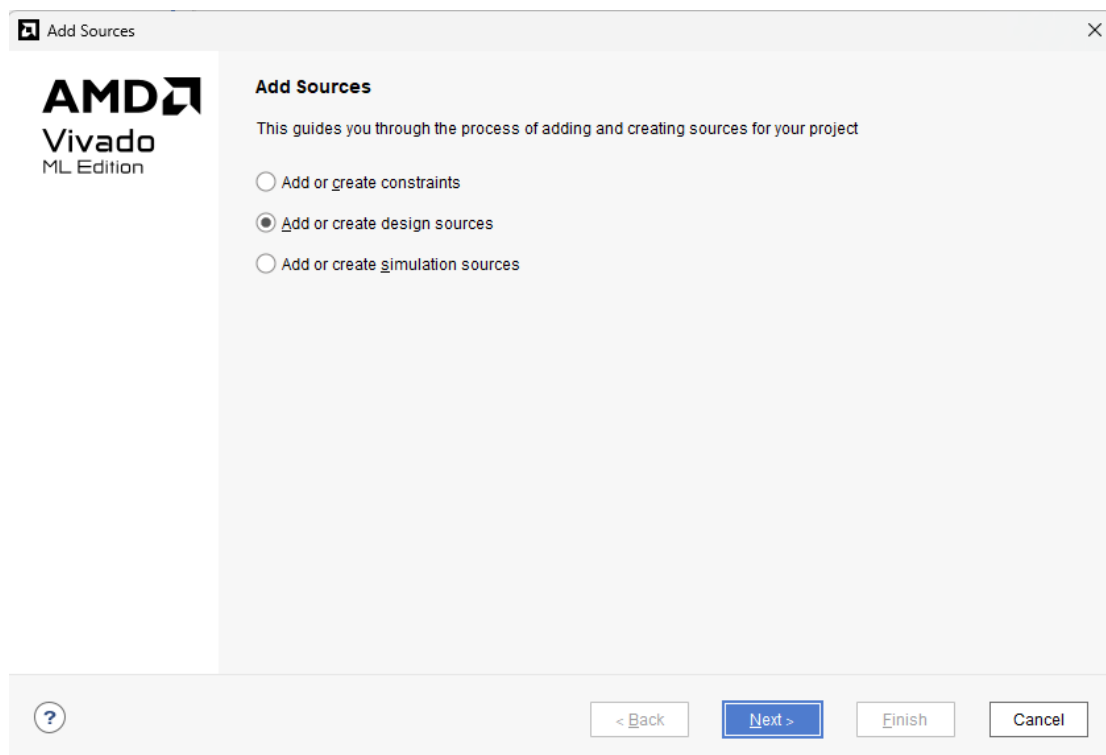
? OK Cancel

שלב 3:

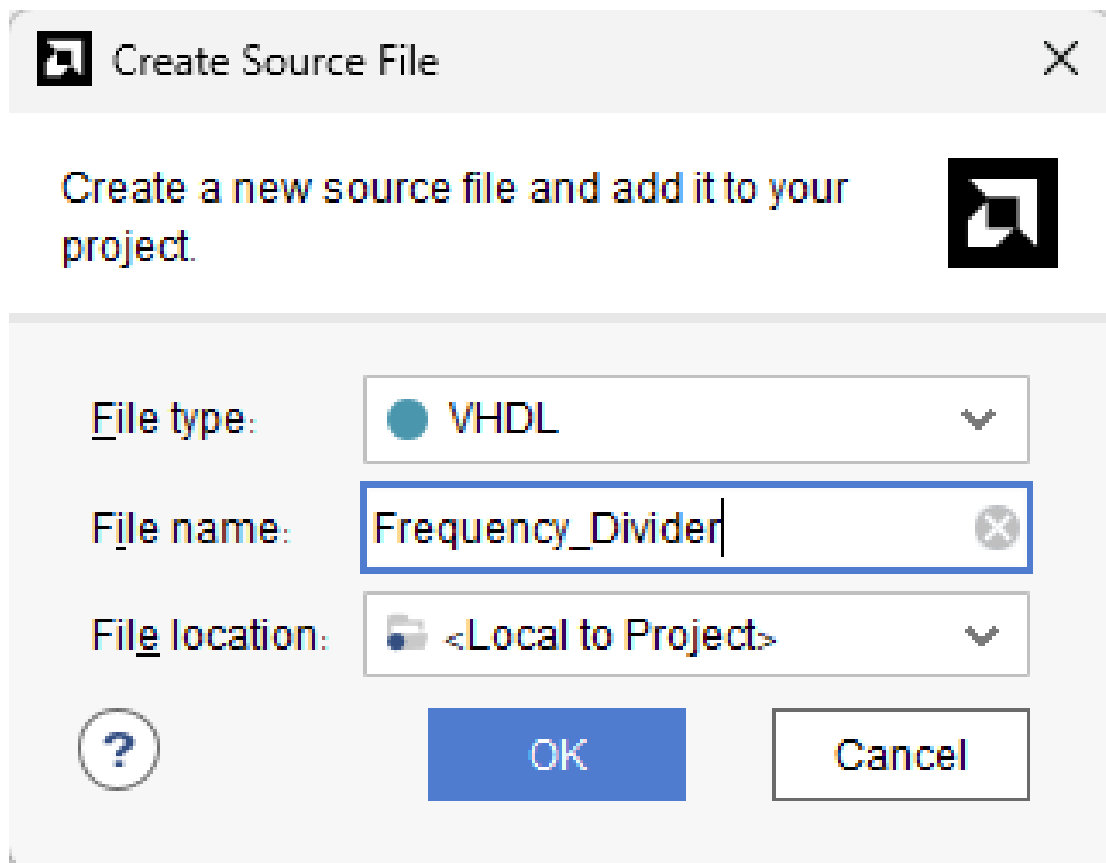
ניצור את קובץ מחלק התדר. בחלון Sources נלחץ Add Sources.



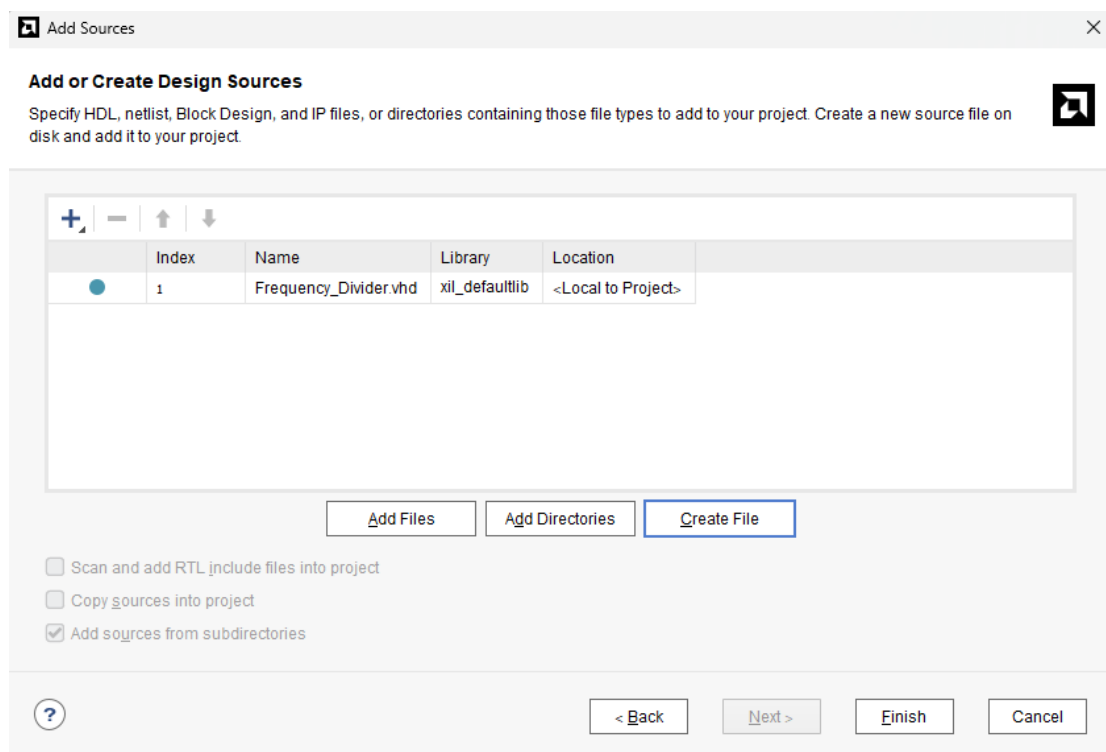
נבחר Add or create design sources ונלחץ Next.



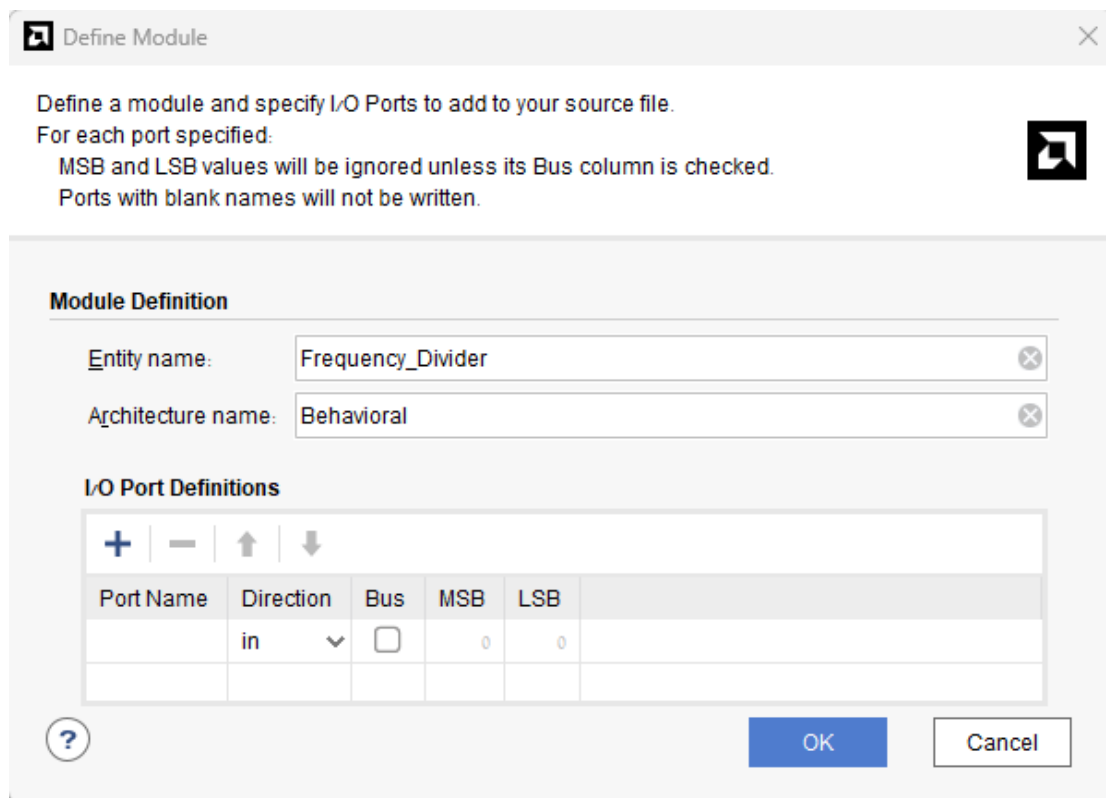
נלחץ Create File ונגדיר קובץ VHDL בשם Frequency_Divider.



הקובץ נוסף לרשימה. נלחץ Finish.



בחלון Define Module נלחץ OK. הפורטים יוגדרו ישירות בקוד.



Define a module and specify I/O Ports to add to your source file.
For each port specified:
MSB and LSB values will be ignored unless its Bus column is checked.
Ports with blank names will not be written.

Module Definition

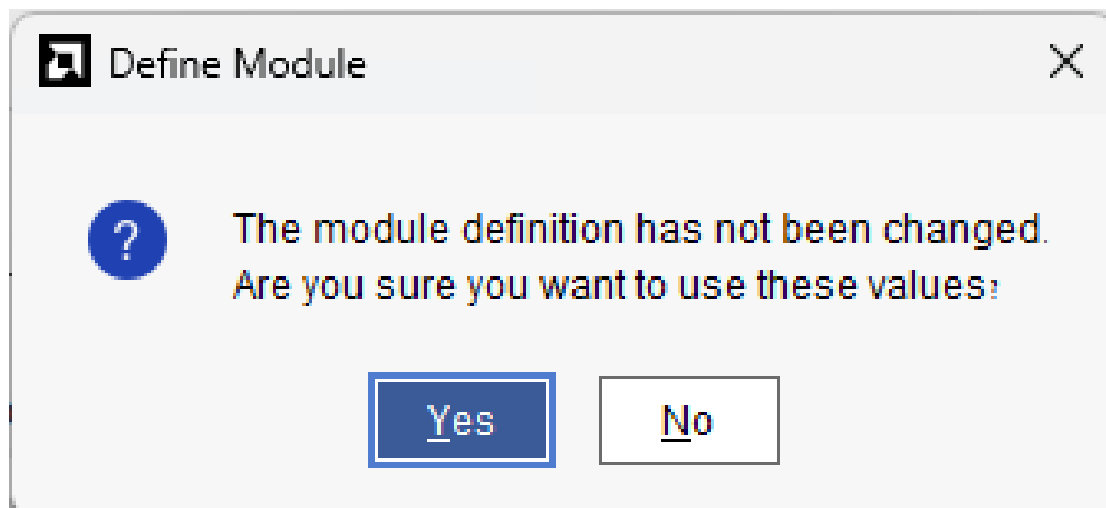
Entity name:

Architecture name:

I/O Port Definitions

Port Name	Direction	Bus	MSB	LSB
	in	☐	0	0

נאשר את ההודעה ב-Yes.



Define Module

 The module definition has not been changed.
Are you sure you want to use these values?

נכתוב את קוד המחלק. המחלק סופר מחזורי שעון עד הערך `max_count`, המחושב כמנת `input_freq` ב-`output_freq`. בסוף הספירה המונה מתאפס והיציאה `pulse_o` עולה ל-1 למשך מחזור שעון אחד בלבד. עבור שעון כניסה של 100 MHz יציאה של 1 Hz יש להגדיר את `input_freq` לערך 100000000, ומתקבל פולס צר אחד בכל 100,000,000 מחזורים.

```

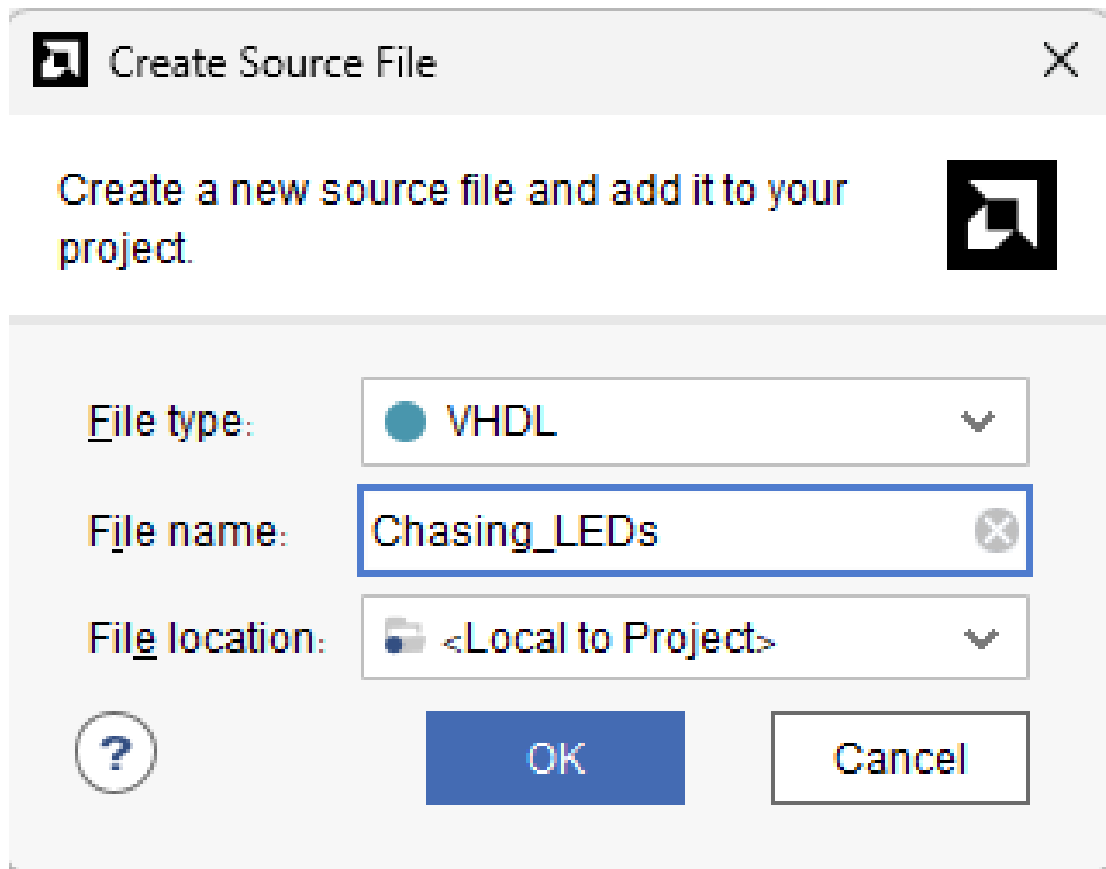
Frequency_Divider.vhd
N:/6412/6412_Comp_DevTeam/Yair_Osh/Yair_Tayri/VCU118_LED_Control_System_Pr

1  --100MHz to 1Hz
2  library ieee;
3  use ieee.std_logic_1164.all;
4  use ieee.numeric_std.all;
5
6  entity Frequency_Divider is
7      generic( input_freq: integer := 100000000;
8              output_freq: integer := 1
9              );
10     port( clk_in: in std_logic;
11          rst: in std_logic;
12          pulse_o: out std_logic
13          );
14 end entity;
15
16 architecture behave of Frequency_Divider is
17     constant max_count: integer := input_freq/output_freq;
18     signal counter: integer range 0 to max_count:=0;
19     signal pulse_en: std_logic := '0';
20 begin
21     process(clk_in,rst)
22     begin
23         if rst = '1' then
24             counter <= 0;
25             pulse_en <= '0';
26         elsif rising_edge(clk_in) then
27             if counter = max_count-1 then
28                 counter <= 0;
29                 pulse_en <= '1';
30             else
31                 counter <= counter +1;
32                 pulse_en <= '0';
33             end if;
34         end if;
35     end process;
36
37     pulse_o <= pulse_en;
38 end architecture;

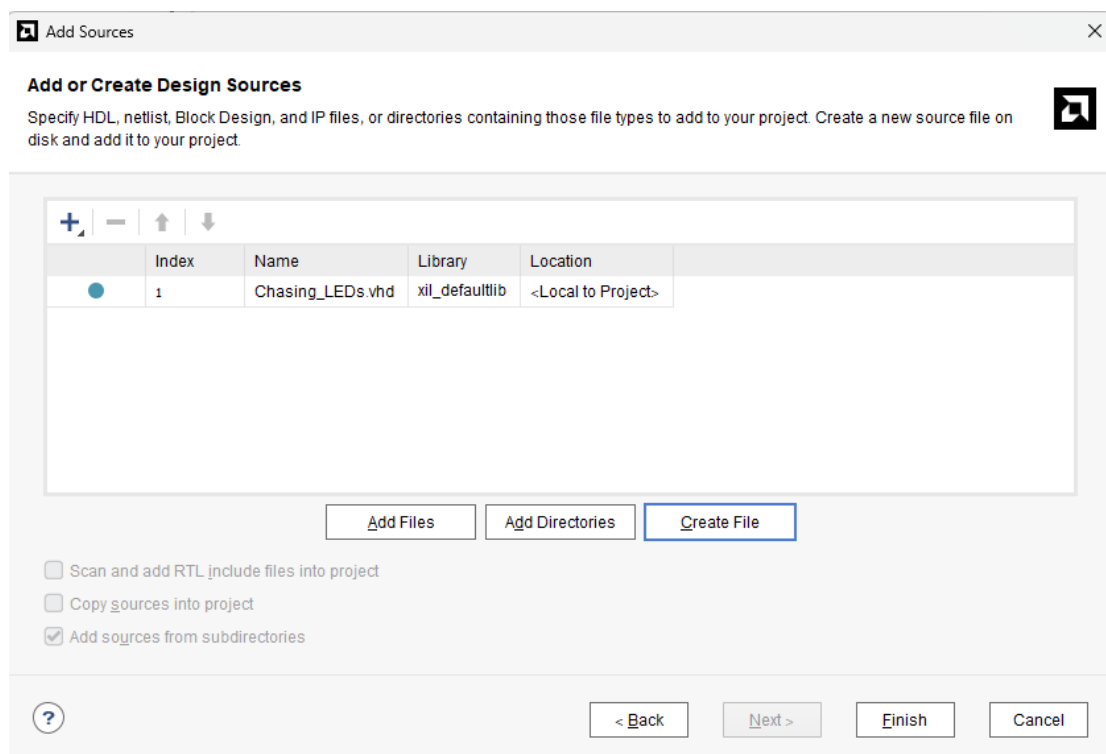
```

שלב 4:

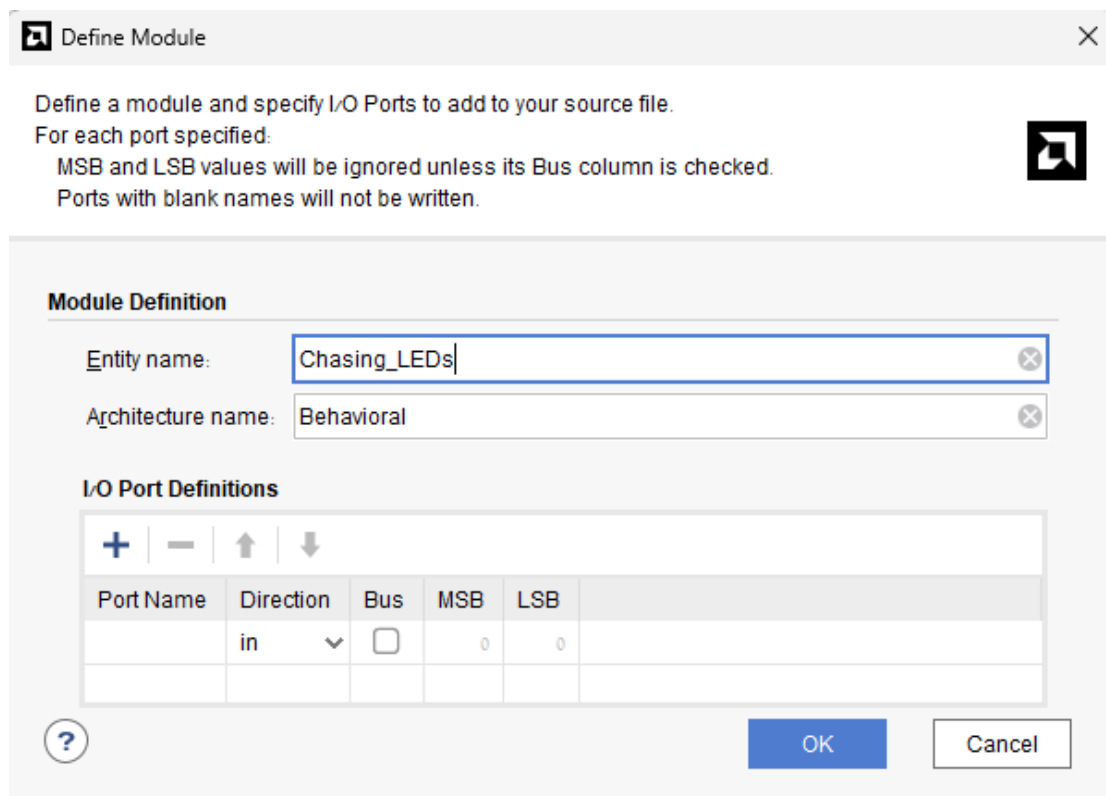
באותו אופן ניצור קובץ VHDL בשם Chasing_LEDs עבור מכונת המצבים.



נלחץ Finish.



נלחץ OK בחלון Define Module.



Define a module and specify I/O Ports to add to your source file.
For each port specified:
MSB and LSB values will be ignored unless its Bus column is checked.
Ports with blank names will not be written.

Module Definition

Entity name:

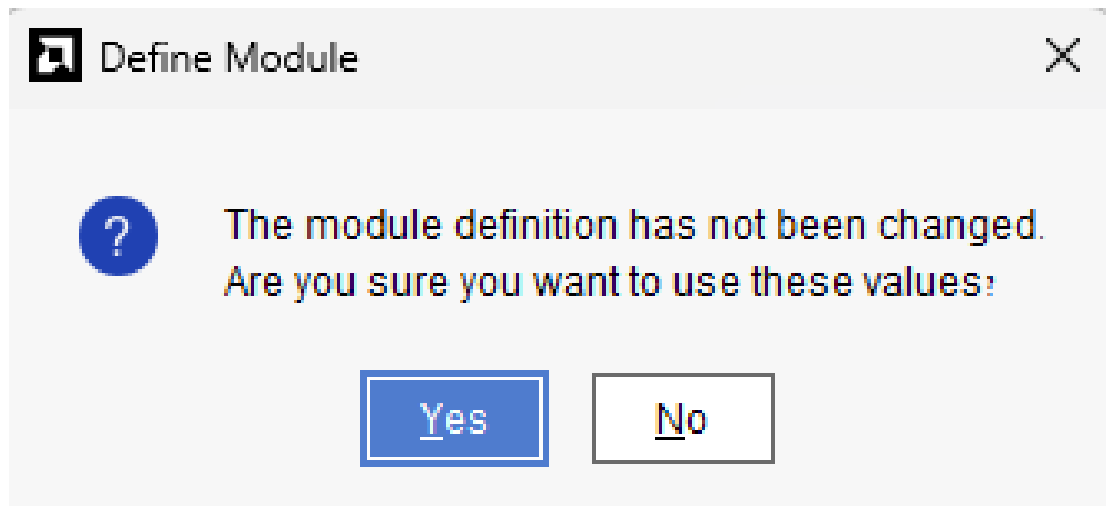
Architecture name:

I/O Port Definitions

Port Name	Direction	Bus	MSB	LSB
	in	☐	0	0

Buttons: ? OK Cancel

נאשר ב-Yes.

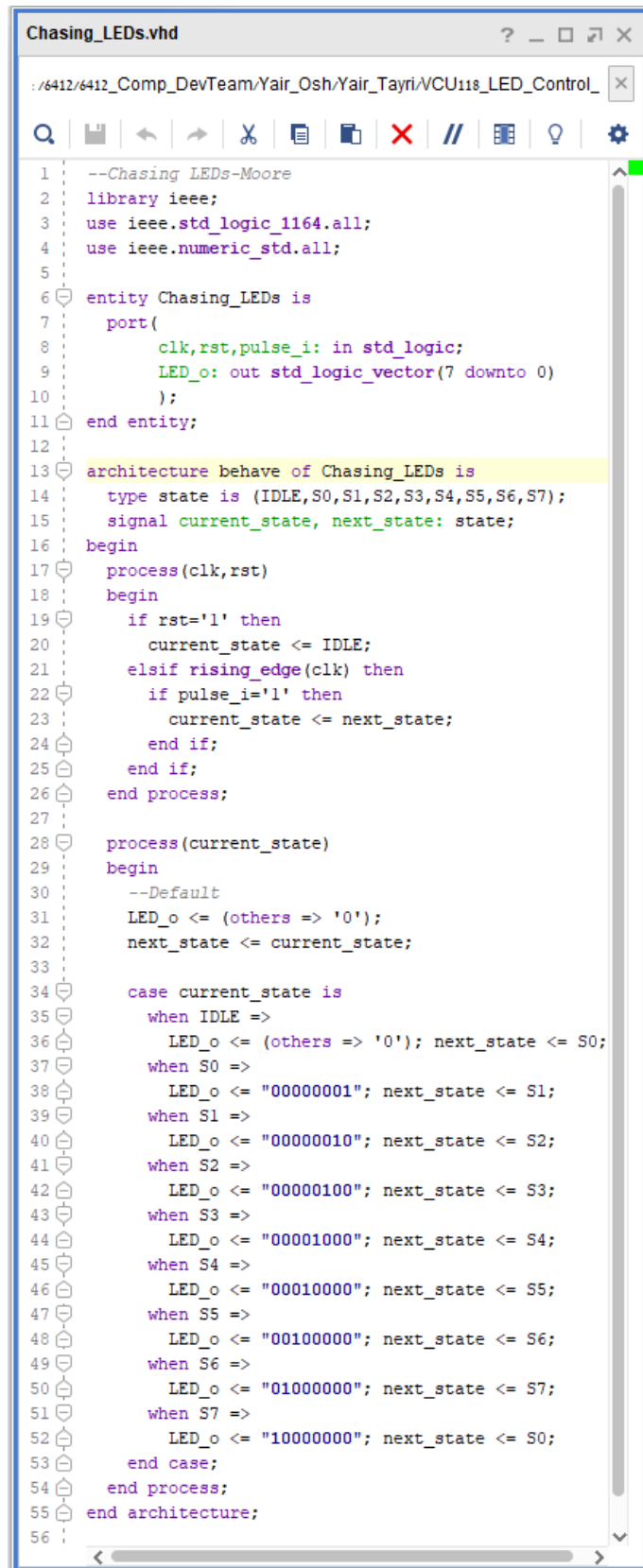


Define Module

? The module definition has not been changed.
Are you sure you want to use these values?

Buttons: Yes No

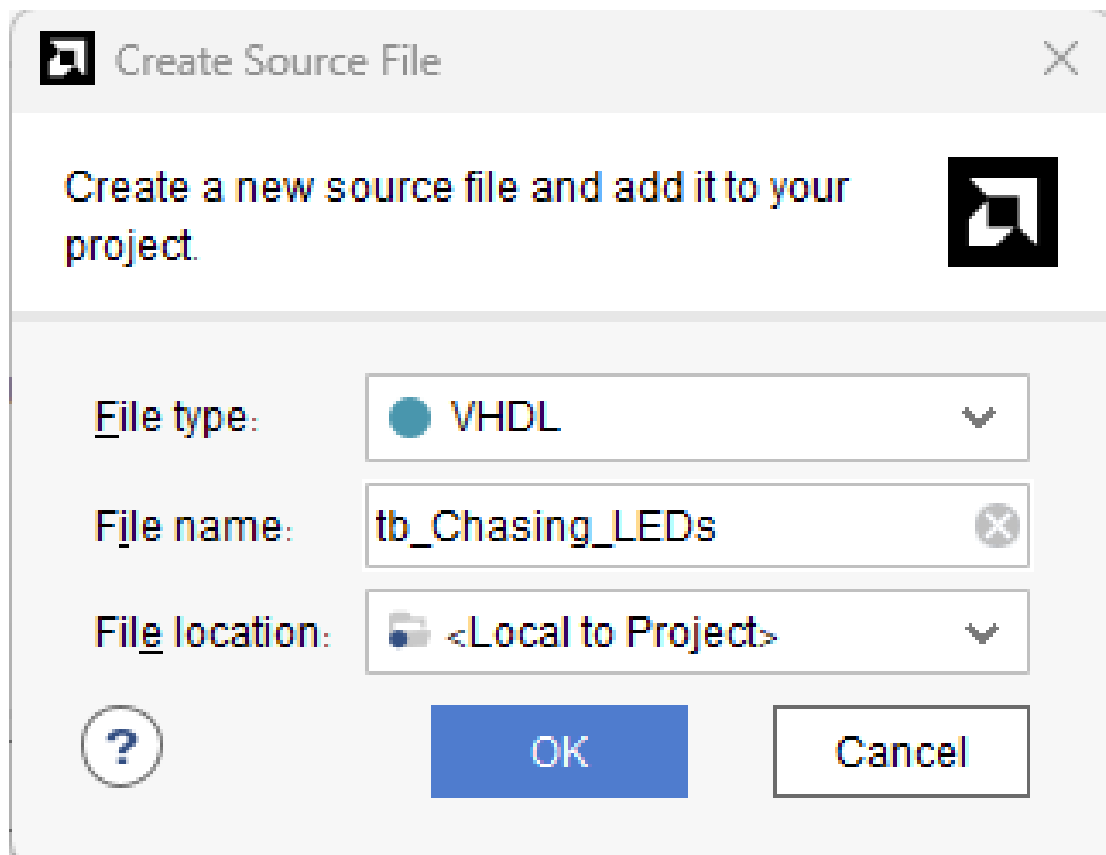
נכתוב את קוד המכונה. בכל עליית שעון שבה כניסת האפשרו pulse_i פעילה, המכונה עוברת למצב הבא ומזיזה את הLED הדולק במקום אחד.



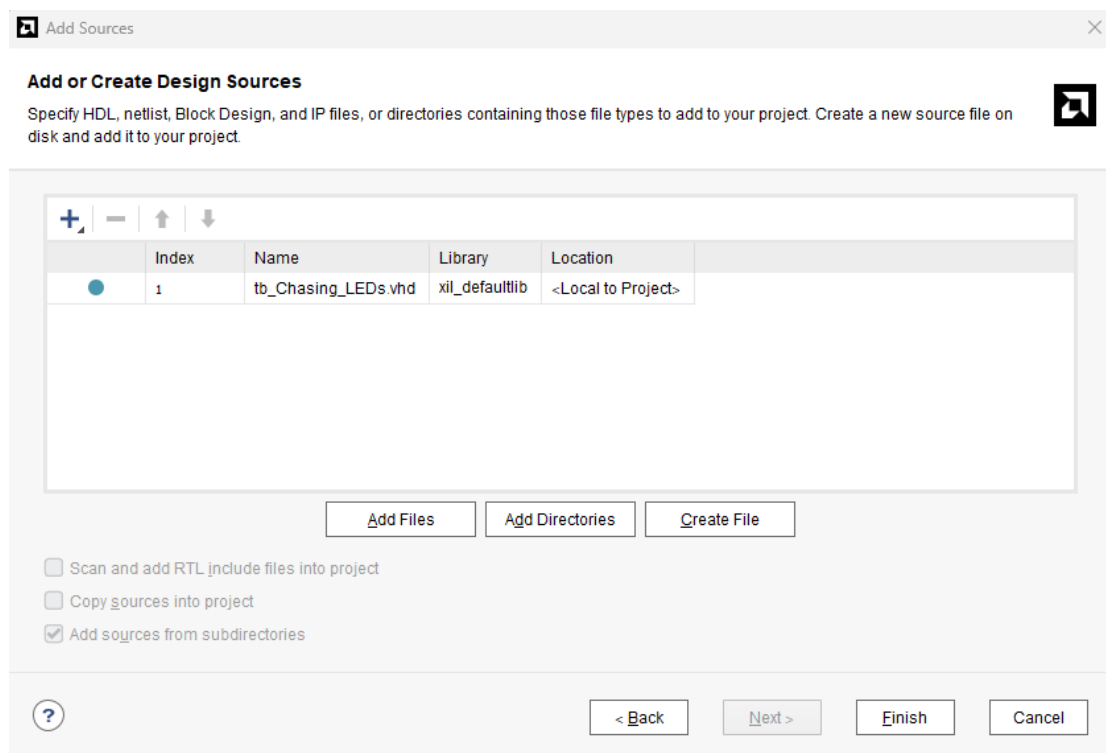
```
1  --Chasing_LEDs-Moore
2  library ieee;
3  use ieee.std_logic_1164.all;
4  use ieee.numeric_std.all;
5
6  entity Chasing_LEDs is
7      port(
8          clk,rst,pulse_i: in std_logic;
9          LED_o: out std_logic_vector(7 downto 0)
10         );
11 end entity;
12
13 architecture behave of Chasing_LEDs is
14     type state is (IDLE,S0,S1,S2,S3,S4,S5,S6,S7);
15     signal current_state, next_state: state;
16 begin
17     process(clk,rst)
18     begin
19         if rst='1' then
20             current_state <= IDLE;
21         elsif rising_edge(clk) then
22             if pulse_i='1' then
23                 current_state <= next_state;
24             end if;
25         end if;
26     end process;
27
28     process(current_state)
29     begin
30         --Default
31         LED_o <= (others => '0');
32         next_state <= current_state;
33
34         case current_state is
35             when IDLE =>
36                 LED_o <= (others => '0'); next_state <= S0;
37             when S0 =>
38                 LED_o <= "00000001"; next_state <= S1;
39             when S1 =>
40                 LED_o <= "00000010"; next_state <= S2;
41             when S2 =>
42                 LED_o <= "00000100"; next_state <= S3;
43             when S3 =>
44                 LED_o <= "00001000"; next_state <= S4;
45             when S4 =>
46                 LED_o <= "00010000"; next_state <= S5;
47             when S5 =>
48                 LED_o <= "00100000"; next_state <= S6;
49             when S6 =>
50                 LED_o <= "01000000"; next_state <= S7;
51             when S7 =>
52                 LED_o <= "10000000"; next_state <= S0;
53         end case;
54     end process;
55 end architecture;
56
```

שלב 5:

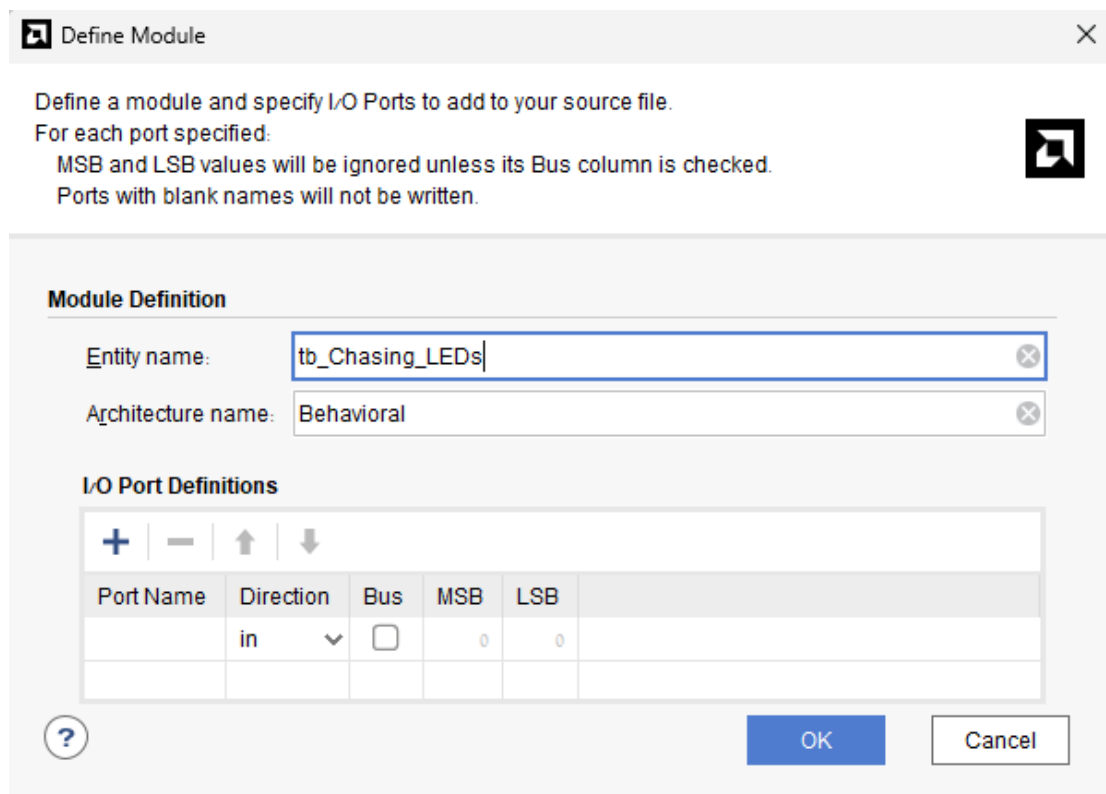
ניצור קובץ בדיקה בשם tb_Chasing_LEDs.



לחץ Finish.



נלחץ OK בחלון Define Module.



Define Module

Define a module and specify I/O Ports to add to your source file.
For each port specified:
MSB and LSB values will be ignored unless its Bus column is checked.
Ports with blank names will not be written.

Module Definition

Entity name:

Architecture name:

I/O Port Definitions

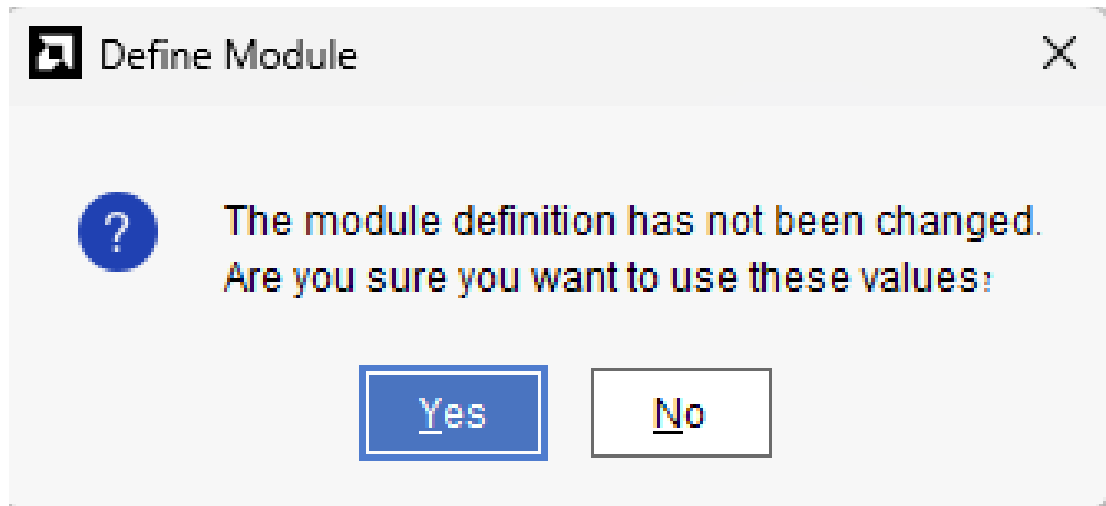
+ - ↑ ↓

Port Name	Direction	Bus	MSB	LSB
	in	☐	0	0

?

OK Cancel

נאשר ב-Yes.



Define Module

?

The module definition has not been changed.
Are you sure you want to use these values?

Yes No

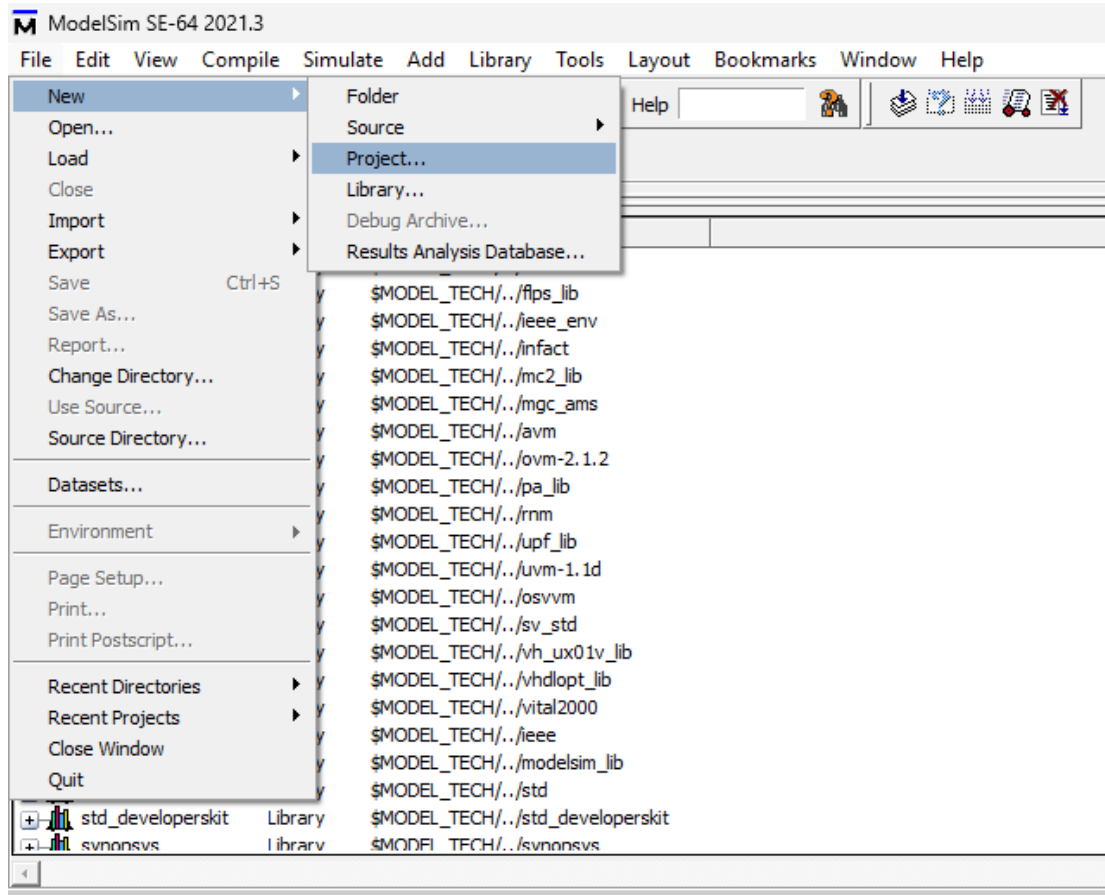
נכתוב את קוד הבדיקה. הקובץ מייצר שעון ואות איפוס ומפעיל את מכונת המצבים לצורך אימות בסימולציה.

```
tb_Chasing_LEDs.vhd
N:\6412\6412_Comp_DevTeam\Yair_Osh\Yair_Tayri\VCU118_LED_Control_System_Pr

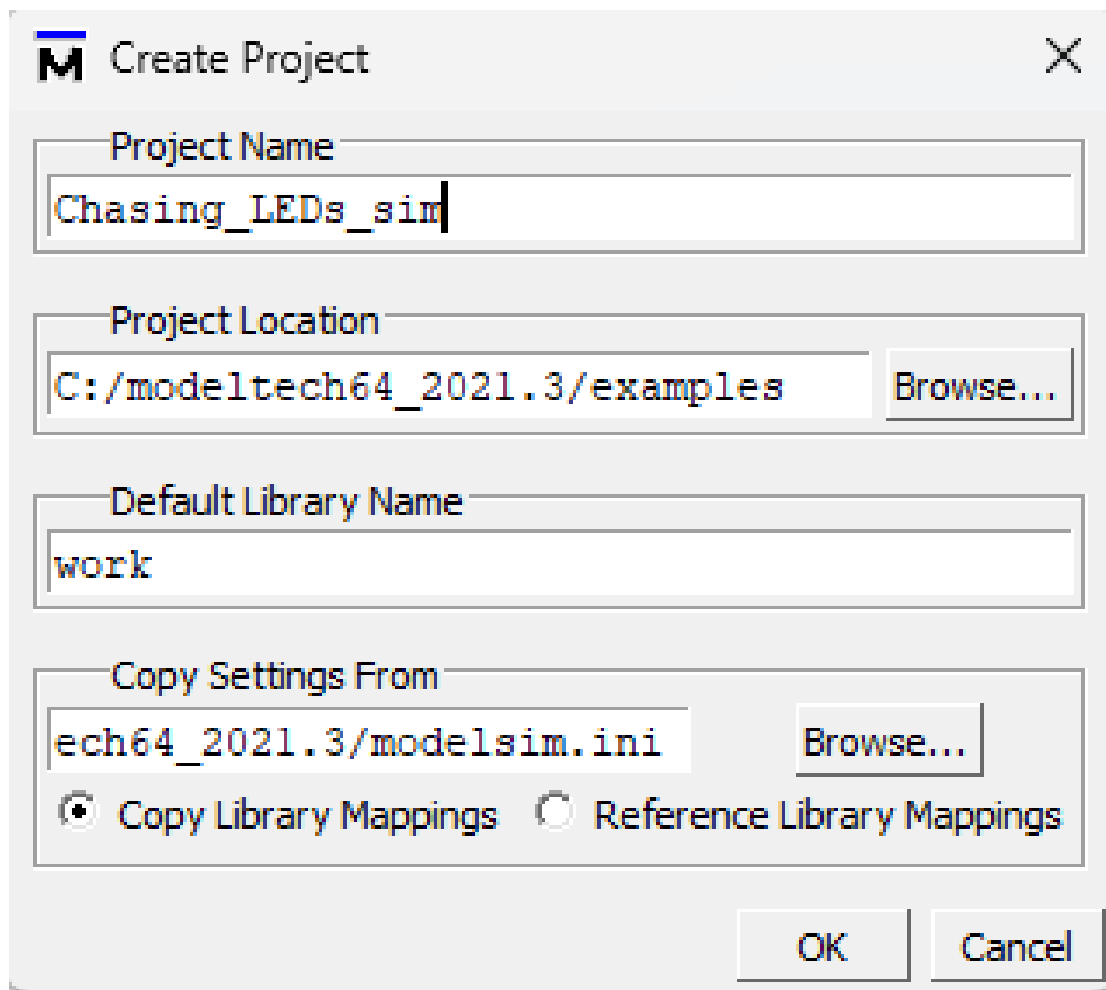
1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity tb_Chasing_LEDs is
5  end tb_Chasing_LEDs;
6
7  architecture behavior of tb_Chasing_LEDs is
8
9      component Chasing_LEDs is
10         port(
11             clk      : in std_logic;
12             rst       : in std_logic;
13             LED_o     : out std_logic_vector(7 downto 0)
14         );
15     end component;
16
17     signal clk_tb      : std_logic := '0';
18     signal rst_tb      : std_logic := '1';
19     signal LED_o_tb    : std_logic_vector(7 downto 0);
20
21     constant CLK_PERIOD : time := 10 ns;
22
23 begin
24
25     uut: Chasing_LEDs port map (
26         clk => clk_tb,
27         rst => rst_tb,
28         LED_o => LED_o_tb
29     );
30
31     clk_process : process
32     begin
33         clk_tb <= '0';
34         wait for CLK_PERIOD/2;
35         clk_tb <= '1';
36         wait for CLK_PERIOD/2;
37     end process;
38
39     stim_proc: process
40     begin
41         rst_tb <= '1';
42         wait for 20 ns;
43         rst_tb <= '0';
44         wait;
45     end process;
46
47 end behavior;
```

שלב 6:

נפתח את ModelSim, ניגש לתפריט File ובבחר New Project.



נזין שם לפרויקט Chasing_LEDs_sim ונלחץ OK.



The image shows a 'Create Project' dialog box with a title bar containing a logo and a close button. The dialog has four main sections, each with a label and a text input field. The first section, 'Project Name', has the text 'Chasing_LEDs_sim'. The second section, 'Project Location', has the text 'C:/modeltech64_2021.3/examples' and a 'Browse...' button. The third section, 'Default Library Name', has the text 'work'. The fourth section, 'Copy Settings From', has the text 'ech64_2021.3/modelsim.ini' and a 'Browse...' button. Below these sections are two radio buttons: 'Copy Library Mappings' (which is selected) and 'Reference Library Mappings'. At the bottom right are 'OK' and 'Cancel' buttons.

Create Project

Project Name
Chasing_LEDs_sim

Project Location
C:/modeltech64_2021.3/examples Browse...

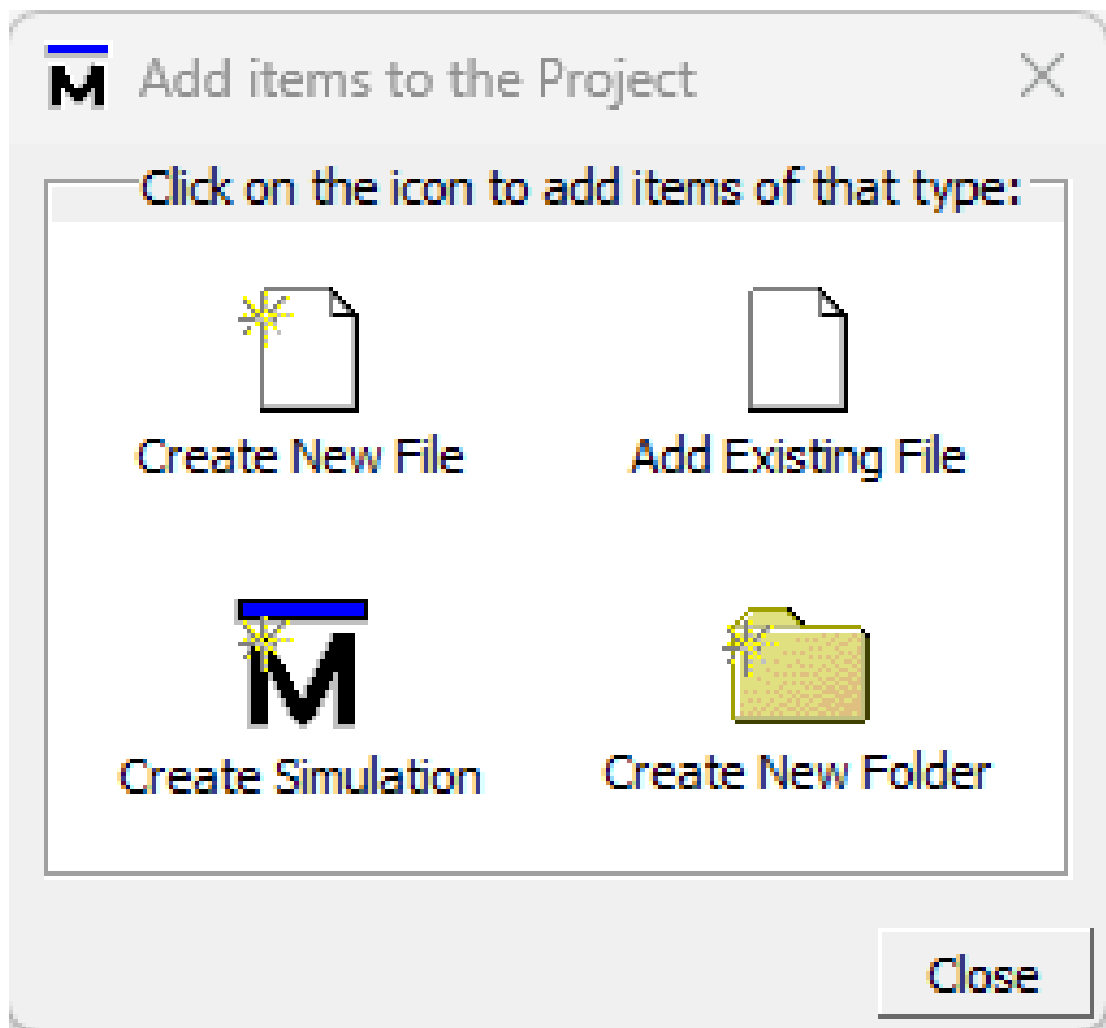
Default Library Name
work

Copy Settings From
ech64_2021.3/modelsim.ini Browse...

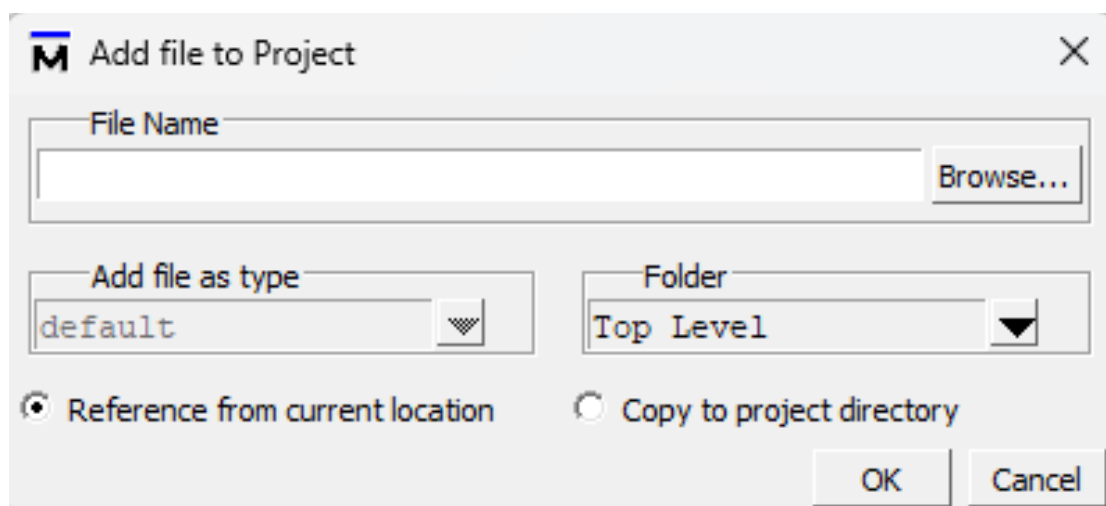
☒ Copy Library Mappings ☐ Reference Library Mappings

OK Cancel

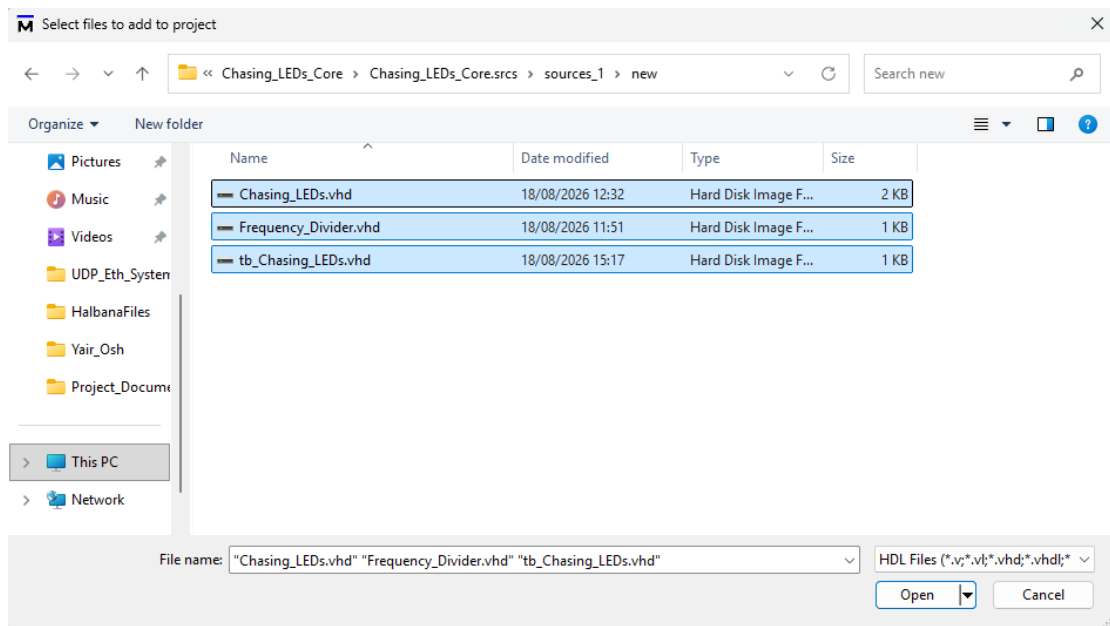
בחלון Add items to the Project נבחר Add Existing File.



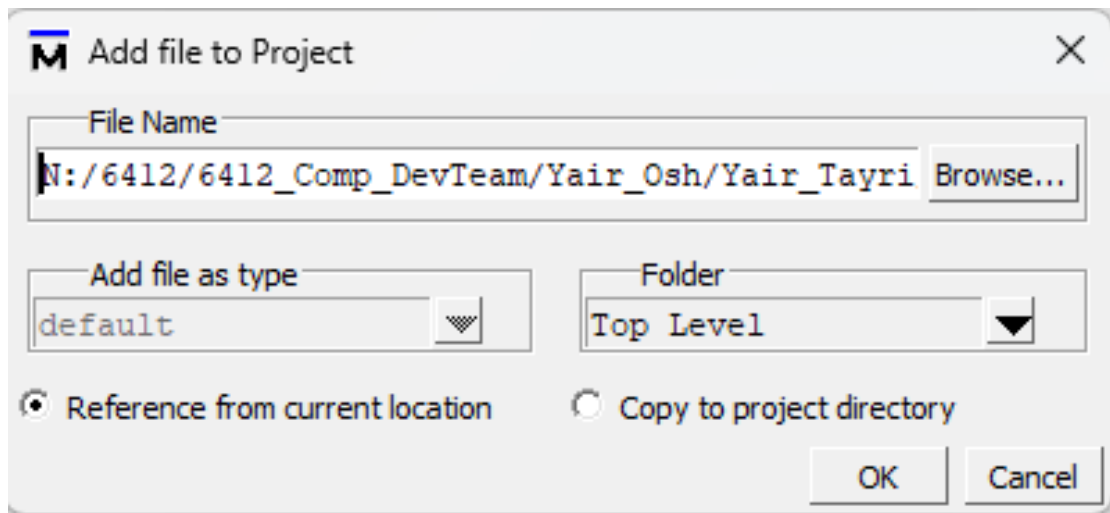
נלחץ Browse לבחירת הקבצים.



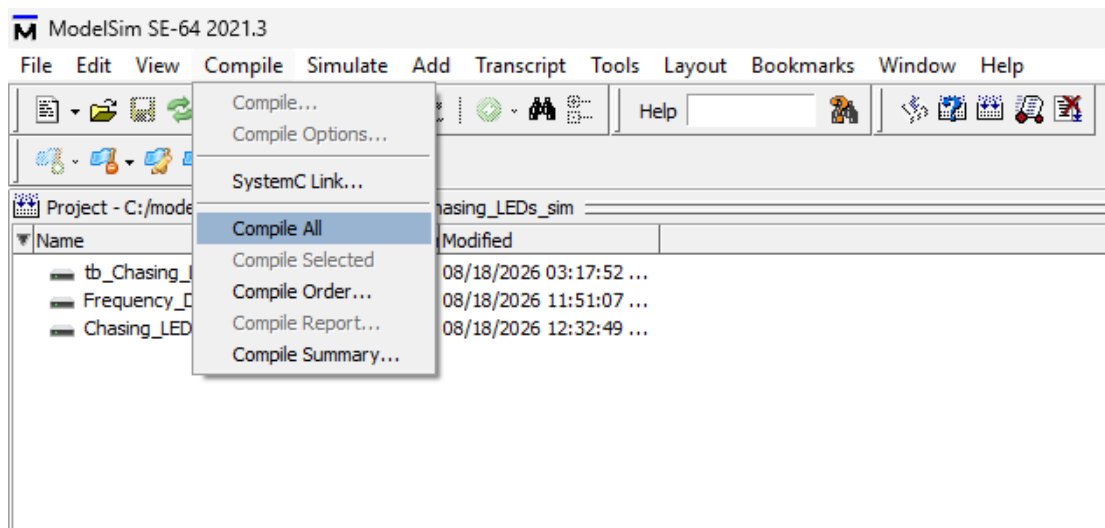
נסמן את שלושת קבצי ה-VHDL ונלחץ Open.



נאשר ב-OK.



נבחר Compile ואז Compile All.



בחלון ה-Transcript נוודא שכל הקבצים עברו הידור ללא שגיאות.

ModelSim SE-64 2021.3

File Edit View Compile Simulate Add Transcript Tools Layout Bookmarks Window Help

Project - C:/modeltech64_2021.3/examples/Chasing_LEDs_sim

Name	Status	Type	Order	Modified
tb_Chasing_LEDs....	✓	VHDL	2	08/18/2026 03:17:52 ...
Frequency_Divider...	✓	VHDL	1	08/18/2026 11:51:07 ...
Chasing_LEDs.vhd	✓	VHDL	0	08/18/2026 12:32:49 ...

Library Project

Transcript

```
# Reading pref.tcl
# // ModelSim SE-64 2021.3 Jul 13 2021
# //
# // Copyright 1991-2021 Mentor Graphics Corporation
# // All Rights Reserved.
# //
# // ModelSim SE-64 and its associated documentation contain trade
# // secrets and commercial or financial information that are the property of
# // Mentor Graphics Corporation and are privileged, confidential,
# // and exempt from disclosure under the Freedom of Information Act,
# // 5 U.S.C. Section 552. Furthermore, this information
# // is prohibited from disclosure under the Trade Secrets Act,
# // 18 U.S.C. Section 1905.
# //
# Loading project Chasing_LEDs_sim
# Compile of Chasing_LEDs.vhd was successful.
# Compile of Frequency_Divider.vhd was successful.
# Compile of tb_Chasing_LEDs.vhd was successful.
# 3 compiles, 0 failed with no errors.

ModelSim>
```

בלשונית Library מופיעים הרכיבים שהודרו לספריית work.

The screenshot displays the ModelSim SE-64 2021.3 software interface. The top menu bar includes File, Edit, View, Compile, Simulate, Add, Library, Tools, Layout, Bookmarks, Window, and Help. Below the menu is a toolbar with various icons for file operations and simulation. The main window is divided into two panes. The top pane, titled 'Library', shows a hierarchical tree of components. The bottom pane, titled 'Transcript', displays the output of the compilation process.

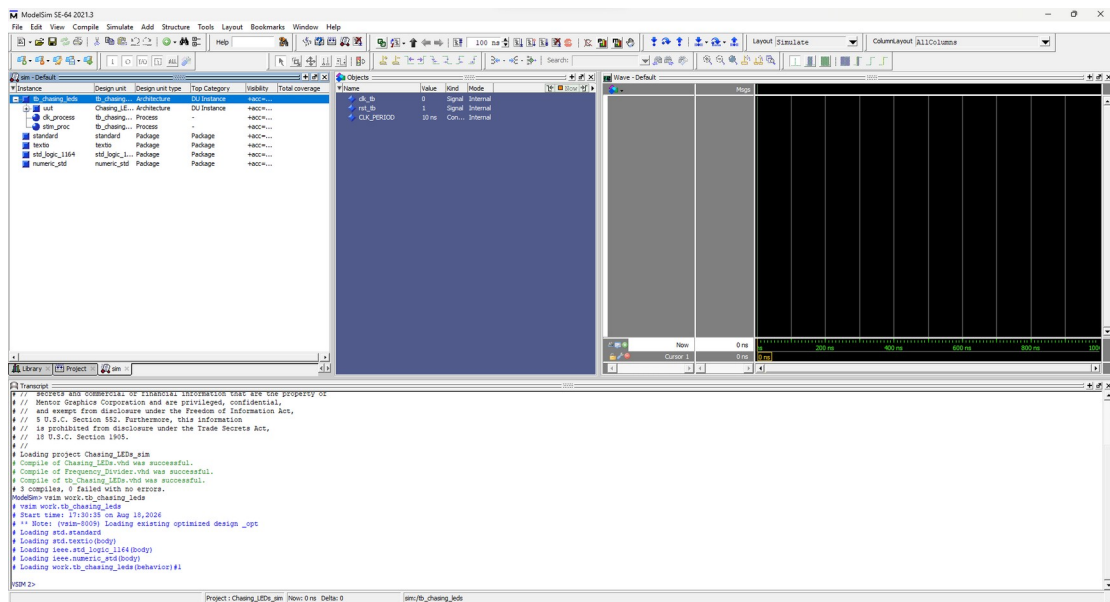
Name	Type	Path
work	Library	work
frequency_divid...	Entity	N:/6412/6412_Comp_DevTeam/Yair_...
chasing_leds	Entity	N:/6412/6412_Comp_DevTeam/Yair_...
tb_chasing_leds	Entity	N:/6412/6412_Comp_DevTeam/Yair_...
_opt1	Optimized...	
_opt	Optimized...	
floatfixlib	Library	\$MODEL_TECH/./floatfixlib
flps_lib	Library	\$MODEL_TECH/./flps_lib
ieee_env (empty)	Library	\$MODEL_TECH/./ieee_env
infact	Library	\$MODEL_TECH/./infact
mc2_lib	Library	\$MODEL_TECH/./mc2_lib
mgc_ams (empty)	Library	\$MODEL_TECH/./mgc_ams
mtiAvm	Library	\$MODEL_TECH/./avm
mtiOvm	Library	\$MODEL_TECH/./ovm-2.1.2
mtiPA (empty)	Library	\$MODEL_TECH/./pa_lib
mtiRnm	Library	\$MODEL_TECH/./rnm
mtiUPF	Library	\$MODEL_TECH/./upf_lib
mtiUvm	Library	\$MODEL_TECH/./uvm-1.1d
osvvm	Library	\$MODEL_TECH/./osvvm
sv_std	Library	\$MODEL_TECH/./sv_std

```

# Reading pref.tcl
# // ModelSim SE-64 2021.3 Jul 13 2021
# //
# // Copyright 1991-2021 Mentor Graphics Corporation
# // All Rights Reserved.
# //
# // ModelSim SE-64 and its associated documentation contain trade
# // secrets and commercial or financial information that are the property of
# // Mentor Graphics Corporation and are privileged, confidential,
# // and exempt from disclosure under the Freedom of Information Act,
# // 5 U.S.C. Section 552. Furthermore, this information
# // is prohibited from disclosure under the Trade Secrets Act,
# // 18 U.S.C. Section 1905.
# //
# Loading project Chasing_LEDs_sim
# Compile of Chasing_LEDs.vhd was successful.
# Compile of Frequency_Divider.vhd was successful.
# Compile of tb_Chasing_LEDs.vhd was successful.
# 3 compiles, 0 failed with no errors.

ModelSim>
  
```

נטען את הסימולציה ונפתח את חלון ה-Wave.

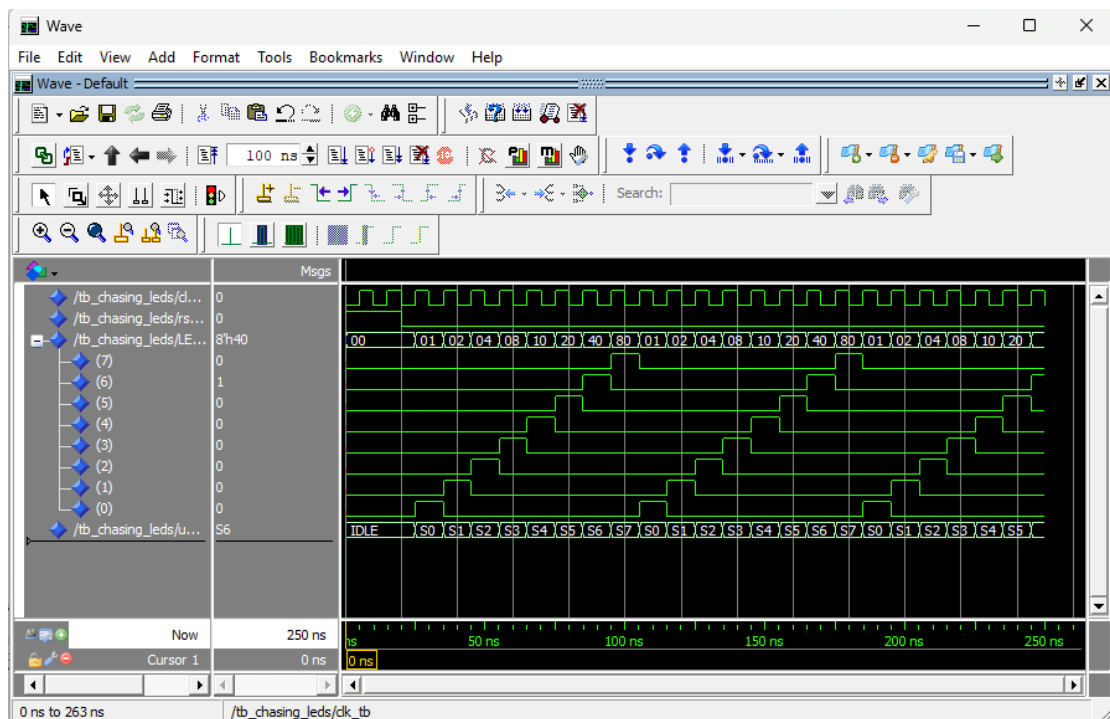


שלב 7:

נריץ את הסימולציה באמצעות הפקודות הבאות בחלון ה-Transcript:

```
vsim -voptargs="+acc" work.tb_chasing_leds
add wave clk_tb rst_tb LED_o_tb uut/current_state
run 250 ns
```

בחלון הגלים ניתן לראות את מעבר המצבים ואת הזזת הled הדולק בווקטור LED_o_tb.

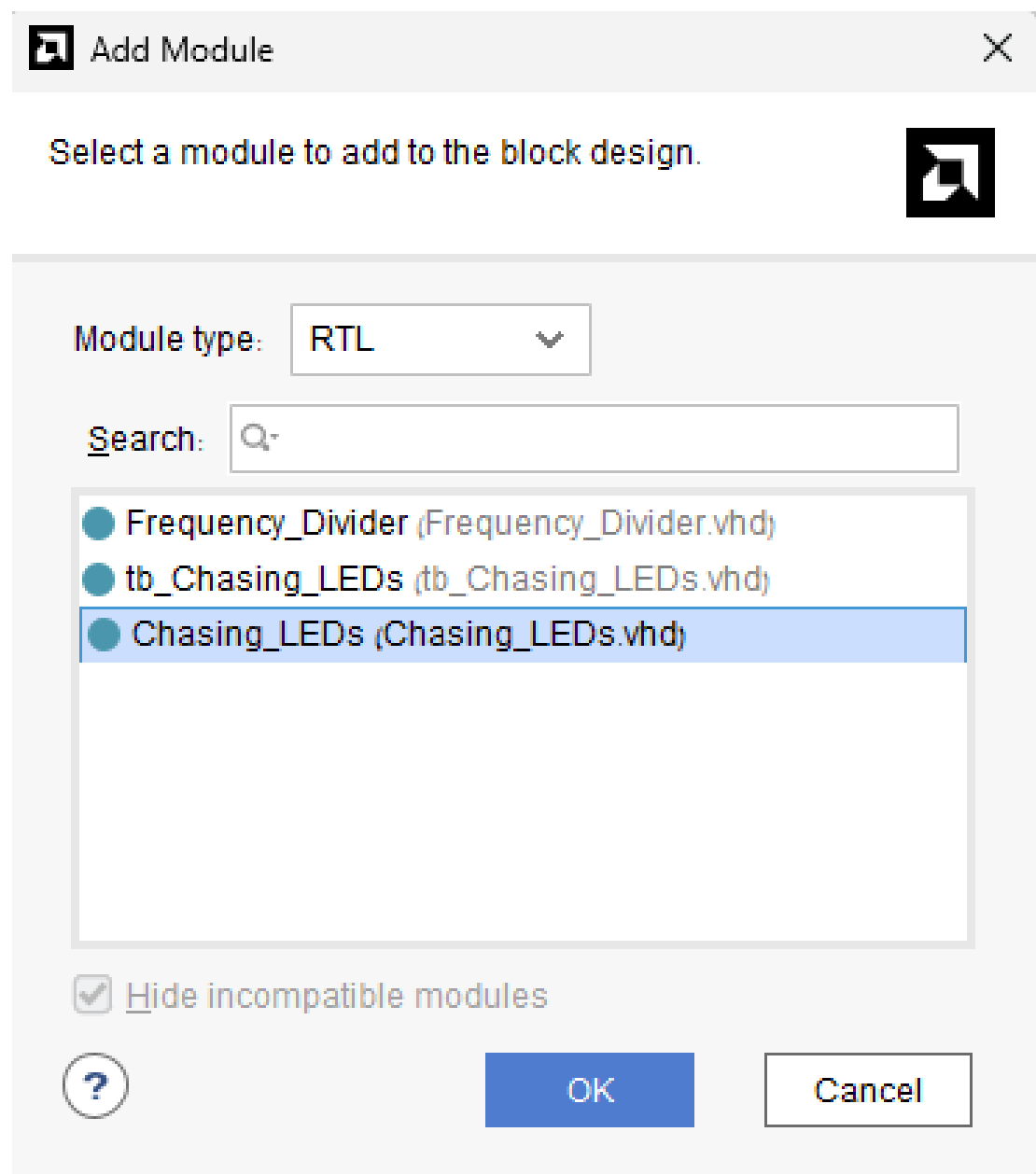


שלב 8:

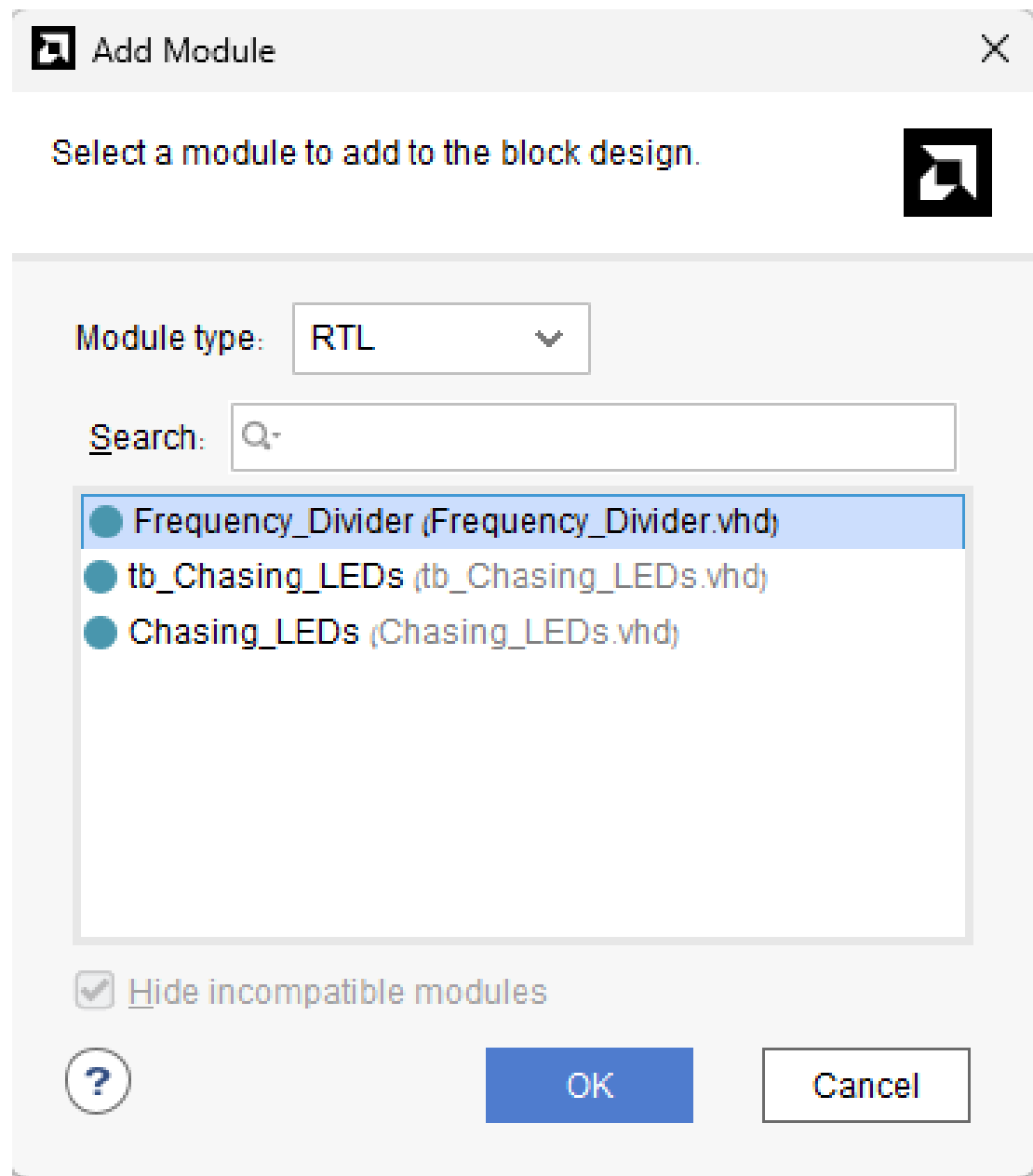
נחזור ל-Block Design, נלחץ קליק ימני ובחר Add Module.

	Properties...	Ctrl+E
✕	Delete	Delete
📄	Copy	Ctrl+C
📄	Paste	Ctrl+V
🔍	Search...	Ctrl+F
🖱️	Select All	Ctrl+A
+	Add IP...	Ctrl+I
	Add Module...	
	Pinning	▶
☑	Validate Design	F6
	Create Hierarchy...	
	Create Comment	
	Create Port...	Ctrl+K
	Create Interface Port...	Ctrl+L
🔄	Regenerate Layout	
📄	Save as PDF File...	

נבחר את Chasing_LEDs ונלחץ OK.

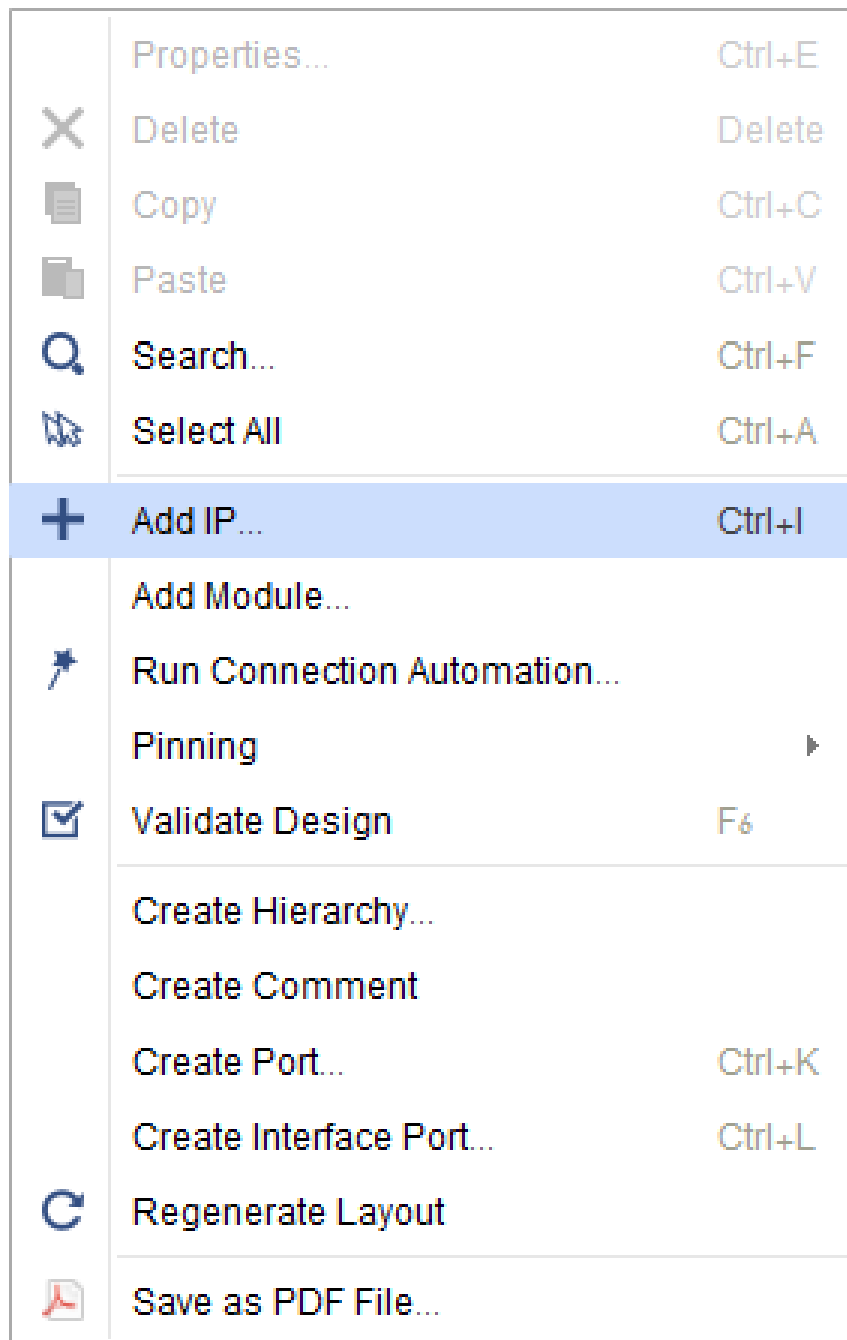


באותו אופן נוסף את Frequency_Divider.

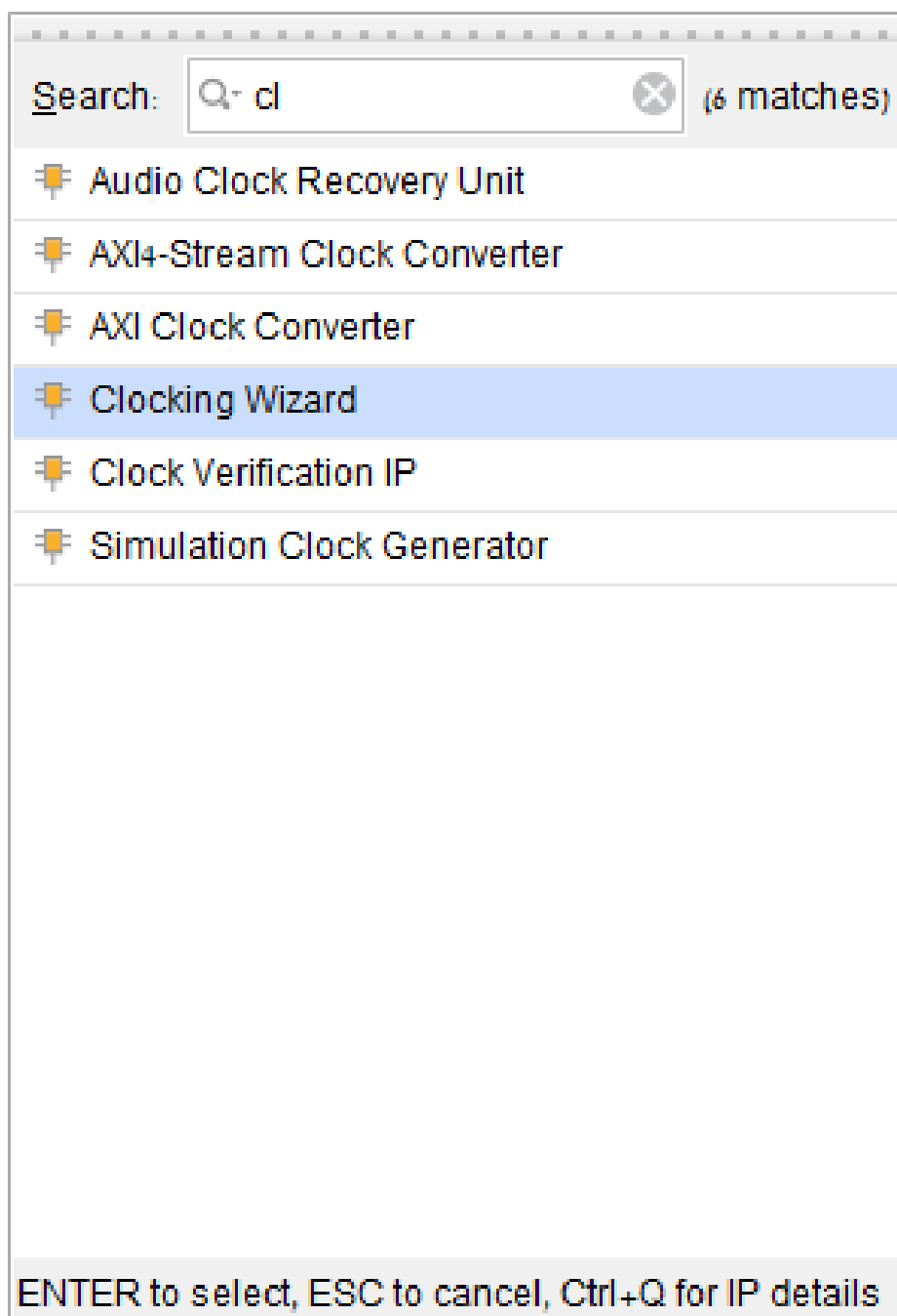


שלב 9:

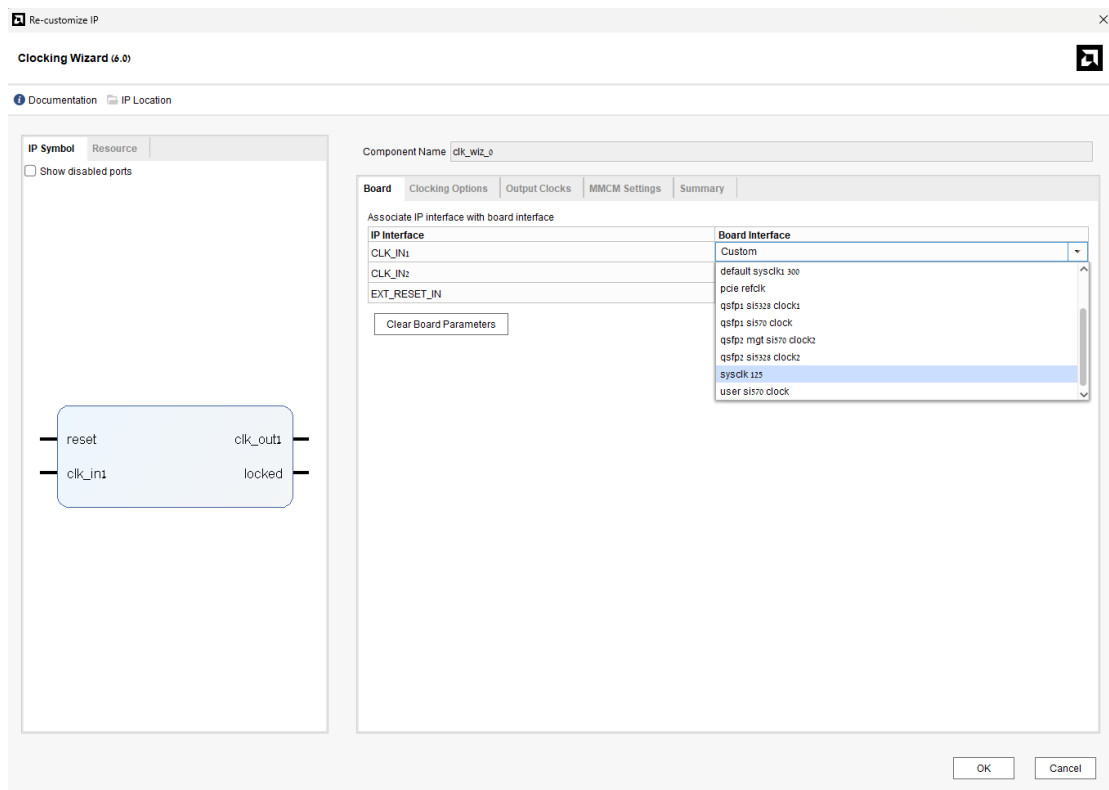
נלחץ קליק ימני ובחר Add IP.



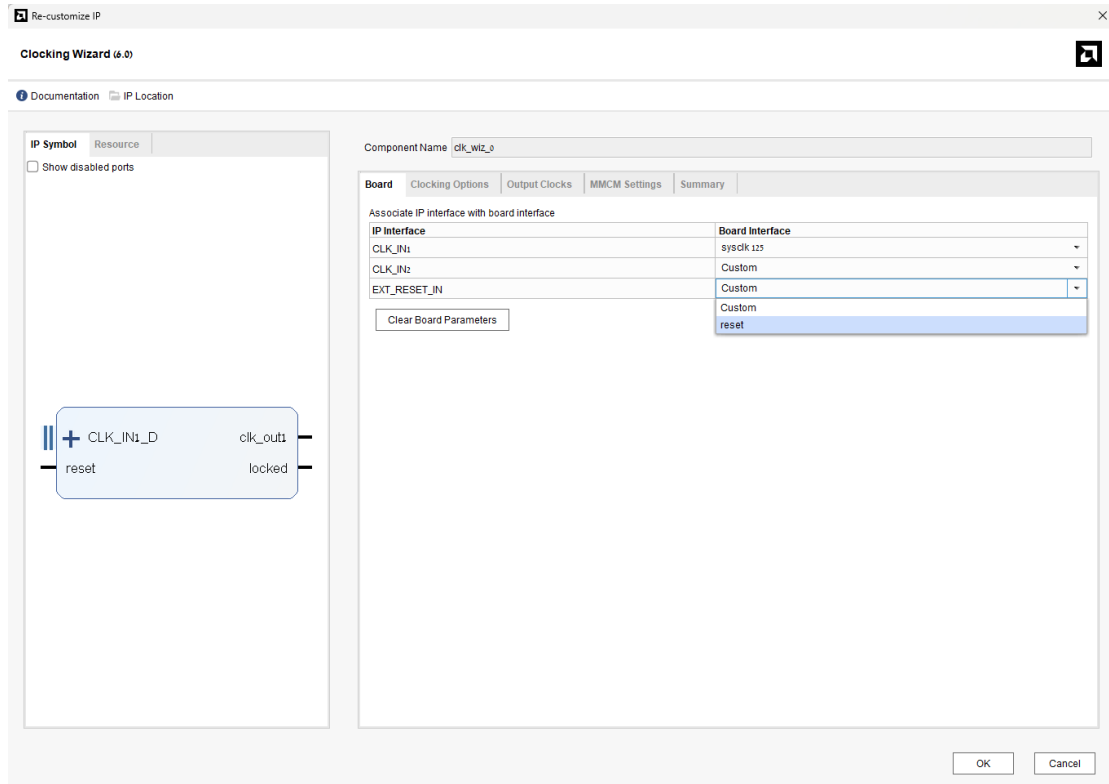
נחפש Clocking Wizard ונוסיף אותו לשרטוט.



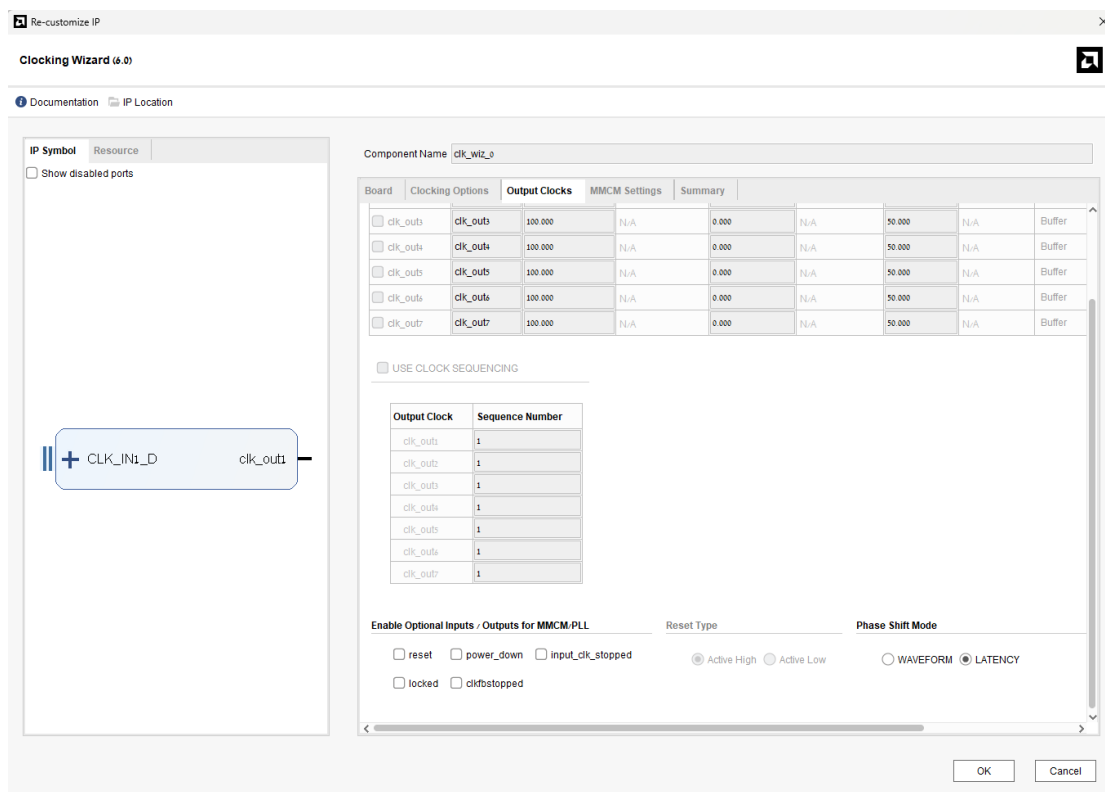
בלשונית Board נשייך את CLK_IN1 למתנד sysclk 125 של הלוח.



באותה לשונית נגדיר את שיוך כניסת האיפוס.

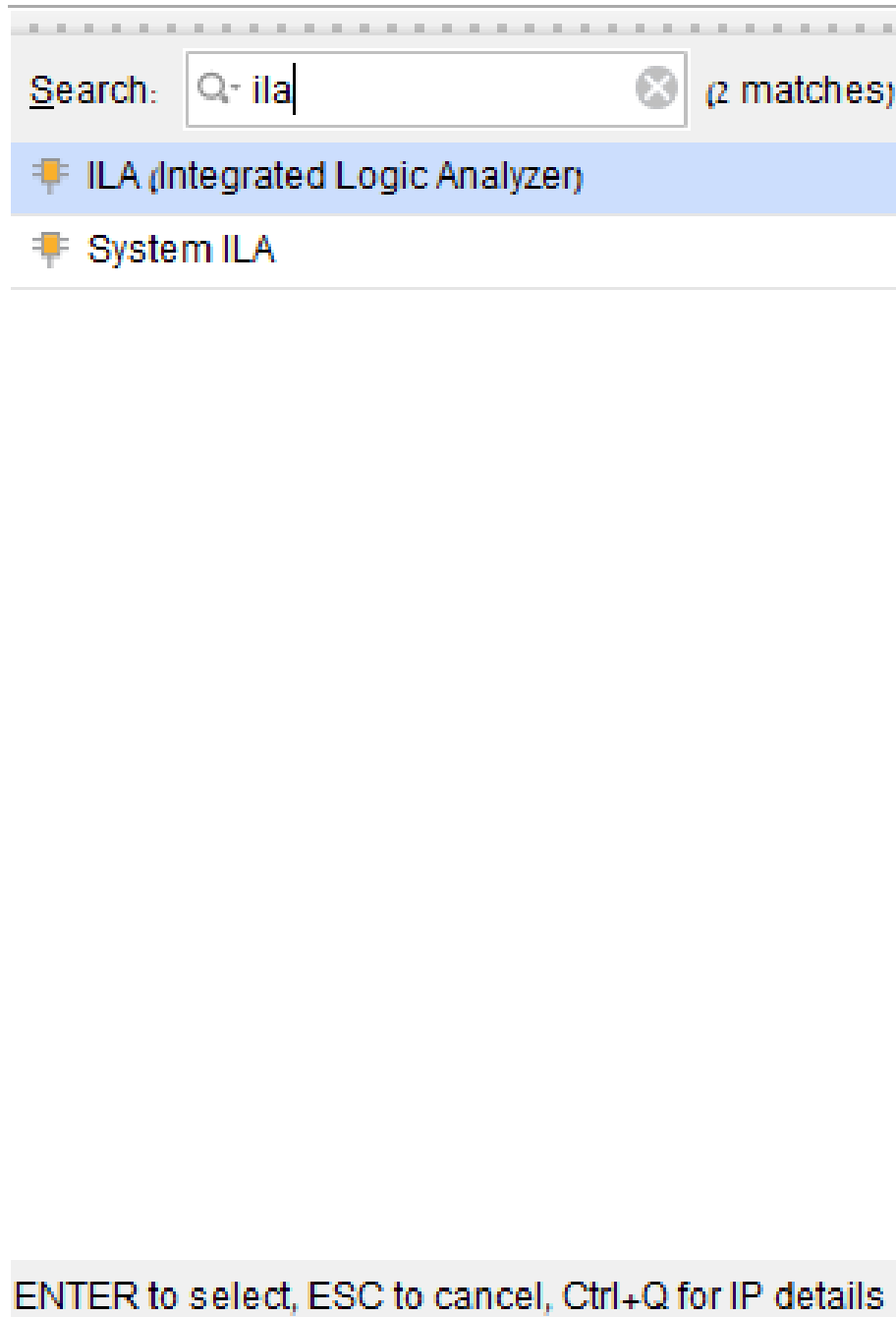


בלשונית Output Clocks נגדיר את clk_out1 ל-100 MHz ונסיר את הסימון מכניסת ה-reset.

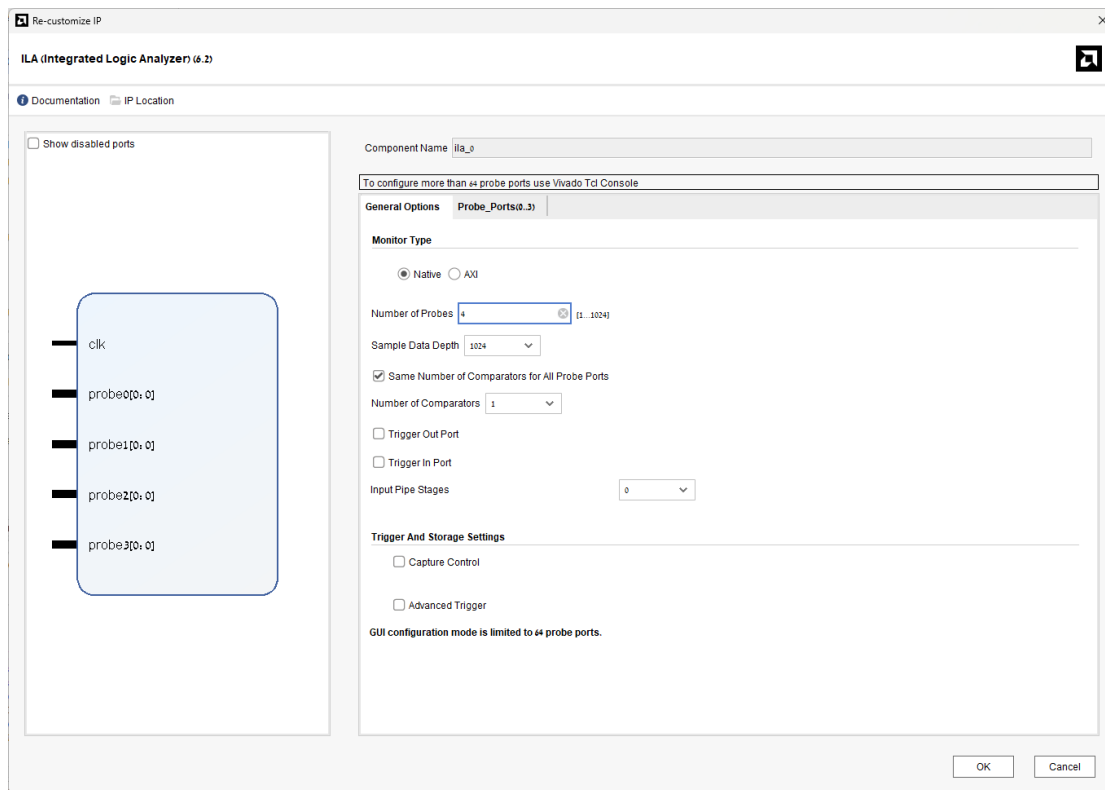


שלב 10:

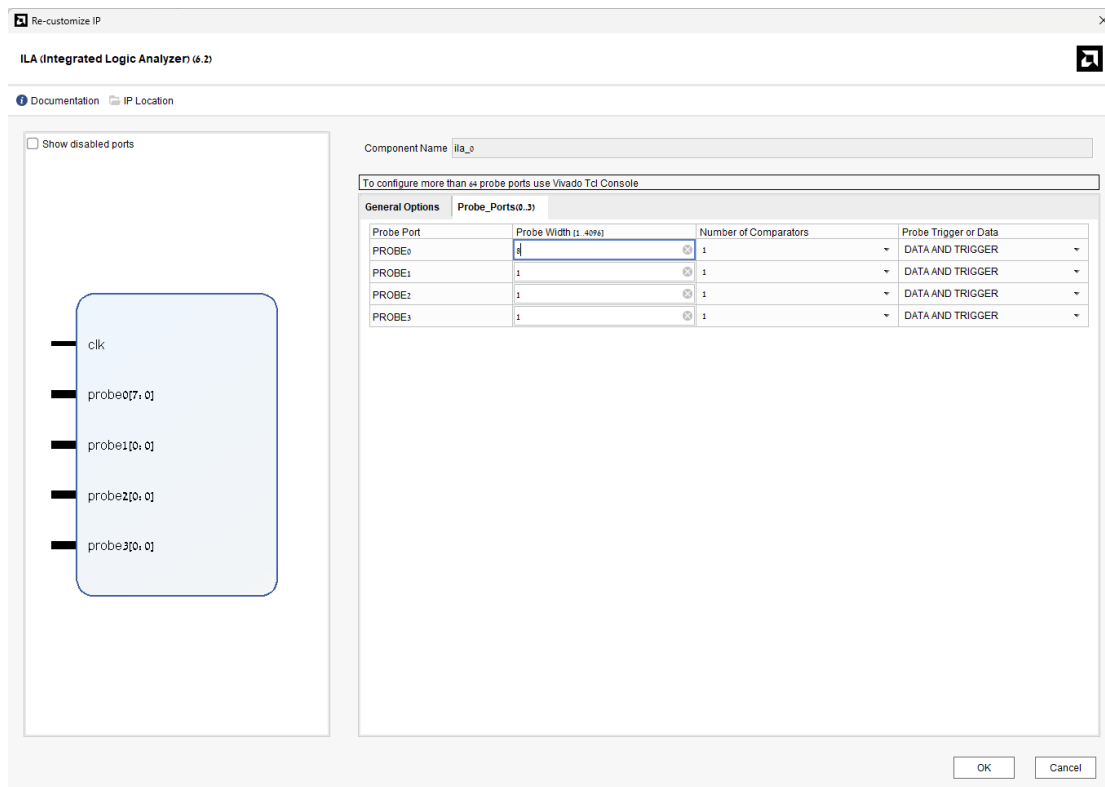
נוסיף רכיב ILA.



בלשונית General Options נגדיר את מספר הפרובים.

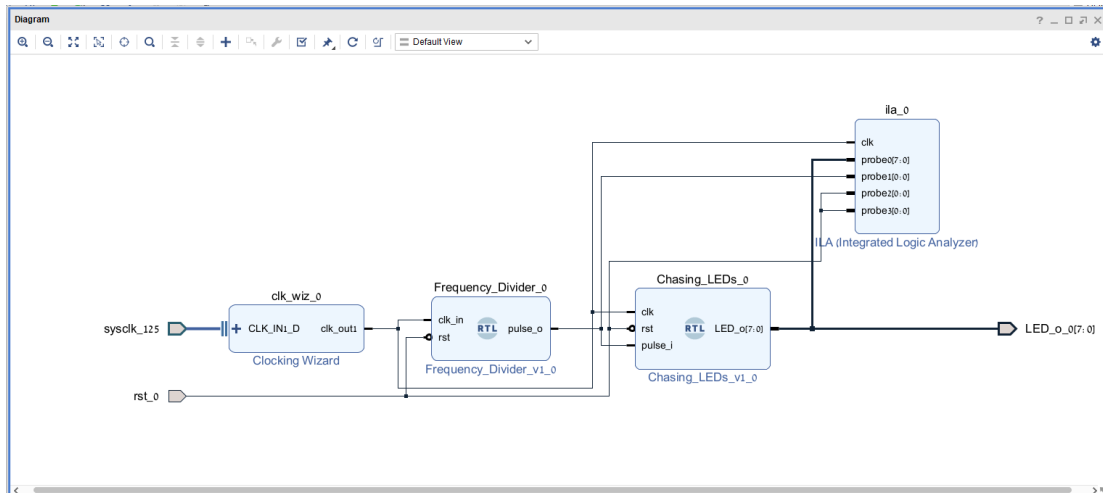


בלשונית Probe Ports נגדיר את רוחב כל פרוב בהתאם לאותות שנרצה לדגום, ובהם יציאת הפולס וקטור הלדים.



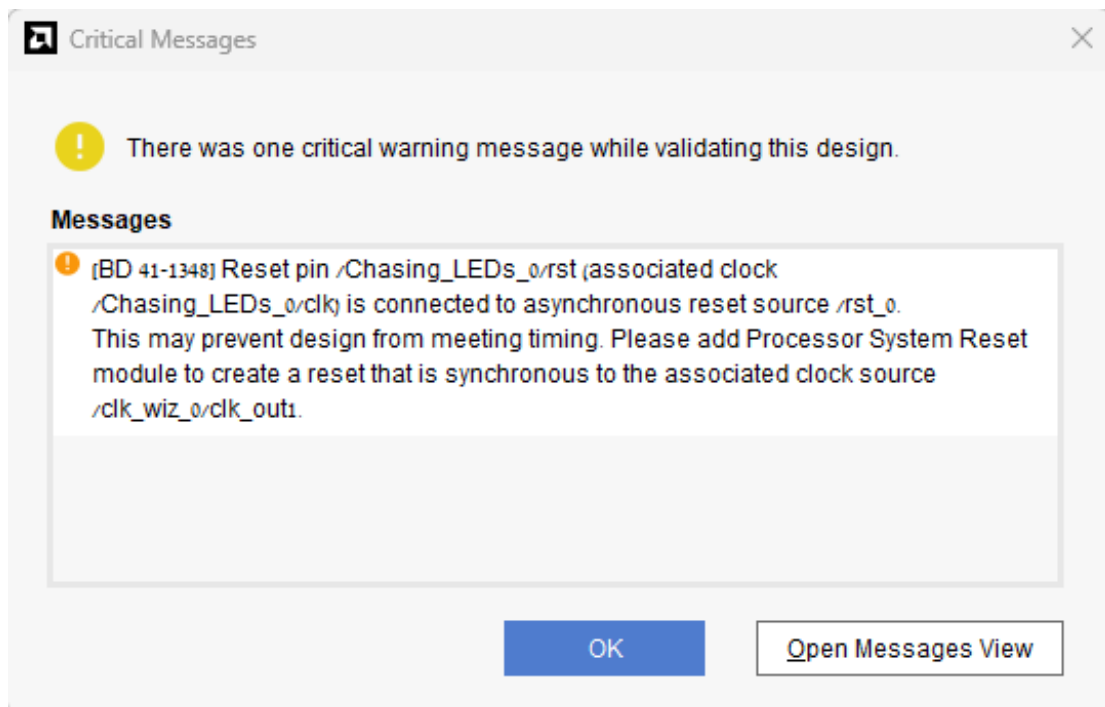
שלב 11:

נחבר את הרכיבים. יציאת clk_out1 בתדר 100 MHz מוזנת גם לכניסת השעון של מחלק התדר וגם לכניסת השעון של מכונת המצבים. יציאת pulse_o של המחלק מחוברת לכניסת pulse_i של המכונה ומשמשת ככניסת אפשר Clock Enable ולא כשעון. כך כל הלוגיקה פועלת על רשת שעון אחת. שימוש בפולס כשעון ישיר היה יוצר תחום שעון נפרד ומגדיל את הסיכון לשגיאות תזמון.



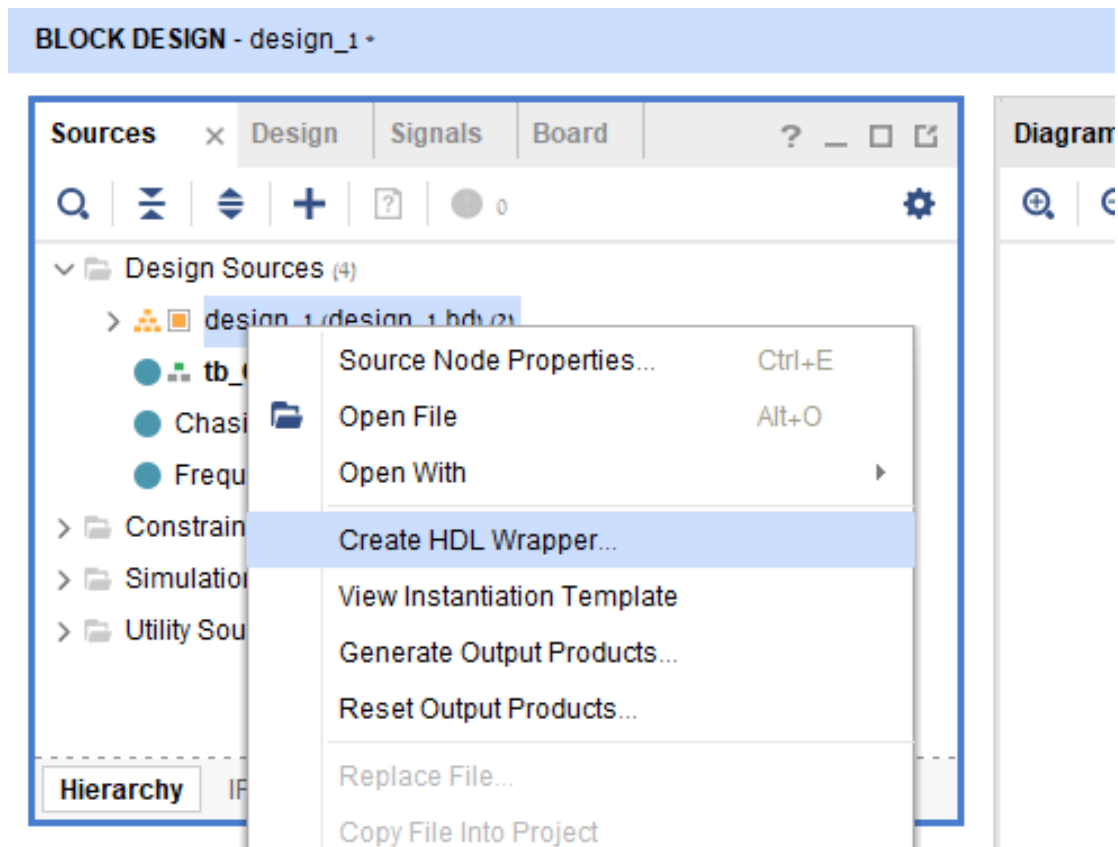
שלב 12:

נלחץ על Validate Design. Vivado מתריע על אות איפוס המחובר למקור איסינכרוני. האזהרה אינה חוסמת את המשך הזרימה בתכן זה, ובפרקים הבאים ישולב רכיב Processor System Reset לסנכרון האיפוס מול השעון.

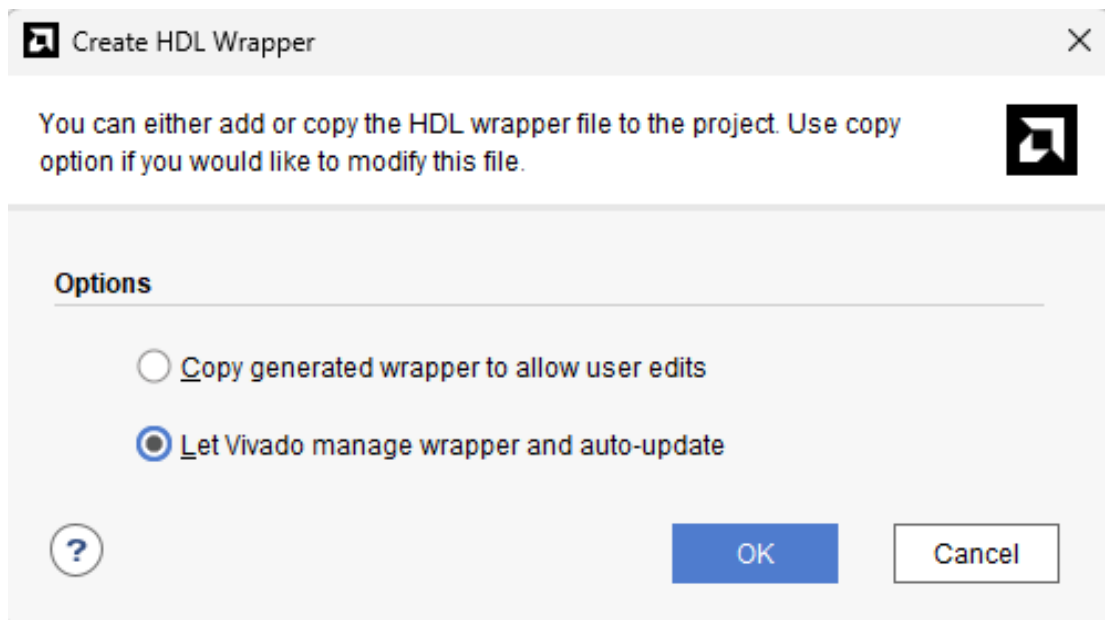


שלב 13:

בחלון Sources נלחץ קליק ימני על design_1 ונבחר Create HDL Wrapper.

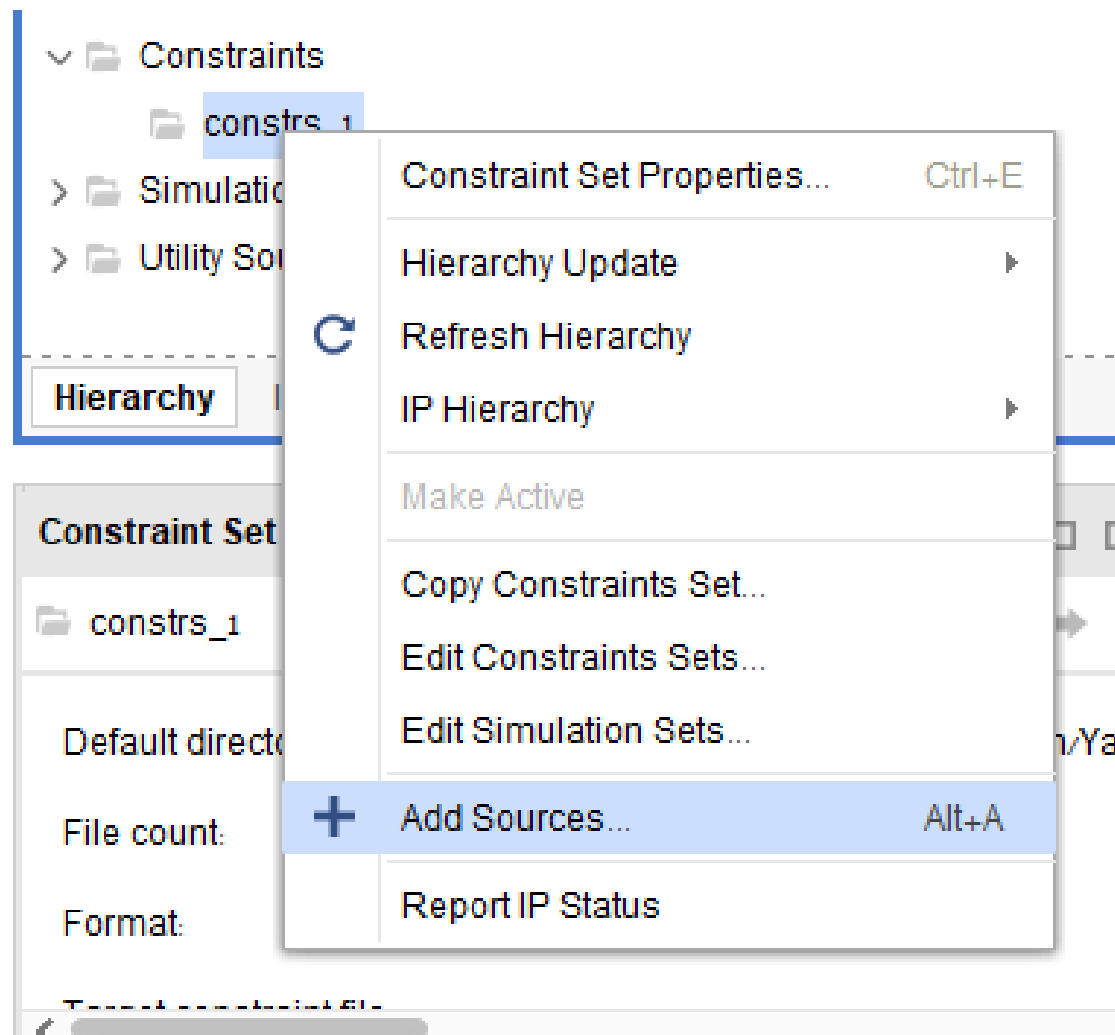


נשאיר את האפשרות לניהול אוטומטי על ידי Vivado ונלחץ OK.

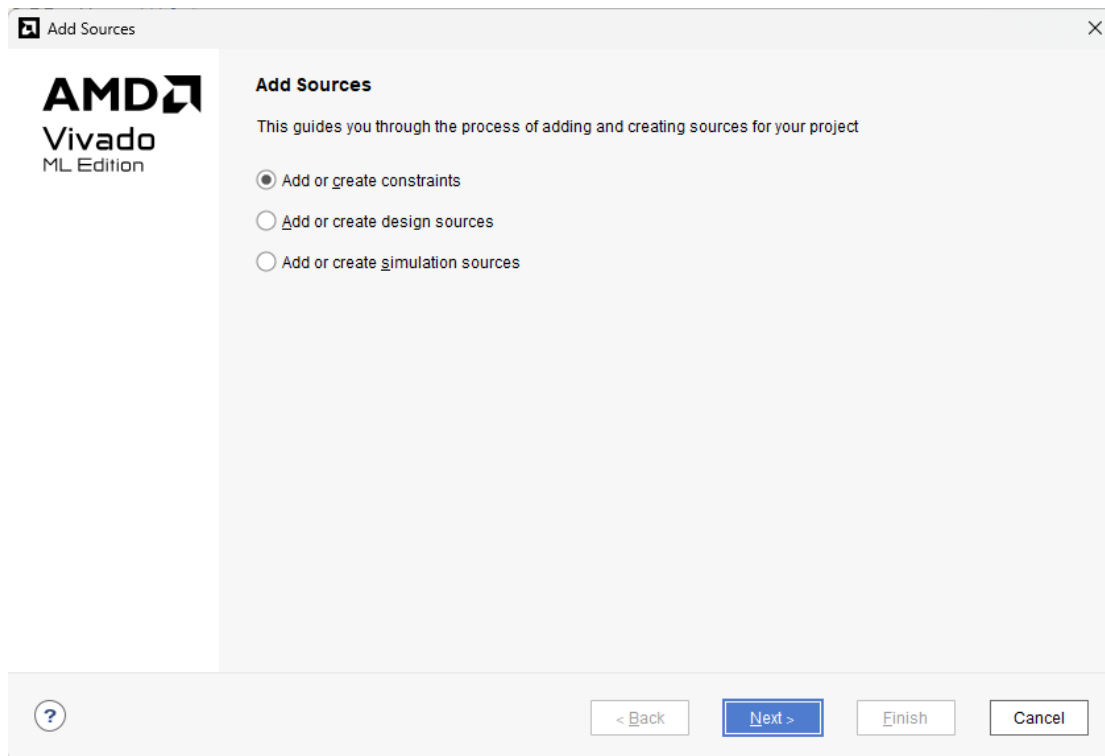


שלב 14:

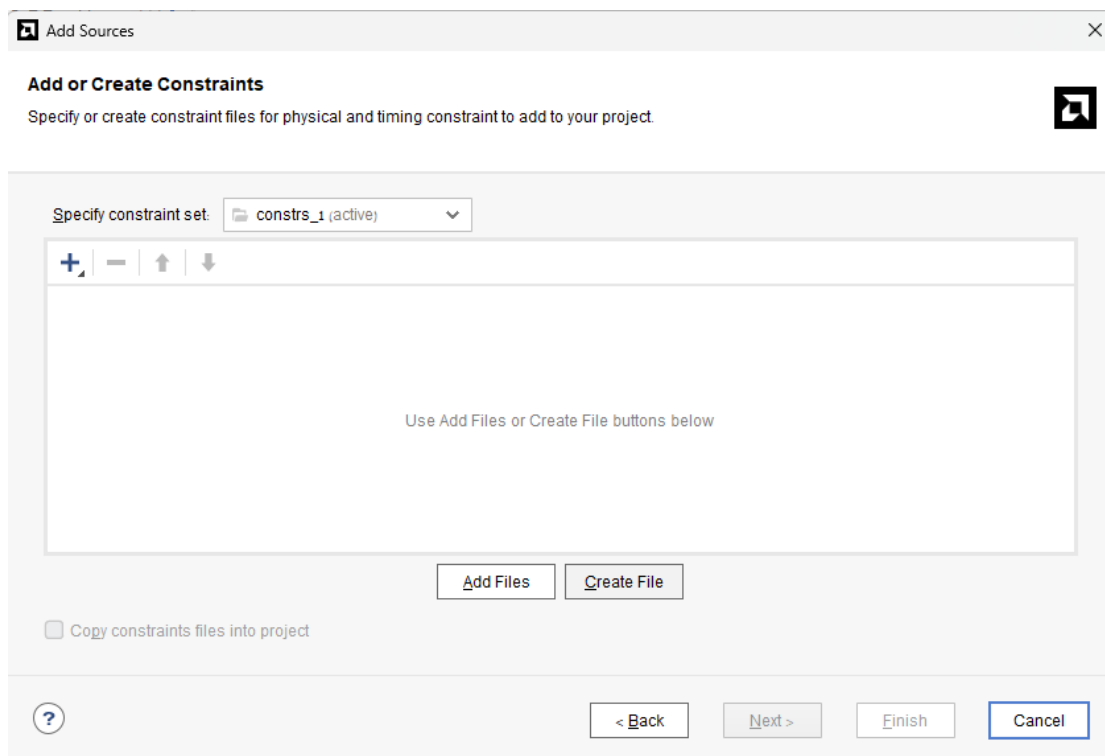
נלחץ קליק ימני על Constraints ונבחר Add Sources.



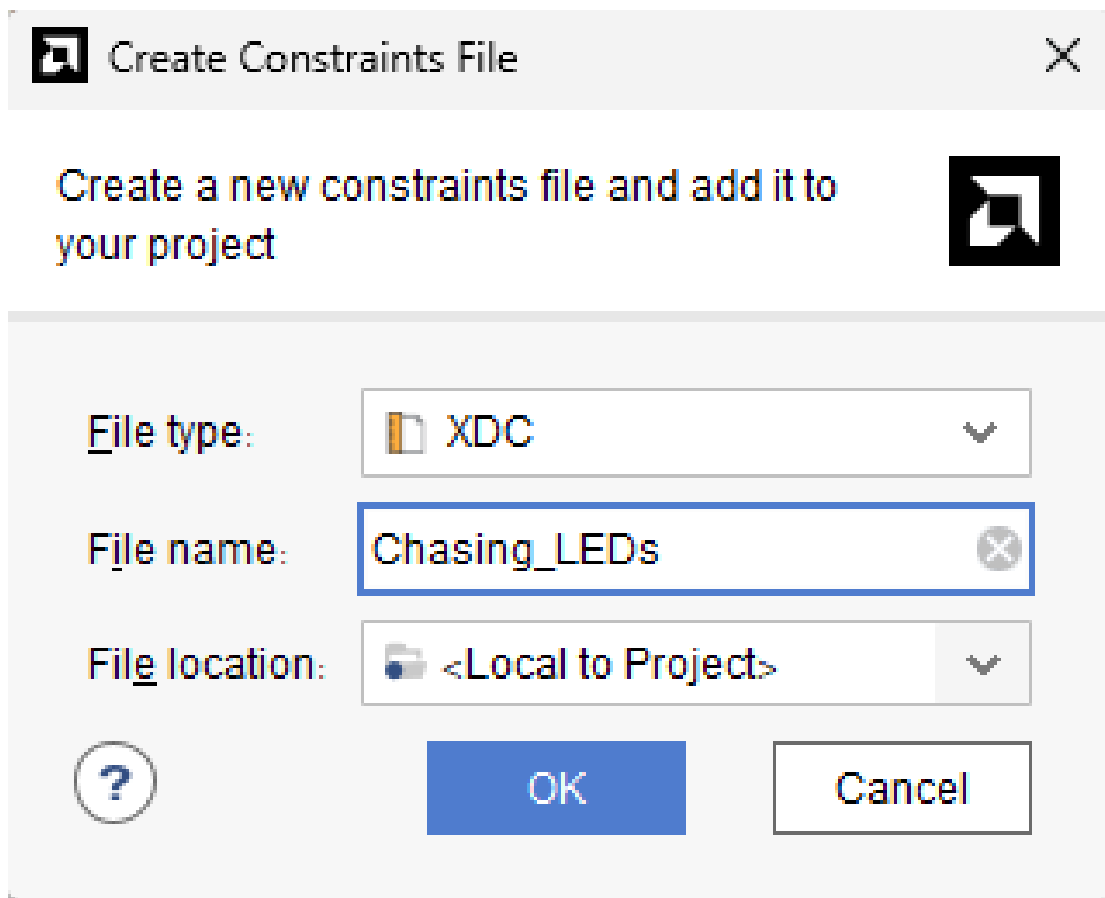
נבחר Add or create constraints ונלחץ Next.



נלחץ Create File.



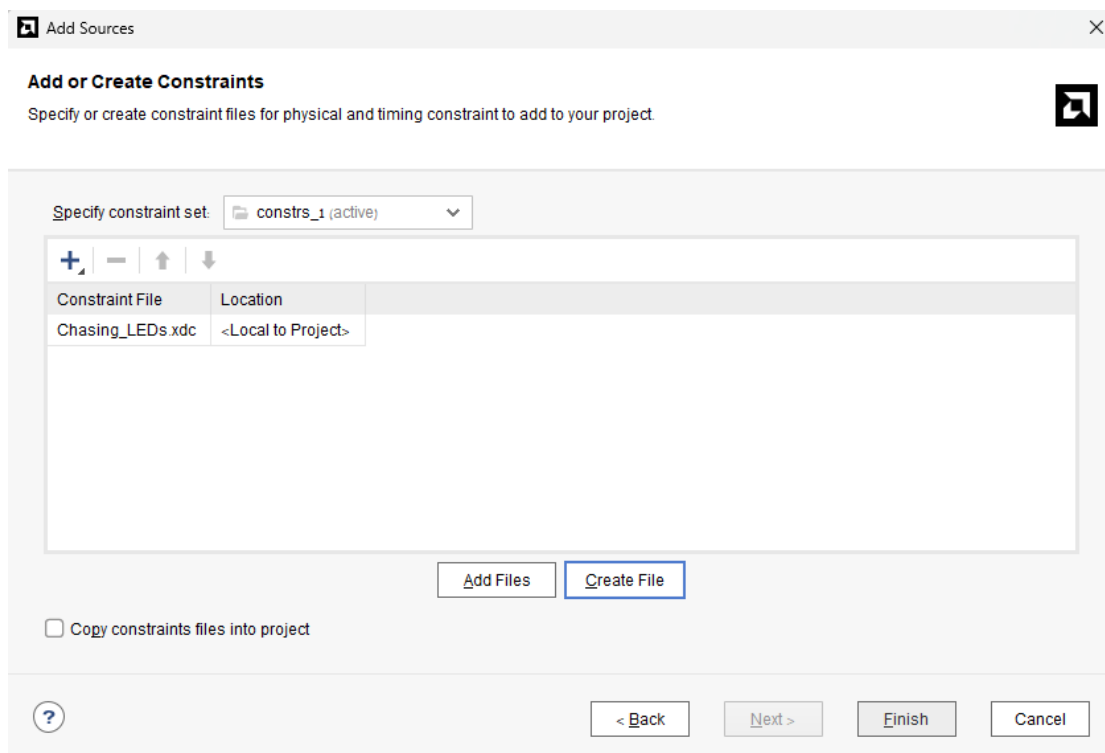
ניצור קובץ XDC בשם Chasing_LEDs.



The dialog box is titled "Create Constraints File" with a close button (X) in the top right corner. It contains the following fields and controls:

- File type:** A dropdown menu showing "XDC" with a downward arrow.
- File name:** A text input field containing "Chasing_LEDs" and a clear button (X) on the right.
- File location:** A dropdown menu showing "<Local to Project>" with a downward arrow.
- A help button (question mark icon) on the bottom left.
- OK** and **Cancel** buttons on the bottom right.

נלחץ Finish.



The dialog box is titled "Add Sources" with a close button (X) in the top right corner. It contains the following elements:

- Add or Create Constraints** section with the instruction: "Specify or create constraint files for physical and timing constraint to add to your project."
- Specify constraint set:** A dropdown menu showing "constrs_1 (active)".
- A table with two columns: "Constraint File" and "Location".

Constraint File	Location
Chasing_LEDs.xdc	<Local to Project>
- Add Files** and **Create File** buttons below the table.
- ☐ Copy constraints files into project
- Navigation buttons at the bottom: **< Back**, **Next >**, **Finish**, and **Cancel**.
- A help button (question mark icon) on the bottom left.

בקובץ נמפה את שמונת הלדים בתקן LVCMOS12, את זוג השעון הדיפרנציאלי sysclk_125 בתקן LVDS ואת כניסת האיפוס. בנוסף נגדיר את שעון ה-dbg_hub הנדרש לרכיב ה-ILA.

```

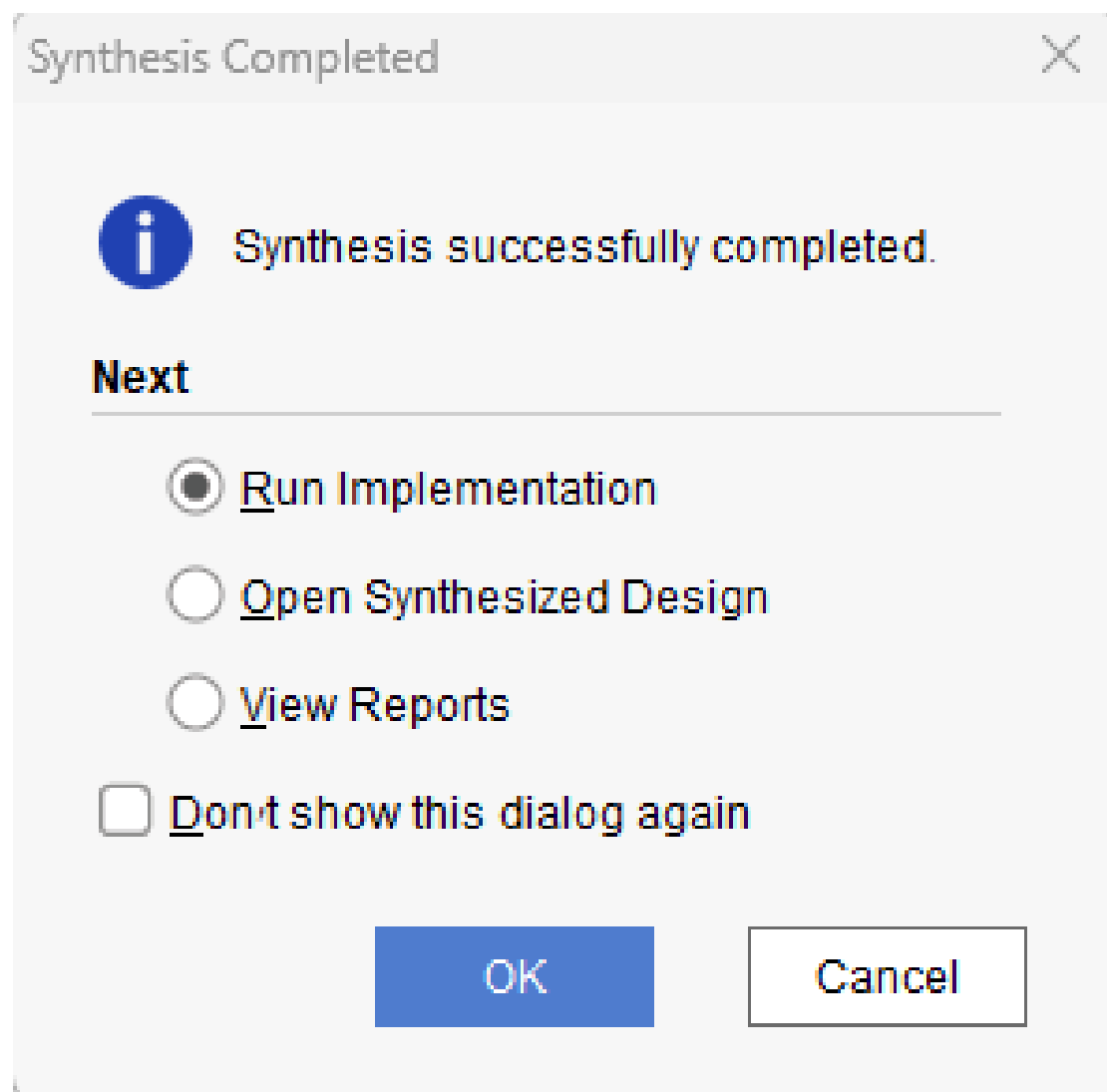
Chasing_LEDs.xdc
N:/6412/6412_Comp_DevTeam/Yair_Osh/Yair_Tayri/VCU118_LED_Control_System_Project/Ch
1 set_property IOSTANDARD LVCMOS12 [get_ports rst_0]
2 set_property IOSTANDARD LVCMOS12 [get_ports {LED_o_0[7]}]
3 set_property IOSTANDARD LVCMOS12 [get_ports {LED_o_0[6]}]
4 set_property IOSTANDARD LVCMOS12 [get_ports {LED_o_0[5]}]
5 set_property IOSTANDARD LVCMOS12 [get_ports {LED_o_0[4]}]
6 set_property IOSTANDARD LVCMOS12 [get_ports {LED_o_0[3]}]
7 set_property IOSTANDARD LVCMOS12 [get_ports {LED_o_0[2]}]
8 set_property IOSTANDARD LVCMOS12 [get_ports {LED_o_0[1]}]
9 set_property IOSTANDARD LVCMOS12 [get_ports {LED_o_0[0]}]
10 set_property PACKAGE_PIN AT32 [get_ports {LED_o_0[7]}]
11 set_property PACKAGE_PIN AV34 [get_ports {LED_o_0[6]}]
12 set_property PACKAGE_PIN AY30 [get_ports {LED_o_0[5]}]
13 set_property PACKAGE_PIN BB32 [get_ports {LED_o_0[4]}]
14 set_property PACKAGE_PIN BF32 [get_ports {LED_o_0[3]}]
15 set_property PACKAGE_PIN AU37 [get_ports {LED_o_0[2]}]
16 set_property PACKAGE_PIN AV36 [get_ports {LED_o_0[1]}]
17 set_property PACKAGE_PIN BA37 [get_ports {LED_o_0[0]}]
18 set_property PACKAGE_PIN L19 [get_ports rst_0]
19 set_property IOSTANDARD LVDS [get_ports sysclk_125_clk_p]
20 set_property IOSTANDARD LVDS [get_ports sysclk_125_clk_n]
21 set_property PACKAGE_PIN AY24 [get_ports sysclk_125_clk_p]
22 set_property C_CLK_INPUT_FREQ_HZ 300000000 [get_debug_cores dbg_hub]
23 set_property C_ENABLE_CLK_DIVIDER false [get_debug_cores dbg_hub]
24 set_property C_USER_SCAN_CHAIN 1 [get_debug_cores dbg_hub]
25 connect_debug_port dbg_hub/clk [get_nets clk]
  
```

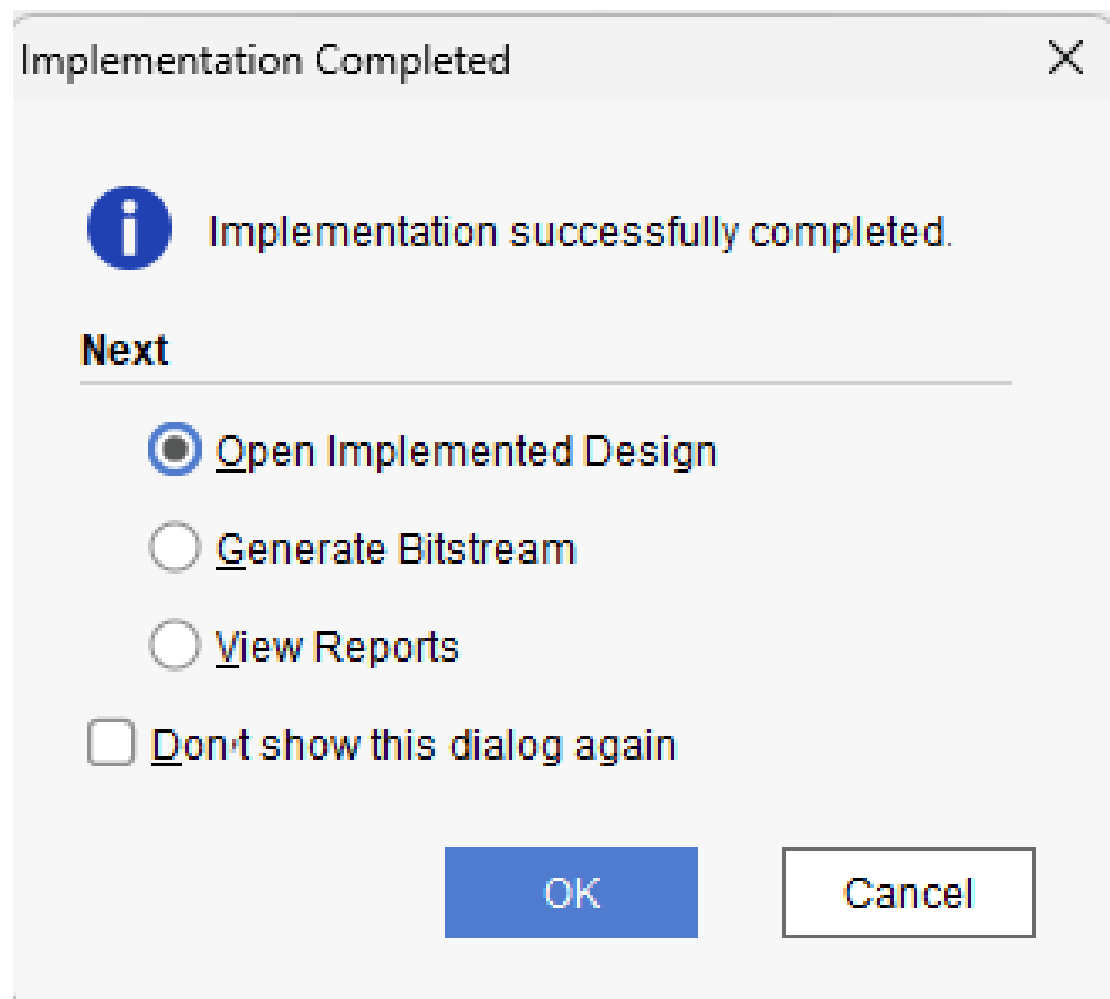
▼ SYNTHESIS

▶ Run Synthesis

> Open Synthesized Design

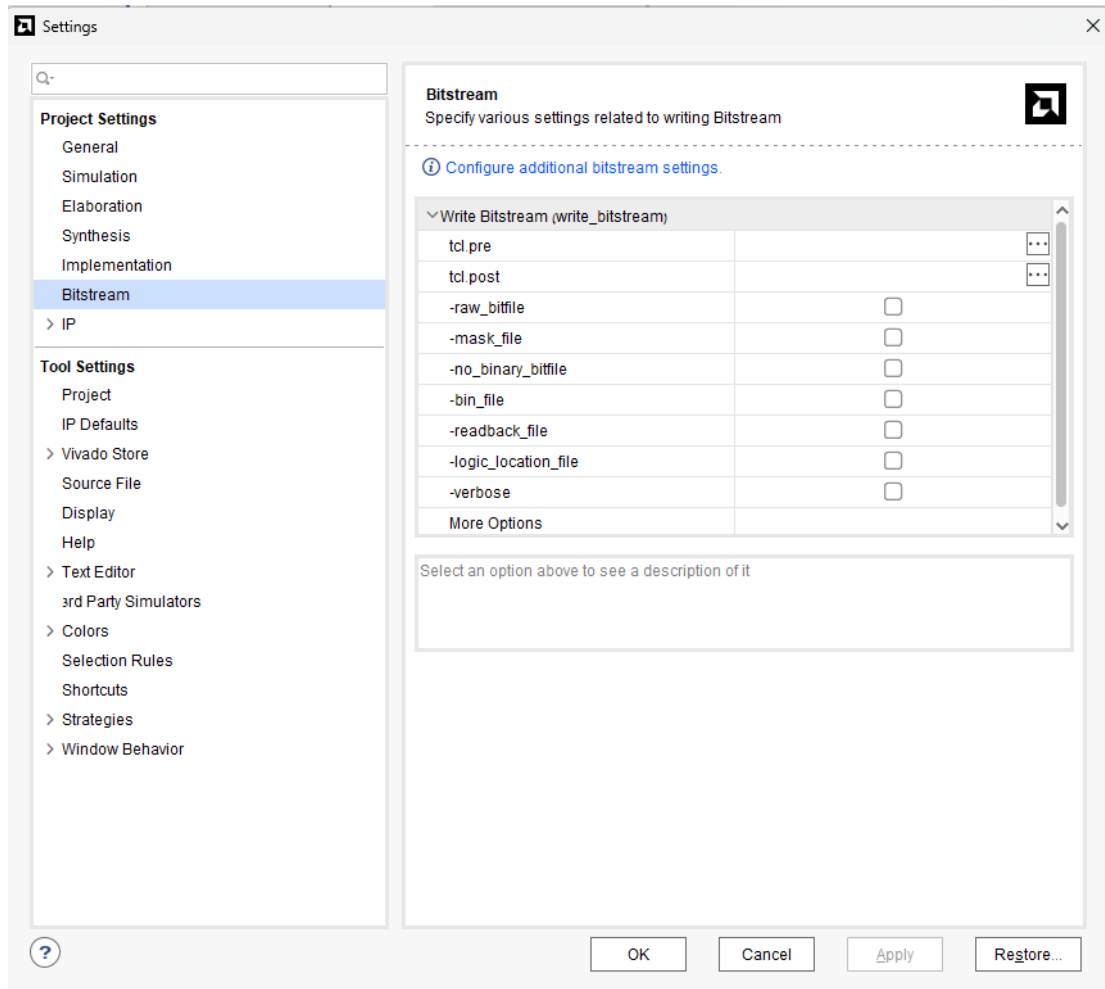
בסיום נבחר Run Implementation ונלחץ OK.



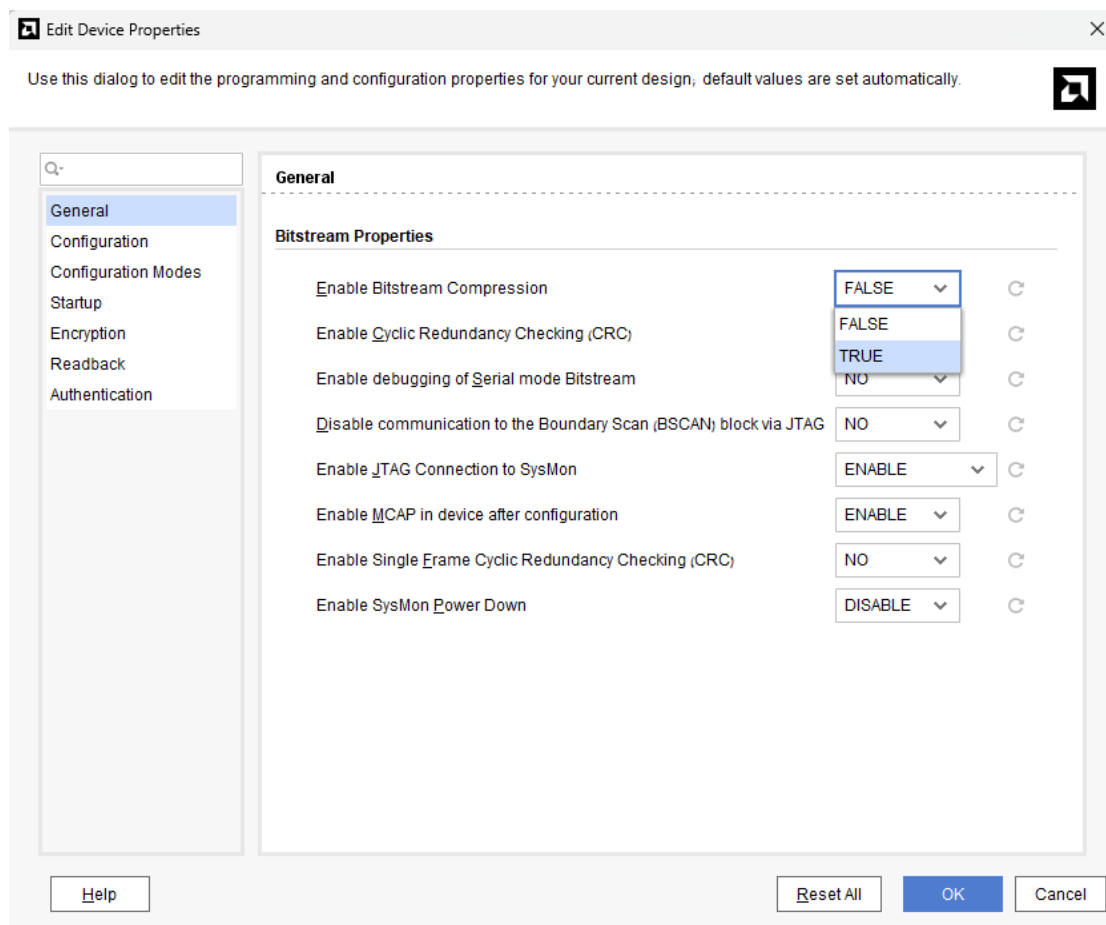


שלב 16:

בהגדרות הפרויקט, בלשונית Bitstream, נלחץ על Configure additional bitstream settings.



נפעיל את Enable Bitstream Compression לצמצום גודל הקובץ וזמן הצריבה.

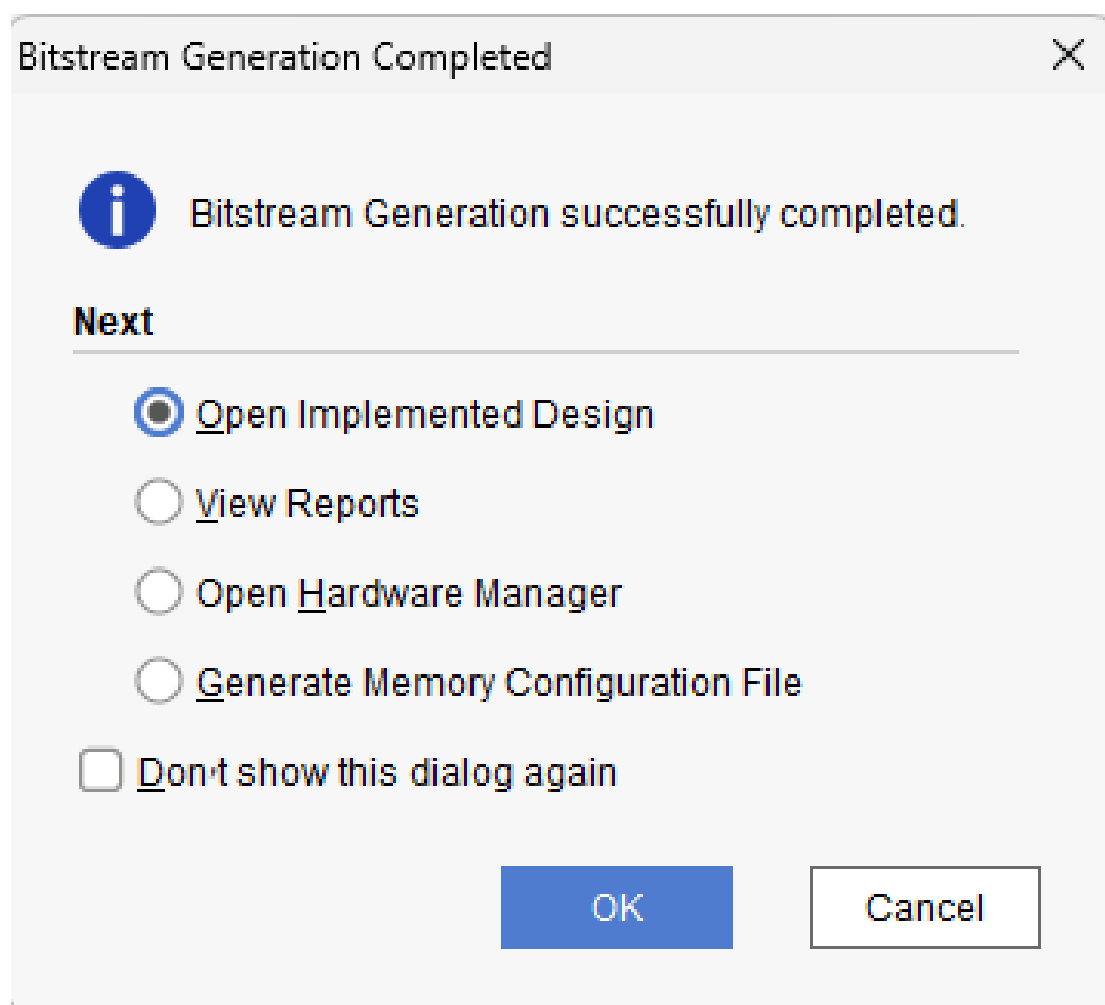


נלחץ Generate Bitstream.

PROGRAM AND DEBUG

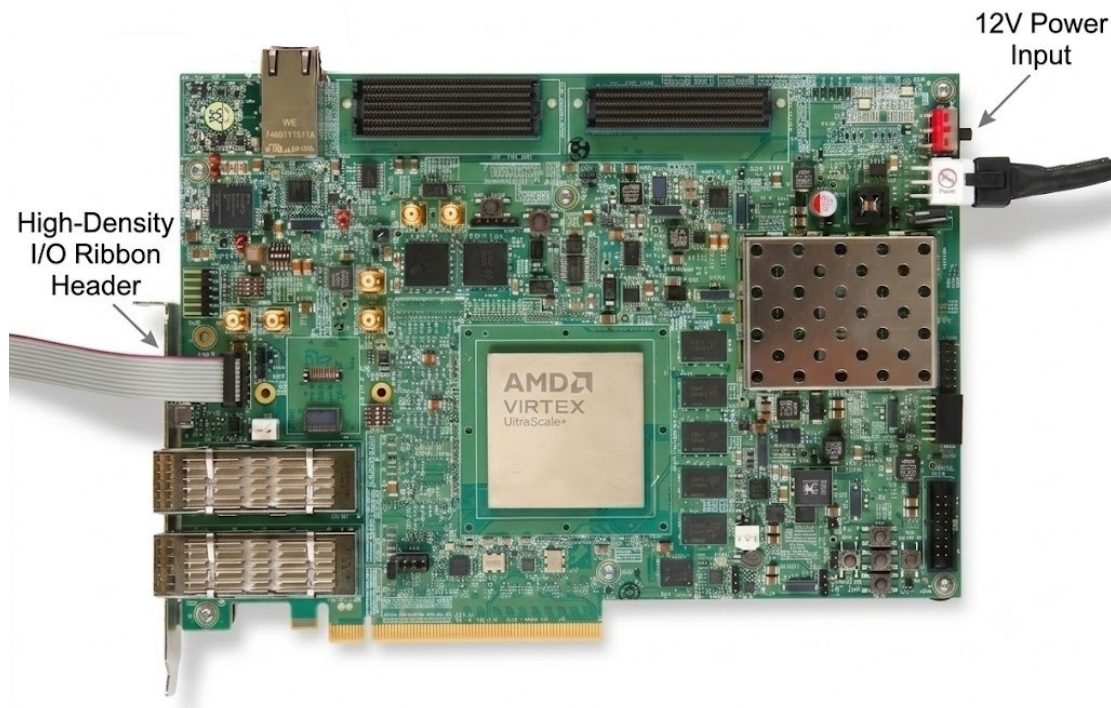


Generate Bitstream

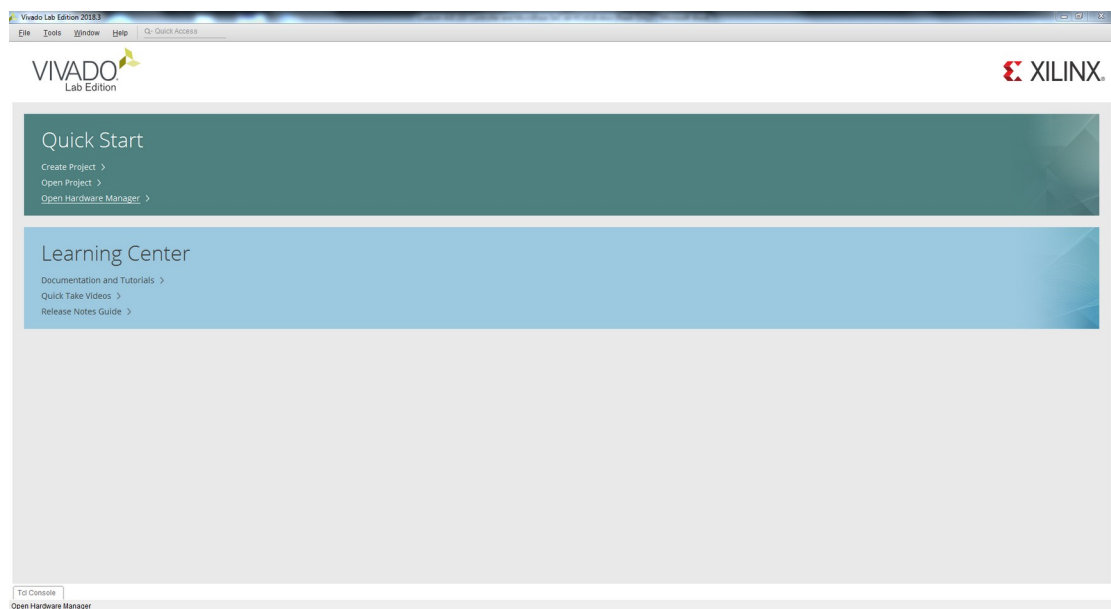


שלב 17:

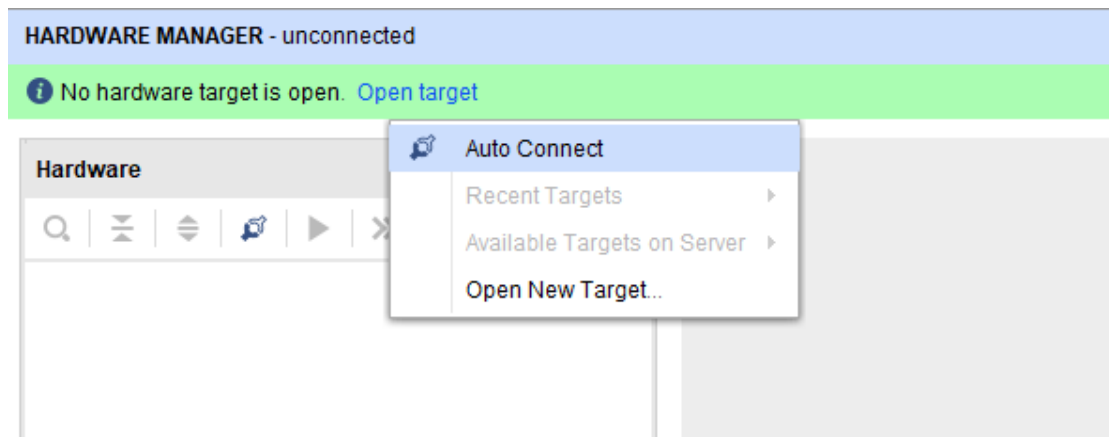
נחבר את הלוח למחשב באמצעות כבל ה-JTAG ונפעיל את ספק הכוח.



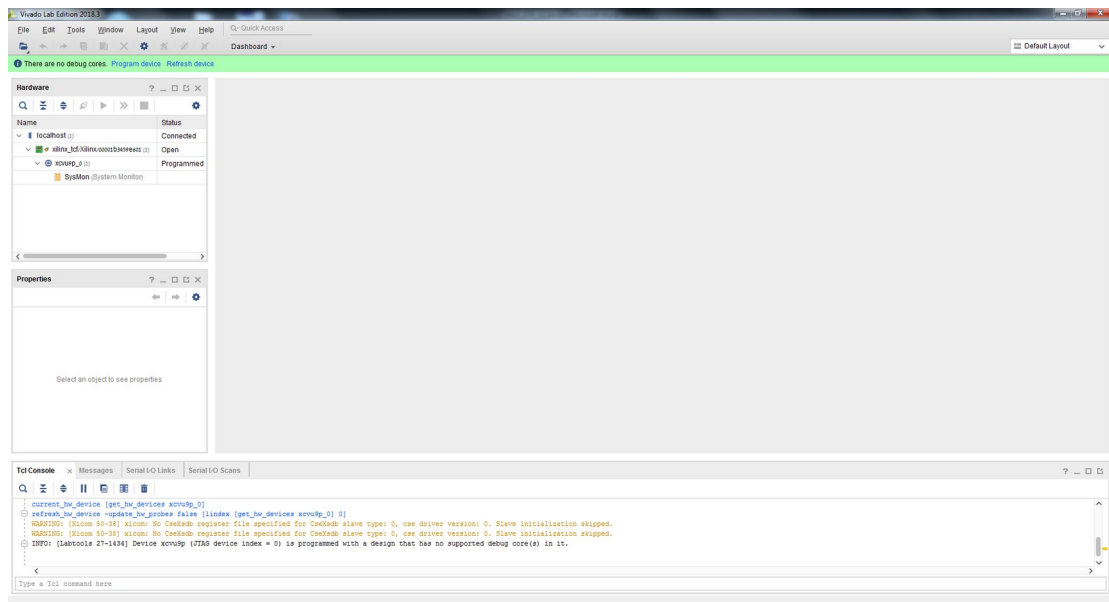
נפתח את Hardware Manager.



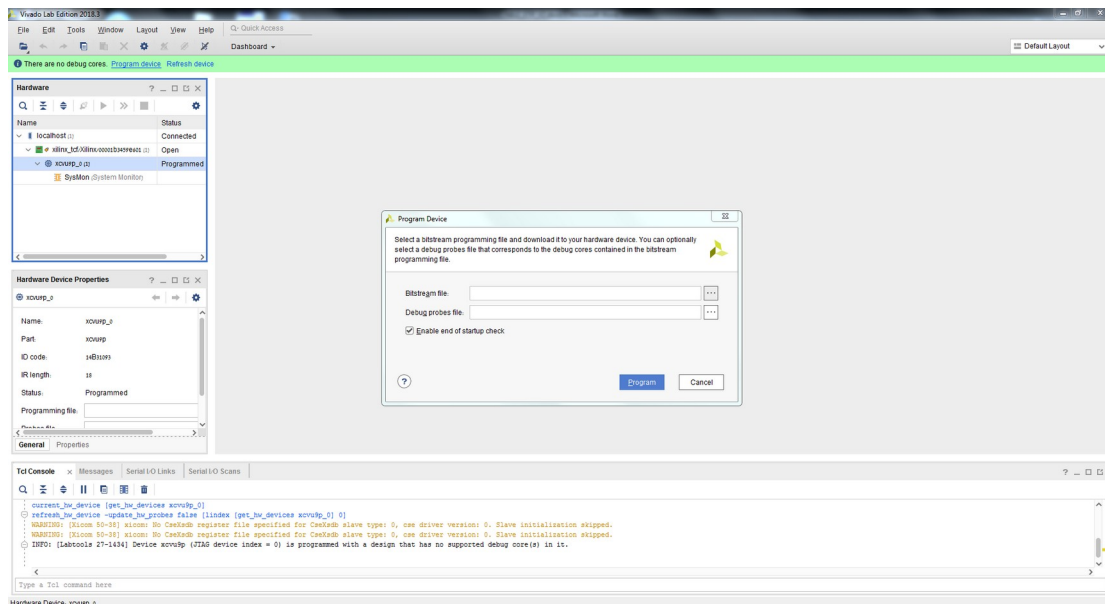
נבחר Open Target ואז Auto Connect.



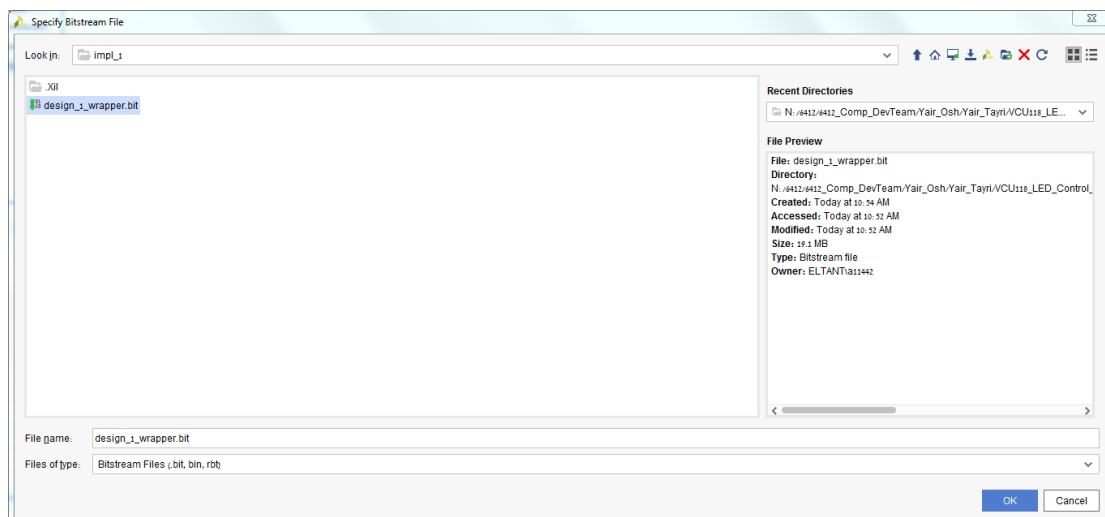
הלוח מזוהה בחלון Hardware.



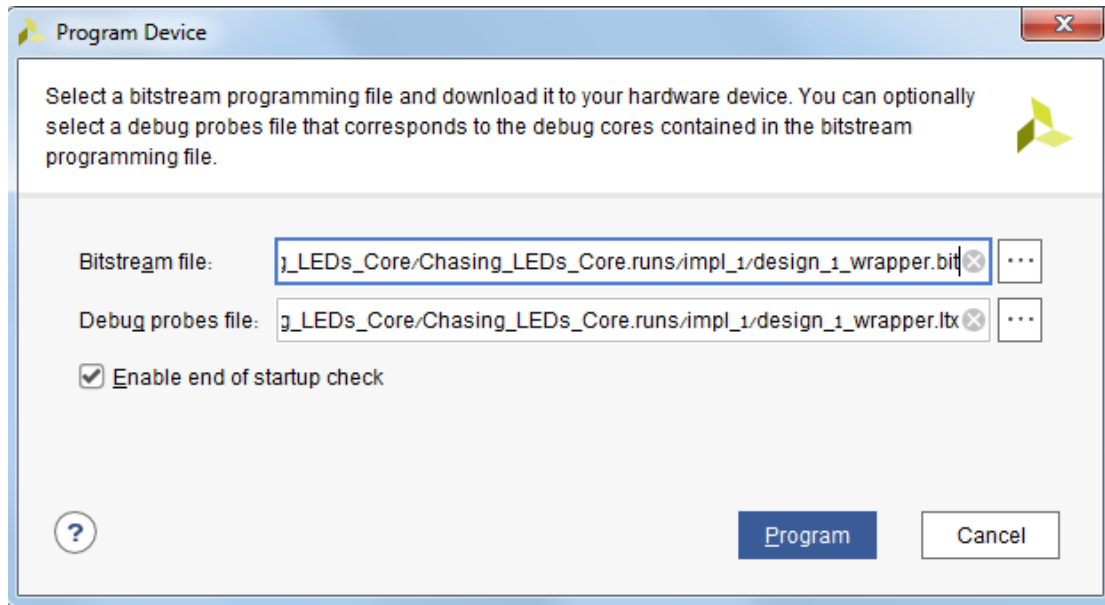
נבחר Program Device.



נבחר את קובץ ה-Bitstream מתיקיית הפרויקט.

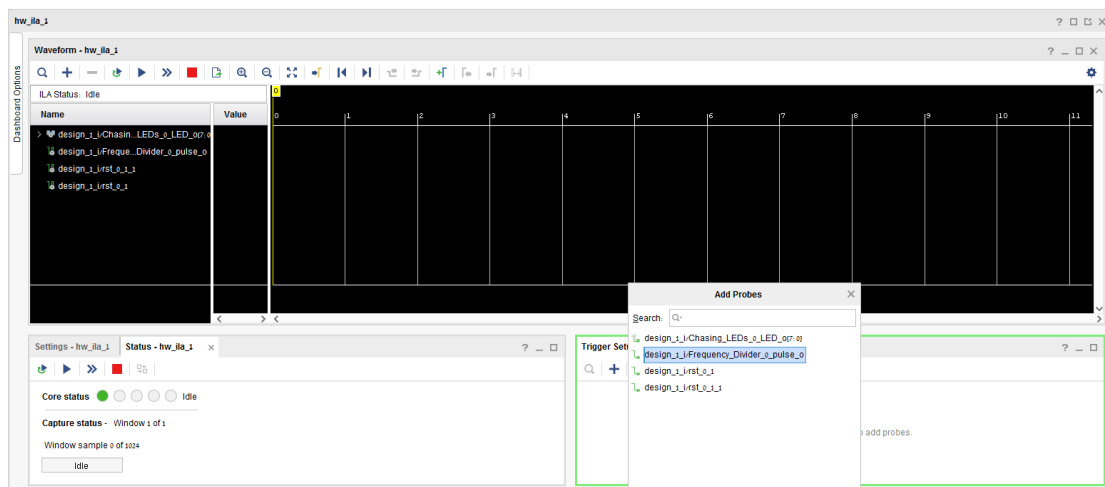


נוודא שנבחרו קובץ ה-Bitstream וקובץ ה-Itx של ה-ILA ונלחץ Program. לאחר הצריבה הולדים רצים על גבי הלוח.

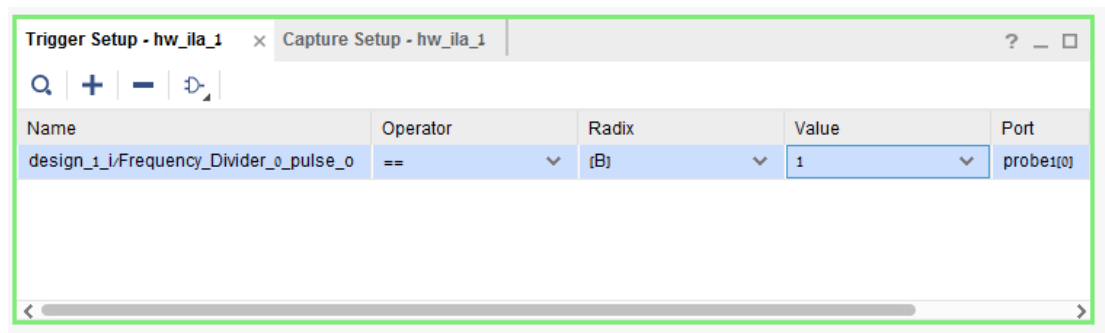


שלב 18:

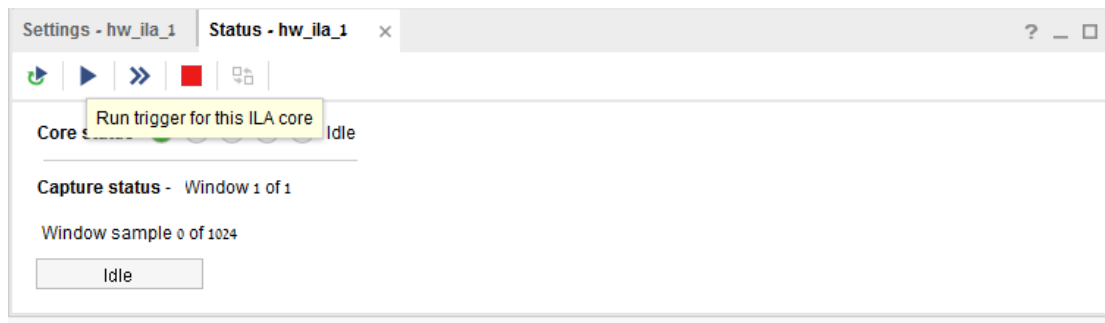
בחלון ה-ILA נוסף את האות pulse_0 ל-Trigger Setup.



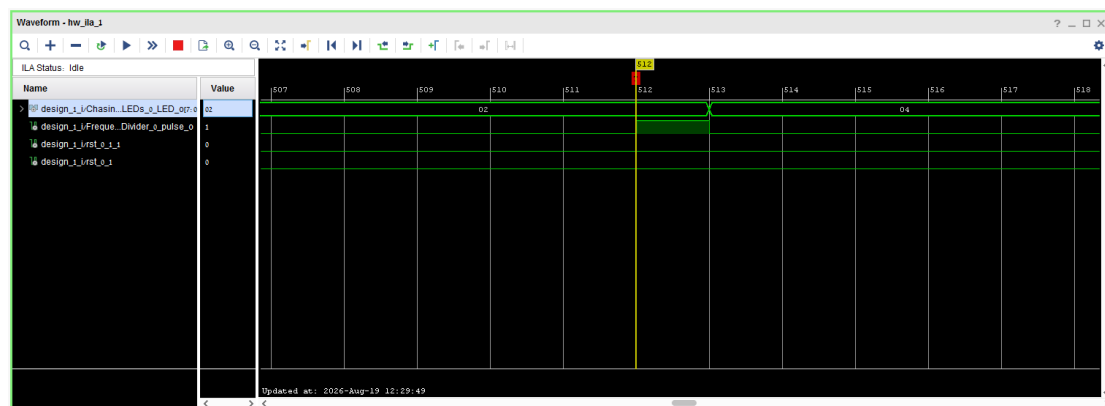
נגדיר את ערך ההדק ל-1.



נלחץ Run Trigger ונמתין ללכידה.



לאחר הלכידה נבצע Zoom In סביב דגימה 512. הפולס עולה ל-1 למחזור שעון בודד, ומיד לאחריו וקטור הלדים מתחלף מ-02 ל-04. תוצאה זו מאמתת בחומרה את מנגנון ה-Clock Enable.



Chapter 2: MicroBlaze Processor Infrastructure and UART Communication on Zynq-7000

מבוא:

בפרק זה נקים מערכת מעבד בסיסית על כרטיס מבוסס Zynq XC7Z020CLG400-1. מעבד ה-MicroBlaze ממומש בלוגיקה המתוכנתת ומתקשר עם המחשב דרך רכיב AXI UART Lite. המטרה היא להריץ תוכנת Hello World מתוך זיכרון BRAM המאותחל ישירות מקובץ ה-Bitstream.

דרישות קדם:

חומרה:

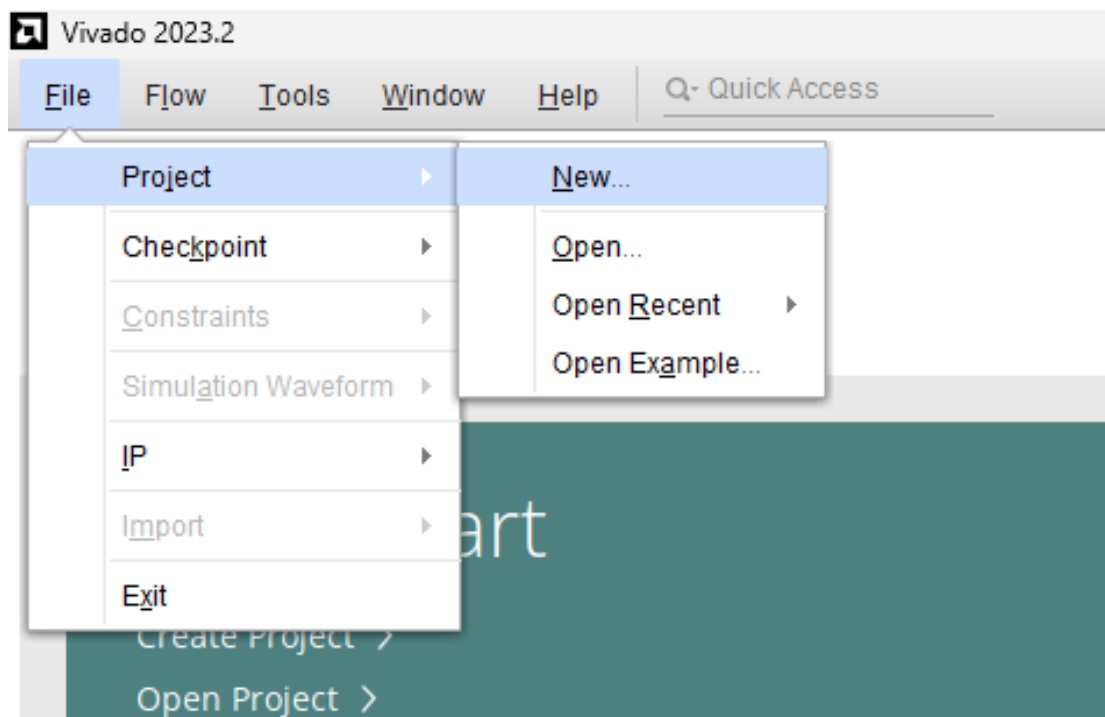
לוח פיתוח מבוסס Zynq XC7Z020CLG400-1
כבל צריבה JTAG
מתאם USB-to-UART לחיבור טורי למחשב

תוכנה:

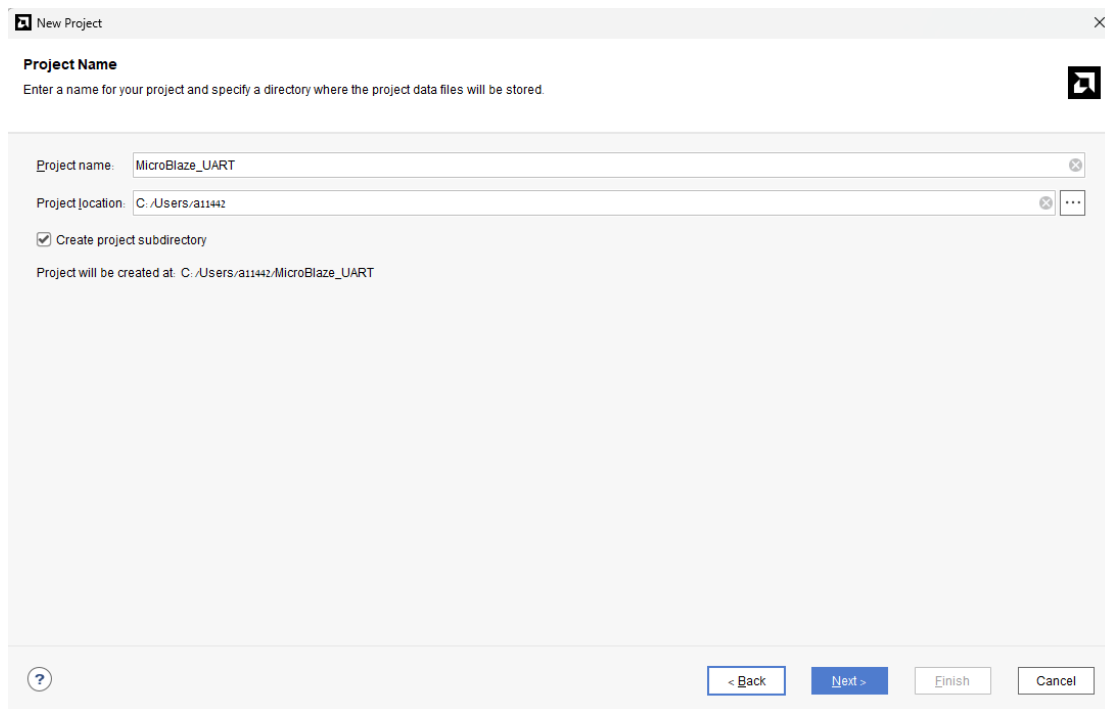
Vivado Design Suite
Vitis Unified IDE
תוכנת טרמינל Tera Term

שלב 1:

נפתח פרויקט חדש דרך File, Project וזו New.



נזין שם לפרויקט MicroBlaze_UART.



New Project

Project Name
Enter a name for your project and specify a directory where the project data files will be stored.

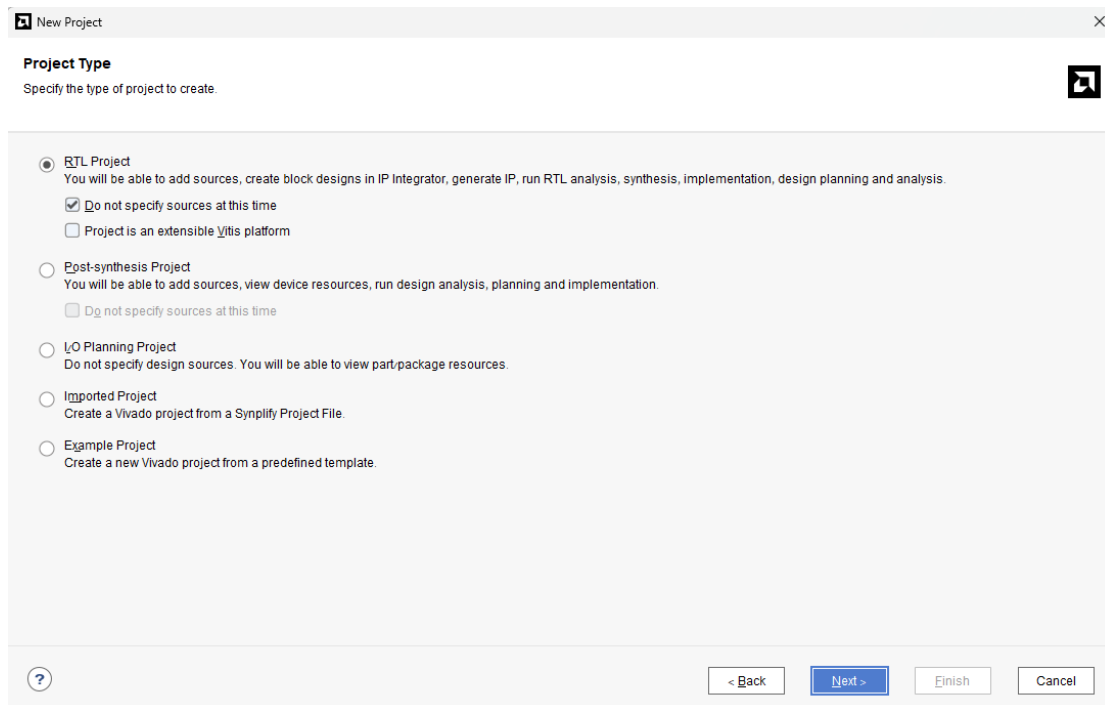
Project name: MicroBlaze_UART

Project location: C:/Users/a11442

☒ Create project subdirectory
Project will be created at: C:/Users/a11442/MicroBlaze_UART

< Back Next > Finish Cancel

נבחר RTL Project ונסמן שלא להגדיר קבצי מקור כרגע.



New Project

Project Type
Specify the type of project to create.

☒ **RTL Project**
You will be able to add sources, create block designs in IP Integrator, generate IP, run RTL analysis, synthesis, implementation, design planning and analysis.
☒ Do not specify sources at this time
☐ Project is an extensible Vitis platform

☐ **Post-synthesis Project**
You will be able to add sources, view device resources, run design analysis, planning and implementation.
☐ Do not specify sources at this time

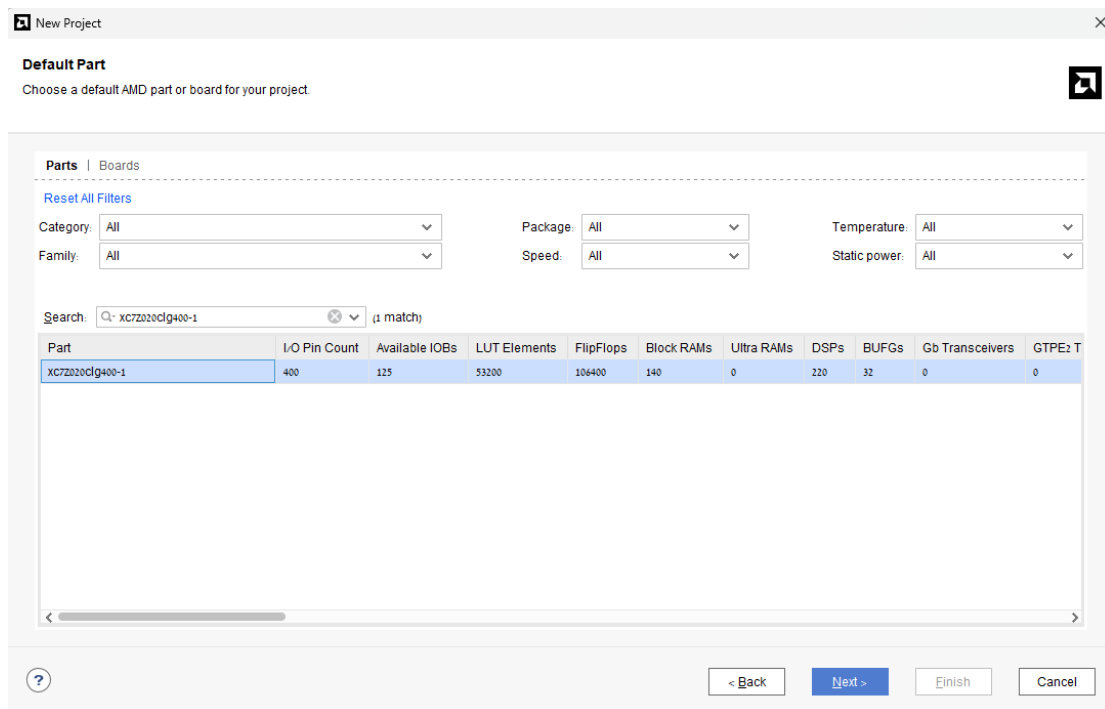
☐ **I/O Planning Project**
Do not specify design sources. You will be able to view part/package resources.

☐ **Imported Project**
Create a Vivado project from a Synplify Project File.

☐ **Example Project**
Create a new Vivado project from a predefined template.

< Back Next > Finish Cancel

בלשונית Parts נבחר את הרכיב xc7z020clg400-1.



Default Part
Choose a default AMD part or board for your project.

Parts | Boards

Reset All Filters

Category: All Package: All Temperature: All
Family: All Speed: All Static power: All

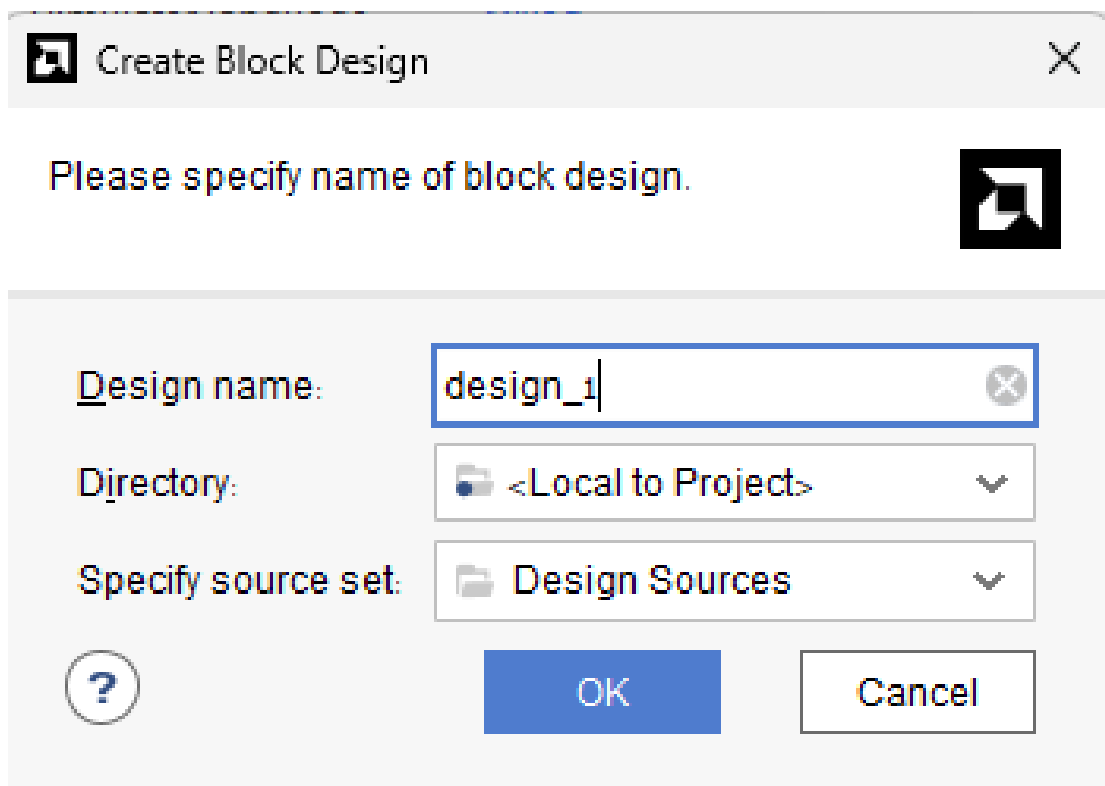
Search: xc7z020clg400-1 (1 match)

Part	I/O Pin Count	Available IOBs	LUT Elements	FlipFlops	Block RAMs	Ultra RAMs	DSPs	BUFs	Gb Transceivers	GTPEz T
xc7z020clg400-1	400	125	53200	106400	140	0	220	32	0	0

< Back Next > Finish Cancel

שלב 2:

ניצור Block Design בשם design_1.



Create Block Design

Please specify name of block design.

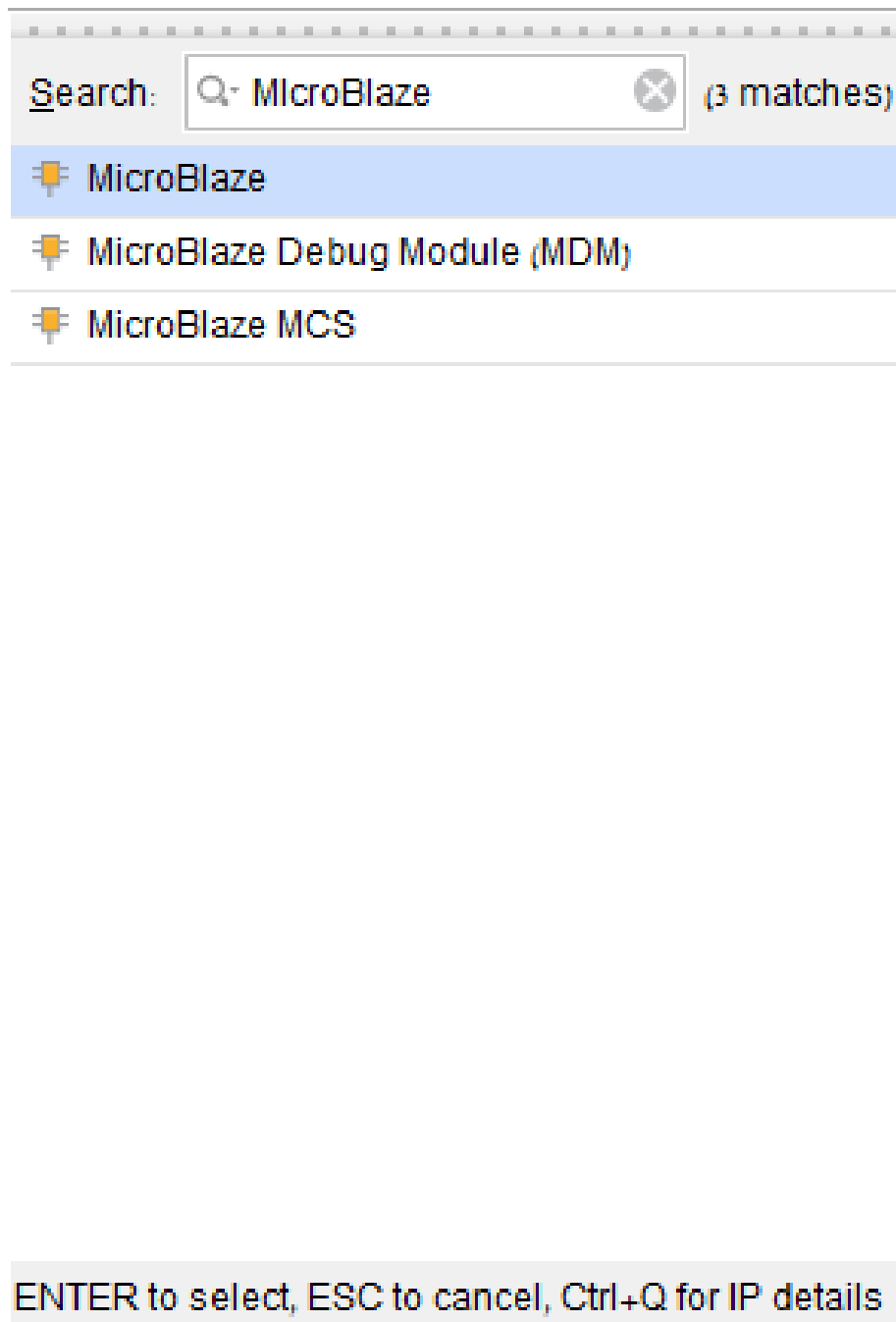
Design name: design_1

Directory: <Local to Project>

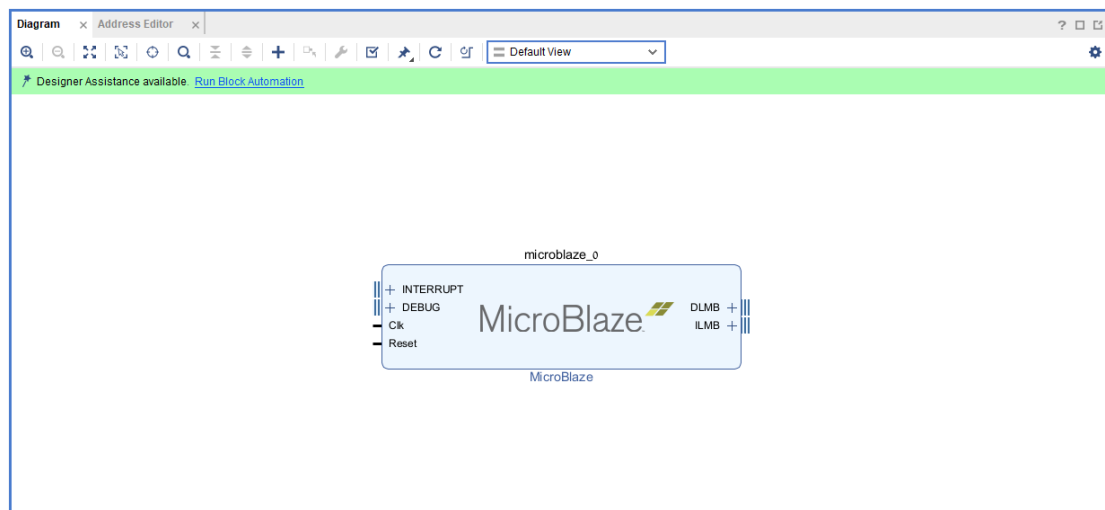
Specify source set: Design Sources

? OK Cancel

נוסיף רכיב MicroBlaze.

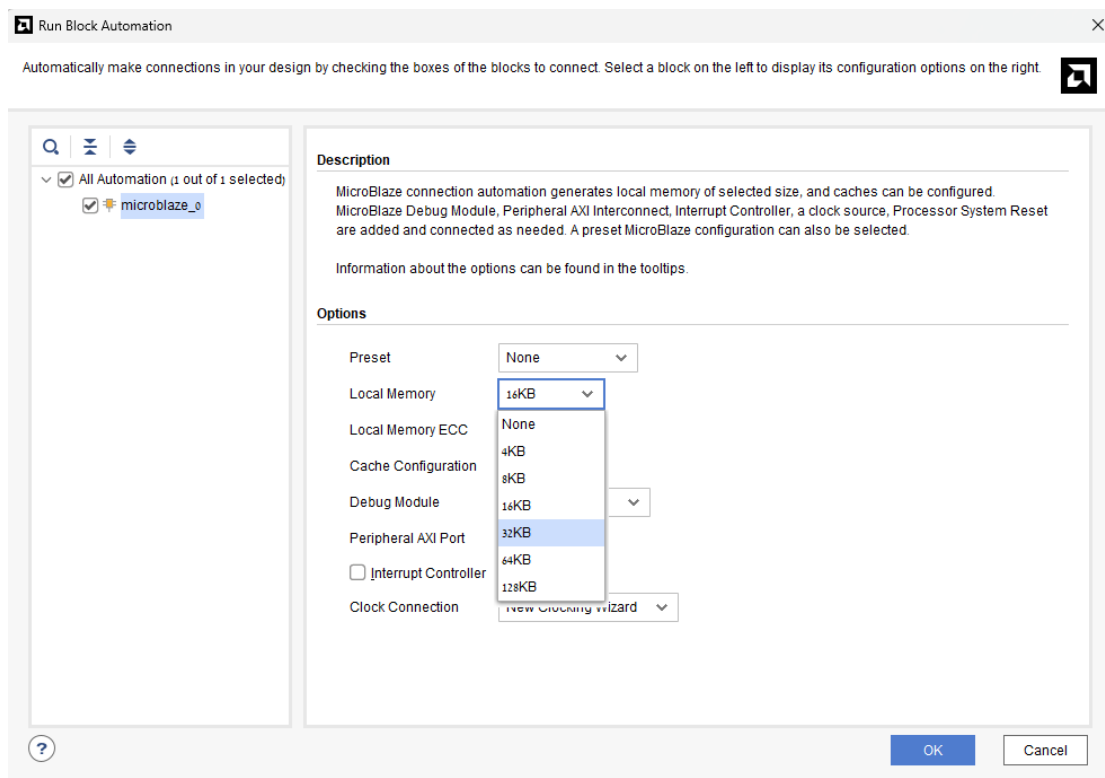


הרכיב נוסף לשרטוט.

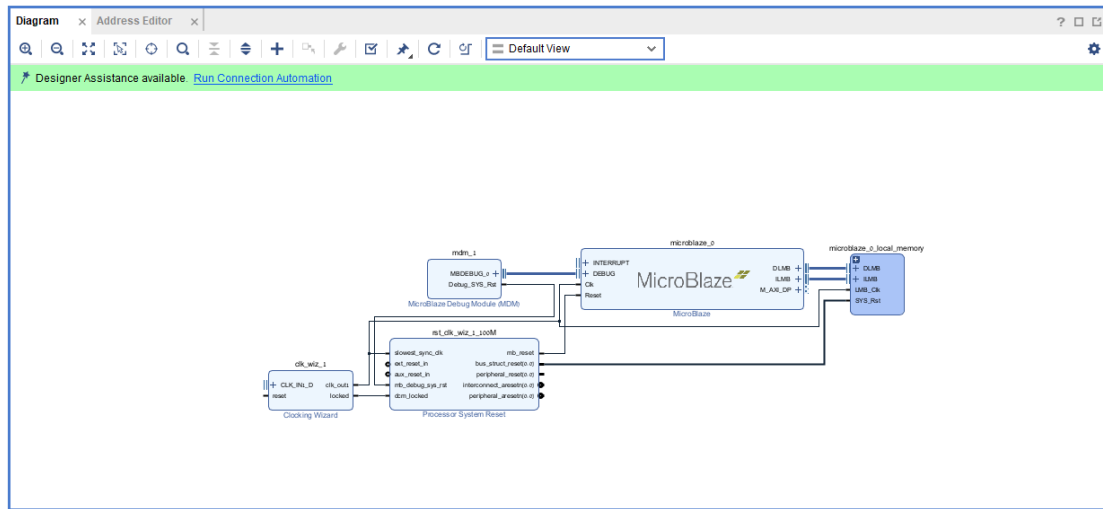


שלב 3:

נלחץ Run Block Automation. בשדה Local Memory נבחר 64KB. ערך זה גדול מהנדרש לתוכנת Hello World ומשאיר שולי ביטחון בזיכרון ה-DLMB וה-ILMB, כך שהקימפול ב-Vitis לא ייכשל על חוסר מקום.

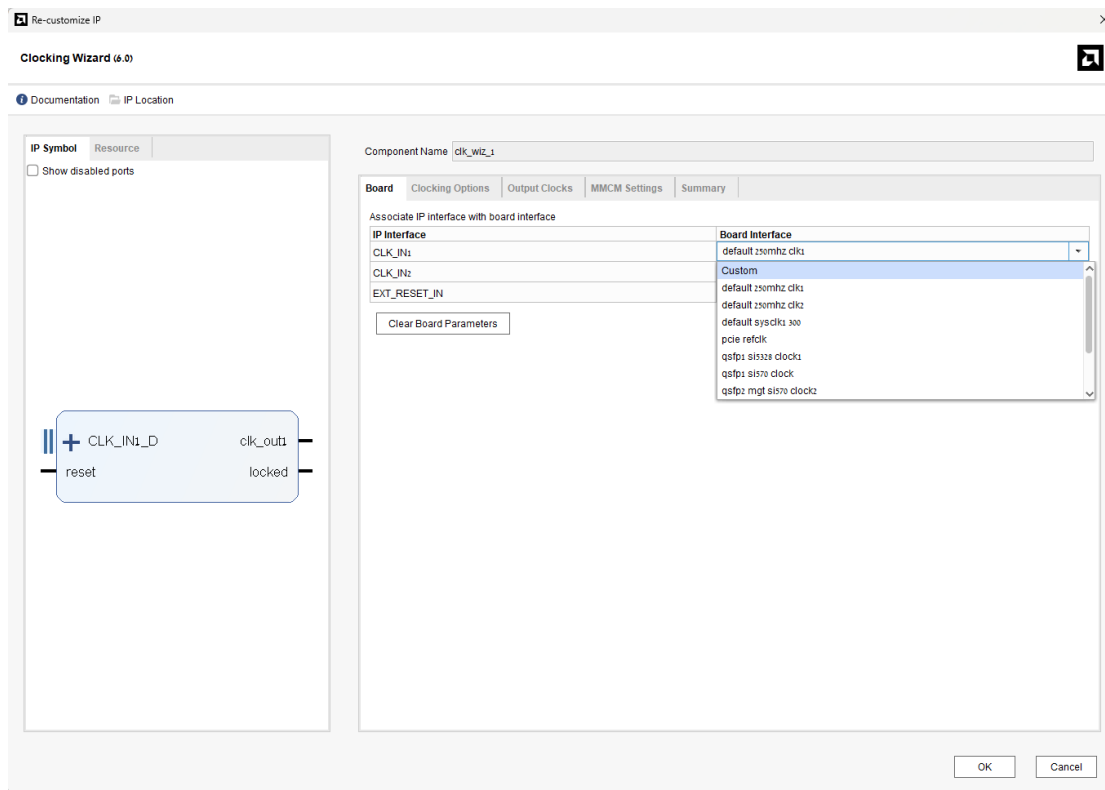


לאחר האוטומציה מתקבלת תשתית המעבד:

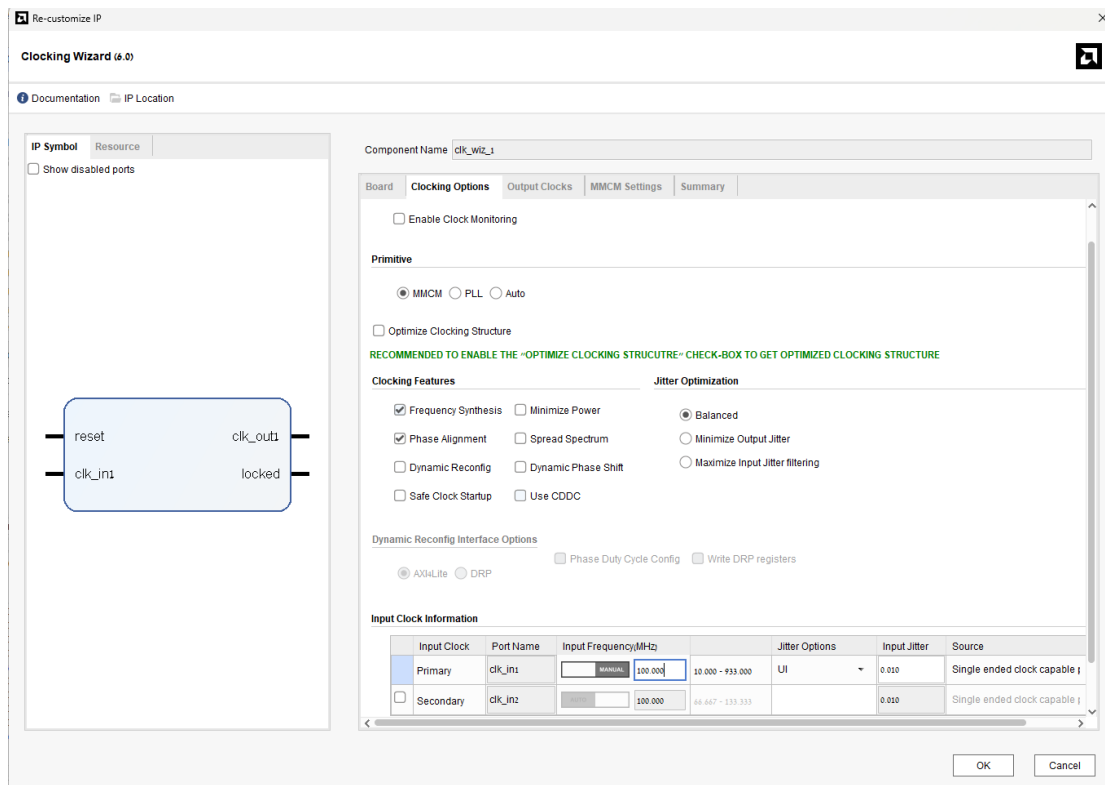


שלב 4:

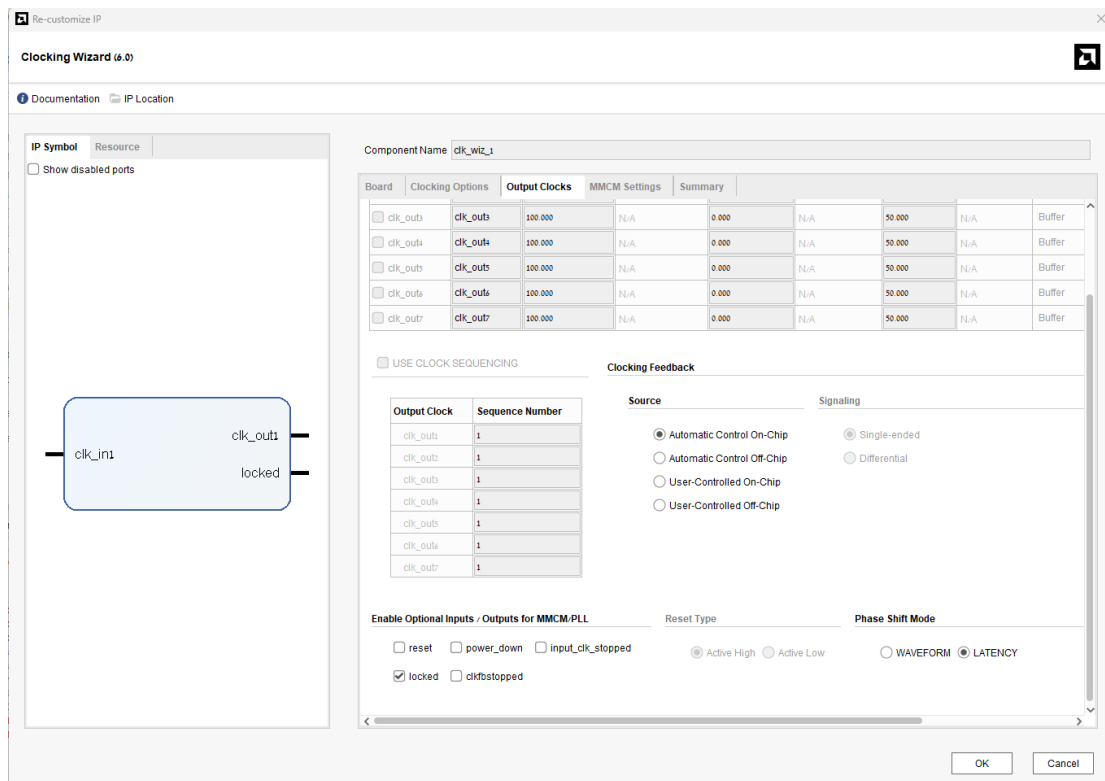
בלשונית Board של ה-Clocking Wizard נגדיר את מקור השעון.



בלשונית Clocking Options נגדיר את תדר הכניסה.



בלשונית Output Clocks נסיר את הסימון מכניסת ה-reset, כך שהשעון פועל מרגע עליית המערכת. את יציאת locked נחבר לכניסת ext_reset_in של רכיב ה-Processor System Reset. יציאה זו עולה ל-1 רק לאחר שה-MMCM ננעל על תדר יציב, ולכן המעבד ורכיב ההיקף יוצאים מאיפוס באופן אוטומטי רק כשהשעון יציב.



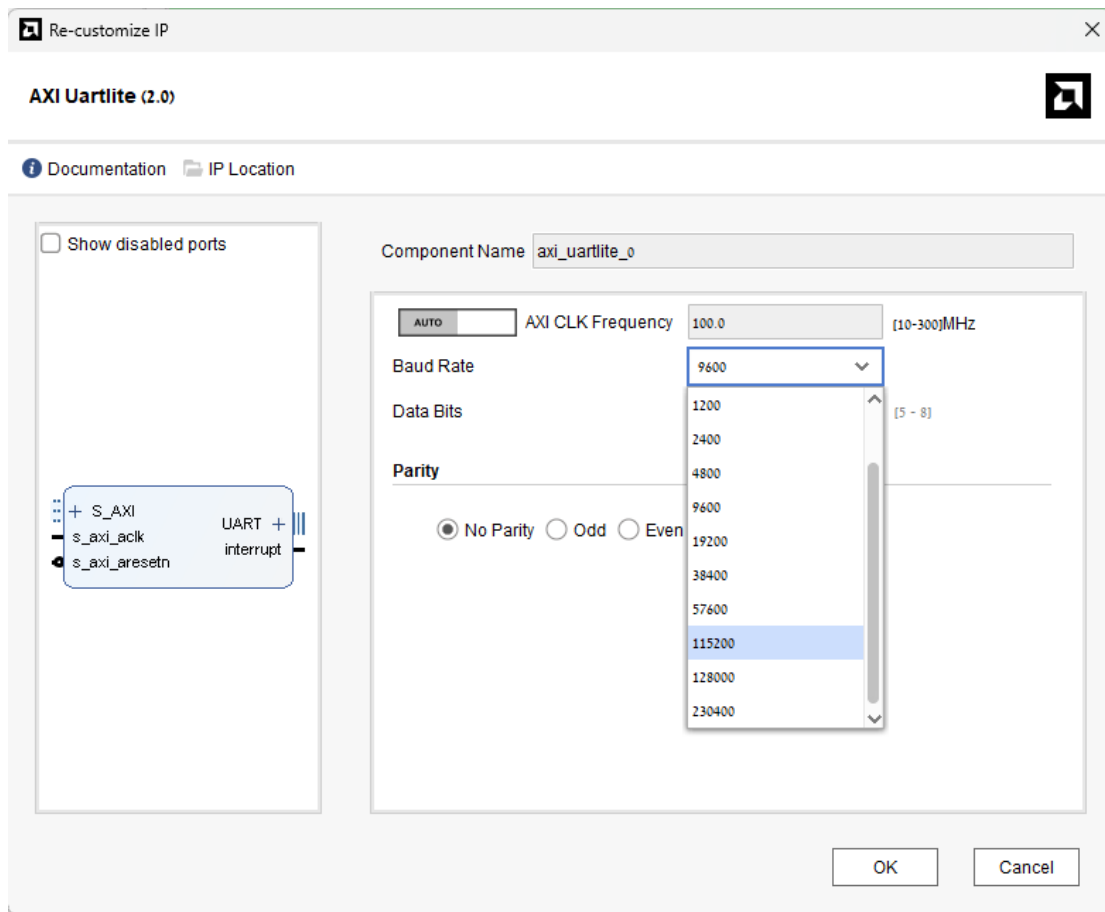
שלב 5:

נוסיף רכיב AXI Uartlite.



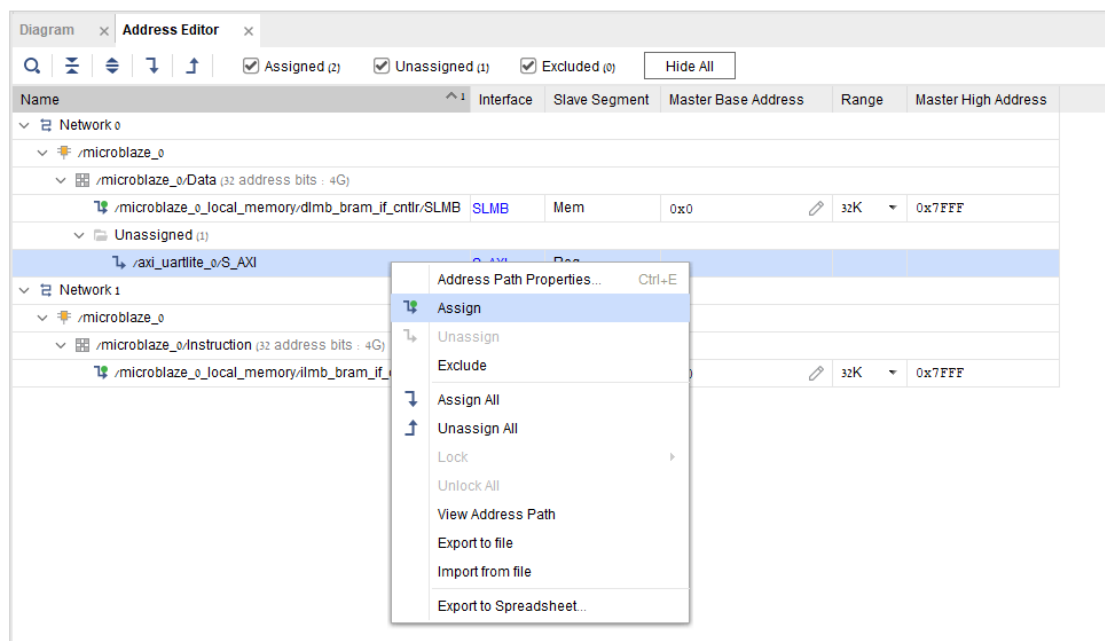
ENTER to select, ESC to cancel, Ctrl+Q for IP details

נגדיר Baud Rate של 115200.

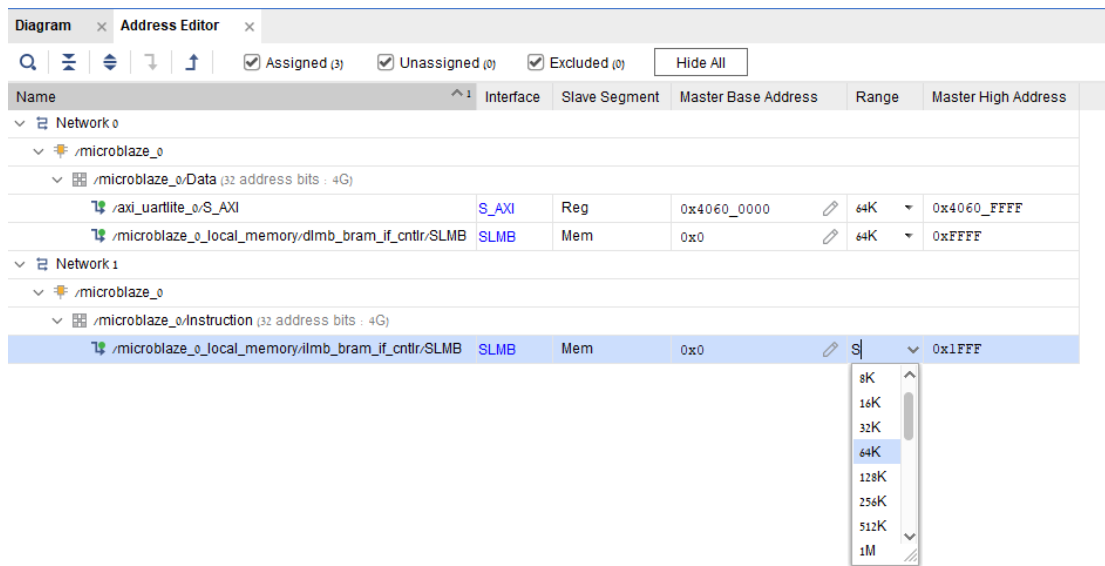


שלב 6:

בחלון Address Editor נלחץ קליק ימני על רכיב ה-UART ונבחר Assign.

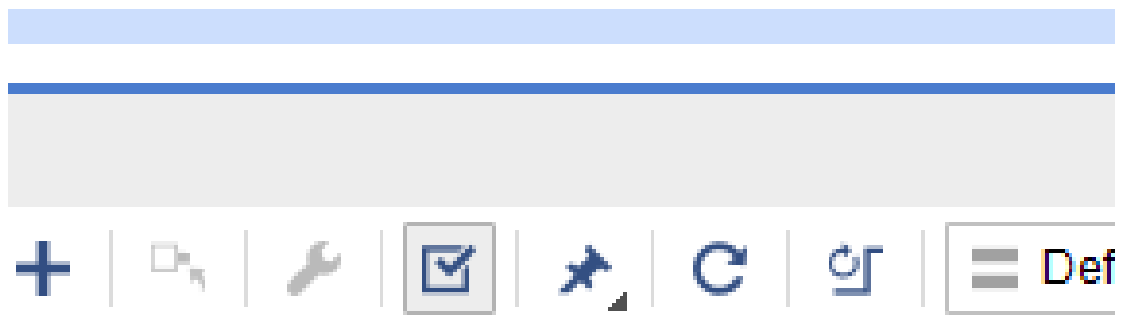


נוודא שטווח הזיכרון המקומי עומד על 64K.



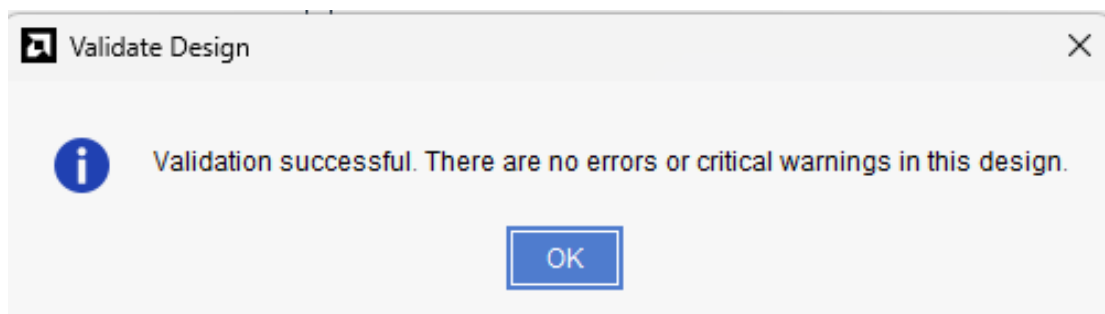
שלב 7:

נלחץ Validate Design.

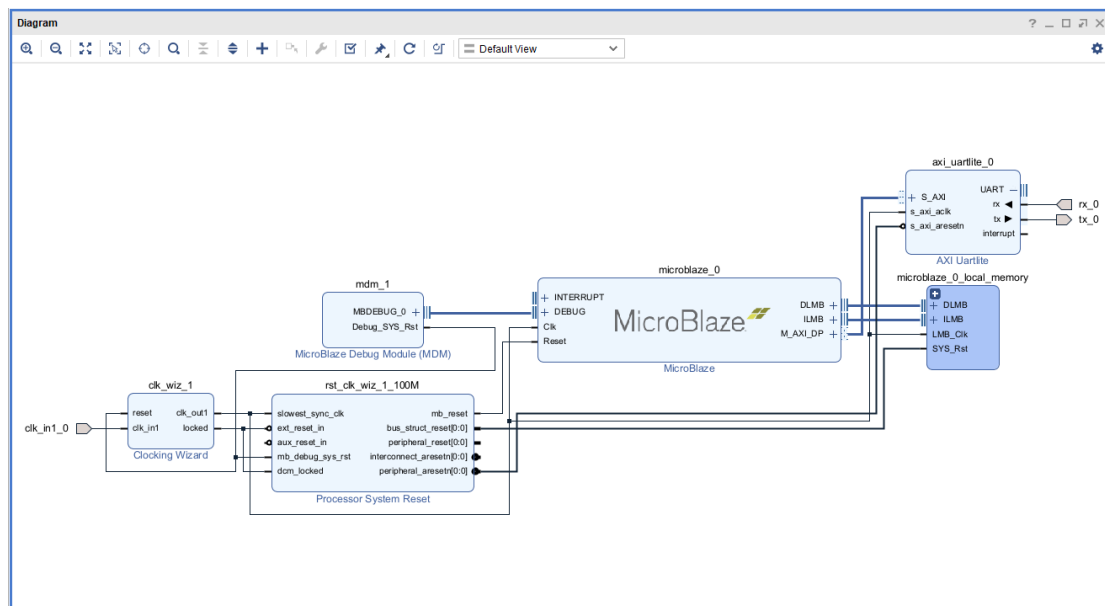


Validate Design (F6)

האימות עבר ללא שגיאות.



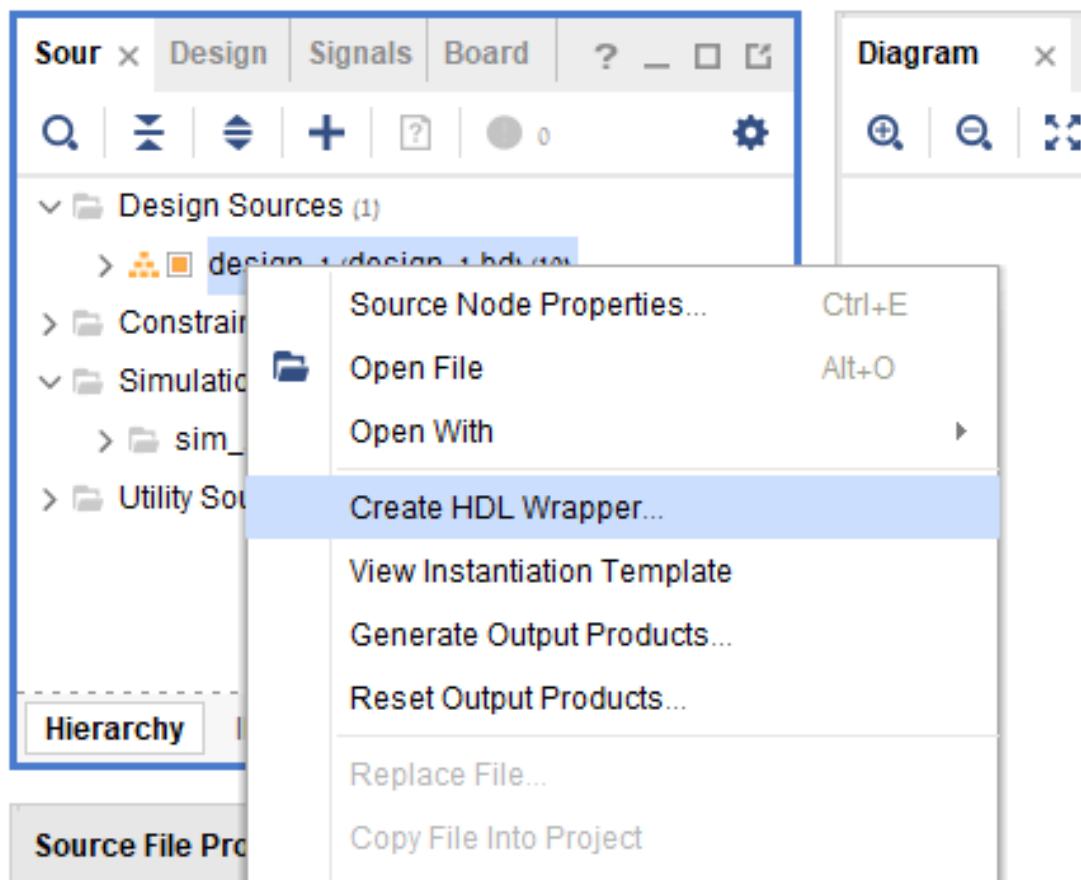
זהו התכן המלא:



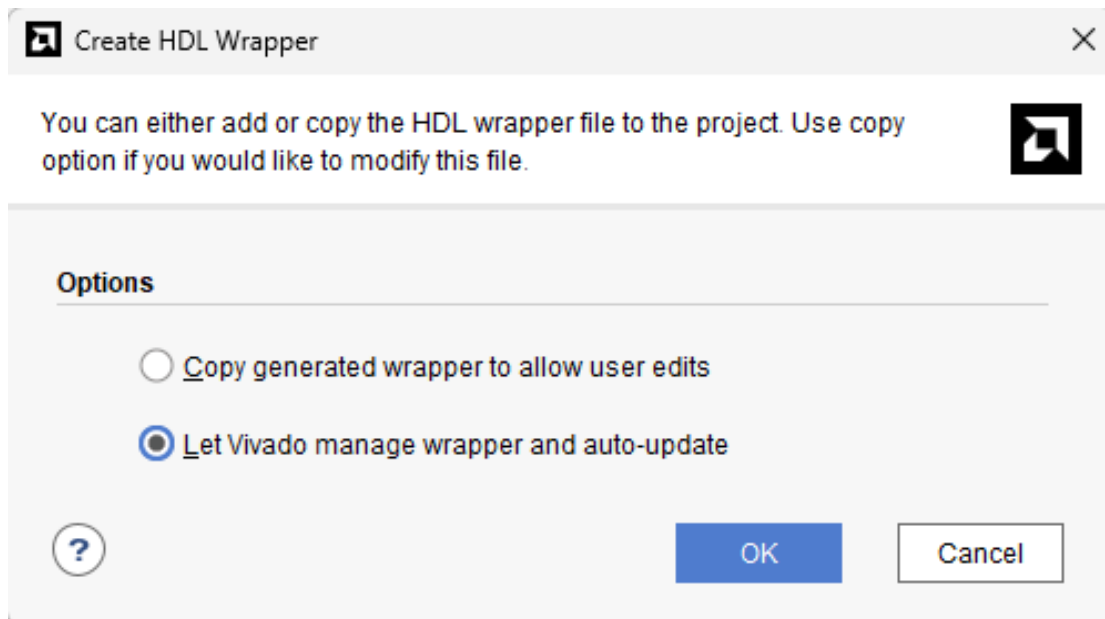
שלב 8:

ניצור HDL Wrapper לתכן.

BLOCK DESIGN - design_1

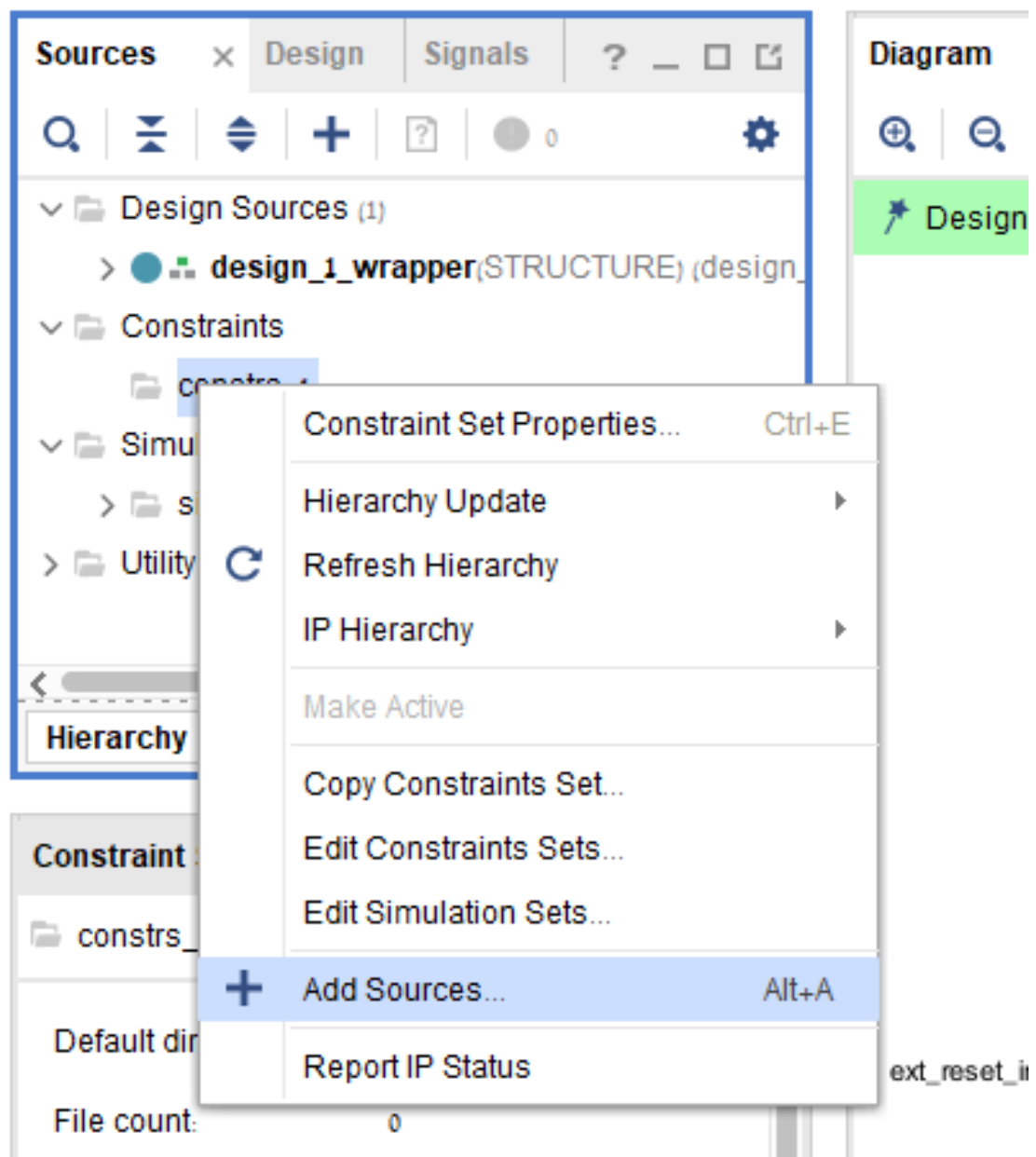


נשאיר את הניהול האוטומטי על ידי Vivado ונלחץ OK.

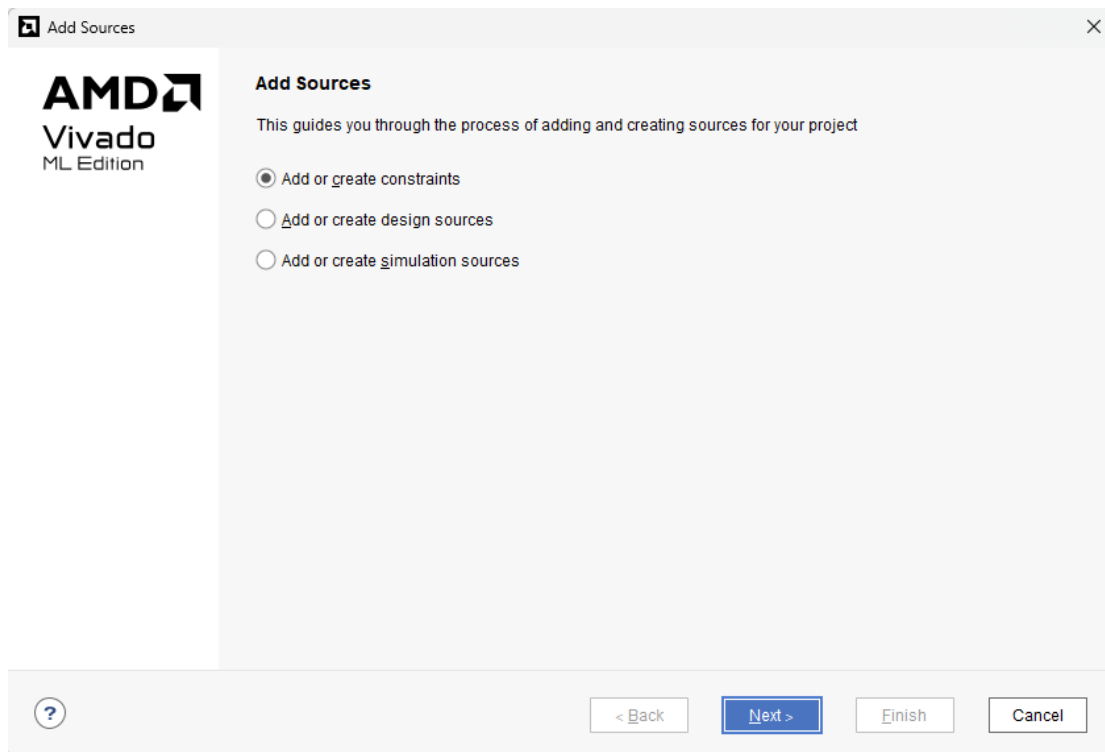


שלב 9:

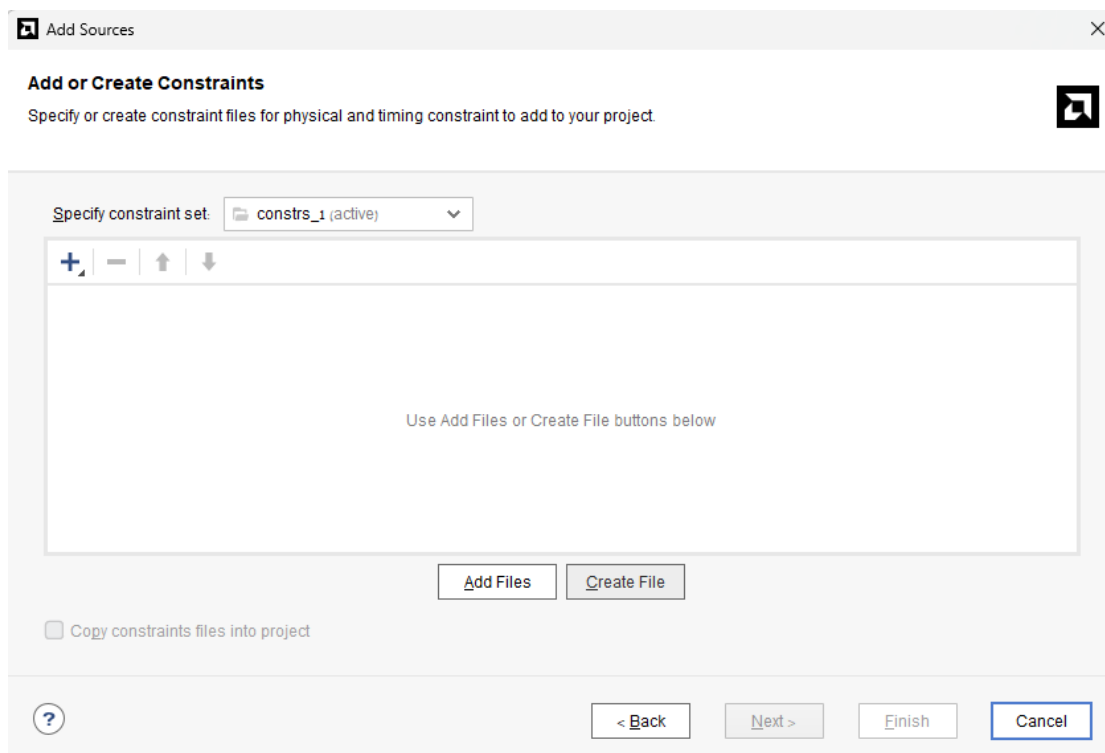
נוסיף קובץ אילוצים דרך Add Sources.



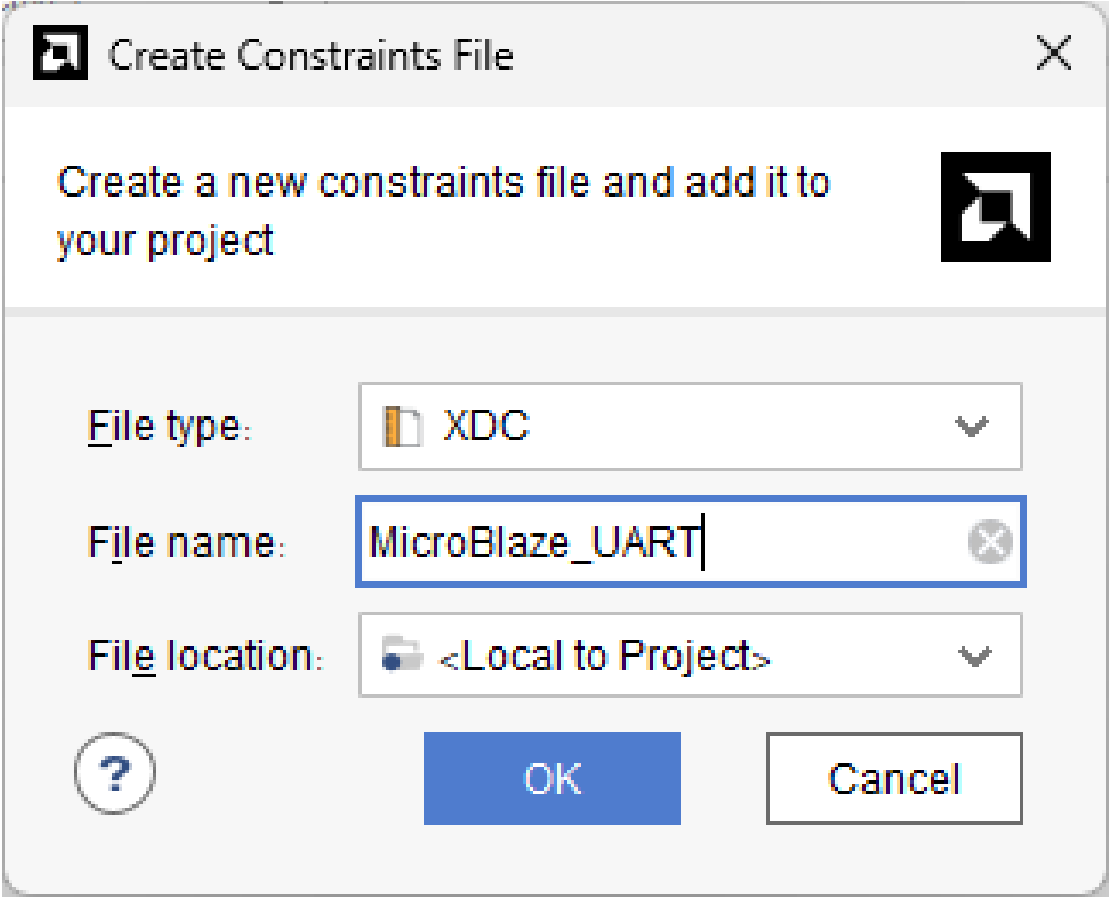
נבחר Add or create constraints ונלחץ Next.



נלחץ Create File.



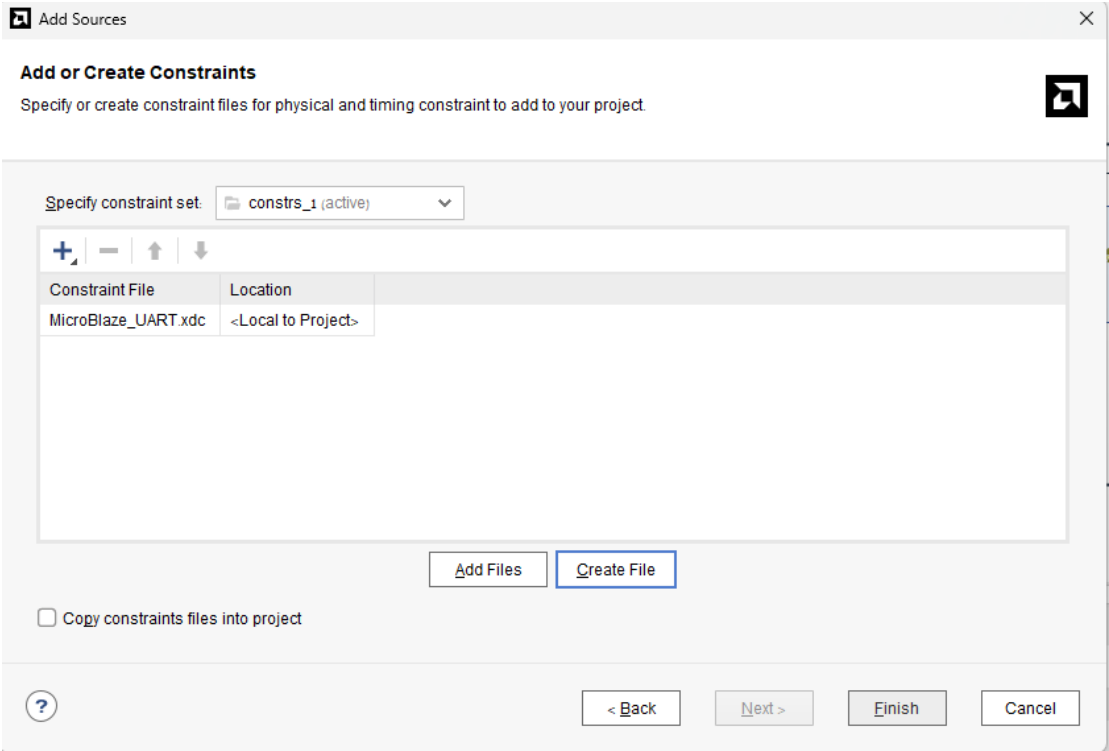
ניצור קובץ XDC בשם MicroBlaze_UART.



The dialog box is titled "Create Constraints File" and contains the following fields and controls:

- File type:** A dropdown menu showing "XDC".
- File name:** A text input field containing "MicroBlaze_UART".
- File location:** A dropdown menu showing "<Local to Project>".
- Buttons:** "OK" (blue), "Cancel", and a help icon (?) in a circle.

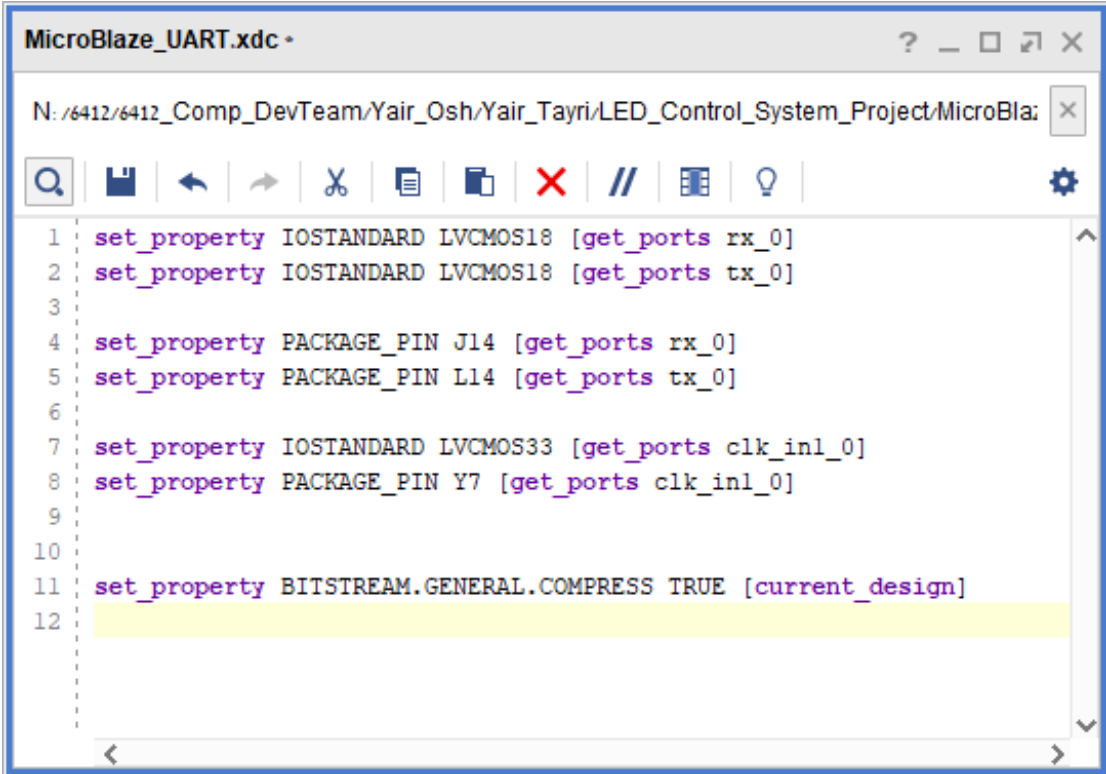
נלחץ Finish.



The dialog box is titled "Add Sources" and contains the following fields and controls:

- Specify constraint set:** A dropdown menu showing "constrs_1 (active)".
- Constraint File Table:** A table with two columns: "Constraint File" and "Location". It contains one row: "MicroBlaze_UART.xdc" and "<Local to Project>".
- Buttons:** "Add Files", "Create File", "< Back", "Next >", "Finish", "Cancel", and a help icon (?) in a circle.
- Checkbox:** "Copy constraints files into project" (unchecked).

בקובץ נמפה את קווי ה-rx וה-tx ואת כניסת השעון clk_in1, ונפעיל דחיסת Bitstream.



```
1 set_property IOSTANDARD LVCMOS18 [get_ports rx_0]
2 set_property IOSTANDARD LVCMOS18 [get_ports tx_0]
3
4 set_property PACKAGE_PIN J14 [get_ports rx_0]
5 set_property PACKAGE_PIN L14 [get_ports tx_0]
6
7 set_property IOSTANDARD LVCMOS33 [get_ports clk_in1_0]
8 set_property PACKAGE_PIN Y7 [get_ports clk_in1_0]
9
10
11 set_property BITSTREAM.GENERAL.COMPRESS TRUE [current_design]
12
```

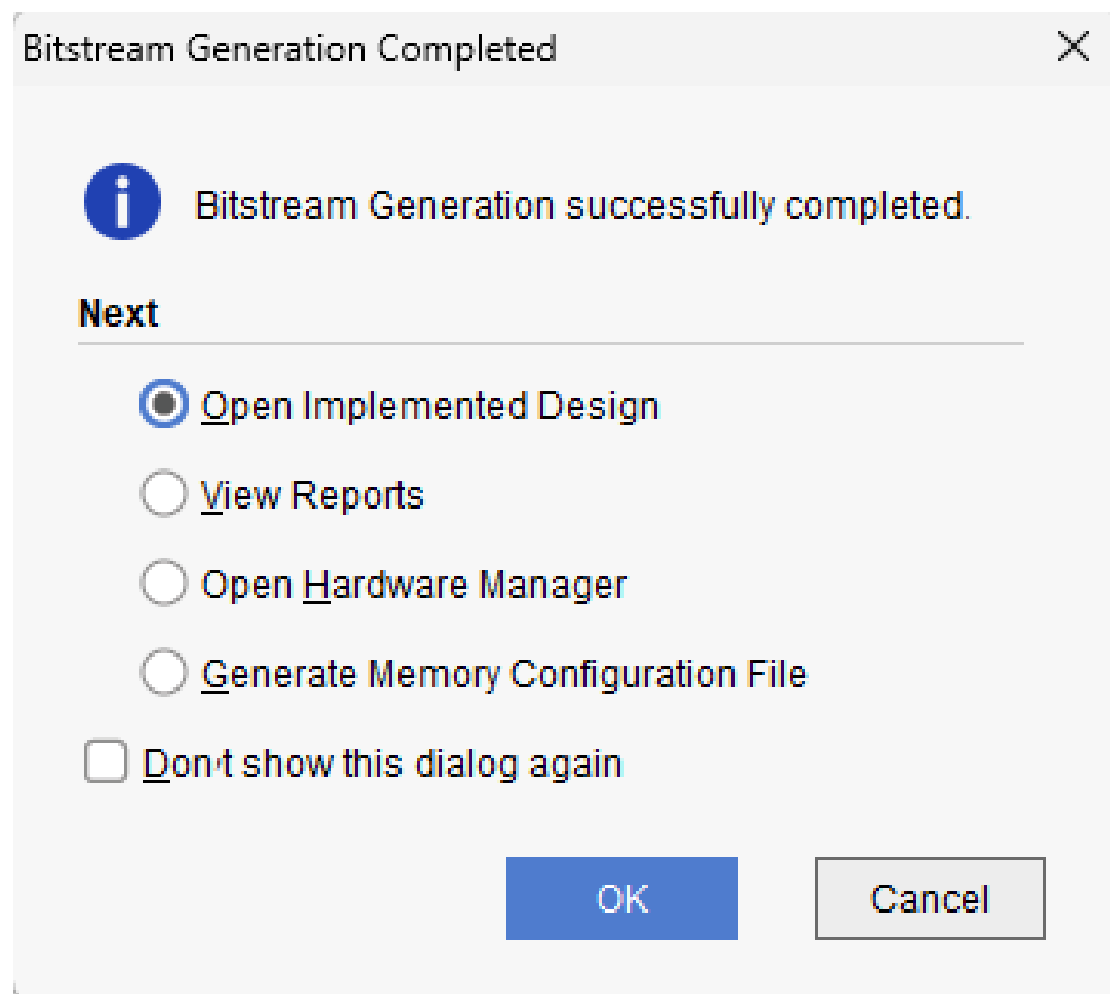
שלב 10:

נלחץ Generate Bitstream.

▼ PROGRAM AND DEBUG

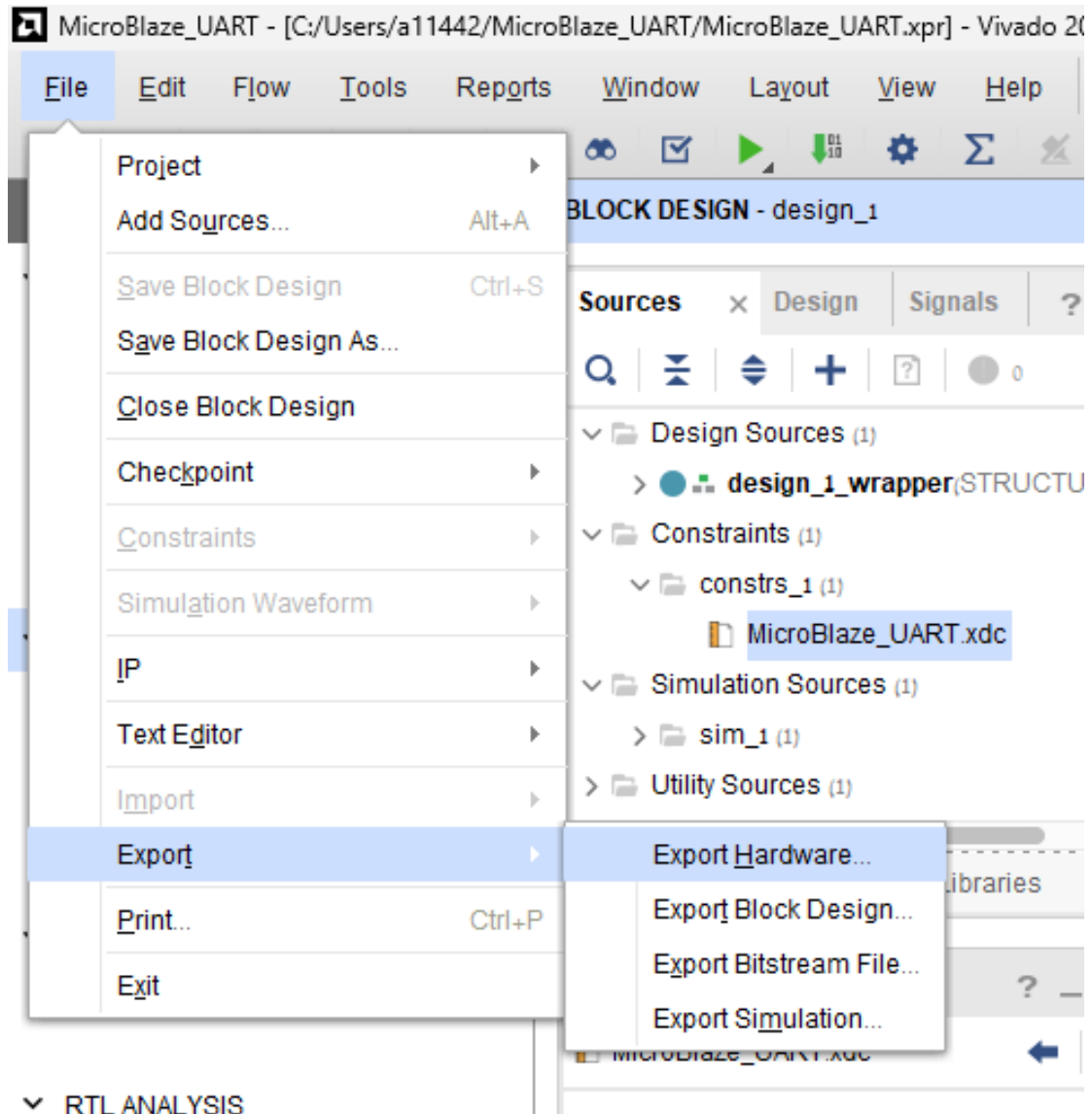
 Generate Bitstream

> Open Hardware Manager

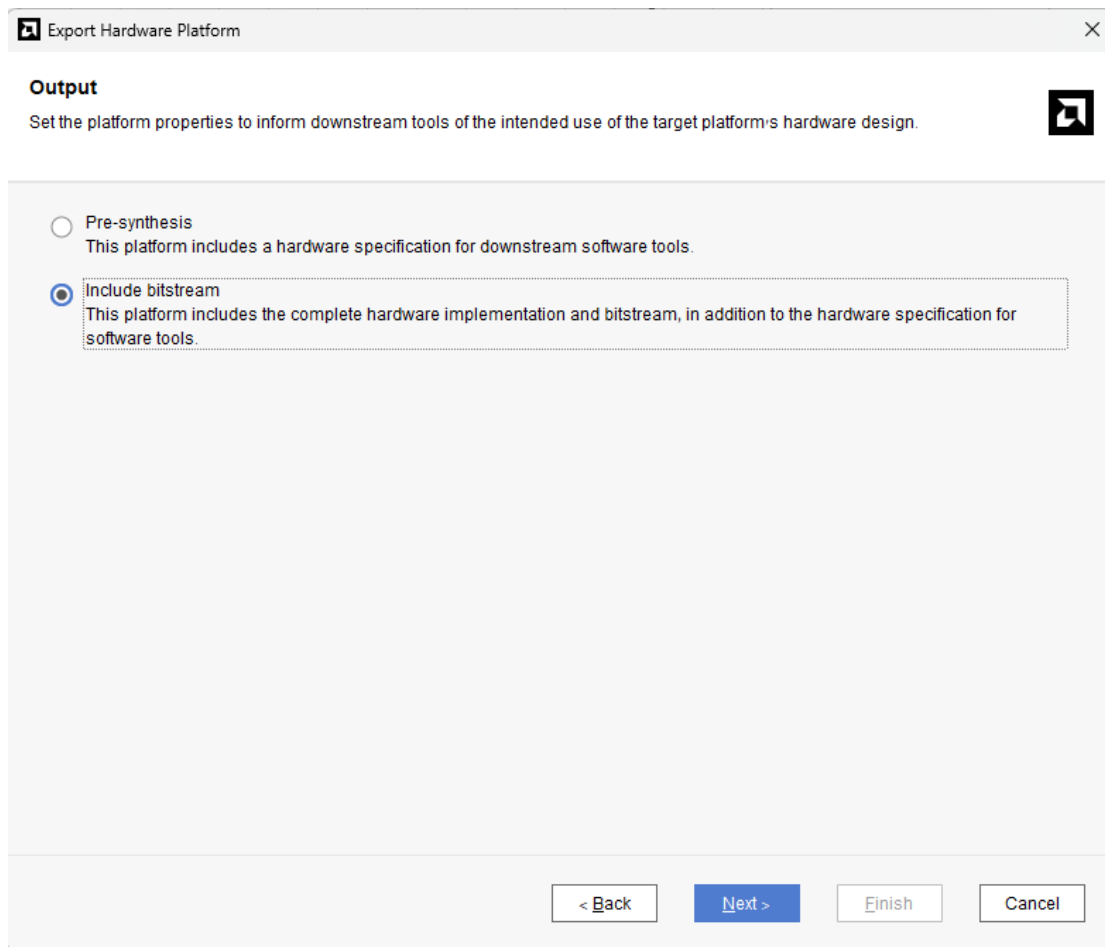


שלב 11:

נייצא את החומרה דרך File, Export Hardware ואז Export Hardware.



נבחר Include bitstream.



The image shows a software dialog box titled "Export Hardware Platform". It has a close button (X) in the top right corner. Below the title bar, there is a section labeled "Output" with a sub-instruction: "Set the platform properties to inform downstream tools of the intended use of the target platform's hardware design." To the right of this text is a small icon of a square with an arrow pointing out. Below this, there are two radio button options. The first option is "Pre-synthesis" with the description "This platform includes a hardware specification for downstream software tools." The second option is "Include bitstream" with the description "This platform includes the complete hardware implementation and bitstream, in addition to the hardware specification for software tools." The "Include bitstream" option is selected, indicated by a blue dot in the radio button. At the bottom of the dialog, there are four buttons: "< Back", "Next >" (highlighted in blue), "Finish", and "Cancel".

Export Hardware Platform X

Output

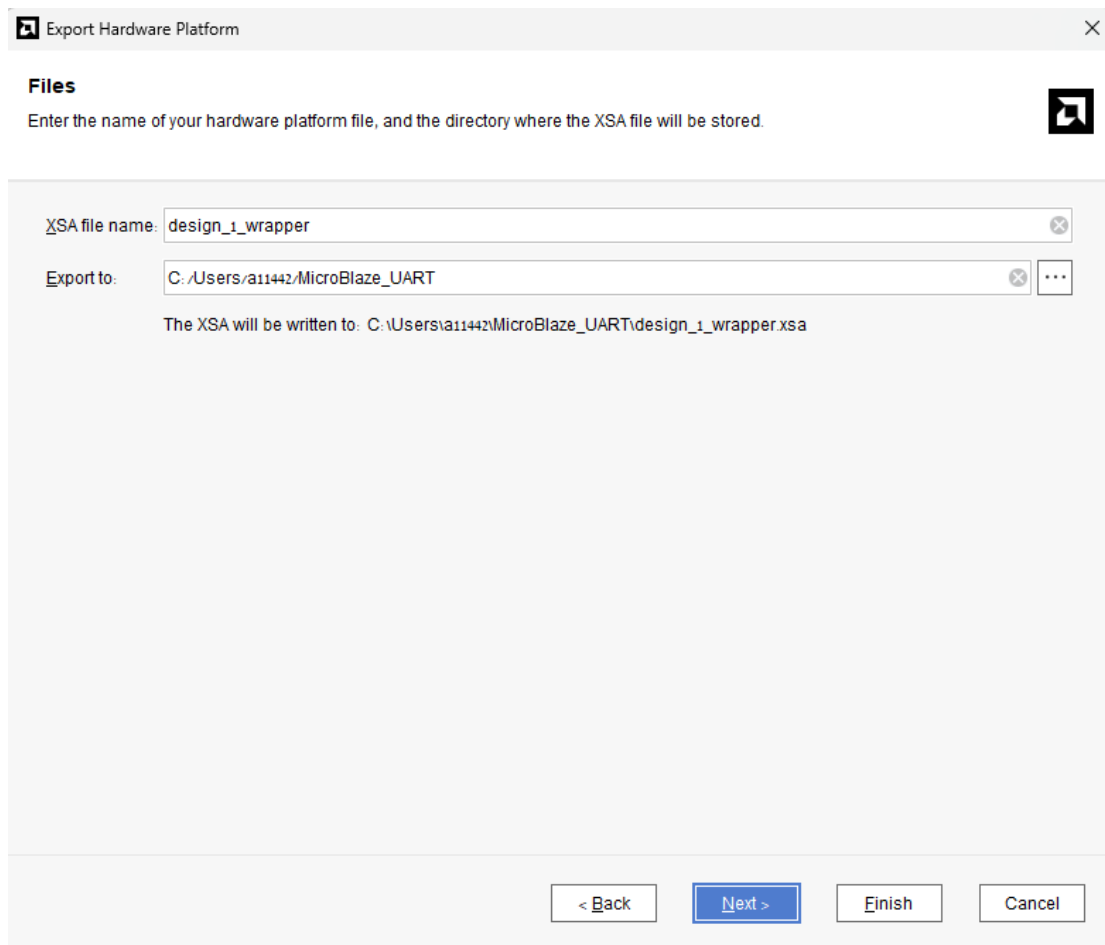
Set the platform properties to inform downstream tools of the intended use of the target platform's hardware design.

☐ Pre-synthesis
This platform includes a hardware specification for downstream software tools.

☒ Include bitstream
This platform includes the complete hardware implementation and bitstream, in addition to the hardware specification for software tools.

< Back Next > Finish Cancel

נשמור את קובץ ה-XSA בשם design_1_wrapper.



The image shows a Windows-style dialog box titled "Export Hardware Platform". It has a close button (X) in the top right corner. Below the title bar, there is a section labeled "Files" with a small icon of a folder and a document. The text below "Files" says: "Enter the name of your hardware platform file, and the directory where the XSA file will be stored." Below this, there are two input fields. The first is labeled "XSA file name:" and contains the text "design_1_wrapper". The second is labeled "Export to:" and contains the path "C:\Users\A11442\MicroBlaze_UART". To the right of the "Export to:" field is a button with three dots "...". Below these fields, a line of text states: "The XSA will be written to: C:\Users\A11442\MicroBlaze_UART\design_1_wrapper.xsa". At the bottom of the dialog, there are four buttons: "< Back", "Next >" (highlighted in blue), "Finish", and "Cancel".

Export Hardware Platform

Files

Enter the name of your hardware platform file, and the directory where the XSA file will be stored.

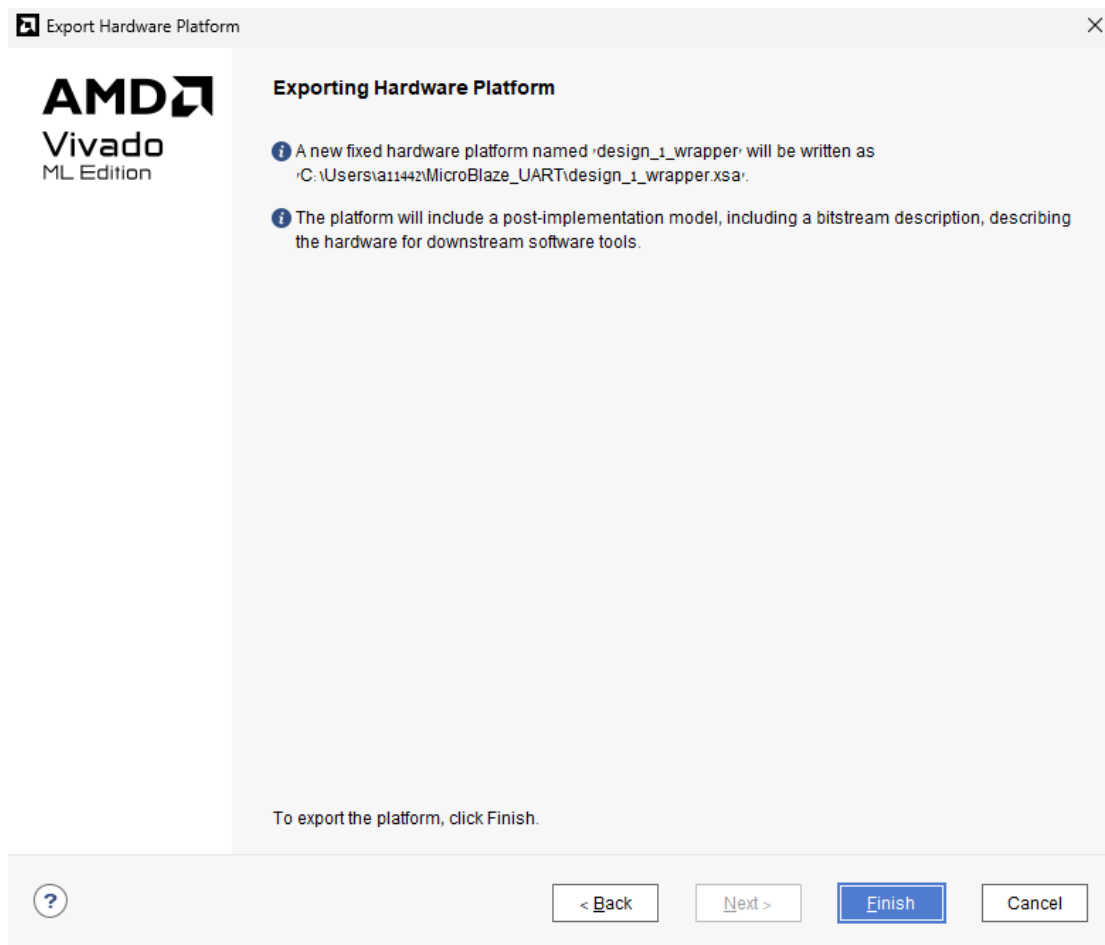
XSA file name: design_1_wrapper

Export to: C:\Users\A11442\MicroBlaze_UART

The XSA will be written to: C:\Users\A11442\MicroBlaze_UART\design_1_wrapper.xsa

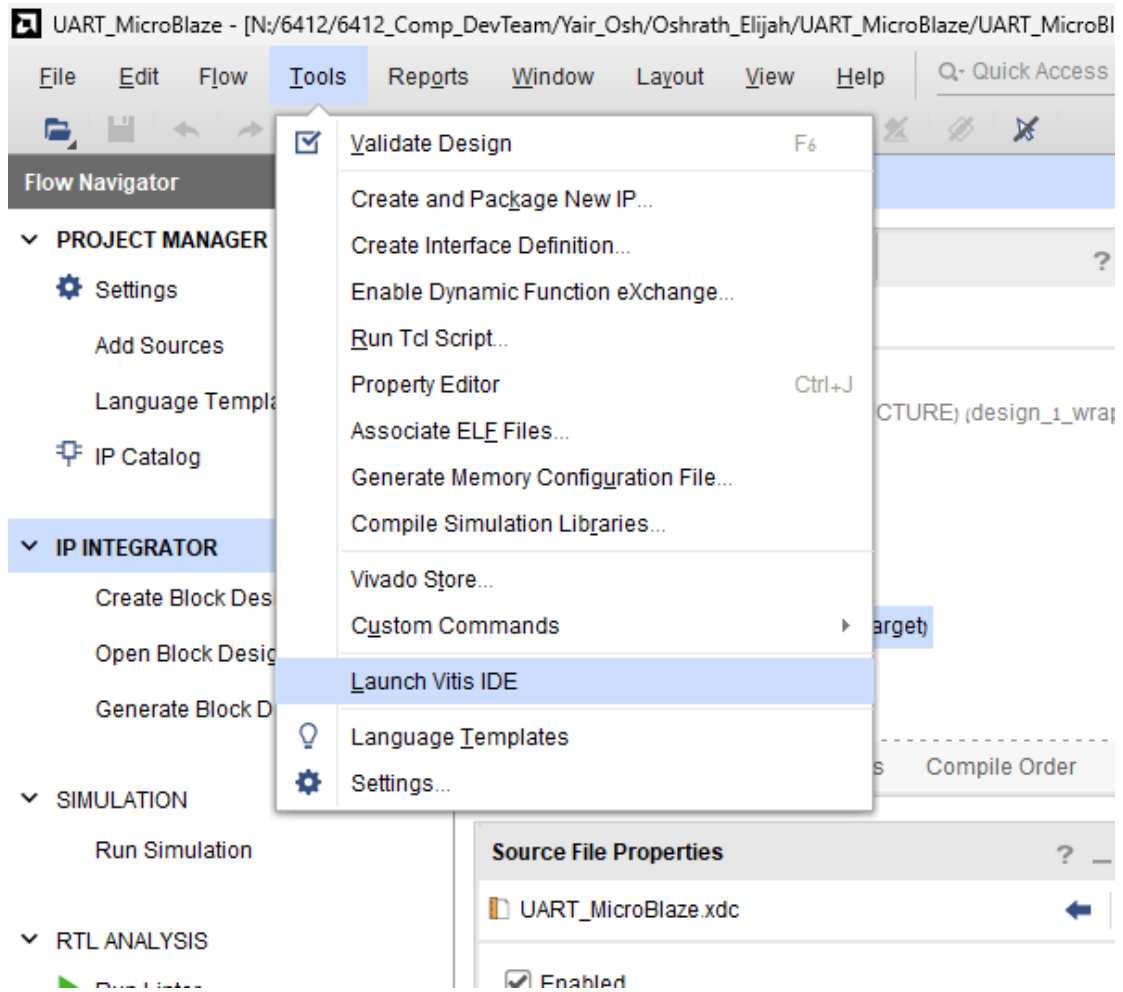
< Back Next > Finish Cancel

גמר. Finish

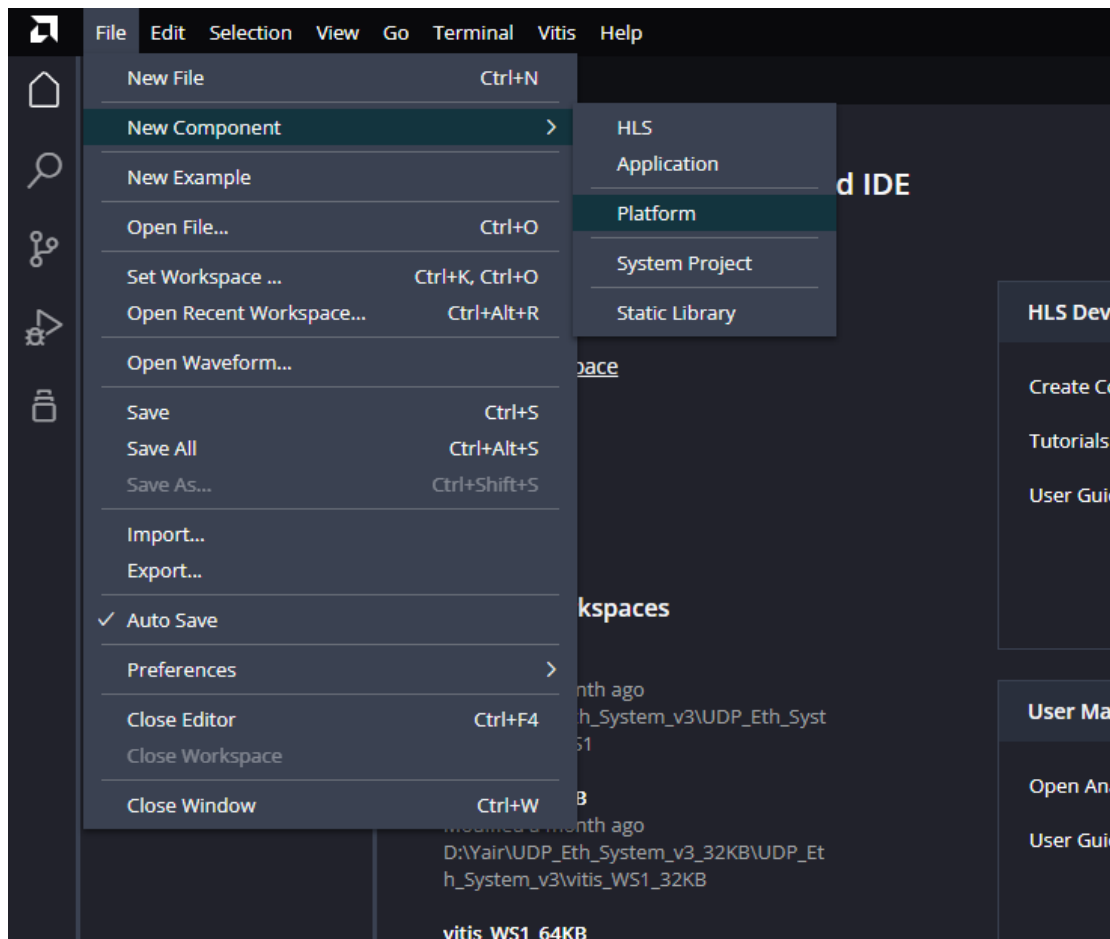


שלב 12:

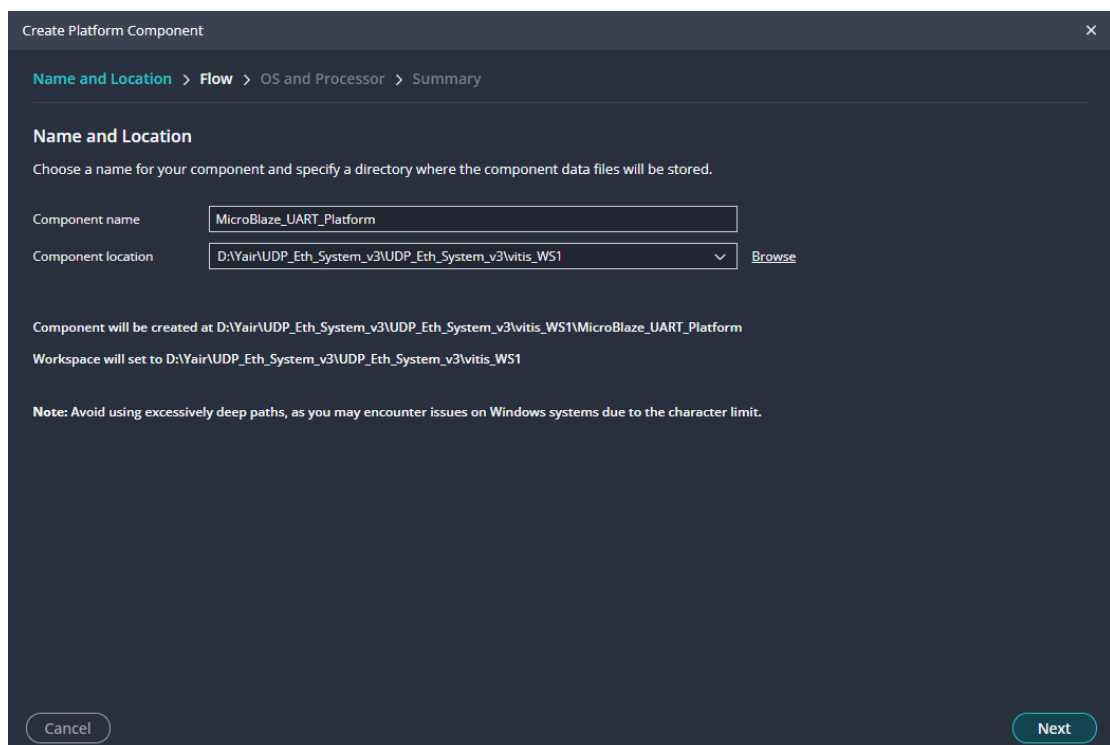
נפעיל את Vitis דרך Tools ואז Launch Vitis IDE.



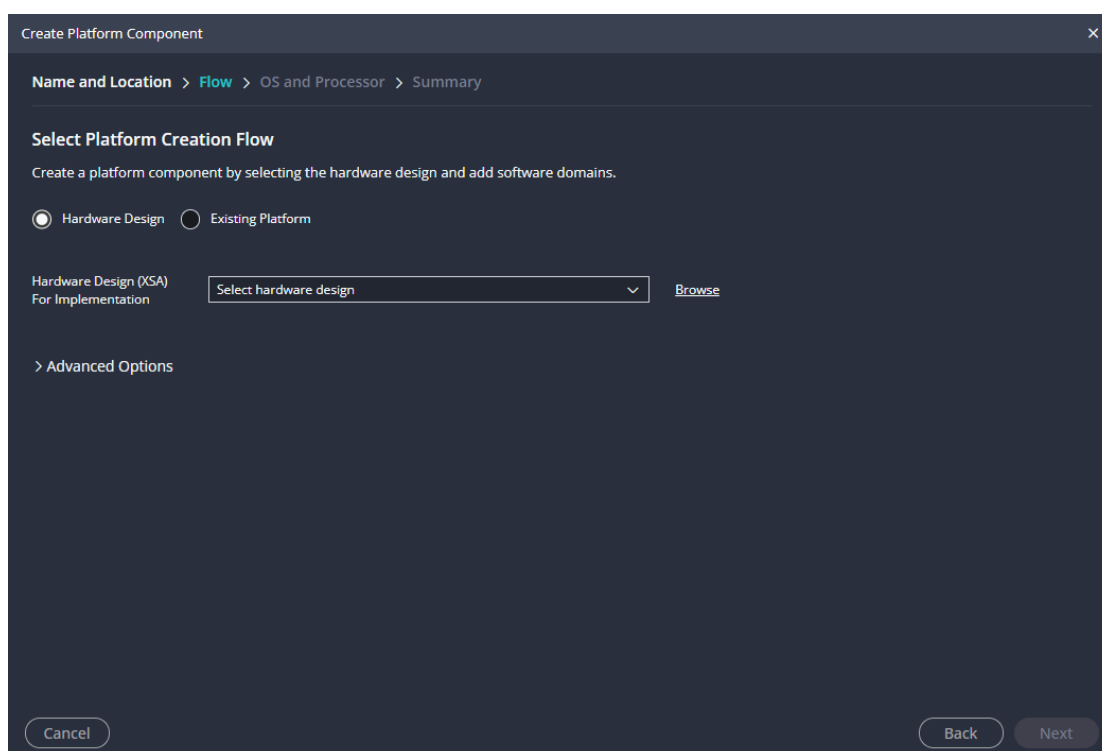
ניצור רכיב חדש דרך File, New Component ואז Platform.



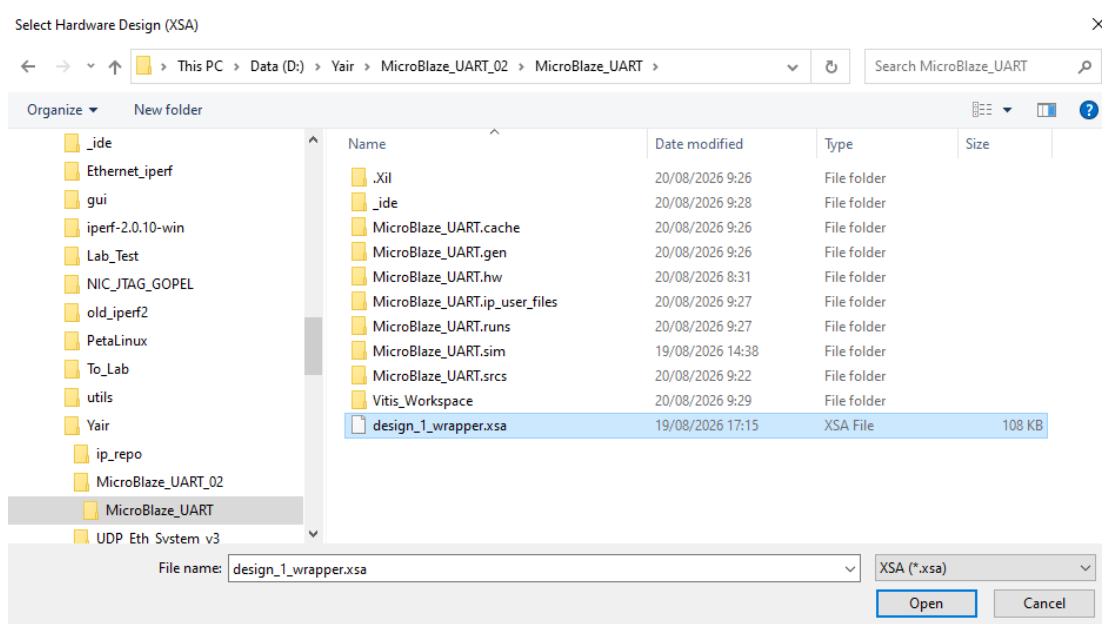
נקרא לפלטפורמה MicroBlaze_UART_Platform.



נבחר Hardware Design ונלחץ Browse.



נבחר את קובץ ה-XSA שיוצא מ-Vivado.

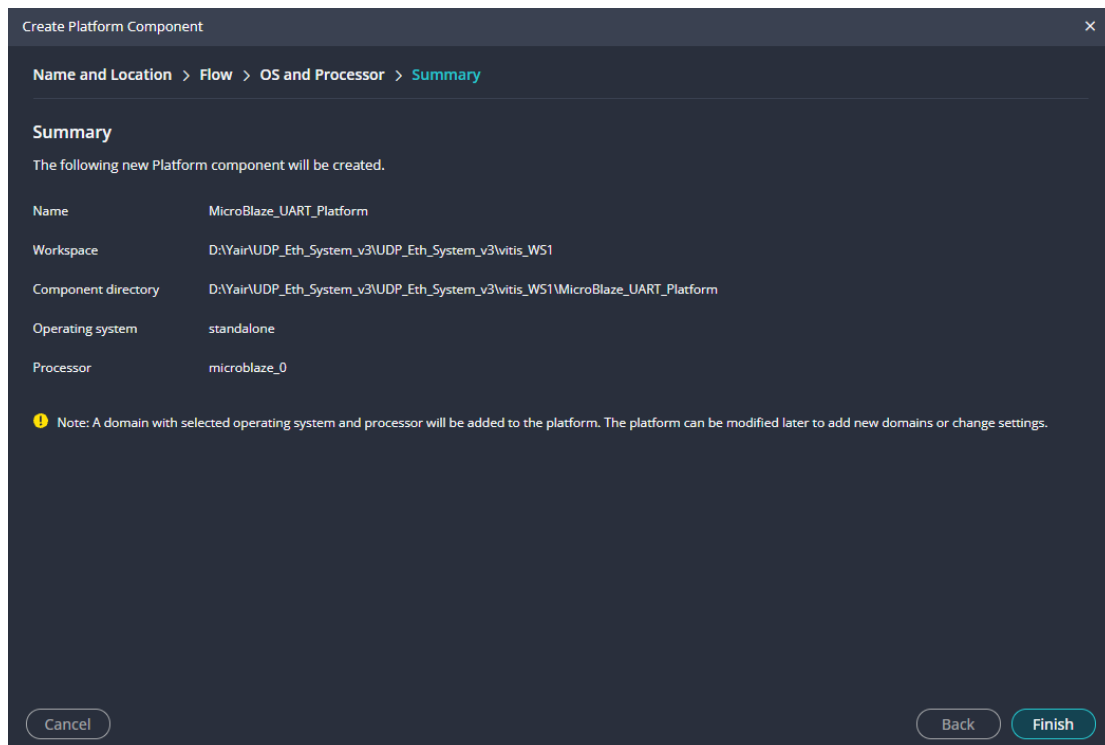


נוודא שהקובץ נבחר ונלחץ Next.

The screenshot shows the 'Create Platform Component' dialog box with the 'OS and Processor' tab selected. The breadcrumb trail is 'Name and Location > Flow > OS and Processor > Summary'. The section 'Select Platform Creation Flow' contains two radio buttons: 'Hardware Design' (selected) and 'Existing Platform'. Below this, the 'Hardware Design (XSA) For Implementation' section shows a file path 'D:\Yair\MicroBlaze_UART_02\MicroBlaze_UART\design_1_wrapper.x...' in a text box, with a 'Browse' button to its right. An '> Advanced Options' link is also present. At the bottom, there are 'Cancel', 'Back', and 'Next' buttons.

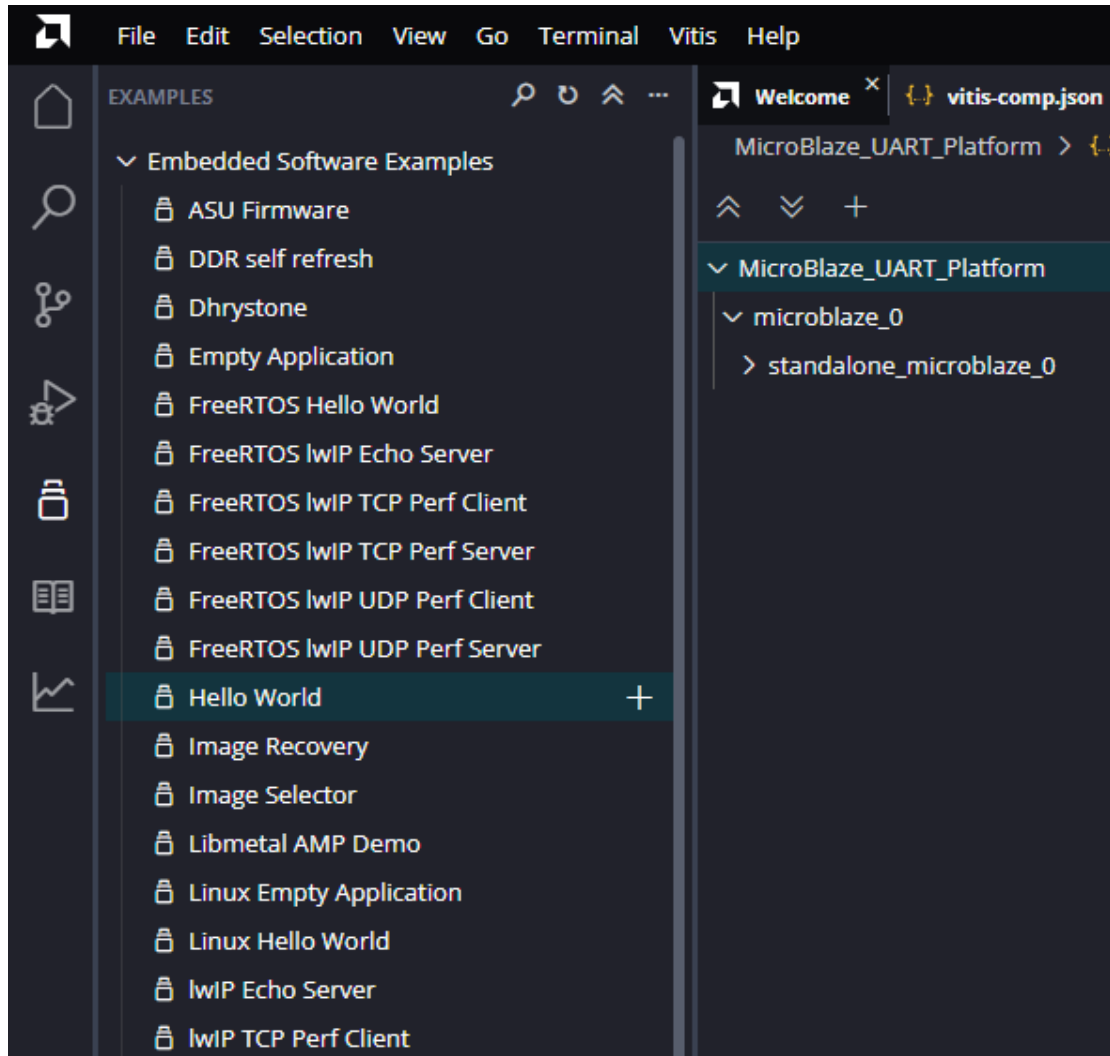
נגדיר מערכת הפעלה standalone ומעבד microblaze_0.

This screenshot shows the same 'Create Platform Component' dialog box, but with configuration options for the operating system and processor. The breadcrumb trail remains 'Name and Location > Flow > OS and Processor > Summary'. The section 'Select Operating System and Processor' includes a descriptive text: 'Specify the details for the initial domain to be added to the platform component. The platform can be modified later to add new domains or change settings by opening the platform editor.' Below this text are two dropdown menus: 'Operating system:' with 'standalone' selected, and 'Processor:' with 'microblaze_0' selected. The 'Cancel', 'Back', and 'Next' buttons are at the bottom.

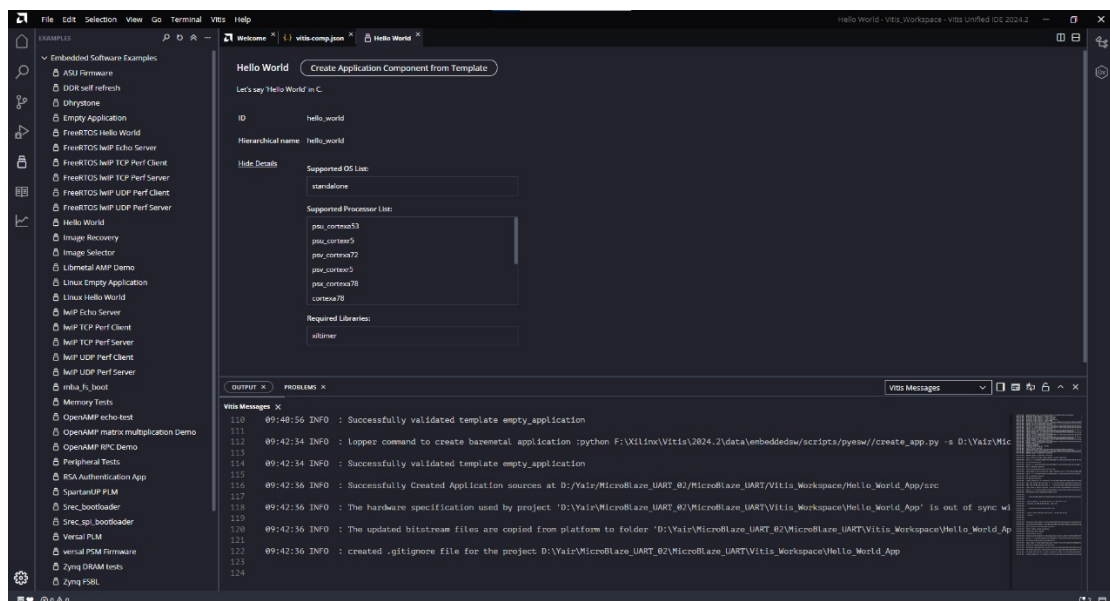


שלב 13:

מתוך הדוגמאות נבחר Hello World.



נלחץ Create Application Component from Template.



נקרא לאפליקציה Hello_World_App.

Create Application Component - Hello World

Name and Location > Hardware > Domain > Summary

Name and Location

Choose a name for your component and specify a directory where the component data files will be stored.

Component name: Hello_World_App

Component location: D:\Yair\MicroBlaze_UART_02\MicroBlaze_UART\Vitis_Workspace [Browse](#)

Component will be created at D:\Yair\MicroBlaze_UART_02\MicroBlaze_UART\Vitis_Workspace\Hello_World_App

Note: Avoid using excessively deep paths, as you may encounter issues on Windows systems due to the character limit.

[Cancel](#) [Next](#)

נבחר את הפלטפורמה MicroBlaze_UART_Platform.

Create Application Component - Hello World

Name and Location > Hardware > Domain > Summary

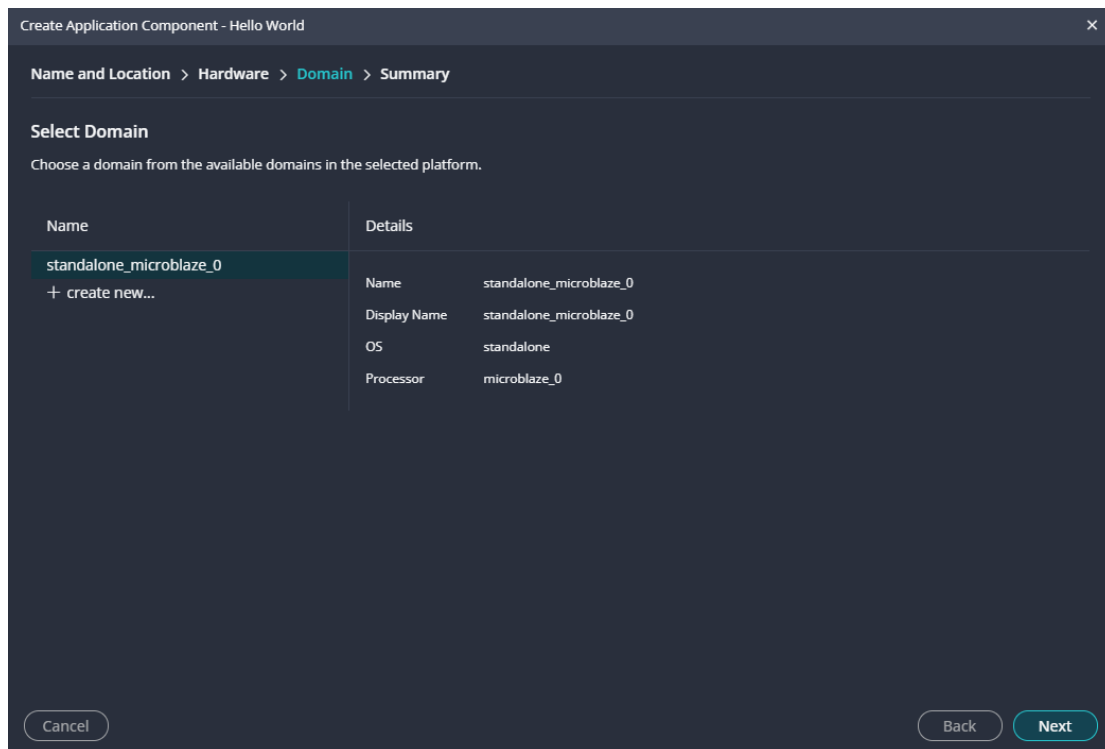
Select Platform

Platforms from your repositories that support the selected example. To create a new platform, use "File -> New Component -> Platform"

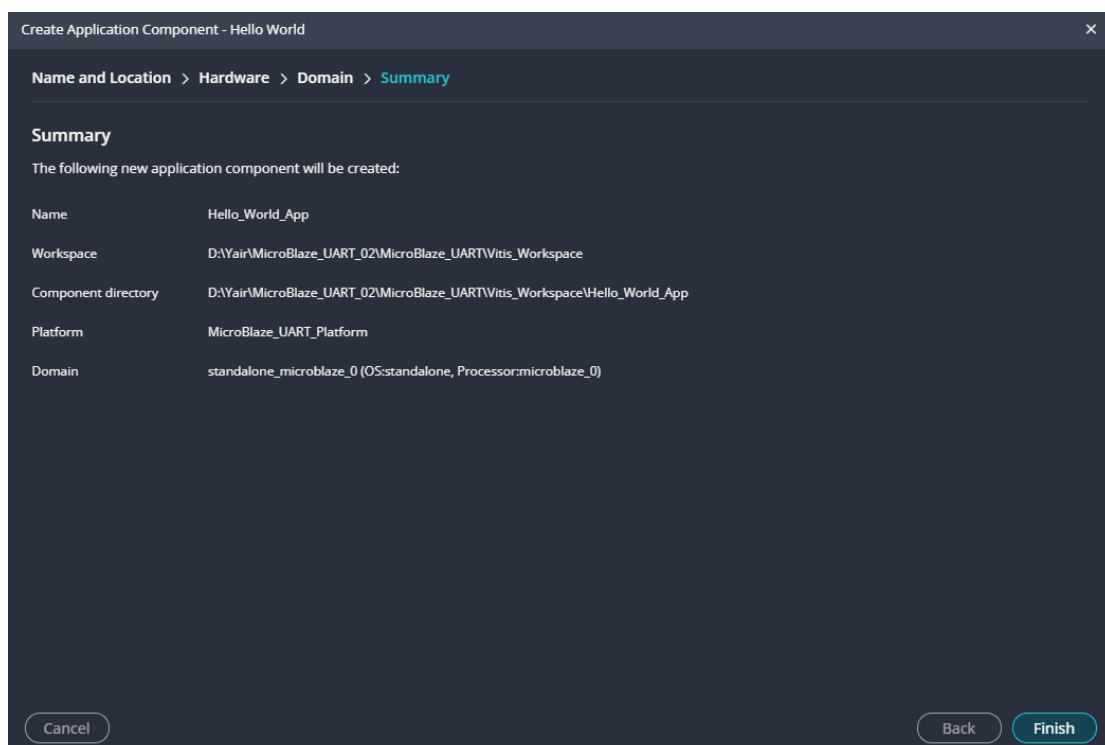
NAME	BOARD	FLOW	VENDOR	PATH
D:\Yair\MicroBlaze_UART_02\MicroBlaze_UART\Vitis_Workspace (1)				
MicroBlaze_UART_Platform		Embedded	xilinx.com	...icroBlaze_UART_Platform\MicroBlaze_UART_Platform.xpfm Info

[Cancel](#) [Back](#) [Next](#)

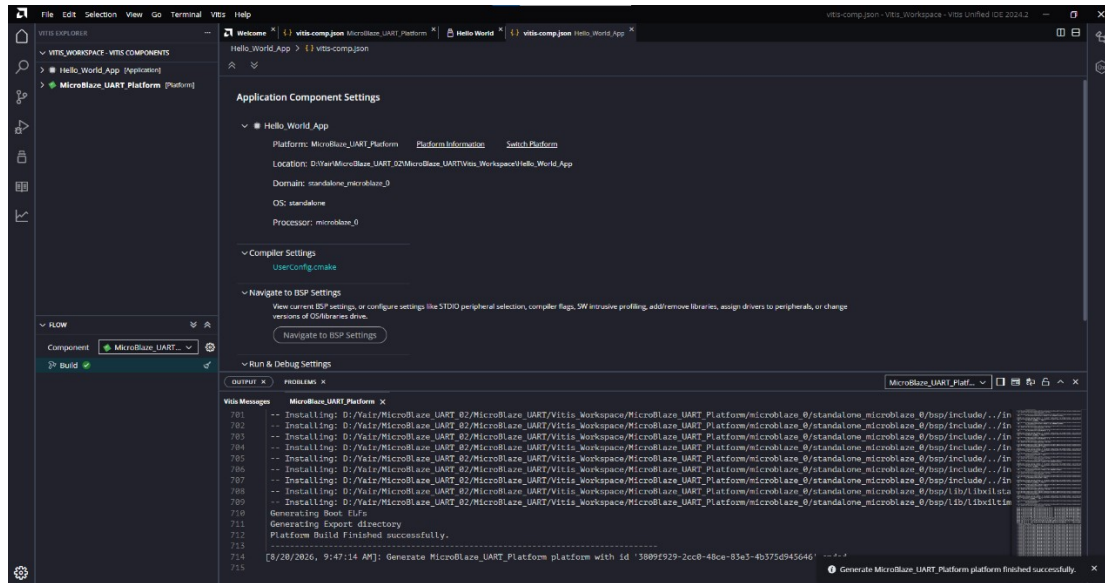
נבחר את הדומיין standalone_microblaze_0.



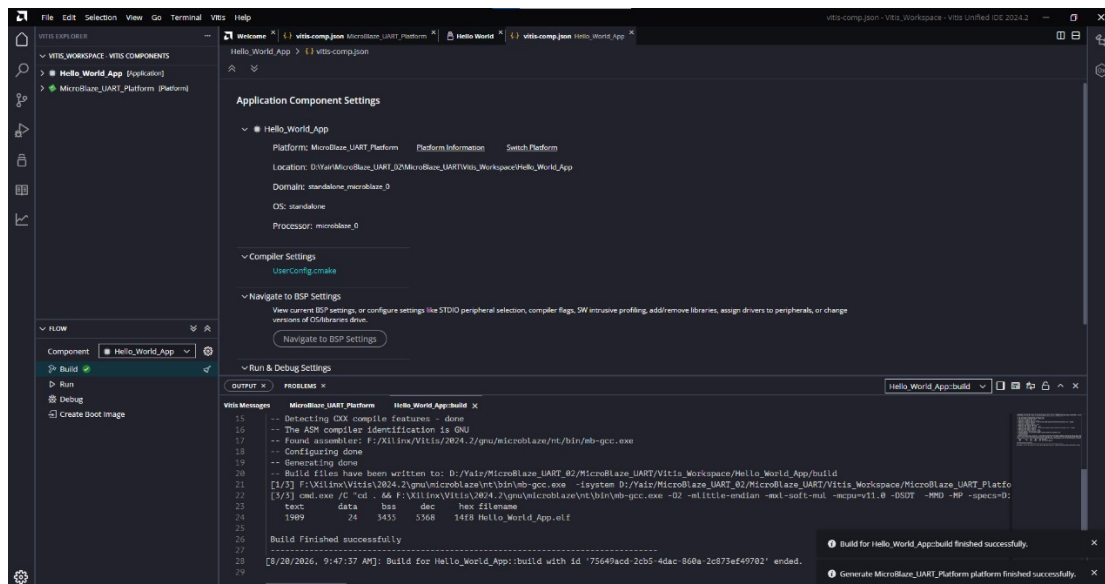
נלחץ Finish.



נבצע Build לאפליקציה.

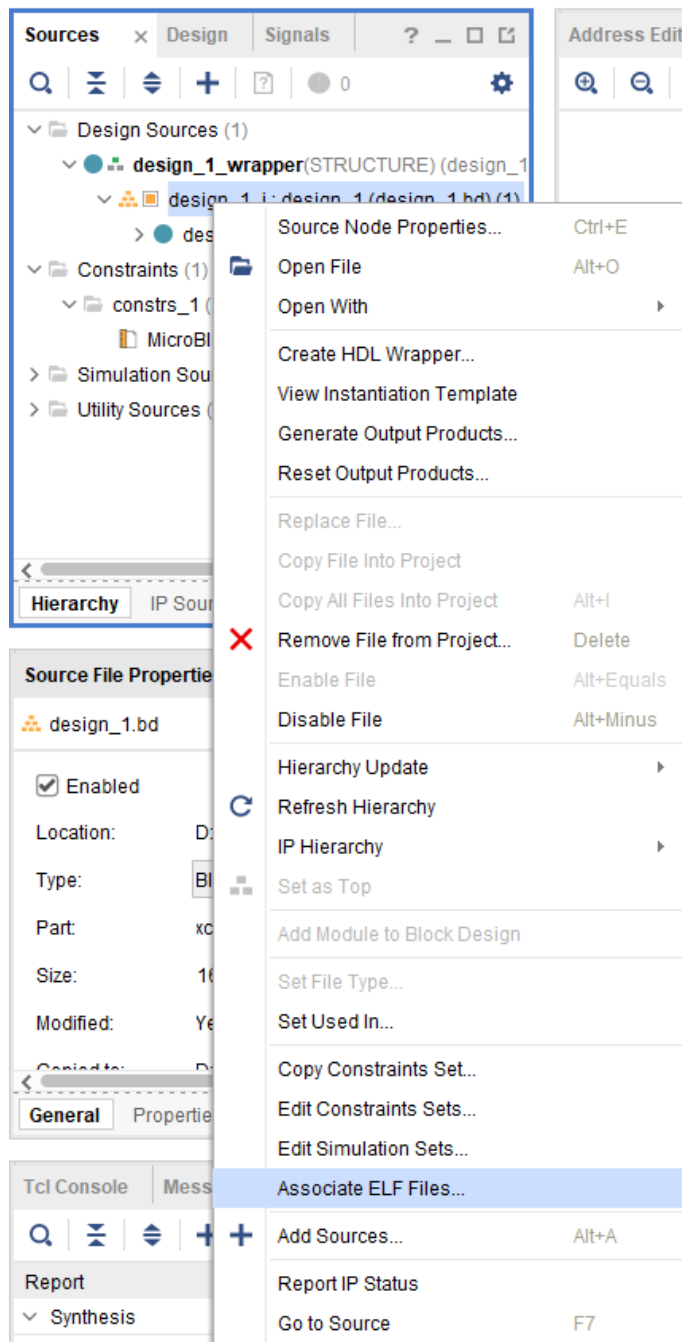


ה-Build הסתיים בהצלחה.

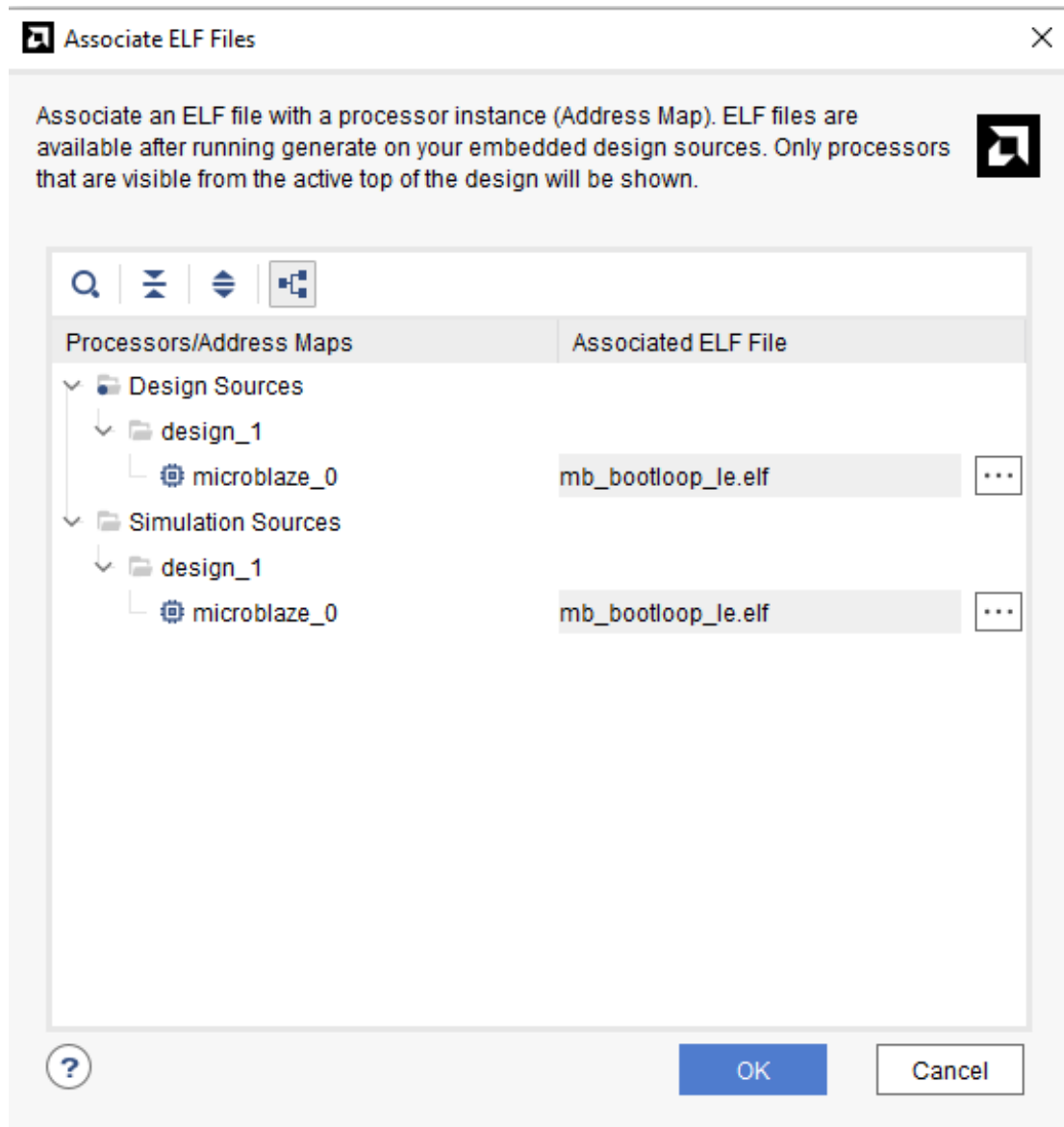


שלב 14:

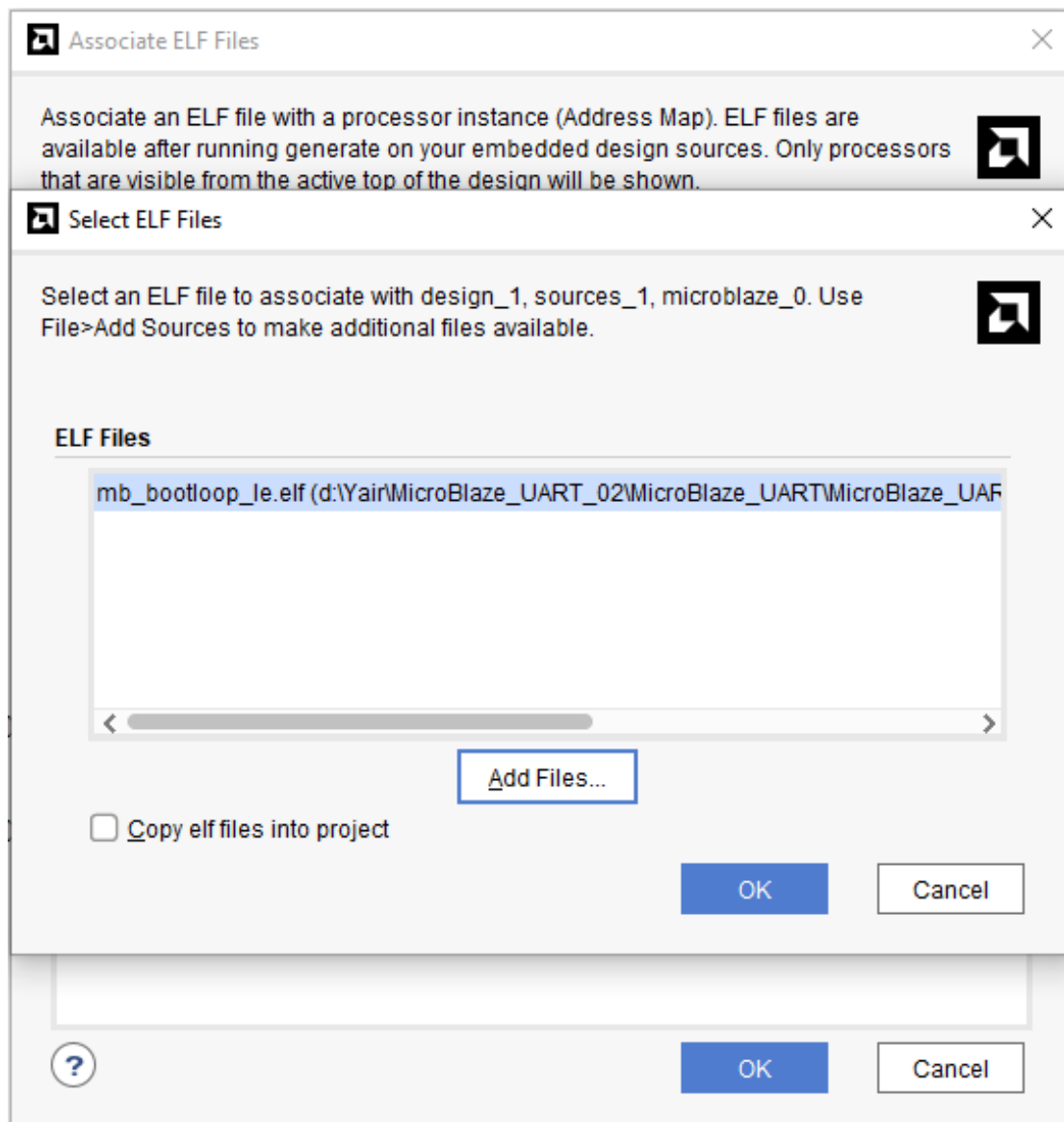
נחזור ל-Vivado. בחלון Sources נלחץ קליק ימני ובחר Associate ELF Files.



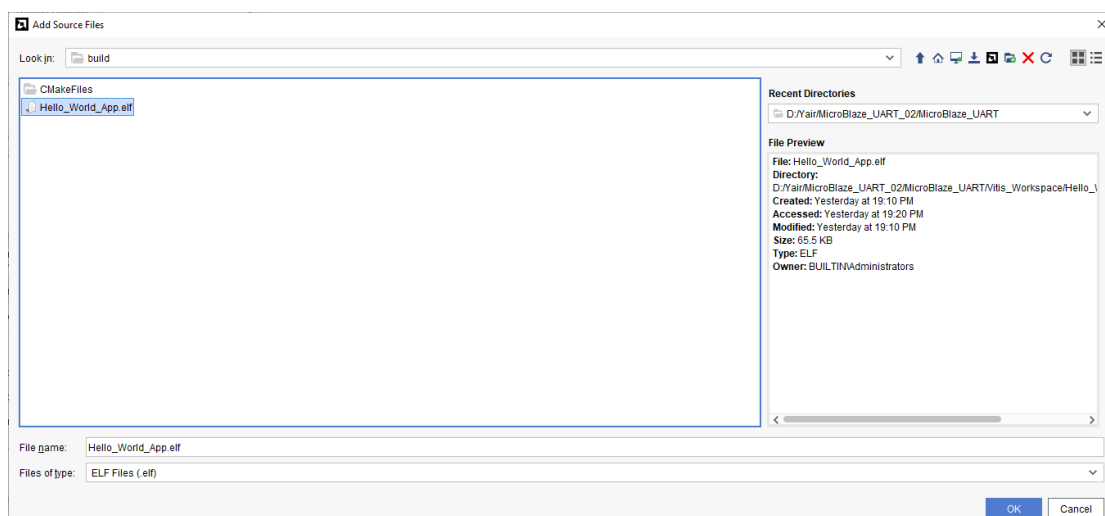
כברירת מחדל משויך הקובץ `mb_bootloop_le.elf`, המחזיק את המעבד בלולאה ריקה. נלחץ על כפתור הבחירה.



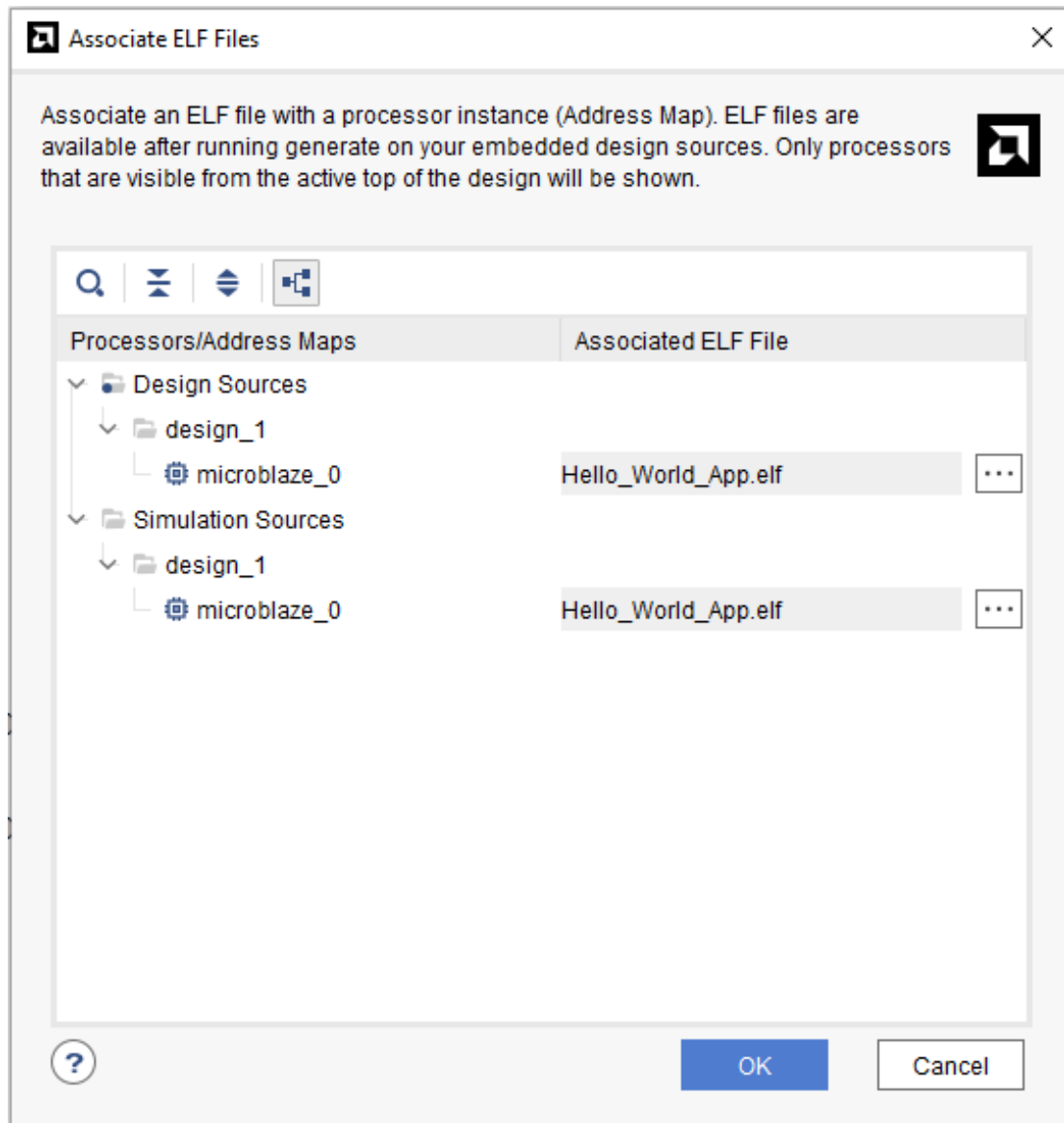
נלחץ Add Files.



נבחר את הקובץ Hello_World_App.elf.



נוודא שהקובץ שויך עבור Design Sources ועבור Simulation Sources. ביצירת ה-Bitstream, הבא Vivado יטען את קוד התוכנה לזיכרון ה-BRAM, והמעבד יריץ אותו מיד לאחר הצריבה.



שלב 15:

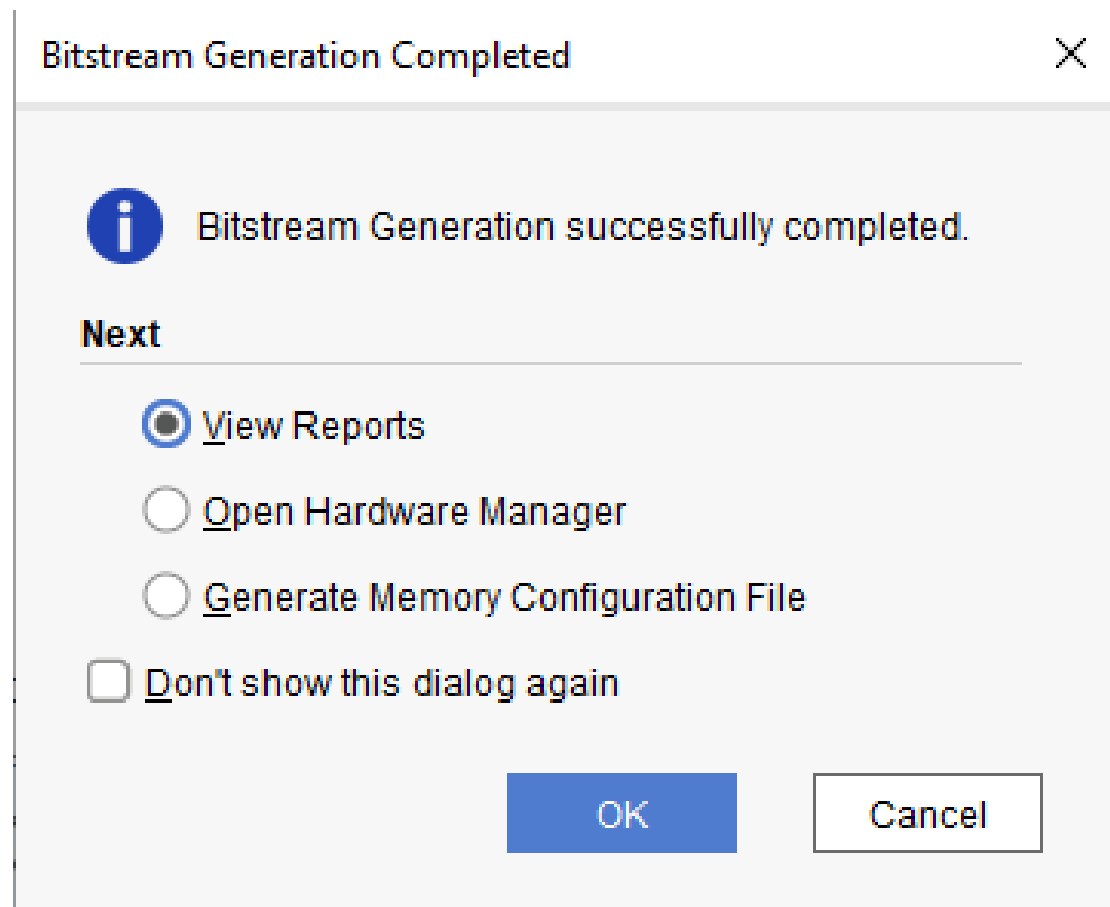
נלחץ Generate Bitstream.

▼ PROGRAM AND DEBUG

↓ Generate Bitstream

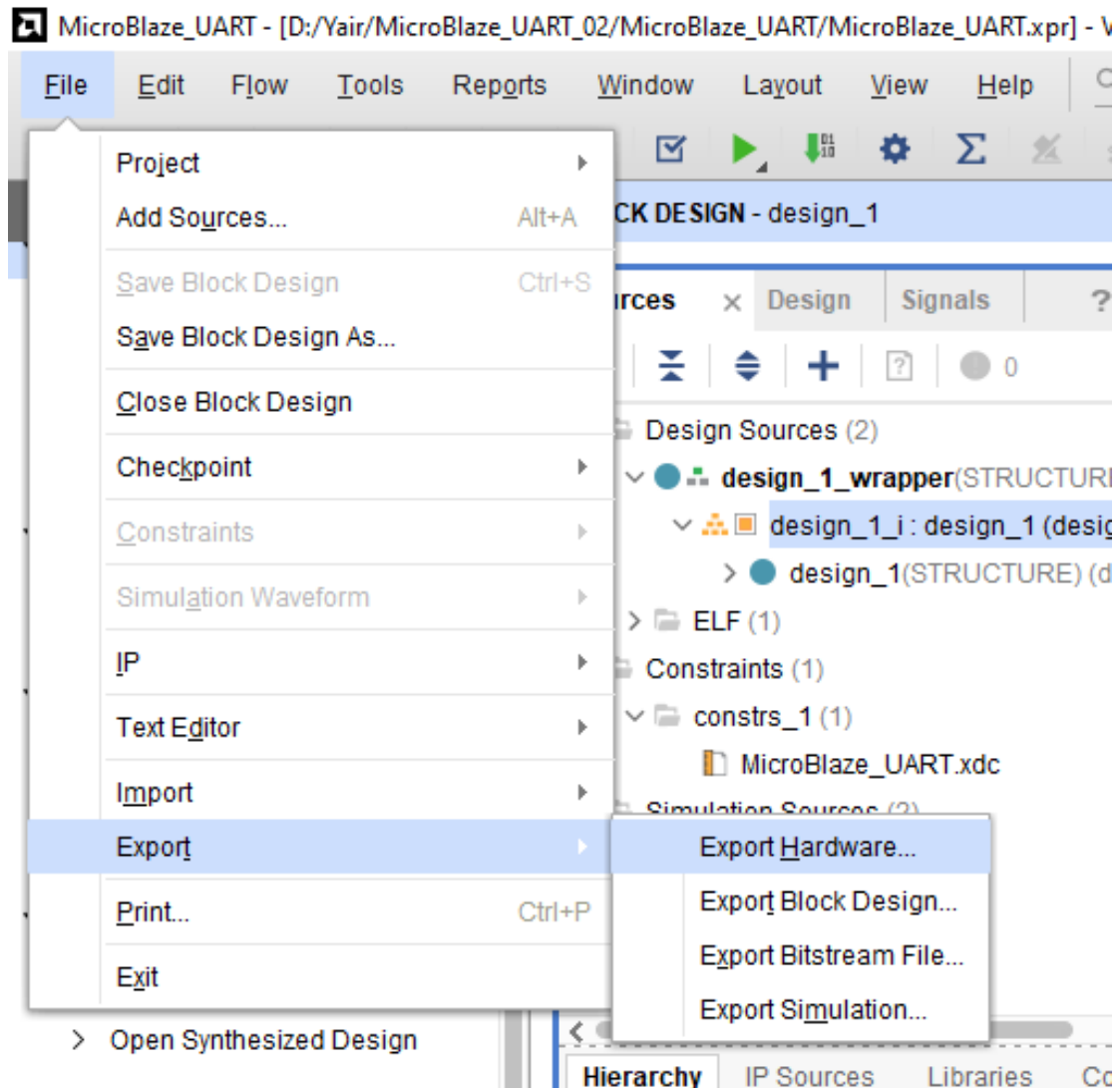
> Open Hardware Manager

ה-Bitstream כולל כעת את קוד התוכנה.

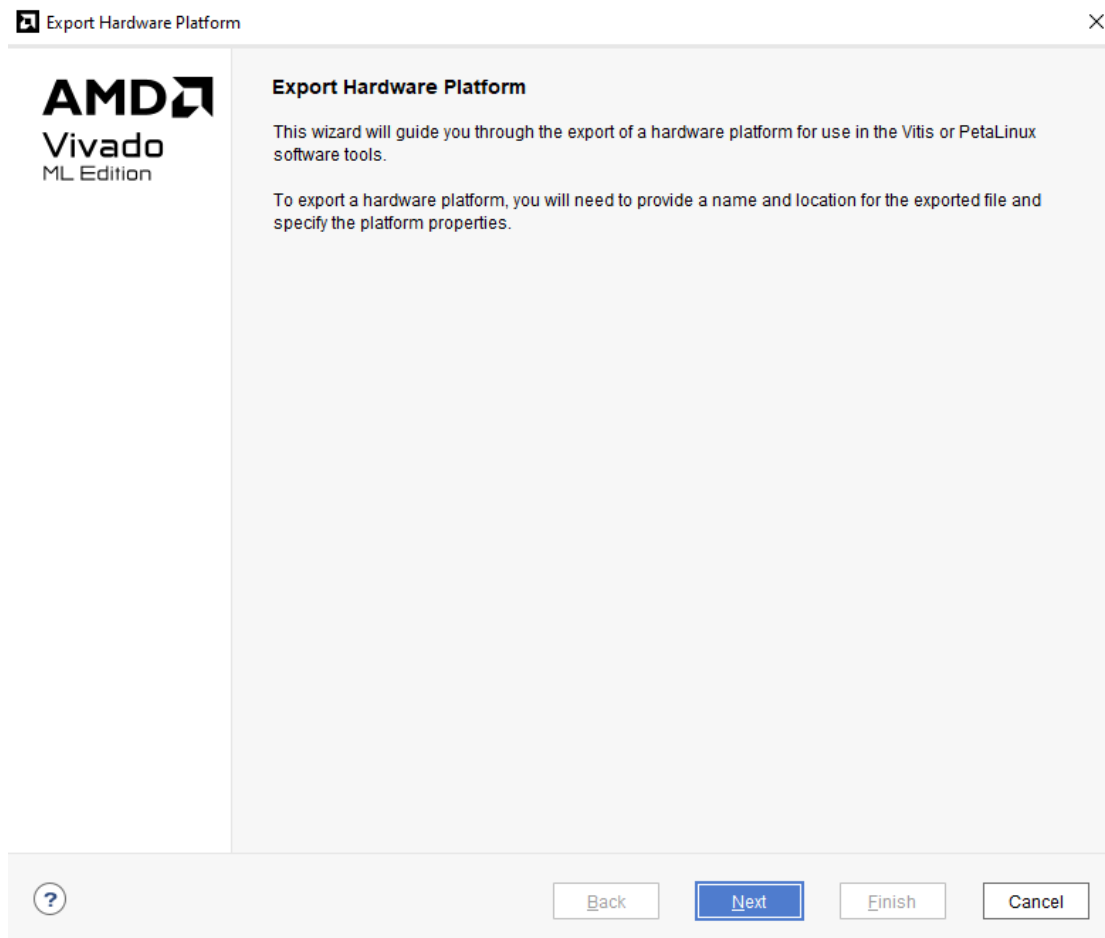


שלב 16:

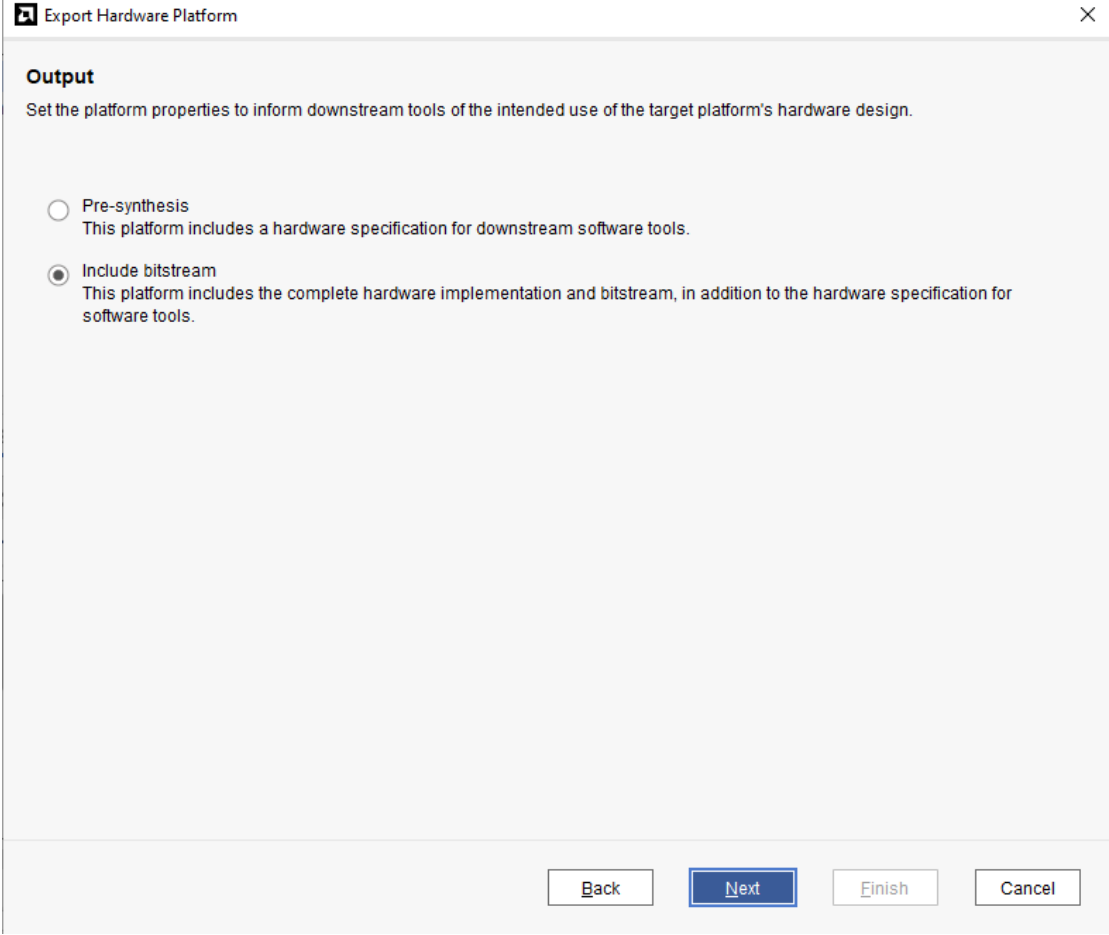
נמצא שוב את החומרה דרך Export Hardware.



נלחץ Next.



נבחר Include bitstream.



The image shows a software dialog box titled "Export Hardware Platform" with a close button (X) in the top right corner. The main area is titled "Output" and contains a descriptive sentence: "Set the platform properties to inform downstream tools of the intended use of the target platform's hardware design." Below this, there are two radio button options. The first option is "Pre-synthesis" with the description "This platform includes a hardware specification for downstream software tools." The second option is "Include bitstream" with the description "This platform includes the complete hardware implementation and bitstream, in addition to the hardware specification for software tools." The "Include bitstream" option is selected, indicated by a filled radio button. At the bottom of the dialog, there are four buttons: "Back", "Next" (which is highlighted with a blue border), "Finish", and "Cancel".

Export Hardware Platform ✕

Output

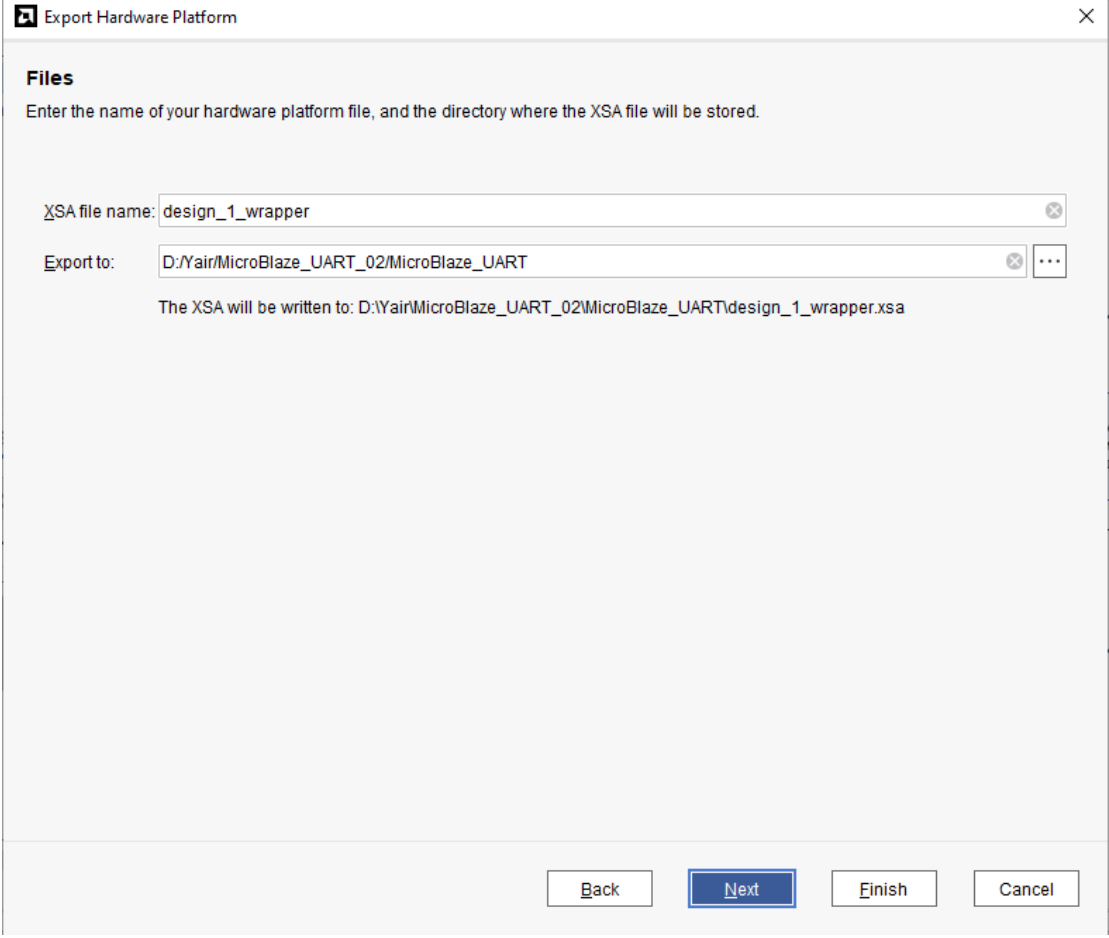
Set the platform properties to inform downstream tools of the intended use of the target platform's hardware design.

☐ Pre-synthesis
This platform includes a hardware specification for downstream software tools.

☒ Include bitstream
This platform includes the complete hardware implementation and bitstream, in addition to the hardware specification for software tools.

Back Next Finish Cancel

נשמור באותו שם ובאותו מיקום.



The dialog box is titled "Export Hardware Platform" and has a close button (X) in the top right corner. It contains a section titled "Files" with the instruction: "Enter the name of your hardware platform file, and the directory where the XSA file will be stored." Below this, there are two input fields. The first is labeled "XSA file name:" and contains the text "design_1_wrapper". The second is labeled "Export to:" and contains the path "D:\Yair\MicroBlaze_UART_02\MicroBlaze_UART". To the right of the "Export to:" field is a button with three dots "...". Below these fields, a line of text states: "The XSA will be written to: D:\Yair\MicroBlaze_UART_02\MicroBlaze_UART\design_1_wrapper.xsa". At the bottom of the dialog, there are four buttons: "Back", "Next" (which is highlighted with a blue border), "Finish", and "Cancel".

Files

Enter the name of your hardware platform file, and the directory where the XSA file will be stored.

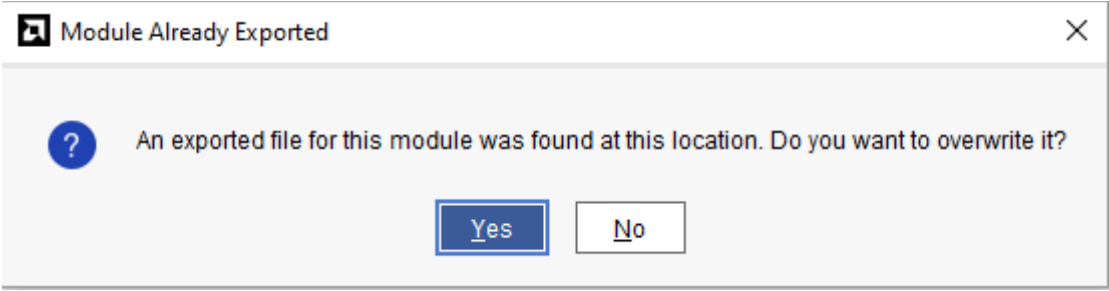
XSA file name: design_1_wrapper

Export to: D:\Yair\MicroBlaze_UART_02\MicroBlaze_UART

The XSA will be written to: D:\Yair\MicroBlaze_UART_02\MicroBlaze_UART\design_1_wrapper.xsa

Back Next Finish Cancel

נאשר דריסה של הקובץ הקיים.



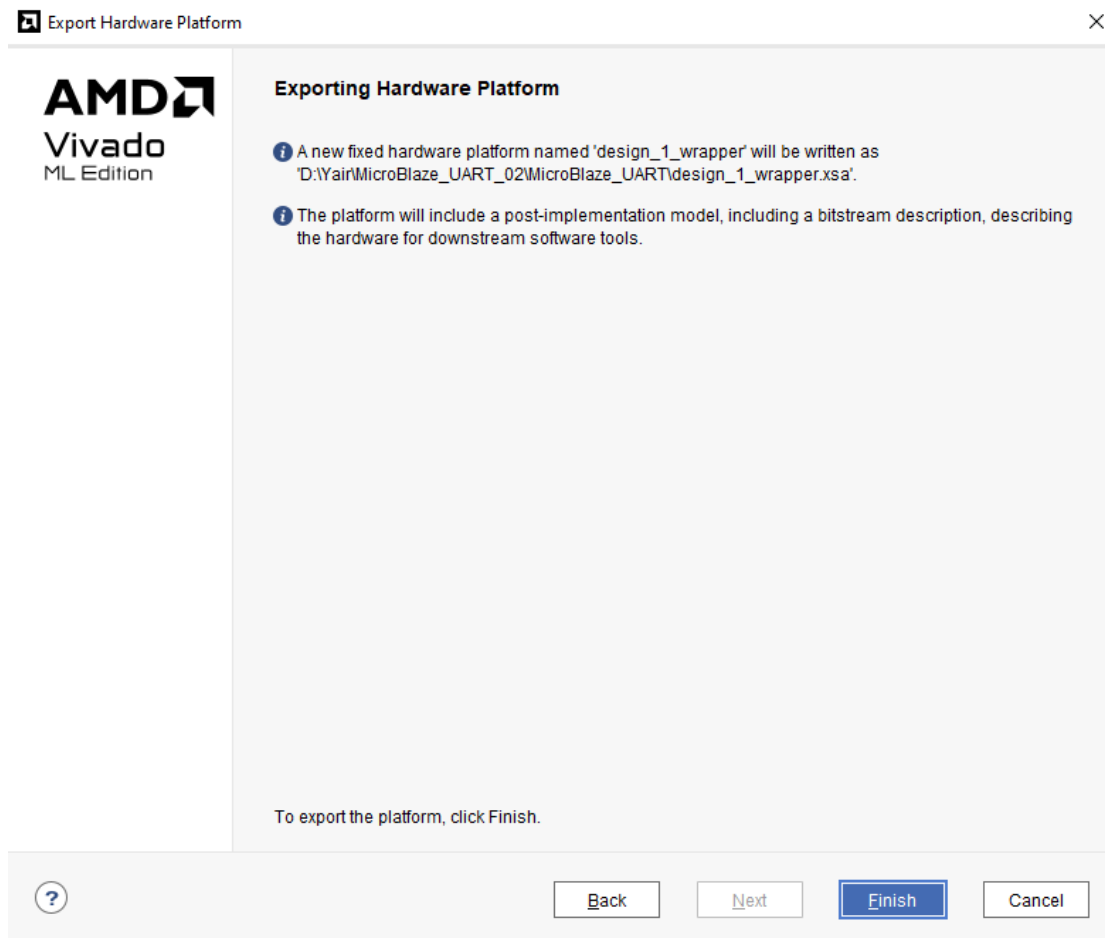
The dialog box is titled "Module Already Exported" and has a close button (X) in the top right corner. It contains a question mark icon in a blue circle followed by the text: "An exported file for this module was found at this location. Do you want to overwrite it?". At the bottom, there are two buttons: "Yes" (highlighted with a blue border) and "No".

Module Already Exported

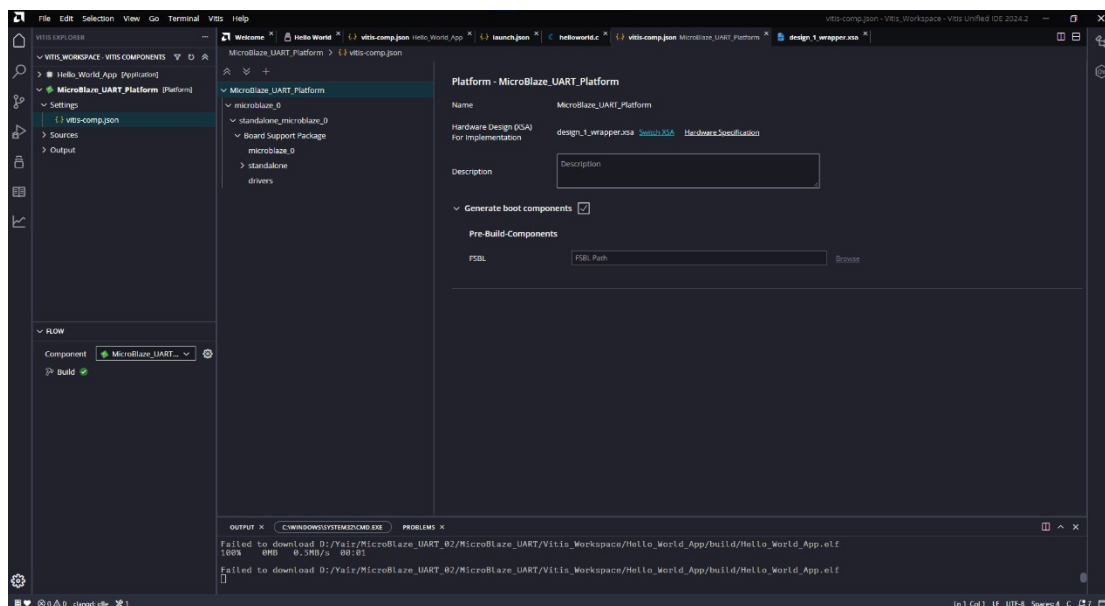
? An exported file for this module was found at this location. Do you want to overwrite it?

Yes No

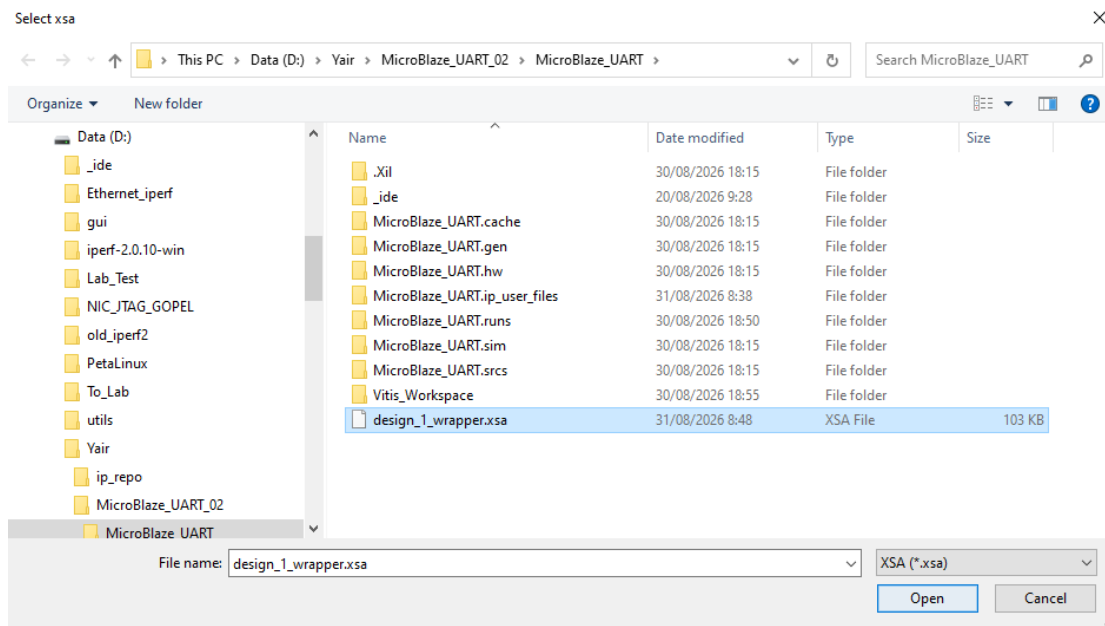
נלחץ Finish.



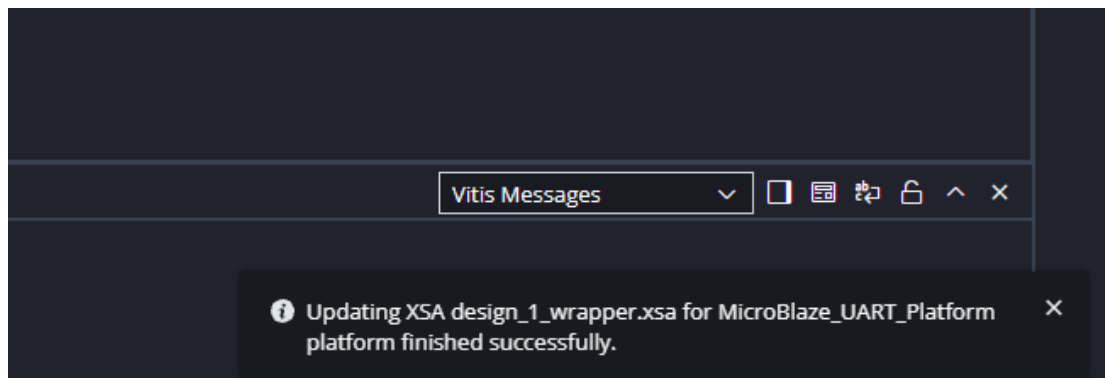
ב-Vitis נפתח את הגדרות הפלטפורמה ונעדכן את קובץ ה-XSA.



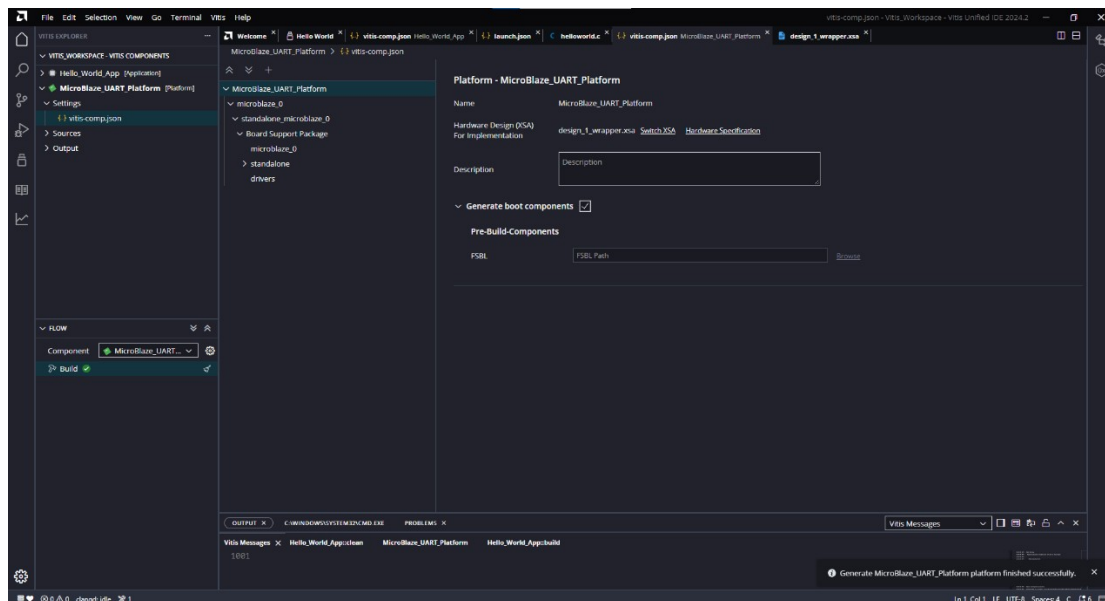
נבחר את קובץ ה-XSA החדש.



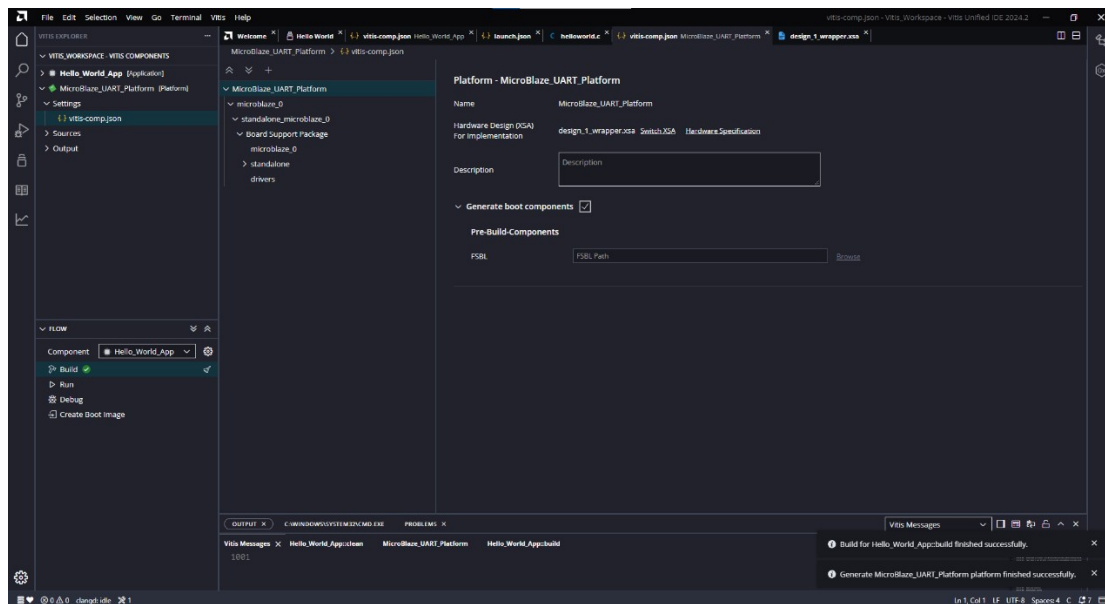
העדכון הסתיים בהצלחה.



נבנה את הפלטפורמה מחדש.

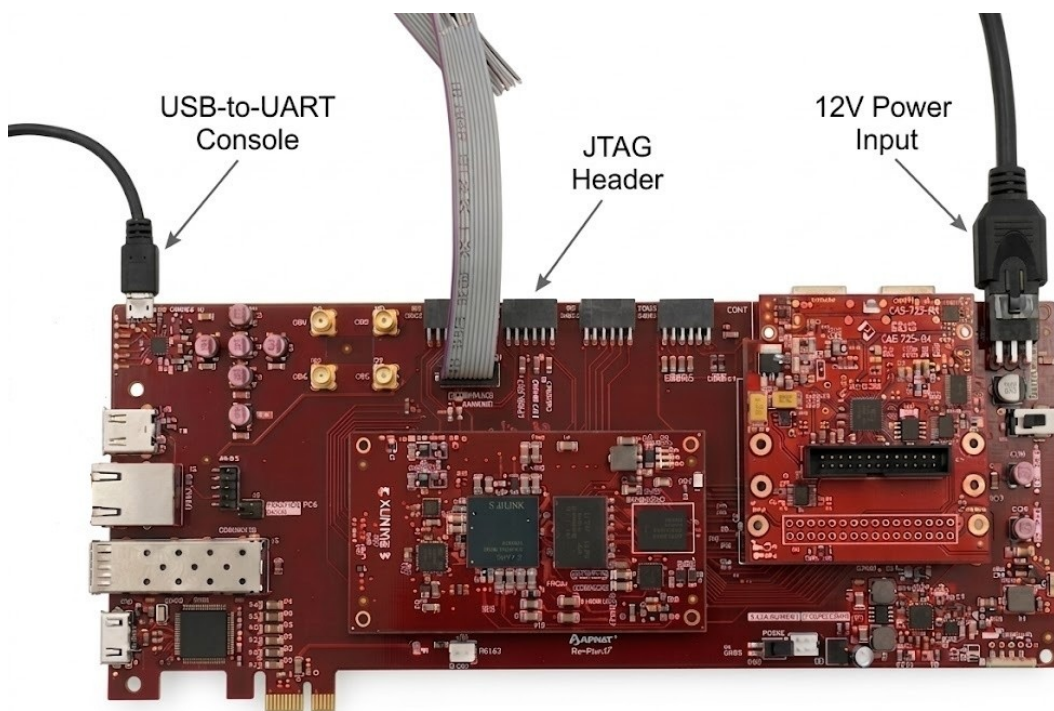


נבנה מחדש גם את האפליקציה.

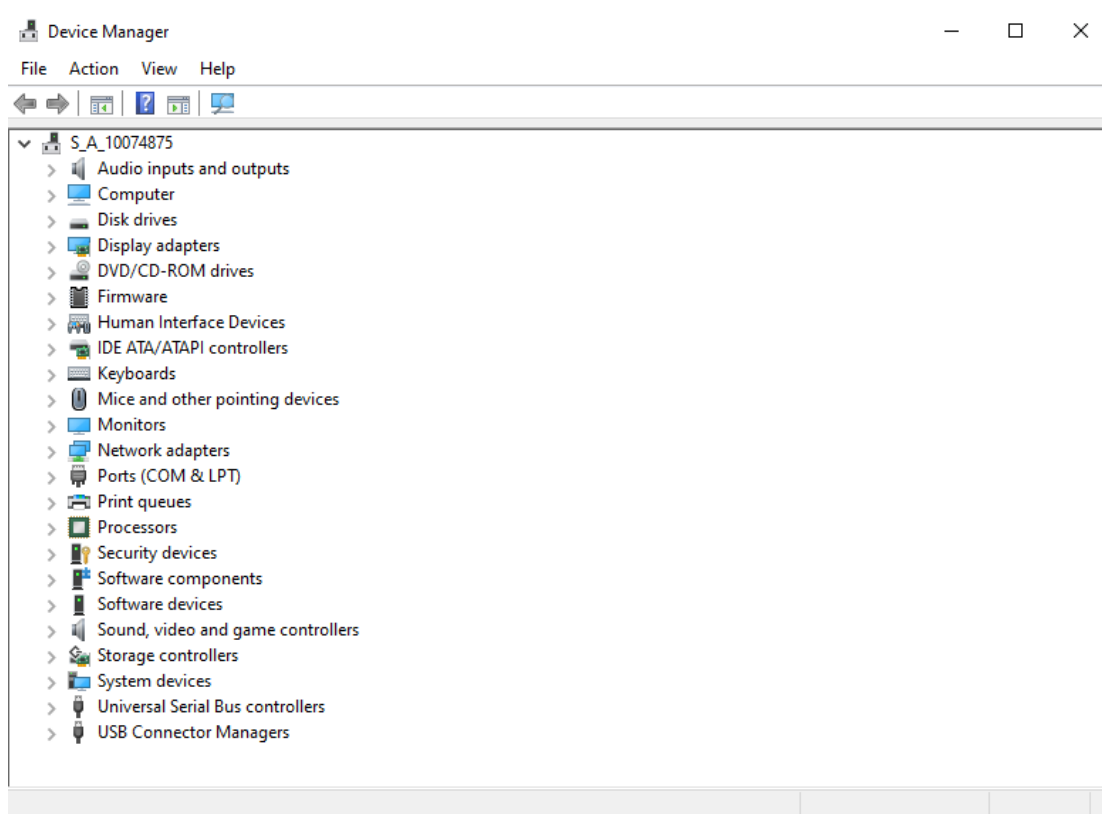


שלב 17:

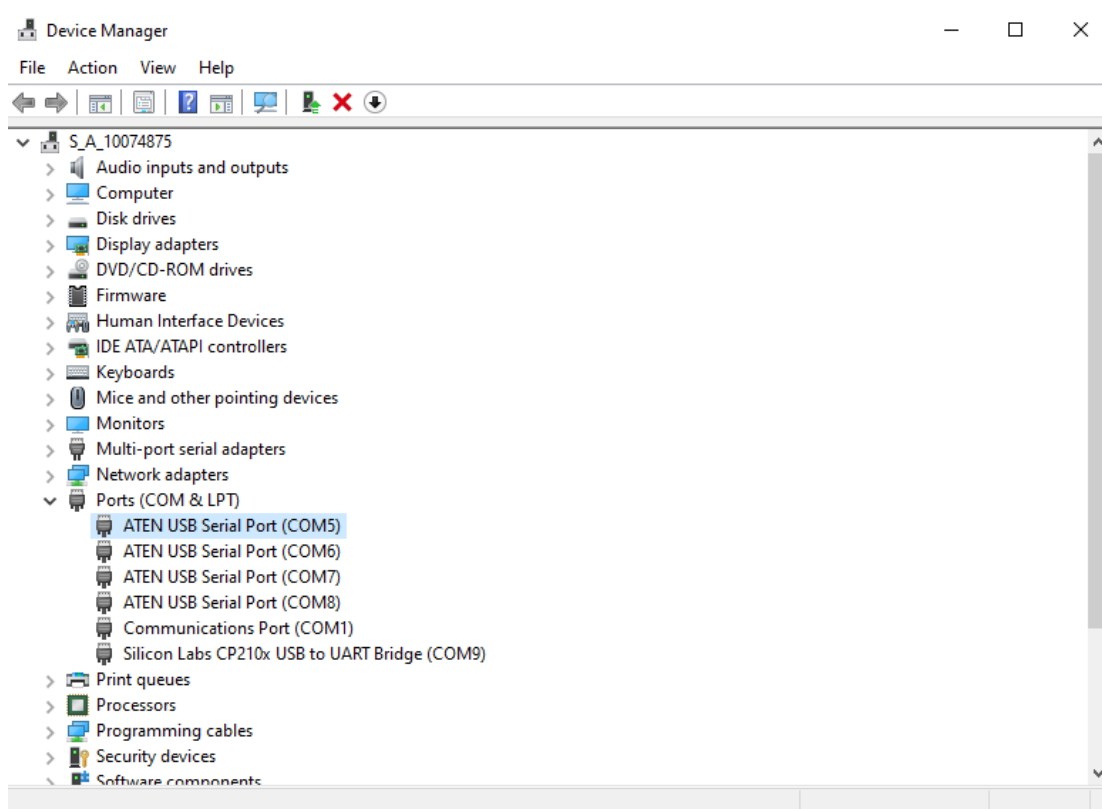
נחבר את הלוח למחשב באמצעות כבל ה-JTAG ומתאם ה-UART.



נפתח את מנהל ההתקנים של Windows.

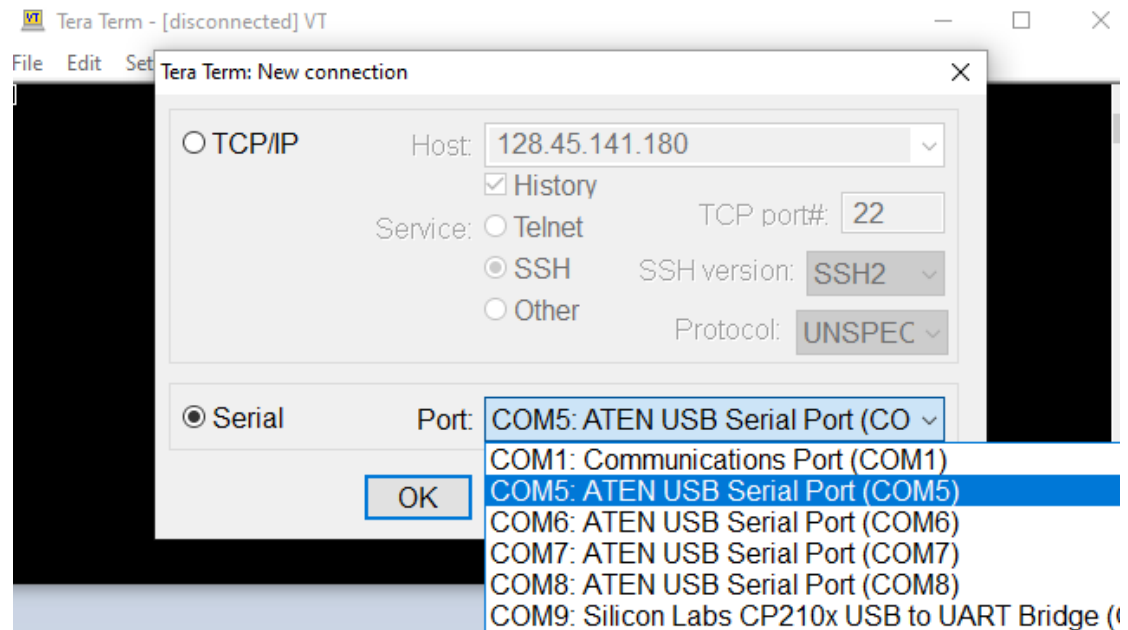


תחת Ports נזהה את יציאת ה-COM של מתאם ה-UART.

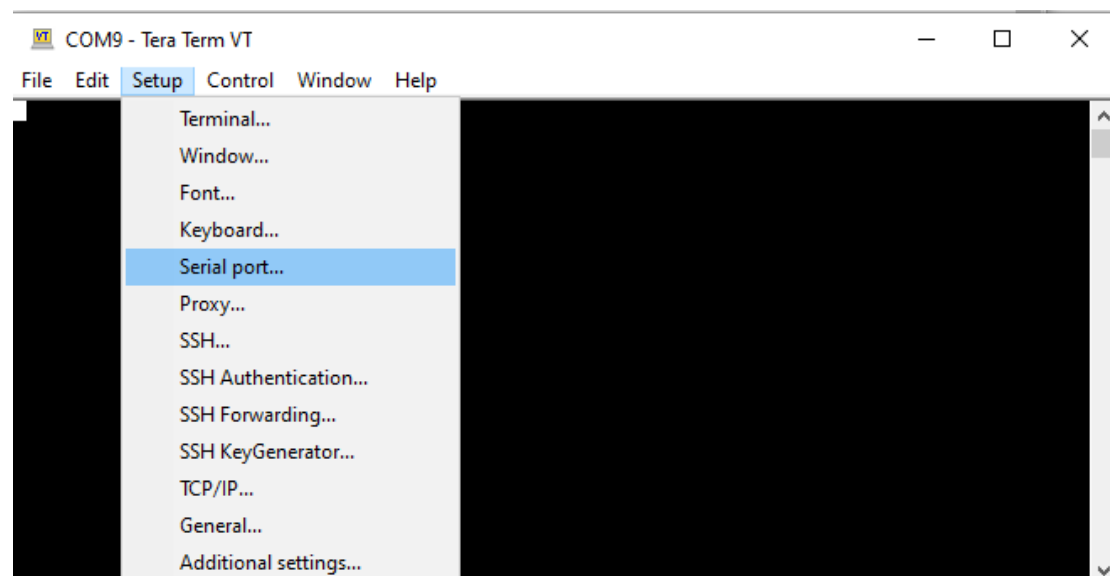


שלב 18:

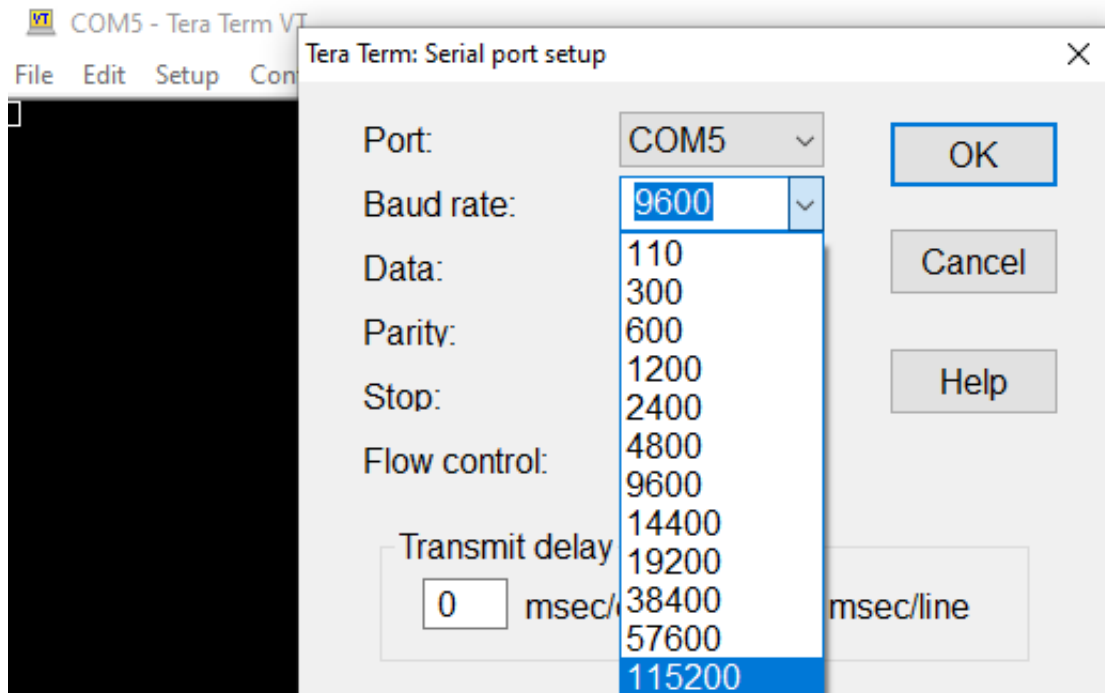
נפתח את Tera Term ובבחר חיבור Serial ביציאת ה-COM שזוהתה.



ניגש ל-Setup ובבחר Serial port.

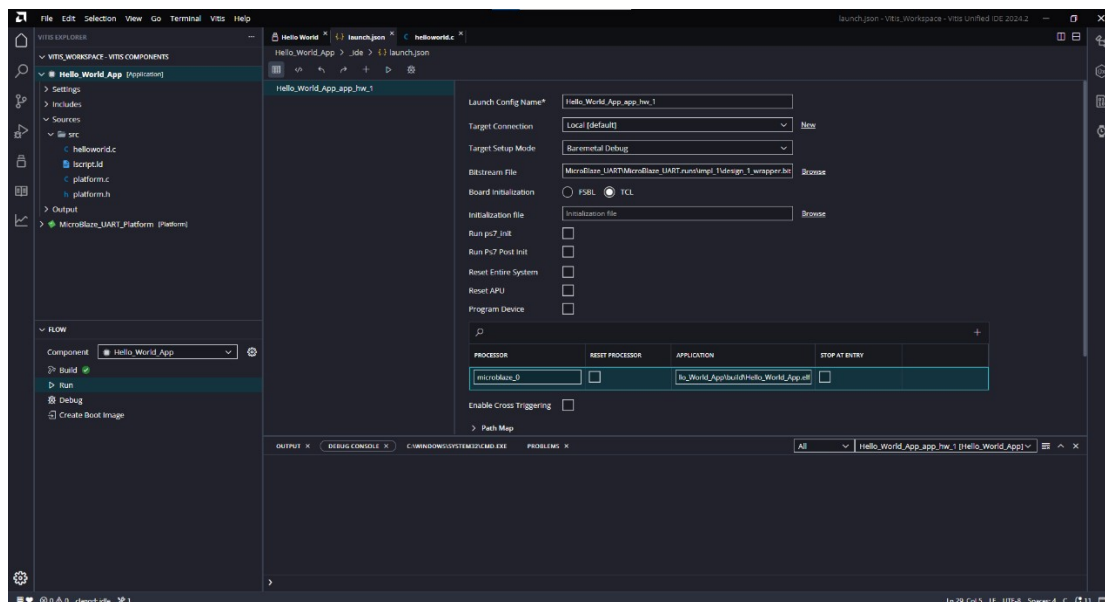


נגדיר קצב של 115200.

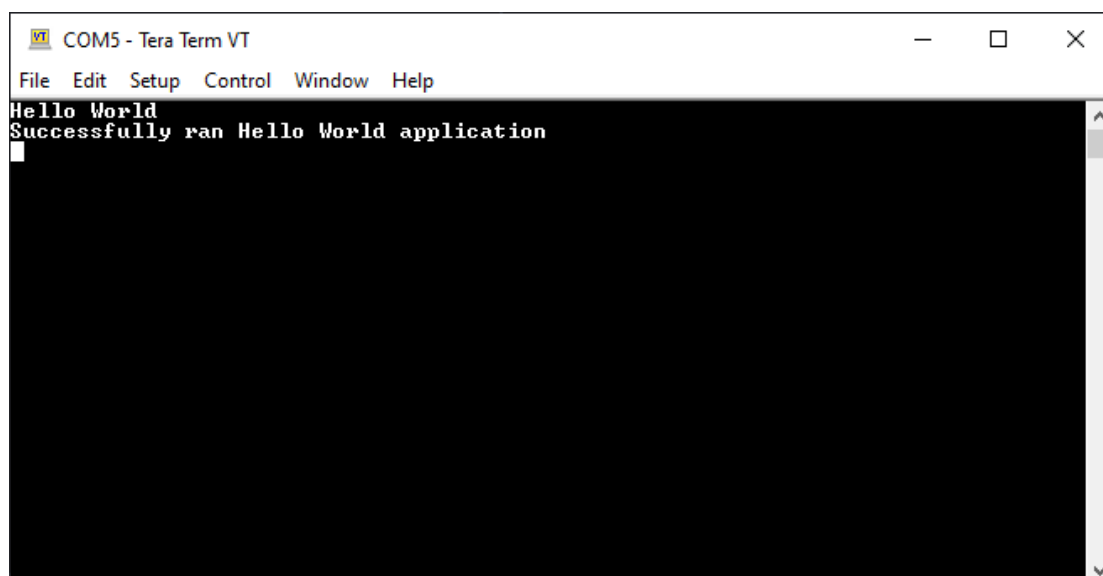


שלב 19:

בהגדרות ההרצה ב-Vitis נסיר את הסימון מ-Reset Processor ו-Entire System. אפשרויות אלו מיועדות לאתחול מעבד ה-PS של ה-Zynq, שאינו בשימוש בתכן זה. נריץ את האפליקציה.



בחלון ה-Tera Term מתקבלת הודעת ה-Hello World:



Chapter 3: Custom AXI4-Lite IP with a Read-Only Version Register on VCU118

מבוא:

בפרק זה נפתח רכיב IP מותאם בממשק AXI4-Lite השולט במצב הלדים. הרכיב כולל רגיסטר כתיבה לבחירת מצב התצוגה ורגיסטר לקריאה בלבד המחזיר את גרסת הרכיב. את הרכיב נבדוק בחומרה באמצעות JTAG to AXI Master ופקודות Tcl.

דרישות קדם:

חומרה:

Xilinx VCU118 לוח פיתוח

כבל USB-JTAG

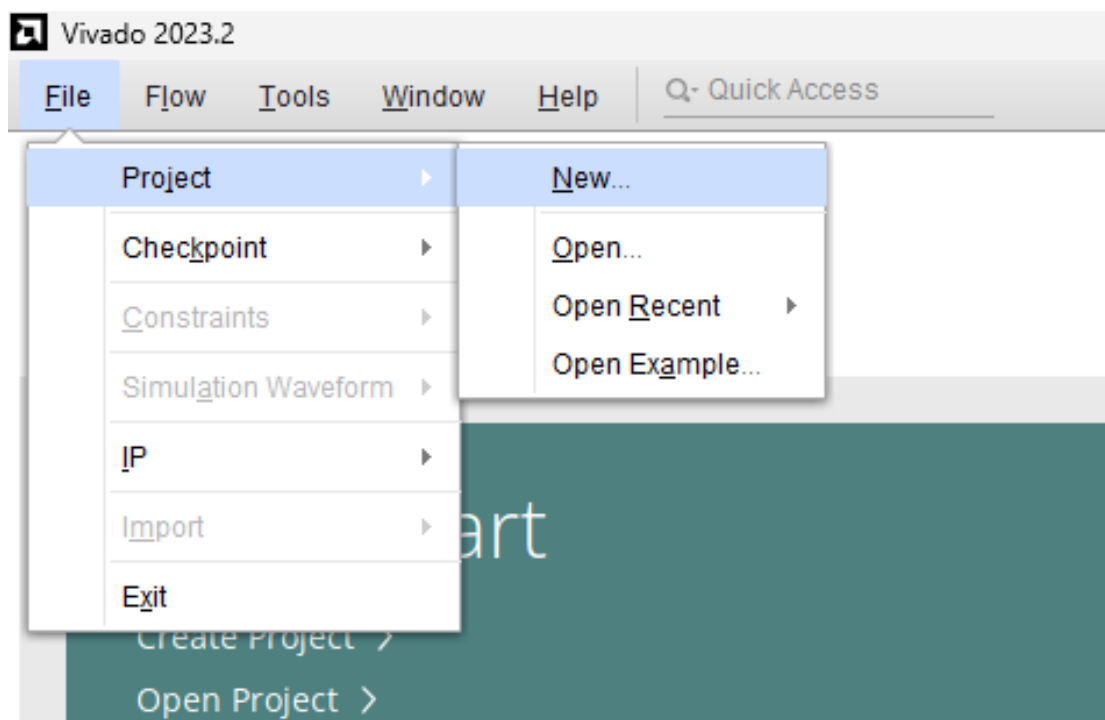
ספק כוח 12 וולט ללוח

תוכנה:

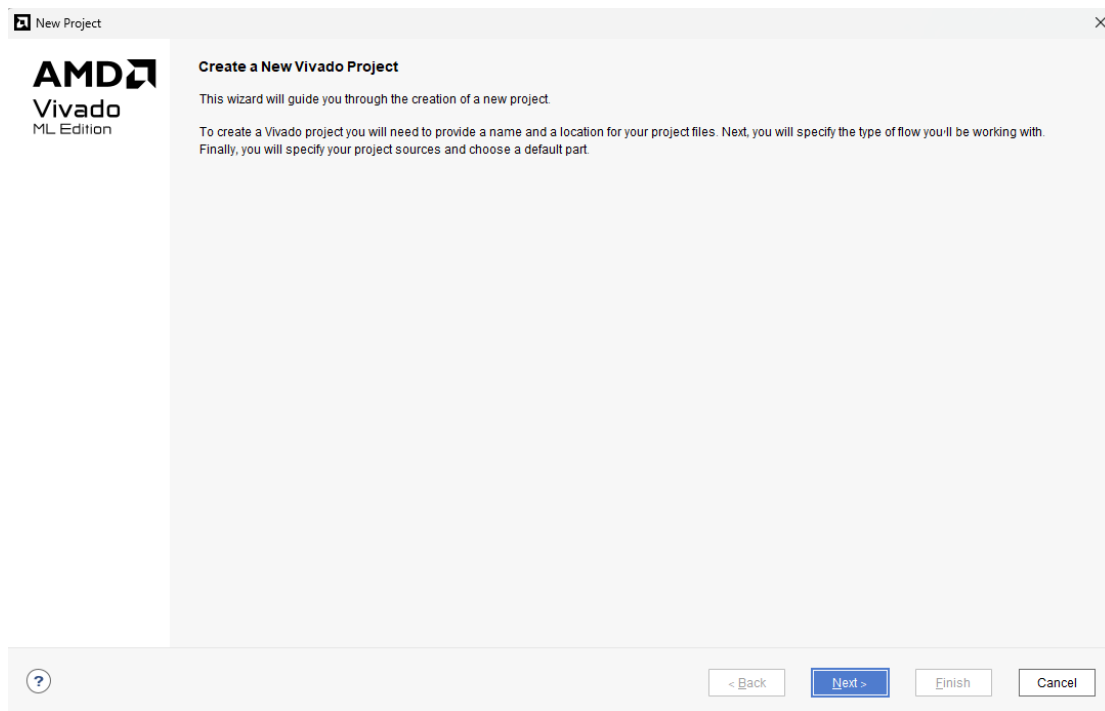
Vivado Design Suite

שלב 1:

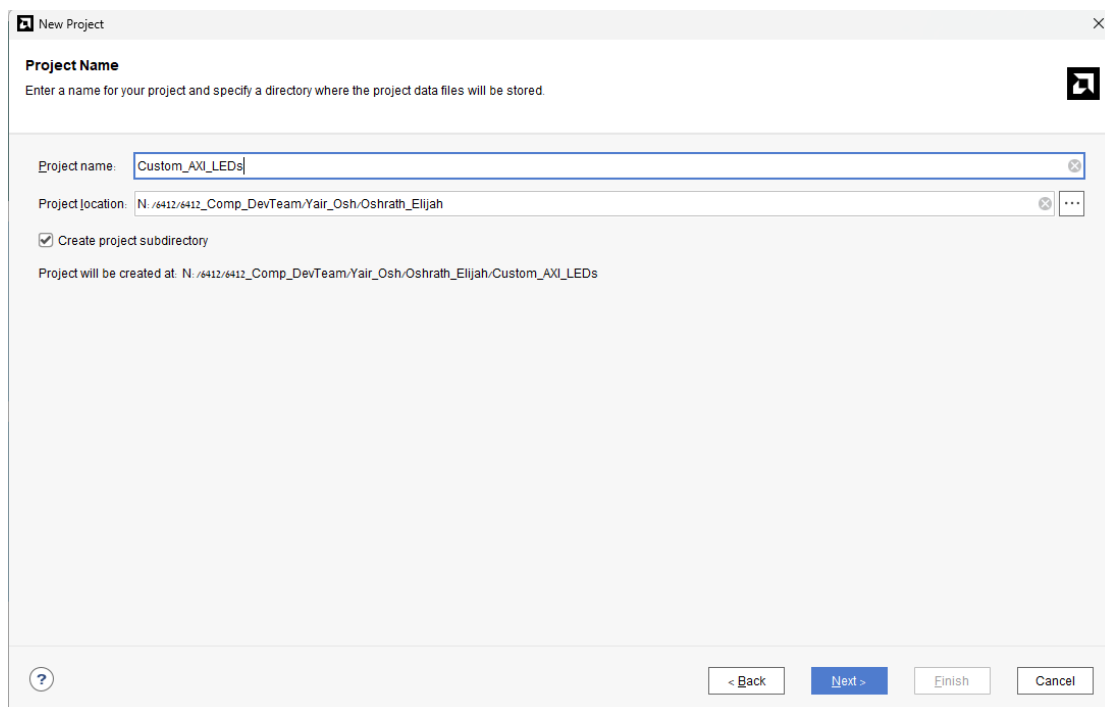
ניצור פרויקט חדש דרך File, Project ואז New.



נלחץ Next.



נקרא לפרויקט Custom_AXI_LEDs.



נבחר RTL Project.

New Project

Project Type
Specify the type of project to create.

☒ **RTL Project**
You will be able to add sources, create block designs in IP Integrator, generate IP, run RTL analysis, synthesis, implementation, design planning and analysis.
☒ Do not specify sources at this time
☐ Project is an extensible Vitis platform

☐ **Post-synthesis Project**
You will be able to add sources, view device resources, run design analysis, planning and implementation.
☐ Do not specify sources at this time

☐ **I/O Planning Project**
Do not specify design sources. You will be able to view part/package resources.

☐ **Imported Project**
Create a Vivado project from a Synplify Project File.

☐ **Example Project**
Create a new Vivado project from a predefined template.

[?](#) [< Back](#) [Next >](#) [Finish](#) [Cancel](#)

בלשונית Boards נבחר את לוח ה-VCU118.

New Project

Default Part
Choose a default AMD part or board for your project.

Parts | **Boards**

To fetch the latest available boards from git repository, click on 'Refresh' button. [Dismiss](#)

[Reset All Filters](#)

Vendor: Name: Board Rev:

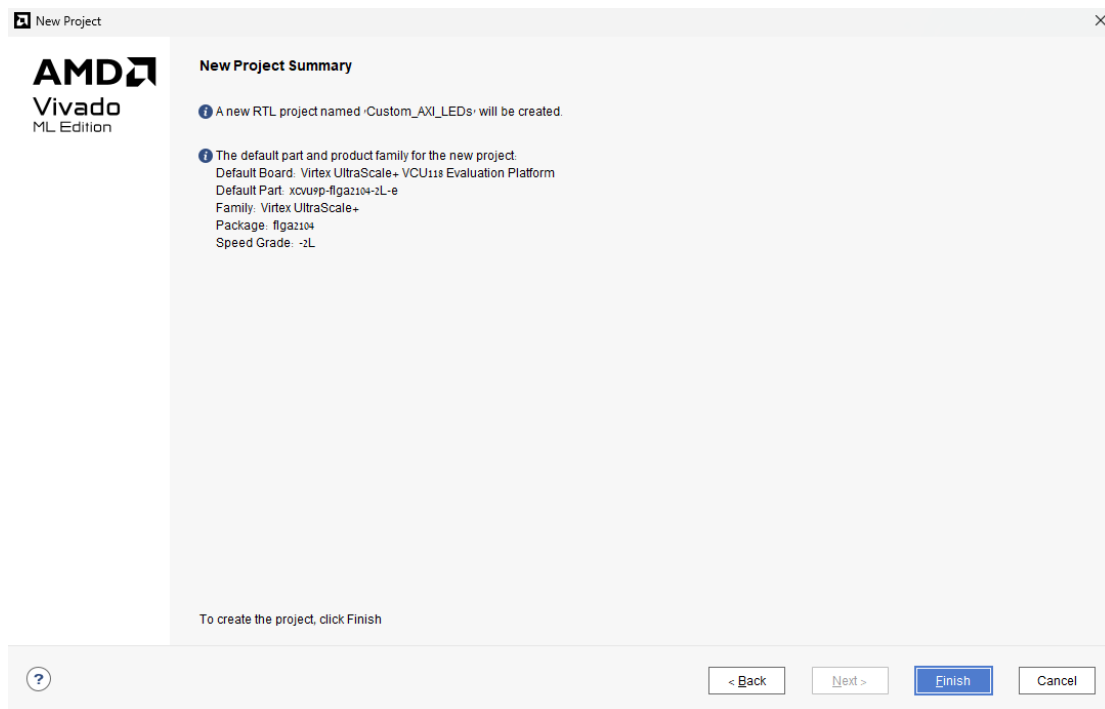
(1 match)

Display Name	Preview	Status	Vendor	File Version	Part	I/O Pin Count
Virtex UltraScale+ VCU118 Evaluation Platform		Installed	xilinx.com	2.4	xcvu9p-flga2104-zL-e	2104

[Refresh](#)

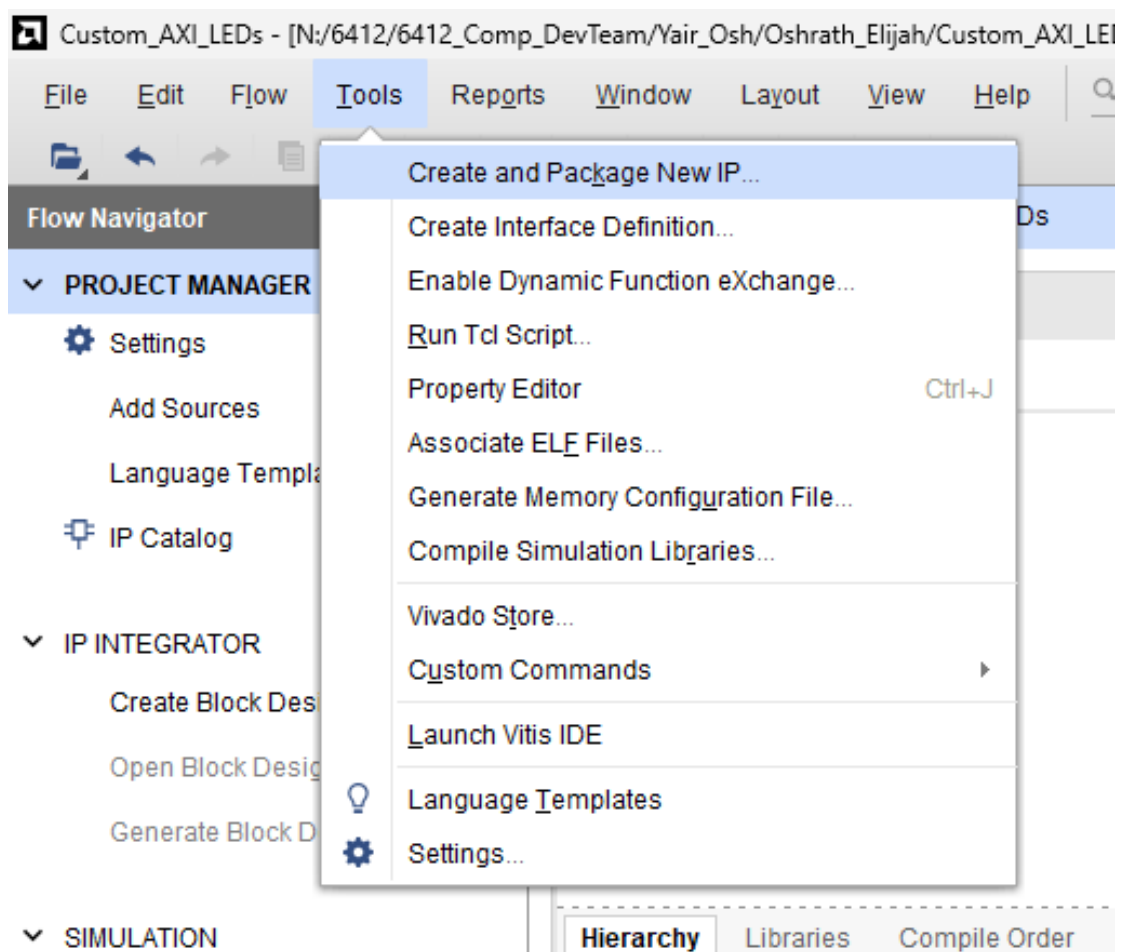
[?](#) [< Back](#) [Next >](#) [Finish](#) [Cancel](#)

נלחץ Finish.

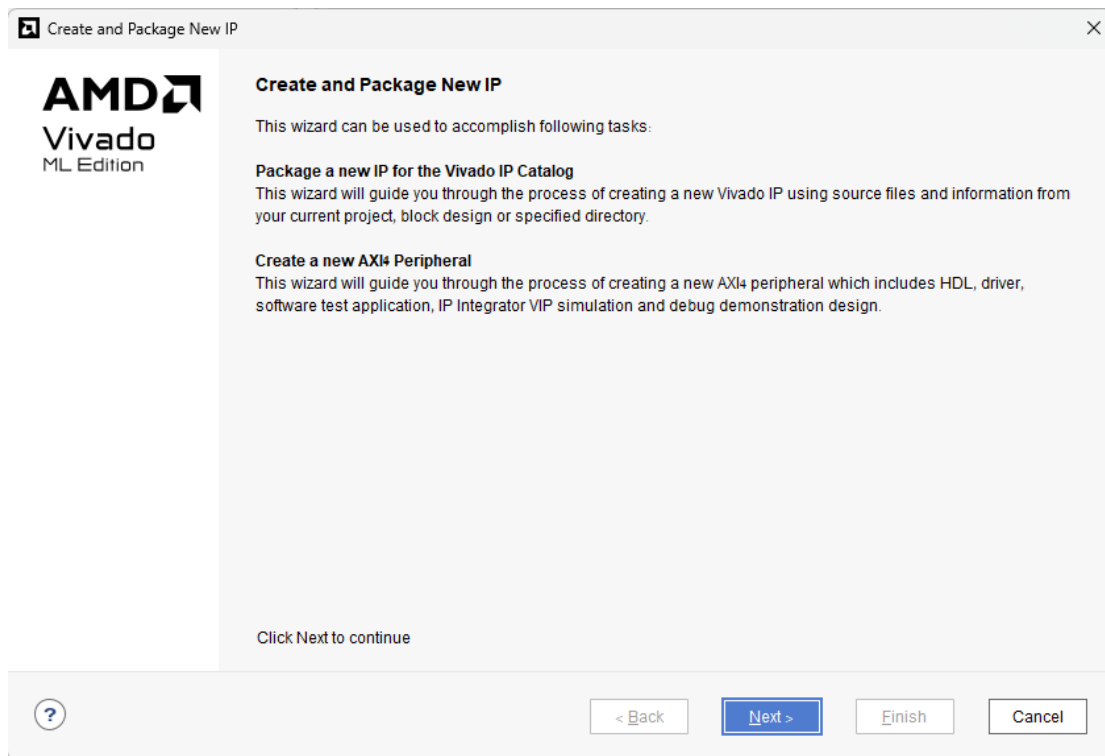


שלב 2:

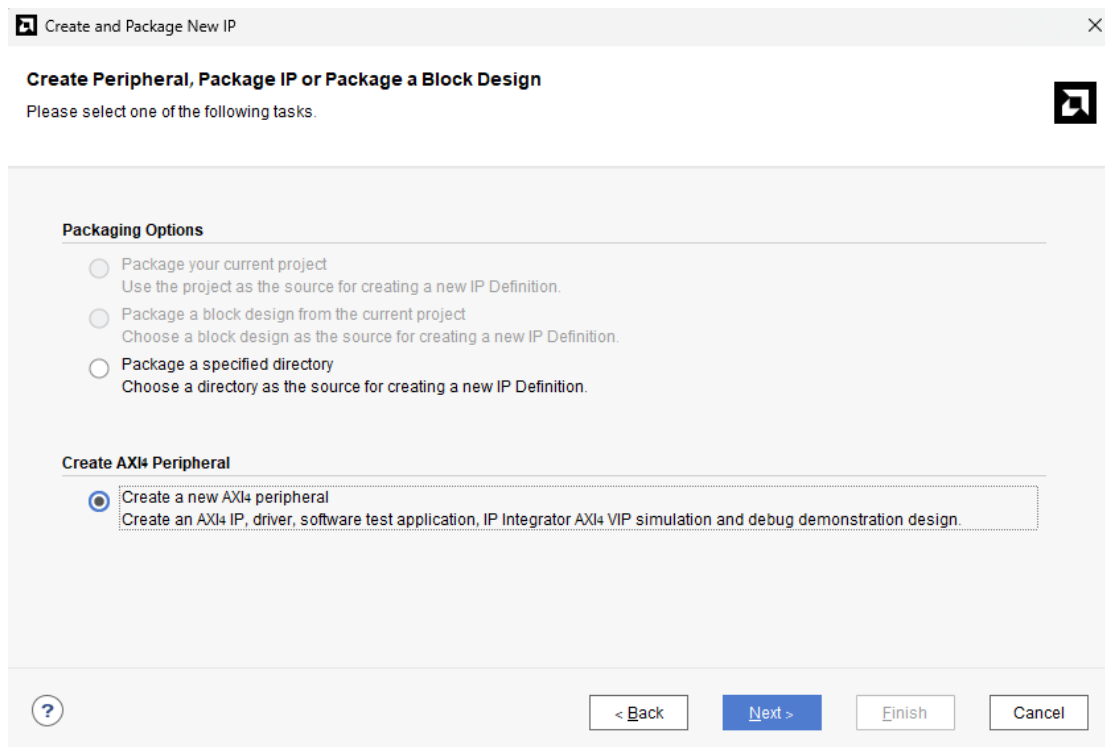
נבחר Tools ואז Create and Package New IP.



נלחץ Next.



נבחר Create a new AXI4 peripheral.



נקרא לרכיב S_AXI_LEDs.

Create and Package New IP

Peripheral Details
Specify name, version and description for the new peripheral

Name: S_AXI_LEDs

Version: 1.0

Display name: S_AXI_LEDs_v1.0

Description: New AXI IP that change the state of the leds by registers

IP location: N:/6412/6412_Comp_DevTeam/Yair_Osh/Oshrath_Elijah/Custom_AXI_LEDs/./ip_repo

☐ Overwrite existing

< Back Next > Finish Cancel

נגדיר ממשק AXI4-Lite מסוג Slave ברוחב נתונים של 32 ביט.

Create and Package New IP

Add Interfaces
Add AXI4 interfaces supported by your peripheral

☐ Enable Interrupt Support

Interfaces

- S00_AXI

S00_AXI
S_AXI_LEDs_v1.0

Name: S00_AXI

Interface Type: Lite

Interface Mode: Slave

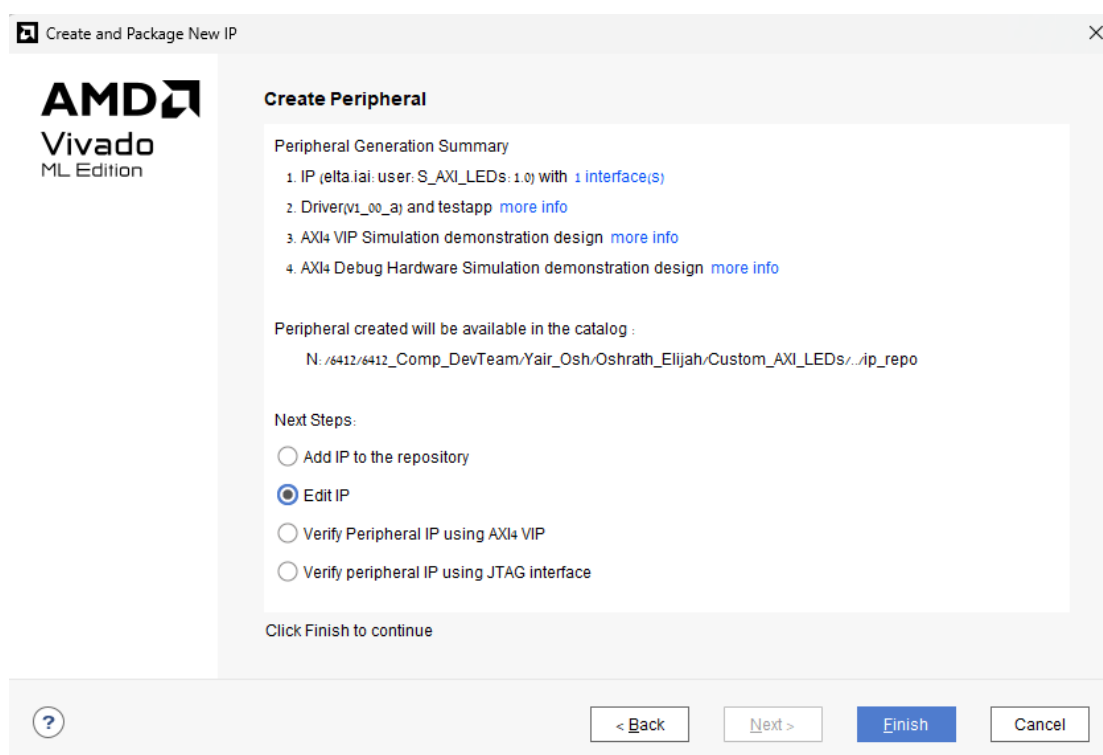
Data Width (Bits): 32

Memory Size (Bytes): 64

Number of Registers: 4 [4..512]

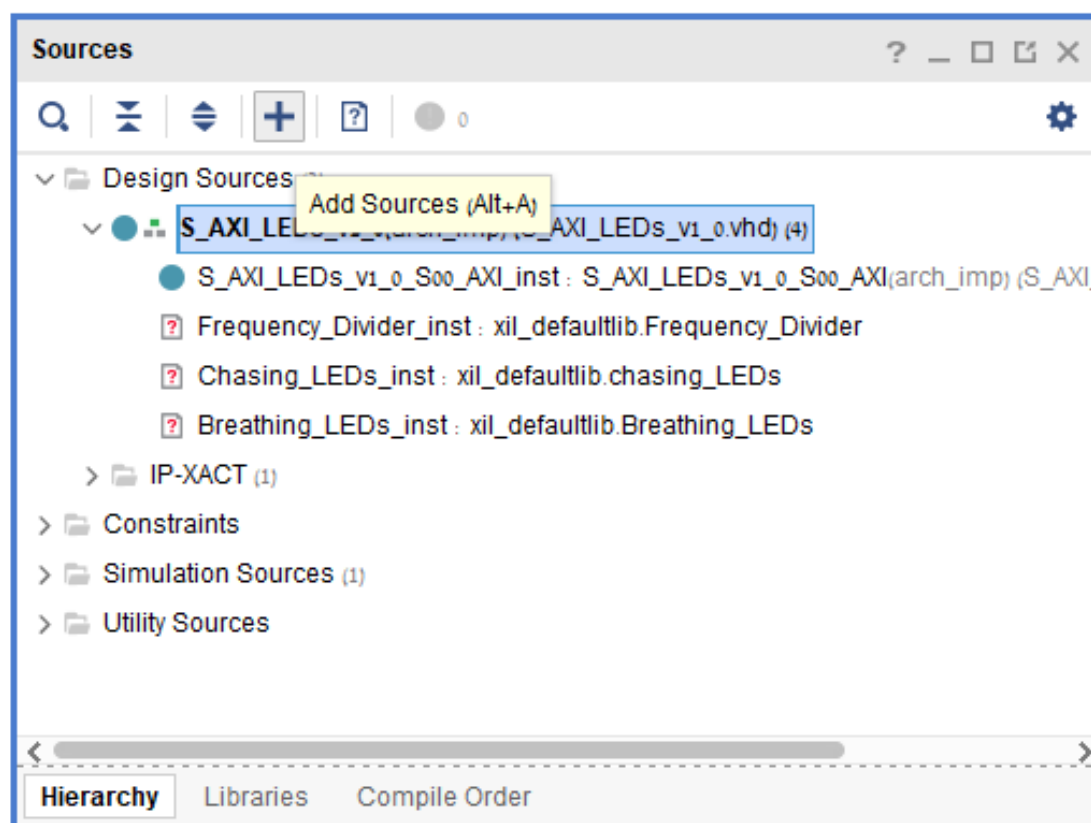
< Back Next > Finish Cancel

נבחר Edit IP ונלחץ Finish.

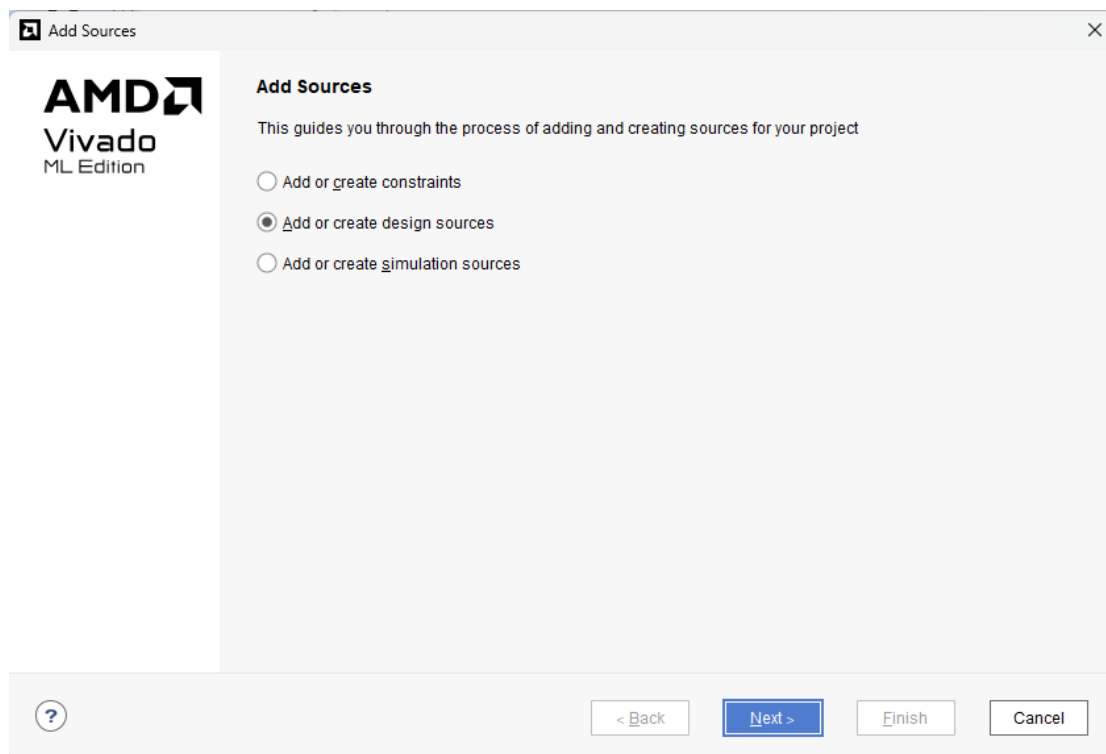


שלב 3:

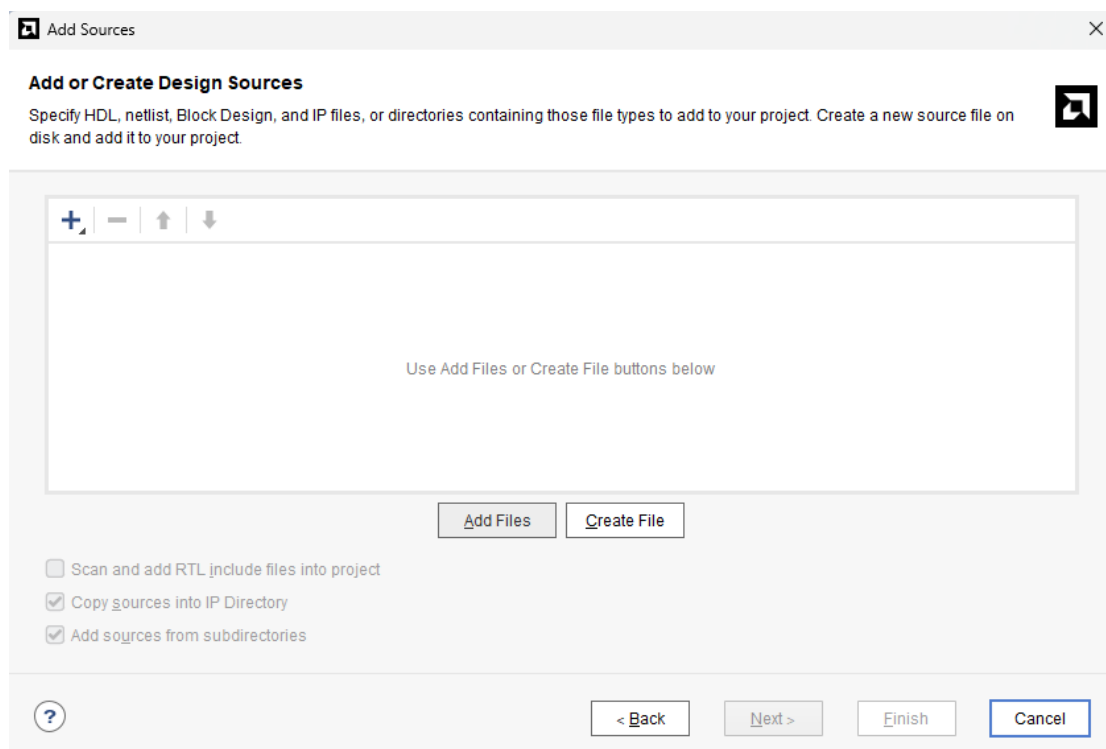
בפרויקט העריכה של ה-IP נלחץ Add Sources.



נבחר Add or create design sources.



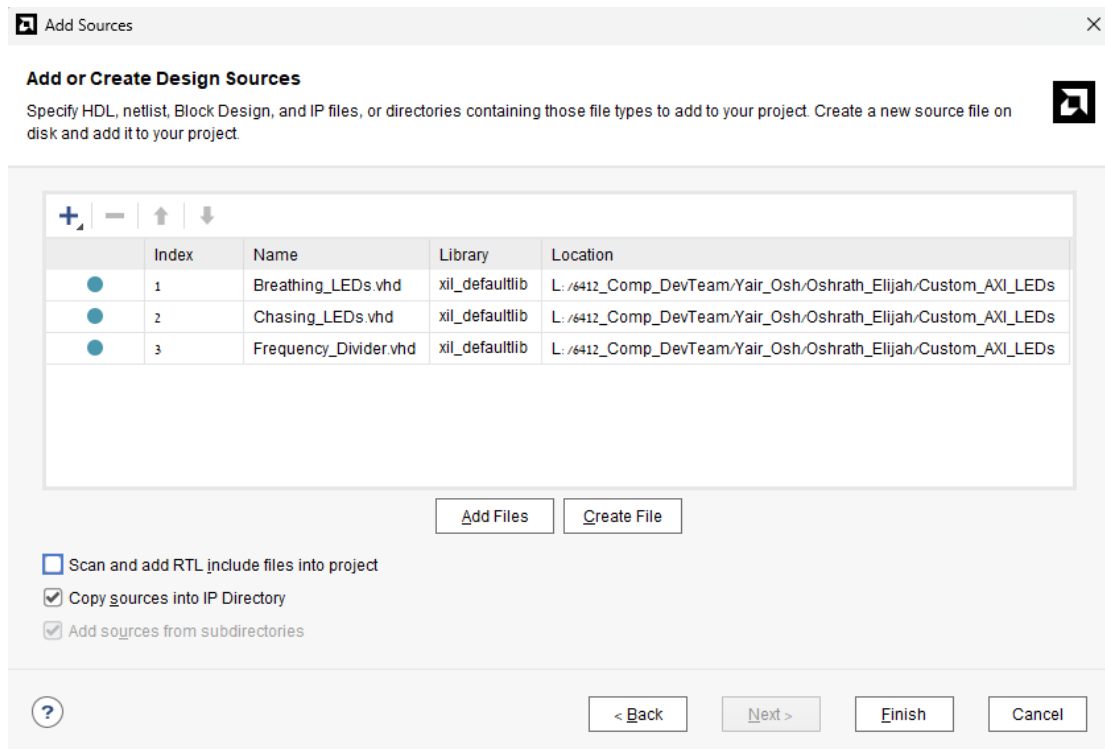
נלחץ Add Files.



נבחר את הקבצים Breathing_LEDs ו-Frequency_Divider, Chasing_LEDs.



נסמן העתקה לתיקיית ה-Finish ונלחץ Finish.



שלב 4:

נפתח את הקובץ העליון S_AXI_LEDs_v1_0.vhd ונוסיף generic בשם IP_LEDs_Version עם הערך "20251123".

```
S_AXI_LEDs_v1_0.vhd
n:/6412/6412_Comp_DevTeam/Yair_Osh/Oshrath_Elijah/ip_repo/S_AXI_LEDs_1_0/hdl/S_AXI_LEDs_v1_0.vhd

1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.numeric_std.all;
4
5  entity S_AXI_LEDs_v1_0 is
6      generic (
7          -- Users to add parameters here
8          IP_LEDs_Version      : std_logic_vector(31 downto 0) := x"20251123";
9          -- User parameters ends
```

פרמטרי ה-AXI נשארים ברוחב 32 ביט.

```
10      -- Do not modify the parameters beyond this line
11
12
13      -- Parameters of Axi Slave Bus Interface S00_AXI
14      C_S00_AXI_DATA_WIDTH    : integer    := 32;
15      C_S00_AXI_ADDR_WIDTH    : integer    := 32
16  );
```

נוסיף יציאה LED_o ברוחב 8 ביטים.

```
17      port (
18          -- Users to add ports here
19          LED_o      : out std_logic_vector(7 downto 0);
20          -- User ports ends
21          -- Do not modify the ports beyond this line
22
23
24      -- Ports of Axi Slave Bus Interface S00_AXI
25      s00_axi_aclk    : in std_logic;
26      s00_axi_aresetn : in std_logic;
27      s00_axi_awaddr   : in std_logic_vector(C_S00_AXI_ADDR_WIDTH-1 downto 0);
28      s00_axi_awprot   : in std_logic_vector(2 downto 0);
29      s00_axi_awvalid  : in std_logic;
30      s00_axi_awready  : out std_logic;
31      s00_axi_wdata    : in std_logic_vector(C_S00_AXI_DATA_WIDTH-1 downto 0);
32      s00_axi_wstrb    : in std_logic_vector((C_S00_AXI_DATA_WIDTH/8)-1 downto 0);
33      s00_axi_wvalid   : in std_logic;
34      s00_axi_wready   : out std_logic;
35      s00_axi_bresp    : out std_logic_vector(1 downto 0);
36      s00_axi_bvalid   : out std_logic;
37      s00_axi_bready   : in std_logic;
38      s00_axi_araddr   : in std_logic_vector(C_S00_AXI_ADDR_WIDTH-1 downto 0);
39      s00_axi_arprot   : in std_logic_vector(2 downto 0);
40      s00_axi_arvalid  : in std_logic;
41      s00_axi_arready  : out std_logic;
42      s00_axi_rdata    : out std_logic_vector(C_S00_AXI_DATA_WIDTH-1 downto 0);
43      s00_axi_rresp    : out std_logic_vector(1 downto 0);
44      s00_axi_rvalid   : out std_logic;
45      s00_axi_rready   : in std_logic;
46  );
47  end S_AXI_LEDs_v1_0;
```

בהצהרת הרכיב S00_AXI נוסיף את ה-generic של הגרסה.

```
48 :  
49 architecture arch_imp of S_AXI_LEDs_v1_0 is  
50 :  
51 -- component declaration  
52 component S_AXI_LEDs_v1_0_S00_AXI is  
53 generic (  
54 IP_LEDs_Version : std_logic_vector(31 downto 0) := x"20251108";
```

פרמטרי הרוחב של הרכיב נשארים ללא שינוי.

```
55 C_S_AXI_DATA_WIDTH : integer := 32;  
56 C_S_AXI_ADDR_WIDTH : integer := 32  
57 );
```

נוסיף לרכיב את היציאה LEDs_mode_o.

```
58 port (  
59 LEDs_mode_o : out std_logic_vector(1 downto 0);  
60 S_AXI_ACLK : in std_logic;  
61 S_AXI_ARESETN : in std_logic;  
62 S_AXI_AWADDR : in std_logic_vector(C_S_AXI_ADDR_WIDTH-1 downto 0);  
63 S_AXI_AWPROT : in std_logic_vector(2 downto 0);  
64 S_AXI_AWVALID : in std_logic;  
65 S_AXI_AWREADY : out std_logic;  
66 S_AXI_WDATA : in std_logic_vector(C_S_AXI_DATA_WIDTH-1 downto 0);  
67 S_AXI_WSTRB : in std_logic_vector((C_S_AXI_DATA_WIDTH/8)-1 downto 0);  
68 S_AXI_WVALID : in std_logic;  
69 S_AXI_WREADY : out std_logic;  
70 S_AXI_BRESP : out std_logic_vector(1 downto 0);  
71 S_AXI_BVALID : out std_logic;  
72 S_AXI_BREADY : in std_logic;  
73 S_AXI_ARADDR : in std_logic_vector(C_S_AXI_ADDR_WIDTH-1 downto 0);  
74 S_AXI_ARPROT : in std_logic_vector(2 downto 0);  
75 S_AXI_ARVALID : in std_logic;  
76 S_AXI_ARREADY : out std_logic;  
77 S_AXI_RDATA : out std_logic_vector(C_S_AXI_DATA_WIDTH-1 downto 0);  
78 S_AXI_RRESP : out std_logic_vector(1 downto 0);  
79 S_AXI_RVALID : out std_logic;  
80 S_AXI_RREADY : in std_logic  
81 );  
82 end component S_AXI_LEDs_v1_0_S00_AXI;  
83  
84 begin
```

נכריז על הרכיב Chasing_LEDs.

```
84 --Component of ChasingLEDs  
85 component chasing_LEDs is  
86 port(  
87 clk,rst,pulse_i: in std_logic;  
88 LED_o : out std_logic_vector(7 downto 0)  
89 );  
90 end component;
```

נכריז על הרכיב Breathing_LEDs.

```
92 --Component of BreathingLEDs  
93 component Breathing_LEDs is  
94 port(  
95 clk : in std_logic;  
96 rst : in std_logic;  
97 pulse_i : in std_logic;  
98 LED_o : out std_logic_vector(7 downto 0)  
99 );  
100 end component;
```

נכריז על הרכיב Frequency_Divider.

```
102 -- Component of Frequency_Divider
103 component Frequency_Divider is
104   generic(
105     input_freq : integer := 100000000;
106     output_freq: integer := 1
107   );
108   port(
109     clk_in : in std_logic;
110     rst    : in std_logic;
111     pulse_o: out std_logic
112   );
113 end component;
```

נגדיר את האותות הפנימיים.

```
114 --signals
115
116 signal pulse_1HZ_o      : std_logic;
117 signal output_chasing   : std_logic_vector(7 downto 0);
118 signal output_Breathing : std_logic_vector(7 downto 0);
119 signal rst_active_high  : std_logic;
120 signal LEDs_mode_i      : std_logic_vector(1 downto 0);
121
122
123
124
125
```

במיקום רכיב ה-AXI נעביר את ה-generic של הגרסה.

```
127
128 -- Instantiation of Axi Bus Interface S00_AXI
129 S_AXI_LEDs_v1_0_S00_AXI_inst : S_AXI_LEDs_v1_0_S00_AXI
130   generic map (
131     IP_LEDs_Version => IP_LEDs_Version,
132     C_S_AXI_DATA_WIDTH => C_S00_AXI_DATA_WIDTH,
133     C_S_AXI_ADDR_WIDTH => C_S00_AXI_ADDR_WIDTH
134   )
```

נחבר את היציאה LEDs_mode_o לאות הפנימי.

```
135   port map (
136     LEDs_mode_o => LEDs_mode_i,
137     S_AXI_ACLK => s00_axi_aclk,
138     S_AXI_ARESETN => s00_axi_aresetn,
139     S_AXI_AWADDR => s00_axi_awaddr,
140     S_AXI_AWPROT => s00_axi_awprot,
141     S_AXI_AWVALID => s00_axi_awvalid,
142     S_AXI_AWREADY => s00_axi_awready,
143     S_AXI_WDATA => s00_axi_wdata,
144     S_AXI_WSTRB => s00_axi_wstrb,
145     S_AXI_WVALID => s00_axi_wvalid,
146     S_AXI_WREADY => s00_axi_wready,
147     S_AXI_BRESP => s00_axi_bresp,
148     S_AXI_BVALID => s00_axi_bvalid,
149     S_AXI_BREADY => s00_axi_bready,
150     S_AXI_ARADDR => s00_axi_araddr,
151     S_AXI_ARPROT => s00_axi_arprot,
152     S_AXI_ARVALID => s00_axi_arvalid,
153     S_AXI_ARREADY => s00_axi_arready,
154     S_AXI_RDATA => s00_axi_rdata,
155     S_AXI_RRESP => s00_axi_rresp,
156     S_AXI_RVALID => s00_axi_rvalid,
157     S_AXI_RREADY => s00_axi_rready
158   );
159
```

נמקם את מחלק התדר.

```
160 -- Add user logic here
161 rst_active_high <= not s00_axi_aresetn;
162 Frequency_Divider_inst : Frequency_Divider
163     port map(
164         clk_in => s00_axi_aclk,
165         rst     => rst_active_high,
166         pulse_o=> pulse_1HZ_o
167     );
168
```

נמקם את Chasing_LEDs.

```
169 Chasing_LEDs_inst : Chasing_LEDs
170     port map(
171         clk     => s00_axi_aclk,
172         rst     => rst_active_high,
173         pulse_i=> pulse_1HZ_o,
174         LED_o  => output_chasing
175     );
176
```

נמקם את Breathing_LEDs ונגדיר את הבורר. LED_o מקבל את מצב הריצה כאשר LEDs_mode שווה 00 ואת מצב הנשימה כאשר הוא שווה 01.

```
177 Breathing_LEDs_inst : Breathing_LEDs
178     port map(
179         clk     => s00_axi_aclk,
180         rst     => rst_active_high,
181         pulse_i=> pulse_1HZ_o,
182         LED_o  => output_Breathing
183     );
184
185 LED_o <= output_chasing when LEDs_mode_i="00" else
186         output_Breathing when LEDs_mode_i="01" else
187         (others => '0');
188
189 -- User logic ends
190
191 end arch_imp;
192
```

שלב 5:

נפתח את הקובץ S_AXI_LEDs_v1_0_S00_AXI.vhd ונוסיף את ה-generic של הגרסה.

```
S_AXI_LEDs_v1_0_S00_AXI.vhd
n:/6412/6412_Comp_DevTeam/Yair_Osh/Oshrath_Elijah/ip_repo/S_AXI_LEDs_1_0/hdl/S_AXI_LEDs_v1_0_S00_AXI.vhd

1 library ieee;
2 use ieee.std_logic_1164.all;
3 use ieee.numeric_std.all;
4
5 entity S_AXI_LEDs_v1_0_S00_AXI is
6     generic (
7         -- Users to add parameters here
8         IP_LEDs_Version : std_logic_vector(31 downto 0) := x"20251123";
9         -- User parameters ends
10        -- Do not modify the parameters beyond this line
11    );
12
```

פרמטרי הרוחב נשארים ללא שינוי.

```
12 -- Width of S_AXI data bus
13 C_S_AXI_DATA_WIDTH : integer := 32;
14 -- Width of S_AXI address bus
15 C_S_AXI_ADDR_WIDTH : integer := 32
16 );
```

נוסיף יציאה LEDs_mode_o.

```
17 port (
18     -- Users to add ports here
19     LEDs_mode_o : out std_logic_vector(1 downto 0);
20     -- User ports ends
21     -- Do not modify the ports beyond this line
22 );
```

פורטי ממשק ה-AXI נשארים ללא שינוי.

```
23 -- Global Clock Signal
24 S_AXI_ACLK : in std_logic;
25 -- Global Reset Signal. This Signal is Active LOW
26 S_AXI_ARESETN : in std_logic;
27 -- Write address (issued by master, accepted by Slave)
28 S_AXI_AWADDR : in std_logic_vector(C_S_AXI_ADDR_WIDTH-1 downto 0);
29 -- Write channel Protection type. This signal indicates the
30 -- privilege and security level of the transaction, and whether
31 -- the transaction is a data access or an instruction access.
32 S_AXI_AWPROT : in std_logic_vector(2 downto 0);
33 -- Write address valid. This signal indicates that the master signaling
34 -- valid write address and control information.
35 S_AXI_AWVALID : in std_logic;
36 -- Write address ready. This signal indicates that the slave is ready
37 -- to accept an address and associated control signals.
38 S_AXI_AWREADY : out std_logic;
39 -- Write data (issued by master, accepted by Slave)
40 S_AXI_WDATA : in std_logic_vector(C_S_AXI_DATA_WIDTH-1 downto 0);
41 -- Write strobes. This signal indicates which byte lanes hold
42 -- valid data. There is one write strobe bit for each eight
43 -- bits of the write data bus.
44 S_AXI_WSTRB : in std_logic_vector((C_S_AXI_DATA_WIDTH/8)-1 downto 0);
45 -- Write valid. This signal indicates that valid write
46 -- data and strobes are available.
47 S_AXI_WVALID : in std_logic;
48 -- Write ready. This signal indicates that the slave
49 -- can accept the write data.
50 S_AXI_WREADY : out std_logic;
51 -- Write response. This signal indicates the status
52 -- of the write transaction.
53 S_AXI_BRESP : out std_logic_vector(1 downto 0);
54 -- Write response valid. This signal indicates that the channel
55 -- is signaling a valid write response.
56 S_AXI_BVALID : out std_logic;
57 -- Response ready. This signal indicates that the master
58 -- can accept a write response.
59 S_AXI_BREADY : in std_logic;
60 -- Read address (issued by master, accepted by Slave)
61 S_AXI_ARADDR : in std_logic_vector(C_S_AXI_ADDR_WIDTH-1 downto 0);
62 -- Protection type. This signal indicates the privilege
63 -- and security level of the transaction, and whether the
64 -- transaction is a data access or an instruction access.
65 S_AXI_ARPROT : in std_logic_vector(2 downto 0);
66 -- Read address valid. This signal indicates that the channel
67 -- is signaling valid read address and control information.
68 S_AXI_ARVALID : in std_logic;
69 -- Read address ready. This signal indicates that the slave is
70 -- ready to accept an address and associated control signals.
71 S_AXI_ARREADY : out std_logic;
```

נמשיך לאותות הפנימיים של הרכיב.

```

72 :      -- Read data (issued by slave)
73 :      S_AXI_RDATA : out std_logic_vector(C_S_AXI_DATA_WIDTH-1 downto 0);
74 :      -- Read response. This signal indicates the status of the
75 :      -- read transfer.
76 :      S_AXI_RRESP : out std_logic_vector(1 downto 0);
77 :      -- Read valid. This signal indicates that the channel is
78 :      -- signaling the required read data.
79 :      S_AXI_RVALID : out std_logic;
80 :      -- Read ready. This signal indicates that the master can
81 :      -- accept the read data and response information.
82 :      S_AXI_RREADY : in std_logic
83 :    );
84 : end S_AXI_LEDS_v1_0_S00_AXI;
85 :
86 : architecture arch_imp of S_AXI_LEDS_v1_0_S00_AXI is
87 :
88 :     -- AXI4LITE signals
89 :     signal axi_awaddr : std_logic_vector(C_S_AXI_ADDR_WIDTH-1 downto 0);
90 :     signal axi_awready : std_logic;
91 :     signal axi_wready : std_logic;
92 :     signal axi_bresp : std_logic_vector(1 downto 0);
93 :     signal axi_bvalid : std_logic;
94 :     signal axi_araddr : std_logic_vector(C_S_AXI_ADDR_WIDTH-1 downto 0);
95 :     signal axi_arready : std_logic;
96 :     signal axi_rdata : std_logic_vector(C_S_AXI_DATA_WIDTH-1 downto 0);
97 :     signal axi_rresp : std_logic_vector(1 downto 0);
98 :     signal axi_rvalid : std_logic;
99 :
100 :    -- Example-specific design signals
101 :    -- local parameter for addressing 32 bit / 64 bit C_S_AXI_DATA_WIDTH
102 :    -- ADDR_LSB is used for addressing 32/64 bit registers/memories
103 :    -- ADDR_LSB = 2 for 32 bits (n downto 2)
104 :    -- ADDR_LSB = 3 for 64 bits (n downto 3)
105 :    constant ADDR_LSB : integer := (C_S_AXI_DATA_WIDTH/32)+ 1;
106 :    constant OPT_MEM_ADDR_BITS : integer := 1;

```

נגדיר את OPT_MEM_ADDR_BITS לערך 0, כך שאות הכתובת loc_addr הוא ביט בודד המבחין בין שני רגיסטרים.

```

100 :    -- Example-specific design signals
101 :    -- local parameter for addressing 32 bit / 64 bit C_S_AXI_DATA_WIDTH
102 :    -- ADDR_LSB is used for addressing 32/64 bit registers/memories
103 :    -- ADDR_LSB = 2 for 32 bits (n downto 2)
104 :    -- ADDR_LSB = 3 for 64 bits (n downto 3)
105 :    constant ADDR_LSB : integer := (C_S_AXI_DATA_WIDTH/32)+ 1;
106 :    constant OPT_MEM_ADDR_BITS : integer := 0;

```

לוגיקת ה-handshake של ה-AXI נשאת כבירת המחדל.

```

107 -----
108 ---- Signals for user logic register space example
109 -----
110 ---- Number of Slave Registers 4
111 signal slv_reg0 :std_logic_vector(C_S_AXI_DATA_WIDTH-1 downto 0);
112 signal slv_reg1 :std_logic_vector(C_S_AXI_DATA_WIDTH-1 downto 0);
113 signal slv_reg_rden : std_logic;
114 signal slv_reg_wren : std_logic;
115 signal reg_data_out :std_logic_vector(C_S_AXI_DATA_WIDTH-1 downto 0);
116 signal byte_index   : integer;
117 signal aw_en        : std_logic;
118
119 begin
120     -- I/O Connections assignments
121
122     S_AXI_AWREADY  <= axi_awready;
123     S_AXI_WREADY   <= axi_wready;
124     S_AXI_BRESP    <= axi_bresp;
125     S_AXI_BVALID   <= axi_bvalid;
126     S_AXI_ARREADY  <= axi_arready;
127     S_AXI_RDATA    <= axi_rdata;
128     S_AXI_RRESP    <= axi_rresp;
129     S_AXI_RVALID   <= axi_rvalid;
130     -- Implement axi_awready generation
131     -- axi_awready is asserted for one S_AXI_ACLK clock cycle when both
132     -- S_AXI_AWVALID and S_AXI_WVALID are asserted. axi_awready is
133     -- de-asserted when reset is low.
134     -- AXI write address handshake
135     process (S_AXI_ACLK)
136     begin
137         if rising_edge(S_AXI_ACLK) then
138             if S_AXI_ARESETN = '0' then
139                 axi_awready <= '0';
140                 aw_en <= '1';
141             else
142                 if (axi_awready = '0' and S_AXI_AWVALID = '1' and S_AXI_WVALID = '1' and aw_en =
143                     -- slave is ready to accept write address when
144                     -- there is a valid write address and write data
145                     -- on the write address and data bus. This design
146                     -- expects no outstanding transactions.
147                     axi_awready <= '1';
148                     aw_en <= '0';
149                     elsif (S_AXI_BREADY = '1' and axi_bvalid = '1') then
150                         aw_en <= '1';
151                         axi_awready <= '0';
152                     else
153                         axi_awready <= '0';
154                     end if;
155                 end if;
156             end if;
157         end process;

```

גם לוגיקת קבלת הכתובת נשארת ללא שינוי.

```
158
159 -- Implement axi_awaddr latching
160 -- This process is used to latch the address when both
161 -- S_AXI_AWVALID and S_AXI_WVALID are valid.
162
163 process (S_AXI_ACLK)
164 begin
165     if rising_edge(S_AXI_ACLK) then
166         if S_AXI_ARESETN = '0' then
167             axi_awaddr <= (others => '0');
168         else
169             if (axi_awready = '0' and S_AXI_AWVALID = '1' and S_AXI_WVALID = '1' and aw_en = '1') then
170                 -- Write Address latching
171                 axi_awaddr <= S_AXI_AWADDR;
172             end if;
173         end if;
174     end if;
175 end process;
176
177 -- Implement axi_wready generation
178 -- axi_wready is asserted for one S_AXI_ACLK clock cycle when both
179 -- S_AXI_AWVALID and S_AXI_WVALID are asserted. axi_wready is
180 -- de-asserted when reset is low.
181
182 process (S_AXI_ACLK)
183 begin
184     if rising_edge(S_AXI_ACLK) then
185         if S_AXI_ARESETN = '0' then
186             axi_wready <= '0';
187         else
188             if (axi_wready = '0' and S_AXI_WVALID = '1' and S_AXI_AWVALID = '1' and aw_en = '1') then
189                 -- slave is ready to accept write data when
190                 -- there is a valid write address and write data
191                 -- on the write address and data bus. This design
192                 -- expects no outstanding transactions.
193                 axi_wready <= '1';
194             else
195                 axi_wready <= '0';
196             end if;
197         end if;
198     end if;
199 end process;
200
```

רגיסטר 0 בהיסט 0x00 מקבל כתיבה ושולט במצב הלדים.

```

201 : -- Implement memory mapped register select and write logic generation
202 : -- The write data is accepted and written to memory mapped registers when
203 : -- axi_awready, S_AXI_WVALID, axi_wready and S_AXI_WVALID are asserted. Write strobes are used to
204 : -- select byte enables of slave registers while writing.
205 : -- These registers are cleared when reset (active low) is applied.
206 : -- Slave register write enable is asserted when valid address and data are available
207 : -- and the slave is ready to accept the write address and write data.
208 : slv_reg_wren <= axi_wready and S_AXI_WVALID and axi_awready and S_AXI_WVALID ;
209 :
210 : -- write to the register file
211 : process (S_AXI_ACLK)
212 : variable loc_addr :std_logic_vector(OPT_MEM_ADDR_BITS downto 0);
213 : begin
214 :   if rising_edge(S_AXI_ACLK) then
215 :     if S_AXI_ARESETN = '0' then
216 :       slv_reg0 <= (others => '0'); -- slv_reg1 is read only
217 :     else
218 :       loc_addr := axi_awaddr(ADDR_LSB + OPT_MEM_ADDR_BITS downto ADDR_LSB); -- bit 2 differentiates between the two registers
219 :       if (slv_reg_wren = '1') then
220 :         case loc_addr is
221 :           when "0" =>
222 :             for byte_index in 0 to (C_S_AXI_DATA_WIDTH/8-1) loop
223 :               if ( S_AXI_WSTRB(byte_index) = '1' ) then
224 :                 -- Respective byte enables are asserted as per write strobes
225 :                 -- slave register 0
226 :                 slv_reg0(byte_index*8+7 downto byte_index*8) <= S_AXI_WDATA(byte_index*8+7 downto byte_index*8);
227 :               end if;
228 :             end loop;
229 :           when "1" =>
230 :             null;
231 :           when others =>
232 :             null;
233 :         end case;
234 :       end if;
235 :     end if;
236 :   end if;
237 : end process;

```

לוגיקת תגובת הכתיבה נשארת ללא שינוי.

```

238 : -- Implement write response logic generation
239 : -- The write response and response valid signals are asserted by the slave
240 : -- when axi_wready, S_AXI_WVALID, axi_wready and S_AXI_WVALID are asserted.
241 : -- This marks the acceptance of address and indicates the status of
242 : -- write transaction.
243 :
244 : process (S_AXI_ACLK)
245 : begin
246 :   if rising_edge(S_AXI_ACLK) then
247 :     if S_AXI_ARESETN = '0' then
248 :       axi_bvalid <= '0';
249 :       axi_bresp <= "00"; --need to work more on the responses
250 :     else
251 :       if (axi_awready = '1' and S_AXI_WVALID = '1' and axi_wready = '1' and S_AXI_WVALID = '1' and axi_bvalid = '0' ) then
252 :         axi_bvalid <= '1';
253 :         axi_bresp <= "00";
254 :       elsif (S_AXI_BREADY = '1' and axi_bvalid = '1') then --check if bready is asserted while bvalid is high)
255 :         axi_bvalid <= '0'; -- (there is a possibility that bready is always asserted high)
256 :       end if;
257 :     end if;
258 :   end if;
259 : end process;
260 :
261 : -- Implement axi_arready generation
262 : -- axi_arready is asserted for one S_AXI_ACLK clock cycle when
263 : -- S_AXI_ARVALID is asserted. axi_arready is
264 : -- de-asserted when reset (active low) is asserted.
265 : -- The read address is also latched when S_AXI_ARVALID is
266 : -- asserted. axi_araddr is reset to zero on reset assertion.
267 :
268 : process (S_AXI_ACLK)
269 : begin
270 :   if rising_edge(S_AXI_ACLK) then
271 :     if S_AXI_ARESETN = '0' then
272 :       axi_arready <= '0';
273 :       axi_araddr <= (others => '1');
274 :     else
275 :       if (axi_arready = '0' and S_AXI_ARVALID = '1') then
276 :         -- indicates that the slave has accepted the valid read address
277 :         axi_arready <= '1';
278 :         -- Read Address latching
279 :         axi_araddr <= S_AXI_ARADDR;
280 :       else
281 :         axi_arready <= '0';
282 :       end if;
283 :     end if;
284 :   end if;
285 : end process;
286 :

```

לוגיקת ערוץ הקריאה נשארת ללא שינוי.

```
287 : -- Implement axi_rvalid generation
288 : -- axi_rvalid is asserted for one S_AXI_ACLK clock cycle when both
289 : -- S_AXI_ARVALID and axi_arready are asserted. The slave registers
290 : -- data are available on the axi_rdata bus at this instance. The
291 : -- assertion of axi_rvalid marks the validity of read data on the
292 : -- bus and axi_rresp indicates the status of read transaction.axi_rvalid
293 : -- is deasserted on reset (active low). axi_rresp and axi_rdata are
294 : -- cleared to zero on reset (active low).
295 : process (S_AXI_ACLK)
296 : begin
297 :   if rising_edge(S_AXI_ACLK) then
298 :     if S_AXI_ARESETN = '0' then
299 :       axi_rvalid <= '0';
300 :       axi_rresp <= "00";
301 :     else
302 :       if (axi_arready = '1' and S_AXI_ARVALID = '1' and axi_rvalid = '0') then
303 :         -- Valid read data is available at the read data bus
304 :         axi_rvalid <= '1';
305 :         axi_rresp <= "00"; -- 'OKAY' response
306 :       elsif (axi_rvalid = '1' and S_AXI_RREADY = '1') then
307 :         -- Read data is accepted by the master
308 :         axi_rvalid <= '0';
309 :       end if;
310 :     end if;
311 :   end if;
312 : end process;
```

בבורר הקריאה, רגיסטר 1 בהיסט 0x04 מחזיר את ערך הגרסה הקשיו 20251123"א" ומוגדר לקריאה בלבד.

```
315 : -- Implement memory mapped register select and read logic generation
316 : -- Slave register read enable is asserted when valid address is available
317 : -- and the slave is ready to accept the read address.
318 : slv_reg_rden <= axi_arready and S_AXI_ARVALID and (not axi_rvalid) ;
319 :
320 : process (slv_reg0, slv_reg1, axi_araddr, S_AXI_ARESETN, slv_reg_rden)
321 :   variable loc_addr : std_logic_vector(OPT_MEM_ADDR_BITS downto 0);
322 : begin
323 :   -- Address decoding for reading registers
324 :   loc_addr := axi_araddr(ADDR_LSB + OPT_MEM_ADDR_BITS downto ADDR_LSB);
325 :   case loc_addr is
326 :     when "0" =>
327 :       reg_data_out <= slv_reg0;
328 :     when "1" =>
329 :       reg_data_out <= IP_LEDS_Version;
330 :     when others =>
331 :       reg_data_out <= (others => '0');
332 :   end case;
333 : end process;
334 :
335 : -- Output register or memory read data
336 : process( S_AXI_ACLK ) is
337 : begin
338 :   if (rising_edge (S_AXI_ACLK)) then
339 :     if ( S_AXI_ARESETN = '0' ) then
340 :       axi_rdata <= (others => '0');
341 :     else
342 :       if (slv_reg_rden = '1') then
343 :         -- When there is a valid read address (S_AXI_ARVALID) with
344 :         -- acceptance of read address by the slave (axi_arready),
345 :         -- output the read data
346 :         -- Read address mux
347 :         axi_rdata <= reg_data_out; -- register read data
348 :       end if;
349 :     end if;
350 :   end if;
351 : end process;
```

הנתון הנקרא מועבר לאפיק ה-AXI.

```
354 ; -- Output register or memory read data
355 process( S_AXI_ACLK ) is
356 begin
357   if (rising_edge (S_AXI_ACLK)) then
358     if ( S_AXI_ARESETN = '0' ) then
359       axi_rdata <= (others => '0');
360     else
361       if (slv_reg_rden = '1') then
362         -- When there is a valid read address (S_AXI_ARVALID) with
363         -- acceptance of read address by the slave (axi_arready),
364         -- output the read data
365         -- Read address mux
366         axi_rdata <= reg_data_out; -- register read data
367       end if;
368     end if;
369   end if;
370 end process;
371
372
373
```

בסוף הקובץ נחבר את `s_mode LEDs` לביטים התחתונים של רגיסטר 0.

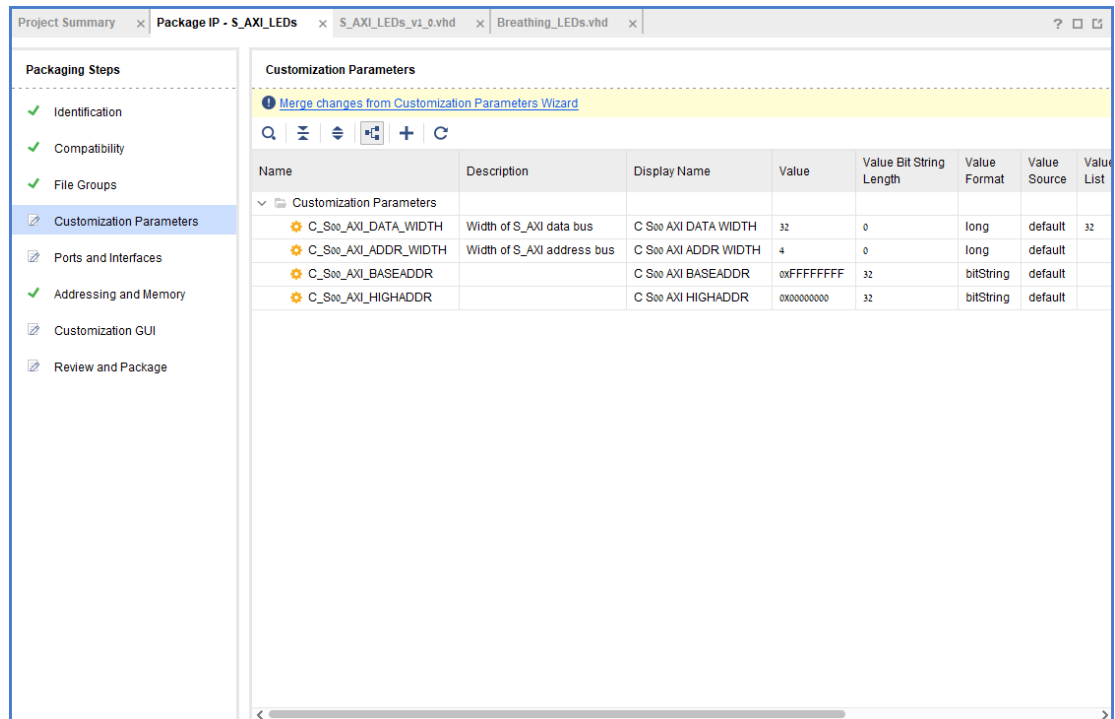
```
374 ; -- Add user logic here
375
376 --Decode LEDs mode based on slv_reg0
377 -- 0 -> "00" chaising LEDs
378 -- 1 -> "01" Breathing LEDs
379 with slv_reg0(0) select
380   LEDs_mode_o <= "00" when '0',
381                  "01" when '1',
382                  "10" when others; --default LEDs will be turned off
383
384 -- User logic ends
385
386 end arch_imp;
```

שלב 6:

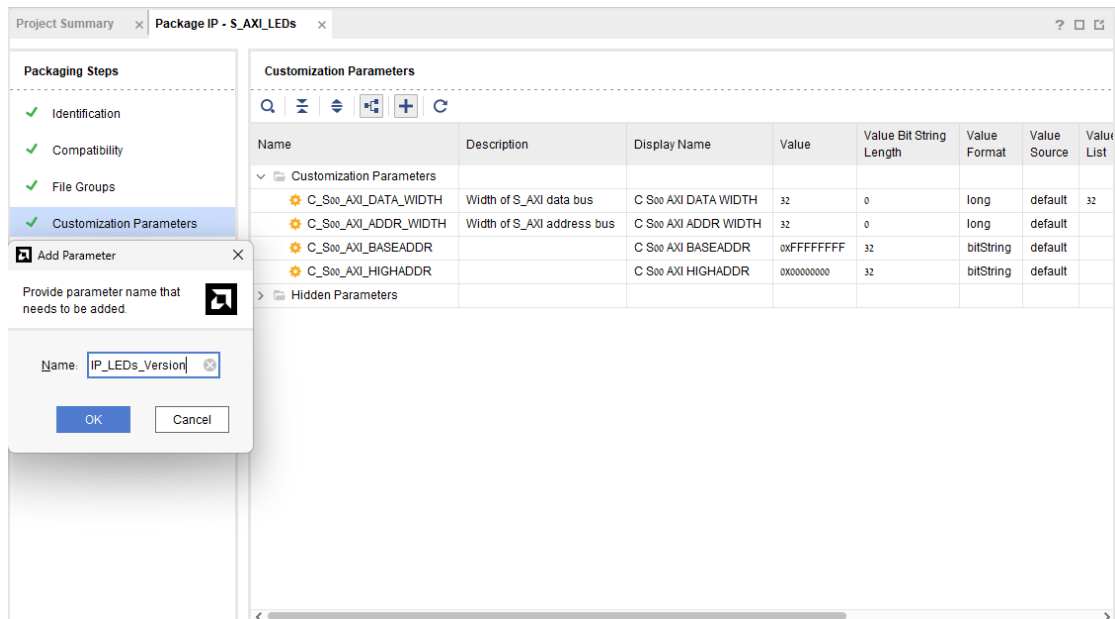
בחלון Package IP, בלשונית File Groups, נבצע Merge changes.

Name	Library Name	Type	Is Include	Model Name	File Group Name
Standard			☐		
Advanced			☐		
> VHDL Synthesis (2)			☐	S_AXI_LEDS_v1_0	
> VHDL Simulation (2)			☐	S_AXI_LEDS_v1_0	
> Software Driver (6)			☐		
> UI Layout (1)			☐		
> Block Diagram (1)			☐		

בלשונית Customization Parameters נבצע Merge changes



נוסיף את הפרמטר IP_LEDs_Version



נגדיר את מאפייני הפרמטר ונלחץ OK.

 Edit IP Parameter ✕

Use the options below to customize how the parameter will appear in the Customization GUI for users of the IP. 

Name:

☒ Visible in Customization GUI

☒ Show Name

Display Name: ✕

Tooltip: ✕

Format: ▼

Editable: ▼

Dependency: ▼

☐ Specify Range

Type: ▼

+ - ↑ ↓

Press the + button to add a value

Show As:

Layout:

Default Value: ✕

OK Cancel

בלשונית Ports and Interfaces נלחץ קליק ימני על ממשק השעון ובחר Edit Interface.

Packaging Steps

- Identification
- Compatibility
- File Groups
- Customization Parameters
- Ports and Interfaces**
- Addressing and Memory
- Customization GUI
- Review and Package

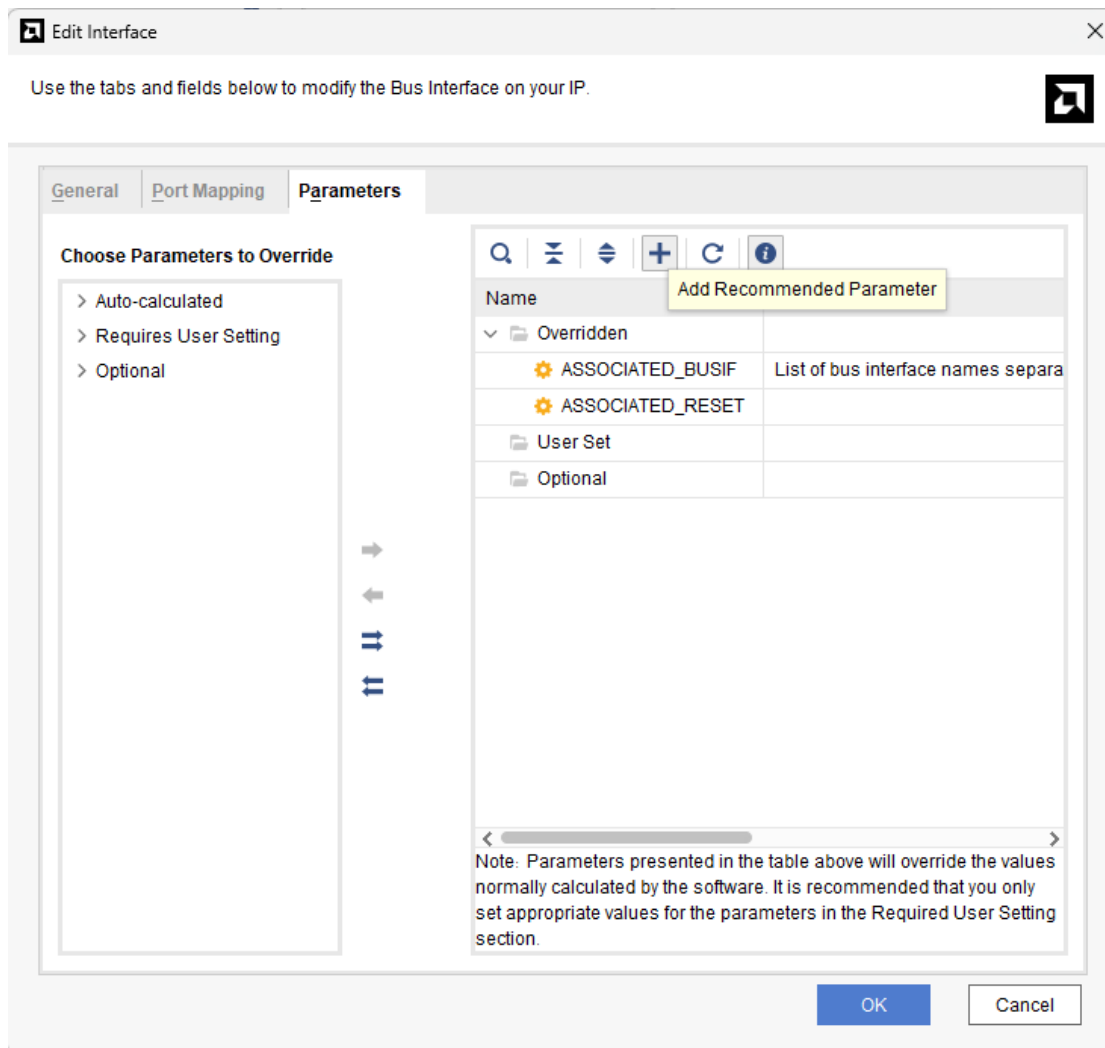
Ports and Interfaces 1

Name	Interface Mode	Enablement Dependency	Direction	Driver Value	Size Left	Size Right	Size Left Dependency	Size Right Dependency	Type Name
> Soo_AXI	slave								
> Soo_AXI_RST	slave								
> LED_o	out				7	0			std_logic_vector

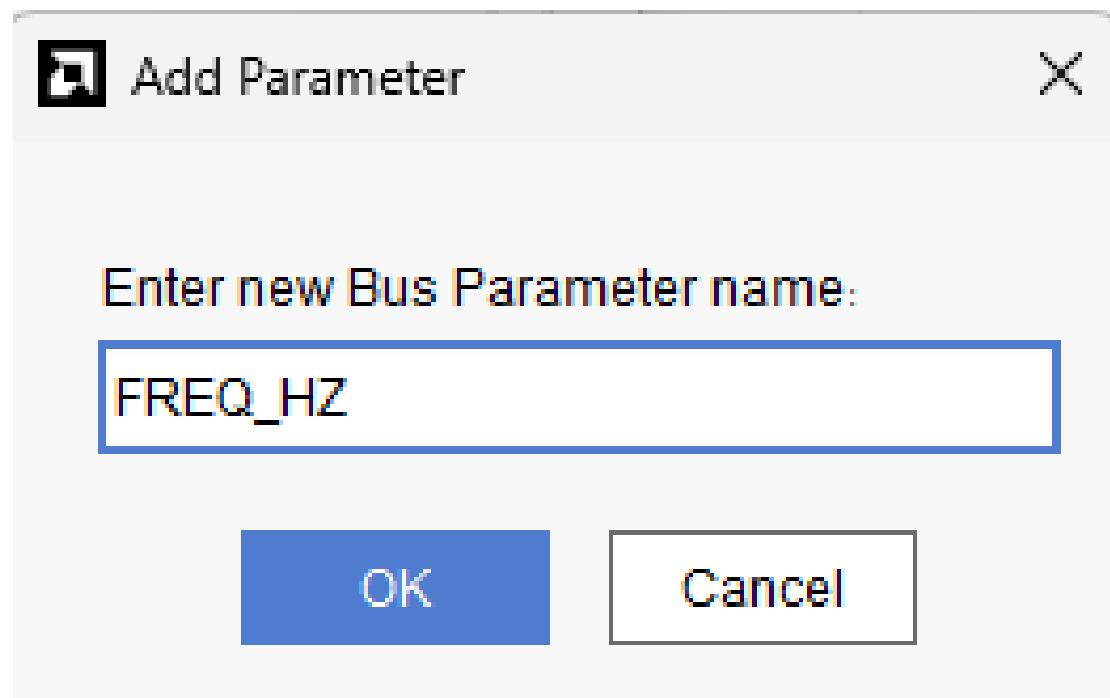
Context Menu for LED_o:

- Edit Port...
- Edit Interface...**
- Associate Clocks...
- + Add Bus Interface...
- Create Interface Definition...
- Remove Port
- Remove Interface
- Import IP Ports...
- Auto Infer Interface...
- Edit Bus Definition...
- Refresh Table
- Export to Spreadsheet...

בלשונית Parameters נלחץ Add Recommended Parameter.



נוסיף את הפרמטר FREQ_HZ.

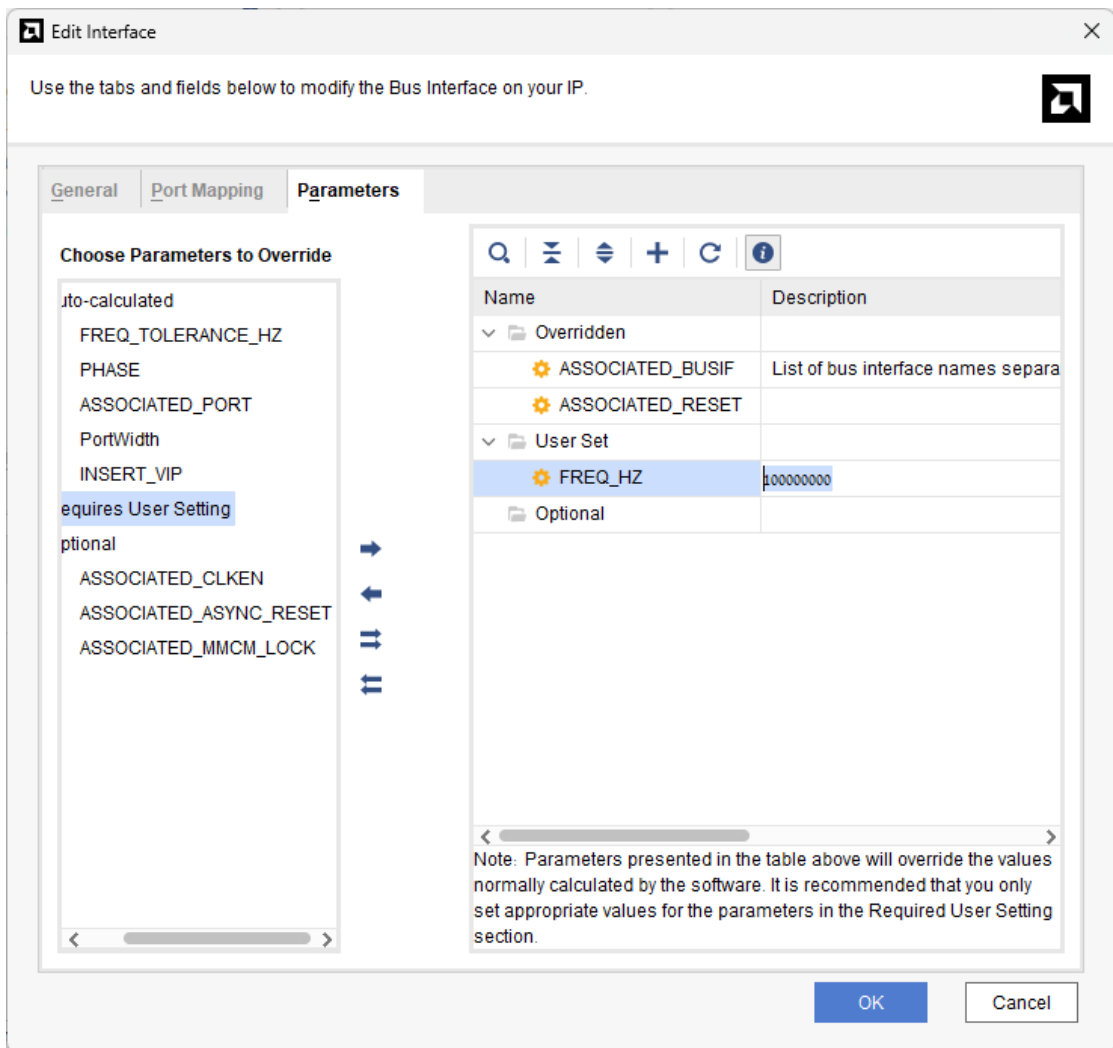


The image shows a software dialog box titled "Add Parameter". It contains a text input field with the text "FREQ_HZ" and two buttons at the bottom: "OK" and "Cancel".

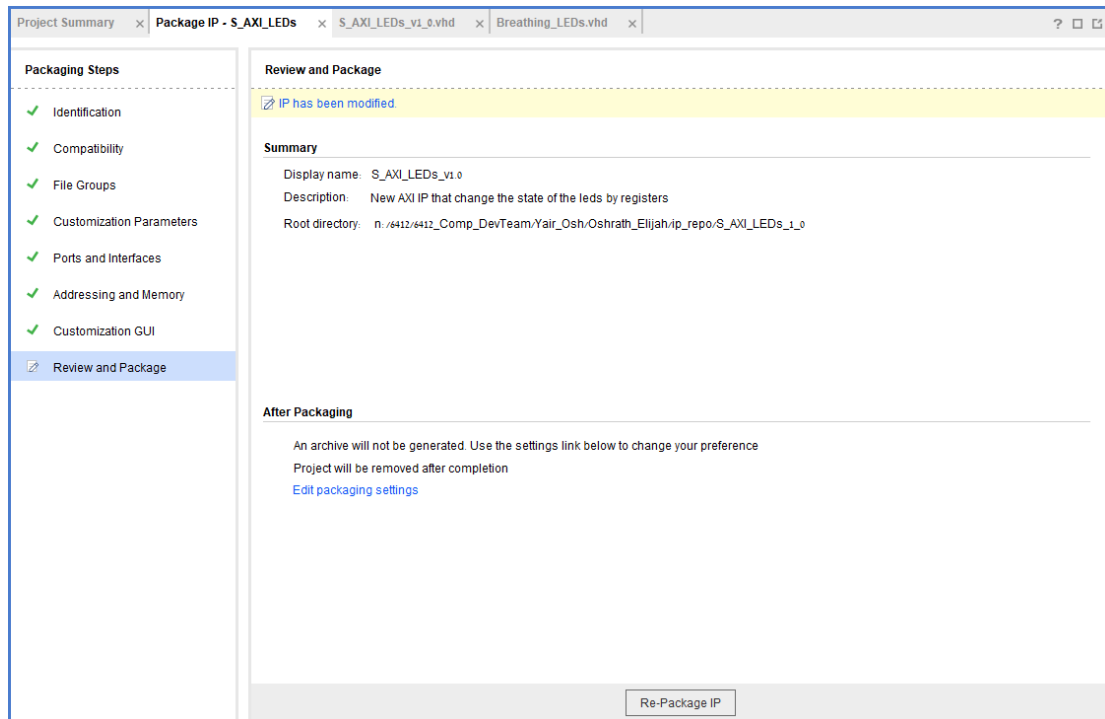
Enter new Bus Parameter name:

FREQ_HZ

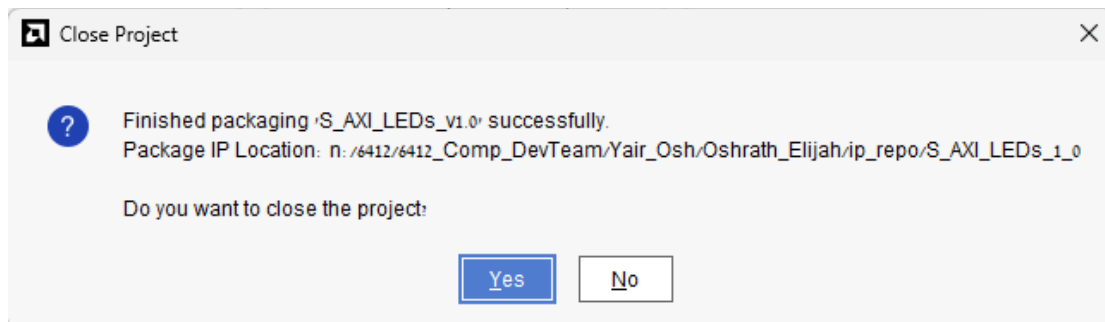
OK Cancel



בלשונית Review and Package נלחץ Re-Package IP.



האריזה הסתיימה בהצלחה.



שלב 7:

ניצור Block Design.

 Create Block Design 

Please specify name of block design. 

Design name:

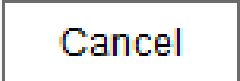


Directory:

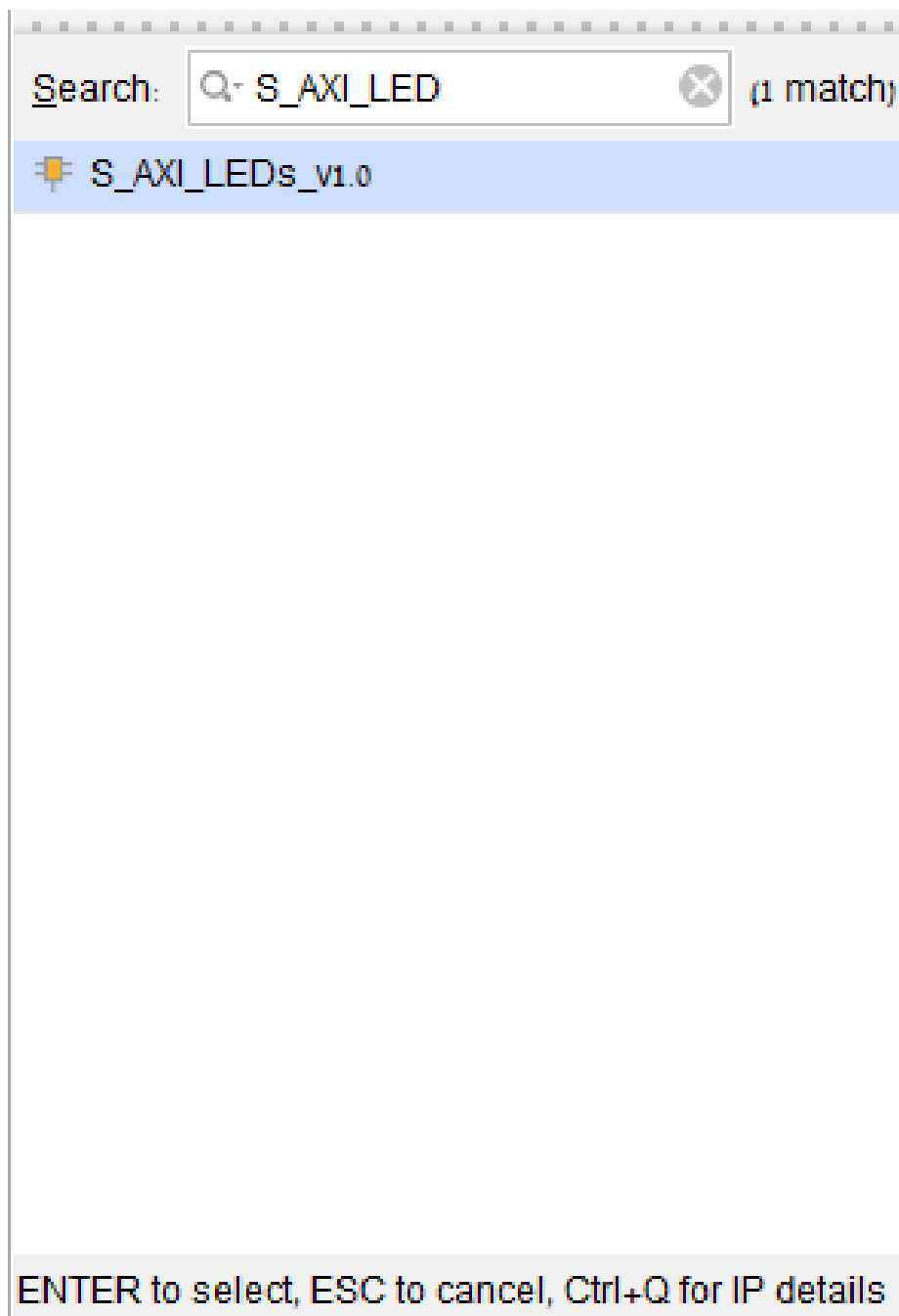
 <Local to Project> 

Specify source set:

 Design Sources 

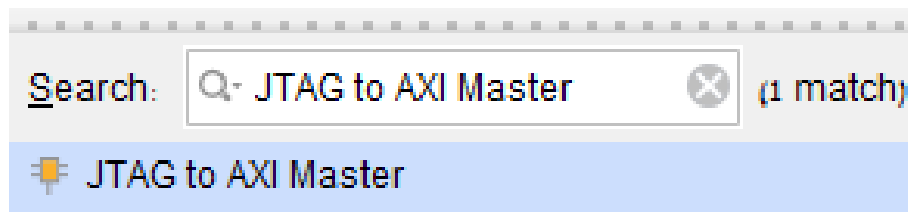
  

נוסיף את הרכיב S_AXI_LEDs.



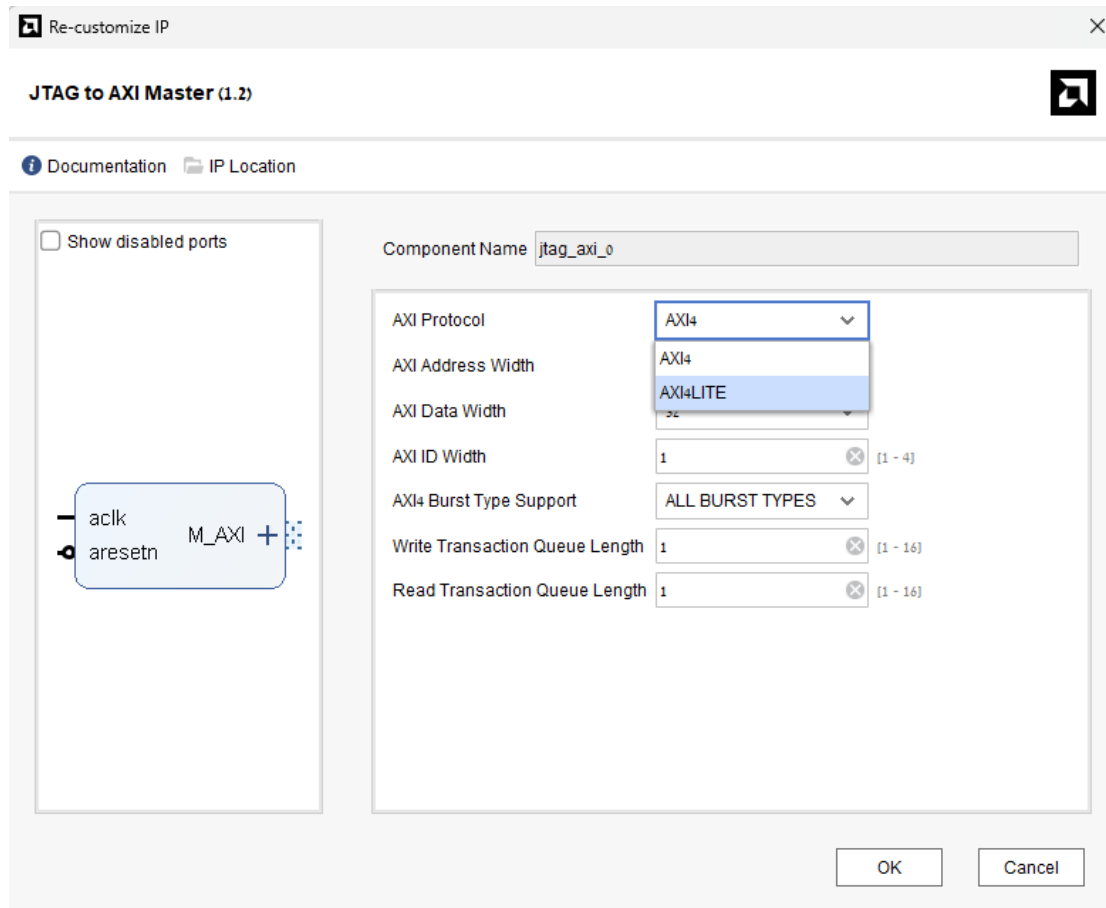
שלב 8:

נוסיף רכיב JTAG to AXI Master.



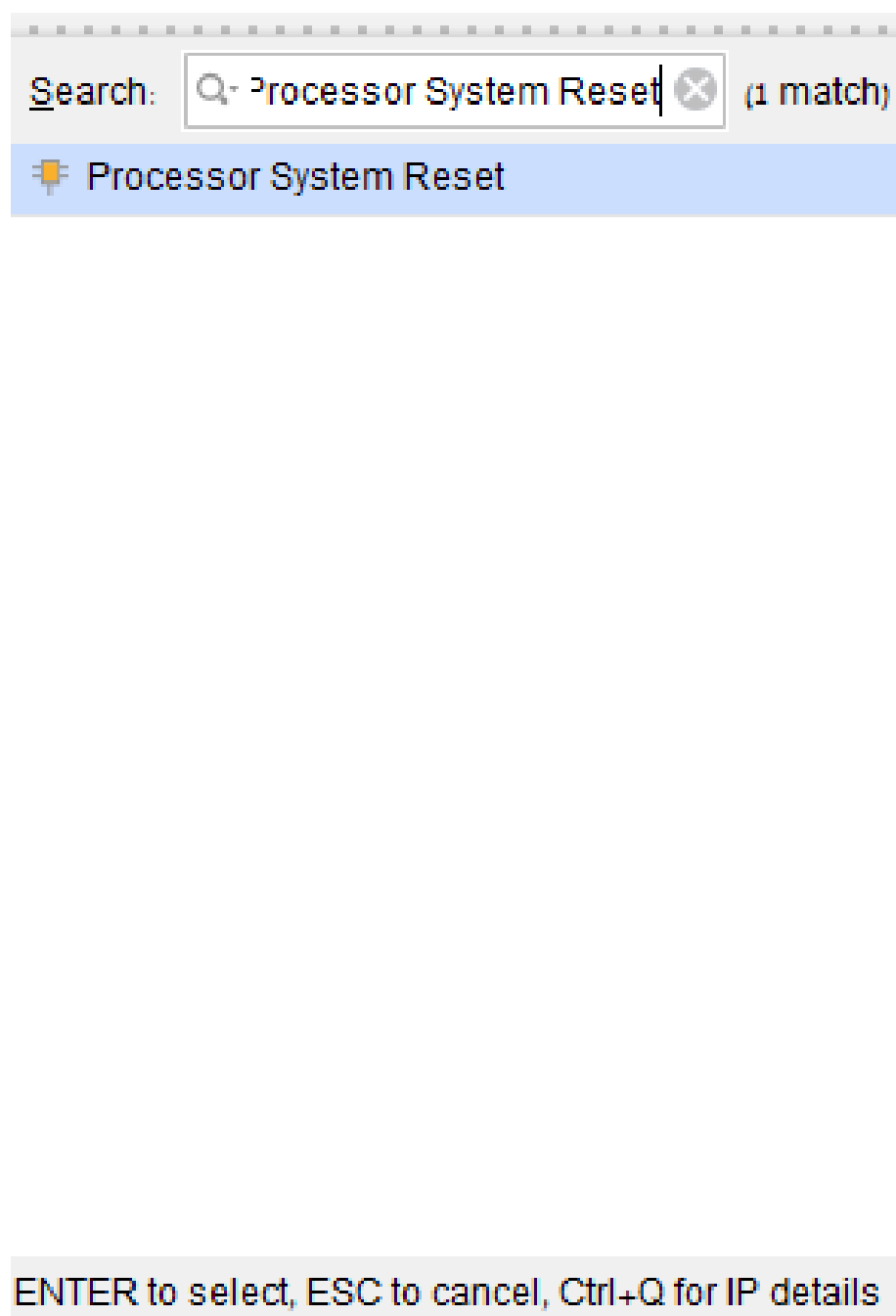
ENTER to select, ESC to cancel, Ctrl+Q for IP details

נגדיר את הפרוטוקול ל-AXI4LITE. רכיב זה מאפשר גישה לרגיסטרים של ה-IP ישירות מ-Vivado, ללא מעבד.

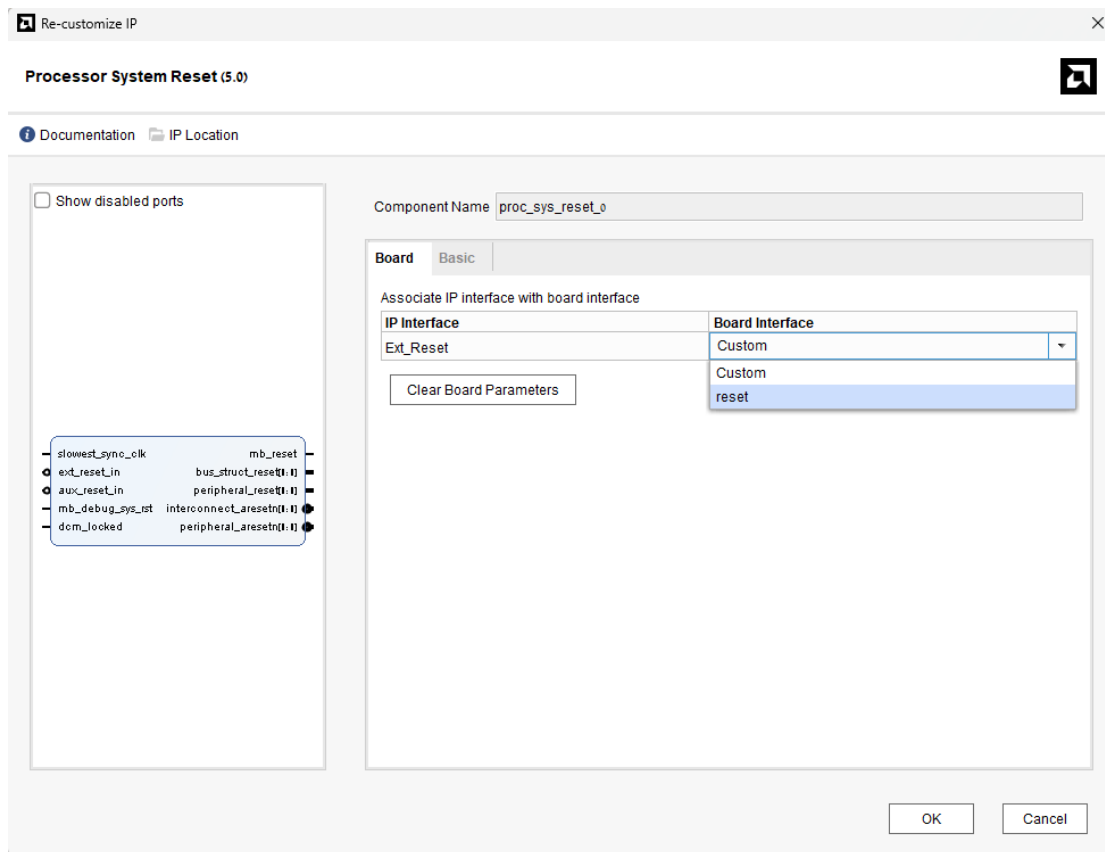


שלב 9:

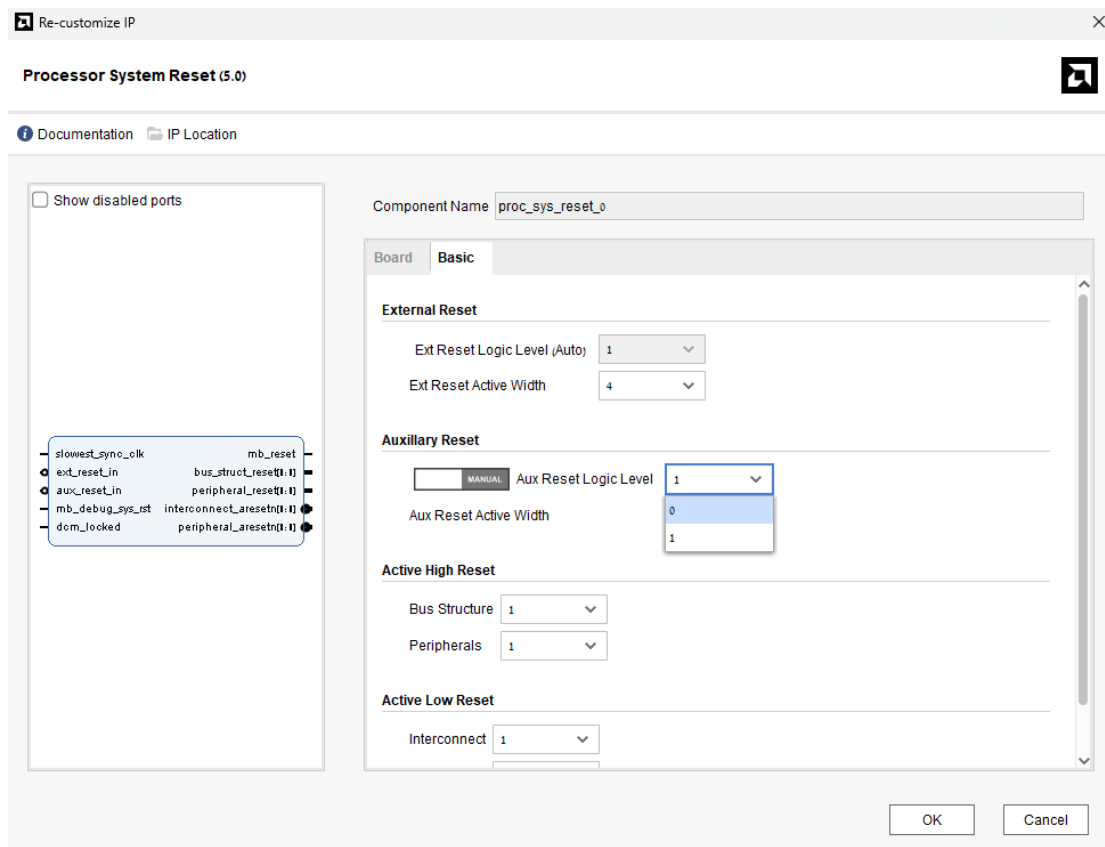
נוסיף רכיב Processor System Reset.



בלשונית Board נשייך את Ext_Reset ל-reset של הלוח.

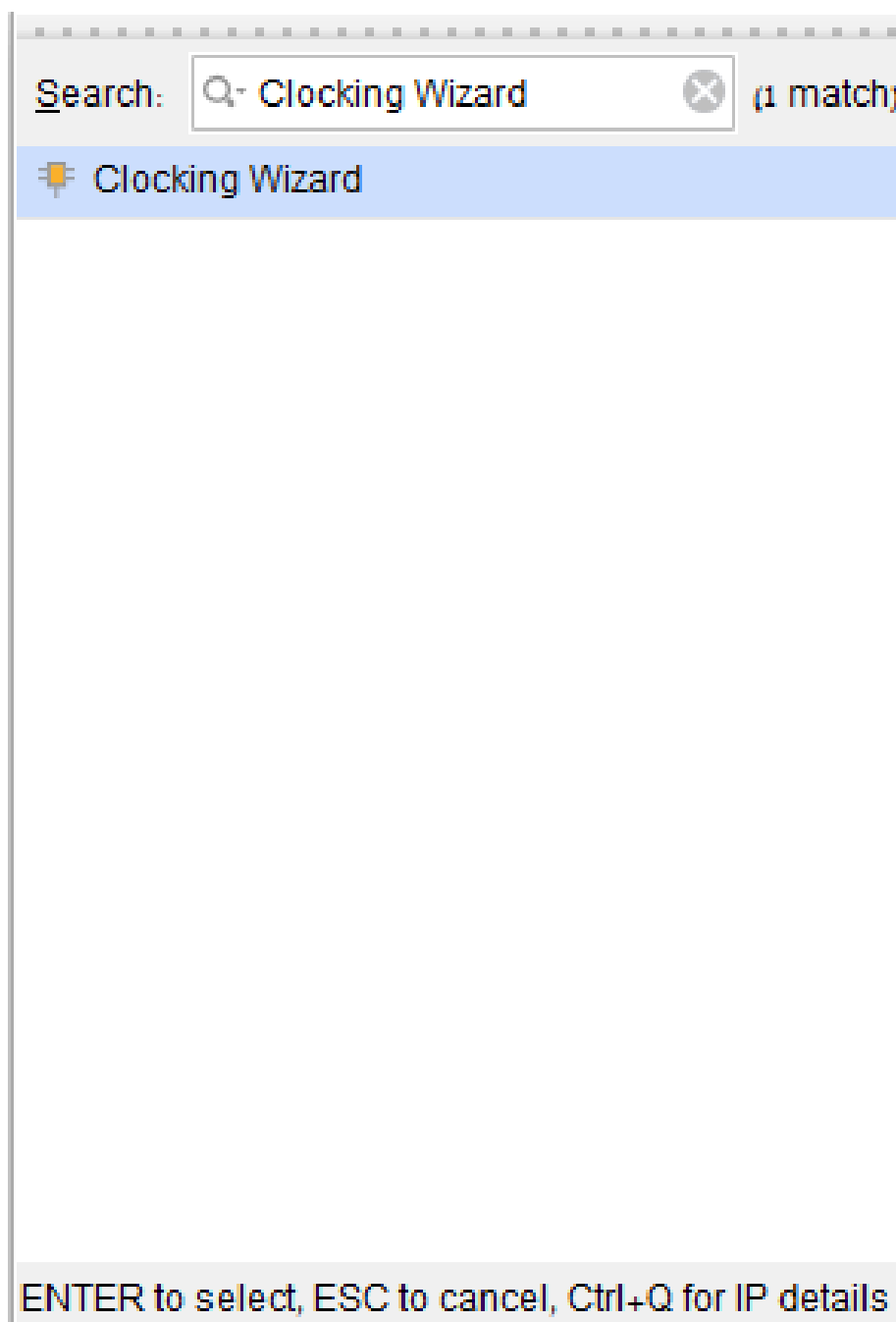


בלשונית Basic נגדיר את מאפייני האיפוס.

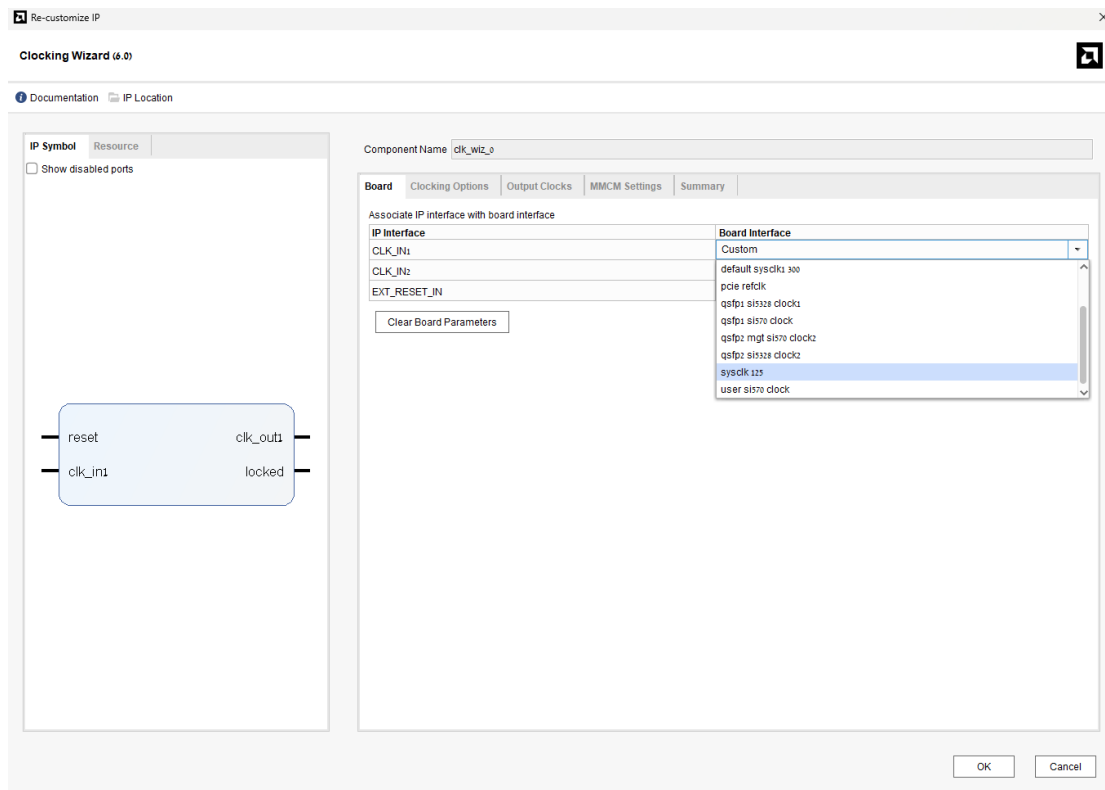


שלב 10:

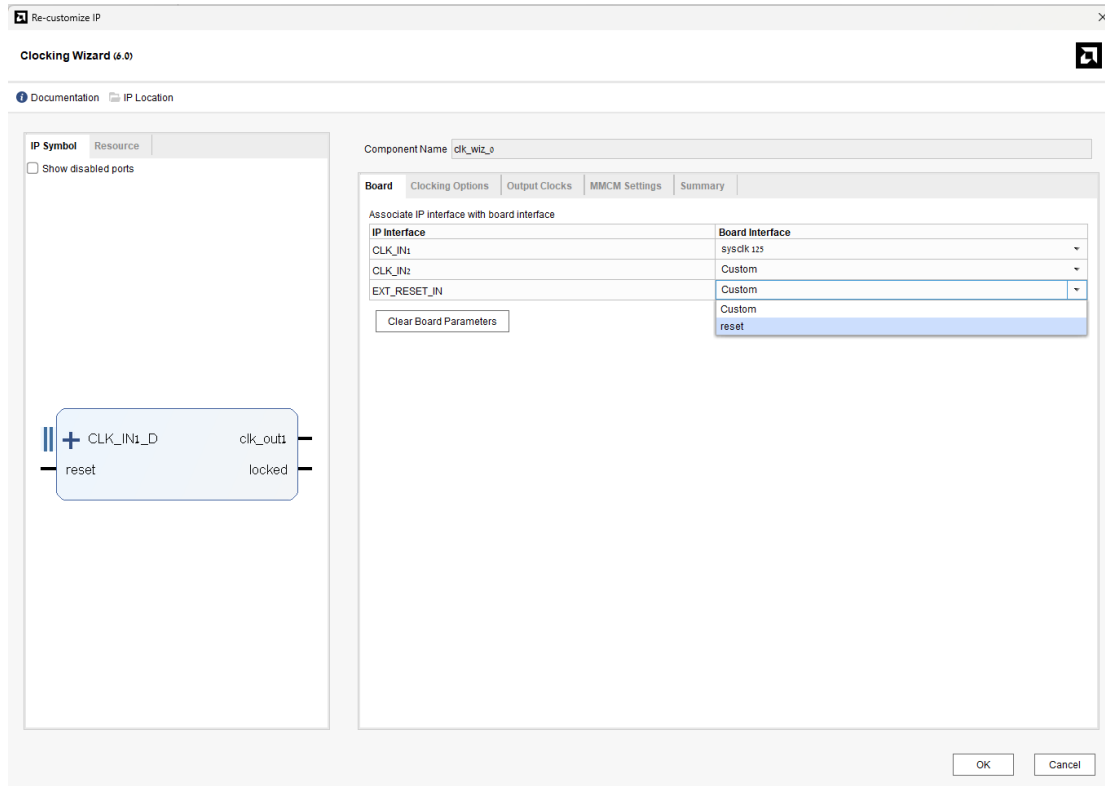
נוסיד Clocking Wizard.



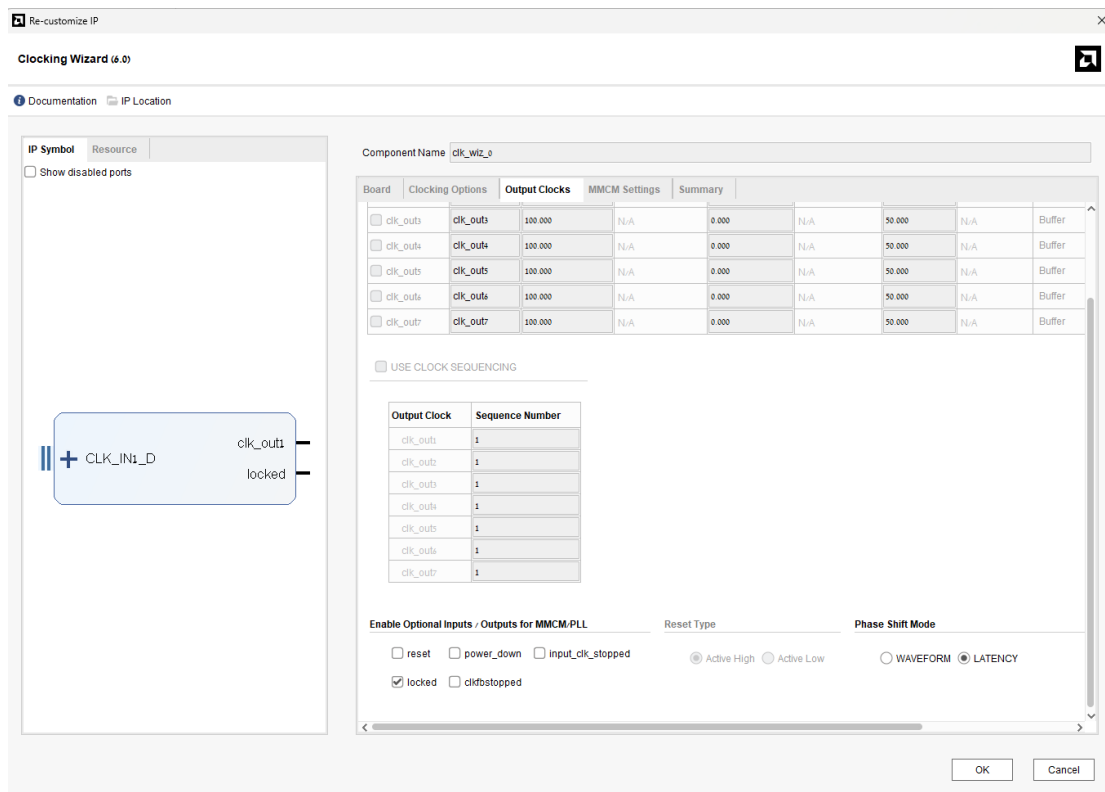
בלשונית Board נשייך את CLK_IN1 למתנד sysclk 125.



נגדיר את שיוך כניסת האיפוס.

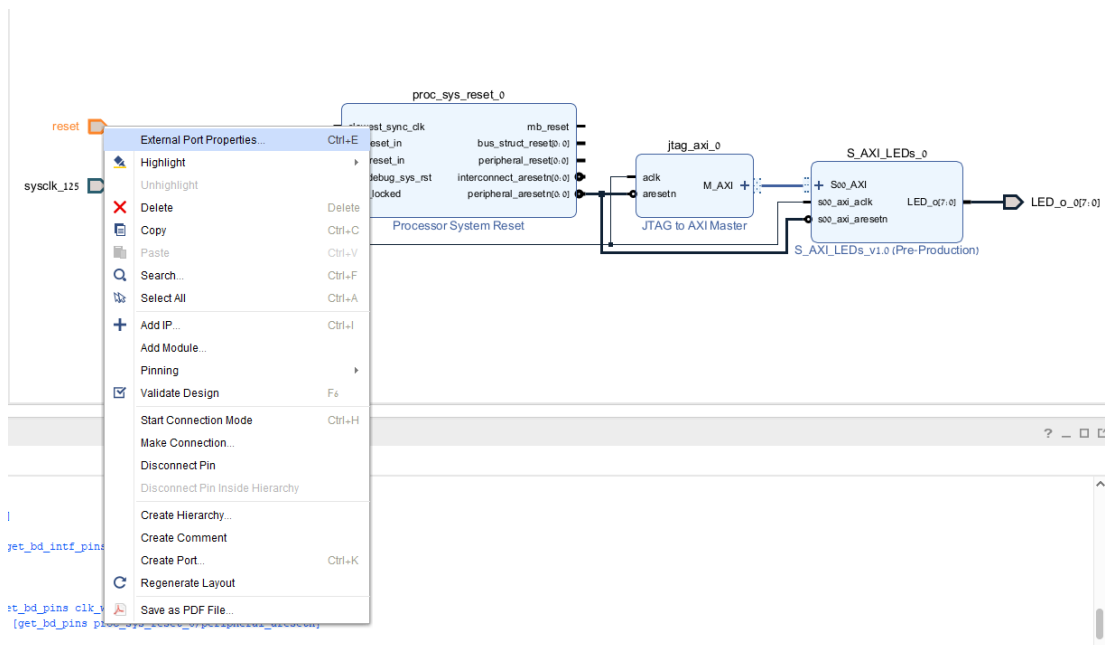


בלשונית Output Clocks נסיר את כניסת ה-reset מהרכיב.

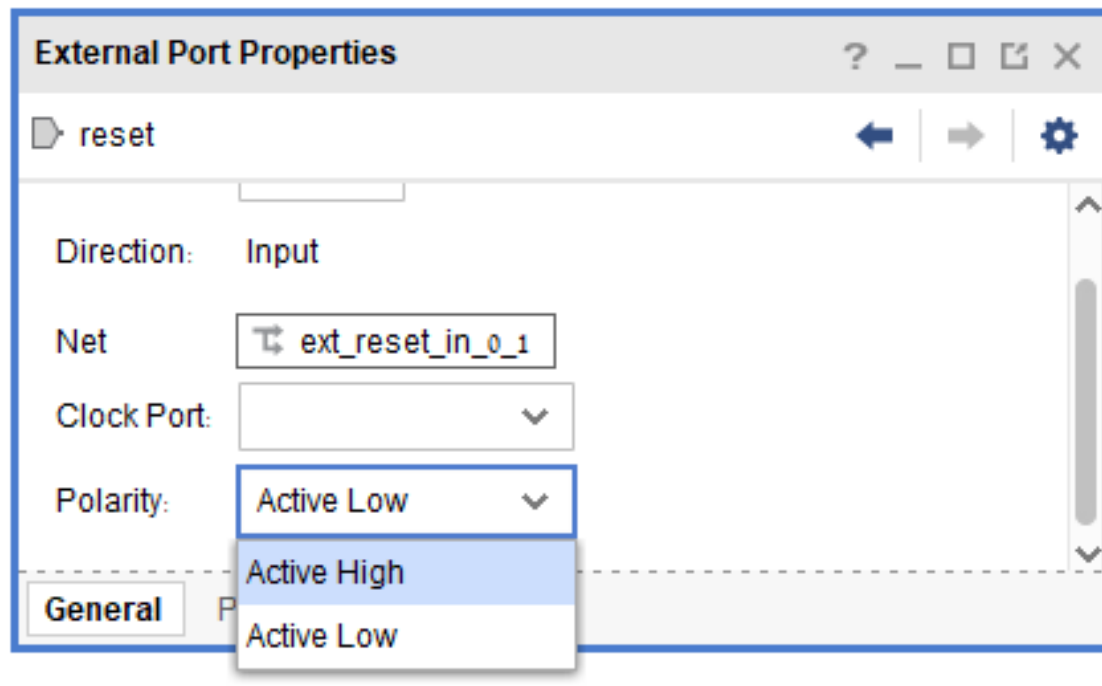


שלב 11:

נלחץ קליק ימני על הפורט reset ונבחר External Port Properties.



נגדיר את הקוטביות כך שתתאים לאות האיפוס בלוח.

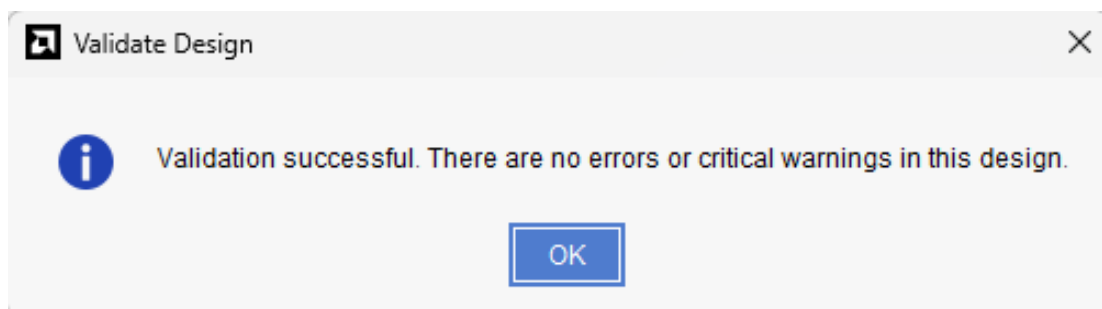


שלב 12:

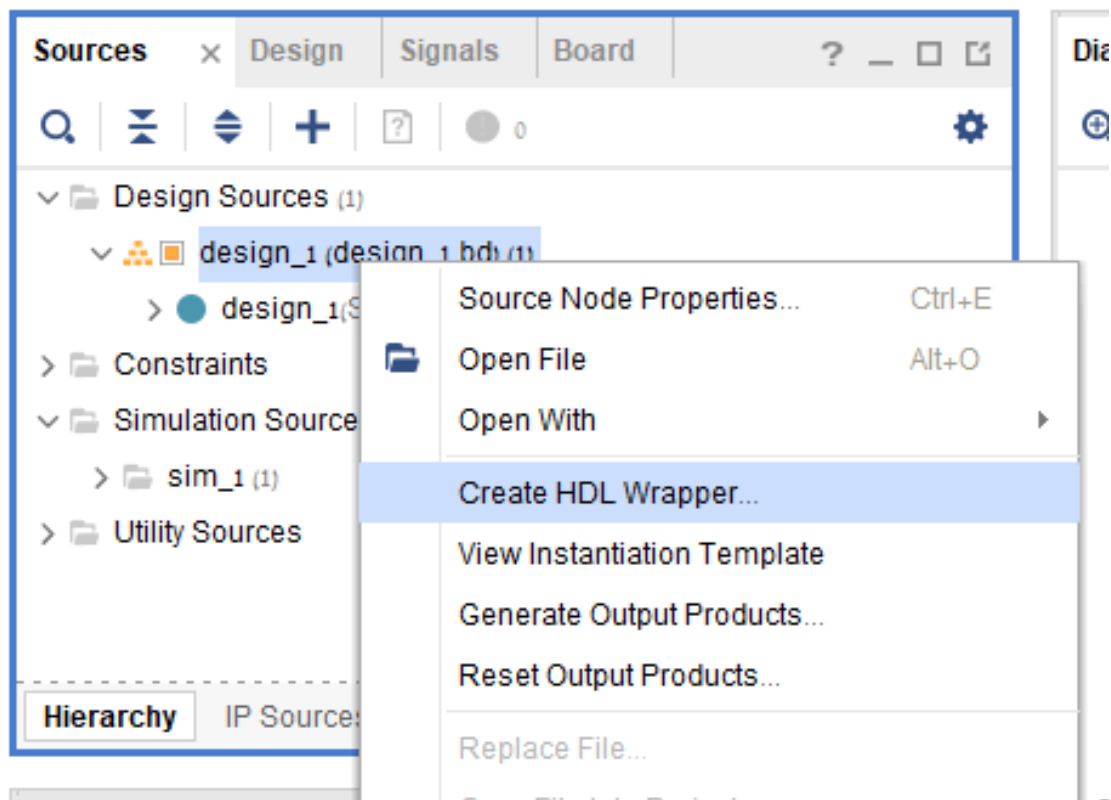
נלחץ Validate Design.



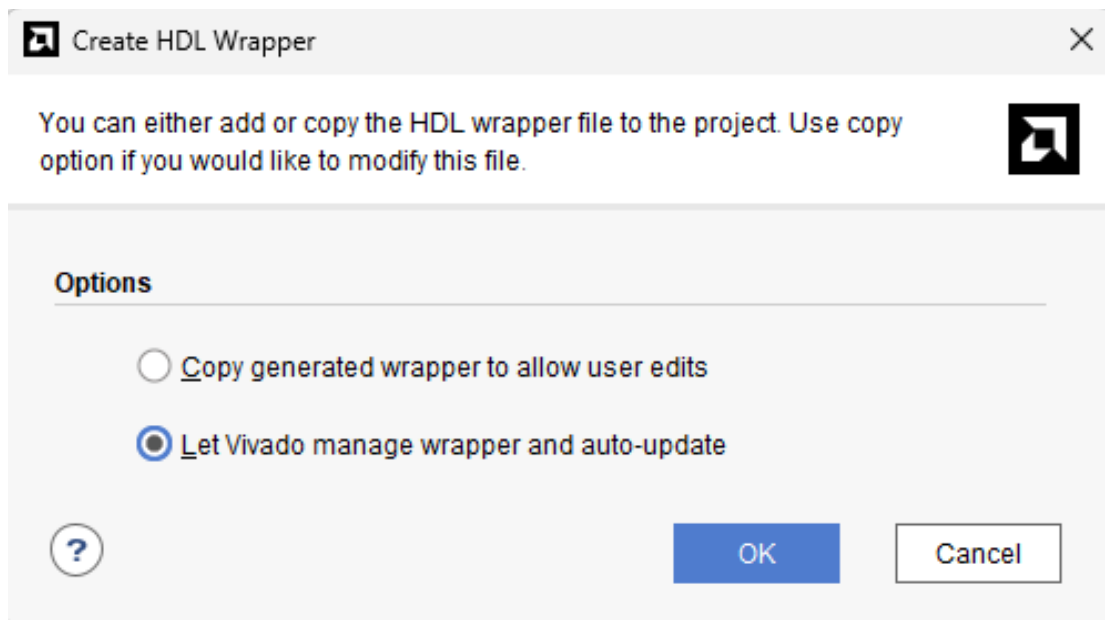
האימות עבר ללא שגיאות.



ניצור HDL Wrapper.

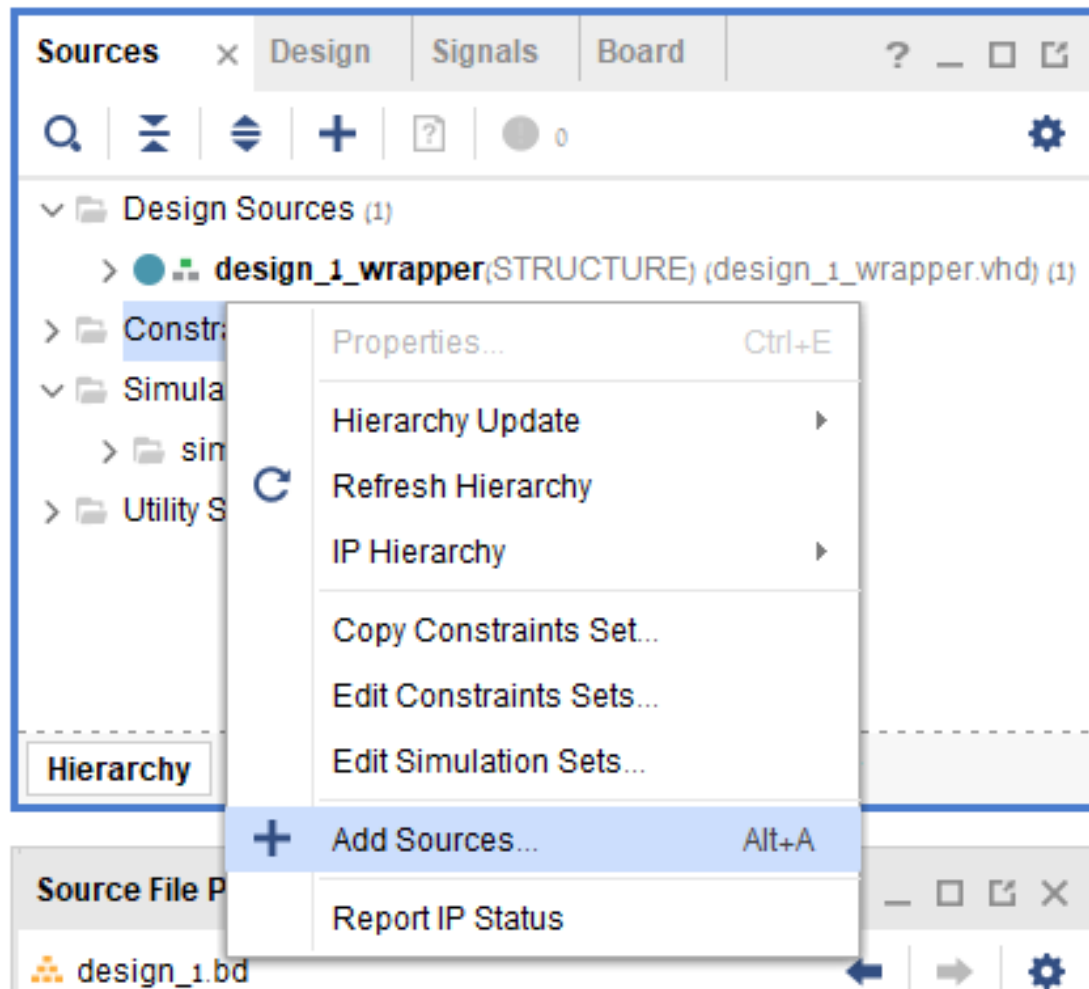


נשאיר את הניהול האוטומטי ונלחץ OK.

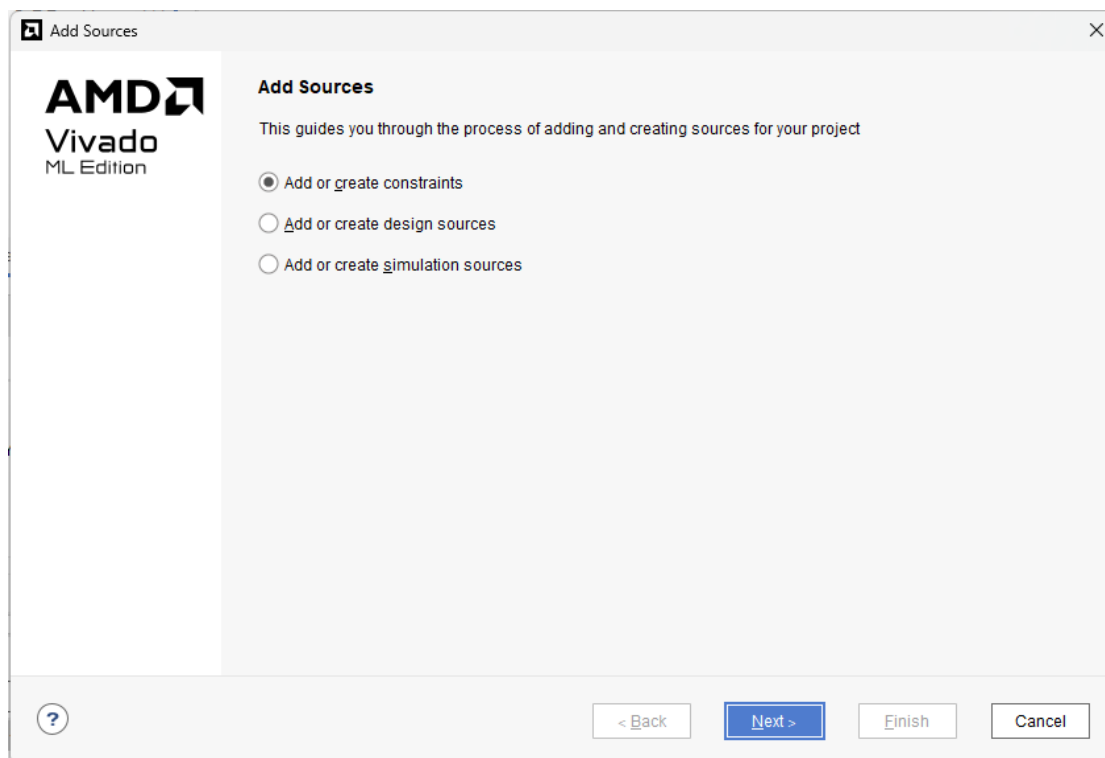


שלב 13:

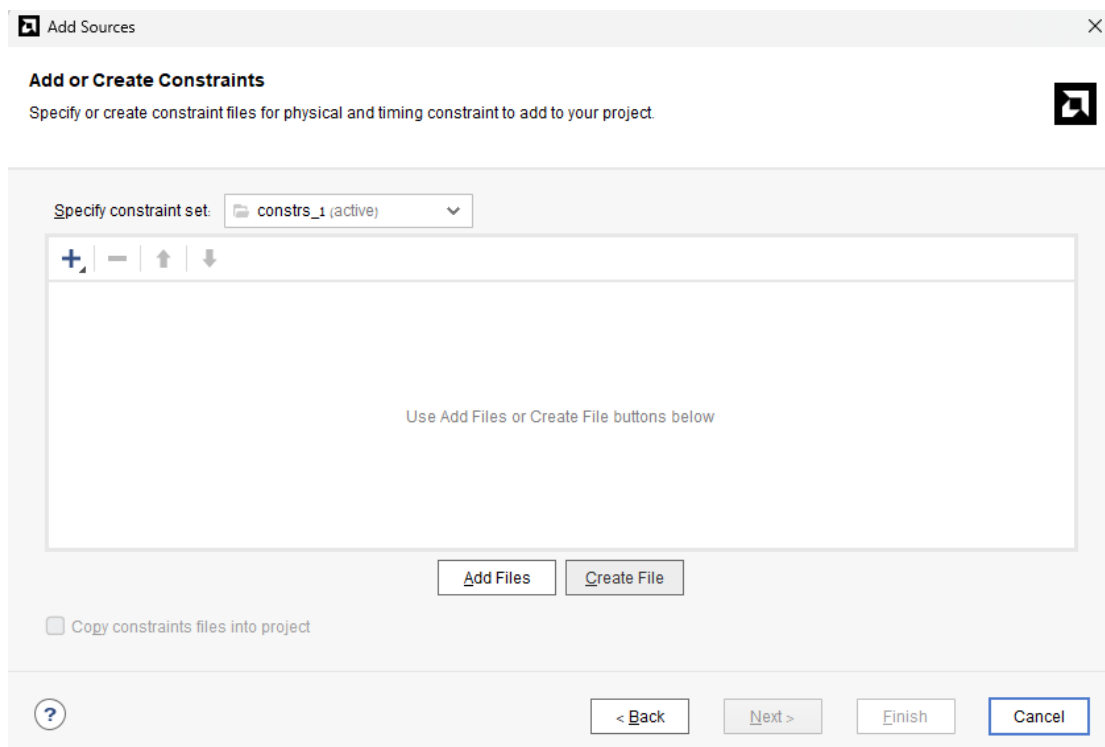
נוסיף קובץ אילוצים דרך Add Sources.



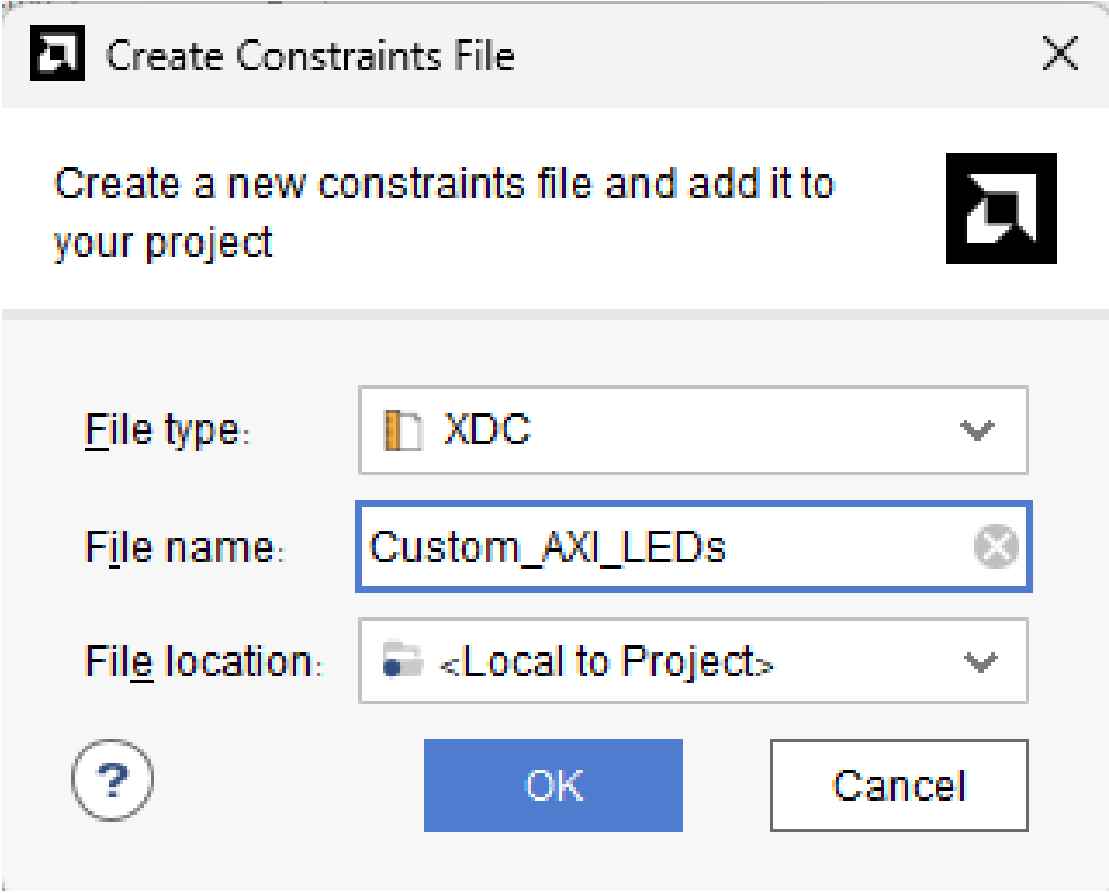
נבחר Add or create constraints.



נלחץ Create File.



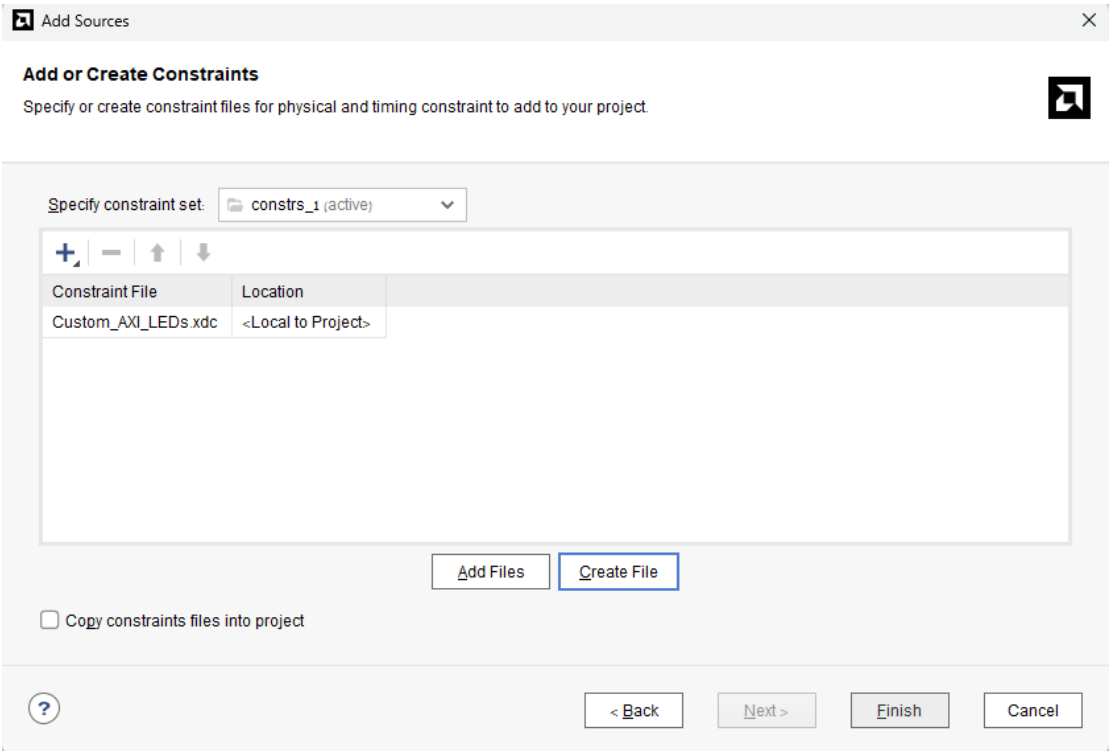
ניצור קובץ XDC בשם Custom_AXI_LEDs.



The dialog box is titled "Create Constraints File" and contains the following fields and buttons:

- File type:** A dropdown menu showing "XDC".
- File name:** A text input field containing "Custom_AXI_LEDs".
- File location:** A dropdown menu showing "<Local to Project>".
- Buttons:** A question mark icon, an "OK" button, and a "Cancel" button.

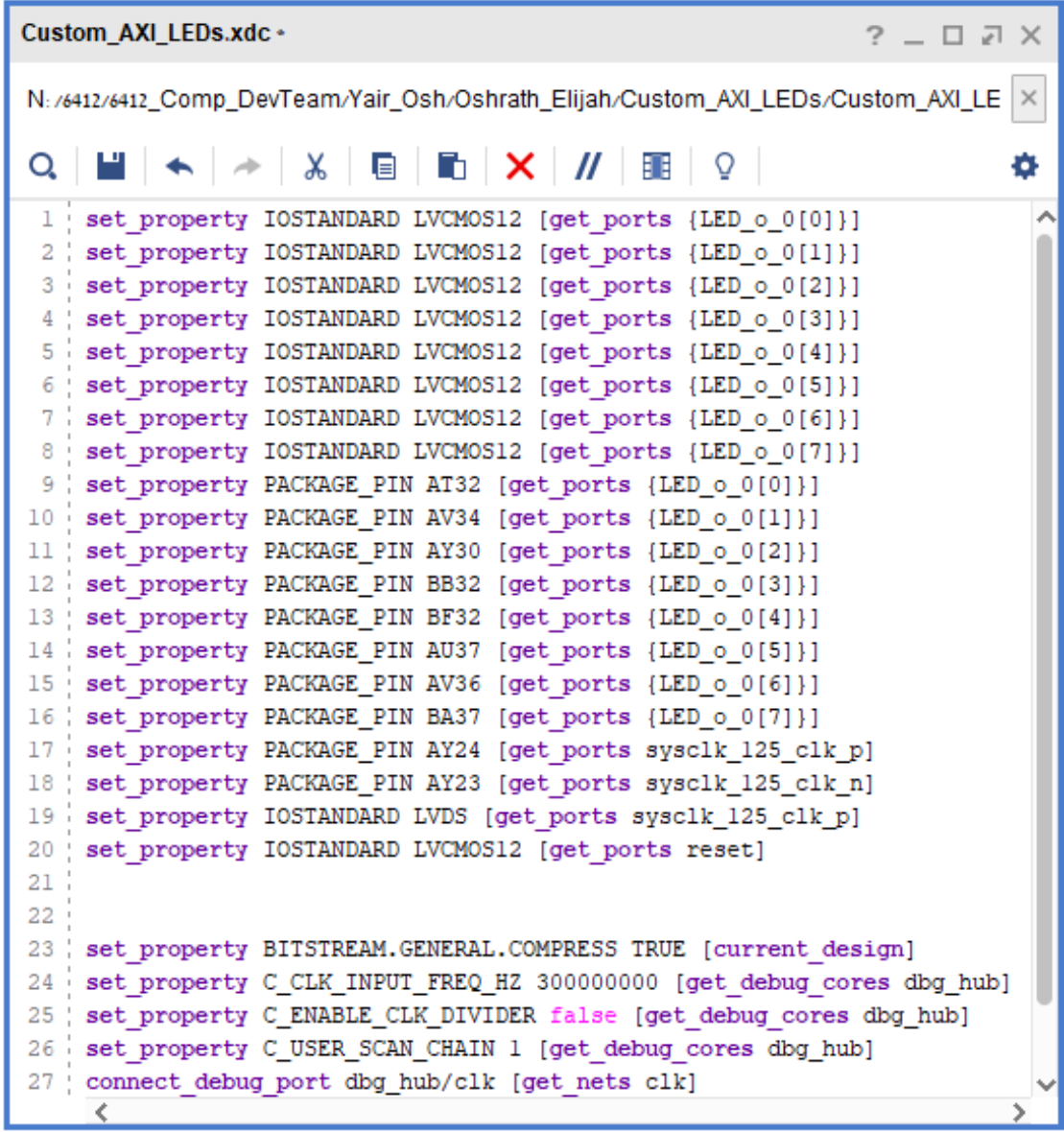
נלחץ Finish.



The dialog box is titled "Add Sources" and contains the following elements:

- Section:** "Add or Create Constraints".
- Text:** "Specify or create constraint files for physical and timing constraint to add to your project."
- Specify constraint set:** A dropdown menu showing "constrs_1 (active)".
- Table:** A table with two columns: "Constraint File" and "Location". It contains one row: "Custom_AXI_LEDs.xdc" and "<Local to Project>".
- Buttons:** "Add Files", "Create File", "Copy constraints files into project" (checkbox), "< Back", "Next >", "Finish", and "Cancel".

בקובץ נמפה את הלדים, את השעון הדיפרנציאלי ואת כניסת האיפוס.



```
Custom_AXI_LEDs.xdc
N: /6412/6412_Comp_DevTeam/Yair_Osh/Oshrath_Elijah/Custom_AXI_LEDs/Custom_AXI_LE

1  set_property IOSTANDARD LVCMOS12 [get_ports {LED_o_0[0]}]
2  set_property IOSTANDARD LVCMOS12 [get_ports {LED_o_0[1]}]
3  set_property IOSTANDARD LVCMOS12 [get_ports {LED_o_0[2]}]
4  set_property IOSTANDARD LVCMOS12 [get_ports {LED_o_0[3]}]
5  set_property IOSTANDARD LVCMOS12 [get_ports {LED_o_0[4]}]
6  set_property IOSTANDARD LVCMOS12 [get_ports {LED_o_0[5]}]
7  set_property IOSTANDARD LVCMOS12 [get_ports {LED_o_0[6]}]
8  set_property IOSTANDARD LVCMOS12 [get_ports {LED_o_0[7]}]
9  set_property PACKAGE_PIN AT32 [get_ports {LED_o_0[0]}]
10 set_property PACKAGE_PIN AV34 [get_ports {LED_o_0[1]}]
11 set_property PACKAGE_PIN AY30 [get_ports {LED_o_0[2]}]
12 set_property PACKAGE_PIN BB32 [get_ports {LED_o_0[3]}]
13 set_property PACKAGE_PIN BF32 [get_ports {LED_o_0[4]}]
14 set_property PACKAGE_PIN AU37 [get_ports {LED_o_0[5]}]
15 set_property PACKAGE_PIN AV36 [get_ports {LED_o_0[6]}]
16 set_property PACKAGE_PIN BA37 [get_ports {LED_o_0[7]}]
17 set_property PACKAGE_PIN AY24 [get_ports sysclk_125_clk_p]
18 set_property PACKAGE_PIN AY23 [get_ports sysclk_125_clk_n]
19 set_property IOSTANDARD LVDS [get_ports sysclk_125_clk_p]
20 set_property IOSTANDARD LVCMOS12 [get_ports reset]
21
22
23 set_property BITSTREAM.GENERAL.COMPRESS TRUE [current_design]
24 set_property C_CLK_INPUT_FREQ_HZ 300000000 [get_debug_cores dbg_hub]
25 set_property C_ENABLE_CLK_DIVIDER false [get_debug_cores dbg_hub]
26 set_property C_USER_SCAN_CHAIN 1 [get_debug_cores dbg_hub]
27 connect_debug_port dbg_hub/clk [get_nets clk]
```

שלב 14:

בכל שינוי בקוד ה-IP יש לחזור על אריזת הרכיב ב-Re-Package IP, לעדכן אותו ב-Block Design דרך Report IP Status ו-Upgrade Selected, ולבצע Reset Output Products לפני יצירת Bitstream חדש. כך הסינתזה משתמשת בגרסה העדכנית של הרכיב.

נלחץ Generate Bitstream.

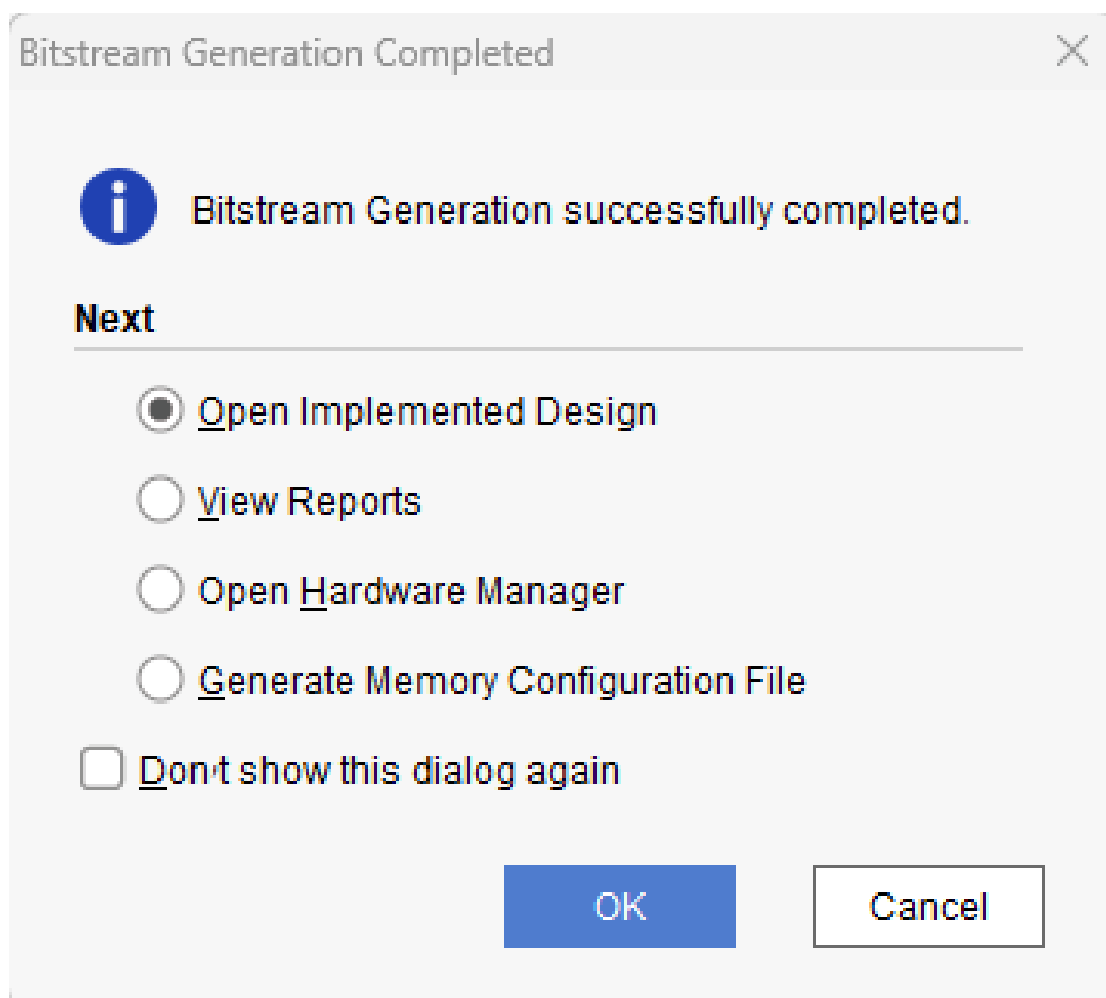
▼ PROGRAM AND DEBUG



Generate Bitstream

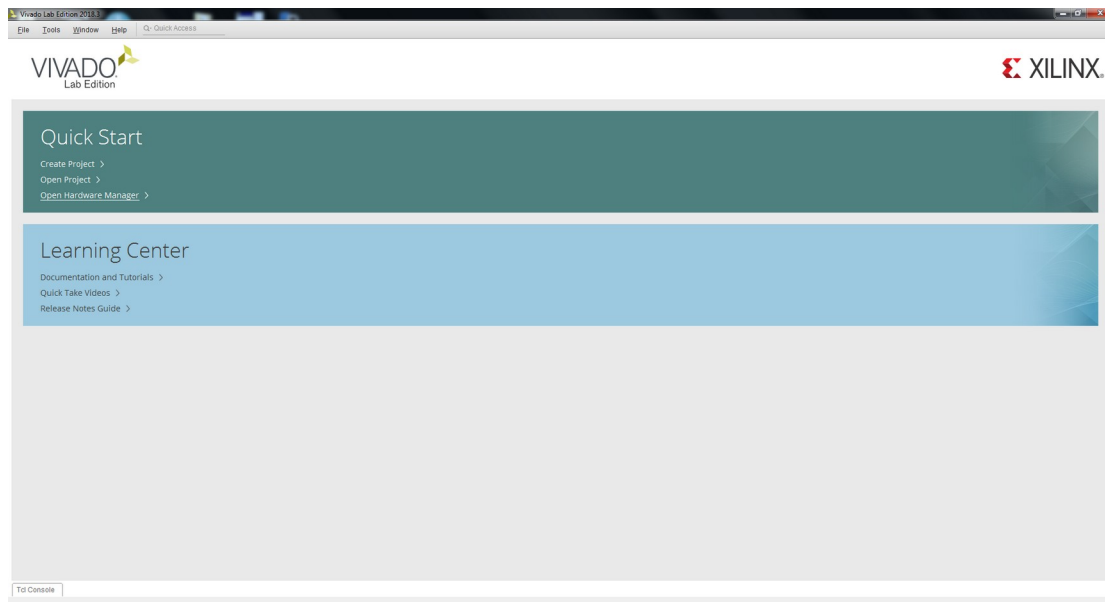
> Open Hardware Manager

יצירת ה-Bitstream הסתיימה בהצלחה.

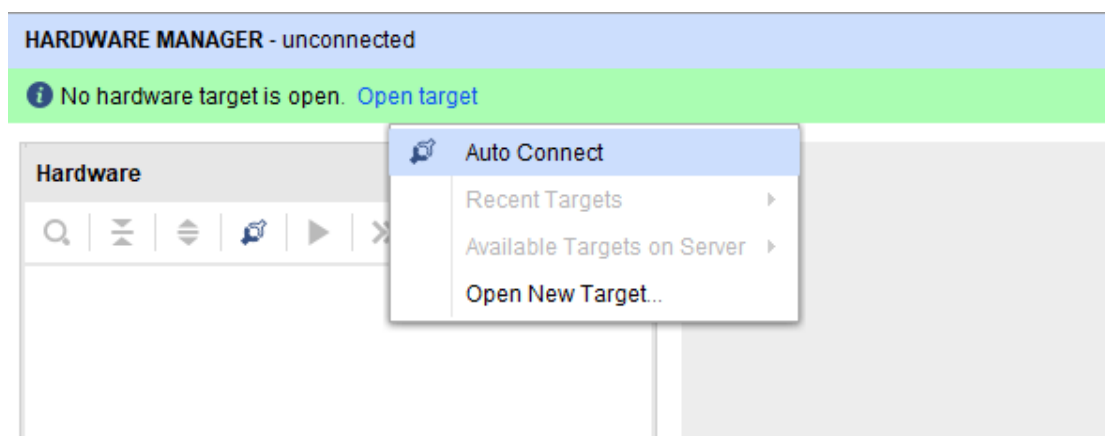


שלב 15:

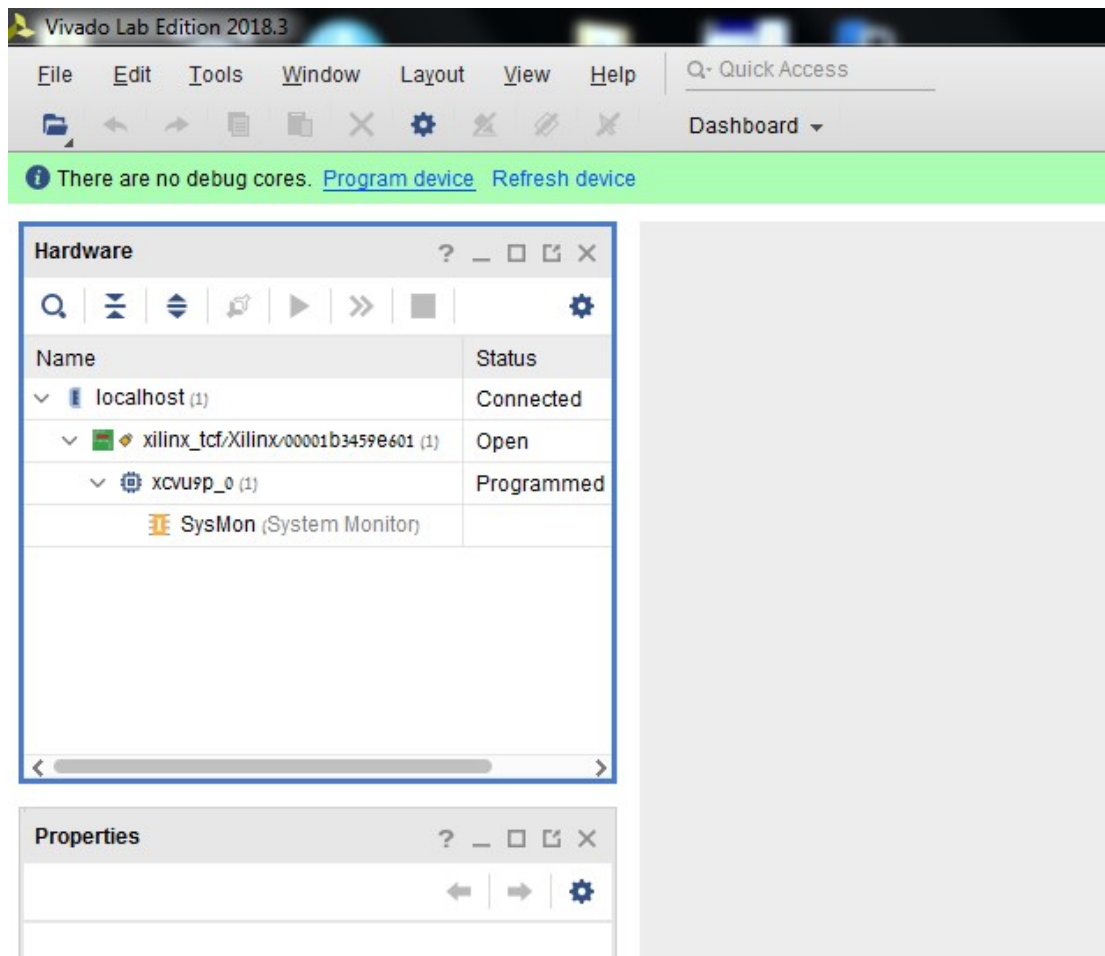
נפתח את Hardware Manager.



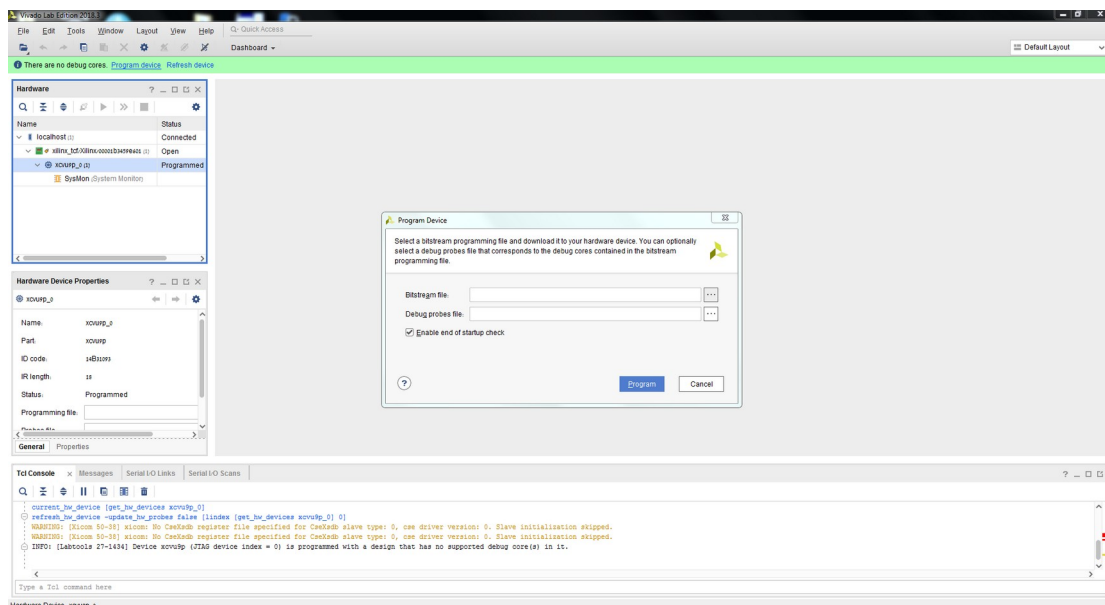
נבחר Open Target ואז Auto Connect.



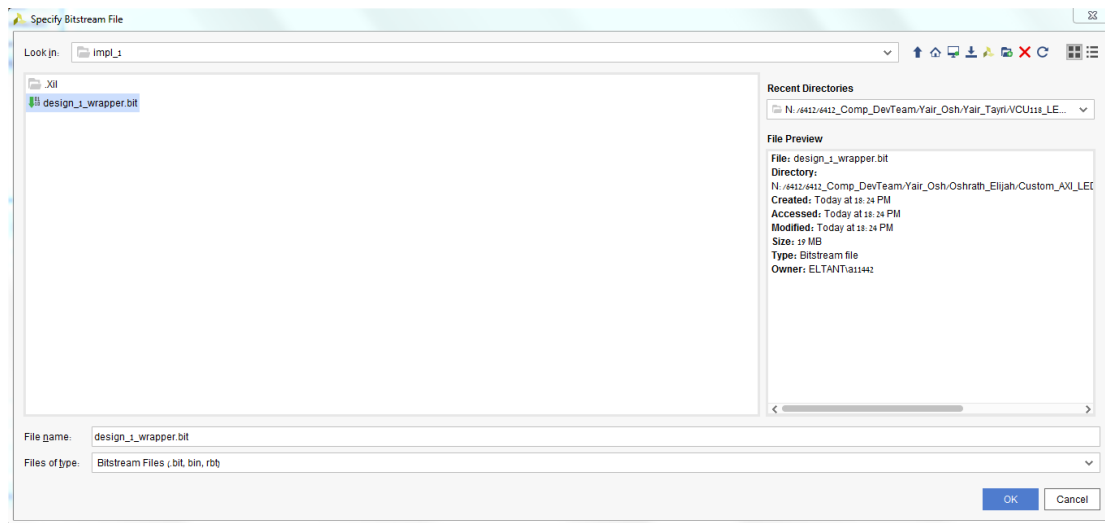
הלוח מזוהה. נבחר Program Device.



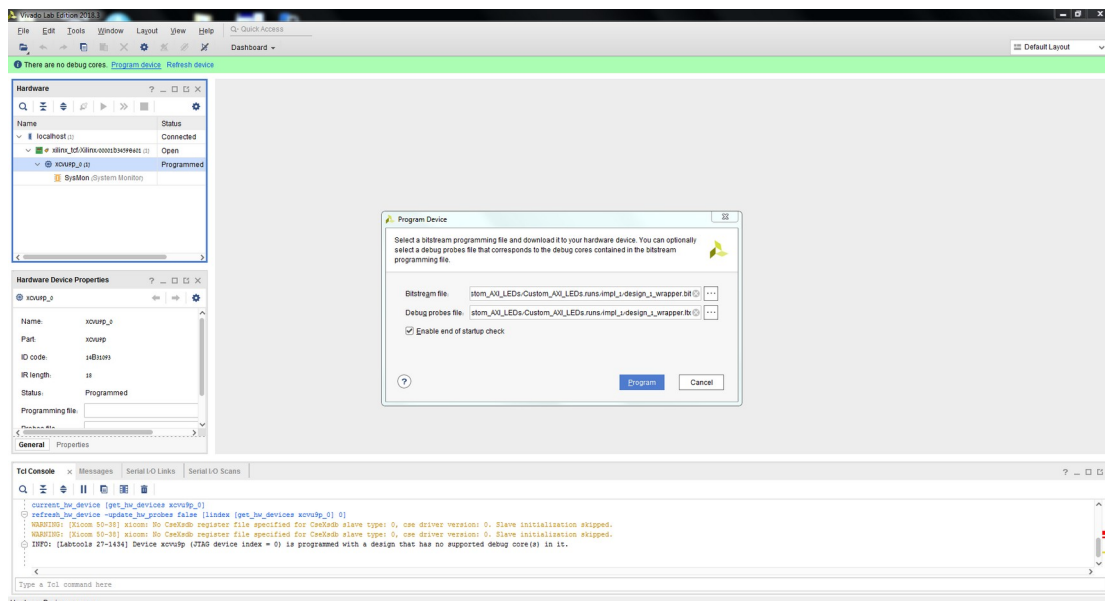
נבחר את קובץ ה-Bitstream.



נאתר את הקובץ בתיקיית הפרויקט.



נלחץ Program.

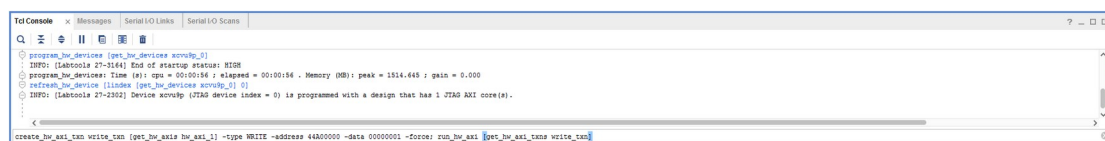


שלב 16:

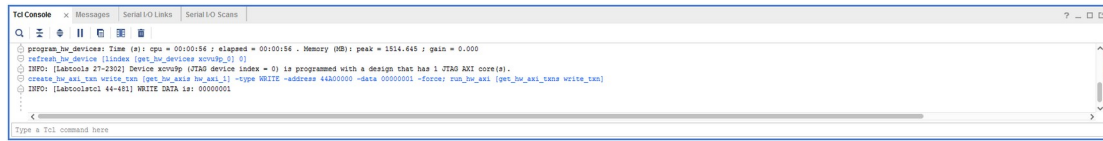
כברירת מחדל הלדים פועלים במצב ריצה. כדי לעבור למצב נשימה נכתוב 1 לרגיסטר 0. נקליד ב-Tcl Console:

```
create_hw_axi_txn write_txn [get_hw_axis hw_axi_1] -type WRITE -address 44A00000 -data 00000001 -force; run_hw_axi [get_hw_axi_txn write_txn]
```

הפקודה בחלון ה-Tcl Console:



הכתיבה בוצעה והלדים עוברים למצב נשימה:



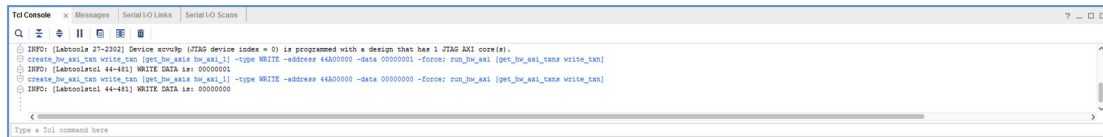
כדי לחזור למצב ריצה נכתוב 0 לאותו רגיסטר:

```
create_hw_axi_txn write_txn [get_hw_axis hw_axi_1] -type WRITE -address 44A00000 -data 00000000 -force; run_hw_axi [get_hw_axi_txns write_txn]
```

הפקודה בחלון ה-Tcl Console:



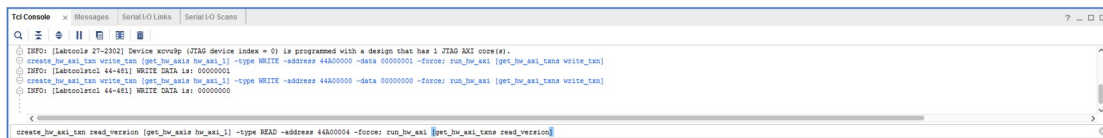
הכתיבה בוצעה והלדים חוזרים למצב ריצה:



לבדיקת ערוץ הקריאה של ה-AXI נקרא את רגיסטר 1 בהיסט 0x04 מכתובת הבסיס:

```
create_hw_axi_txn read_version [get_hw_axis hw_axi_1] -type READ -
address 44A00004 -force; run_hw_axi [get_hw_axi_txns read_version]
```

הפקודה בחלון ה-Tcl Console:



הפקודה מחזירה את הערך 20251123, התואם את ערך הגרסה הקשיח שהוגדר ברכיב.



Chapter 4: System Integration and Software Control via JTAG-UART on VCU118

מבוא:

בפרק זה נשלב את רכיב ה-LED המותאם במערכת מבוססת MicroBlaze ונשלט בו מתוכנה בזמן אמת. התקשורת עם המחשב מתבצעת דרך ממשק JTAG-UART של רכיב ה-MDM, ולכן אין צורך ברכיב UART נוסף או בפינים חיצוניים.

דרישות קדם:

חומרה:

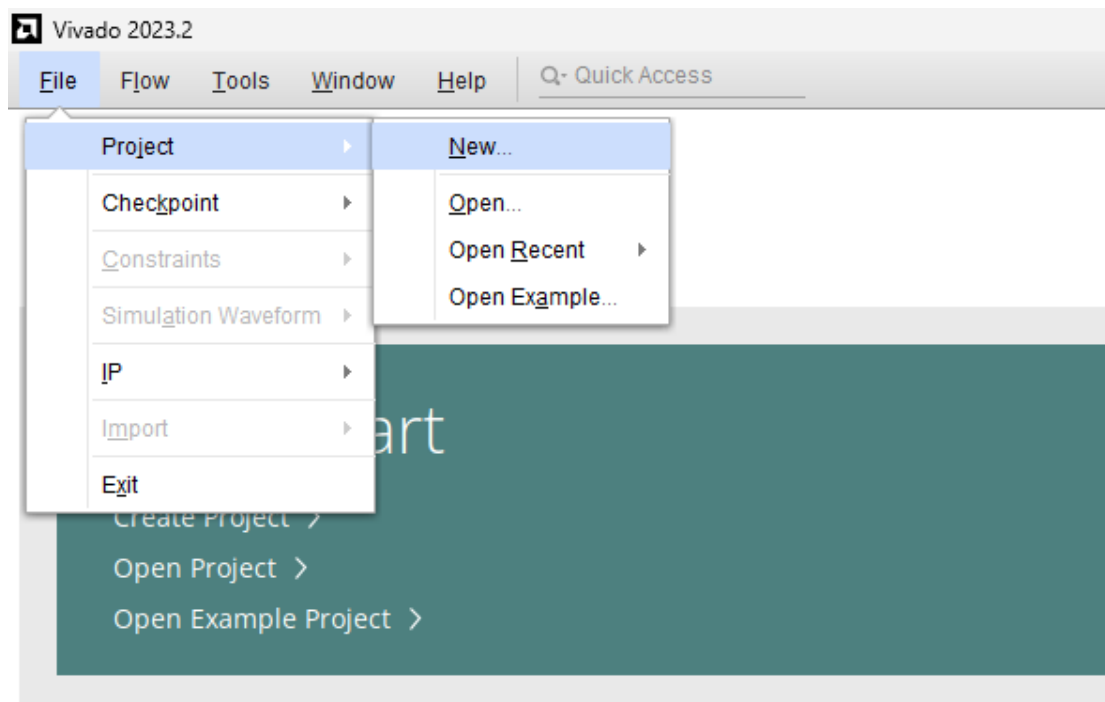
לוח פיתוח Xilinx VCU118
כבל USB-JTAG לצריבה ולתקשורת
ספק כוח 12 וולט ללוח

תוכנה:

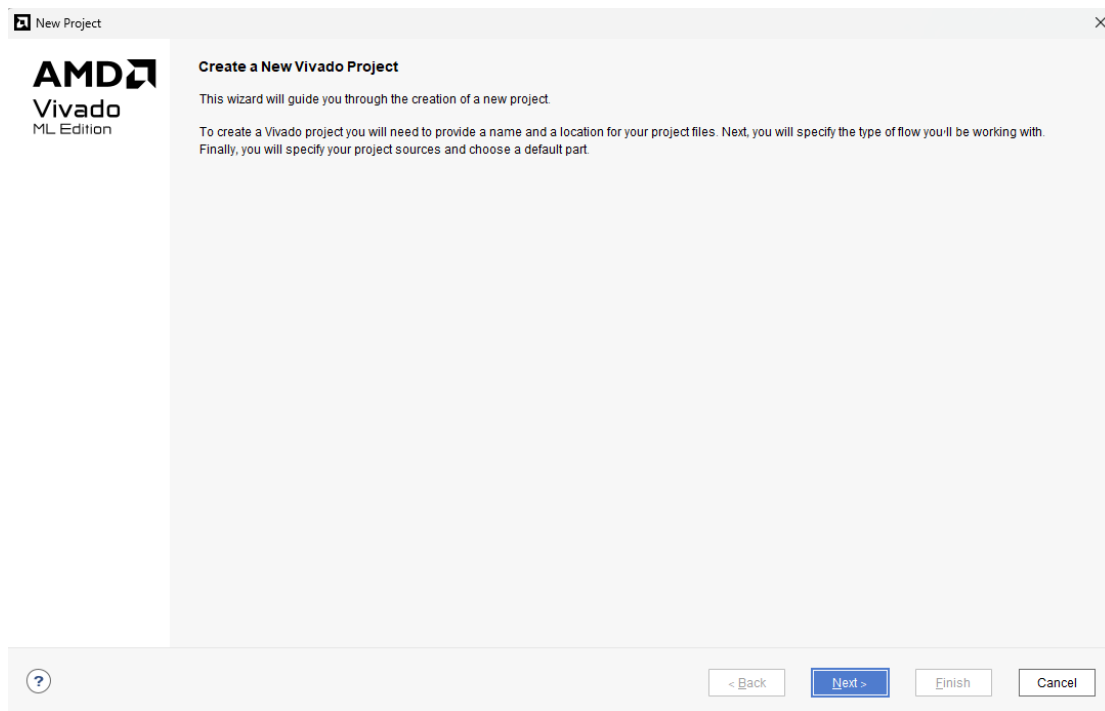
Vivado Design Suite
Vitis Unified IDE
XSCT Console

שלב 1:

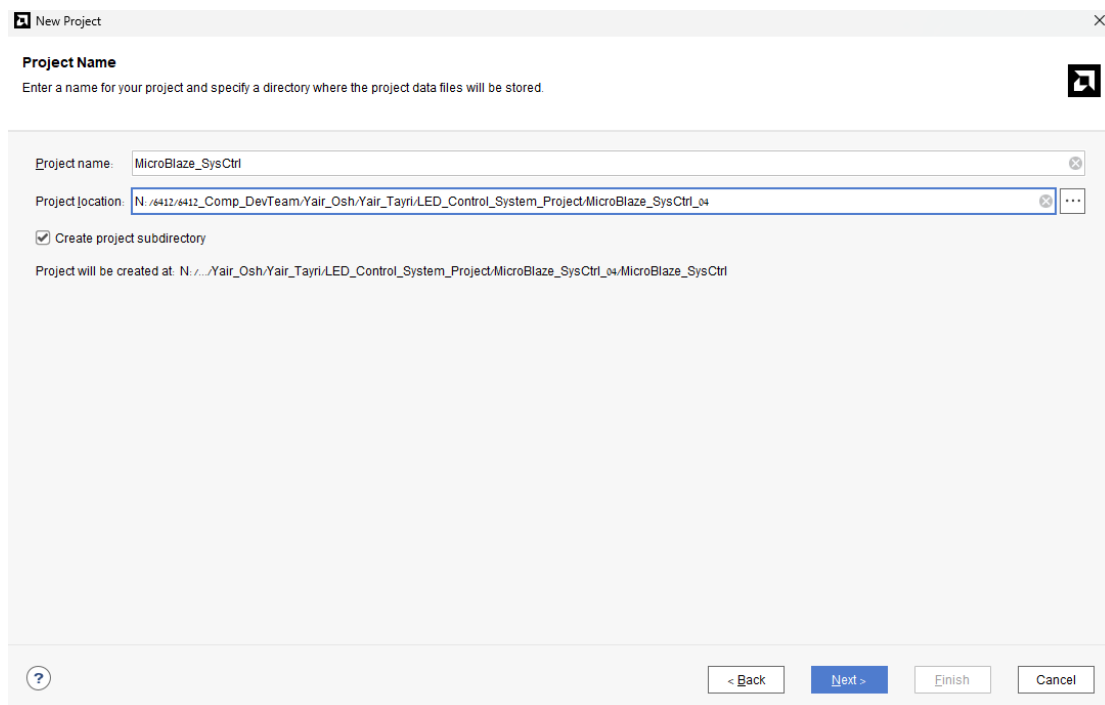
ניצור פרויקט חדש דרך File, Project ואז New.



נלחץ Next.



נקרא לפרויקט MicroBlaze_SysCtrl.



נבחר RTL Project.

New Project

Project Type
Specify the type of project to create.

☒ **RTL Project**
You will be able to add sources, create block designs in IP Integrator, generate IP, run RTL analysis, synthesis, implementation, design planning and analysis.
☒ Do not specify sources at this time
☐ Project is an extensible Vitis platform

☐ **Post-synthesis Project**
You will be able to add sources, view device resources, run design analysis, planning and implementation.
☐ Do not specify sources at this time

☐ **I/O Planning Project**
Do not specify design sources. You will be able to view part/package resources.

☐ **Imported Project**
Create a Vivado project from a Synplify Project File.

☐ **Example Project**
Create a new Vivado project from a predefined template.

[?](#) [< Back](#) [Next >](#) [Finish](#) [Cancel](#)

בלשונית Boards נבחר את לוח ה-VCU118.

New Project

Default Part
Choose a default AMD part or board for your project.

Parts | **Boards**

To fetch the latest available boards from git repository, click on <Refresh> button. [Dismiss](#)

[Reset All Filters](#)

Vendor: Name: Board Rev:

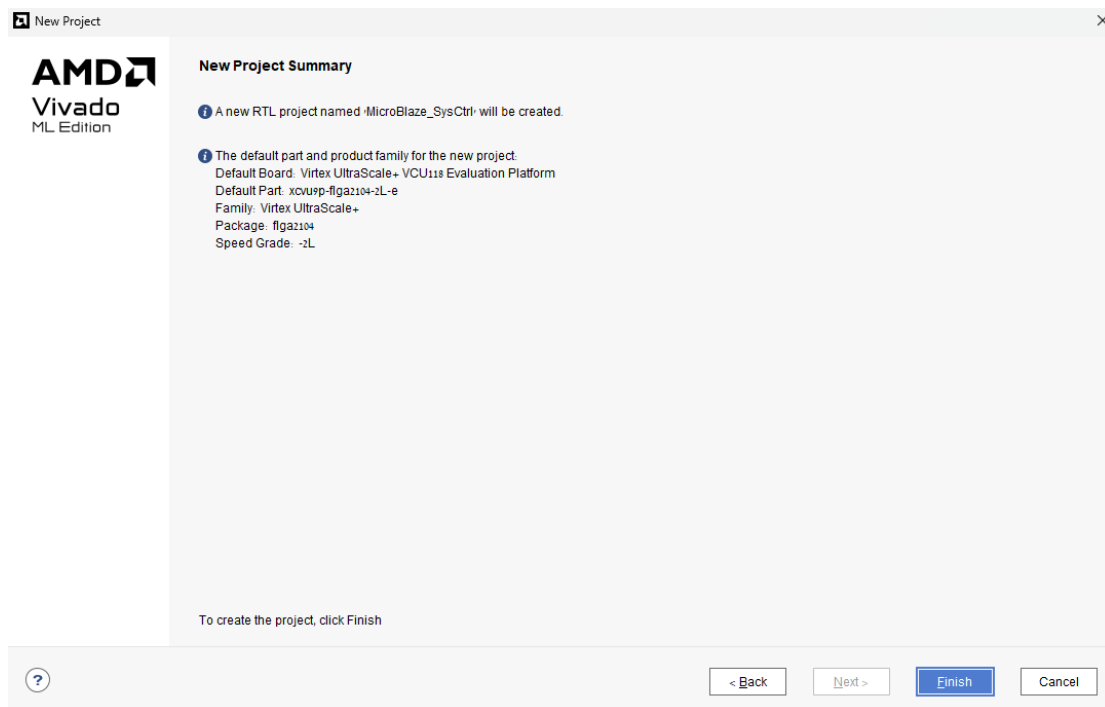
Search: (1 match)

Display Name	Preview	Status	Vendor	File Version	Part	I/O Pin Count
Virtex UltraScale+ VCU118 Evaluation Platform		Installed	xilinx.com	2.4	xcvuup-flga2104-2L-e	2104

[Refresh](#)

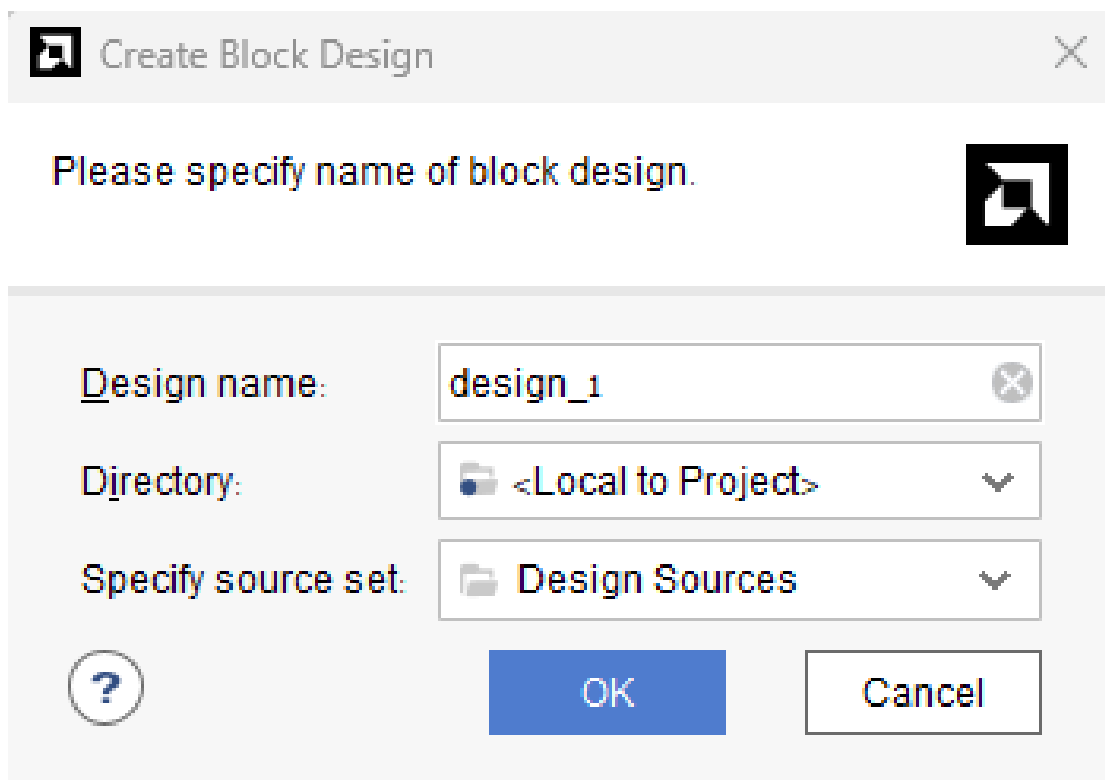
[?](#) [< Back](#) [Next >](#) [Finish](#) [Cancel](#)

נלחץ Finish.

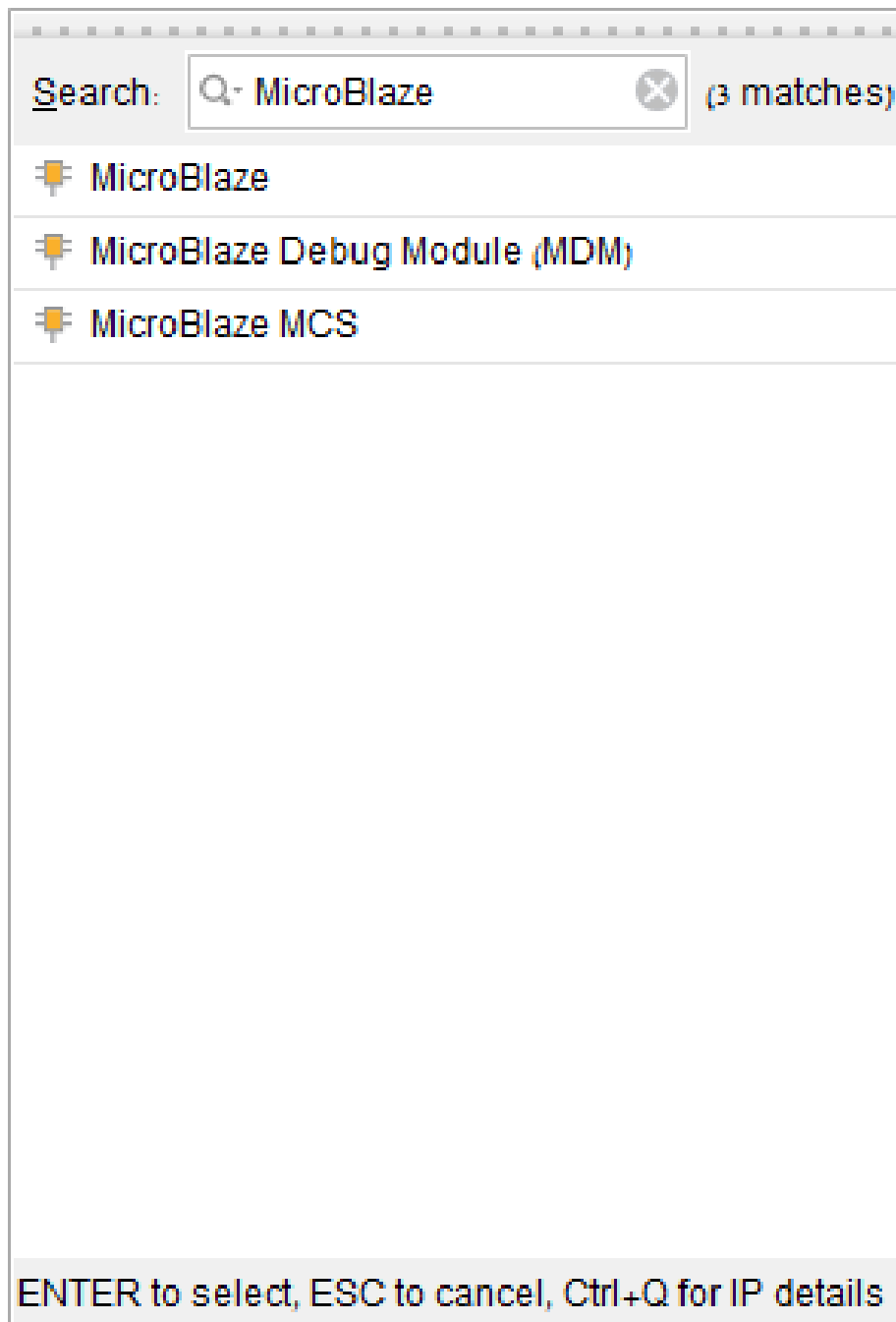


שלב 2:

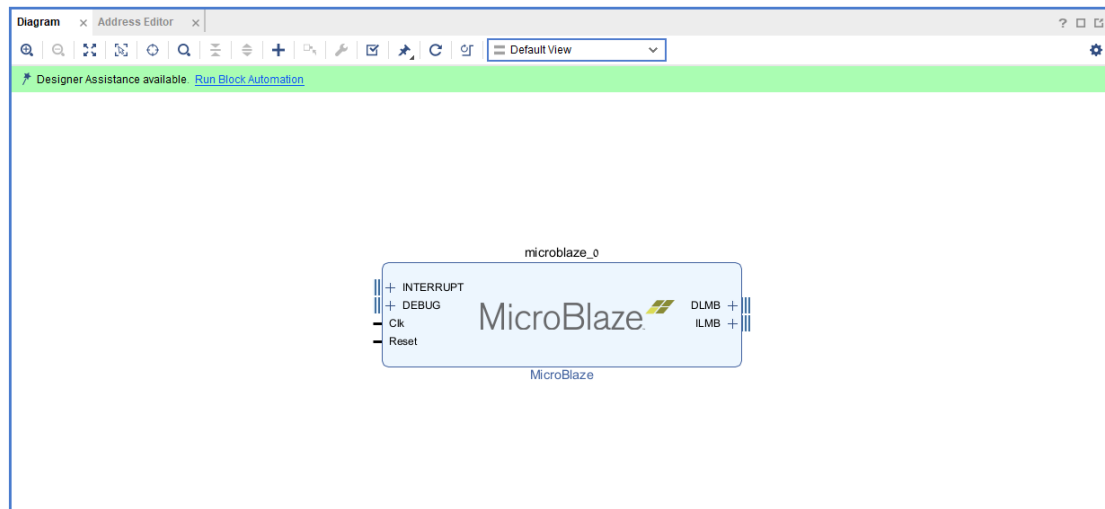
ניצור Block Design.



נוסיף רכיב MicroBlaze.

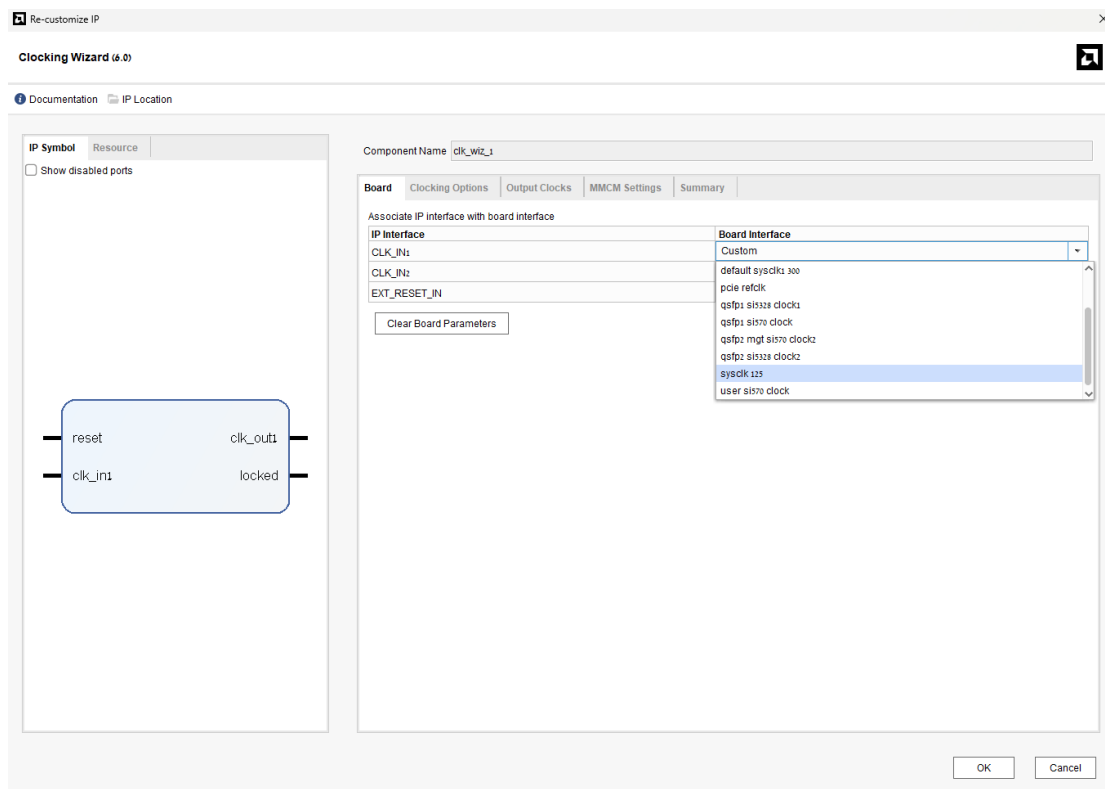


הרכיב נוסף לשרטוט.



שלב 3:

נוסוף Clocking Wizard. בלשונית Board נשנה את CLK_IN1 מ-Custom ל-sysclk 125 ואת EXT_RESET_IN ל-reset.



בלשונית Output Clocks נסיר את הסימון מכניסת ה-reset. רכיב ה-LED המותאם מסונכרן לתדר 125MHz, בהתאם להגדרות מחלק התדר שבתוכו.

Re-customize IP

Clocking Wizard (6.0)

Documentation IP Location

IP Symbol Resource

☐ Show disabled ports

Component Name clk_wiz_1

Board	Clocking Options	Output Clocks	MMCM Settings	Summary
☐ clk_out0	clk_out0	100.000	N/A	0.000
☐ clk_out1	clk_out1	100.000	N/A	0.000
☐ clk_out2	clk_out2	100.000	N/A	0.000
☐ clk_out3	clk_out3	100.000	N/A	0.000
☐ clk_out4	clk_out4	100.000	N/A	0.000
☐ clk_out5	clk_out5	100.000	N/A	0.000
☐ clk_out6	clk_out6	100.000	N/A	0.000

☐ USE CLOCK SEQUENCING

Output Clock	Sequence Number
clk_out0	1
clk_out1	1
clk_out2	1
clk_out3	1
clk_out4	1
clk_out5	1
clk_out6	1

Enable Optional Inputs / Outputs for MMCM/PLL

☐ reset ☐ power_down ☐ input_clk_stopped ☒ locked ☐ clkfbstopped

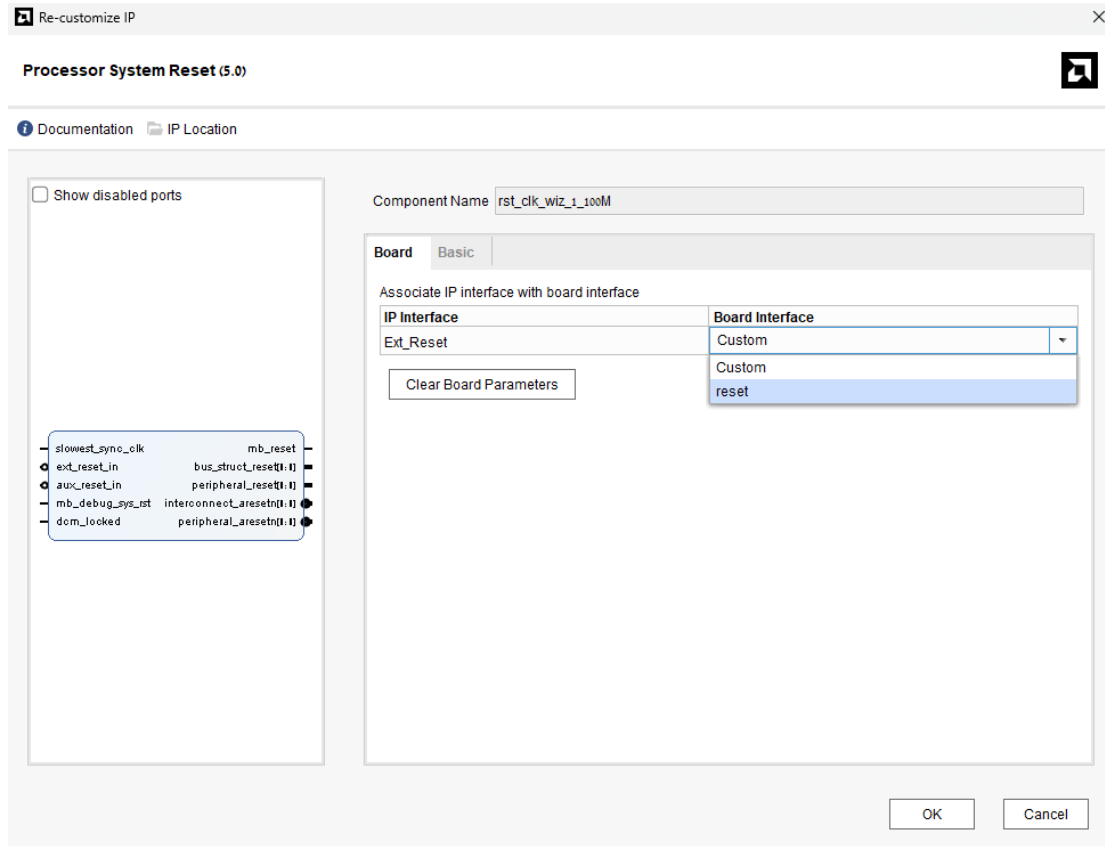
Reset Type: ☒ Active High ☐ Active Low

Phase Shift Mode: ☐ WAVEFORM ☒ LATENCY

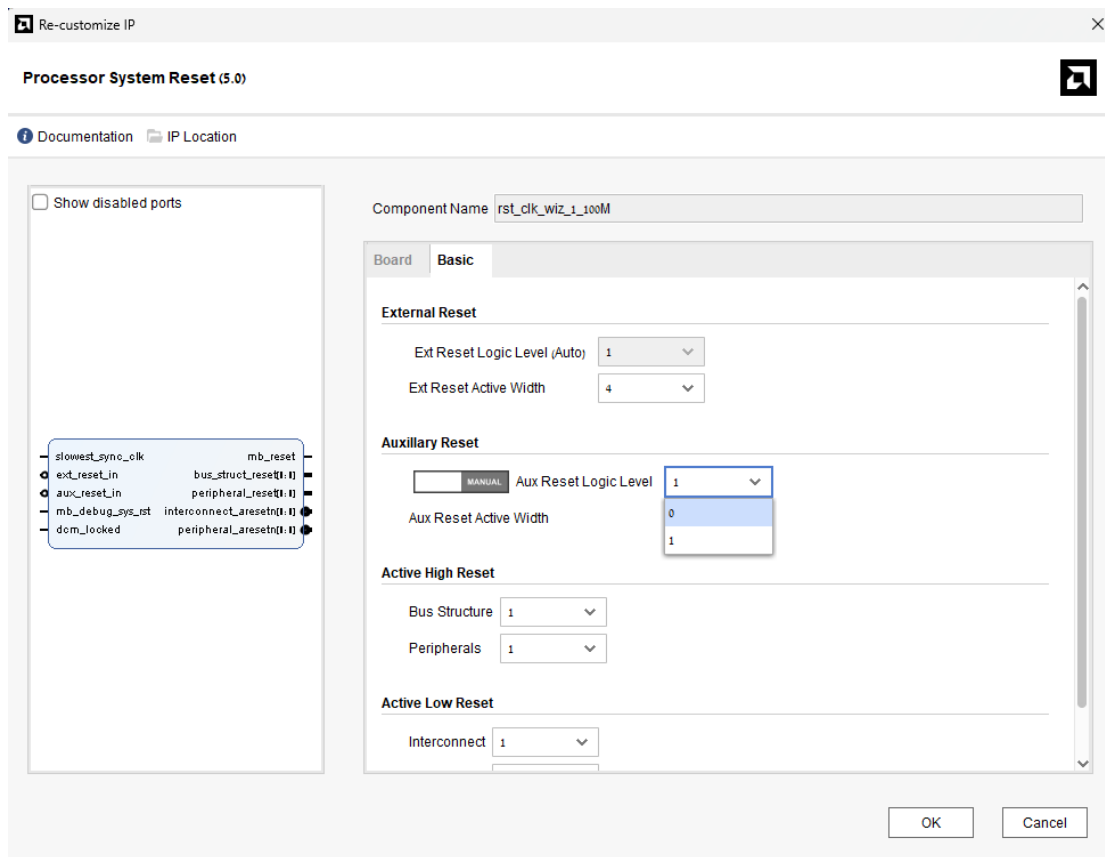
OK Cancel

שלב 4:

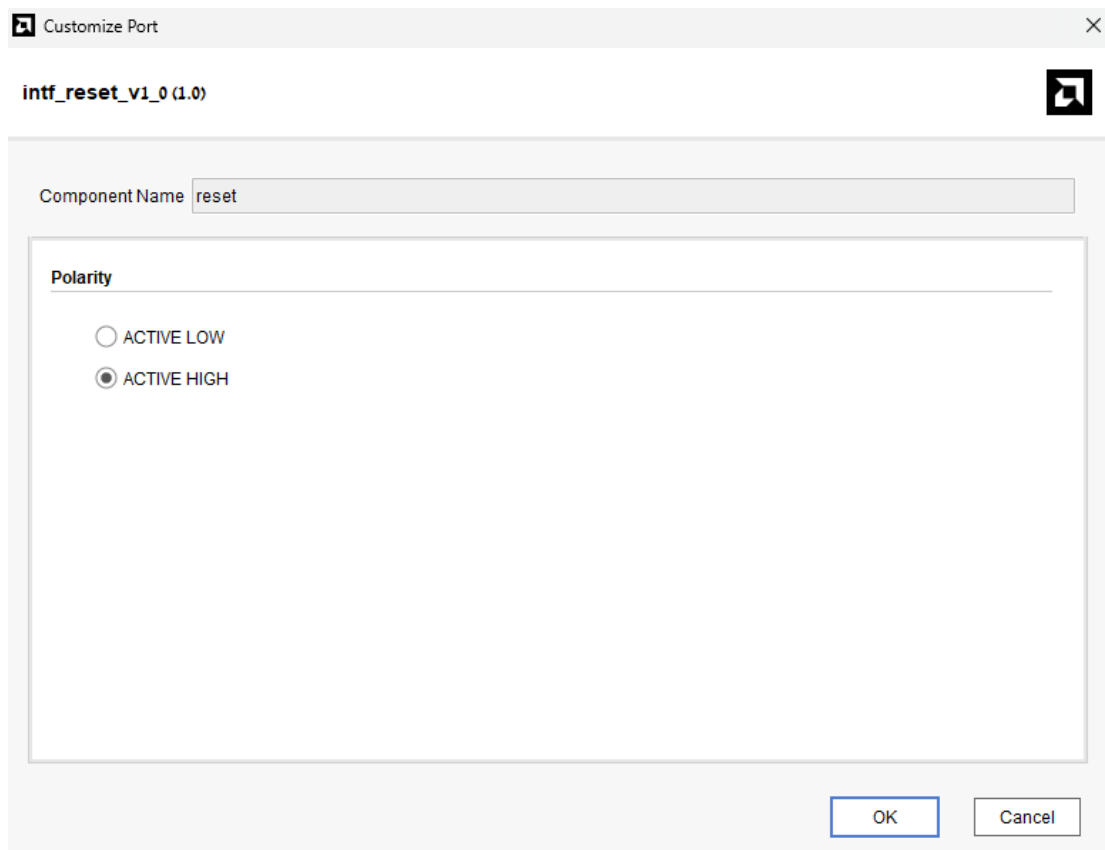
נוסוף Processor System Reset ונשייך את Ext_Reset ל-reset.



בלשונית Basic נגדיר את מאפייני האיפוס.

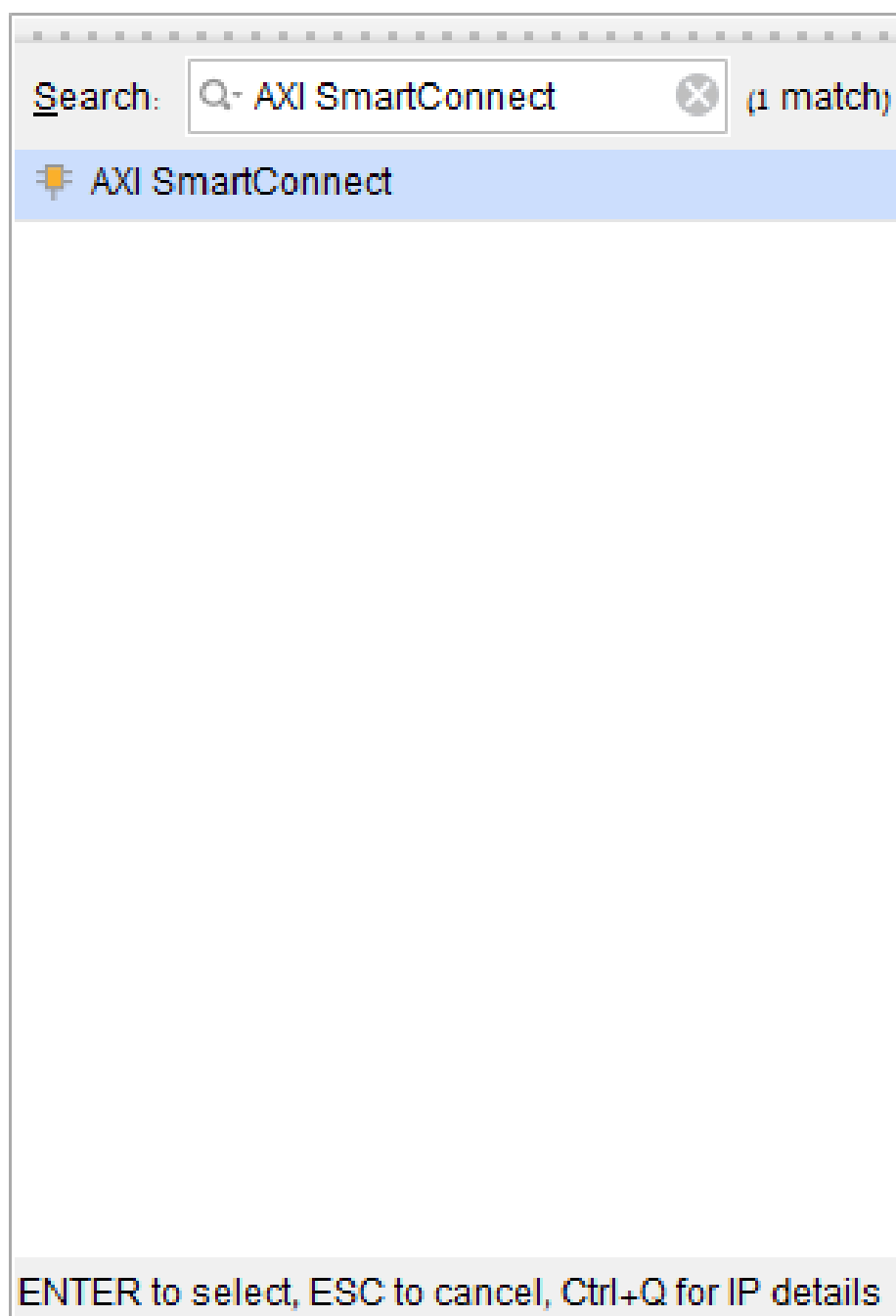


נגדיר את קוטביות האיפוס ל-ACTIVE HIGH.

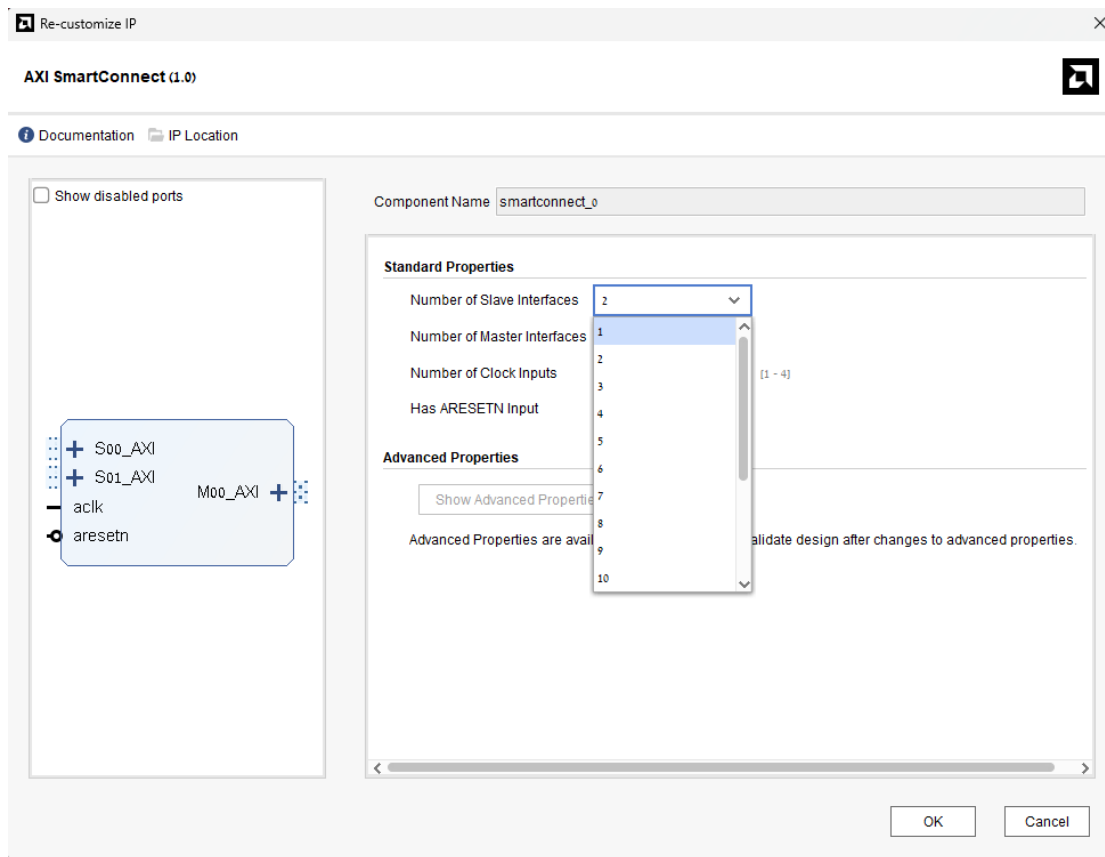


שלב 5:

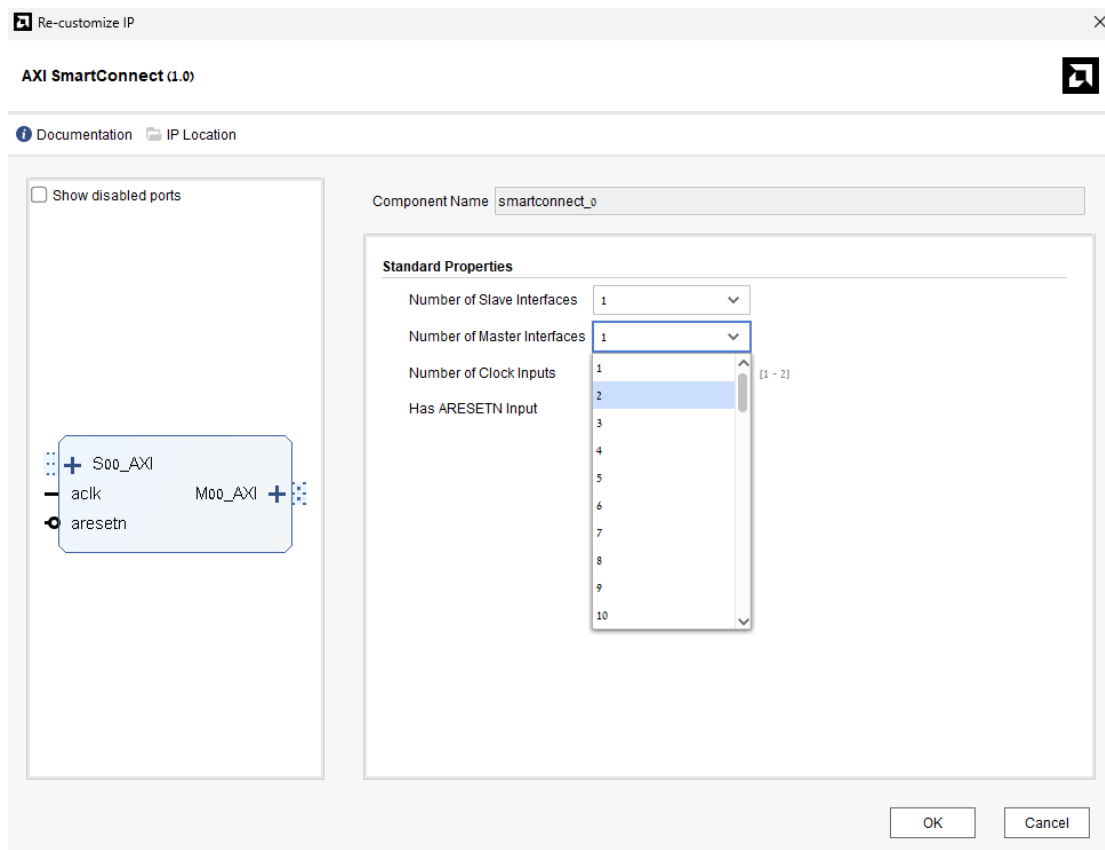
נוסיף רכיב AXI SmartConnect.



נגדיר את מספר ממשקי ה-Slave.

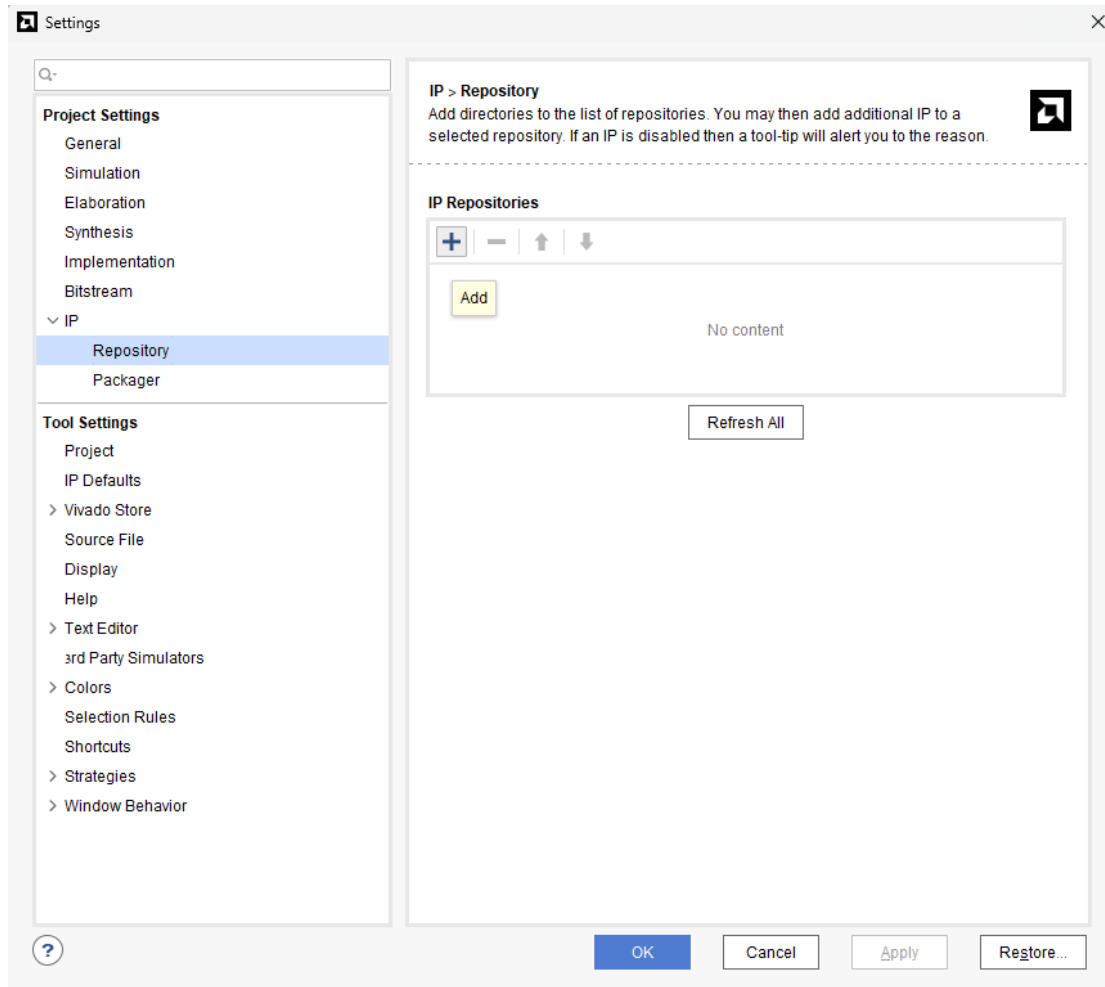


נגדיר את מספר ממשקי ה-Master.

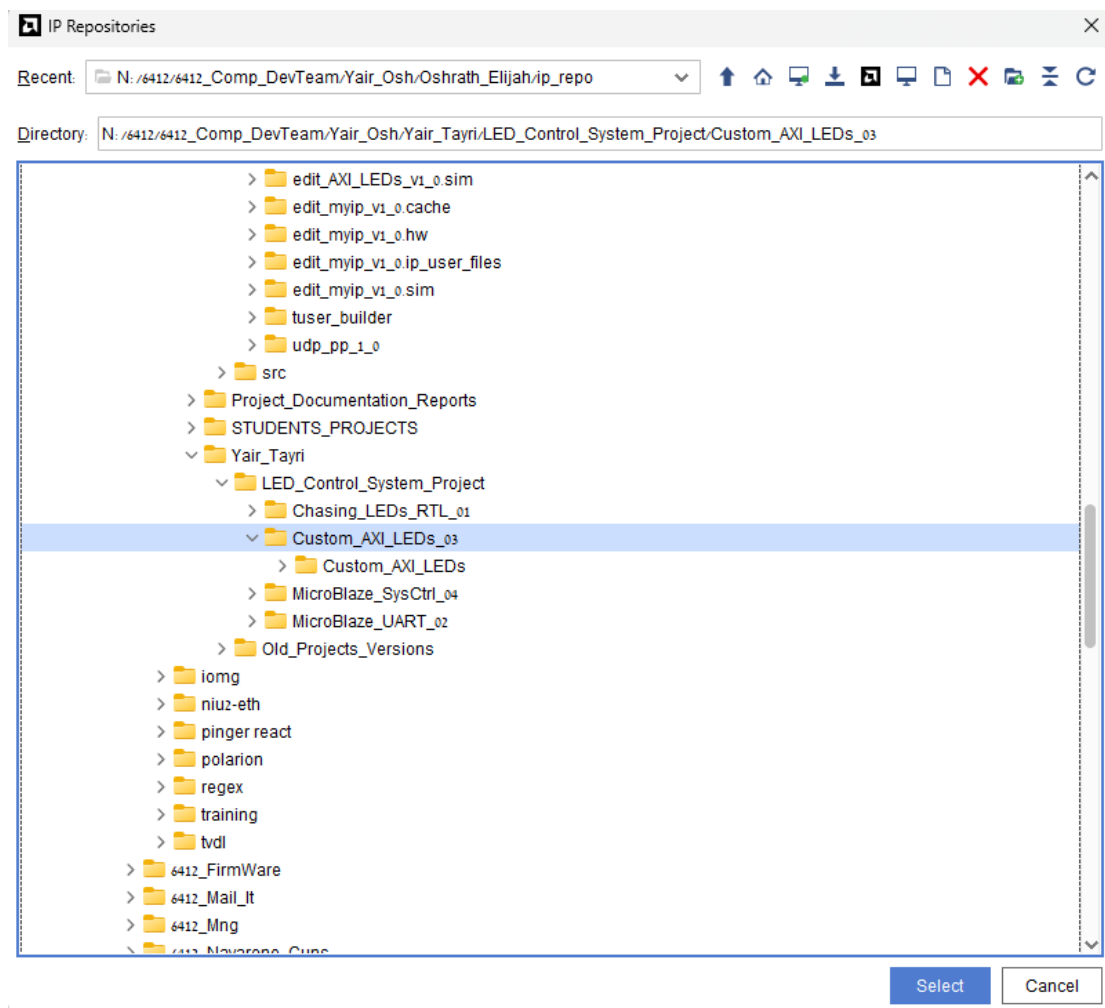


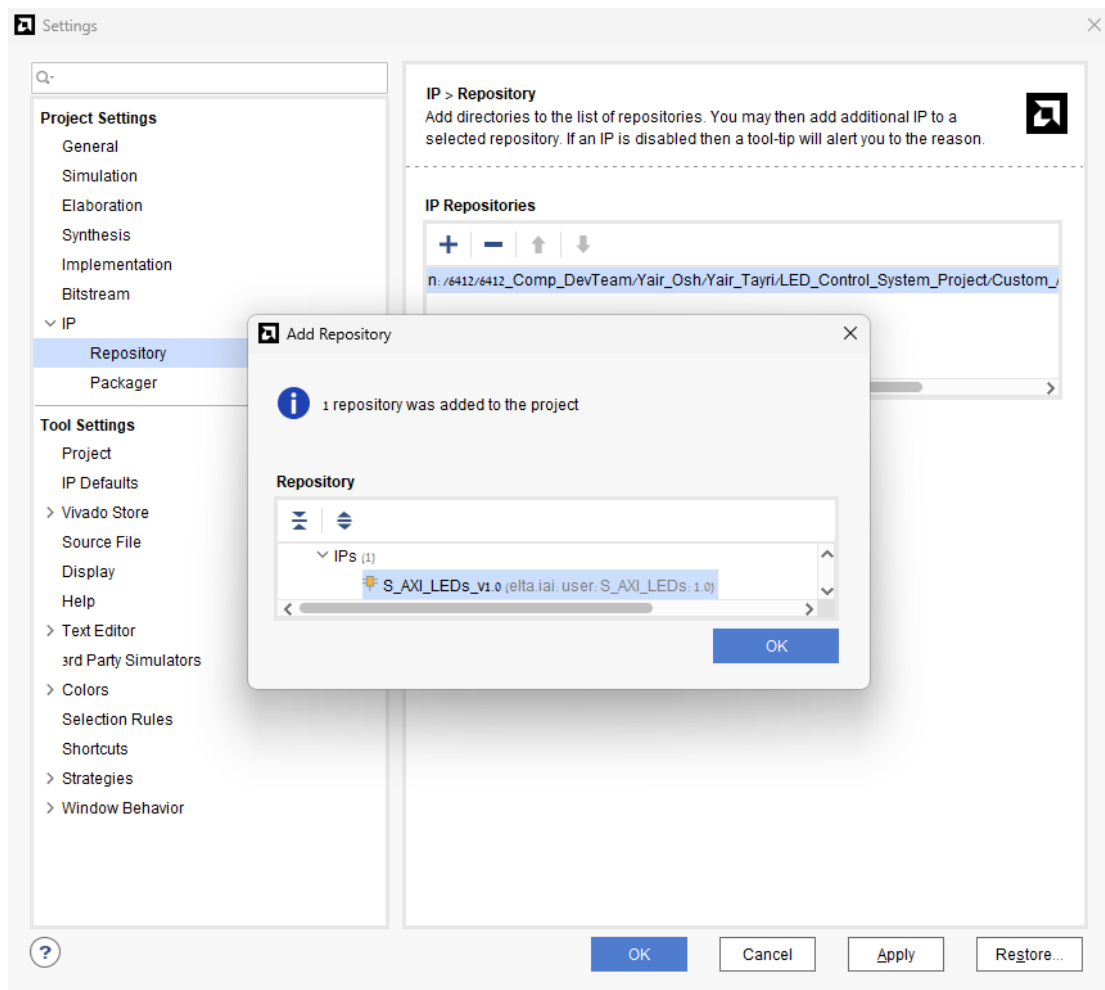
שלב 6:

ב-Settings תחת IP ואז Repository נלחץ על סימן הפלוס.

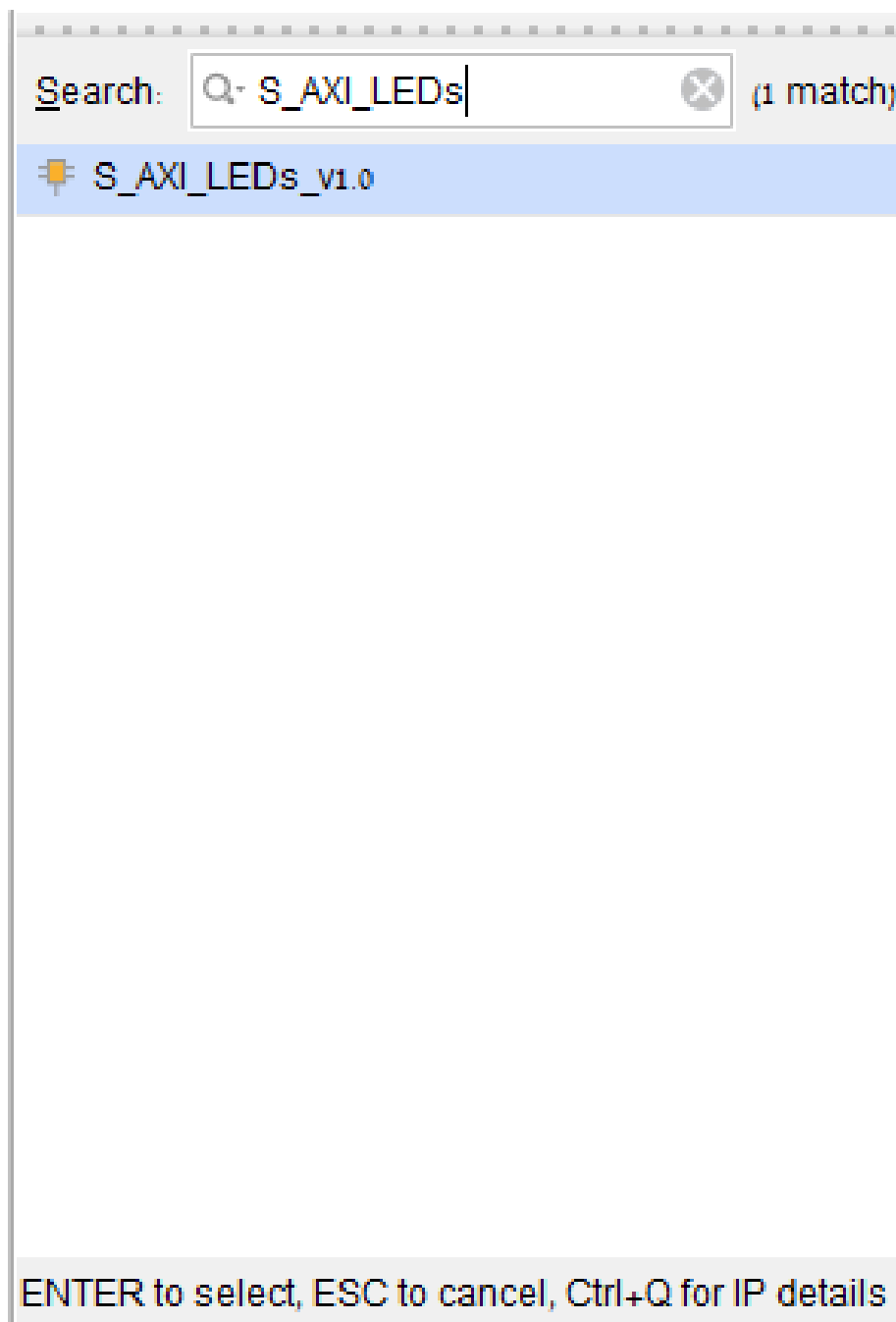


נבחר את תיקיית ה-`ip_repo` של הרכיב המותאם.





נוסיף את S_AXI_LEDs ל-Block Design.



שלב 7:

נייצר את ה-Output Products של התכן.

 Generate Output Products ✕

The following output products will be generated. 

Preview

▼  design_1.bd (OOC per IP)

☐ Synthesis

☐ Simulation

☐ Implementation

☐ Low Power Handoff

Synthesis Options

☐ Global

☒ Out of context per IP

☐ Out of context per Block Design

Run Settings

☒ On local host: Number of jobs: 16 ▼

☐ Generate scripts only

☐ Do not launch



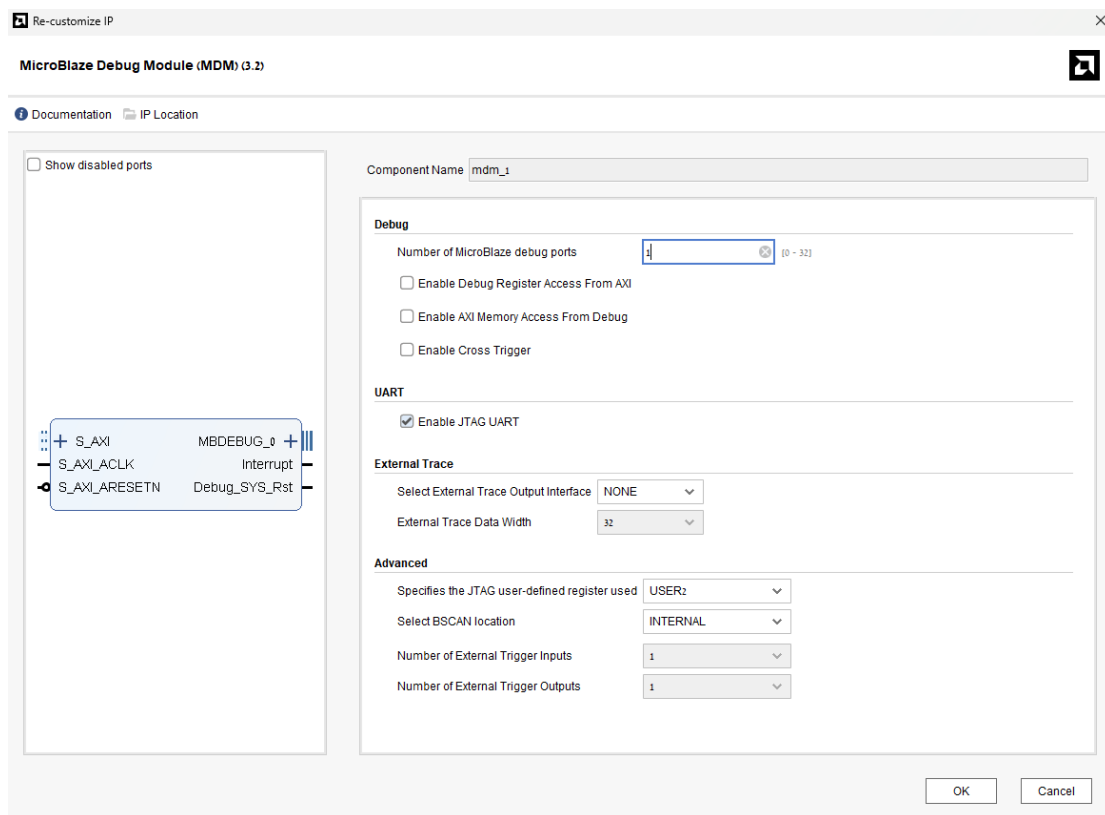
Apply

Generate

Skip

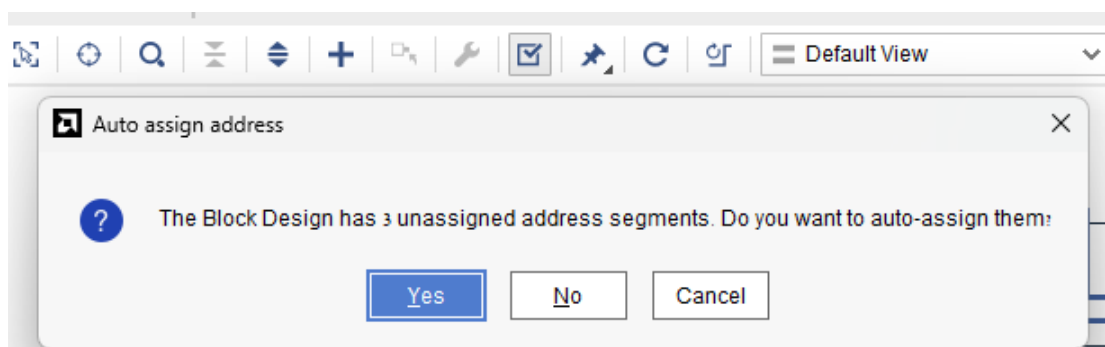
שלב 8:

בהגדרות רכיב ה-MDM נסמן Enable JTAG UART. הגדרה זו מממשת ערוץ UART על גבי חיבור ה-JTAG הקיים.

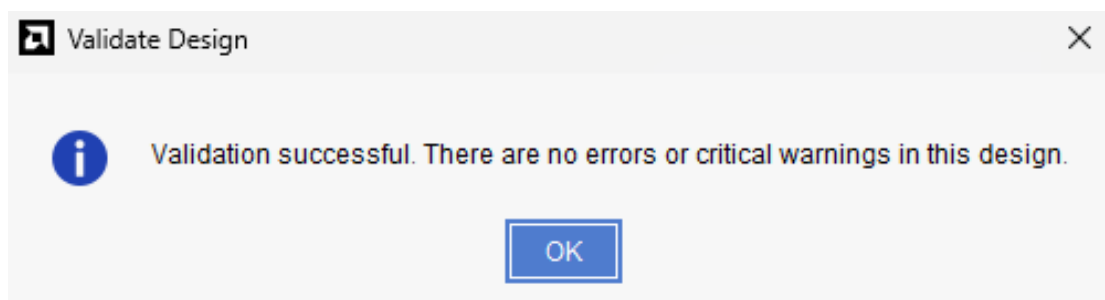


שלב 9:

נאשר הקצאת כתובות אוטומטית.



נריץ Validate Design. האימות עבר ללא שגיאות.



שלב 10:

בחלון Address Editor נעדכן את טווחי הכתובות לערכים שבטבלה.

Address Editor

MicroBlaze_SysCtrl.xdc | design_1_wrapper.vhd

Q

Filter

Assigned (4)

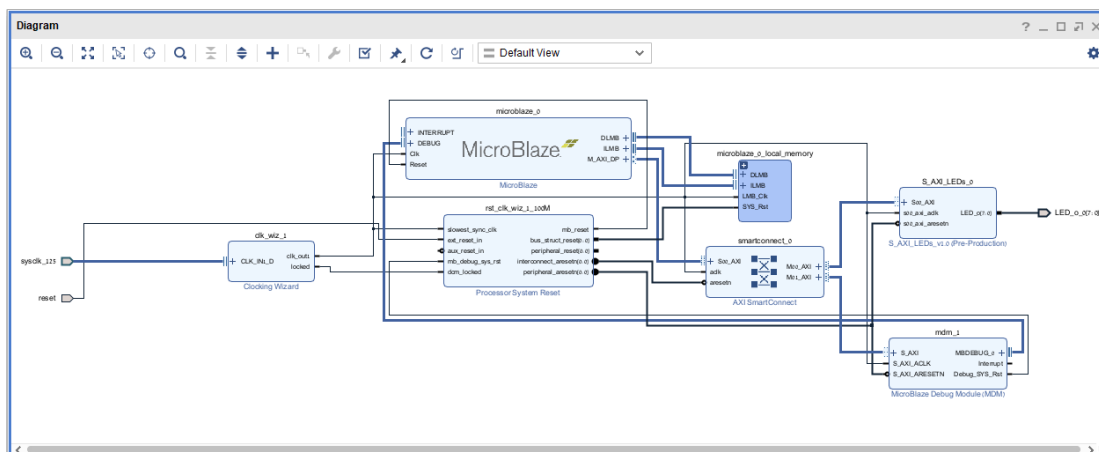
Unassigned (0)

Excluded (0)

Hide All

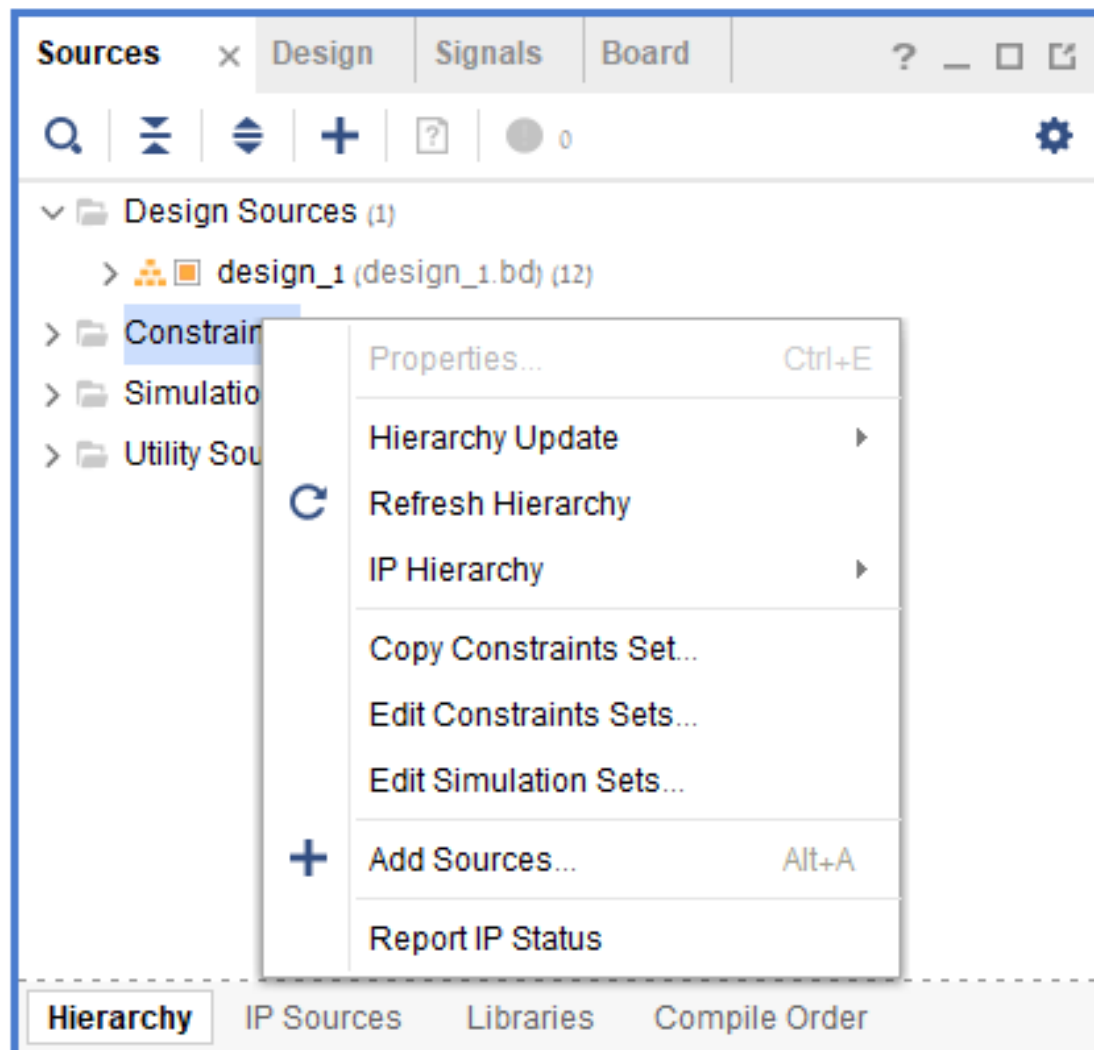
Name	Interface	Slave Segment	Master Base Address	Range	Master High Address
▼ Network 0					
▼ /microblaze_0					
▼ /microblaze_0/Data (32 address bits - 4G)					
/microblaze_0/local_memory/dlmb_bram_if_cntlr/SLMB	SLMB	Mem	0x0	16K	0x3FFF
/S_AXI_LEDS_0/S_AXI	Svc_AXI	Svc_AXI_reg	0x44A0_0000	128K	0x44A1_FFFF
/mdm_1/S_AXI	S_AXI	Reg	0x4140_0000	4K	0x4140_0FFF
▼ Network 1					
▼ /microblaze_0					
▼ /microblaze_0/Instruction (32 address bits - 4G)					
/microblaze_0/local_memory/ilmb_bram_if_cntlr/SLMB	SLMB	Mem	0x0	16K	0x3FFF

זהו התכן המלא:

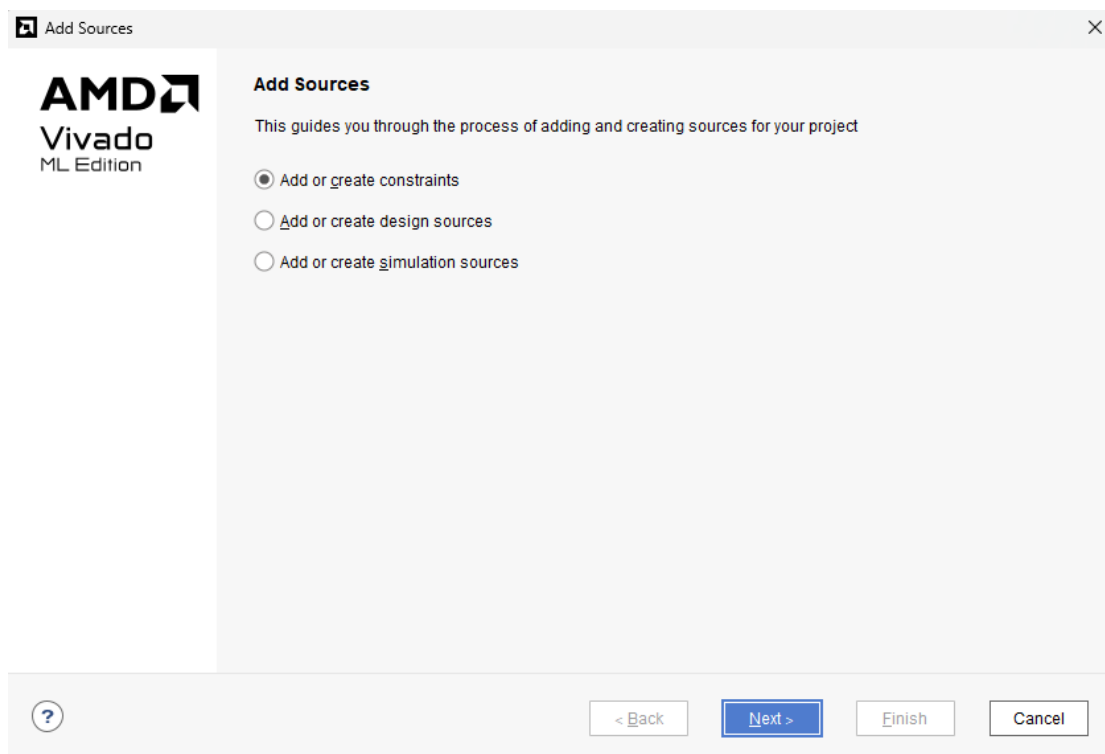


שלב 11:

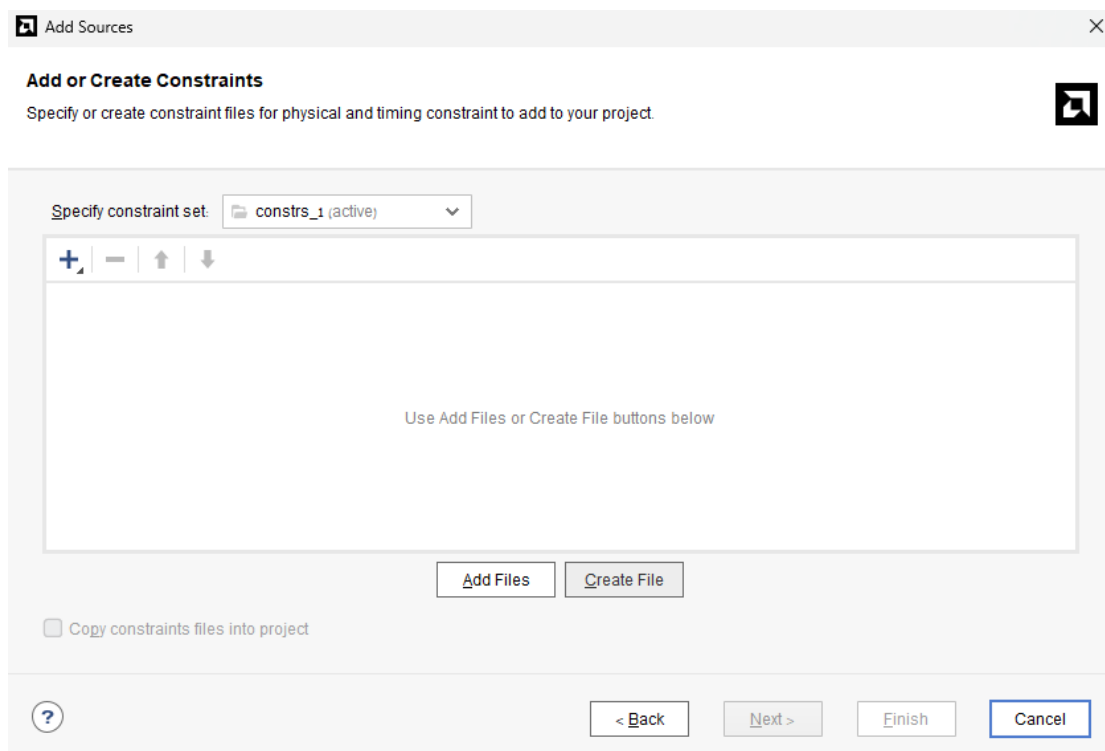
נוסיף קובץ אילוצים דרך Add Sources.



נבחר Add or create constraints.



נלחץ Create File.



ניצור קובץ XDC בשם MicroBlaze_SysCtrl ונלחץ Finish.

Add Sources

Add or Create Constraints

Specify or create constraint files for physical and timing constraint to add to your project.

Specify constraint set:

constrs_1 (active)

Constraint File	Location
MicroBlaze_SysCtrl.xdc	<Local to Project>

Add Files

Create File

☐ Copy constraints files into project

?

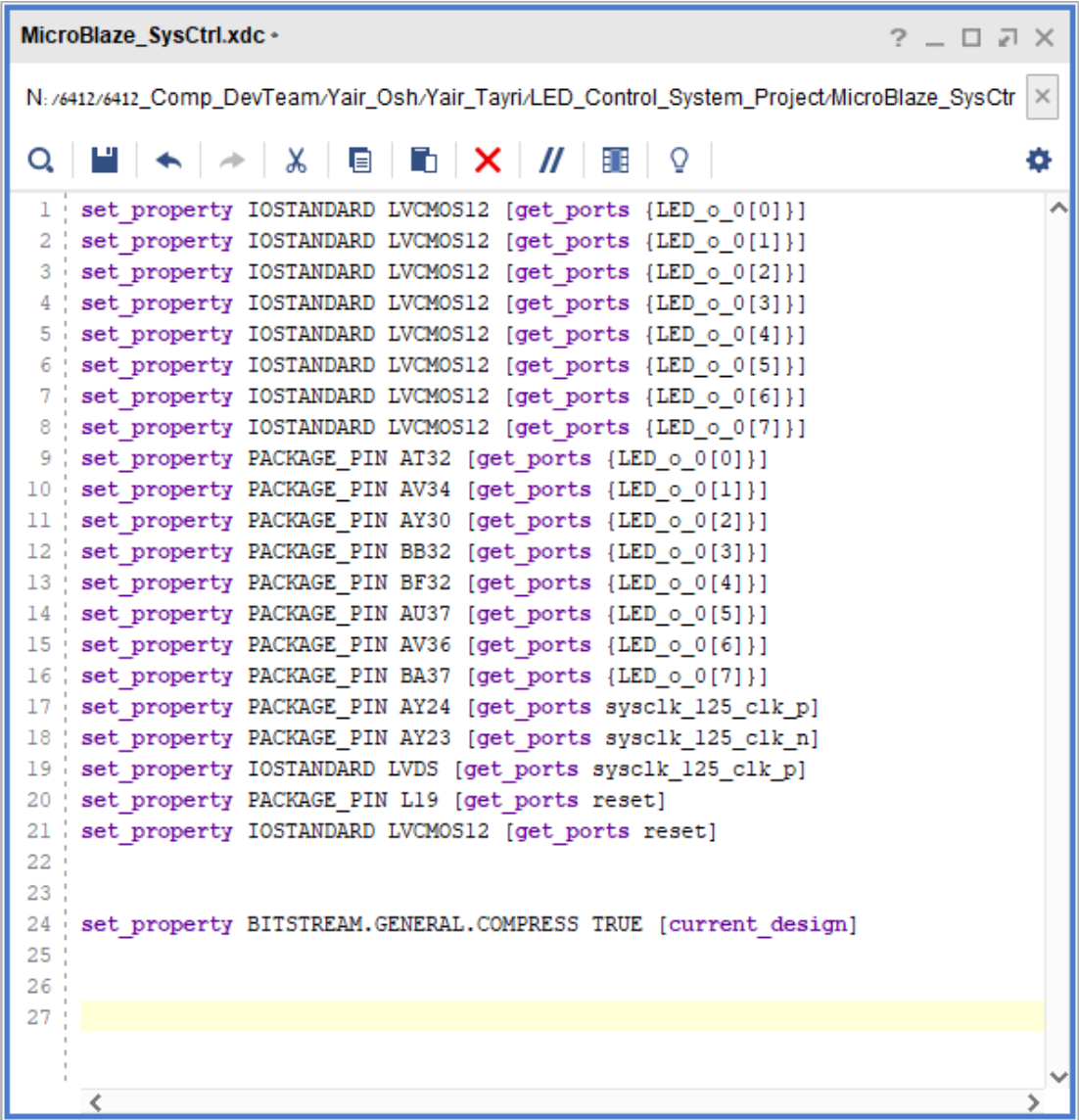
< Back

Next >

Finish

Cancel

בקובץ נמפה את הלדים, את השעון ואת כניסת האיפוס.

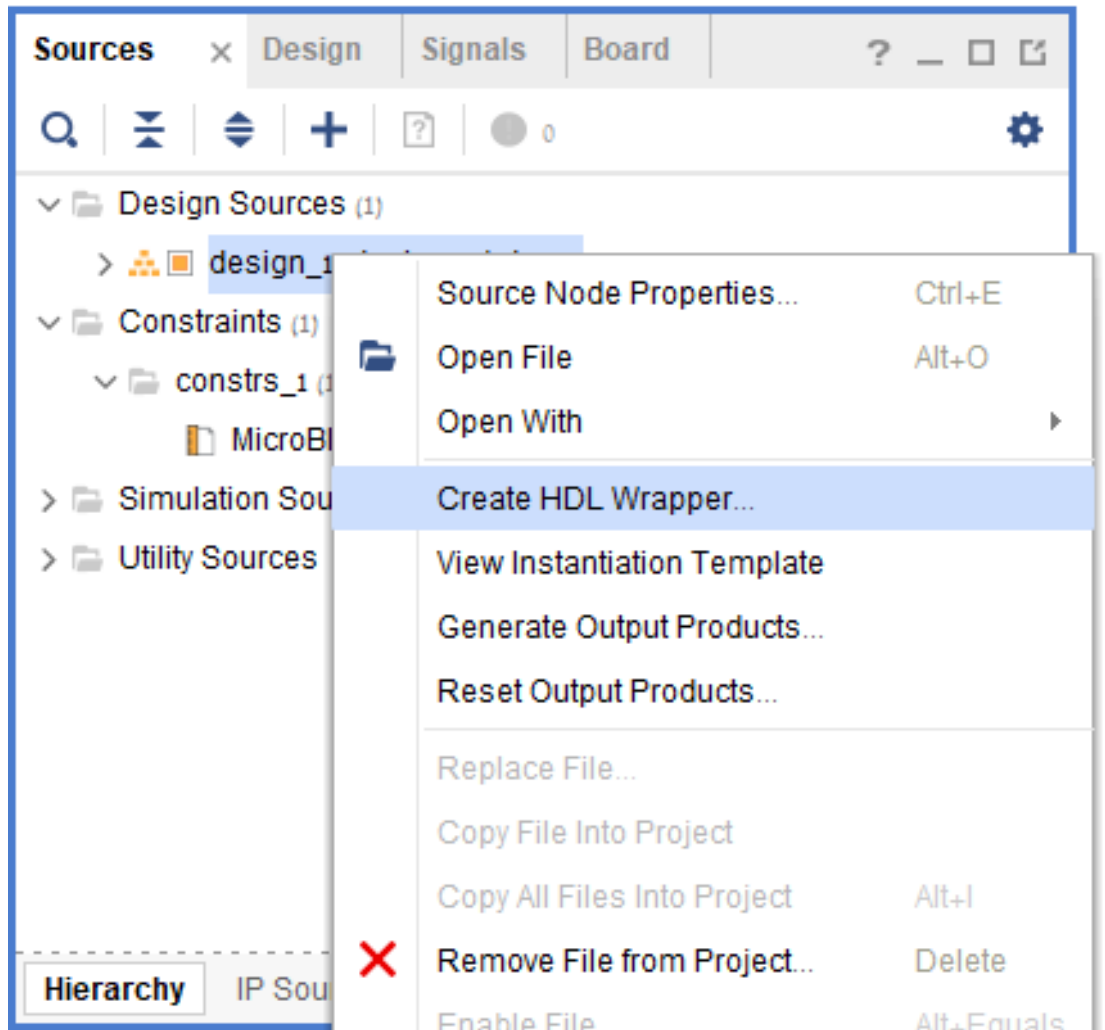


```
MicroBlaze_SysCtrl.xdc
N: /6412/6412_Comp_DevTeam/Yair_Osh/Yair_Tayri/LED_Control_System_Project/MicroBlaze_SysCtrl

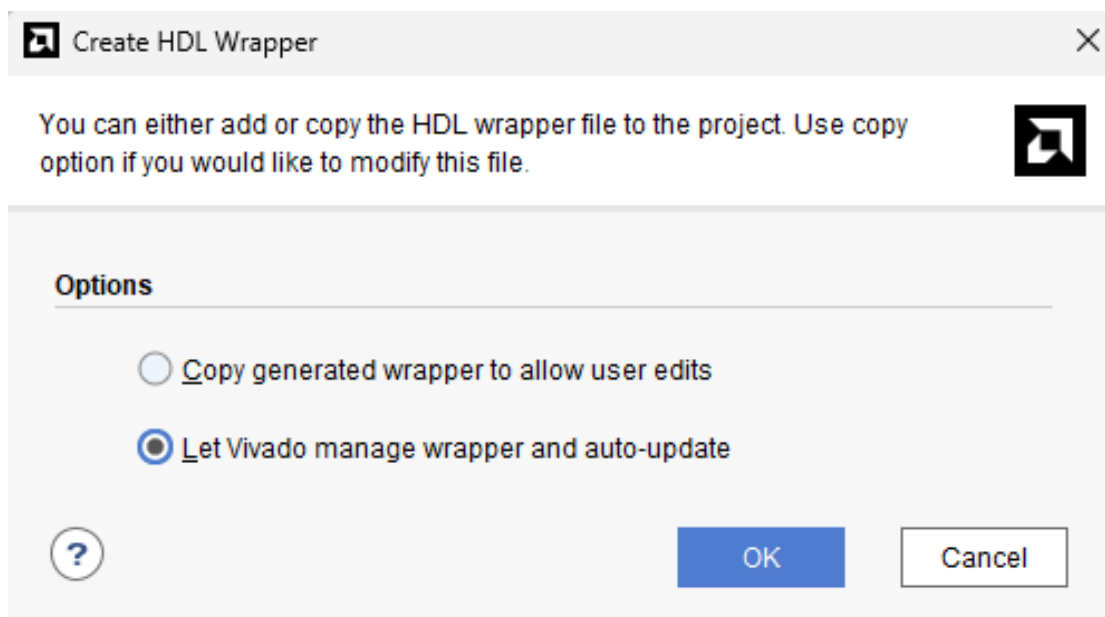
1 set_property IOSTANDARD LVCMOS12 [get_ports {LED_o_0[0]}]
2 set_property IOSTANDARD LVCMOS12 [get_ports {LED_o_0[1]}]
3 set_property IOSTANDARD LVCMOS12 [get_ports {LED_o_0[2]}]
4 set_property IOSTANDARD LVCMOS12 [get_ports {LED_o_0[3]}]
5 set_property IOSTANDARD LVCMOS12 [get_ports {LED_o_0[4]}]
6 set_property IOSTANDARD LVCMOS12 [get_ports {LED_o_0[5]}]
7 set_property IOSTANDARD LVCMOS12 [get_ports {LED_o_0[6]}]
8 set_property IOSTANDARD LVCMOS12 [get_ports {LED_o_0[7]}]
9 set_property PACKAGE_PIN AT32 [get_ports {LED_o_0[0]}]
10 set_property PACKAGE_PIN AV34 [get_ports {LED_o_0[1]}]
11 set_property PACKAGE_PIN AY30 [get_ports {LED_o_0[2]}]
12 set_property PACKAGE_PIN BB32 [get_ports {LED_o_0[3]}]
13 set_property PACKAGE_PIN BF32 [get_ports {LED_o_0[4]}]
14 set_property PACKAGE_PIN AU37 [get_ports {LED_o_0[5]}]
15 set_property PACKAGE_PIN AV36 [get_ports {LED_o_0[6]}]
16 set_property PACKAGE_PIN BA37 [get_ports {LED_o_0[7]}]
17 set_property PACKAGE_PIN AY24 [get_ports sysclk_125_clk_p]
18 set_property PACKAGE_PIN AY23 [get_ports sysclk_125_clk_n]
19 set_property IOSTANDARD LVDS [get_ports sysclk_125_clk_p]
20 set_property PACKAGE_PIN L19 [get_ports reset]
21 set_property IOSTANDARD LVCMOS12 [get_ports reset]
22
23
24 set_property BITSTREAM.GENERAL.COMPRESS TRUE [current_design]
25
26
27
```

שלב 12:

ניצור HDL Wrapper.



נשאיר את הניהול האוטומטי ונלחץ OK.



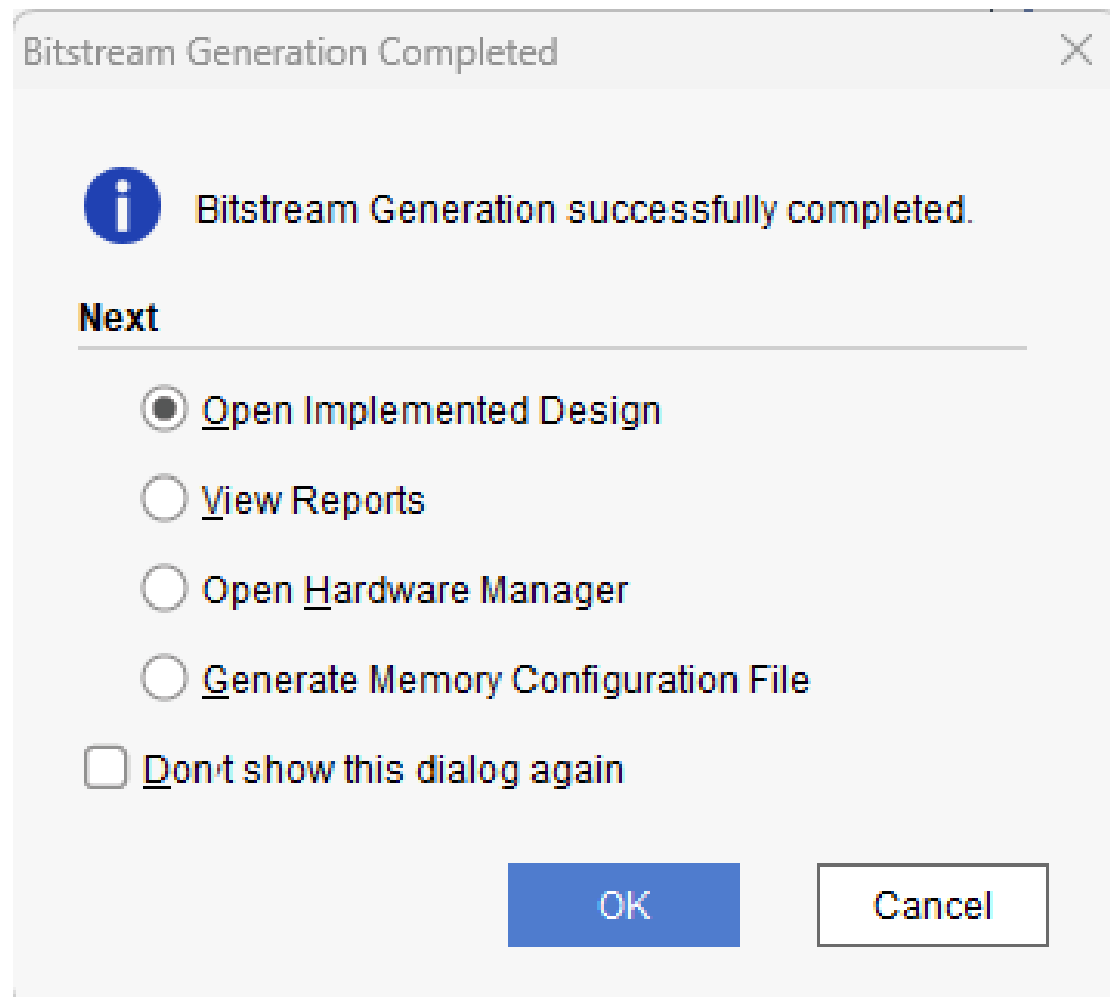
▼ PROGRAM AND DEBUG



Generate Bitstream

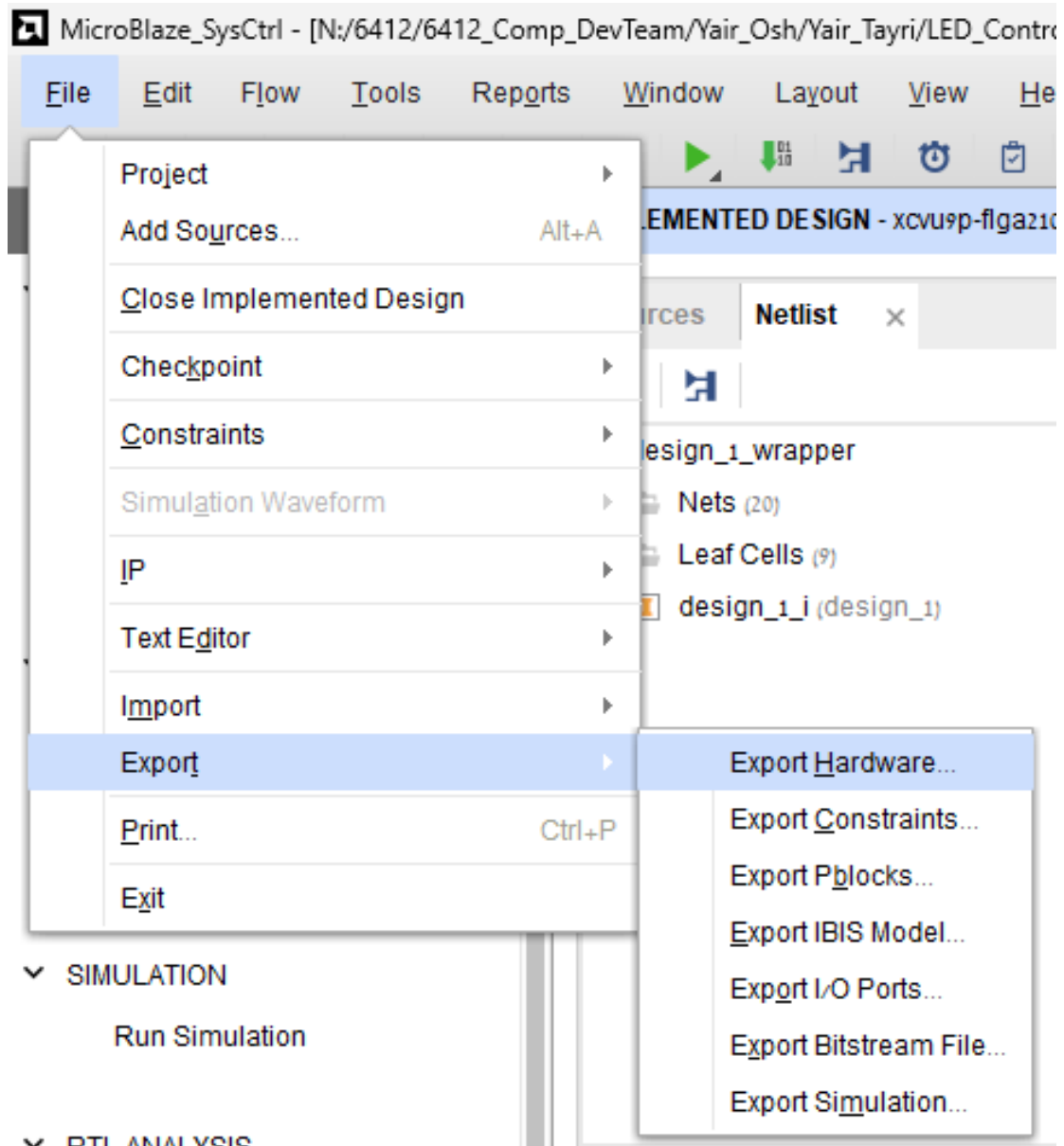
> Open Hardware Manager

יצירת ה-Bitstream הסתיימה בהצלחה.

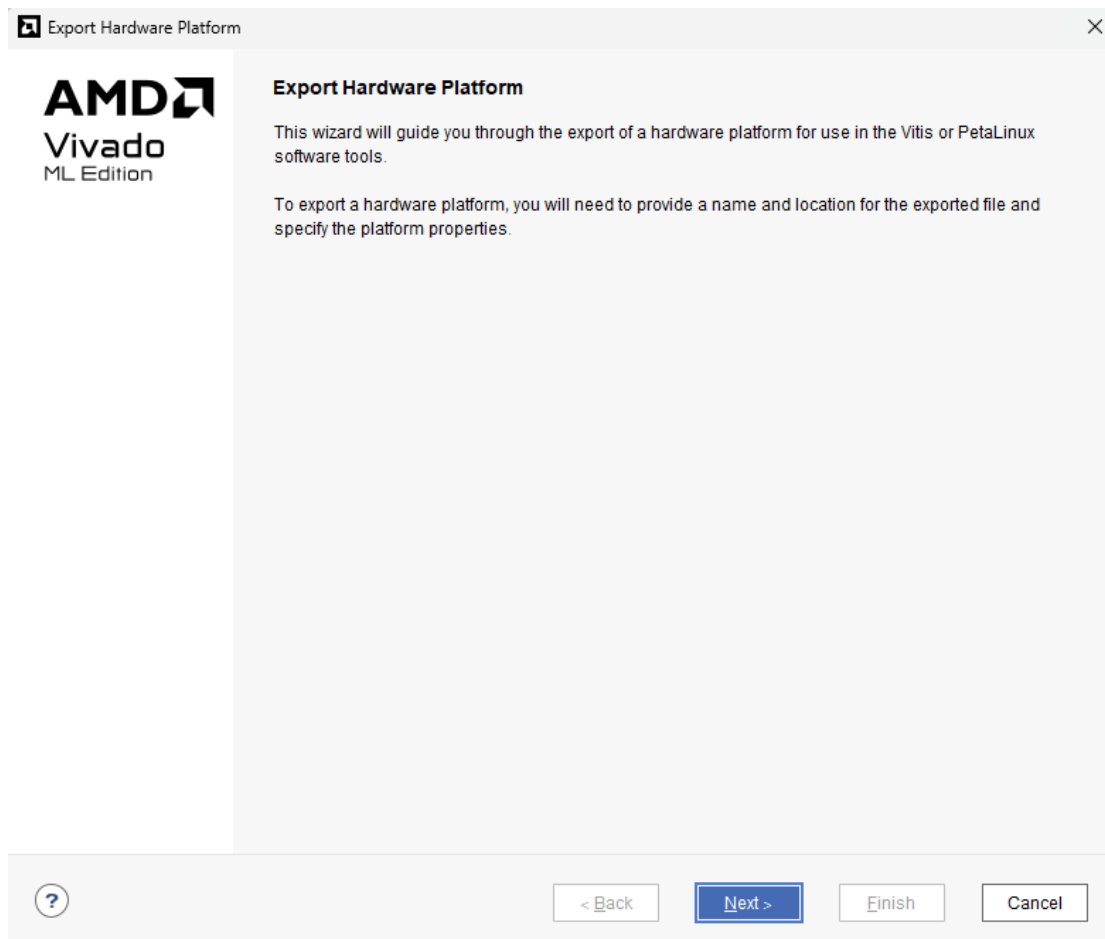


שלב 13:

נייצא את החומרה דרך File, Export Hardware.



הלחץ Next.



נבחר Include bitstream.

 Export Hardware Platform ×

Output 

Set the platform properties to inform downstream tools of the intended use of the target platform's hardware design.

☐ Pre-synthesis
This platform includes a hardware specification for downstream software tools.

☒ Include bitstream
This platform includes the complete hardware implementation and bitstream, in addition to the hardware specification for software tools.

< Back

Next >

Finish

Cancel

נגדיר שם ומיקום לקובץ ה-XSA.

 Export Hardware Platform ✕

Files 

Enter the name of your hardware platform file, and the directory where the XSA file will be stored.

XSA file name:

Export to: ...

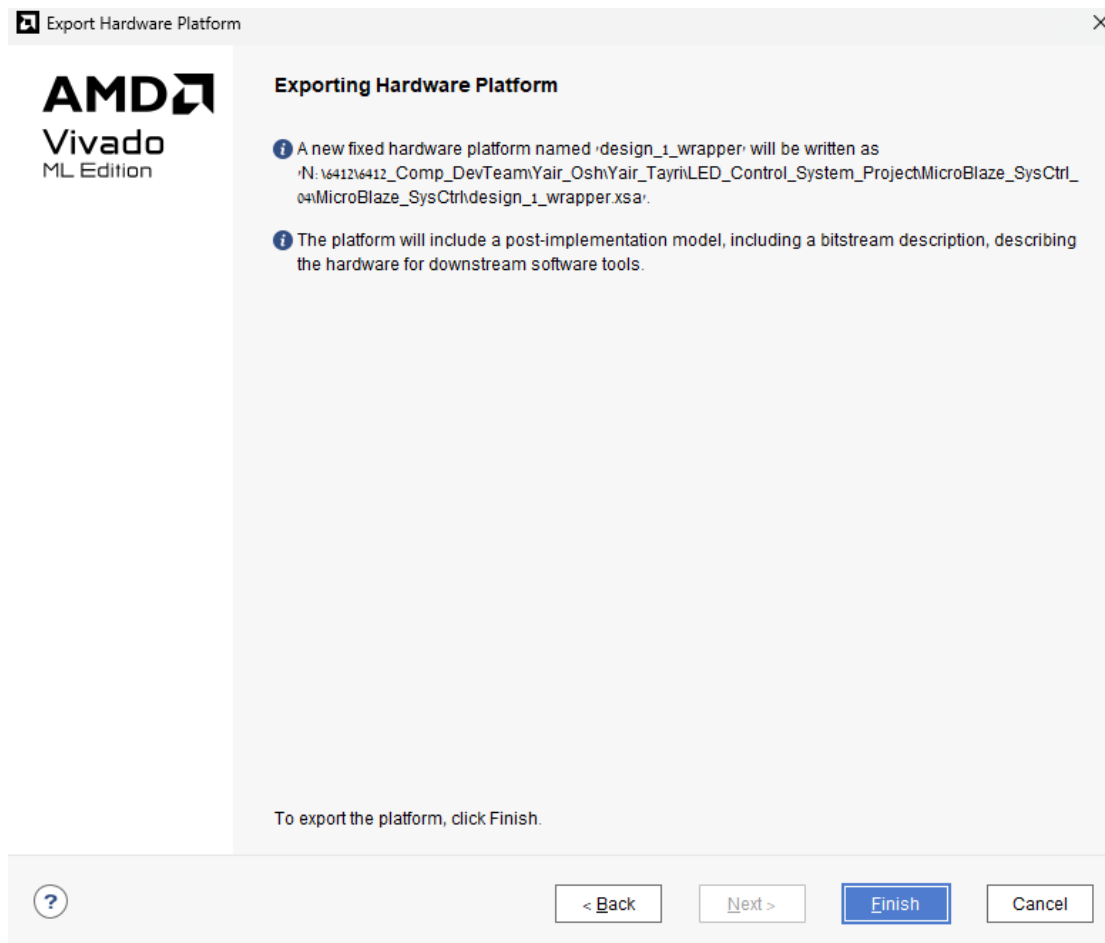
The XSA will be written to: N:\6412\6412_Comp_DevTeam\Yair_Osh\Yair_Tayri\LED_Control_System_Project\MicroBlaze_SysCtrl_04\

< Back

Next >

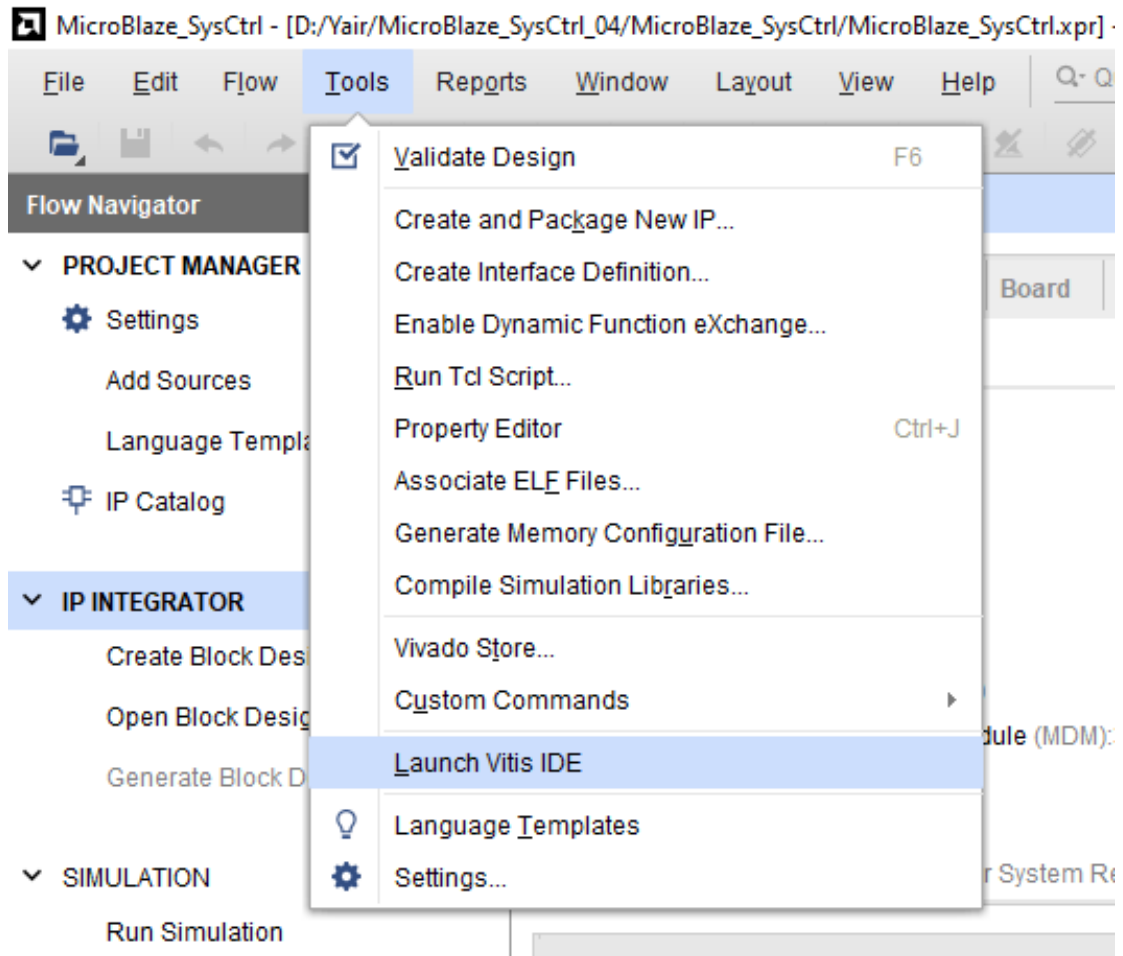
Finish

Cancel

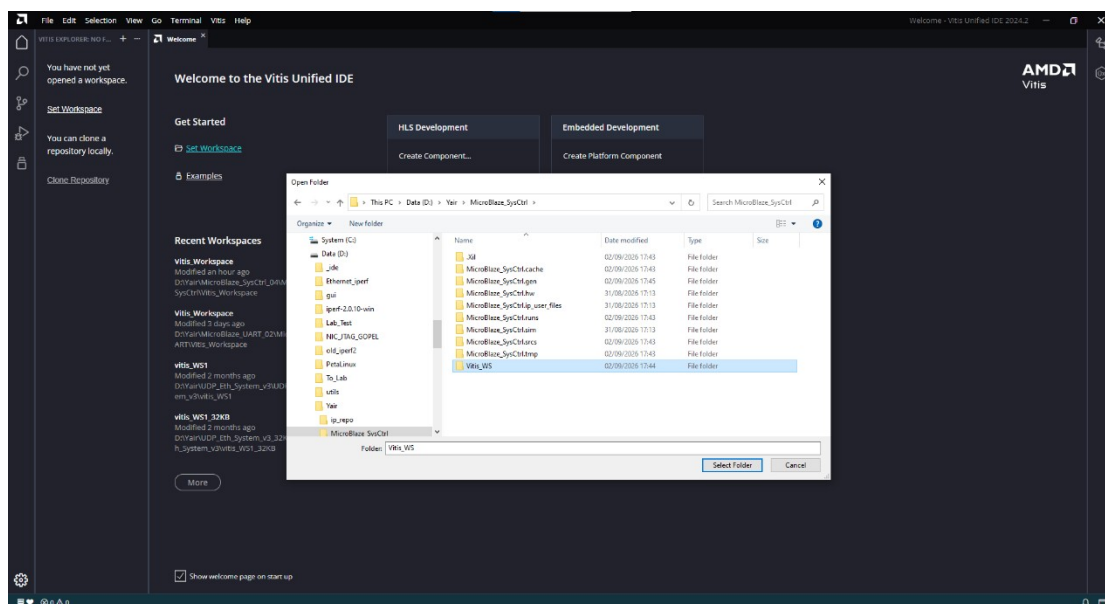


שלב 14:

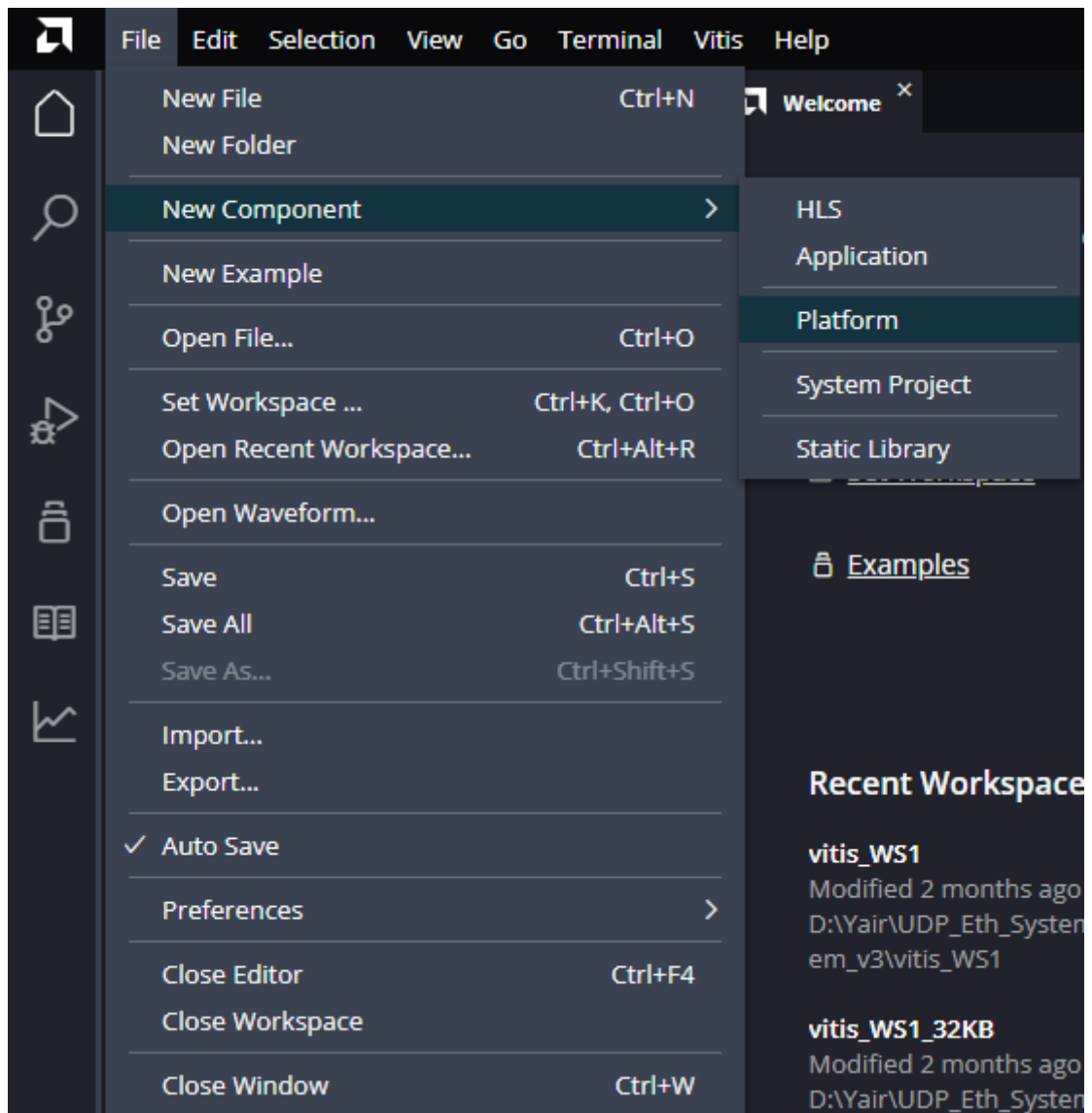
נפעיל את Vitis דרך Tools וזו Launch Vitis IDE.



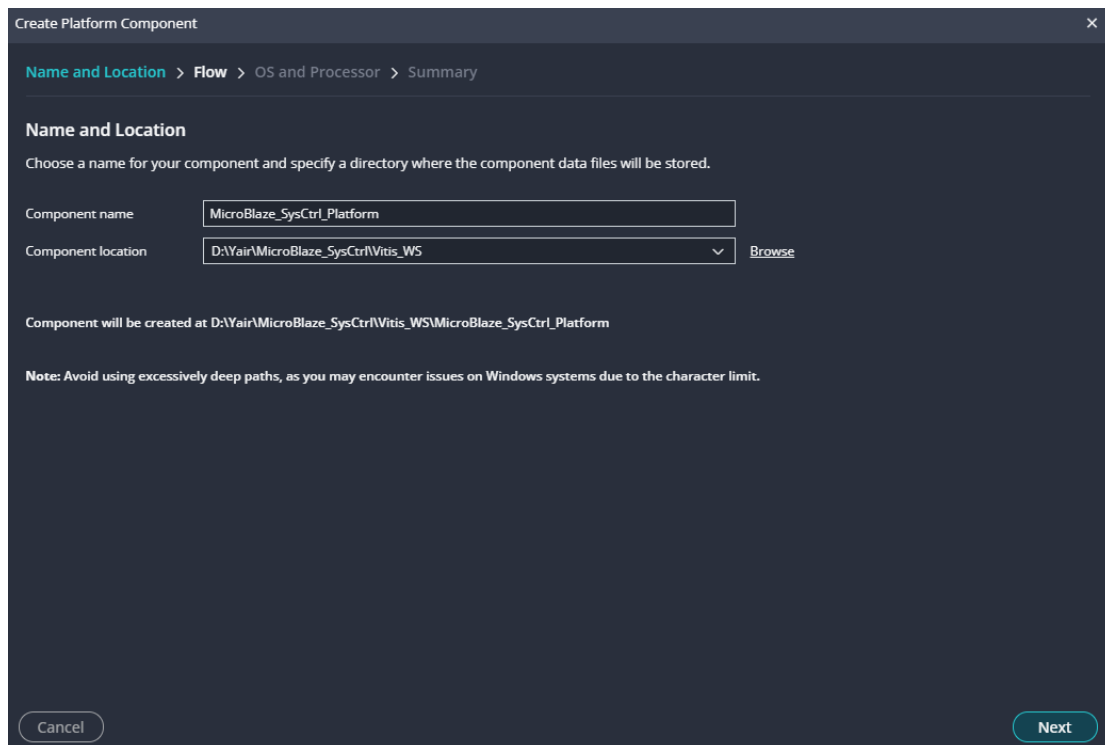
נבחר תיקיית Workspace.



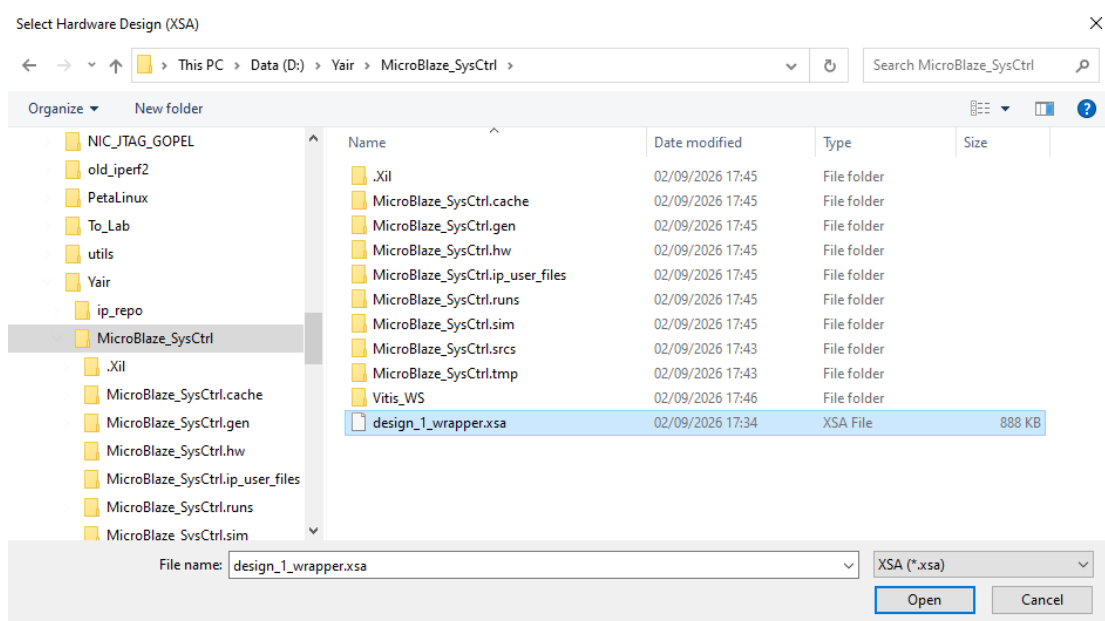
ניצור רכיב חדש מסוג Platform.



נקרא לפלטפורמה MicroBlaze_SysCtrl_Platform.



נבחר את קובץ ה-XSA.



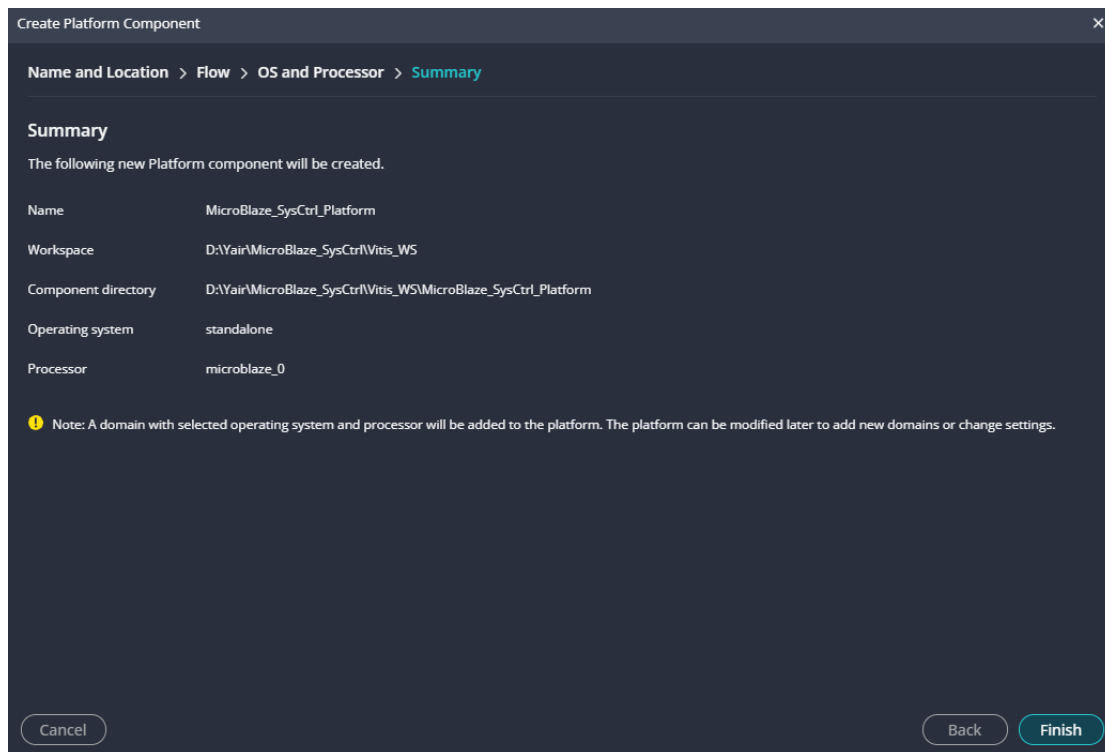
נגדיר מערכת הפעלה standalone ומעבד microblaze_0.

The screenshot shows the 'Create Platform Component' dialog box with the 'OS and Processor' tab selected. The breadcrumb trail is 'Name and Location > Flow > OS and Processor > Summary'. The title is 'Select Operating System and Processor'. Below the title, there is a descriptive text: 'Specify the details for the initial domain to be added to the platform component. The platform can be modified later to add new domains or change settings by opening the platform editor.' There are two dropdown menus: 'Operating system:' with 'standalone' selected, and 'Processor:' with 'microblaze_0' selected. At the bottom, there are three buttons: 'Cancel', 'Back', and 'Next'.

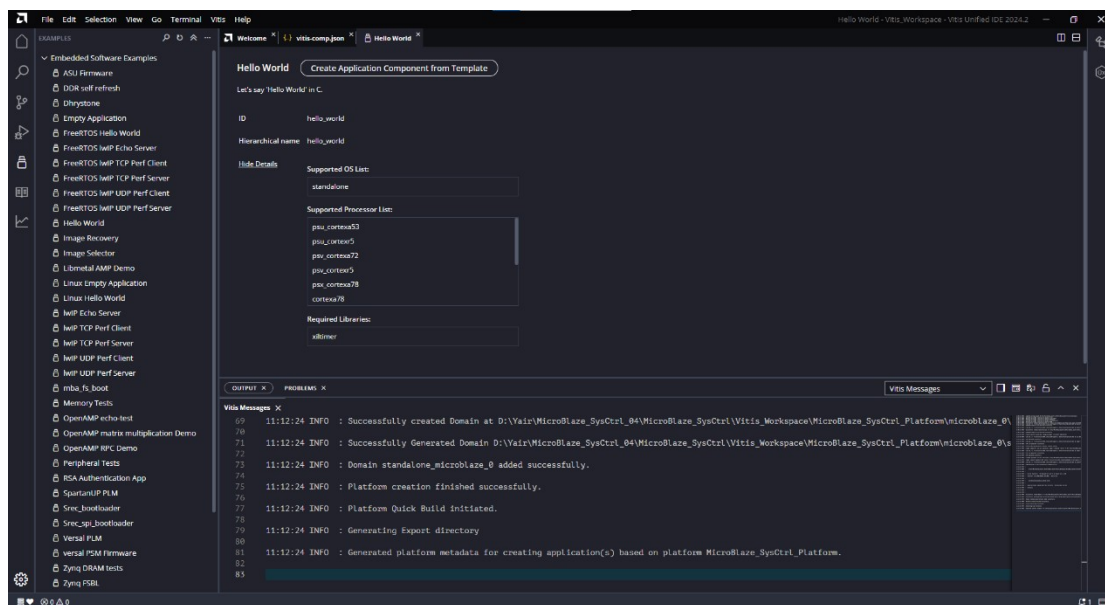
נוודא שקובץ ה-XSA נבחר ונלחץ Next.

The screenshot shows the 'Create Platform Component' dialog box with the 'Flow' tab selected. The breadcrumb trail is 'Name and Location > Flow > OS and Processor > Summary'. The title is 'Select Platform Creation Flow'. Below the title, there is a descriptive text: 'Create a platform component by selecting the hardware design and add software domains.' There are two radio buttons: 'Hardware Design' (selected) and 'Existing Platform'. Below the radio buttons, there is a section for 'Hardware Design (XSA) For Implementation' with a dropdown menu showing 'D:\Vair\MicroBlaze_SysCtrl\design_1_wrapper.xsa' and a 'Browse' button. At the bottom, there is a section for '> Advanced Options'. At the bottom of the dialog, there are three buttons: 'Cancel', 'Back', and 'Next'.

נלחץ Finish.



בהגדרות ה-BSP של הפלטפורמה ננתב את stdin ו-stdout לרכיב ה-MDM.



שלב 15:

ניצור אפליקציה בשם LED_Control_App מתבנית Hello World.

Create Application Component - Hello World

Name and Location > Hardware > Domain > Summary

Name and Location

Choose a name for your component and specify a directory where the component data files will be stored.

Component name:

Component location: [Browse](#)

Component will be created at D:\Yair\MicroBlaze_SysCtrl\Vitis_WS\LED_Control_App

Note: Avoid using excessively deep paths, as you may encounter issues on Windows systems due to the character limit.

[Cancel](#) [Next](#)

נבחר את הפלטפורמה MicroBlaze_SysCtrl_Platform.

Create Application Component - Hello World

Name and Location > Hardware > Domain > Summary

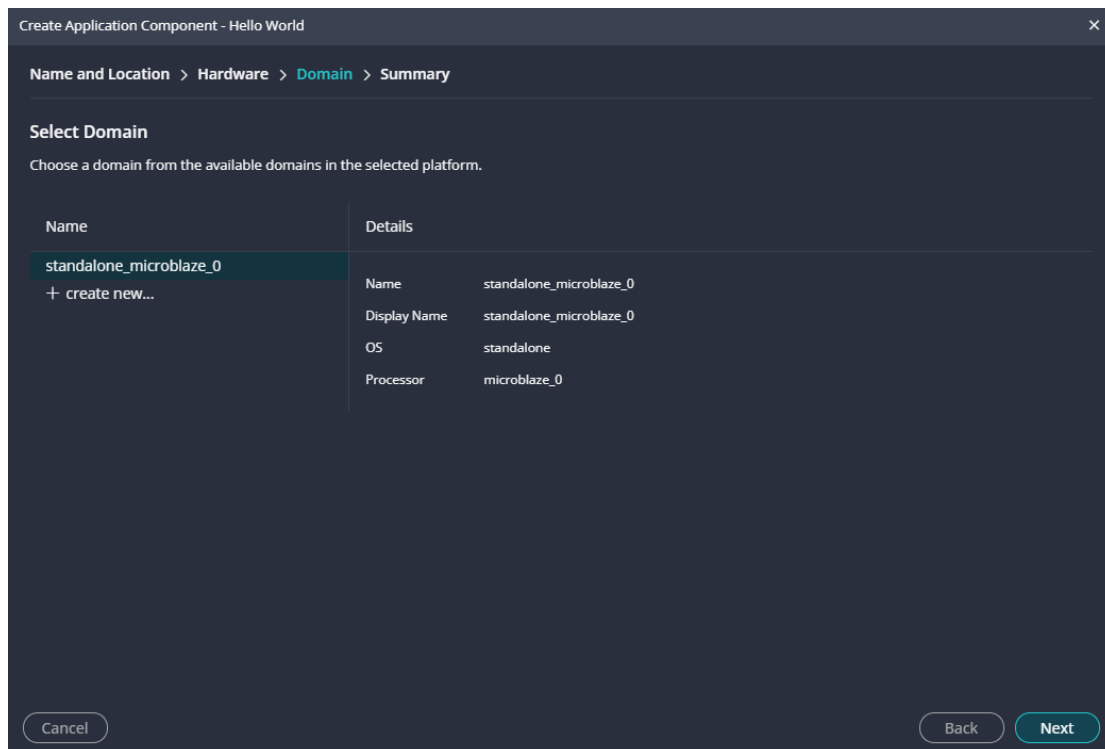
Select Platform

Platforms from your repositories that support the selected example. To create a new platform, use "File -> New Component -> Platform"

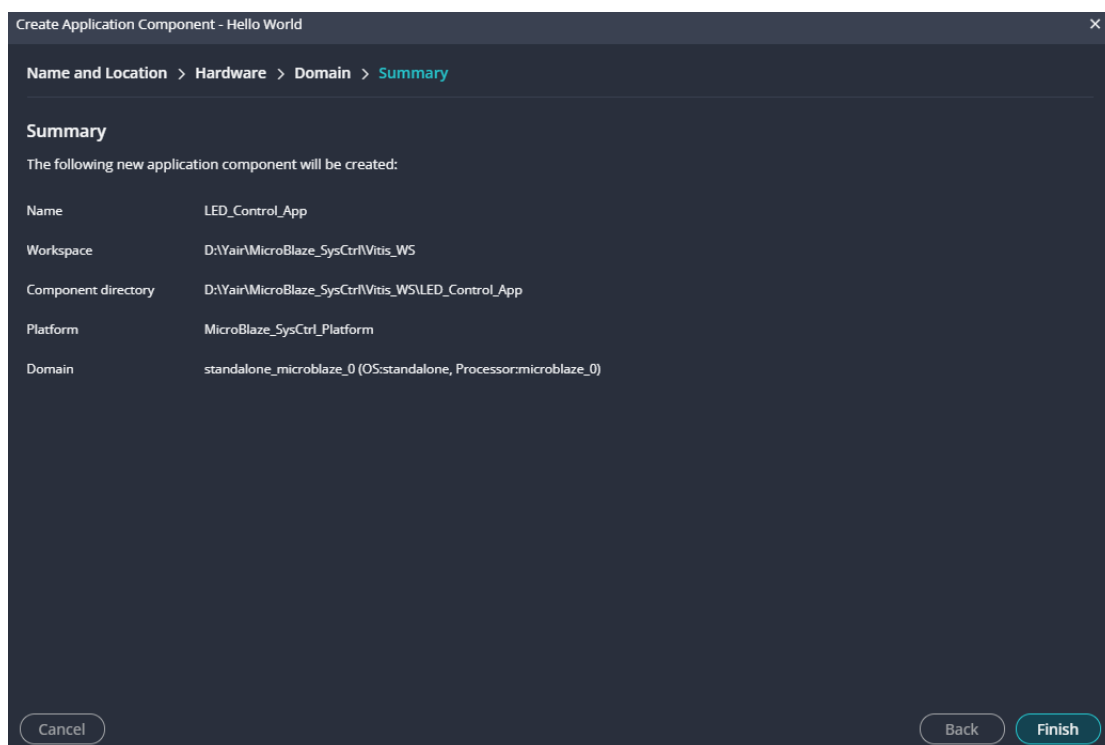
NAME	BOARD	FLOW	VENDOR	PATH
D:\Yair\MicroBlaze_SysCtrl\Vitis_WS\MicroBlaze_SysCtrl_Platform (1)				...e_SysCtrl_Platform\export\MicroBlaze_SysCtrl_Platform
MicroBlaze_SysCtrl_Platform	vcu118	Embedded	xilinx.com	...aze_SysCtrl_Platform\MicroBlaze_SysCtrl_Platform.xpfm

[Cancel](#) [Back](#) [Next](#)

נבחר את הדומיין standalone_microblaze_0.



נלחץ Finish.



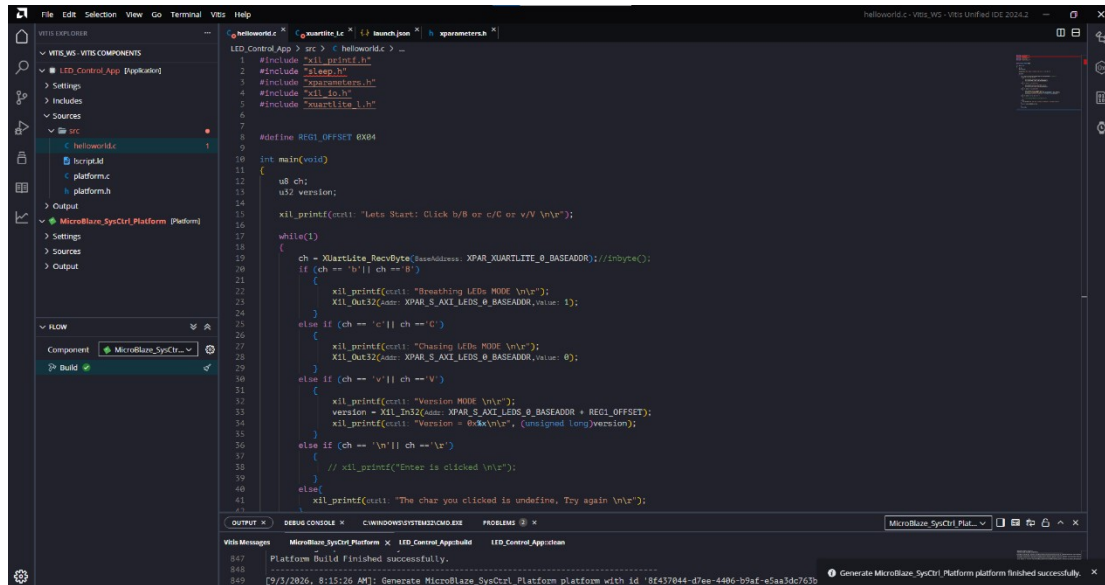
שלב 16:

נכתוב את קוד הבקרה. הלולאה הראשית קולטת תו מממשק ה-UART של ה-MDM. התו b מפעיל מצב נשימה באמצעות כתיבת 1 לרגיסטר 0 בפונקציה Xil_Out32. התו c כותב 0 ומחזיר את הלדים למצב ריצה. התו v קורא את רגיסטר 1 בהיסט REG1_OFFSET בפונקציה Xil_In32 ומדפיס את הגרסה. כל תו אחר מחזיר הודעת שגיאה.

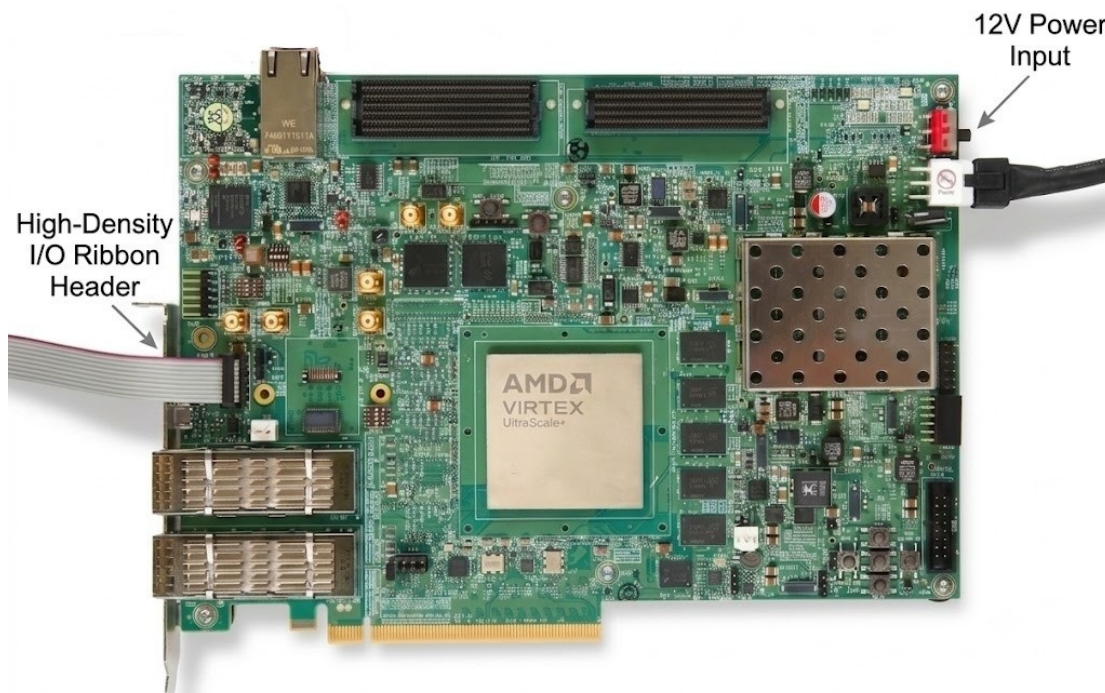
```
1  #include "xil_printf.h"
2  #include "sleep.h"
3  #include "xparameters.h"
4  #include "xil_io.h"
5  #include "xuartlite_l.h"
6
7
8  #define REG1_OFFSET 0X04
9
10 int main(void)
11 {
12     u8 ch;
13     u32 version;
14
15     xil_printf(ctrl1: "Lets Start: Click b/B or c/C or v/V \n\r");
16
17     while(1)
18     {
19         ch = XUartLite_RecvByte(BaseAddress: XPAR_XUARTLITE_0_BASEADDR); //inbyte();
20         if (ch == 'b' || ch == 'B')
21         {
22             xil_printf(ctrl1: "Breathing LEDs MODE \n\r");
23             Xil_Out32(Addr: XPAR_S_AXI_LEDS_0_BASEADDR, Value: 1);
24         }
25         else if (ch == 'c' || ch == 'C')
26         {
27             xil_printf(ctrl1: "Chasing LEDs MODE \n\r");
28             Xil_Out32(Addr: XPAR_S_AXI_LEDS_0_BASEADDR, Value: 0);
29         }
30         else if (ch == 'v' || ch == 'V')
31         {
32             xil_printf(ctrl1: "Version MODE \n\r");
33             version = Xil_In32(Addr: XPAR_S_AXI_LEDS_0_BASEADDR + REG1_OFFSET);
34             xil_printf(ctrl1: "Version = 0x%x\n\r", (unsigned long)version);
35         }
36         else if (ch == '\n' || ch == '\r')
37         {
38             // xil_printf("Enter is clicked \n\r");
39         }
40         else{
41             xil_printf(ctrl1: "The char you clicked is undefine, Try again \n\r");
42         }
43         for(int i=0;i<1000000;i++){
44
45         }
46         return(0);
47     }
```

שלב 17:

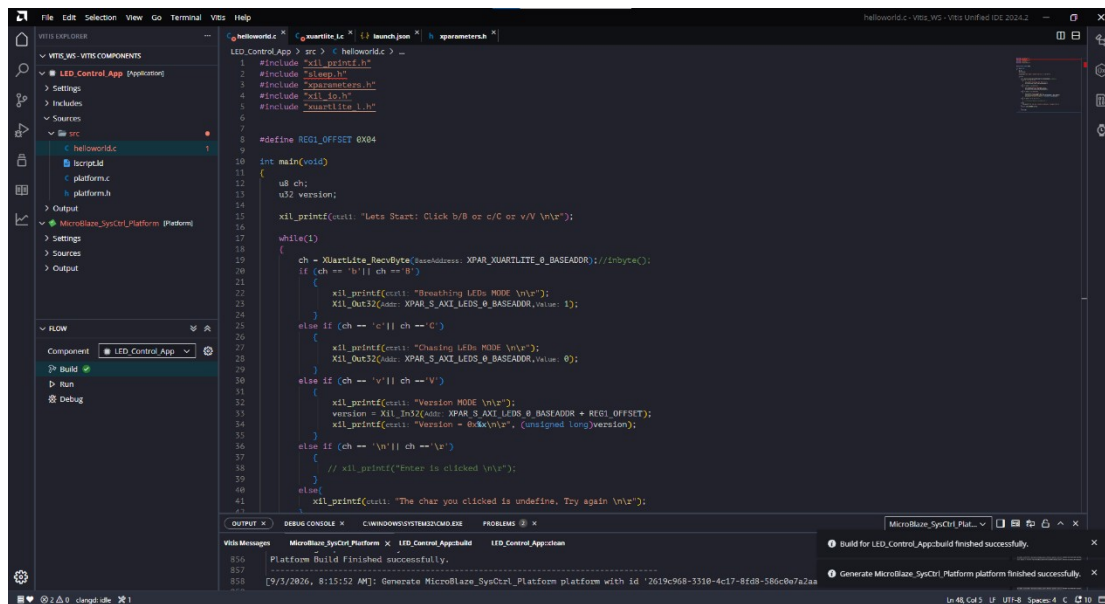
נבצע Build לאפליקציה.



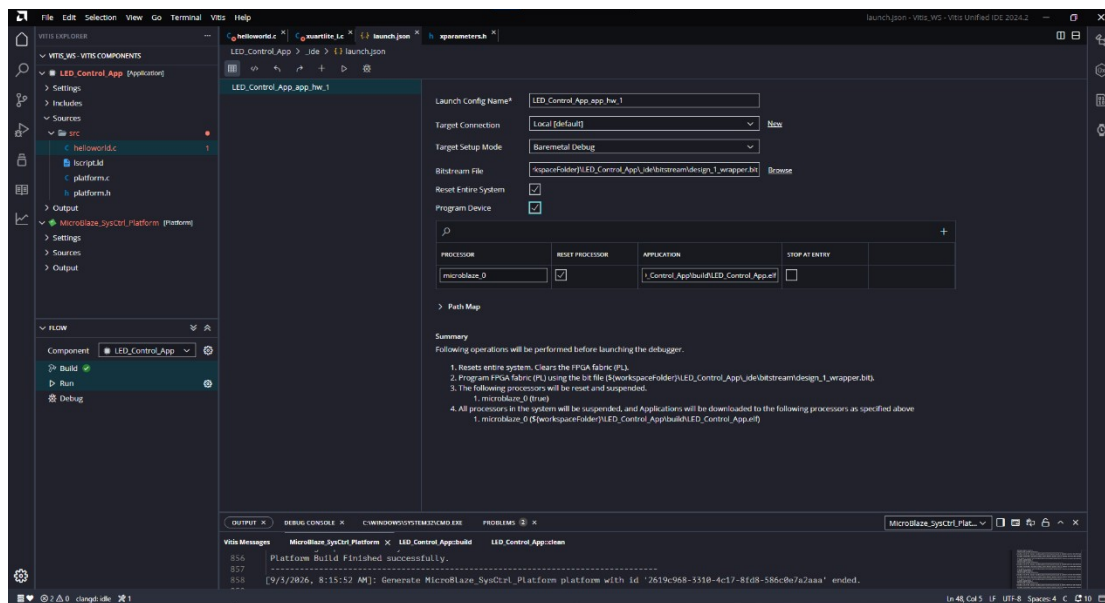
נחבר את הלוח למחשב באמצעות כבל ה-JTAG ונפעיל את ספק הכוח.



נגדיר את תצורת ההרצה.



נרץ את האפליקציה על הלוח.



שלב 18:

בחלון XSCT Console נקליד את הפקודות הבאות. הפקודה target מציגה את יעדי ה-JTAG, הפקודה target 2 בוחרת את רכיב ה-MDM, והפקודה jtagterminal -start פותחת טרמינל המחובר לערוץ ה-JTAG-UART:

```
target
target 2
jtagterminal -start
```

בטרמינל נקליד c, b ו-v. המערכת עוברת בין מצבי הלדים בזמן אמת, וקריאת הגרסה מחזירה 0x20251123.

```
F:\Xilinx\Vitis\2024.2\bin\unwrapped\win64.o\tclsh86t.exe
JTAG-based Hyperterminal.
Connected to JTAG-based Hyperterminal over TCP port : 64996
(using socket : sock000001E33E745C90)
Help :
Terminal requirements :
  (i) Processor's STDOUT is redirected to the ARM DCC/MDM UART
  (ii) Processor's STDIN is redirected to the ARM DCC/MDM UART.
  Then, text input from this console will be sent to DCC/MDM's UART port.
NOTE: This is a line-buffered console and you have to press "Enter"
      to send a string of characters to DCC/MDM.

Lets Start: Click b/B or c/C or v/V
b
Breathing LEDs MODE
c
Chasing LEDs MODE
v
Version MODE
Version = 0x20251123
d
The char you clicked is undefine, Try again
_
```

Chapter 5: DDR Memory Integration and AXI GPIO Pushbutton Interface on Artix-7 AC701

מבוא:

בפרק זה נבנה מערכת MicroBlaze על לוח Artix-7 AC701 הכוללת בקר זיכרון DDR מסוג MIG 7 Series. למערכת יתווסף רכיב AXI GPIO לקריאת מצב הלחצנים והלדים, ואת פעולתו נאמת בתוכנה דרך Tera Term ובחומרה באמצעות ILA.

דרישות קדם:

חומרה:

לוח פיתוח Xilinx Artix-7 AC701

כבל USB-JTAG

כבל USB-to-UART לחיבור טורי למחשב

ספק כוח 12 וולט ללוח

תוכנה:

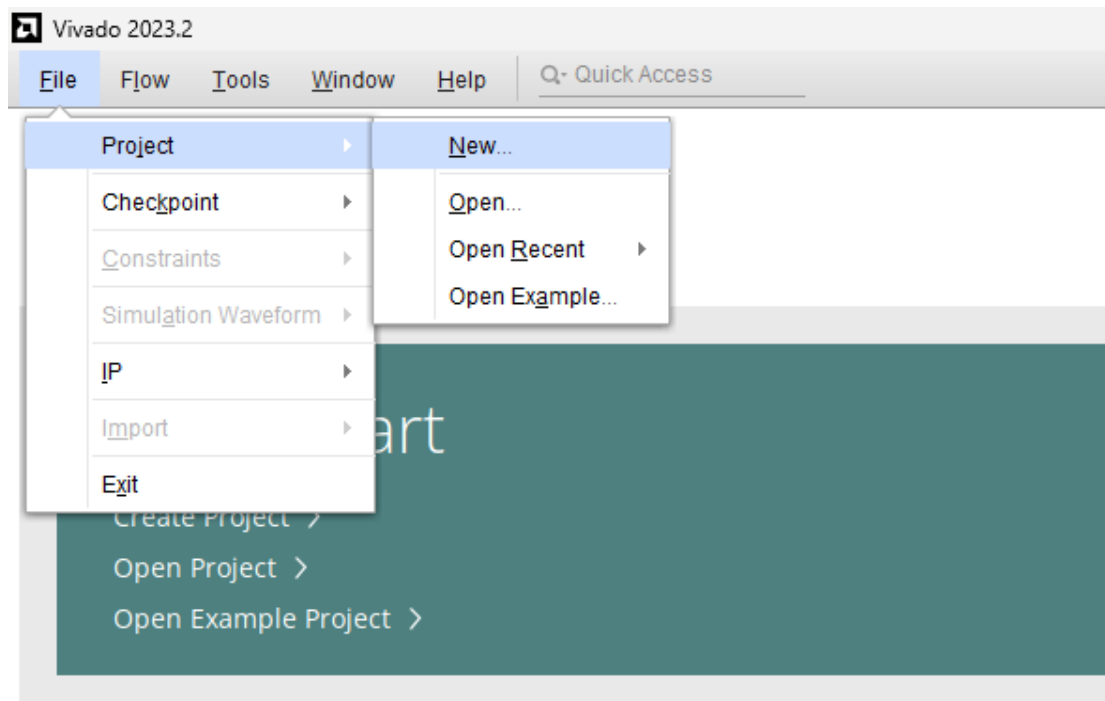
Vivado Design Suite

Vitis Unified IDE

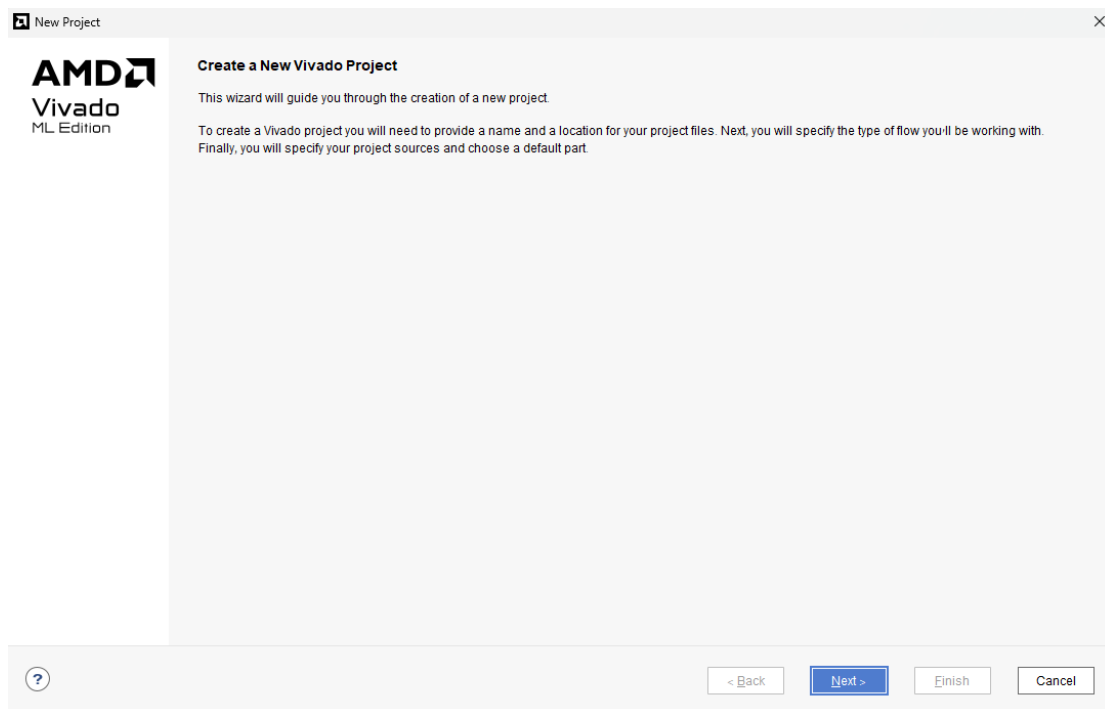
תוכנת טרמינל Tera Term

שלב 1:

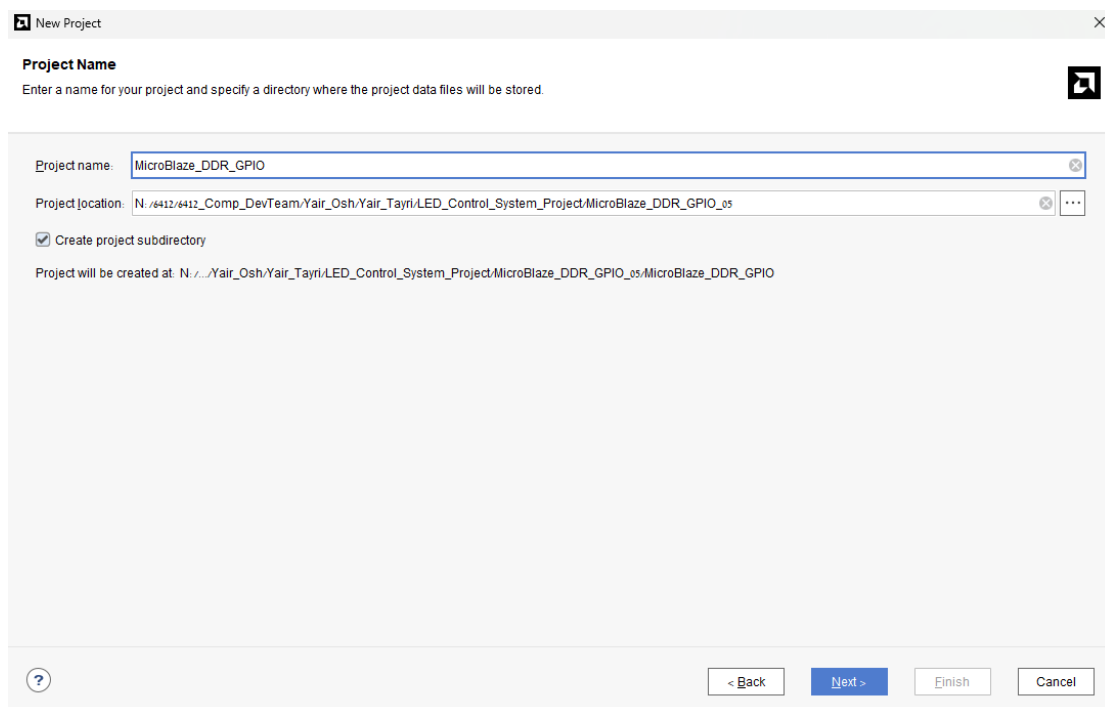
ניצור פרויקט חדש דרך File, Project ואז New.



נלחץ Next.



נקרא לפרויקט MicroBlaze_DDR_GPIO.



נבחר RTL Project.

New Project
✕

Project Type
Specify the type of project to create.

☒ **RTL Project**
You will be able to add sources, create block designs in IP Integrator, generate IP, run RTL analysis, synthesis, implementation, design planning and analysis.
☒ Do not specify sources at this time
☐ Project is an extensible Vitis platform

☐ **Post-synthesis Project**
You will be able to add sources, view device resources, run design analysis, planning and implementation.
☐ Do not specify sources at this time

☐ **I/O Planning Project**
Do not specify design sources. You will be able to view part/package resources.

☐ **Imported Project**
Create a Vivado project from a Synplify Project File.

☐ **Example Project**
Create a new Vivado project from a predefined template.

?
< Back
Next >
Finish
Cancel

בלשונית Boards נבחר את לוח ה-AC701.

New Project
✕

Default Part
Choose a default AMD part or board for your project.

Parts
Boards

To fetch the latest available boards from git repository, click on 'Refresh' button. [Dismiss](#)

Reset All Filters

Vendor: All Name: All Board Rev: Latest

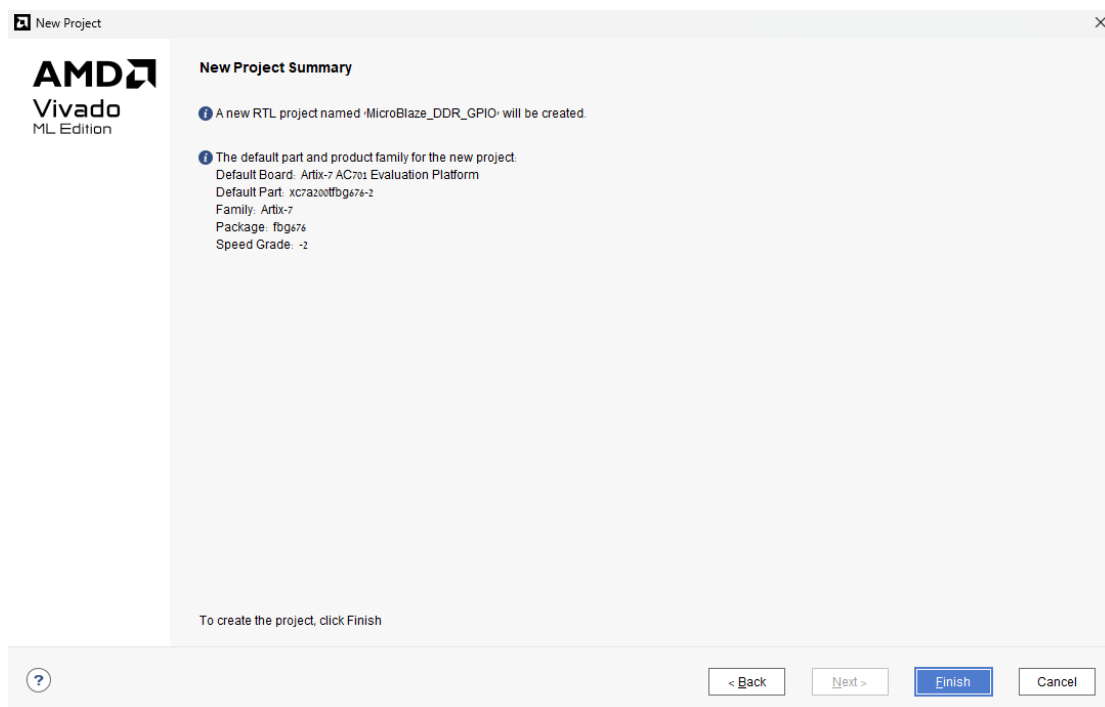
Search: Artix-7 (1 match)

Display Name	Preview	Status	Vendor	File Version	Part	I/O Pin Count
Artix-7 AC701 Evaluation Platform Add Companion Card Connections		Installed	xilinx.com	1.4	xc7a200tfg676-2	676

Refresh

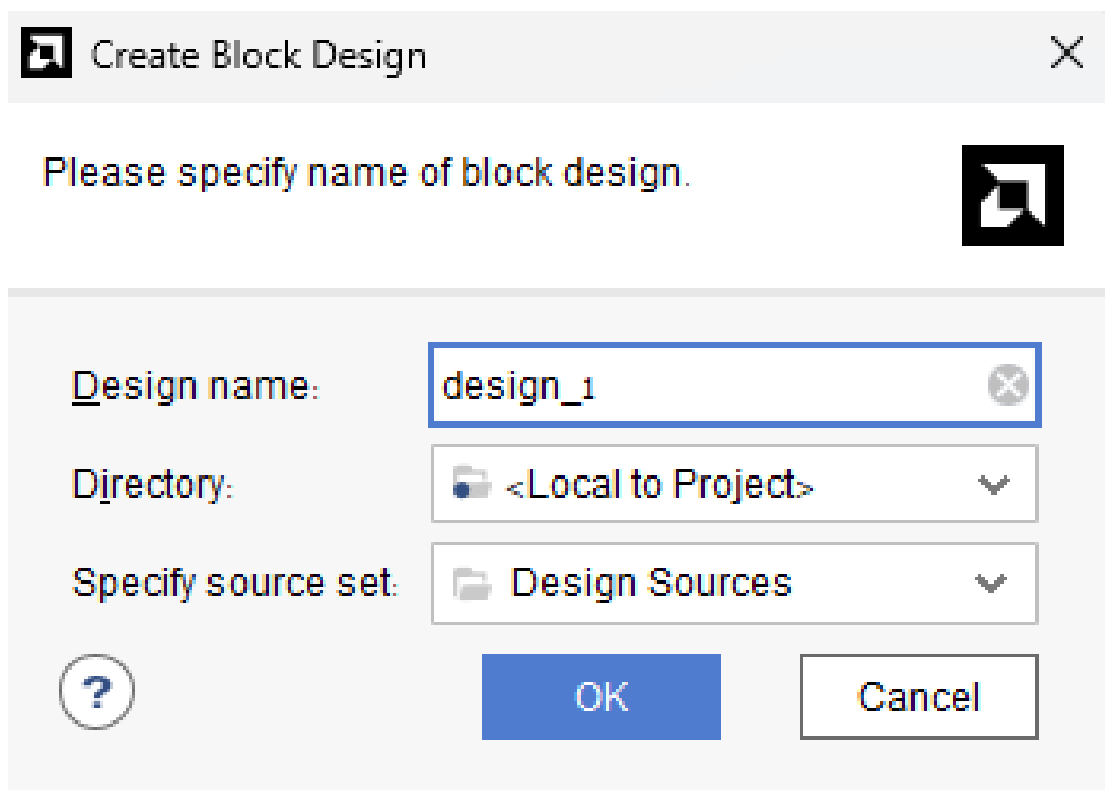
?
< Back
Next >
Finish
Cancel

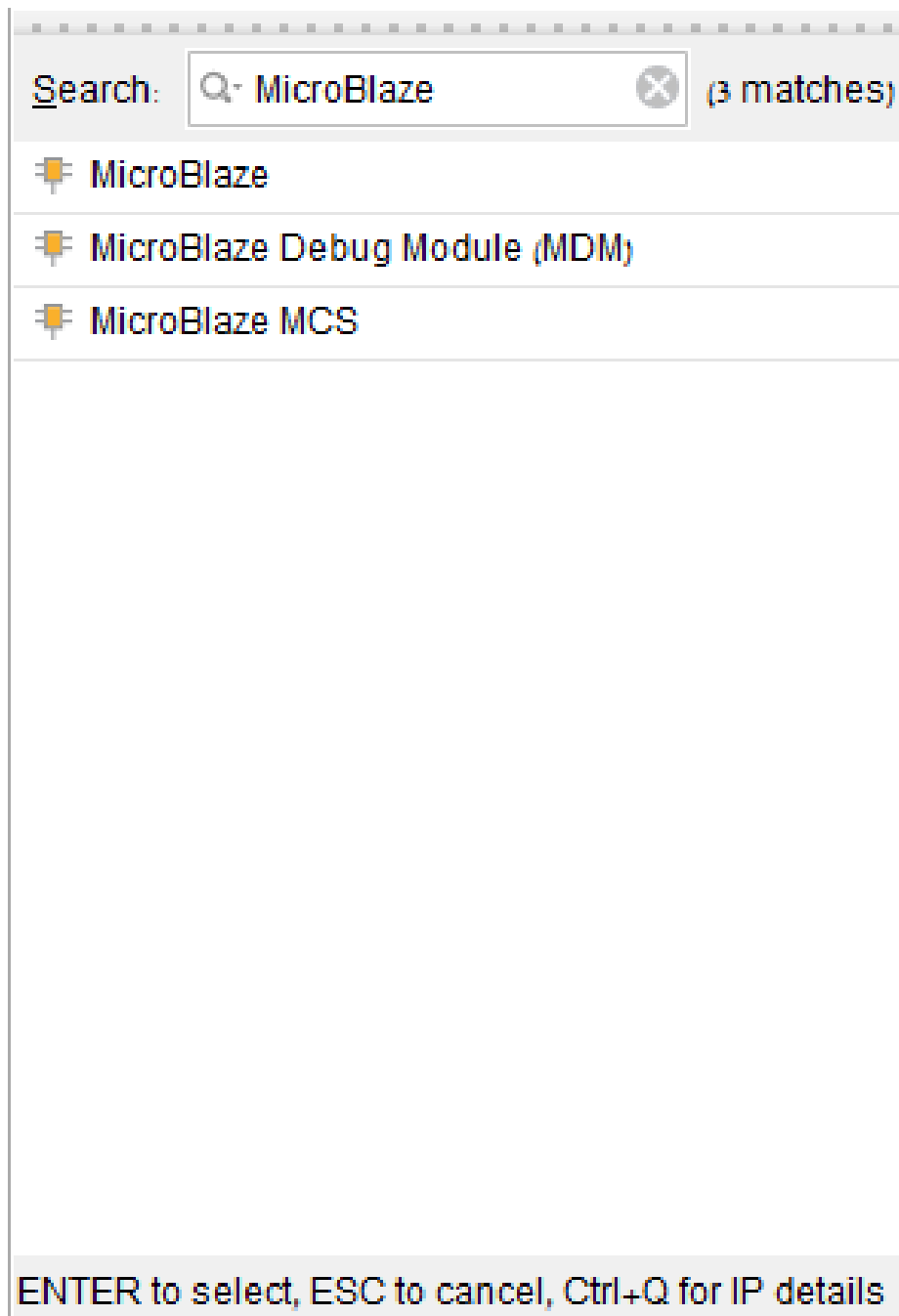
נלחץ Finish.



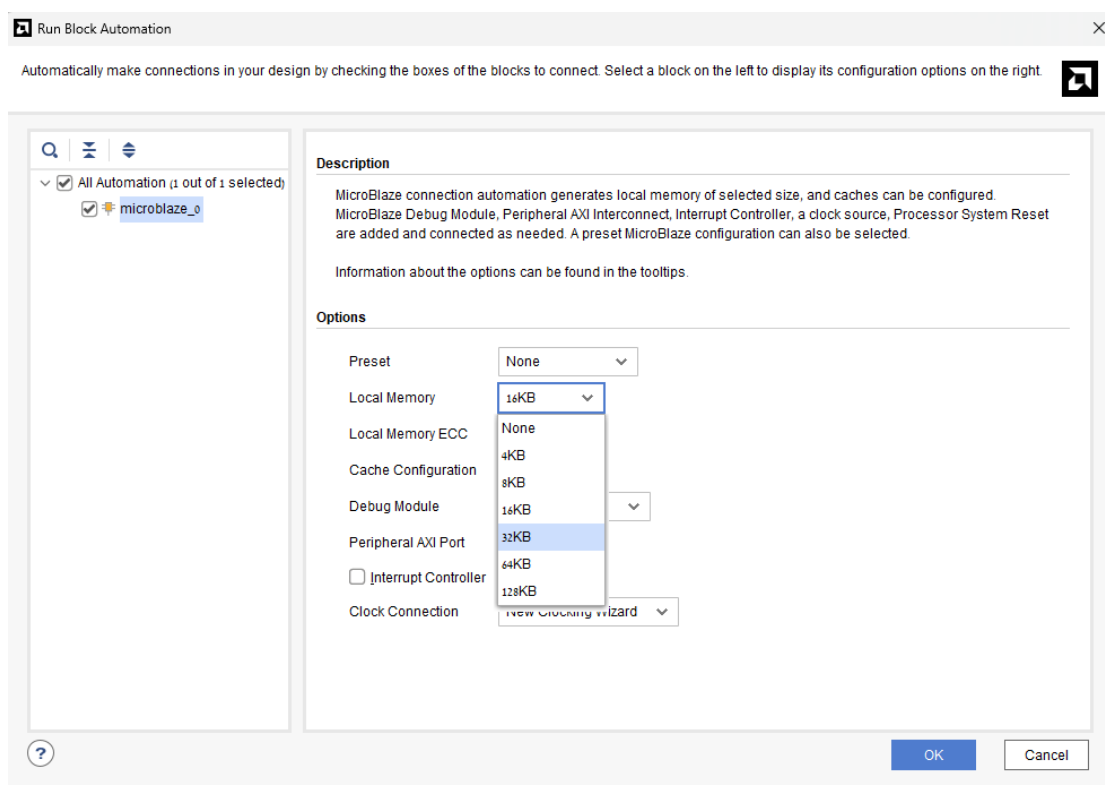
שלב 2:

ניצור Block Design.



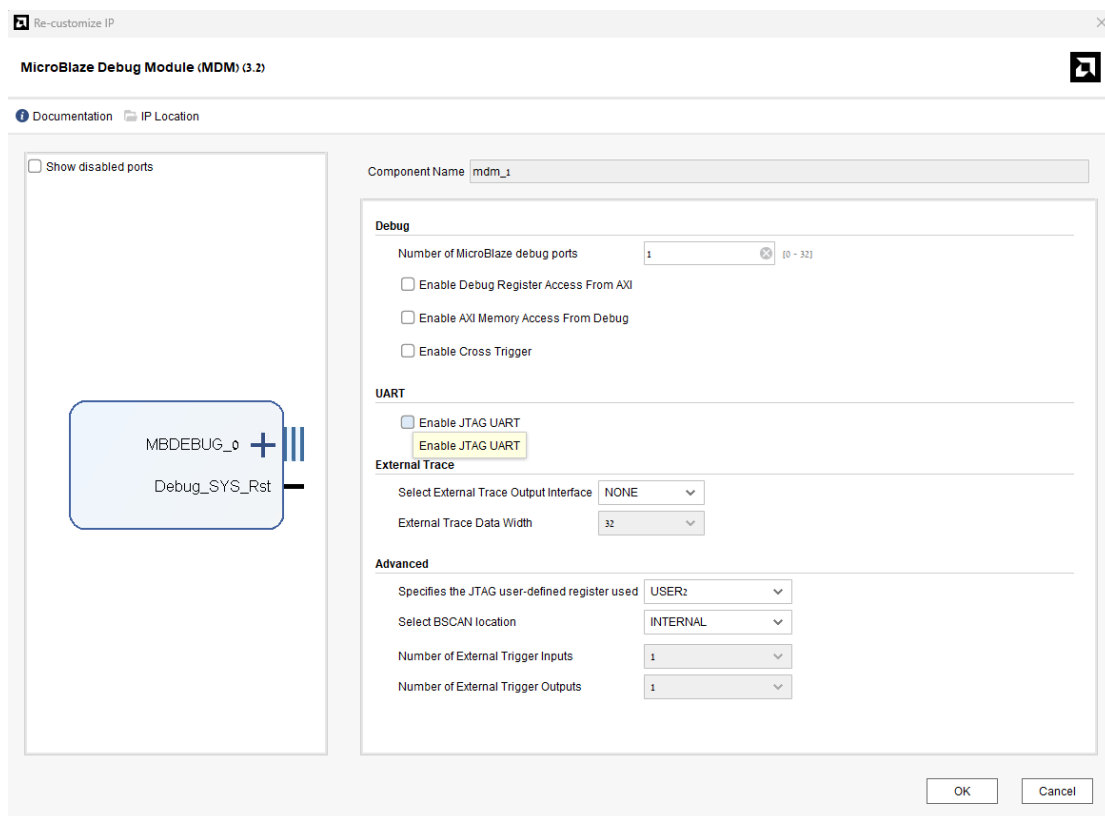


נריץ Run Block Automation ונגדיר את הזיכרון המקומי.



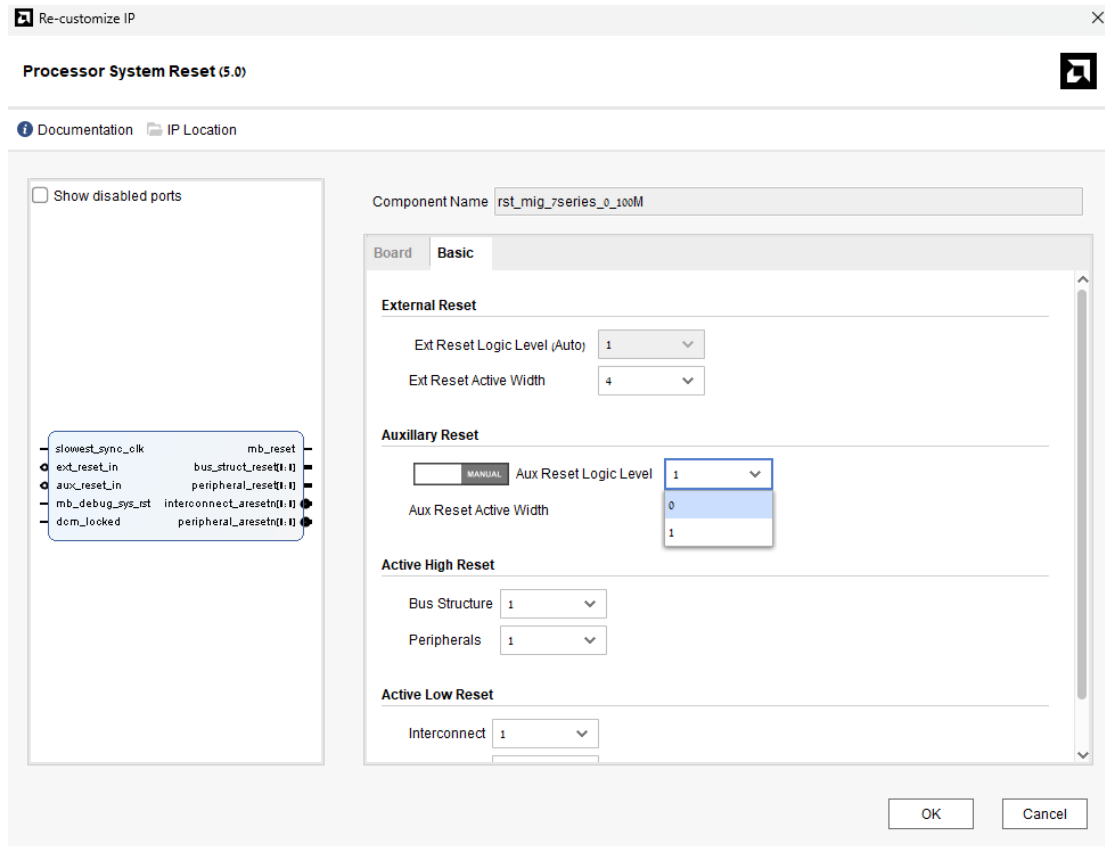
שלב 3:

בהגדרות רכיב ה-MDM נסמן Enable JTAG UART.



שלב 4:

נגדיר את רכיב ה-Processor System Reset.



שלב 5:

נפתח את הגדרות ה-MicroBlaze.

Re-customize IP

MicroBlaze (11.0)

Documentation IP Location Advanced

IP Symbol Resource Estimates

Frequency Area Performance

Resource Estimates

Percent (%)

100.0
90.0
80.0
70.0
60.0
50.0
40.0
30.0
20.0
10.0
0.0

0.0

Resource Usage

BRAM: 0 DSP48E: 0

Component Name microblaze_0

Debug

MicroBlaze Debug Module Interface BASIC

Hardware Breakpoints

Number of PC Breakpoints 1 (0 - 8)

Number of Write Address Watchpoints 0 (0 - 4)

Number of Read Address Watchpoints 0 (0 - 4)

Performance Monitoring

Number of Performance Monitor Event Counters 5 (0 - 48)

Number of Performance Monitor Latency Counters 1 (0 - 7)

Performance Monitor Counter Width 32

Trace & Profiling

External Trace

Trace Buffer Size 4KB

Profile Buffer Size NONE

< Back Next > Page 3 of 4

OK Cancel

בלשונית Buses נסמן Enable Peripheral AXI Instruction Interface.

Re-customize IP

MicroBlaze (11.0)

Documentation IP Location Advanced

IP Symbol Resource Estimates

Frequency Area Performance

Resource Estimates

Percent (%)

100.0
90.0
80.0
70.0
60.0
50.0
40.0
30.0
20.0
10.0
0.0

0.0

Resource Usage

BRAM: 0 DSP48E: 0

Component Name microblaze_0

Buses

Local Memory Bus Interfaces

☒ Enable Local Memory Bus Instruction Interface

☒ Enable Local Memory Bus Data Interface

AXI and ACE Interfaces

Select Bus Interface AXI

☒ Enable Peripheral AXI Instruction Interface

☒ Enable Peripheral AXI Data Interface

Stream Interfaces

Number of Stream Links 0 (0 - 14)

Other Interfaces

☐ Enable Trace Bus Interface

Lockstep Interface NONE

< Back Next > Page 4 of 4

OK Cancel

הגדרה זו מאפשרת למעבד להביא פקודות דרך אפיק ה-AXI מזיכרון חיצוני, בנוסף לזיכרון המקומי.

Re-customize IP

MicroBlaze (11.0)

Documentation
IP Location
Advanced

IP Symbol

Resource Estimates

Frequency

Area

Performance

Resource Estimates

Resource Usage

BRAM:	DSP48E:
0	0

Component Name: microblaze_o

Buses

Local Memory Bus Interfaces

☒ Enable Local Memory Bus Instruction Interface
☒ Enable Local Memory Bus Data Interface

AXI and ACE Interfaces

Select Bus Interface: AXI

☐ Enable Peripheral AXI Instruction Interface

☒ Enable AXI peripheral instruction interface. If this parameter is set, the peripheral AXI-Lite instruction interface is available. In many cases, this interface is not needed, in particular if the Instruction Cache is enabled and C_ICACHE_ALWAYS_USED is set.

Stream Interfaces

Number of Stream Links: 0 (0 - 16)

Other Interfaces

☐ Enable Trace Bus Interface

Lockstep Interface: NONE

< Back

Next >

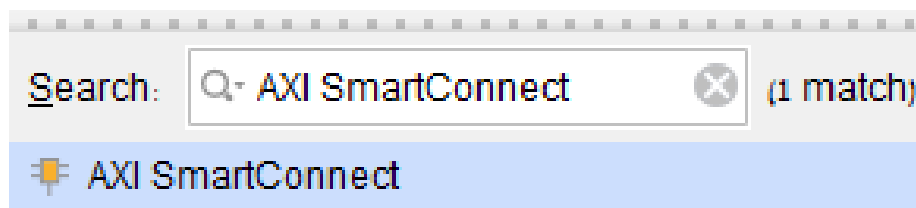
Page 4 of 4

OK

Cancel

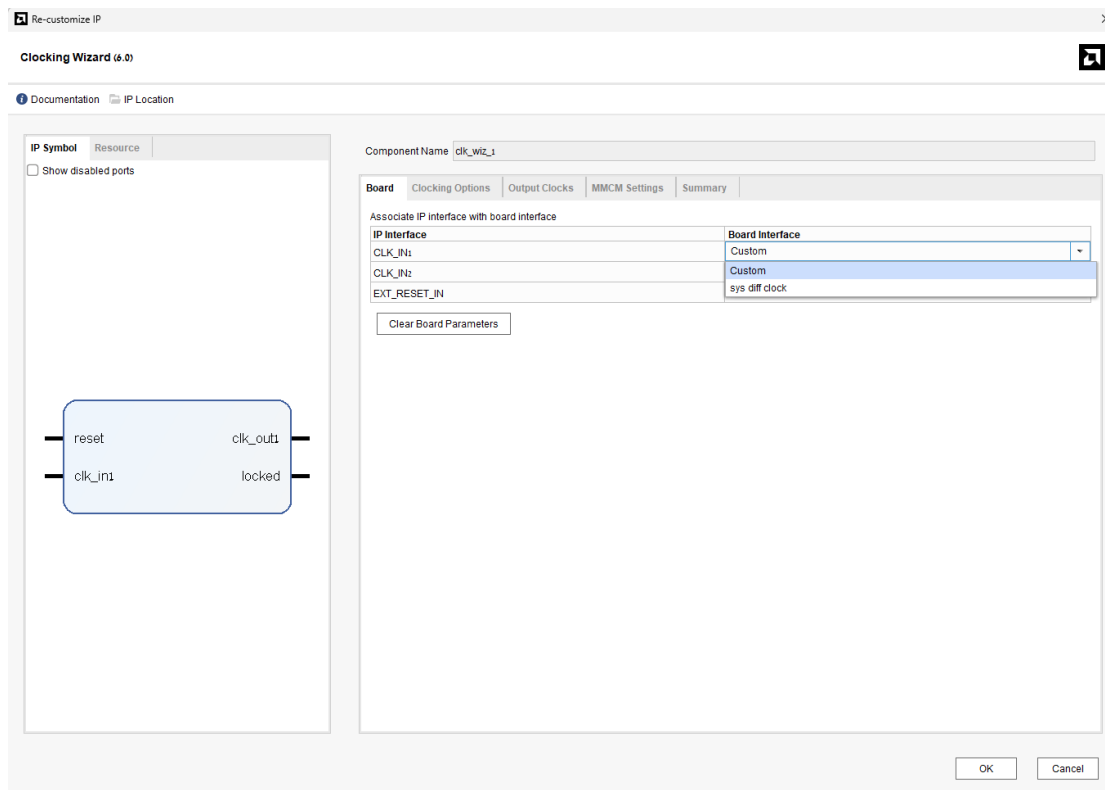
שלב 6:

נוסיד AXI SmartConnect.

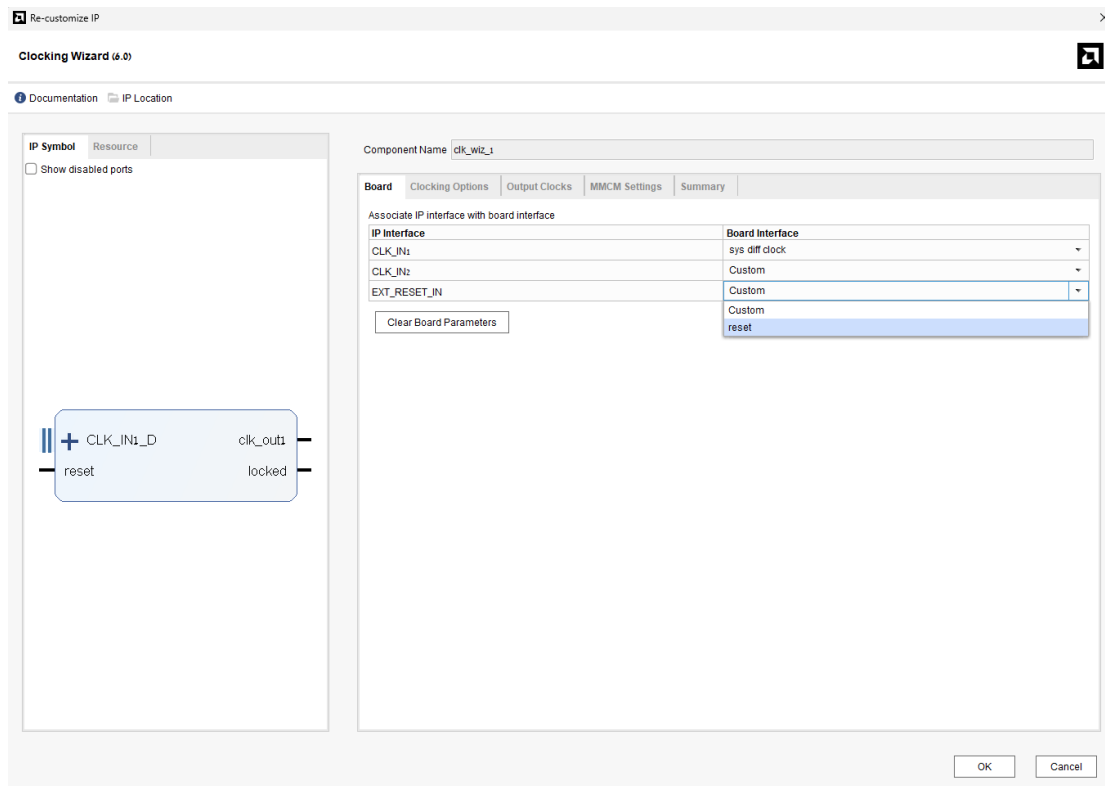


ENTER to select, ESC to cancel, Ctrl+Q for IP details

בהגדרות ה-Clocking Wizard, בלשונית Board, נגדיר את מקור השעון.



נגדיר את שיוך כניסת האיפוס.



בלשונית Output Clocks נשנה את תדר clk_out1 מ-100MHz ל-200MHz ונוסיף יציאת שעון נוספת.

Re-customize IP

Clocking Wizard (6.0)

Documentation IP Location

Component Name: clk_wiz_1

Board Clocking Options Output Clocks MMCM Settings Summary

The phase is calculated relative to the active input clock

Output Clock	Port Name	Output Freq (MHz)		Phase (degrees)		Duty Cycle (%)		Drives
		Requested	Actual	Requested	Actual	Requested	Actual	
☒ clk_out1	clk_out1	200.000	200.000000	0.000	0.000	50.000	50.0	BUFG
☒ clk_out2	clk_out2	12.000	12.000000	0.000	0.000	50.000	50.0	BUFG
☐ clk_out3	clk_out3	100.000	N/A	0.000	N/A	50.000	N/A	BUFG
☐ clk_out4	clk_out4	100.000	N/A	0.000	N/A	50.000	N/A	BUFG
☐ clk_out5	clk_out5	100.000	N/A	0.000	N/A	50.000	N/A	BUFG
☐ clk_out6	clk_out6	100.000	N/A	0.000	N/A	50.000	N/A	BUFG
☐ clk_out7	clk_out7	100.000	N/A	0.000	N/A	50.000	N/A	BUFG

☐ USE CLOCK SEQUENCING

Clocking Feedback

Output Clock	Sequence Number
clk_out1	1
clk_out2	1
clk_out3	1
clk_out4	1
clk_out5	1
clk_out6	1
clk_out7	1

Source

☒ Automatic Control On-Chip
☐ Automatic Control Off-Chip
☐ User-Controlled On-Chip
☐ User-Controlled Off-Chip

Signaling

☒ Single-ended
☐ Differential

IP Symbol Resource

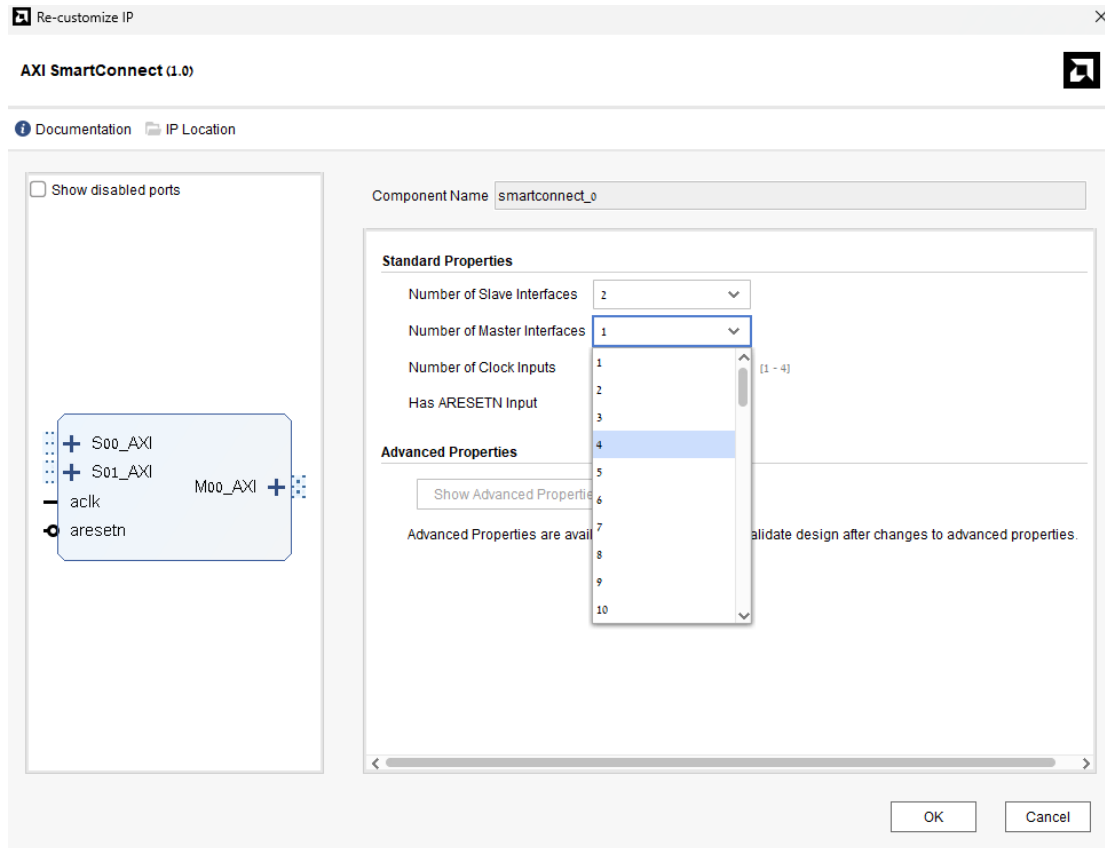
☐ Show disabled ports

CLK_IN1_D
reset
clk_out1
clk_out2
locked

OK Cancel

שלב 7:

נגדיר ב-SmartConnect את מספר הממשקים.



נגדיר את מספר כניסות השעון.

Re-customize IP

AXI SmartConnect (1.0)

Documentation IP Location

☐ Show disabled ports

Component Name: smartconnect_0

Standard Properties

Number of Slave Interfaces: 2

Number of Master Interfaces: 4

Number of Clock Inputs: 1 (1 - 7)

Has ARESETN Input: 1

Advanced Properties

Has ARESETN Input

Show Advanced Properties

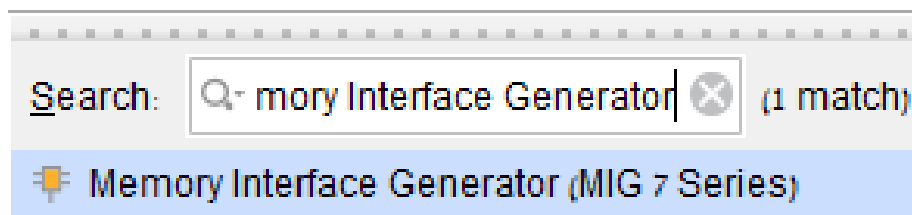
Advanced Properties are available after validation. Re-validate design after changes to advanced properties.

OK Cancel



שלב 8:

נוסיף רכיב Memory Interface Generator MIG 7 Series ונריץ Run Block Automation.



ENTER to select, ESC to cancel, Ctrl+Q for IP details

בהגדרות הבקר נגדיר את שעון המערכת כ-No Buffer.

AMD Vivado ML Edition

Pin Compatible FPGAs
Memory Selection
Controller Options
AXI Parameter
Memory Options
FPGA Options
Extended FPGA Options
IO Planning Options
Bank Selection
System Signals Selection
Summary
Simulation Options
PCB information
Design Notes

System Clock
Choose the desired input clock configuration. Design clock can be Differential or Single-Ended.
System Clock: Differential

Reference Clock
Choose the desired reference clock configuration. Reference clock can be Differential or Single-Ended.
Reference Clock: Differential (dropdown menu open showing: Differential, Single-Ended, No Buffer, Differential)

System Reset Polarity
Choose the desired System Reset Polarity.
System Reset Polarity: ACTIVE LOW

Internal Vref
Internal Vref can be used to allow the use of the Vref pins as normal IO pins. This option can only be used at 800 Mbps and lower data rates. This can free 2 pins per bank where inputs are used. This setting has no effect on banks with only outputs.
Internal Vref: ☐

IO Power Reduction
Significantly reduces average IO power by automatically disabling DQ/DQS IBUFs and internal terminations during WRITES and periods of inactivity.
IO Power Reduction: ON

XADC Instantiation
The memory interface uses the temperature reading from the XADC block to perform temperature compensation and keep the read DQS centered in the data window. There is one XADC block per device. If the XADC is not currently used anywhere in the design, enable this option to have the block instantiated. If the XADC is already used, disable this MIG option. The user is then required to provide the temperature value to the top level 12-bit device_temp_i input port. Refer to Answer Record 51487 or the UG584 for detailed information.

User Guide < Back Next > Cancel

נגדיר גם את שעון הייחוס כ-No Buffer, כך שהשעונים מגיעים מה-Clocking Wizard.

AMD Vivado ML Edition

Pin Compatible FPGAs
Memory Selection
Controller Options
AXI Parameter
Memory Options
FPGA Options
Extended FPGA Options
IO Planning Options
Bank Selection
System Signals Selection
Summary
Simulation Options
PCB information
Design Notes

System Clock
Choose the desired input clock configuration. Design clock can be Differential or Single-Ended.
System Clock: Differential

Reference Clock
Choose the desired reference clock configuration. Reference clock can be Differential or Single-Ended.
Reference Clock: Differential (dropdown menu open showing: Differential, Single-Ended, No Buffer, Use System Clock)

System Reset Polarity
Choose the desired System Reset Polarity.
System Reset Polarity: ACTIVE LOW

Internal Vref
Internal Vref can be used to allow the use of the Vref pins as normal IO pins. This option can only be used at 800 Mbps and lower data rates. This can free 2 pins per bank where inputs are used. This setting has no effect on banks with only outputs.
Internal Vref: ☐

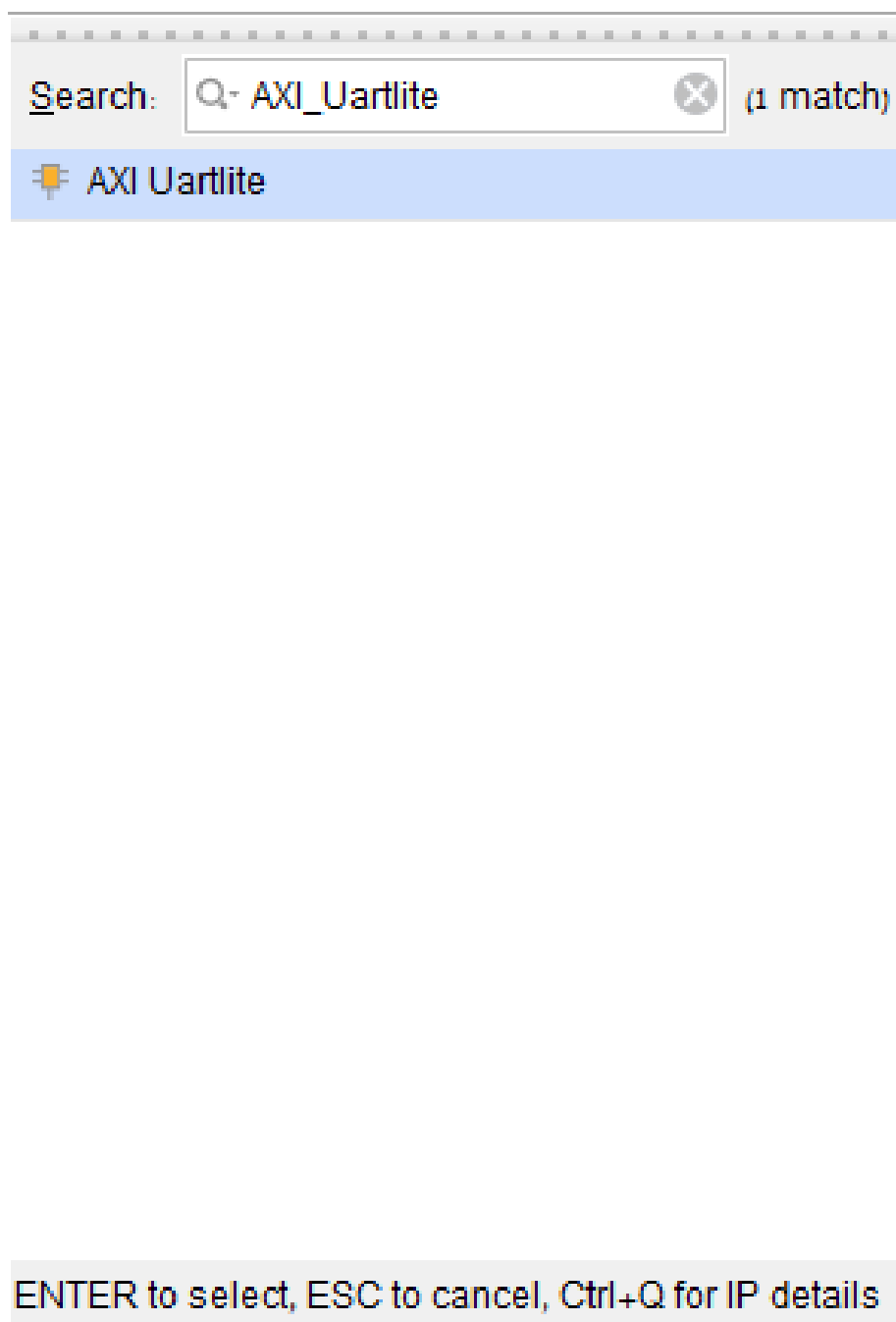
IO Power Reduction
Significantly reduces average IO power by automatically disabling DQ/DQS IBUFs and internal terminations during WRITES and periods of inactivity.
IO Power Reduction: ON

XADC Instantiation
The memory interface uses the temperature reading from the XADC block to perform temperature compensation and keep the read DQS centered in the data window. There is one XADC block per device. If the XADC is not currently used anywhere in the design, enable this option to have the block instantiated. If the XADC is already used, disable this MIG option. The user is then required to provide the temperature value to the top level 12-bit device_temp_i input port. Refer to Answer Record 51487 or the UG584 for detailed information.

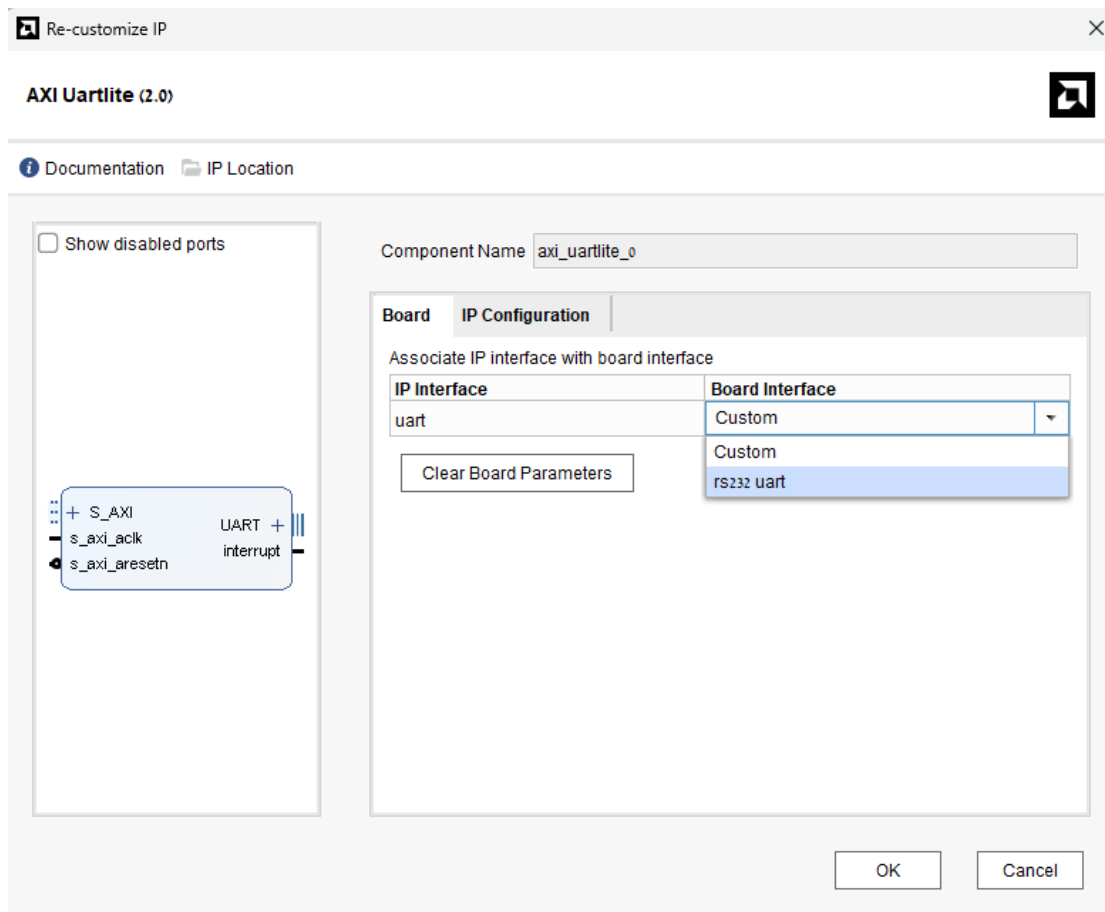
User Guide < Back Next > Cancel

שלב 9:

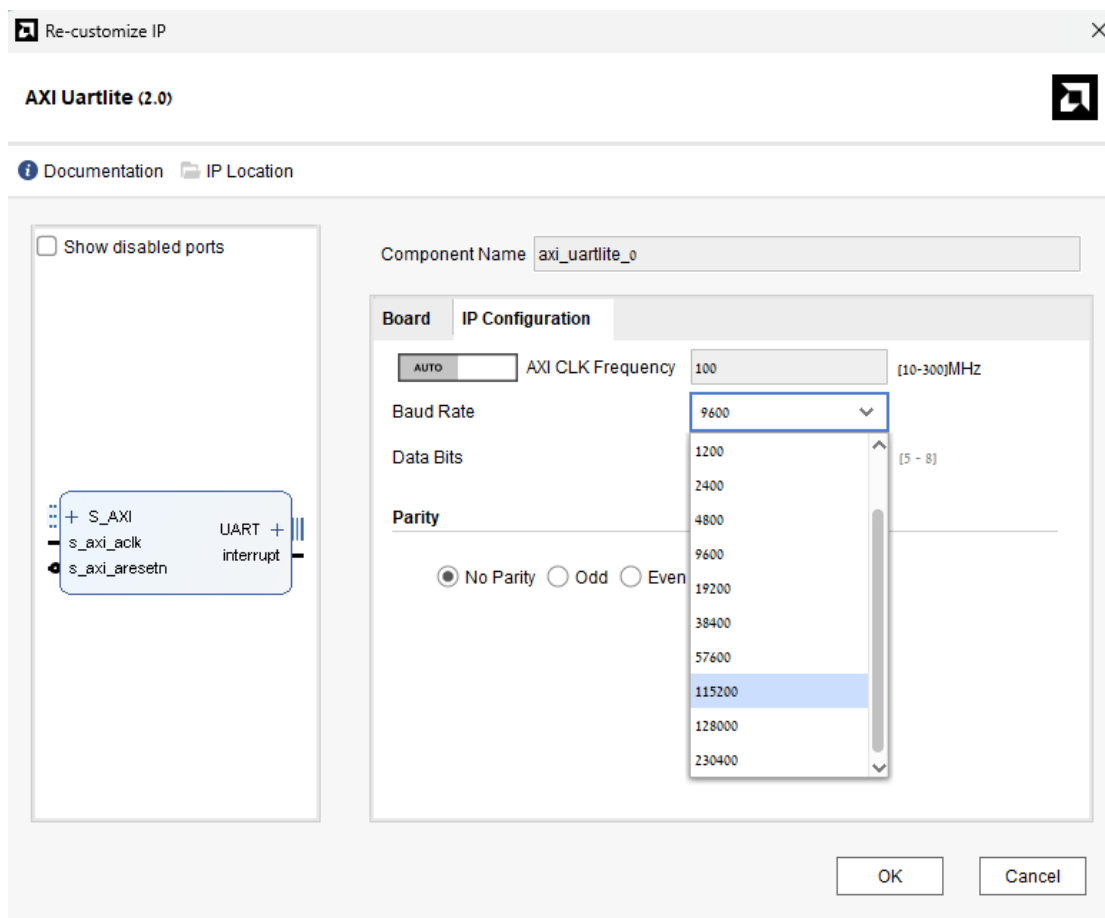
נוסיף רכיב AXI Uartlite.



נשייך אותו לממשק rs232 uart של הלוח.

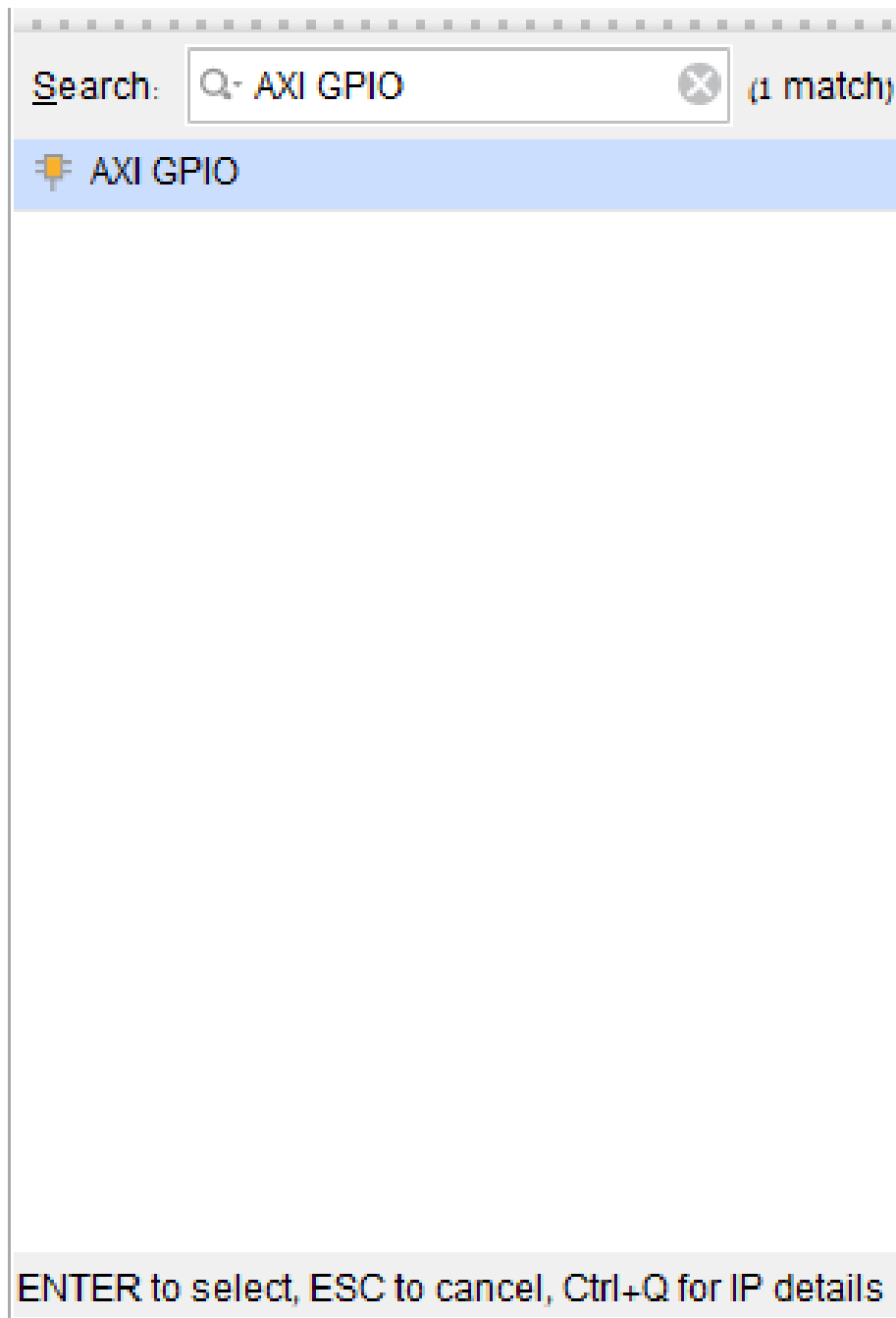


נגדיר קצב של 115200.

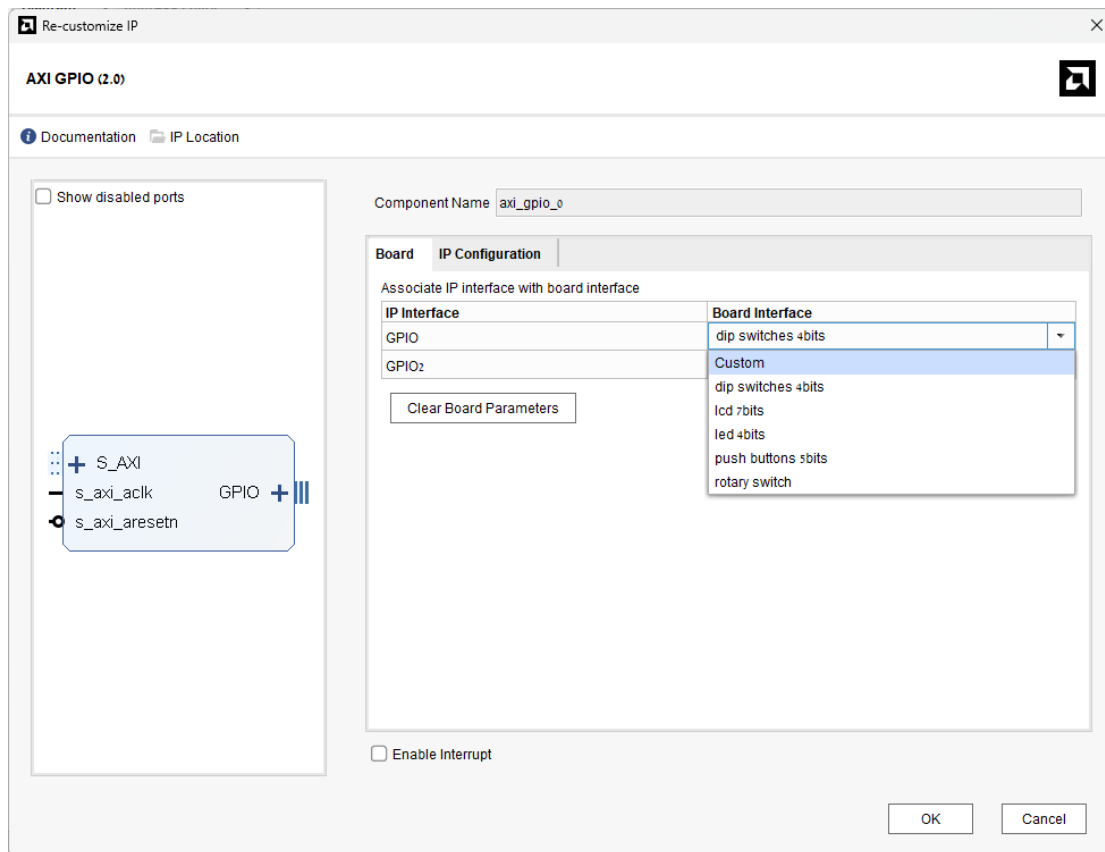


שלב 10:

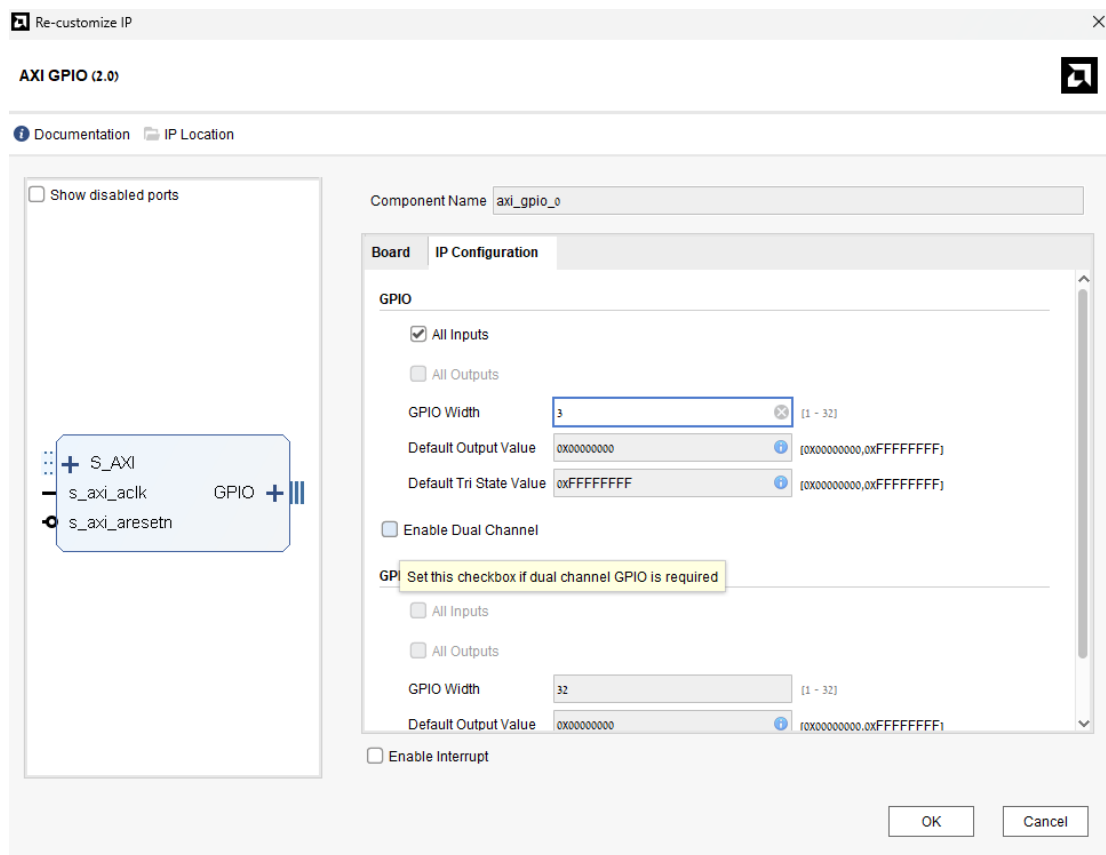
נוסיף רכיב AXI GPIO.



בלשונית Board נשאיר את השיוך Custom.

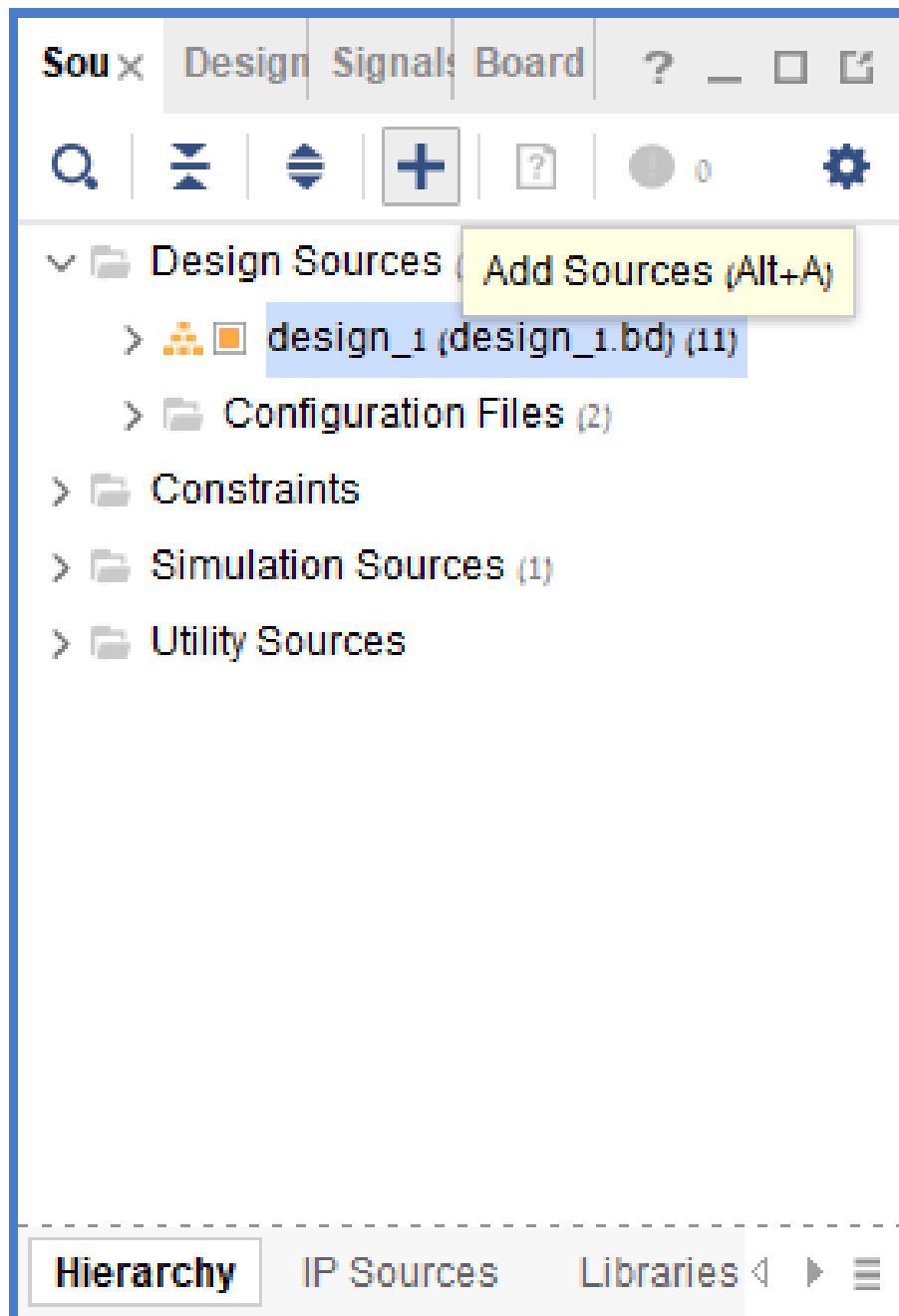


בלשונית IP Configuration נגדיר את הערוץ הראשון כ-All Inputs ברוחב 3 ביטים ונסמן Enable Dual Channel. הערוץ הראשון קורא את מצב הלחצנים והערוץ השני קורא את מצב הלדים.

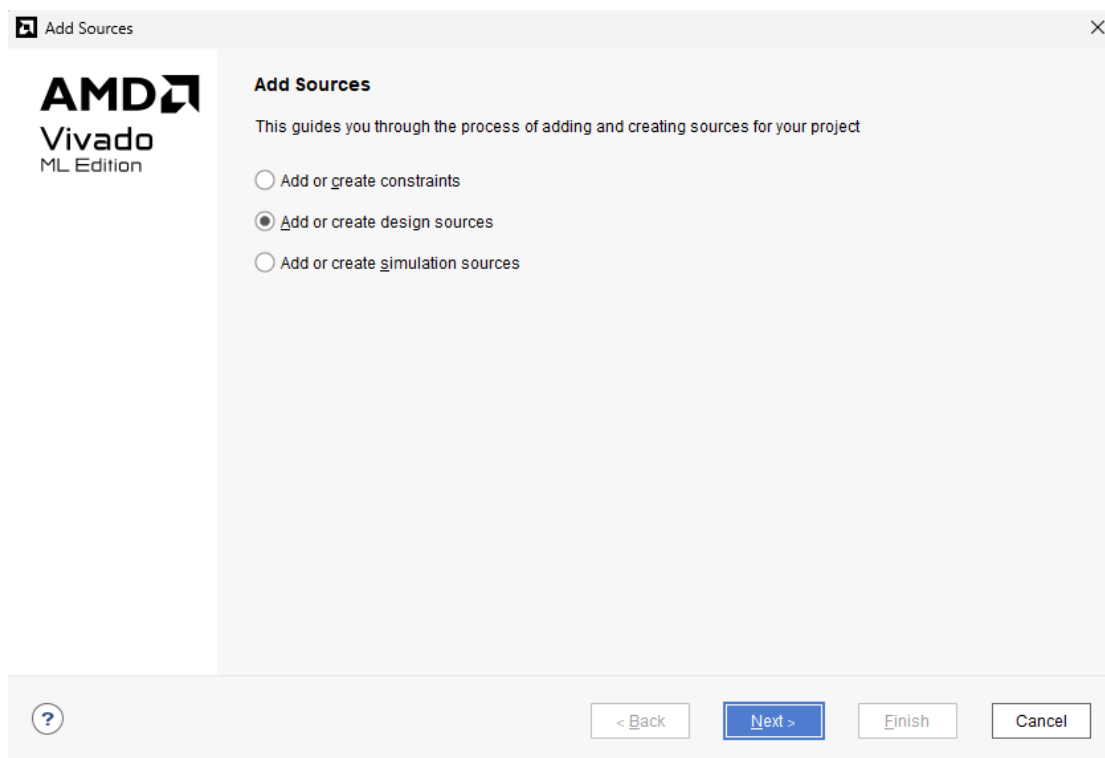


שלב 11:

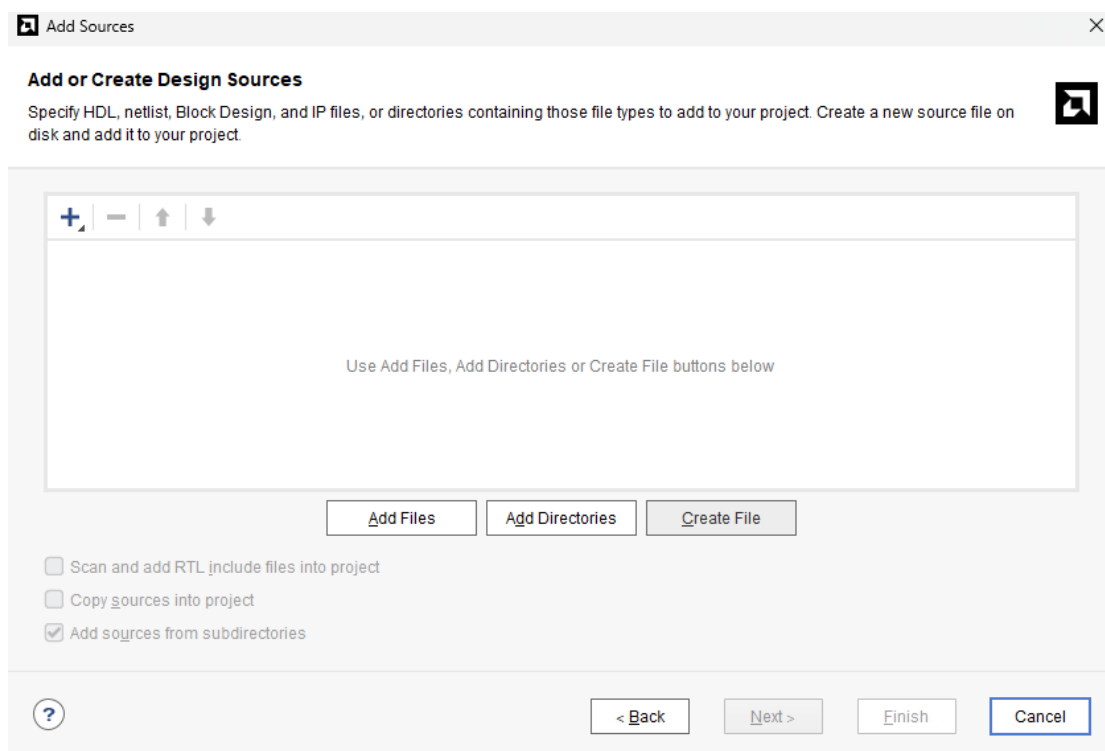
נלחץ Add Sources.



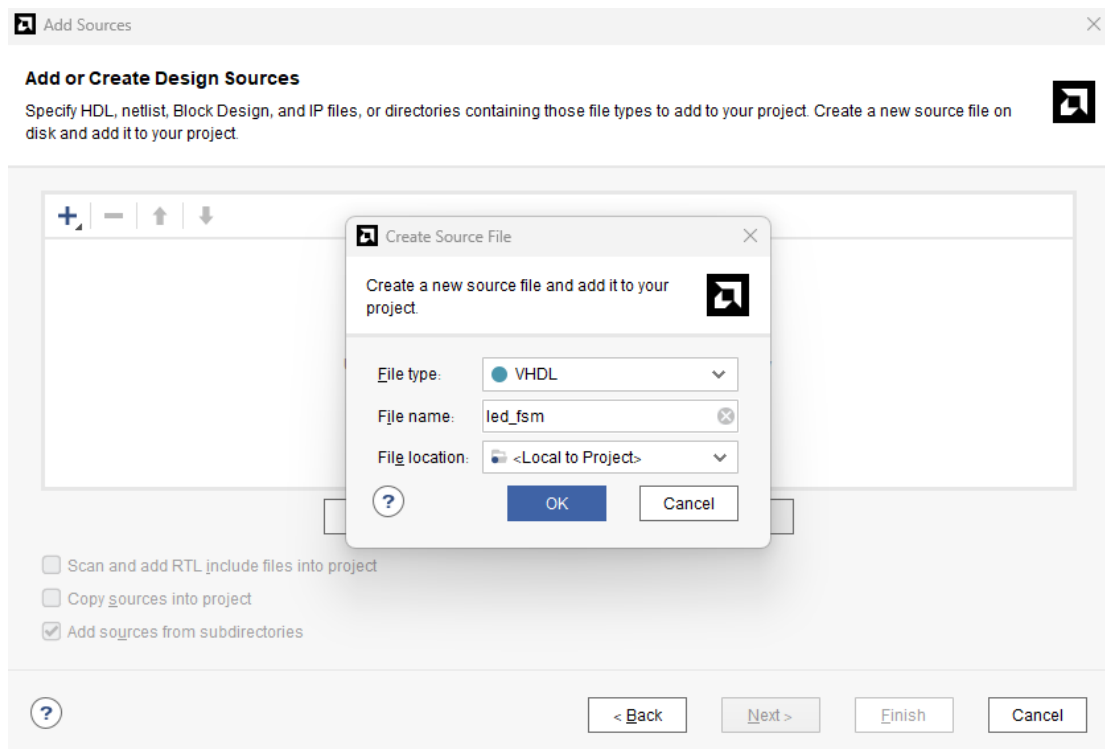
נבחר Add or create design sources.



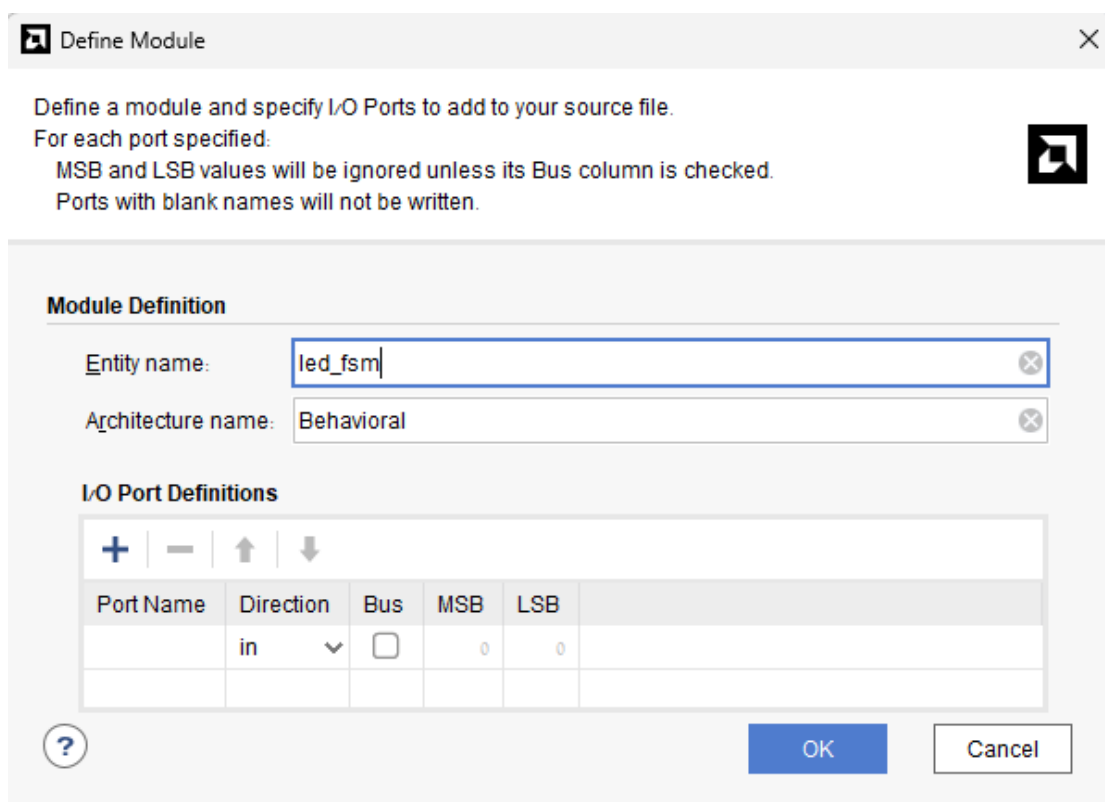
נלחץ Create File.



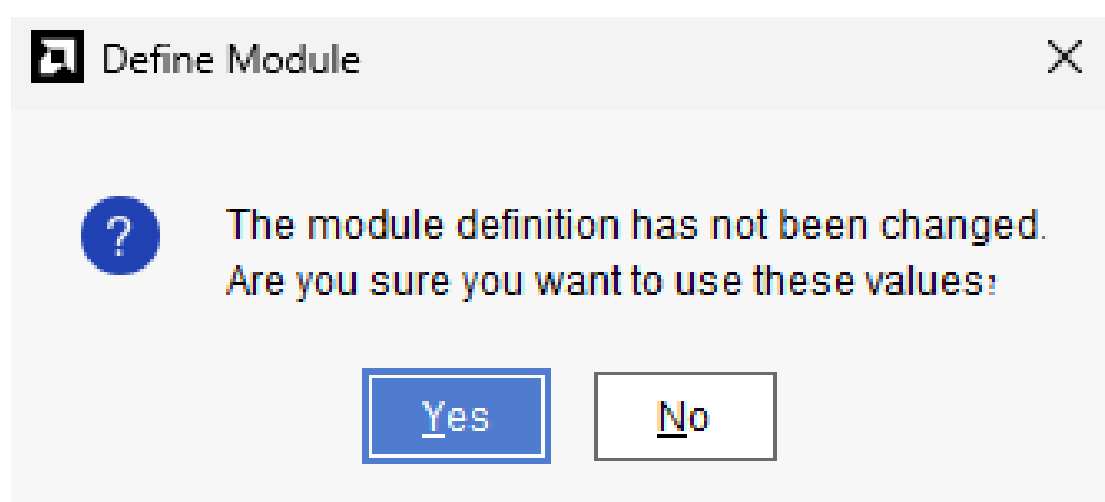
ניצור קובץ VHDL בשם led_fsm.



נלחץ OK בחלון Define Module.



נאשר ב-Yes.



נכתוב את קוד המכונה. במצב idle הלדים כבויים, וכאשר כניסת enable עולה ל-1 המכונה עוברת למצב enab ומעבירה את ערך הכניסה sw_in ליציאה led_enb.

```

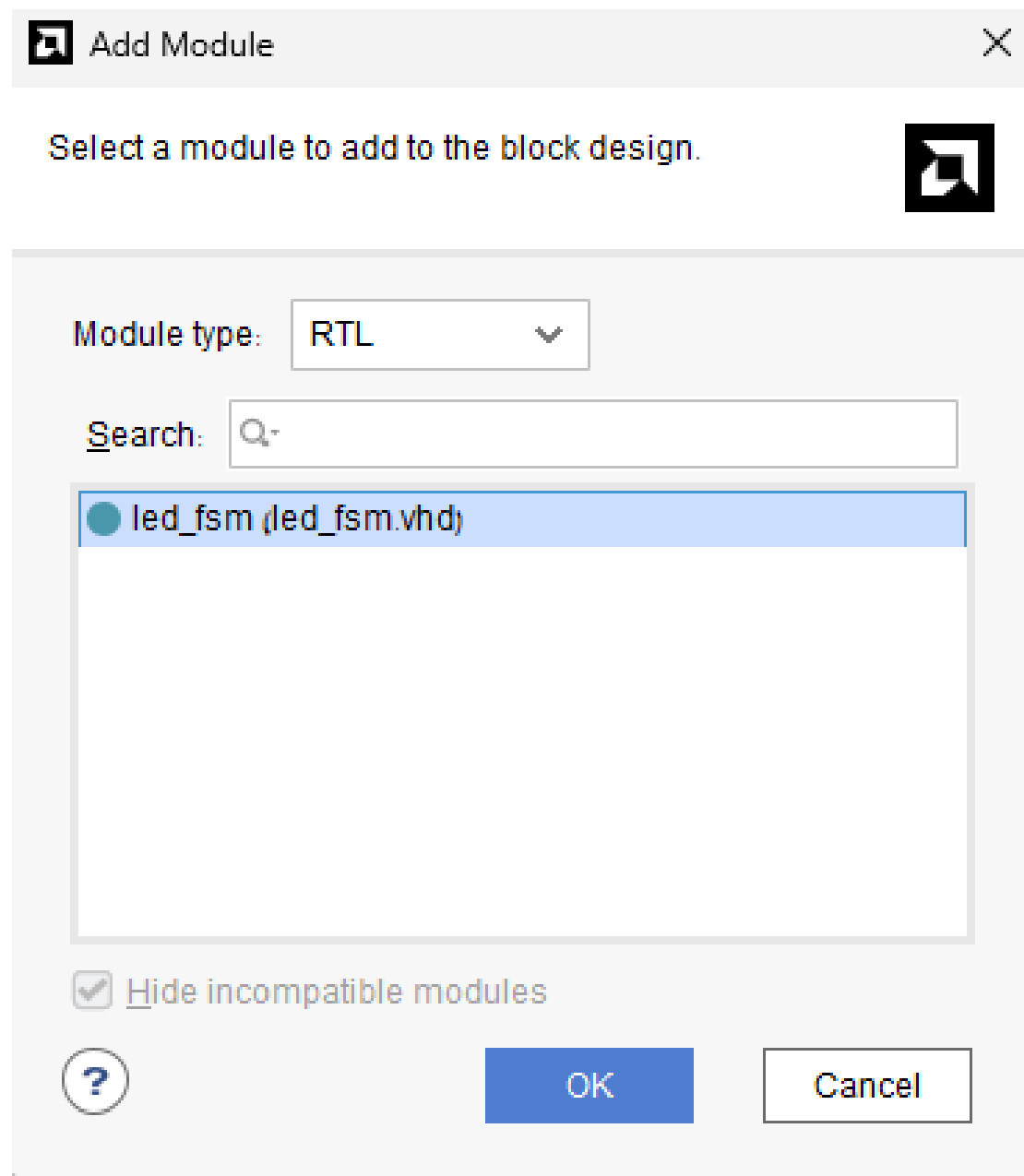
led_fsm.vhd
C:/Users/a11442/Desktop/MicroBlaze_DDR_GPIO_05/MicroBlaze_DDR_GPIO/MicroBlaze_DDR_GPIO
1 library ieee;
2 use ieee.std_logic_1164.all;
3
4 entity led_fsm is
5 port (
6     clk      : in  std_logic;
7     enable   : in  std_logic;
8     sw_in    : in  std_logic_vector(2 downto 0);
9     enb_led  : out std_logic;
10    led_enb   : out std_logic_vector(2 downto 0)
11 );
12 end entity led_fsm;
13
14 architecture rtl of led_fsm is
15     type fsm_t is (idle, enab);
16     signal current_state : fsm_t := idle;
17 begin
18     fsm_proc : process(clk)
19     begin
20         if rising_edge(clk) then
21             case current_state is
22                 when idle =>
23                     enb_led <= '0';
24                     led_enb <= (others => '0');
25                     if enable = '1' then
26                         current_state <= enab;
27                     end if;
28
29                 when enab =>
30                     enb_led <= '1';
31                     led_enb <= sw_in;
32                     if enable = '0' then
33                         current_state <= idle;
34                     end if;
35
36                 when others =>
37                     null;
38             end case;
39         end if;
40     end process;
41 end architecture rtl;
42

```

נלחץ קליק ימני ב-Block Design ובבחר Add Module.

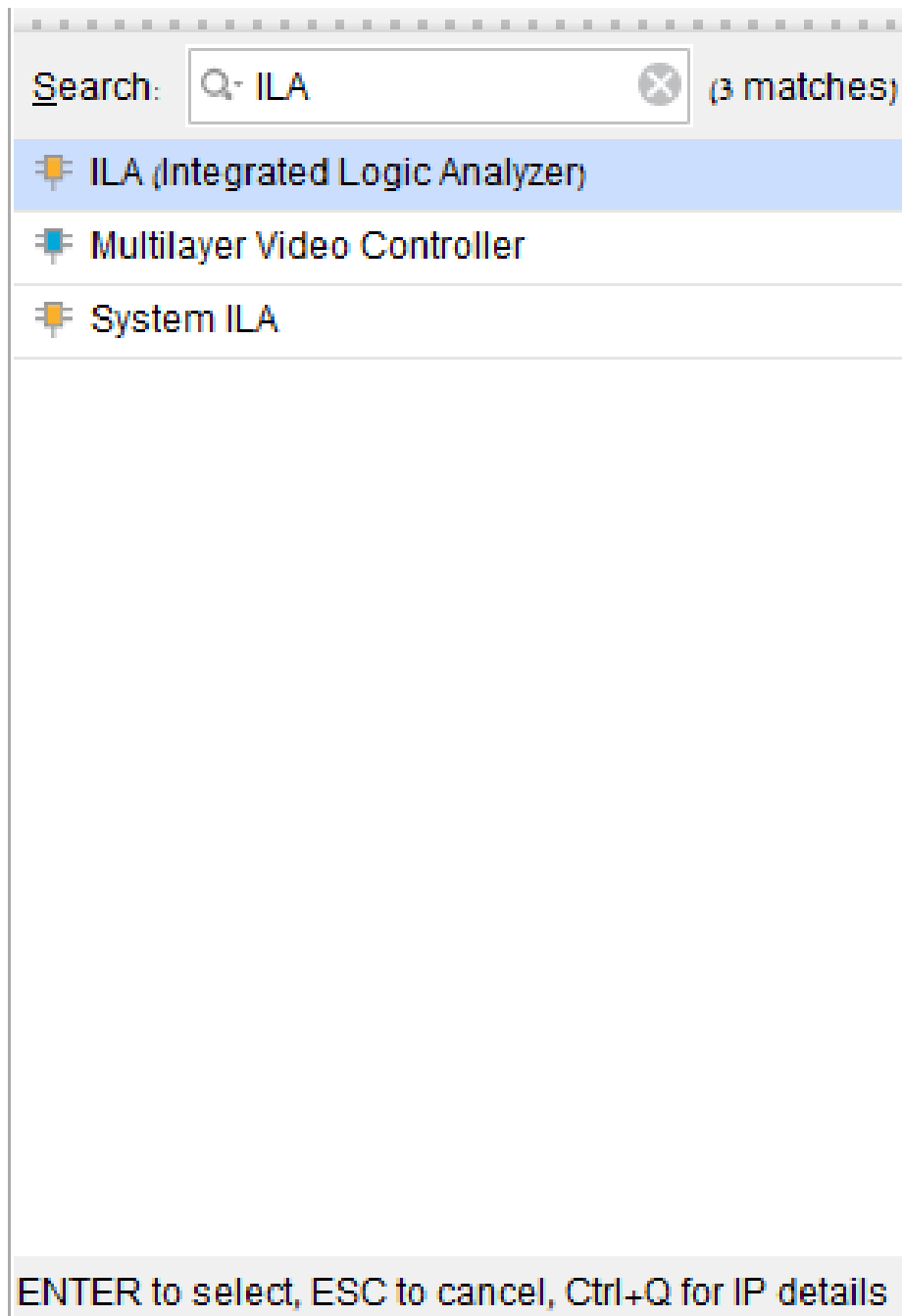
	Properties...	Ctrl+E
	Delete	Delete
	Copy	Ctrl+C
	Paste	Ctrl+V
	Search...	Ctrl+F
	Select All	Ctrl+A
	Add IP...	Ctrl+I
	Add Module...	
	Run Connection Automation...	
	Pinning	▶
	Validate Design	F6
	Expand/Collapse	▶
	Create Hierarchy...	
	Create Comment	
	Create Port...	Ctrl+K
	Create Interface Port...	Ctrl+L
	Regenerate Layout	
	Save as PDF File...	

נבחר את led_fsm ונלחץ OK.

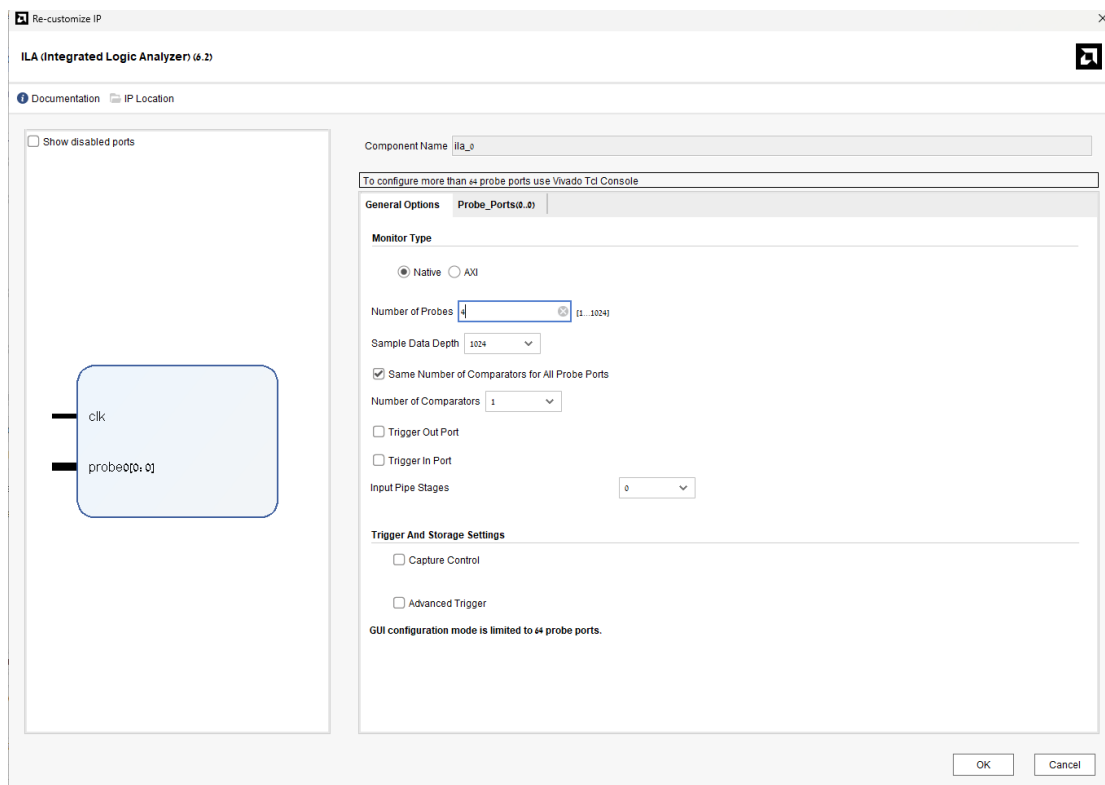


שלב 12:

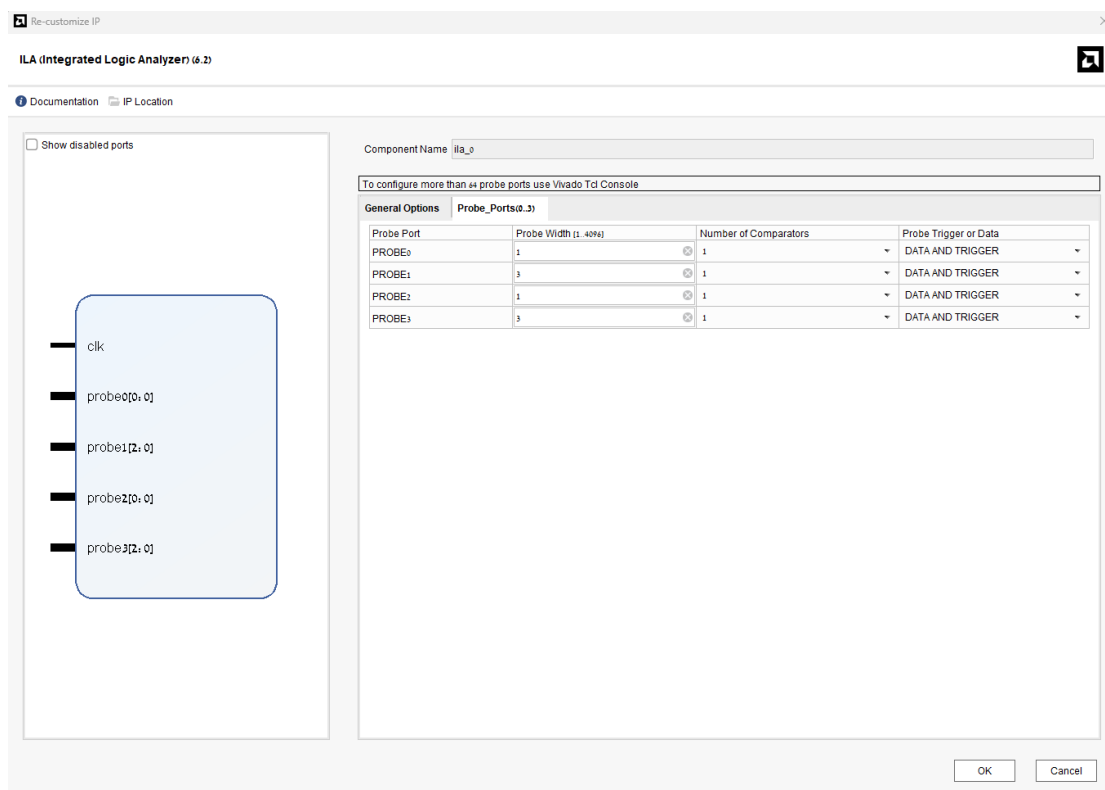
נוסיף רכיב ILA.



נגדיר מצב Native וארבעה פרובים.

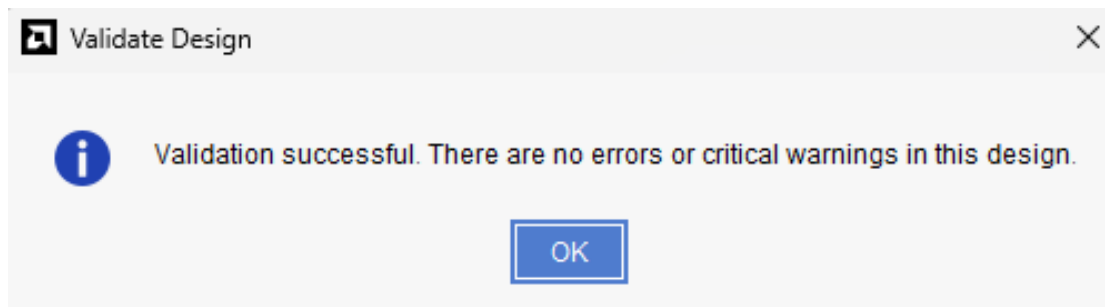


נגדיר את רוחב probe1 ו-probe2 ל-3 ביטים לדגימת וקטורי הלחצנים והלדים.

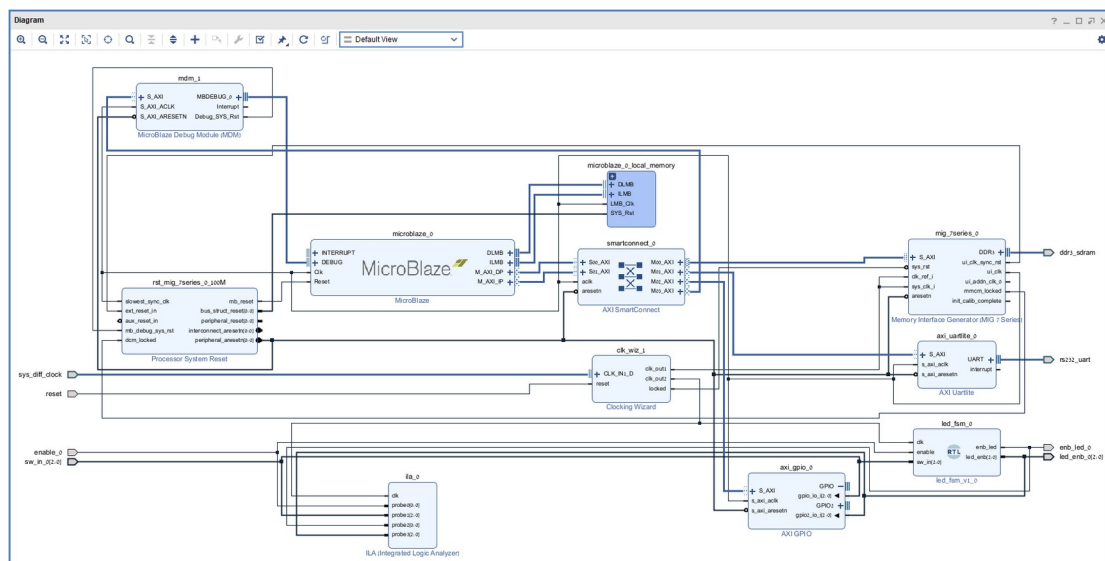


שלב 13:

נריץ Validate Design.

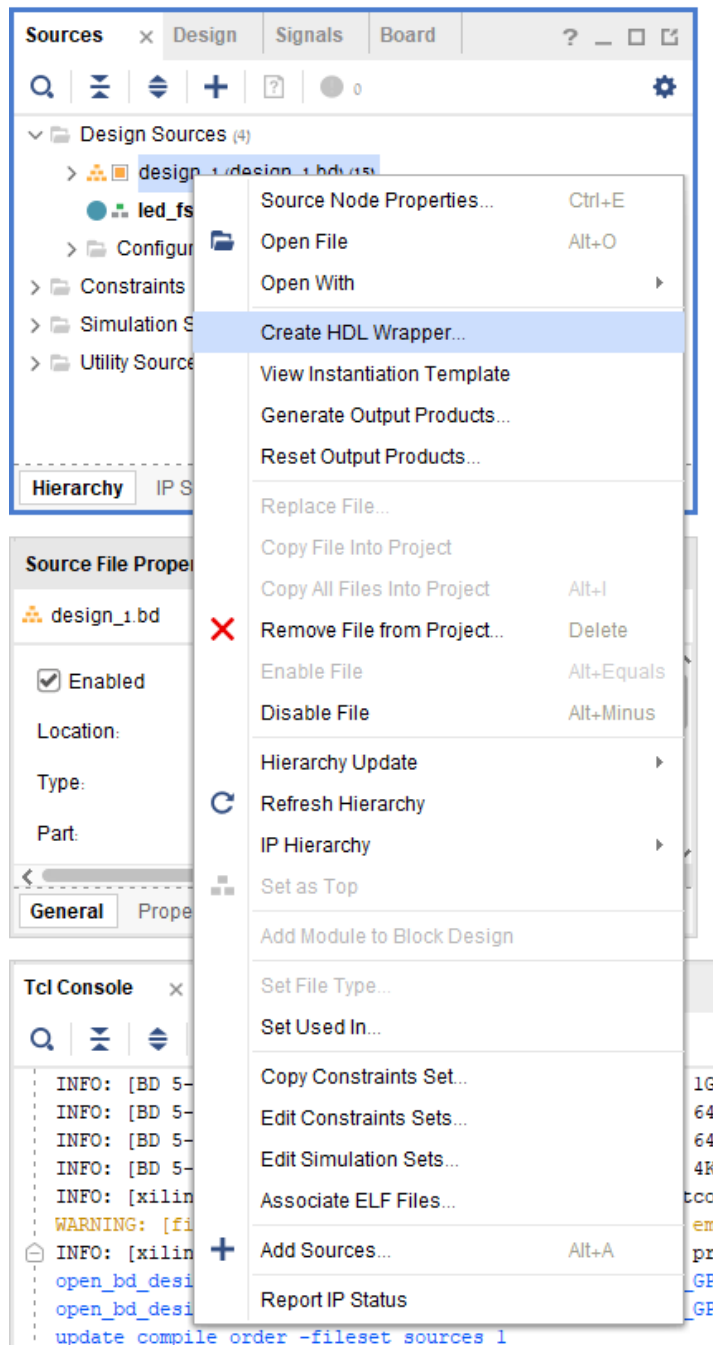


זוהו התכן המלא:

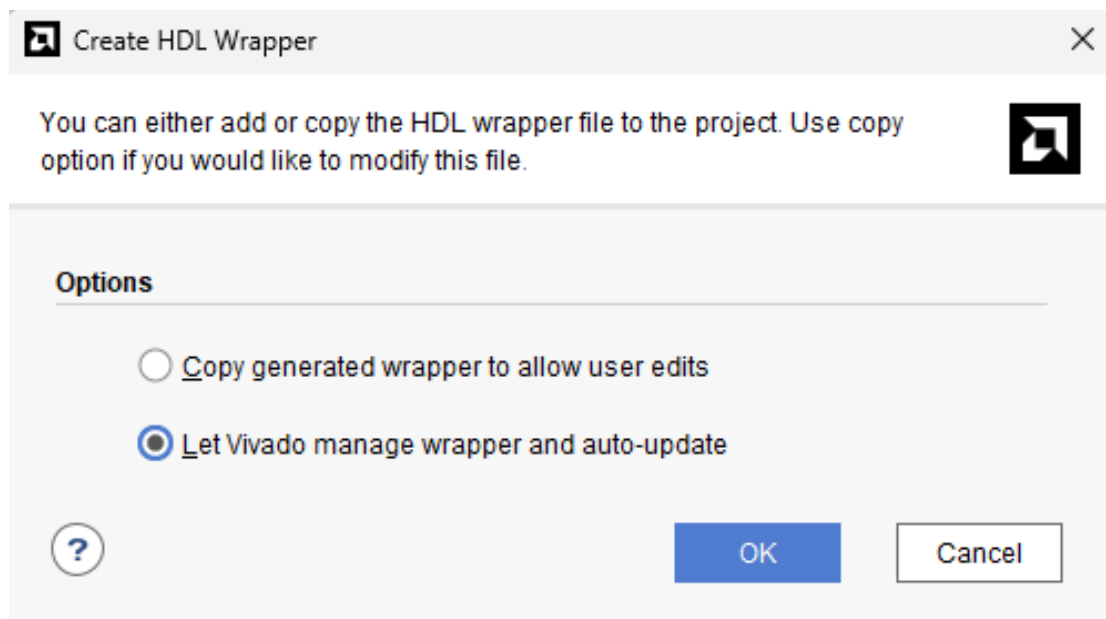


שלב 14:

ניצור HDL Wrapper.

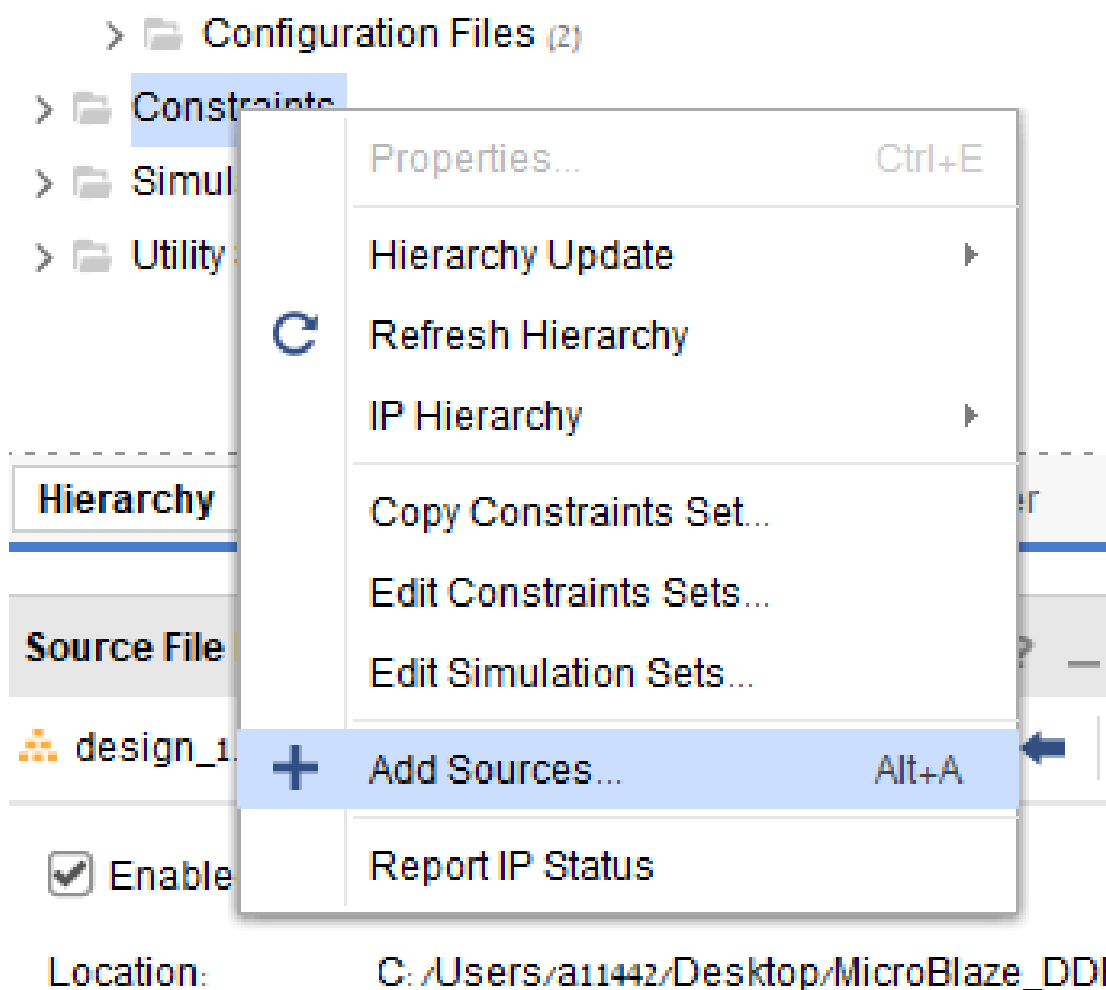


נשאיר את הניהול האוטומטי ונלחץ OK.

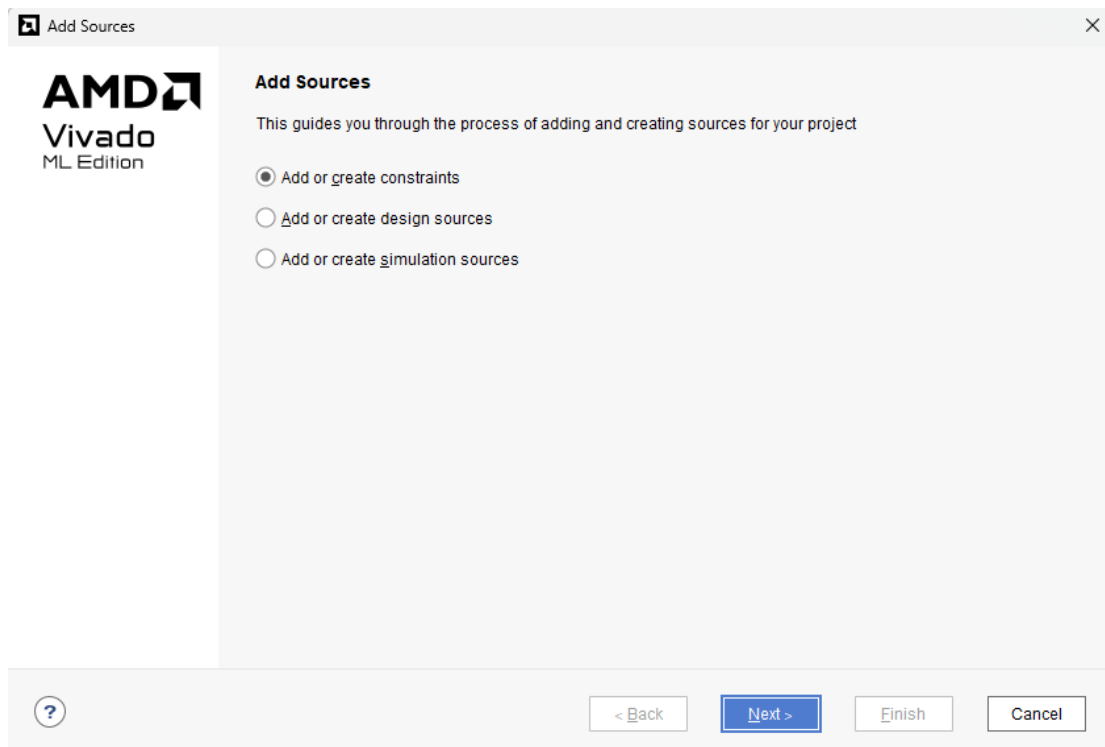


שלב 15:

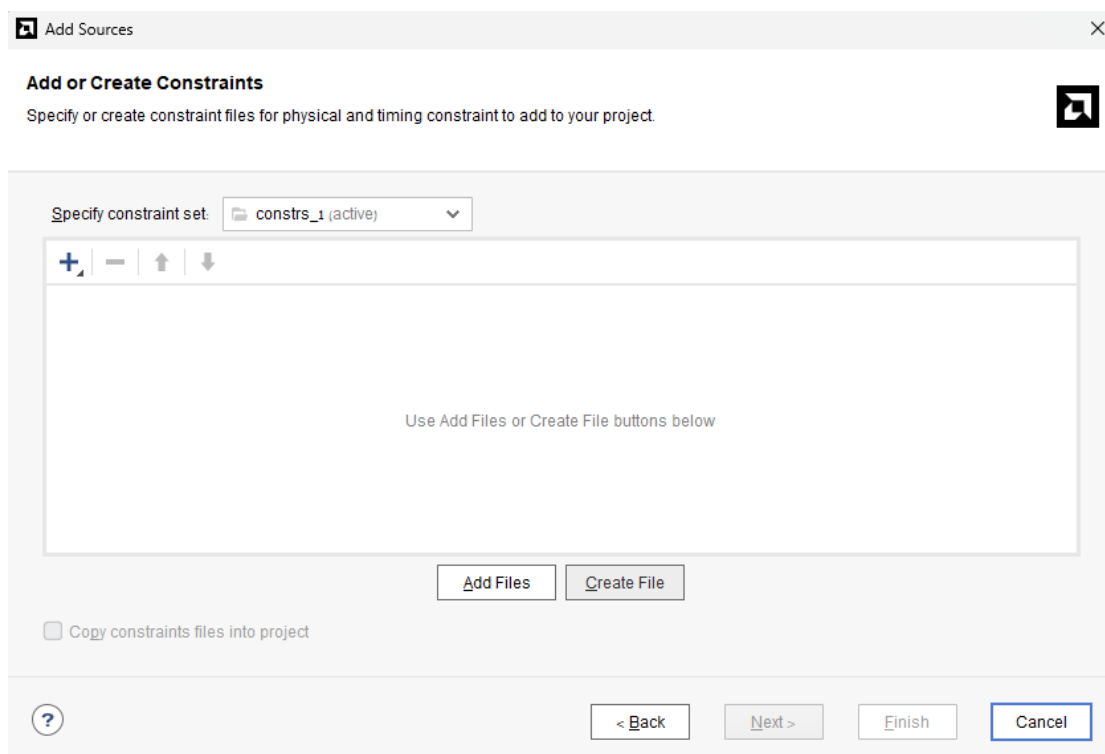
נוסיף קובץ אילוצים דרך Add Sources.



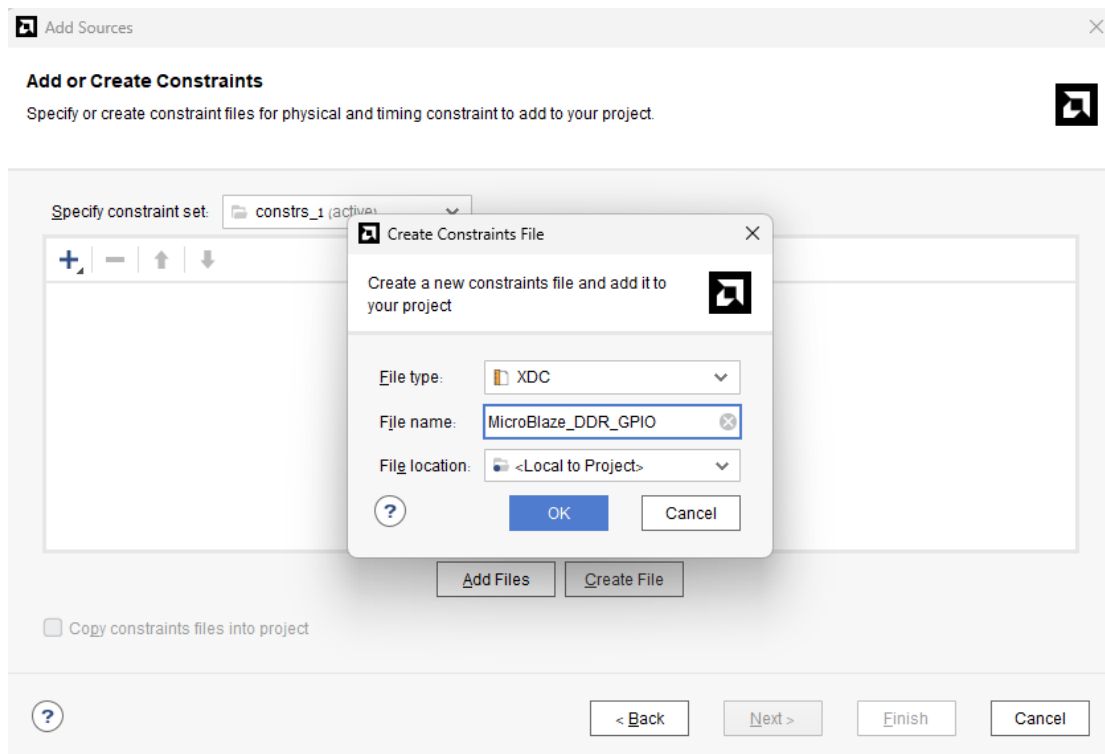
נבחר Add or create constraints.



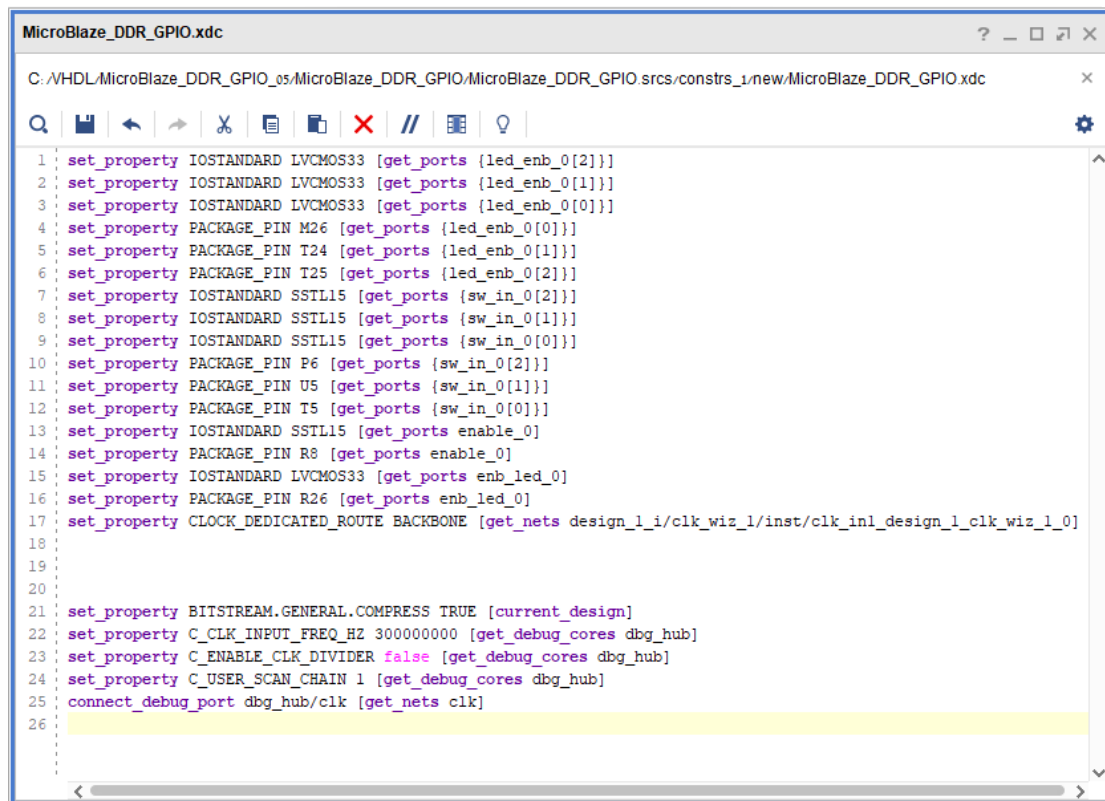
נלחץ Create File.



ניצור קובץ XDC בשם MicroBlaze_DDR_GPIO.



בתכנון המקורי נועדו כניסות [sw_in_0[2:0]] למפסקי ה-DIP, אך בפועל מופו ללחצני הלוח. הלחצנים ממלאים את אותו תפקיד של כניסה בינארית בת שלושה ביטים, והבחירה בהם חסכה עדיין נוסף של קובץ האילוצים וקימפול מחדש.



שלב 16:

בחלון Address Editor נגדיל את טווח הזיכרון המקומי מ-16K ל-32K.

Diagram

Address Editor

☒ Assigned (10)
 ☒ Unassigned (0)
 ☒ Excluded (0)

Hide All

Name	Interface	Slave Segment	Master Base Address	Range	Master High Address
Network 0					
microblaze_0					
microblaze_0/Data (32 address bits : 4G)					
/axi_gpio_0/S_AXI	S_AXI	Reg	0x4000_0000	64K	0x4000_FFFF
/axi_uartlite_0/S_AXI	S_AXI	Reg	0x4060_0000	64K	0x4060_FFFF
/mdm_1/S_AXI	S_AXI	Reg	0x4140_0000	4K	0x4140_0FFF
/microblaze_0_local_memory/dlmb_bram_if_cntlr/SLMB	SLMB	Mem	0x0	32K	0x7FFF
/mig_7series_0/memmap	S_AXI	memaddr	0x8000_0000	1G	0xBFFF_FFFF
microblaze_0/Instruction (32 address bits : 4G)					
/axi_gpio_0/S_AXI	S_AXI	Reg	0x4000_0000	64K	0x4000_FFFF
/axi_uartlite_0/S_AXI	S_AXI	Reg	0x4060_0000	64K	0x4060_FFFF
/mdm_1/S_AXI	S_AXI	Reg	0x4140_0000	4K	0x4140_0FFF
/microblaze_0_local_memory/ilmb_bram_if_cntlr/SLMB	SLMB	Mem	0x0	32K	0x7FFF
/mig_7series_0/memmap	S_AXI	memaddr	0x8000_0000	1G	0xBFFF_FFFF

שלב 17:

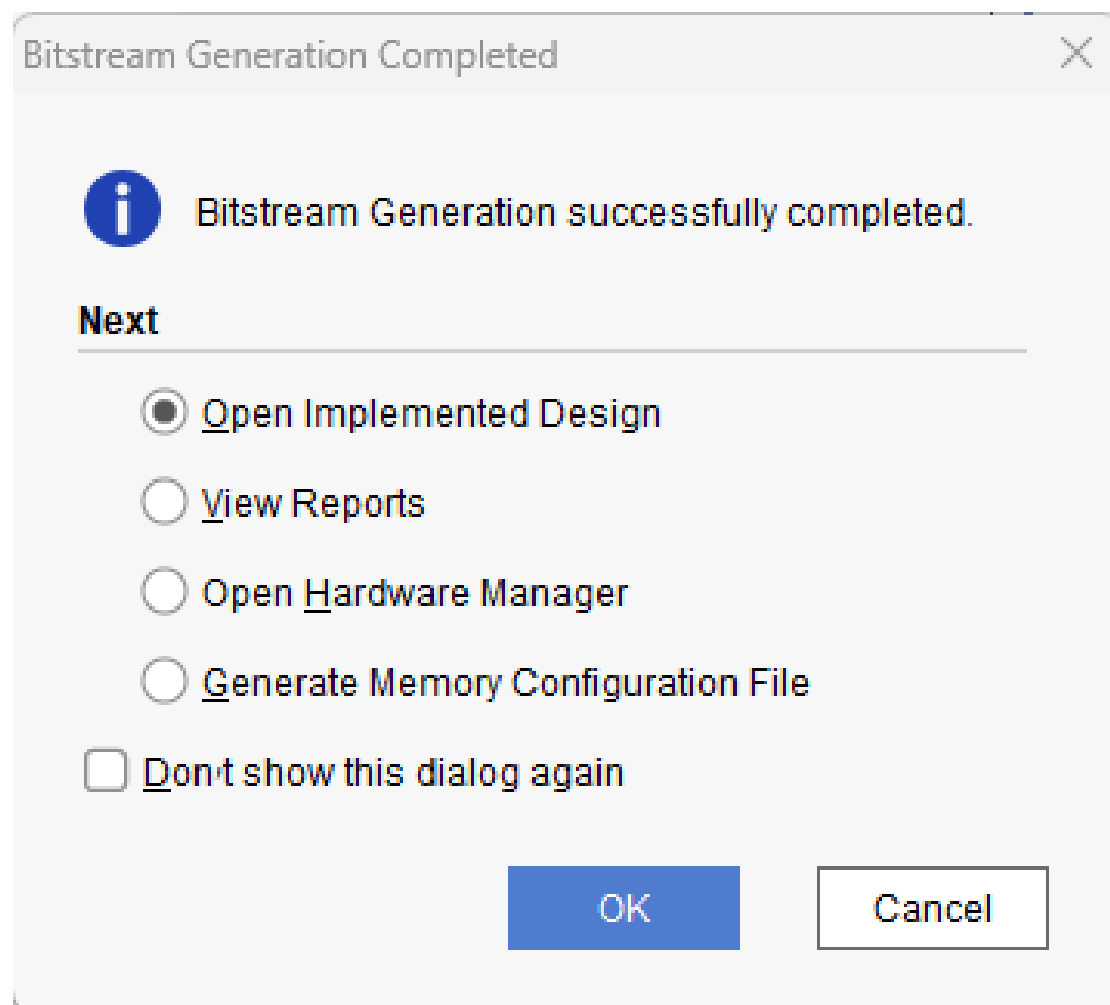
נלחץ Generate Bitstream.

▼ PROGRAM AND DEBUG

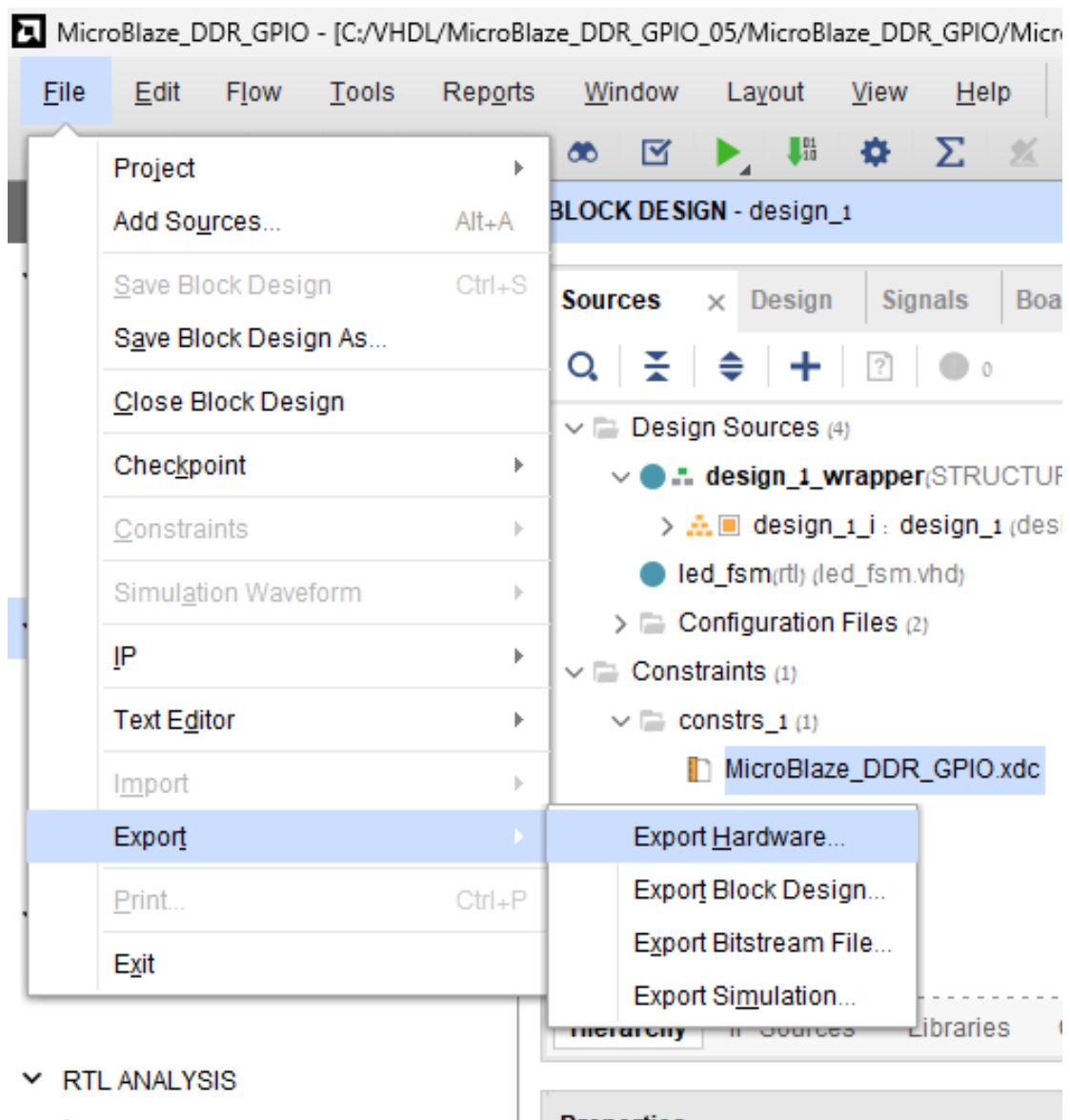


Generate Bitstream

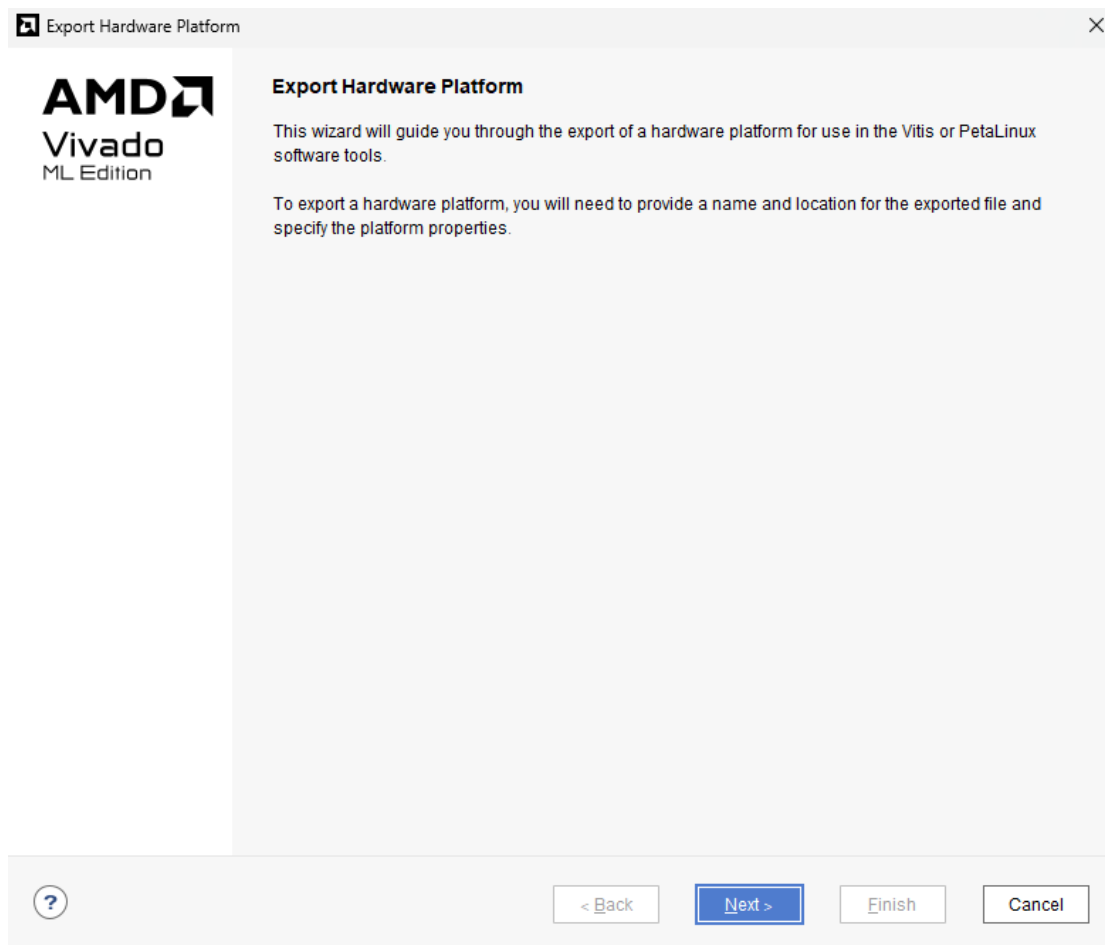
- > Open Hardware Manager



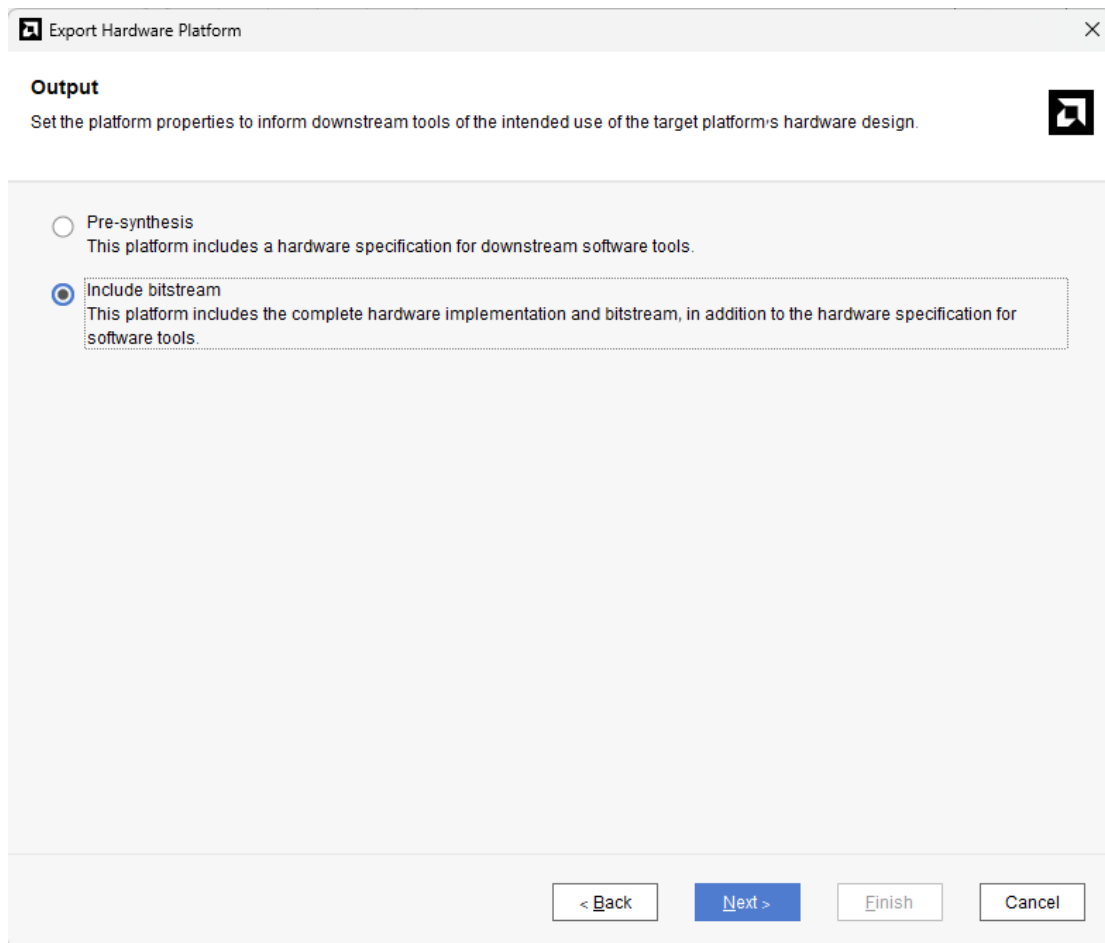
נמצא את החומרה דרך File, Export Hardware.



הלחץ Next.



נבחר Include bitstream.



The image shows a software dialog box titled "Export Hardware Platform". It has a close button (X) in the top right corner. Below the title bar, there is a section labeled "Output" with a description: "Set the platform properties to inform downstream tools of the intended use of the target platform's hardware design." To the right of this text is a small icon of a square with an arrow pointing out. Below this, there are two radio button options. The first option is "Pre-synthesis" with the description "This platform includes a hardware specification for downstream software tools." The second option is "Include bitstream" with the description "This platform includes the complete hardware implementation and bitstream, in addition to the hardware specification for software tools." The "Include bitstream" option is selected, indicated by a blue dot. At the bottom of the dialog, there are four buttons: "< Back", "Next >" (highlighted in blue), "Finish", and "Cancel".

Export Hardware Platform [X]

Output
Set the platform properties to inform downstream tools of the intended use of the target platform's hardware design.

☐ Pre-synthesis
This platform includes a hardware specification for downstream software tools.

☒ Include bitstream
This platform includes the complete hardware implementation and bitstream, in addition to the hardware specification for software tools.

< Back Next > Finish Cancel

נגדיר שם ומיקום לקובץ ה-XSA.

 Export Hardware Platform ✕

Files 

Enter the name of your hardware platform file, and the directory where the XSA file will be stored.

XSA file name:

Export to: ✕ ...

The XSA will be written to: C:\VHDL\MicroBlaze_DDR_GPIO_05\MicroBlaze_DDR_GPIO\design_1_wrapper.xsa

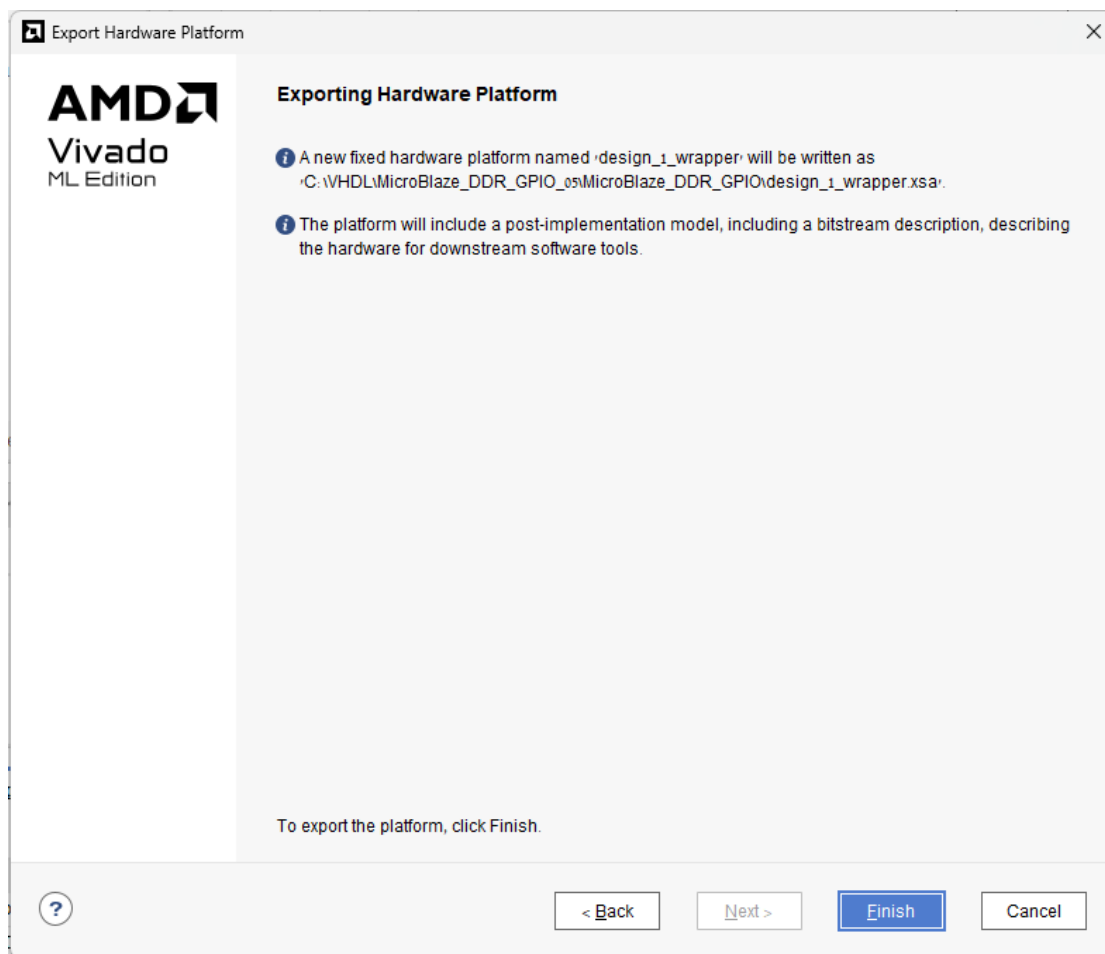
< Back

Next >

Finish

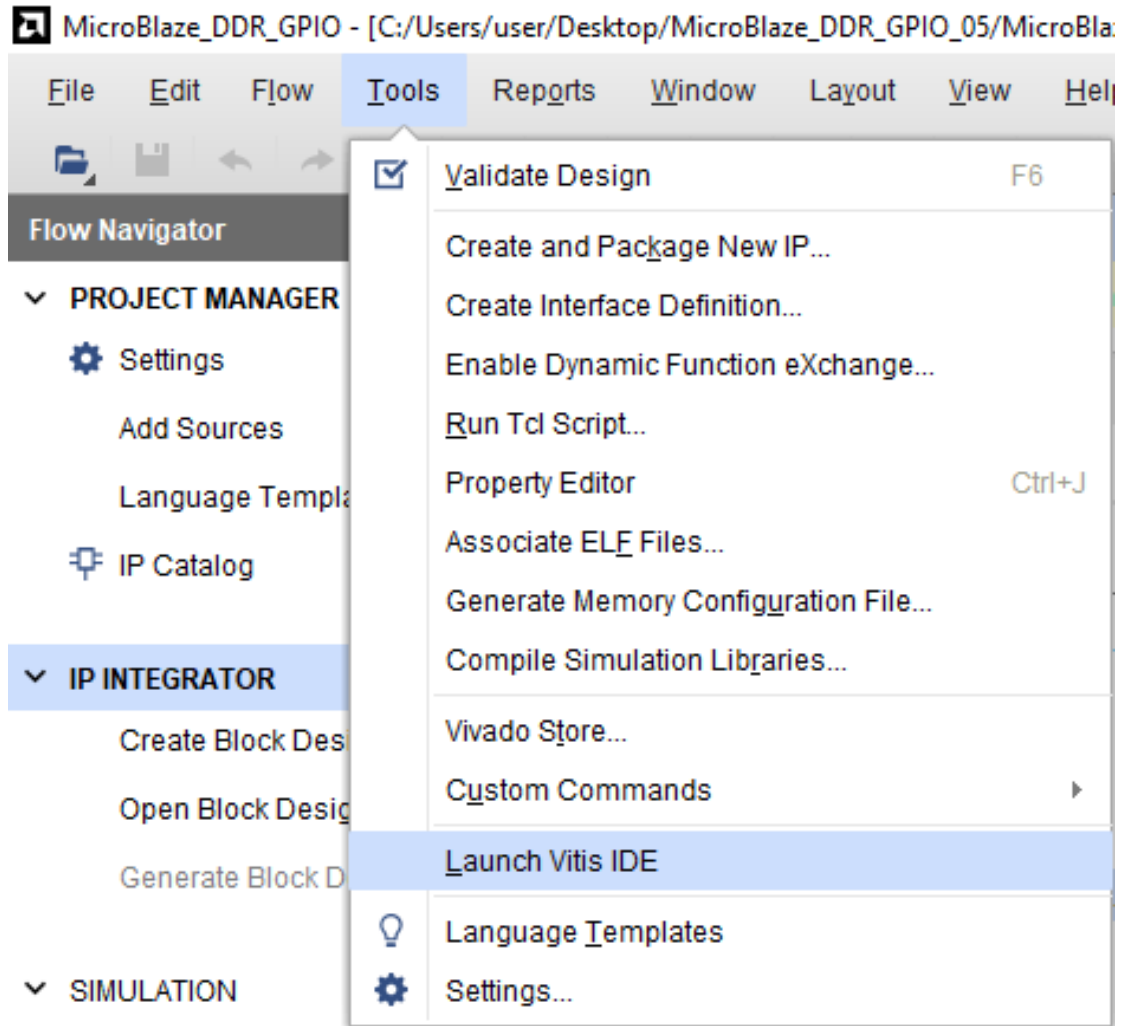
Cancel

לחץ Finish.

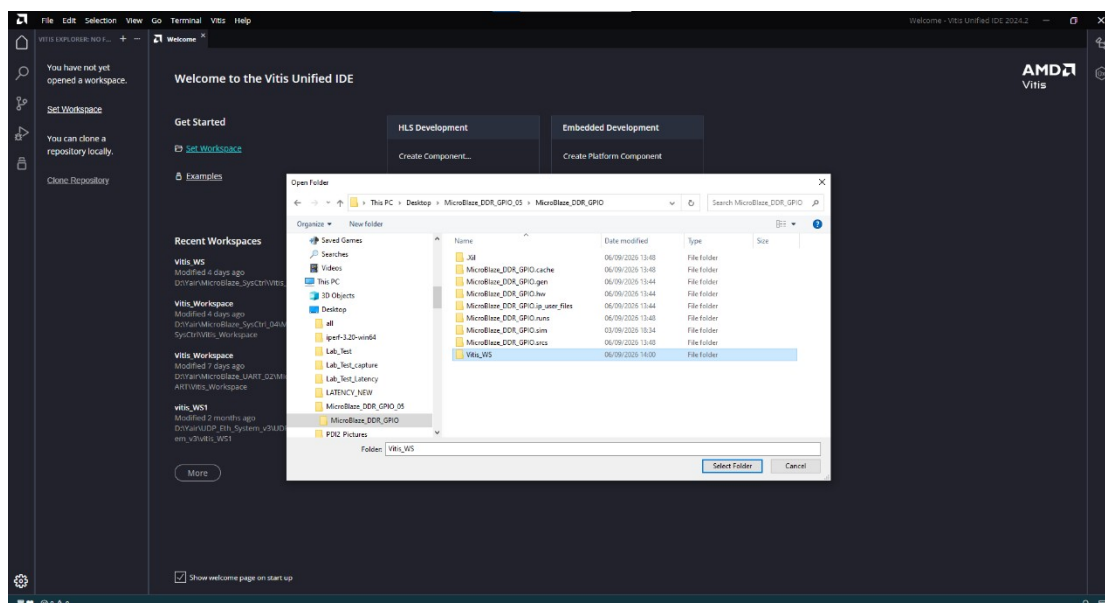


שלב 18:

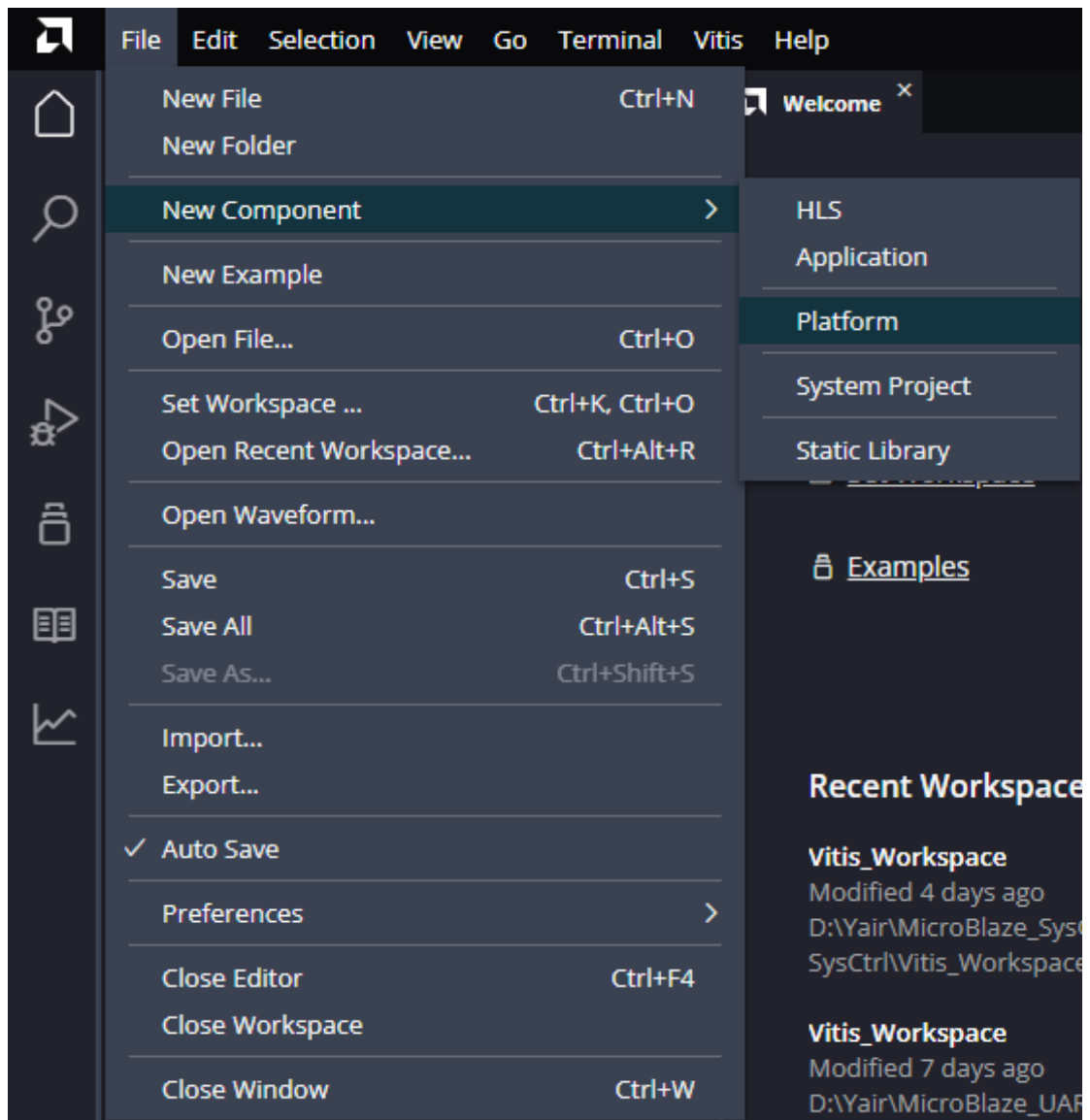
נפעיל את Vitis דרך Tools ואז Launch Vitis IDE.



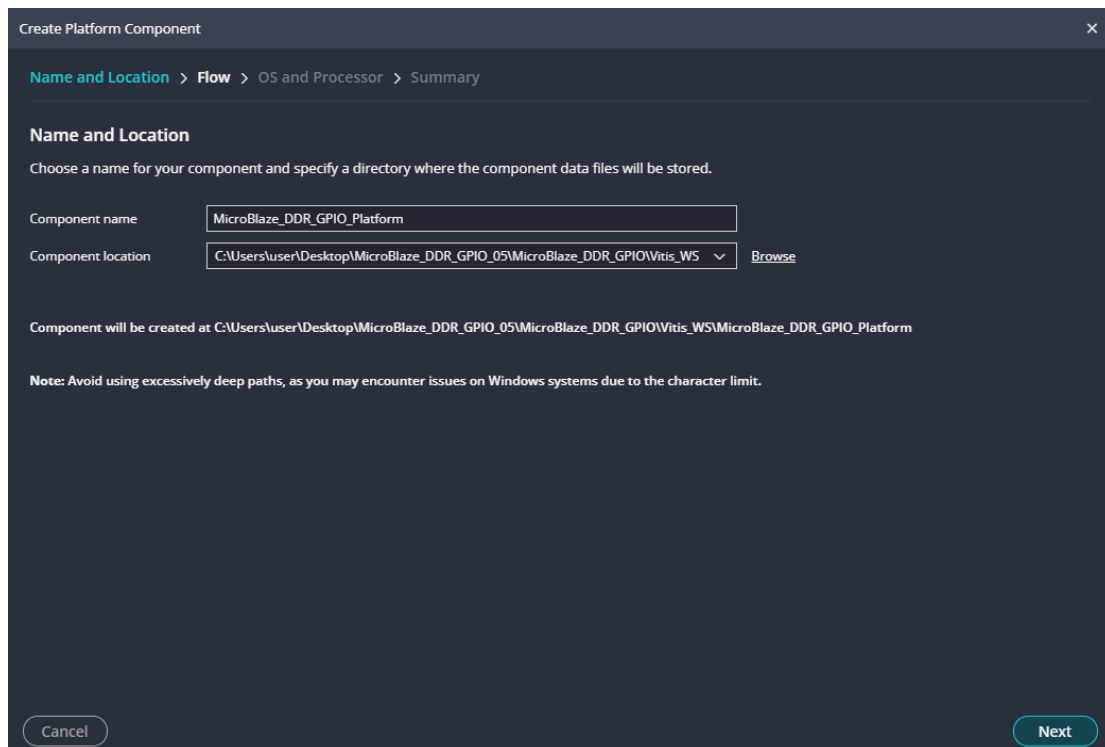
נבחר תיקיית Workspace.



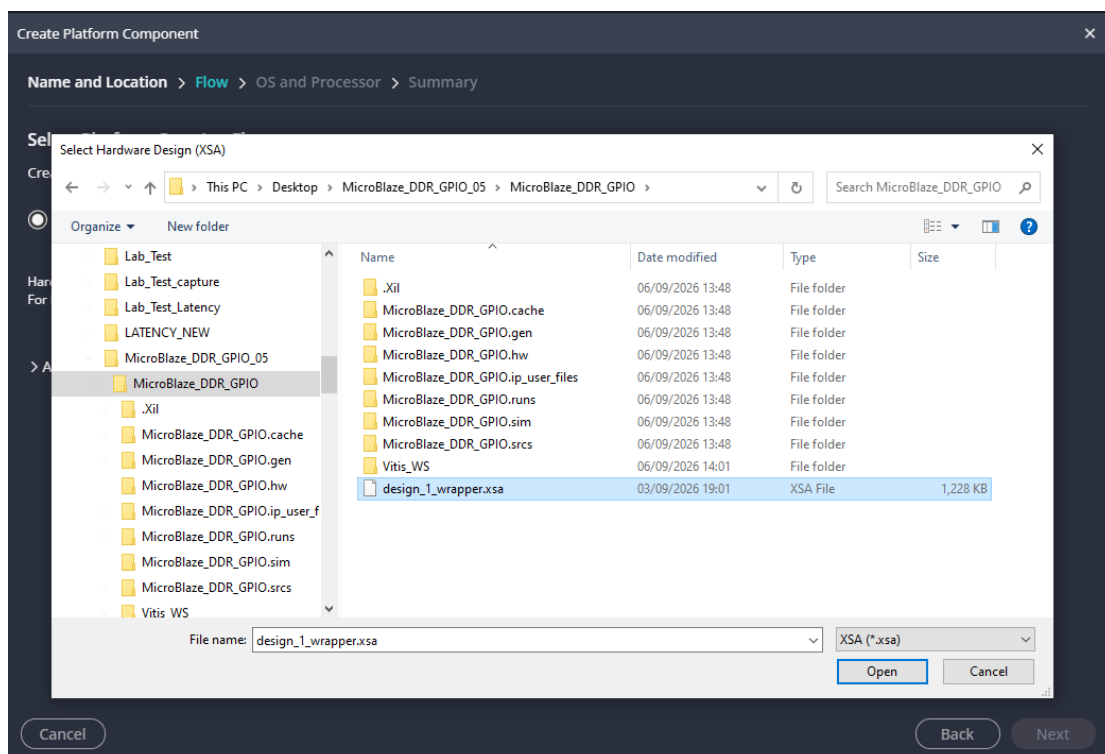
ניצור רכיב חדש מסוג Platform.



נקרא לפלטפורמה MicroBlaze_DDR_GPIO_Platform.



נבחר את קובץ ה-XSA.



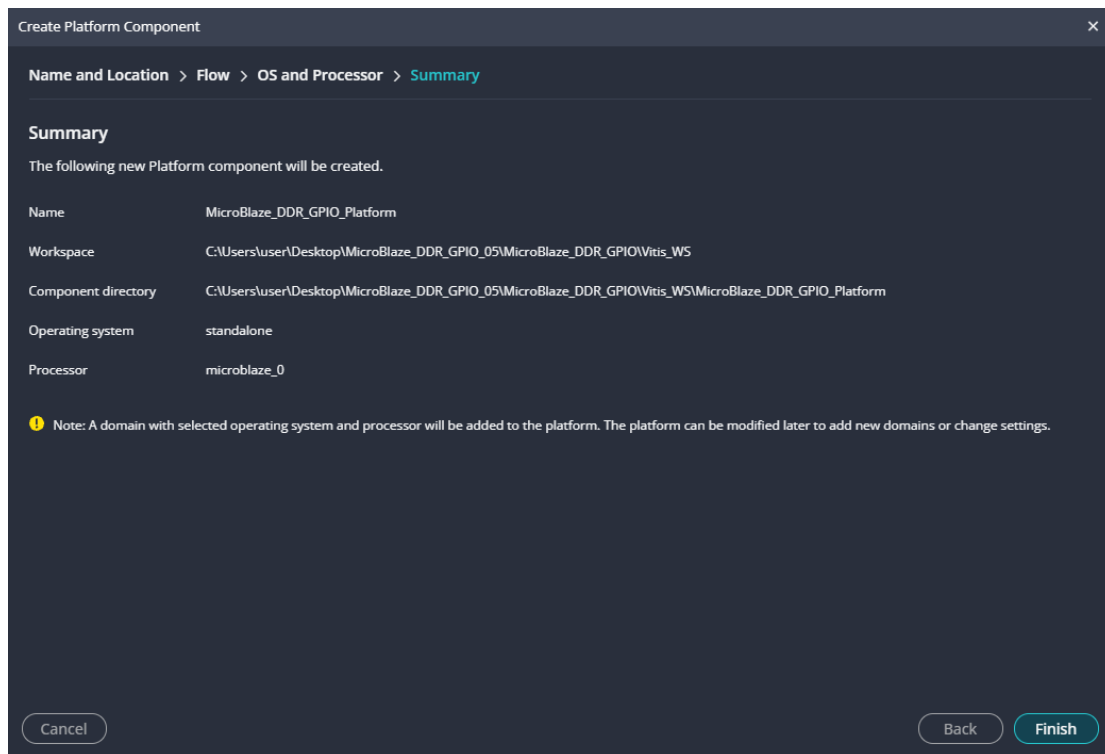
נוודא שהקובץ נבחר ונלחץ Next.

The screenshot shows the 'Create Platform Component' dialog box with the 'Flow' tab selected. The breadcrumb navigation is 'Name and Location > Flow > OS and Processor > Summary'. The section 'Select Platform Creation Flow' contains two radio buttons: 'Hardware Design' (selected) and 'Existing Platform'. Below this, the 'Hardware Design (XSA) For Implementation' section shows a file path 'C:\Users\user\Desktop\MicroBlaze_DDR_GPIO_05\MicroBlaze_DDR...' in a dropdown menu, with a 'Browse' button to its right. An '> Advanced Options' link is also present. At the bottom, there are 'Cancel', 'Back', and 'Next' buttons.

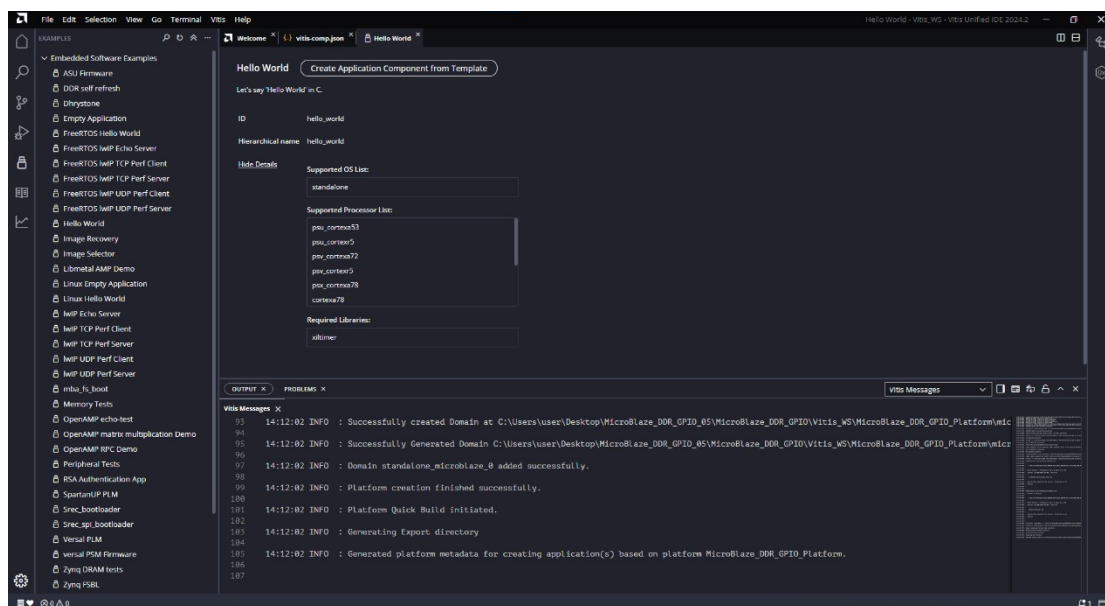
נגדיר מערכת הפעלה standalone ומעבד microblaze_0.

The screenshot shows the 'Create Platform Component' dialog box with the 'OS and Processor' tab selected. The breadcrumb navigation is 'Name and Location > Flow > OS and Processor > Summary'. The section 'Select Operating System and Processor' contains a descriptive paragraph and two dropdown menus. The 'Operating system:' dropdown is set to 'standalone', and the 'Processor:' dropdown is set to 'microblaze_0'. At the bottom, there are 'Cancel', 'Back', and 'Next' buttons.

נלחץ Finish.



הפלטפורמה נוצרה. נבצע לה Build.



שלב 19:

ניצור אפליקציה בשם Micro_DDR_GPIO_App.

Create Application Component - Hello World

Name and Location > Hardware > Domain > Summary

Name and Location

Choose a name for your component and specify a directory where the component data files will be stored.

Component name:

Component location: [Browse](#)

Component will be created at C:\Users\user\Desktop\MicroBlaze_DDR_GPIO_05\MicroBlaze_DDR_GPIO\Vitis_WS\Micro_Blaze_DDR_GPIO_App

Note: Avoid using excessively deep paths, as you may encounter issues on Windows systems due to the character limit.

[Cancel](#) [Next](#)

נבחר את הפלטפורמה MicroBlaze_DDR_GPIO_Platform.

Create Application Component - Hello World

Name and Location > Hardware > Domain > Summary

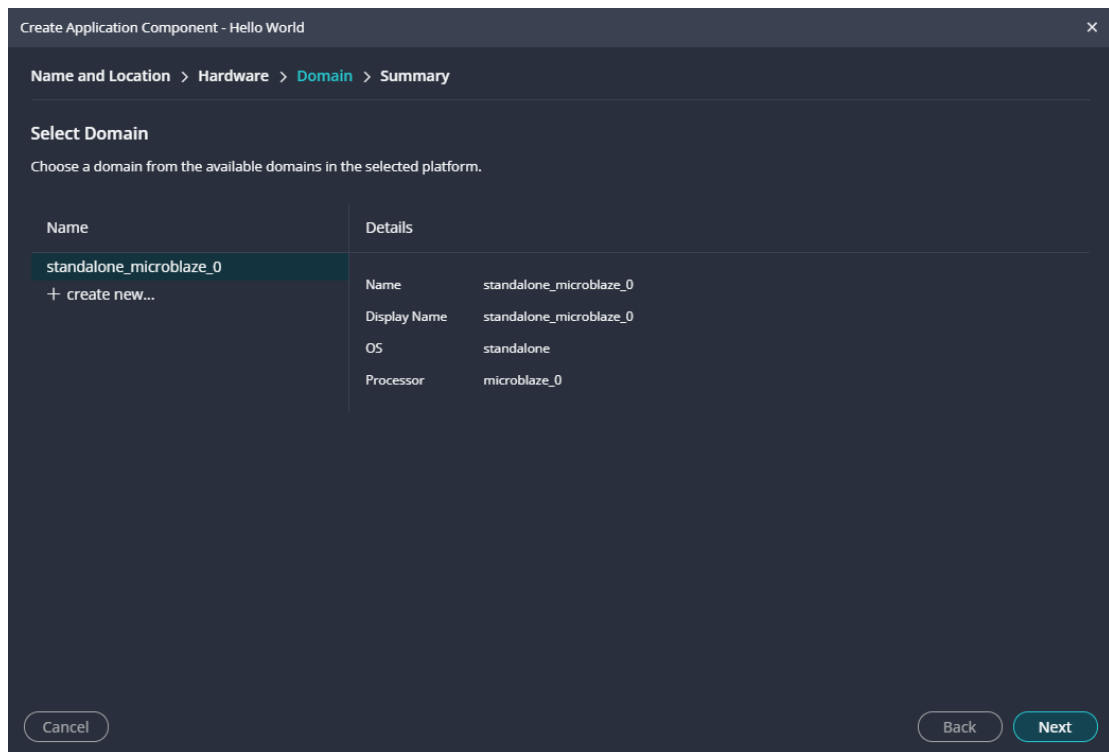
Select Platform

Platforms from your repositories that support the selected example. To create a new platform, use "File -> New Component -> Platform"

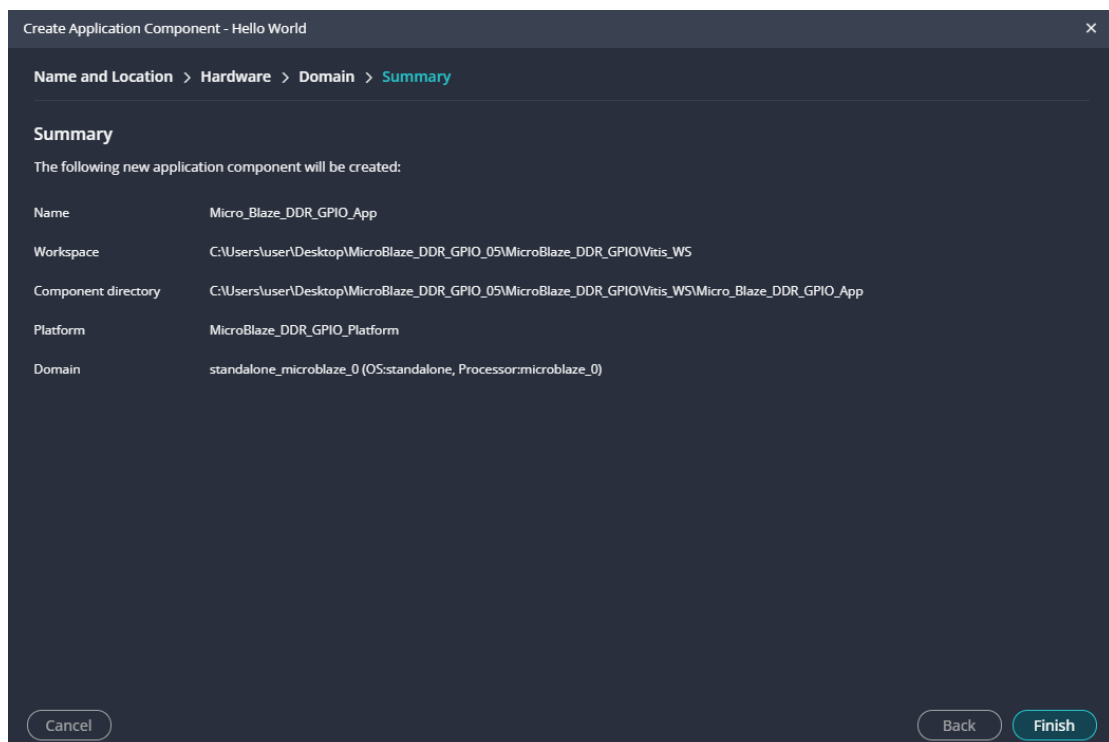
NAME	BOARD	FLOW	VENDOR	PATH
...				...DDR_GPIO_Platform\export\MicroBlaze_DDR_GPIO_Platform
(1)				
☒ MicroBlaze_DDR_GPIO_Platform	ac701	Embedded	xilinx.com	...e_DDR_GPIO_Platform\MicroBlaze_DDR_GPIO_Platform.xpfm

[Cancel](#) [Back](#) [Next](#)

נבחר את הדומיין standalone_microblaze_0.



נלחץ Finish.



נכתוב את הקוד. הקוד מאתחל את רכיב ה-GPIO ומגדיר את שני הערוצים ככניסות. בלולאה הראשית נקרא ערוץ 1 ונבדק אם מצב הלחצנים השתנה. בכל שינוי מודפס הערך החדש של הלחצנים, ולאחריו מצב הלדים הנקרא מערוץ 2.

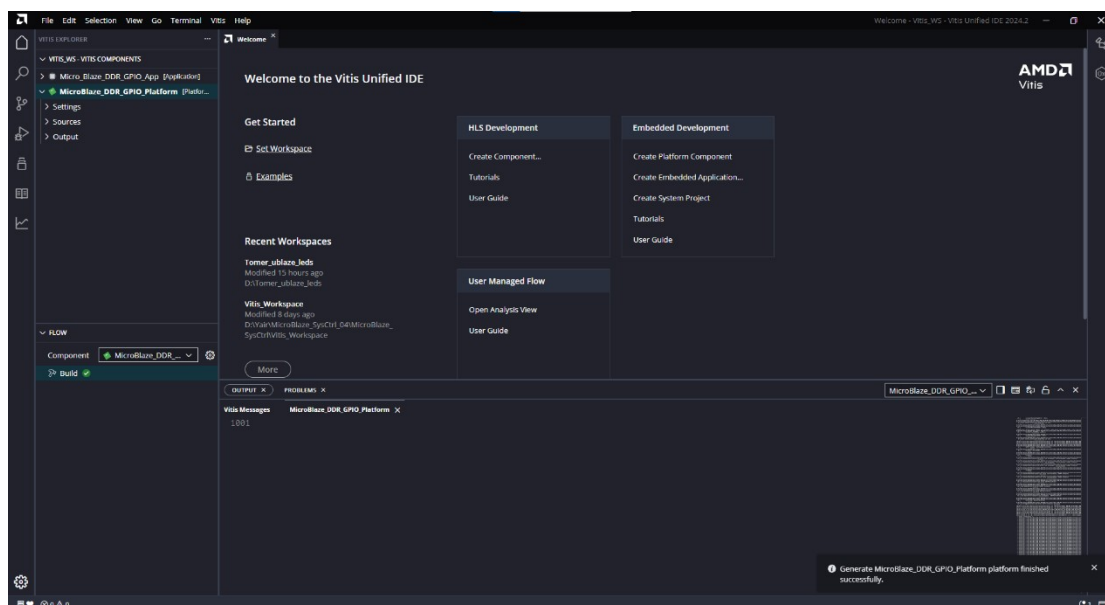
```

1  #include "xparameters.h"
2  #include "xgpio.h"
3  #include "xil_printf.h"
4  #include <xstatus.h>
5
6
7  int main(void)
8  {
9      XGpio Gpio;
10     int init_status;
11     int switches_current_state;
12     int switches_prev_state = -1;
13     int LEDs_current_state;
14
15     init_status = XGpio_Initialize(&Gpio, XPAR_AXI_GPIO_0_BASEADDR);
16     if (init_status != XST_SUCCESS)
17     {
18         xil_printf("GPIO init failed \r\n");
19         return -1;
20     }
21     XGpio_SetDataDirection(&Gpio, 1, 0xFFFFFFFF);
22     XGpio_SetDataDirection(&Gpio, 2, 0xFFFFFFFF);
23     xil_printf("WELCOME\r\n");
24
25     while(1)
26     {
27         switches_current_state = XGpio_DiscreteRead(&Gpio, 1) & 0x7;
28         if (switches_current_state != switches_prev_state)
29         {
30             xil_printf("User changed state of switches : 0x%x\r\n", switches_current_state);
31             switches_prev_state = switches_current_state;
32             LEDs_current_state = XGpio_DiscreteRead(&Gpio, 2) & 0x7;
33             xil_printf("LEDs state changed to: 0x%x\r\n", LEDs_current_state);
34         }
35     }
36 }

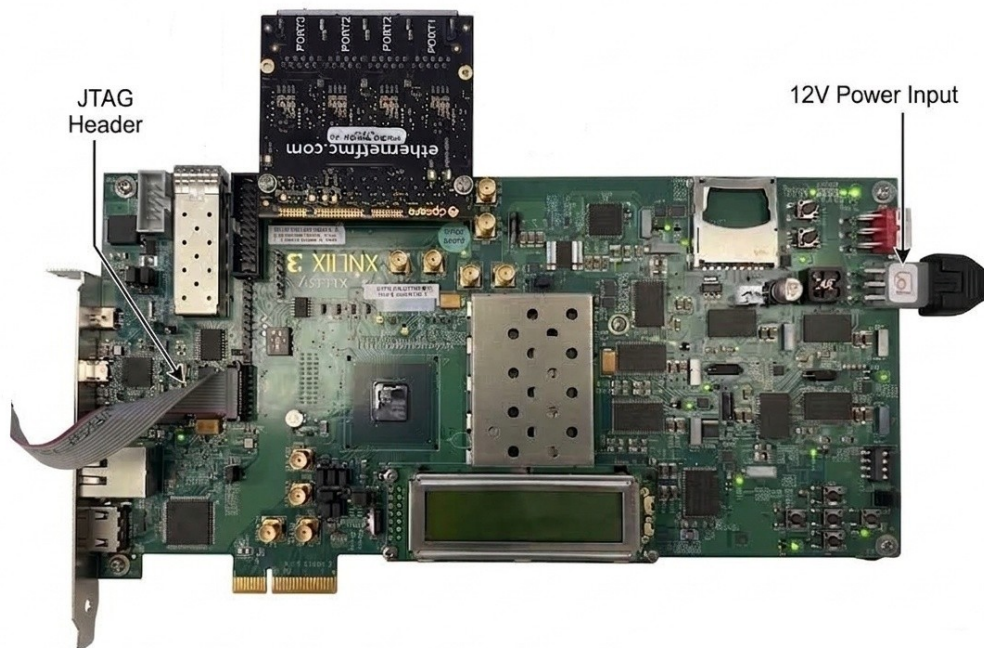
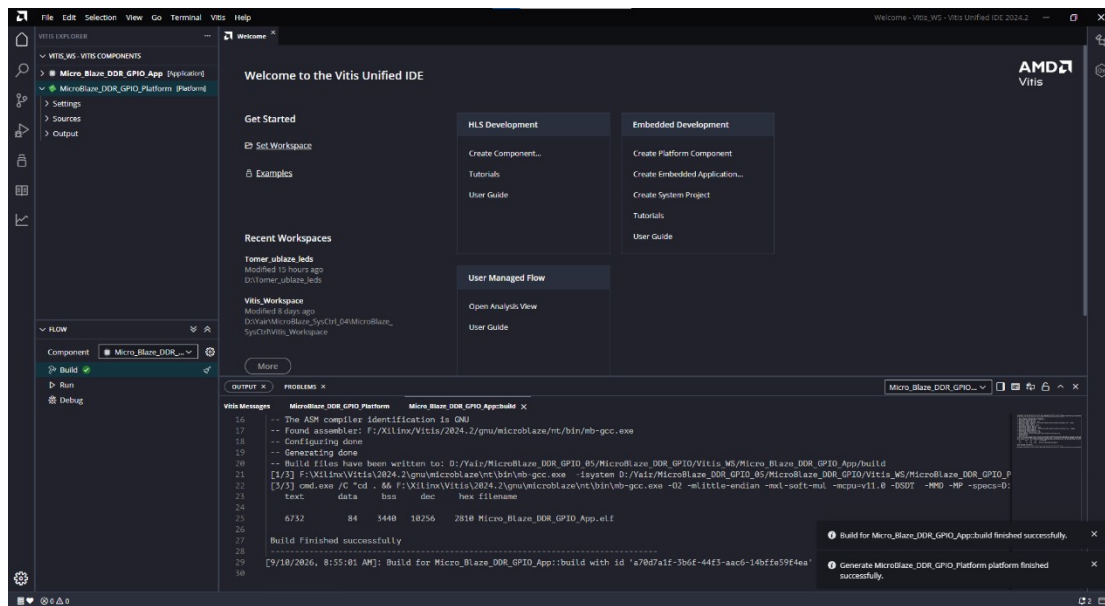
```

שלב 20:

בבצע Build לאפליקציה.

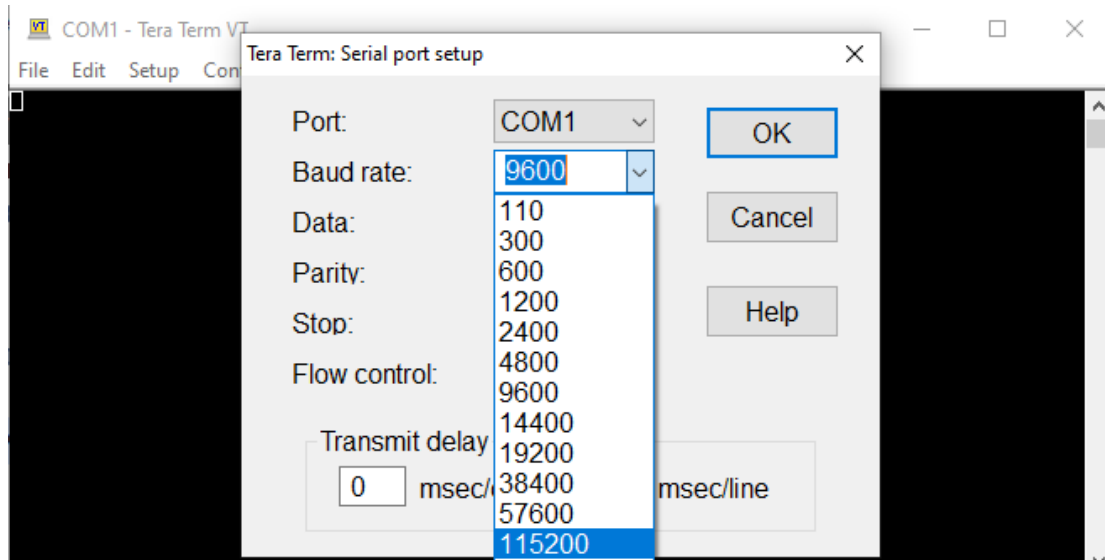


נריץ את האפליקציה על הלוח.

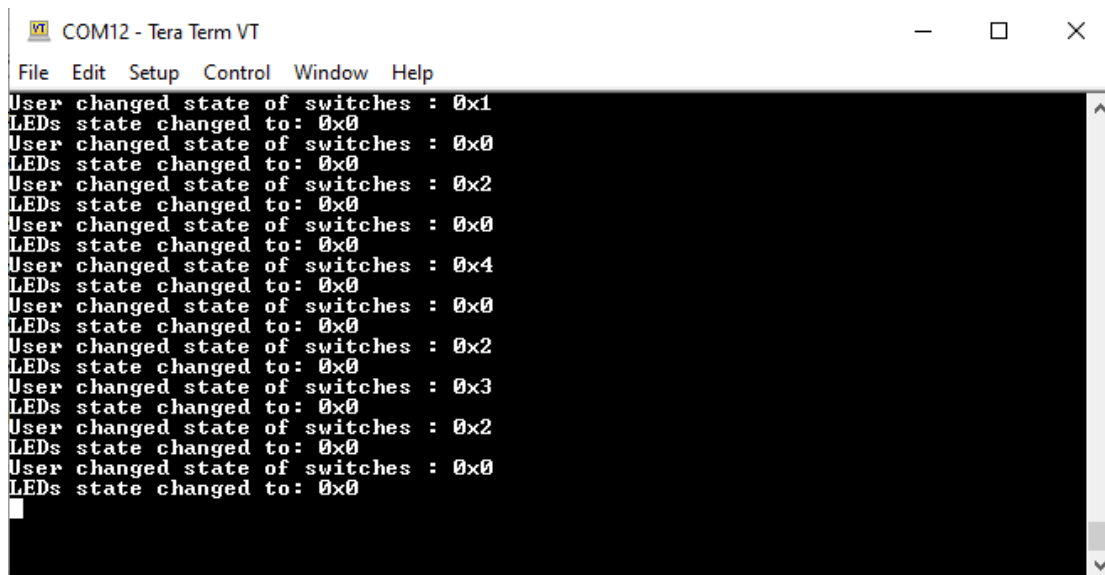


שלב 21:

נפתח את Tera Term ונגדיר חיבור טורי בקצב 115200.

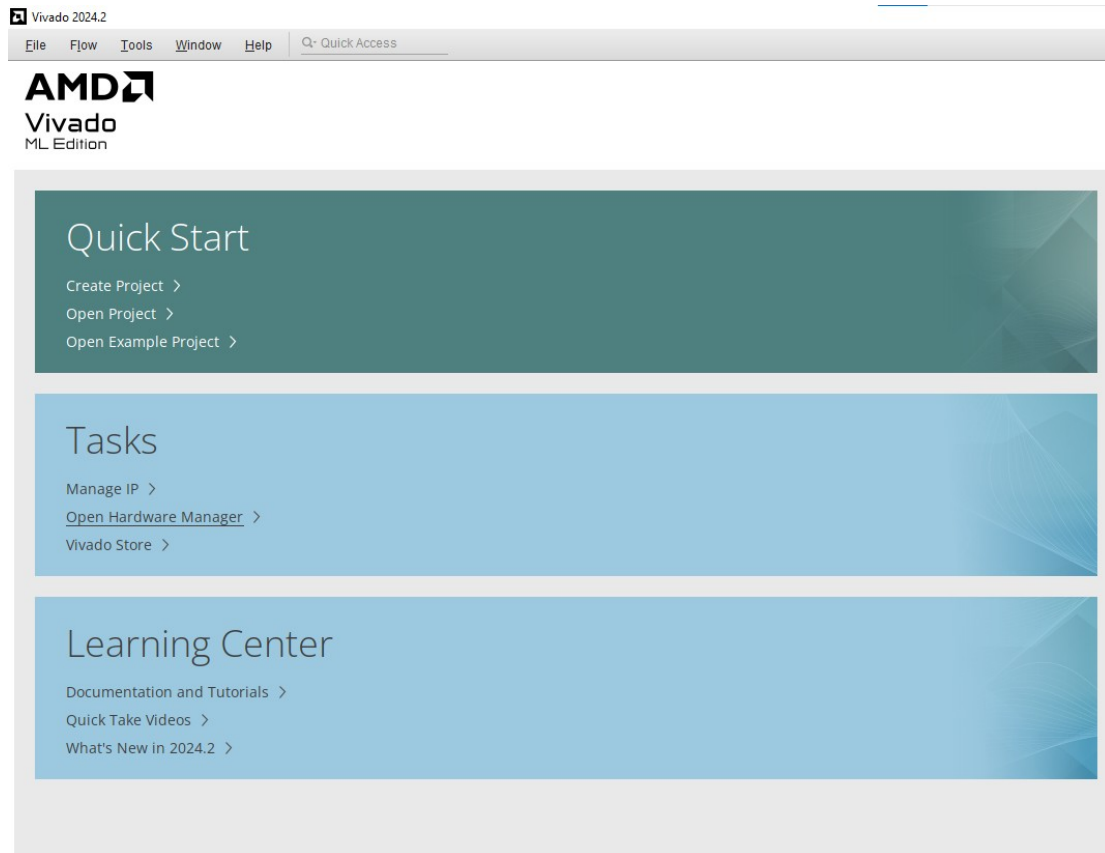


לחיצה על כל אחד מהלחצנים מחזירה את הערכים 0x1, 0x2 ו-0x4. לחיצה משולבת על שני הלחצנים הראשונים מחזירה 0x3, ושחרור הלחצנים מחזיר 0x0.

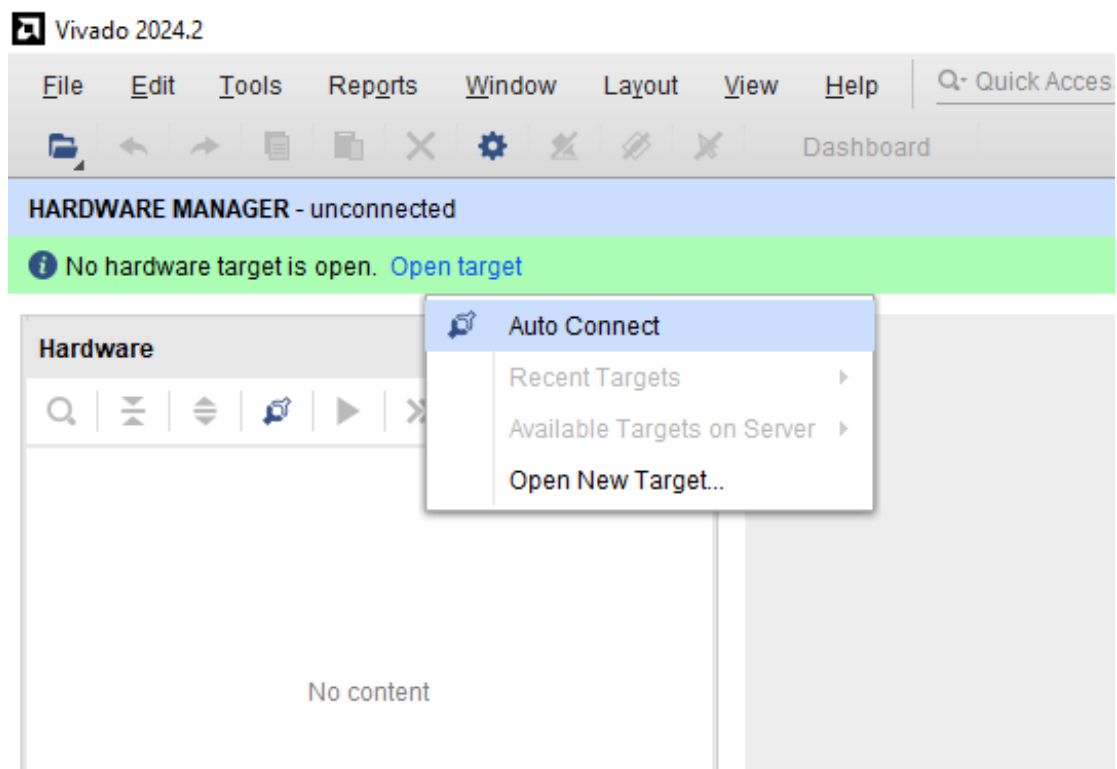


שלב 22:

לאימות בחומרה נפתח את Hardware Manager.



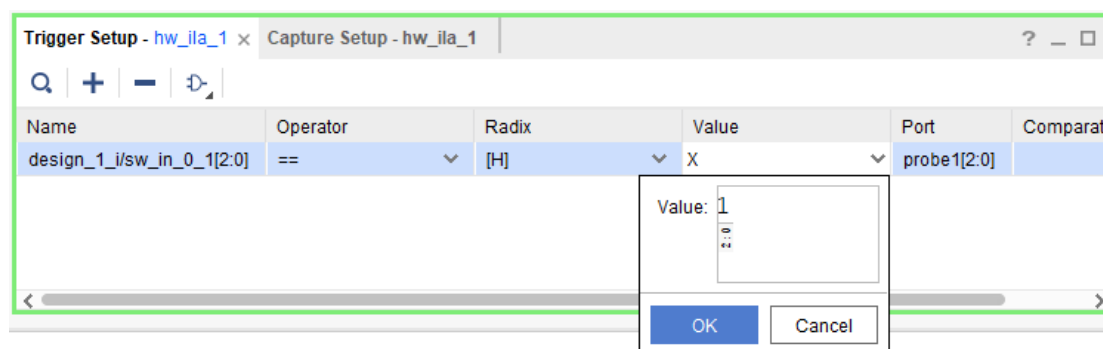
נבחר Open Target ואז Auto Connect.



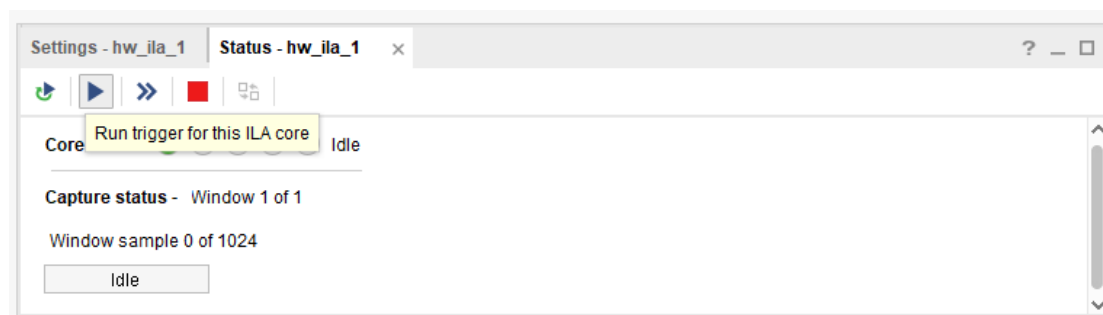
נטען את קובץ ה-Itx דרך Refresh Device.



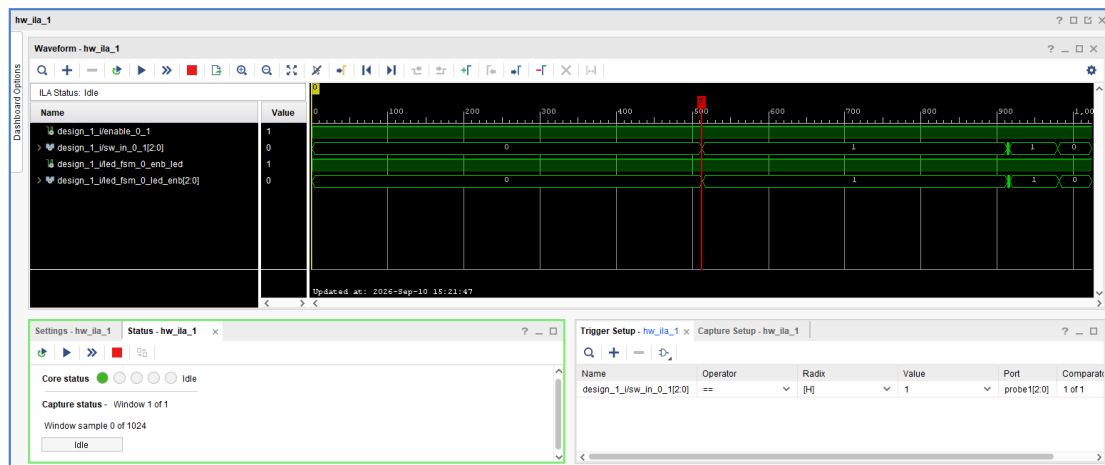
בחלון Trigger Setup נגדיר הדק על וקטור הכניסות sw_in_0 עם הערך 1.



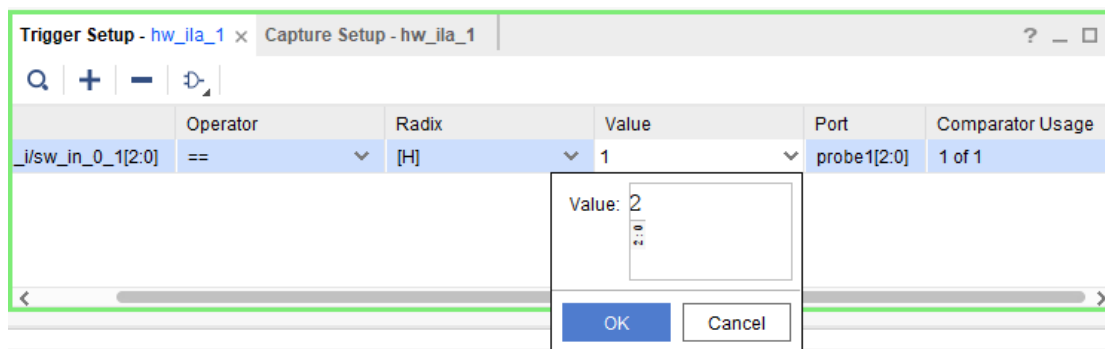
נלחץ Run Trigger.



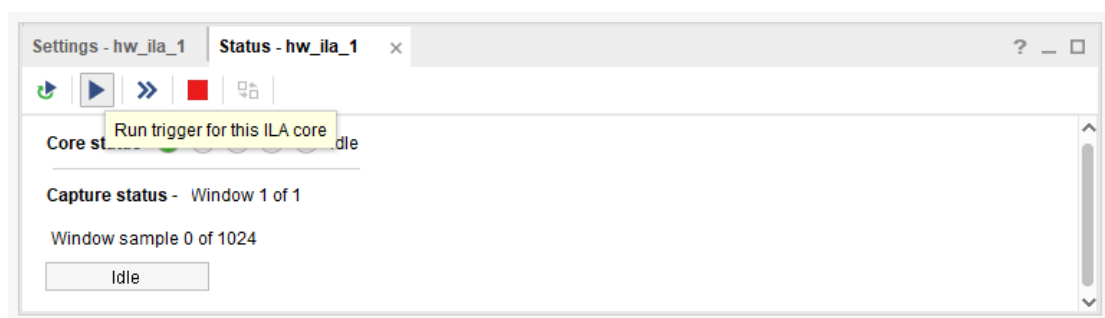
בלחיצה על הלחצן שערכו ה-ILA-0x1, ובחלון הגלים נראית עליית הביט המתאים ברגע הלחיצה.



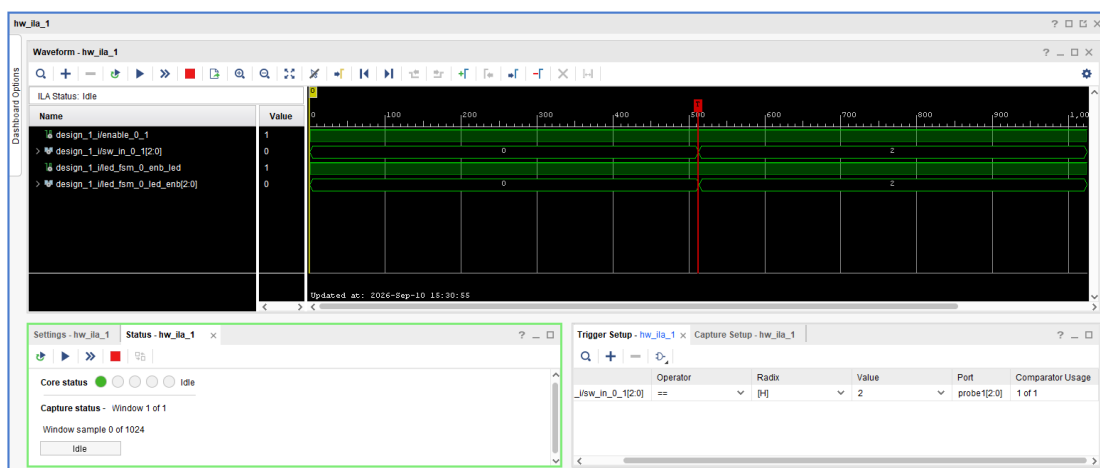
נגדיר ערך הדק 2.



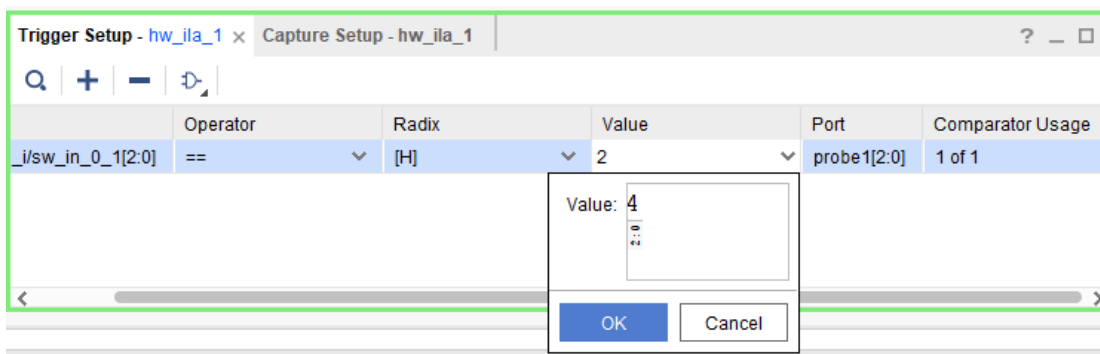
נלחץ Run Trigger.



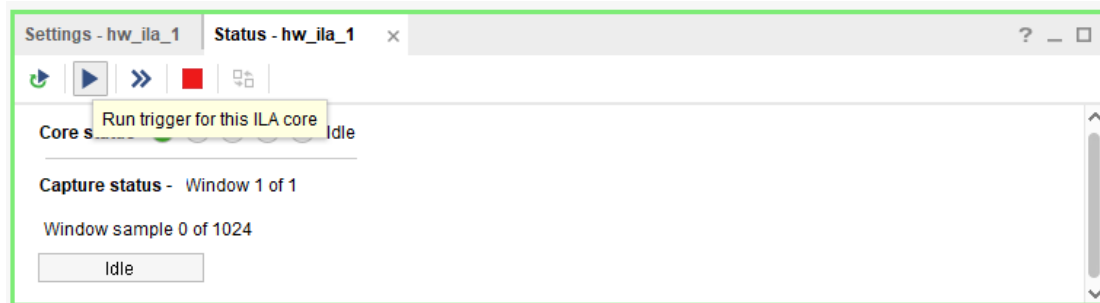
בלחיצה על הלחצן שערכו 0x2:



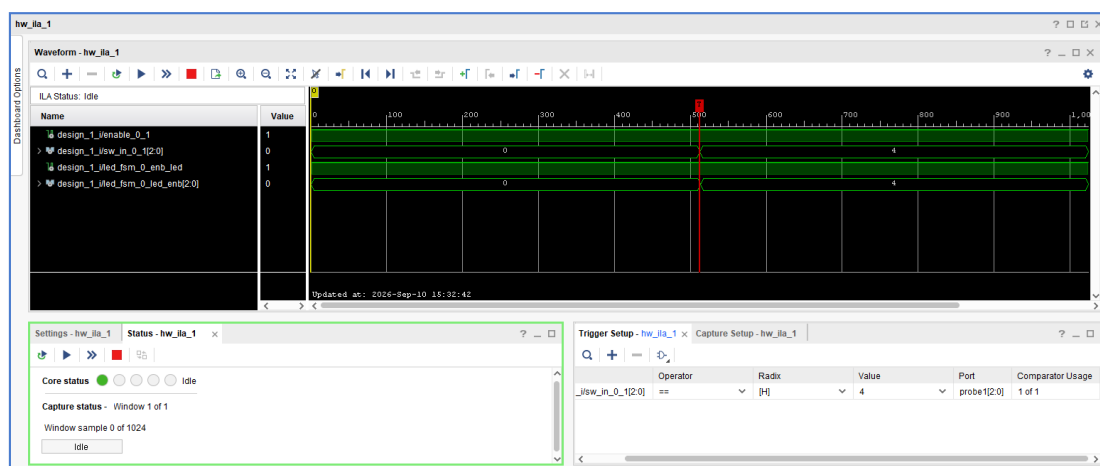
נגדיר ערך הדק 4.



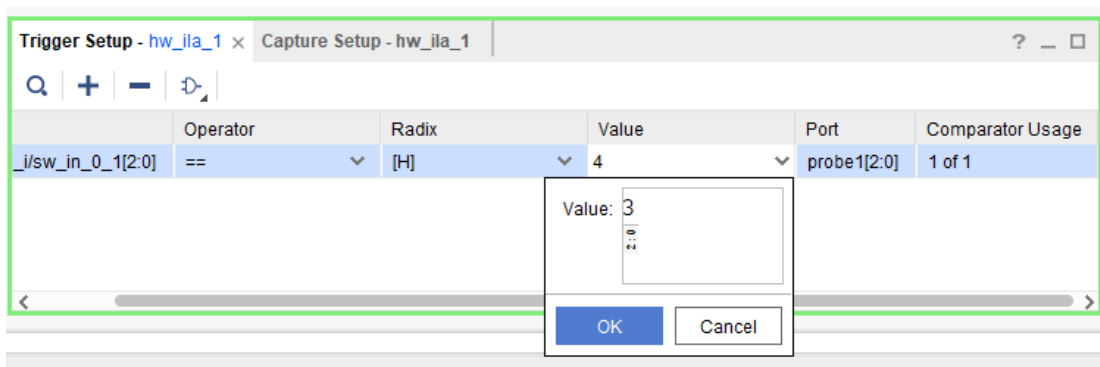
נלחץ Run Trigger.



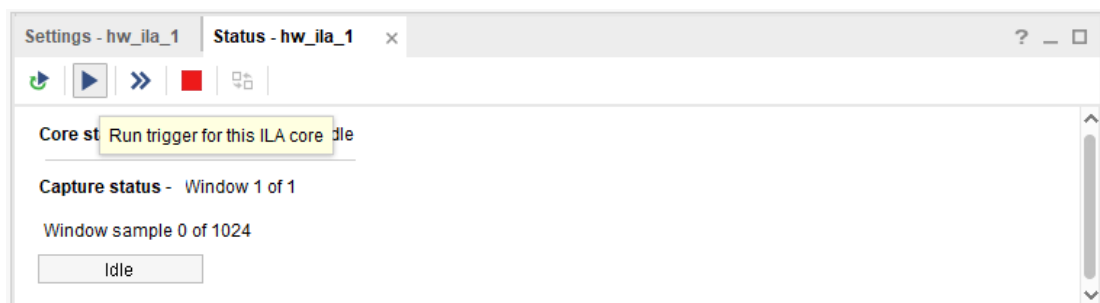
בלחיצה על הלחצן שערכו 0x4:



נגדיר ערך הדק 3.



נלחץ Run Trigger.



בלחיצה משולבת שערכה 0x3 שני הביטים עולים יחד:

